RESEARCH PAPERS

ON

POISONGUARD: AUTOMATED FRAMEWORK FOR DETECTING AND PREVENTING DATA POISONING AND BACKDOOR ATTACKS IN MACHINE LEARNING MODELS

J ANKITHA

25PGOO168

II YEAR MCA

Detecting and Mitigating Data Poisoning Attacks in Machine Learning: A Weighted Average Approach

Yogi Reddy Maramreddy
Department of CSE, GITAM Deemed to be University, Hyderabad, India
iamyogireddy@gmail.com

Kireet Muppavaram
Department of CSE, GITAM Deemed to be University Hyderabad, India
kireet04@gmail.com (corresponding author)

Received: 22 April 2024 | Revised: 9 May 2024 | Accepted: 13 May 2024

Licensed under a CC-BY 4.0 license | Copyright (c) by the authors | *DOI: https://doi.org/10.48084/etasr.7591*

ABSTRACT

Adversarial attacks, in particular data poisoning, can affect the behavior of machine learning models by inserting deliberately designed data into the training set. This study proposes an approach for identifying data poisoning attacks on machine learning models, the Weighted Average Analysis (VWA) algorithm. This algorithm evaluates the weighted averages of the input features to detect any irregularities that could be signs of poisoning efforts. The method finds deviations that can indicate manipulation by adding all the weighted averages and comparing them with the predicted value. Furthermore, it differentiates between binary and multiclass classification instances, accordingly modifying its analysis. The experimental results showed that the VWA algorithm can accurately detect and mitigate data poisoning attacks and improve the robustness and security of machine learning systems against adversarial threats.

Keywords-advanced machine learning attacks; poisoning attacks; attacks in intelligent networks; attack defense methods; security threats

I. INTRODUCTION

Intelligent networks have revolutionized many fields by using cutting-edge Machine Learning (ML) algorithms to generate predictions and judgments based on data. Adversarial attacks are among the most dangerous and sophisticated security risks to which these networks are vulnerable. Poisoning attacks involve malicious alteration of training data in ML models, which compromises system performance and can have disastrous effects [1-3]. Understanding the current status of poisoning attacks and examining potential future options for effective countermeasures are crucial as these intelligent networks become more prevalent and essential in our daily lives [4].

Intelligent network technology has opened the door to revolutionary developments in a variety of industries, including healthcare, finance, transportation, and smart cities. These networks use cutting-edge ML algorithms to handle massive amounts of data, allowing them to get insightful information, streamline procedures, and provide specialized services. The proliferation of Internet of Things (IoT) devices and the widespread connectivity of everyday objects have increased the amount of data that intelligent networks generate and process, making them a prime target for adversaries looking to exploit weaknesses and compromise their integrity [5]. Poisoning attacks have become a danger for ML models amid the rapid development and integration of intelligent networks [6-7]. An adversary conducts a poisoning attack by introducing deliberately induced data or values into the existing training dataset to distort the model learning curve [8]. The accuracy and effectiveness of the model decrease as it assimilates the tainted data during training, resulting in inaccurate forecasts and poor decision-making. These attacks can damage crucial infrastructure and compromise user privacy, which makes them a serious concern for researchers and business professionals.

The development of numerous defense mechanisms has been sparked by efforts to stop poisoning attempts [9]. To mitigate the effects of poisoned data, many studies have examined strategies such as adversarial training, data sanitization, and outlier detection [10-11]. Adversarial training involves adding adversarial cases to the existing training process to increase the model's resistance or performance to poisoning attacks [12]. On the other hand, data sanitization seeks to identify and eliminate poisoned samples from the training dataset to avoid their negative effects on the model's performance [13]. Outlier detection algorithms locate and remove unusual occurrences that could be signs of poisoning attacks [14-15]. Although these defense systems have shown

potential, they also have difficulty performing at their peak levels and adapting to changing threat environments [16].

As intelligent networks continue to develop, it is critical to look for future potential solutions against poisoning attacks. Research on explainable AI, differential privacy, and federated learning has a lot of promise to improve the safety of intelligent networks. Federated learning offers a viable way to reduce poisoning threats, as it allows for the collaborative training of ML models on various devices while protecting data privacy [17]. Differential privacy strategies introduce noise into the data to prevent attackers from extracting private information, protecting against attacks that target specific data points using data poisoning [18]. Additionally, the incorporation of explicable AI approaches improves the transparency of the model, enabling better detection and comprehension of potential poisoning attempts.

II. THREAT MODEL

Threat modeling is necessary to detect and evaluate potential security risks and vulnerabilities in intelligent networking systems, especially poisoning attacks. These types of attacks alter the perception level data to compromise the reliability of ML models. A threat model for poisoning attacks comprises the adversary's knowledge, objective, capacity, and approach. Attacker knowledge can vary from zero knowledge, which means that the attacker knows nothing about the target system and can merely query it, to perfect knowledge, which means that the attacker is completely familiar with the system. The perpetrator's goal encompasses influence on system performance and operation, attack specificity, and security breaches. The ability of the adversary to manipulate features or labels in the training data is referred to as its capability.

III. POISONING ATTACKS IN ML

ML has gained extensive application across a variety of fields, but this advancement has also brought about new security issues, most notably poisoning attacks. Poisoning attacks involve inserting harmful data into the training set, resulting in poor model performance and incorrect predictions during inference. Due to their covert nature, these attacks can have serious effects on critical systems and are difficult to detect. To preserve the integrity and trustworthiness of ML systems, as they continue to play an increasingly important role in decision-making processes, it is critical to develop powerful defenses against poisoning attempts.

A. Using the Support Vector Machine (SVM) to Detect Poisoning Attacks

SVM is an ML technique that is widely used in classification and anomaly detection tasks. SVM can be used as a binary classifier in the context of detecting poisoning attacks to identify cases that differ from the predicted data distribution [19]. The key concept is to create a judgment boundary that maximizes the margin between classes while spotting any outliers that could be fraudulently injected poison data. SVM can successfully identify potential toxic samples that could undermine the integrity of the training data and the final model by utilizing SVM's capacity to understand complex decision boundaries. However, the efficacy of SVM in identifying poisoning attacks depends on the kernel function and hyperparameters used, thus careful adjustment is required for optimal results. Furthermore, the computational cost of SVM can be an issue in large-scale datasets, requiring effective optimization solutions. To address these issues, researchers frequently combine SVM with other techniques, such as ensemble methods or feature selection, to improve detection accuracy and resistance in the face of shifting attack strategies. SVM is an important instrument in the armory of defense measures, greatly contributing to current research in preventing poisoning attempts and enhancing the security of ML systems.

TABLE I. COMPARATIVE STUDY OF DIFFERENT POISONING TECHNIQUES

Learning Taxonomy	Advantages	Disadvantages
Conventional Traditional Supervised Learning	Rapid statistical attack. Algorithm independent	Requires extensive knowledge. The attacker possesses unrealistic knowledge.
Conventional Unsupervised Learning	Can be used in more trustworthy situations.	Vulnerable to outliers.
Reinforcement Learning	Attack only requires a limited number of poison training points to function. The PCA-subspace approach is efficient. Effective at enhancing the loss function of the agent. Effective in combating A3C.	The risk or additional effort. Delays training.
Deep Learning	Avoid the gradient-based optimizing assumption. The utilization of a black-box attack in a realistic scenario.	Inadequate performance evaluation. The pattern of injected triggers is evident.

B. Using Decision Trees to Detect Poisoning Attacks

Decision trees partition data recursively, depending on distinct features, resulting in a tree-like structure that aids decision-making. Decision trees can reveal abnormalities and outliers in the context of poisoning attack detection by analyzing unique paths within the tree structure [21]. During the decision-making process, instances taking unusual or divergent pathways may generate suspicion and be labeled as potential poison data points. The interpretability of decision trees enables researchers to examine the characteristics and conditions that lead to abnormal decisions, providing significant information on the nature of prospective poisoning attacks. Despite their interpretability and ease of use, overfitting can occur in decision trees, especially when dealing with noisy or unbalanced data. Pruning, ensemble approaches (e.g., Random Forests), and regularization are used to improve generalization and increase the durability of decision tree-based detection against adversarial attacks. Additionally, feature engineering and selection are critical to detecting significant patterns and improving detection accuracy. Decision trees are useful tools for identifying poisoning attacks and help design effective defensive mechanisms that protect the integrity and trustworthiness of ML models in a variety of applications.

C. Conventional Unsupervised Learning Poisoning Attacks

Clustering methods in classical unsupervised learning try to group data points based on their similarity without requiring any labeled information. Poisoning attacks in clustering can skew cluster formation, resulting in data point misclassification and compromising the integrity of the clustering model [22]. To change the cluster assignments, attackers may inject fraudulent data, causing genuine data points to be misclassified into erroneous clusters. This can lead to inaccurate insights and decisions, affecting applications such as consumer segmentation, anomaly detection, and pattern recognition.

1) Feature Selection

Identifying the most relevant features that contribute significantly to the model's performance is a critical stage in unsupervised learning. To mislead the selection process, poisoning attacks in feature selection can involve changing feature ranks or introducing redundant or irrelevant features [23]. Attackers can intentionally inject noise or biased information into the system, causing it to select suboptimal or deceptive features. This reduces the model's efficiency, interpretability, and generalization, making it less effective in tasks such as data compression, dimensionality reduction, and data preparation.

2) Principal Component Analysis (PCA)

PCA is a popular unsupervised dimensionality reduction and feature extraction technique. It seeks to highlight the essential patterns of the data by transforming them into a new coordinate system. Poisoning attacks in PCA can involve injecting malicious data to change the directions and magnitudes of the principal components. Attackers can corrupt the converted space by manipulating the data's covariance matrix, resulting in false interpretations and jeopardizing the model's accuracy and robustness in applications such as image recognition, signal processing, and anomaly detection.

D. Poisoning Attacks within Deep Learning

Poisoning attacks within deep learning involve the deliberate introduction of harmful content into the training set. These malicious samples are diligently crafted to influence the model's learning process, resulting in biased or erroneous predictions during inference. Poisoning attacks, unlike typical attacks that target software flaws, target the underlying learning processes, making them extremely difficult to detect. Diversified approaches are required to mitigate the threat of poisoning attacks in deep learning. Techniques such as adversarial training, data sanitization, and model robustness testing are used to improve the model's resilience to hostile cases. In addition, enhancing awareness of potential vulnerabilities and incorporating security considerations throughout model building and deployment are crucial aspects of safeguarding deep learning systems.

The significance of detecting and preventing poisoning attacks cannot be overstated, especially because deep learning is at the forefront of AI research and applications. To harden deep learning models against these complex risks and ensure their trustworthiness and reliability in real-world scenarios, ongoing research, collaboration, and initiatives are required.

IV. DETECTION OF DATA POISONING ATTACKS USING VERIFIED WEIGHTED AVERAGE METHOD

The proposed method, called the Verified Weighted Average (VWA) method, aims to detect data poisoning attacks on ML models by analyzing the weighted averages of input features. The VWA algorithm provides a straightforward, yet effective means of identifying potential tampering within training datasets.

ALGORITHM 1: DATA POISONING ATTACK DETECTION USING THE VWA ALGORITHM

```
Input: Training model
Output: Modified training labels or not
1.  Begin
2.  Consider Input features as I, training model as
    TM, weight as W, and weighted average as WA
    // λ is the sum of a weighted average of input
    // features
3.  Calculate λ= ΣWA
4.  if (λ > 1 || λ < 1 )
5.    if (no of class labels =2 and
          (C1(WA) >C2(WA)) || (C1(WA) <C2(WA)) then
6.      binary classification of only two class
        labels
7.    else if (no of class labels > 2) then
8.      // calculate VWi  = WA * λ for each input
        // feature
9.      for each VWi in I
          if(VWi = Wi) then
            no modification in class labels
10.       else
            modified multiclass label or
            multi-labels
11. else (λ =1) then
      No modification on class labels
12. End
```

λ is the sum of weighted averages

WA is the Weighted Average of input features

VW is the Verified Weight of input features

The VWA data poisoning detection method identifies probable data poisoning attacks on a training dataset. The procedure takes the input features (*I*), the training model (*TM*), and their respective weights (*W*) and generates the weighted average (*WA*) of the features. The algorithm calculates the sum of weighted averages (λ) and examines whether it differs from the predicted value of 1. If there are just two class labels and one class's weighted average is significantly higher or lower than the other, indicating possible manipulation, the method flags it as a binary classification problem. When there are more than two class labels, it calculates the weighted averages for each feature and compares them to their weights. If they match, it means there has been no change to the class labels, otherwise, it indicates that the labels have been modified or poisoned. If the sum of the weighted averages is equal to one, it indicates that no changes were made.

V. IMPLEMENTATION ON DIFFERENT DATASETS

A. Weka Dataset on Weather Prediction

The VWA algorithm was applied to the Weka Weather dataset [24], consisting of four labels, namely outlook, temperature, humidity, windy, and a final label for the decision to play or not. The results are as follows:

TABLE II. RESULTS USING WEKA WEATHER DATASET

Weather	W	WA	VW
Outlook	2	0.01	2
Temperature	74	0.47	74
Humidity	82	0.52	82
Windy	0.43	0	0
	158	1	158

```
@relation weather

@attribute outlook {sunny, overcast,
rainy}
@attribute temperature real
@attribute humidity real
@attribute windy {TRUE, FALSE}
@attribute play {yes, no}

@data
sunny,85,85, FALSE, no
sunny,80,90, TRUE, no
overcast,83,86, FALSE, yes
rainy,70,96, FALSE, yes
rainy,68,80, FALSE, yes
rainy,65,70, TRUE, no
overcast,64,65, TRUE, yes
sunny,72,95, FALSE, no
sunny,69,70, FALSE, yes
rainy,75,80, FALSE, yes
sunny,75,70, TRUE, yes
overcast,72,90, TRUE, yes
overcast,81,75, FALSE, yes
rainy,71,91, TRUE, no
```

Fig. 1. Dataset on weather prediction.

As shown in Table II, λ is calculated as 1, thus there is no modification in the dataset.

B. Glass Identification Dataset

The Weka Glass identification dataset [24] was used, which is modified by an attacker. The proposed VWA algorithm was applied to this dataset. In the third step of the algorithm, the λ value was greater than 1, and the final results are as follows.

TABLE III. RESULTS ON GLASS IDENTIFICATION DATASET

Class label	W	WA	VW_i
1	101.341	0.142809	101.341
2	101.2957	0.142745	101.2957
3	101.4177	0.142917	101.4177
5	101.3451	0.142815	101.3451
6	101.4091	0.142905	101.4091
7	101.4013	0.142894	101.4013
	608.2099	0.857086	608.2099

When the original dataset weights are calculated using the VWA algorithm, the total weight is equal to 709.6251. But in the above table, the sum of all the weights was equal to 608.2099. This observation shows that the VWA algorithm was able to detect the modification in training data, which may be caused by a data poisoning attack.

C. Comparative Performance of VWA with Existing Methods

As shown in Table III, the VWA method demonstrates superior performance compared to standard models in terms of accuracy, precision, recall, and F1 score. Additionally, it offers low computational complexity, making it efficient for practical deployment in real-world scenarios.

TABLE IV. PERFORMANCE COMPARISON

Method	Accuracy	Precision	Recall
VWA method	0.92	0.94	0.91
Influence functions	0.85	0.83	0.88
Feature squeezing	0.88	0.88	0.85

VI. CONCLUSION

This study explored intelligent network poisoning attacks and their consequences on ML models. This study helped define adversarial ML attacks by creating a detailed threat model using data poisoning approaches. The proposed VWA approach, which uses weighted averages of input features to discover manipulation abnormalities, can help detect and mitigate data poisoning attacks. The changing nature of threats and detection and prevention issues highlight the need for ongoing research and innovation. Federated learning and explainable AI may improve the poisoning resilience of ML systems. In the future, this study will seek to investigate poisoning attacks on intelligent networks and design powerful security mechanisms to protect crucial ML applications in an increasingly adversarial environment.

REFERENCES

[1] X. Zhang, Z. Wang, J. Zhao, and L. Wang, "Targeted Data Poisoning Attack on News Recommendation System by Content Perturbation." arXiv, Mar. 2022, https://doi.org/10.48550/arXiv.2203.03560.

[2] Y. Zhao, X. Gong, F. Lin, and X. Chen, "Data Poisoning Attacks and Defenses in Dynamic Crowdsourcing With Online Data Quality Learning," *IEEE Transactions on Mobile Computing*, vol. 22, no. 5, pp. 2569–2581, May 2023, https://doi.org/10.1109/TMC.2021.3133365.

[3] J. Chen, X. Zhang, R. Zhang, C. Wang, and L. Liu, "De-Pois: An Attack-Agnostic Defense against Data Poisoning Attacks," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 3412–3425, 2021, https://doi.org/10.1109/TIFS.2021.3080522.

[4] M. Dibaei *et al.*, "Attacks and defences on intelligent connected vehicles: a survey," *Digital Communications and Networks*, vol. 6, no. 4, pp. 399–421, Nov. 2020, https://doi.org/10.1016/j.dcan.2020.04.007.

[5] A. Qayyum, M. Usama, J. Qadir, and A. Al-Fuqaha, "Securing Connected & Autonomous Vehicles: Challenges Posed by Adversarial Machine Learning and the Way Forward," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 998–1026, 2020, https://doi.org/10.1109/COMST.2020.2975048.

[6] M. B. Ammar, R. Ghodhbani, and T. Saidani, "Enhancing Neural Network Resilence against Adversarial Attacks based on FGSM Technique," *Engineering, Technology & Applied Science Research*, vol. 14, no. 3, pp. 14634–14639, Jun. 2024, https://doi.org/10.48084/etasr.7479.

[7] A. Al-Marghilani, "Comprehensive Analysis of IoT Malware Evasion Techniques," *Engineering, Technology & Applied Science Research*, vol. 11, no. 4, pp. 7495–7500, Aug. 2021, https://doi.org/10.48084/etasr.4296.

[8] N. A. Alsharif, S. Mishra, and M. Alshehri, "IDS in IoT using Machine Learning and Blockchain," *Engineering, Technology & Applied Science Research*, vol. 13, no. 4, pp. 11197–11203, Aug. 2023, https://doi.org/10.48084/etasr.5992.

[9] K. Muppavaram, M. S. Rao, K. Rekanar, and R. S. Babu, "How Safe Is Your Mobile App? Mobile App Attacks and Defense," in *Proceedings of the Second International Conference on Computational Intelligence and Informatics ICCII 2017*, Hyderabad, India, Sep. 2017, pp. 199–207, https://doi.org/10.1007/978-981-10-8228-3_19.

[10] S. Aparna, K. Muppavaram, C. C. V. Ramayanam, and K. S. S. Ramani, "Mask RCNN with RESNET50 for Dental Filling Detection," *International Journal of Advanced Computer Science and Applications*

(IJACSA), vol. 12, no. 10, 2021, https://doi.org/10.14569/IJACSA.2021.0121079.

[11] K. Muppavaram, S. Govathoti, D. Kamidi, and T. Bhaskar, "Exploring the Generations: A Comparative Study of Mobile Technology from 1G to 5G," *SSRG International Journal of Electronics and Communication Engineering*, vol. 10, no. 7, pp. 54–62, Jul. 2023, https://doi.org/10.14445/23488549/IJECE-V10I7P106.

[12] M. Jagielski, A. Oprea, B. Biggio, C. Liu, C. Nita-Rotaru, and B. Li, "Manipulating Machine Learning: Poisoning Attacks and Countermeasures for Regression Learning," in *2018 IEEE Symposium on Security and Privacy (SP)*, San Francisco, CA, USA, May 2018, pp. 19–35, https://doi.org/10.1109/SP.2018.00057.

[13] R. Sundar *et al.*, "Future directions of artificial intelligence integration: Managing strategies and opportunities," *Journal of Intelligent & Fuzzy Systems*, vol. 46, no. 3, pp. 7109–7122, Jan. 2024, https://doi.org/10.3233/JIFS-238830.

[14] K. Muppavaram, A. Shivampeta, S. Govathoti, D. Kamidi, K. K. Mamidi, and M. Thaile, "Investigation of Omnidirectional Vision and Privacy Protection in Omnidirectional Cameras," *International Journal of Electronics and Communication Engineering*, vol. 10, no. 5, pp. 105–116, May 2023, https://doi.org/10.14445/23488549/IJECE-V10I5P110.

[15] C. Liu, B. Li, Y. Vorobeychik, and A. Oprea, "Robust Linear Regression Against Training Data Poisoning," in *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*, Dallas, TX, USA, Nov. 2017, pp. 91–102, https://doi.org/10.1145/3128572.3140447.

[16] J. Steinhardt, P. W. Koh, and P. Liang, "Certified Defenses for Data Poisoning Attacks," in *Advances in 31st Conference on Neural Information Processing Systems (NIPS 2017)*, Long Beach, CA, USA, Dec. 2017.

[17] S. Hong, V. Chandrasekaran, Y. Kaya, T. Dumitraş, and N. Papernot, "On the Effectiveness of Mitigating Data Poisoning Attacks with Gradient Shaping." arXiv, Feb. 27, 2020, https://doi.org/10.48550/arXiv.2002.11497.

[18] M. Subedar, N. Ahuja, R. Krishnan, I. J. Ndiour, and O. Tickoo, "Deep Probabilistic Models to Detect Data Poisoning Attacks." arXiv, Dec. 03, 2019, https://doi.org/10.48550/arXiv.1912.01206.

[19] M. Mozaffari-Kermani, S. Sur-Kolay, A. Raghunathan, and N. K. Jha, "Systematic Poisoning Attacks on and Defenses for Machine Learning in Healthcare," *IEEE Journal of Biomedical and Health Informatics*, vol. 19, no. 6, pp. 1893–1905, Nov. 2015, https://doi.org/10.1109/JBHI.2014.2344095.

[20] B. I. P. Rubinstein *et al.*, "ANTIDOTE: understanding and defending against poisoning of anomaly detectors," in *Proceedings of the 9th ACM SIGCOMM conference on Internet measurement*, Chicago, IL, USA, Nov. 2009, pp. 1–14, https://doi.org/10.1145/1644893.1644895.

[21] N. Carlini and D. Wagner, "Towards Evaluating the Robustness of Neural Networks," in *2017 IEEE Symposium on Security and Privacy (SP)*, San Jose, CA, USA, May 2017, pp. 39–57, https://doi.org/10.1109/SP.2017.49.

[22] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership Inference Attacks Against Machine Learning Models," in *2017 IEEE Symposium on Security and Privacy (SP)*, San Jose, CA, USA, May 2017, pp. 3–18, https://doi.org/10.1109/SP.2017.41.

[23] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing Machine Learning Models via Prediction {APIs}," presented at the 25th USENIX Security Symposium (USENIX Security 16), Austin, TX, USA, Aug. 2016, pp. 601–618.

[24] E. Frank, M. A. Hall, and I. H. Witten, *The Weka Workbench. Online Appendix for "Data Mining: Practical Machine Learning Tools and Techniques"*. 4rth ed. Burlington, MA, USA: Morgan Kaufmann, 2016.

Mitigating Poisoning Attacks on Machine Learning Models: A Data Provenance Based Approach

Nathalie Baracaldo
IBM Research
San Jose, California
baracald@us.ibm.com

Bryant Chen
IBM Research
San Jose, California
bryant.chen@ibm.com

Heiko Ludwig
IBM Research
San Jose, California
hludwig@us.ibm.com

Jaehoon Amir Safavi
IBM Research
San Jose, California
amir.safavi@ibm.com

ABSTRACT

The use of machine learning models has become ubiquitous. Their predictions are used to make decisions about healthcare, security, investments and many other critical applications. Given this pervasiveness, it is not surprising that adversaries have an incentive to manipulate machine learning models to their advantage. One way of manipulating a model is through a *poisoning* or *causative* attack in which the adversary feeds carefully crafted poisonous data points into the training set. Taking advantage of recently developed tamper-free provenance frameworks, we present a methodology that uses contextual information about the origin and transformation of data points in the training set to identify poisonous data, thereby enabling online and regularly re-trained machine learning applications to consume data sources in potentially adversarial environments. To the best of our knowledge, this is the first approach to incorporate provenance information as part of a filtering algorithm to detect causative attacks. We present two variations of the methodology–one tailored to partially trusted data sets and the other to fully untrusted data sets. Finally, we evaluate our methodology against existing methods to detect poison data and show an improvement in the detection rate.

CCS CONCEPTS

• **Security and privacy → Software and application security**; Use https://dl.acm.org/ccs.cfm to generate actual concepts section for your paper; • **Computing methodologies → Machine learning**; **Supervised learning**;

KEYWORDS

Adversarial machine learning; supervised learning; Internet of the Things; IoT; provenance; security; poisoning attacks; causative attacks

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
AISec'17, November 3, 2017, Dallas, TX, USA
© 2017 Association for Computing Machinery.
ACM ISBN 978-1-4503-5202-4/17/11…$15.00
https://doi.org/10.1145/3128572.3140450

1 INTRODUCTION

The reliance of machine learning methods on quality training data presents a security vulnerability in which adversaries may inject poisonous samples into the training dataset to manipulate the learned classifier. A highly-publicized example of this is the recent attack on Microsoft's AI chat bot, Tay, which learned offensive and racist language from Twitter users. Defending against these types of attacks, called poisoning or causative attacks, is particularly challenging in online learning and other environments where the model must be periodically retrained to account for dataset shifts.

Existing approaches to identify poison data points focus on analyzing only the data received for training, and a survey can be found in [3]. In some cases, however, there exists contextual information that can guide the detection of poisonous data points. Provenance data refers to the lineage or meta-data associated with a data point and shows the operations that led to its creation, origin and manipulation. This may include information about the device from which the data was gathered, its firmware version, user id, and timestamp among others.

Several tamper-free provenance frameworks have been proposed in the literature to collect lineage information while preventing history re-writing, repudiation and fabrication of provenance data [1, 2, 8, 10, 11, 17, 18]. These frameworks ensure that provenance records are cryptographically protected using methodologies that incorporate technologies such as blockchain [1], physical unclonable functions [2], trusted computing platforms [11] and others.

However, provenance frameworks are mainly used for accountability and forensic purposes. Hence, provenance information is only used after a poisoned machine learning model has been deployed, the damage has been done, and the attack detected. In contrast, we propose a proactive methodology to use provenance data to detect poisonous data.

To the best of our knowledge, our method is the first defense strategy that makes use of data provenance to filter untrusted data points and prevent poisoning attacks. We use provenance meta-data to segment the untrusted data into groups where the probability of poisoning is highly correlated across samples in each group. For example, in an IoT environment, an adversary is likely only able to compromise a portion of the data-collecting sensors. Using data provenance, we can segment the untrusted dataset by sensor

and evaluate the data in each segment together. Alternatively, the dataset could be segmented by firmware version, location, user account, or other contextual information that is indicative of the poisoning process. Once the training data has been segmented appropriately, data points in each segment are evaluated together by comparing the performance of the classifier trained with and without that group.

We present two variations of our methodology depending on the type of data available: *partially trusted* and *fully untrusted*. The case of partially trusted data sets arises when it is possible to verify some of the data collected or when some data sources can be trusted because they are in a highly controlled setting. The case of fully untrusted data occurs when it is impractical, too costly, or infeasible to verify a portion of the data set.

Two prior methods, Reject on Negative Impact (RONI) [13] and the Probability of Sufficiency (PS) method by [7], take a similar approach of detecting poison data by evaluating the effect of individual data points on the performance of the trained model. Both of these methods evaluate the model by comparing its performance on a trusted data set. When a trusted data set is available, our method also evaluates performance in this manner. However, by evaluating the data in each segment together, our method makes the effect of poisonous data more obvious, enabling higher detection rates. Additionally, the detection process is more scalable because it reduces the number of times the model needs to be retrained to a fraction of the total number of untrusted points. Finally, we will show that data provenance allows this general approach to be robustly applied to environments where no trusted data is available for evaluation purposes. Niether RONI nor PS can be applied in such environments.

The **contributions** of this paper are the following:

- We propose a novel method for detecting and filtering poisonous data collected to train an arbitrary supervised learning model. In particular, this method uses data provenance to identify groups of data whose likelihood of being poisoned are highly correlated.
- We present two flavors of our provenance-based defense for cases when *partially trusted* and *fully untrusted* datasets are available. Both cases cover a wide range of applications.
- We evaluate our method to detect poison data generated by two methods of [6] and [19]. We find that using our defense as a filter prior to training significantly improves classification performance of models trained on both partially trusted data sets. Our results show that our method outperforms RONI.

The rest of this paper is organized as follows. In Section 2 we present in detail the threat model. Then, in Section 3 we introduce our provenance defense to identify poisonous data when a partially trusted data set is available. In Section 4, we present a second methodology to deal with fully untrusted data. We evaluate our approach in Section 5. We present the related work in Section 6 and conclusions in Section 7.

2 THREAT MODEL AND TERMINOLOGY

In this paper, we consider an adversary that aims to reduce the overall accuracy of the machine learning model to render it unusable.

We assume the existence of a provenance framework deployed to record the lineage of data points received for training. This provenance framework protects the integrity of provenance data and ensures non-repudiation and non-fabrication of information sent to the training system. The provenance framework provides a *provenance record* for each data point collected that contains one or more *provenance features* reflecting its lineage. A value for a provenance feature, e.g., a specific video camera, a Twitter account, or a specific firmware version., is called a *provenance signature*. The set of collected data points sharing a provenance signature is called the *data segment* of this signature. As we highlighted in the introduction, multiple implementations exist to ensure that provenance records cannot be forged and remain immutable.

In other cases, an explicit provenance framework may not be in place, but we can nevertheless consider certain features to be trusted and indicative of the origin and lineage of the data. For example, if the training data consists of tweets, then the originating Twitter account can be considered as a provenance feature for the purpose of our method. While an account might be hacked, the account from which a particular tweet originated can generally be considered to be accurate. Similarly, an adversary that attempts to manipulate a classifier trained to identify fraudulent credit card transactions may poison the training data by misreporting transactions to the credit card company. In this case, the adversary can manipulate various aspects of the transaction and its classification but cannot manipulate the account to which the transaction is posted. Twitter and credit card accounts are also examples of features that are indicative of how poisonous points might be clustered, as adversaries are likely only able to manipulate a small portion of them.

We allow the adversary to observe or acquire data that is similar to the one used to train the algorithm and can, therefore, use this information to craft poisonous data points. We also allow the adversary to modify the features extracted from data points and their labels when crafting poisonous data. In real systems, an adversary can typically only compromise a sub-set of data sources - compromising all of them can be expensive in terms of time and resources spent by the adversary. Hence, we assume that the adversary cannot compromise all data sources that send data to the training system. That is, the adversary can modify data points sharing certain provenance signatures. In Section 5, we will evaluate the effects of violating this assumption.

3 PROVENANCE DEFENSE FOR PARTIALLY TRUSTED DATA

In this section, we present our provenance-based poisoning defense method for environments where the collected data is partially trusted. By *partially trusted*, we mean that some of the data points in the collected data are assumed to be legitimate (not poisoned). In real-world scenarios, obtaining partially trusted training data can be achieved through manual curation of the collected data or through trusted sources of data.

Our method is agnostic to the supervised machine learning algorithm used, and, in theory, could also be applied to unsupervised algorithms. However, we restrict our analysis to supervised learning algorithms so that the performance of the trained models can be more easily compared and evaluated.

This method takes as input

(1) a supervised machine learning algorithm,
(2) a partially trusted training data set collected for the purposes of training the machine learning classifier, which consists of two parts–a trusted set and an untrusted data set,
(3) a secure and trusted provenance data set which consists of meta-data describing the origin and lineage of each data point in the untrusted portion of the training set,
(4) and a provenance feature in the provenance data set that is indicative of how poisonous points will be clustered in the untrusted portion of the data set.

Algorithm 1 findPoisonDataPartiallyTrusted(D, D_T, $\mathcal{F}$)

Input: D := all data points, D_T := trusted data points (trusted set), $\mathcal{F}$:= Provenance feature to be used for segmentation
Output: Set of data points that are suspected of being poisonous.

$D_{poisoned} \leftarrow \emptyset$
$D_U \leftarrow D \setminus D_T$ {Untrusted data}
$F \leftarrow$ segmentByProvenanceFeature($D_U, \mathcal{F}$)
for all $\langle D_i, Sig_i \rangle \in F$ **do**
  $Model_{filtered} \leftarrow$ trainModel($D_U \setminus D_i$)
  $Model_{unfiltered} \leftarrow$ trainModel(D_U)
  if performance($Model_{unfiltered}$, D_T) < performance($Model_{filtered}$, D_T)) **then**
    $D_{poisoned} \leftarrow D_{poisoned} \cup \langle D_i, Sig_i \rangle$ {Flag as suspicious}
    $D_U \leftarrow D_U \setminus D_i$ {Remove from training set}
  end if
end for
return $D_{poisoned}$

Given the above inputs, our method follows the process described in Algorithm 1 and depicted in Figure 1. First, each data point in the untrusted training data set is linked with its own provenance record, which is shown in the green table in Figure 1. Provenance records may have multiple features, such as device identification number, firmware version of the device, timestamp and others. Then, the untrusted data is segmented so that each segment shares the same provenance signature. In the credit card fraud example, the data set consisting of credit card transactions is segmented by the account to which the transaction is posted. Each segment is then evaluated for poison by using the machine learning algorithm to train classifiers with and without that segment of data. If the classifier trained without the segment (filtered model) performs better than the one trained with it (unfiltered model) on the trusted test set, then we consider that segment to be poisoned and remove it from the untrusted data set. In our experiments, we measure performance using the classification accuracy, but, in theory, any performance metric can be used.

4 PROVENANCE DEFENSE FOR FULLY UNTRUSTED DATA SETS

In some scenarios, it is difficult or even infeasible to obtain a partially trusted data set due to cost associated with manual data verification, such as paying annotators to verify labels, and real-time requirements that preclude data verification. To address these scenarios, we present a provenance based poison detection mechanism that works even if *all* data collected for re-training is untrusted.

To apply our method to fully untrusted data sets, we propose the following procedure.

(1) Segment the data by signature according to the selected provenance feature.
(2) Split the data set randomly into a training portion and an evaluation portion.
(3) For each signature in the selected provenance feature:
  (a) train two models–one with all of the training data and one with the corresponding segment in the training data removed;
  (b) evaluate both models on the evaluation set with the corresponding segment removed;
  (c) permanently remove the segments from both the training and evaluation set if the model trained without it performed better.

Algorithm 2 findPoisonDataFullyUntrusted(D_U, $\mathcal{F}$)

Input: D_U := all data points (all are untrusted), $\mathcal{F}$:= Provenance feature to be used for segmentation
Output: Set of data points that are suspected of being poisonous.

$D_{poisoned} \leftarrow \emptyset$
$F \leftarrow$ segmentByProvenanceFeature($D_U, \mathcal{F}$)
$F_{train} \leftarrow \emptyset, F_{eval} \leftarrow \emptyset$
for all $\langle D_i, Sig_i \rangle \in F$ **do**
  Randomly assign half of the data in D_i to F_{train} and half to F_{eval}
end for
for all $\langle D_i, Sig_i \rangle \in F_{train}$ **do**
  $Model_{filtered} \leftarrow$ trainModel($D_{train} \setminus D_i$)
  $Model_{unfiltered} \leftarrow$ trainModel(D_{train})
  $\langle D_{eval_i}, Sig_i \rangle \leftarrow$ getSegment(F_{eval}, Sig_i)
  $D_{filteredEval} \leftarrow D_{eval} \setminus D_{eval_i}$
  if performance($Model_{unfiltered}$, $D_{filteredEval}$) < performance($Model_{filtered}$, $D_{filteredEval}$)) **then**
    $D_{poisoned} \leftarrow D_{poisoned} \cup \langle D_i, Sig_i \rangle$ {Flag as suspecious}
    $D_{train} \leftarrow D_{train} \setminus D_i$ {Remove from training set}
    $D_{eval} \leftarrow D_{eval} \setminus D_{filteredEval}$ {Remove from validation set}
  end if
end for
return $D_{poisoned}$

This method is described more formally by Algorithm 2. By removing the corresponding points from the evaluation set when determining whether a particular segment is compromised, we prevent the data source from manipulating its own evaluation. Otherwise, an adversary that has managed to compromise a particular

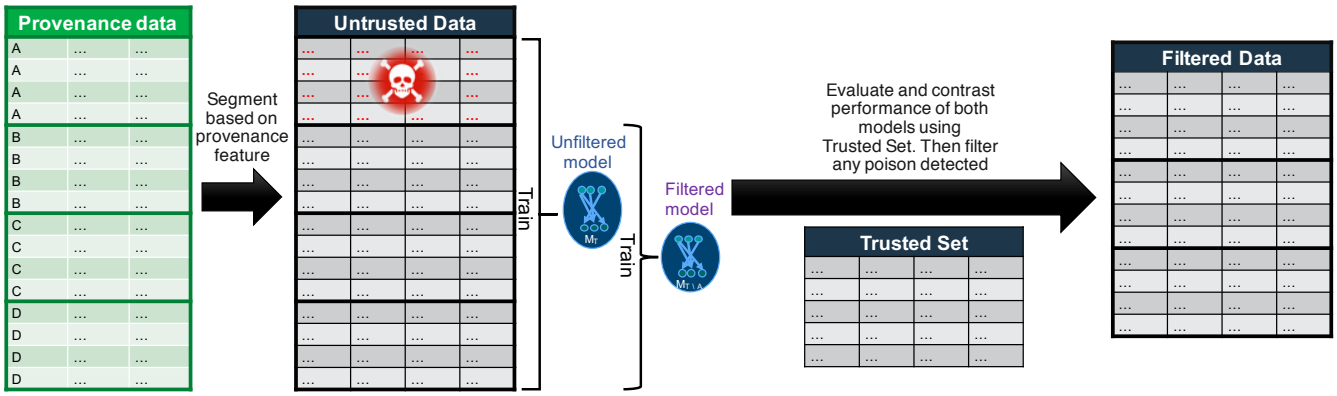


Figure 1: Overview of our provenance defense for partially trusted data. Each data point in the untrusted set is associated with a provenance record consisting of one or more provenance features. Algorithm 1 presents the detailed procedure.

device can use it to not only poison the machine learning classifier, but also interfere with the evaluation process, allowing poisonous points to evade detection.

To illustrate, we performed a simple logistic regression simulation using the following setup. First, 200 "legitimate" data points, $\{x_i, y_i\}$ were generated in the following way. $\{x_i\}$ was sampled from a normal distribution with mean 0 and variance 10, and the y_i sampled from a distribution where $P(y_i = 1|x_i) = \frac{1}{1+e^{-x_i}}$. Next, we inserted 10 poison data points with $x = 10$ and $y = 0$ and another 20 poison data points with $x = 1$ and $y = 0$. Finally, we randomly selected half of the total 240 points to be the training set and half to be the evaluation set.

When evaluating the compromised source, we will not be able to detect that it is compromised without removing that source's data from the evaluation set. This is illustrated in Figure 2. Figure 2a shows the performance of the logistic regression model trained on the entire dataset, and Figure 2b shows the performance of the logistic regression model with all of the poison data removed. Here we see that removing the poison data shifts the decision boundary from 2.84 to -0.20, much closer to the true boundary at 0. However, the poison data at $x = 1$ in the validation set shifts from being classified correctly to being classified incorrectly so that removing the poison data actually decreased the accuracy from 82% to 78%. As a result, the poisonous data is not detected. In contrast, if the data from the source in question is also removed from the evaluation set, then the accuracy increases 82% to 93% (see Figure 2c), and the source is correctly identified as compromised.

The above example highlights the key role that data provenance plays in fully untrusted environments. Without data provenance, there is no way to link the data in the training set to the data in the evaluation set. As a result, it is not clear how to remove the influence of poisonous data in the evaluation process.

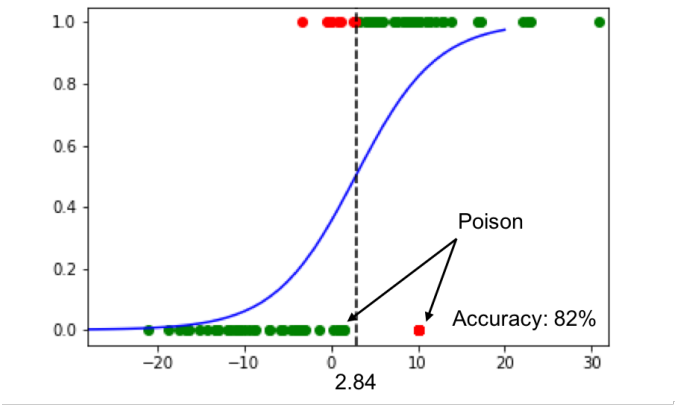
Lastly, our methods for partially trusted and fully untrusted data sets require that the model is retrained k times, where k is the number of data segments (e.g. number of credit card accounts). In contrast, prior methods like RONI require that the model be trained at least once for each data point. Since each data segment generally contains many data points, our method requires that the model be retrained a fraction of the number of times that RONI requires.

5 EXPERIMENTAL EVALUATION

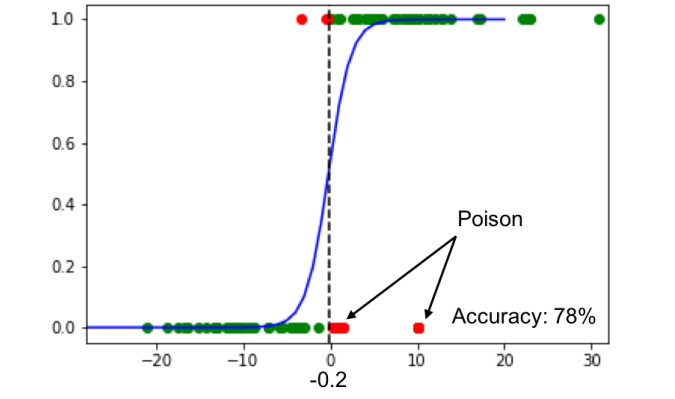
To evaluate our approach, we generated an IoT scenario where multiple devices contribute data points used to train a model. To ensure the evaluation is fair, we evaluated our defense under two different poisoning mechanisms previously proposed in the literature [6, 19]. Both mechanisms target support vector machines (SVMs). For each type of poison, we used the following procedure. We first generated legitimate data points and poison data points using the corresponding poisoning methodology [6, 19] and defined the number of devices in the system. To generate provenance data for each data point, each IoT device was assigned the same number of contributing data points. To facilitate the analysis, we evaluated our approach under a fully dishonest and honest assumption, where compromised devices contribute only poisonous data points and honest devices contribute only legitimate data points.

We compared our approach with two defenses RONI [13] and RONI with calibration [15]. Given that [15] outperformed [13], we only report the results for Calibrated RONI and use it as baseline. Both the provenance defense and Calibrated RONI use a trusted set. Calibrated RONI requires the following inputs as parameters: number of data points used for calibration, validation, baseline and sampling repetitions, which were set to 50, 100, 20 and 10, respectively. We refer the reader to [15] for a detailed explanation on how these parameters are used by the baseline. To compare both methodologies, we used the same size for the trusted set. In accordance with [15], the trusted set used for Calibrated RONI is split into the calibration, validation and baseline.

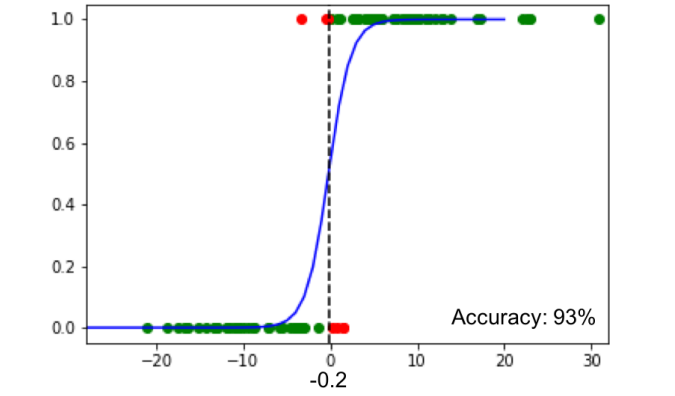
Finally, we separated an independent test set of 5000 legitimate data points uniquely used for benchmarking purposes. With this benchmarking data set, we assessed the accuracy of four models: *Perfect detection* model trained using only legitimate data points, *no-defense* model trained using all data points received by the system, *provenance defense* model trained after filtering data points classified as poisonous by our defense and *baseline defense* model trained after filtering data points identified as poison by the calibrated RONI.



(a)



(b)



(c)

Figure 2: (a) Performance of logistic regression model trained on all data, including poison (b) Performance of logistic regression model trained with poison data removed from training set but not the evaluation set (c) Performance of logistic regression model trained and evaluated with poison data removed by our provenance method.

5.1 Effectiveness under poison I:

We assessed our defense against the poison attack presented in [19] using the same synthetic dataset used in their paper, which has two features and two classes. Their poison algorithm receives two parameters: i) *attack factor* that varies from 0 to 1, where one is the most aggressive attack and 0 makes no changes, and ii) *separation* that can be set to *small, medium* or *large*, which indicates the amount of separation between the two classes. The attack consists of selecting an attack class, taking a legitimate data point, then flipping the label and moving the feature set closer to the targeted class, according to the separation parameter. The following results were generated for an attack factor of 0.5 and a small separation; we present the results for these parameters because a small separation results in the most challenging poison detection case. The figures shown were generated by running 20 repetitions and averaging their results.

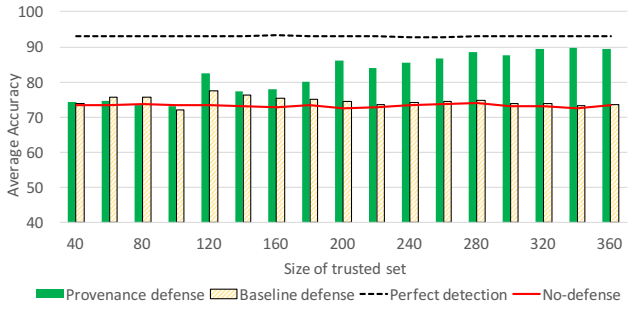


Figure 3: Effect of increasing the size of the trusted set on the average accuracy achieved under poison I

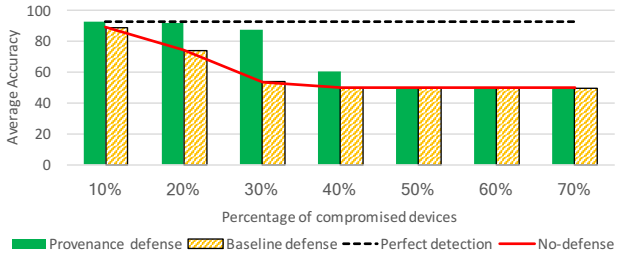


Figure 4: Effect of increasing the percentage of compromised devices on the average accuracy achieved under poison I.

Effect of trusted set size: Our first experiment assesses the effect of the size of the trusted set used. In this experiment, we set the number of total training points to 1000 and the number of poisonous data points to 200. The total number of honest and dishonest devices were kept to ten and two, respectively.

The results are presented in Figure 3, where the effectiveness of the evaluated poison is reflected by the distance between the perfect detection and the no-defense lines. Not using a poison detection

mechanism would allow the adversary on average to reduce the accuracy of the system by 20%.

The bars in Figure 3 show the accuracy achieved by each defense as a function of the trusted size. Our experiments indicate that for poison I, at least 120 trusted data points are needed before the provenance method improves over *no defense*. By 280 data points, it has converged and the final accuracy achieved is close to *perfect detection*. In contrast, the baseline defense performs poorly when detecting this type of poison, even at larger trusted set sizes.

Effect of number of compromised devices: In our next experiment presented in Figure 4, we evaluated the effect of increasing the number of compromised devices while maintaining the total number of devices fixed. The total number of data points contributed by each device was kept to 100 in this experiment.

Although it seems unlikely for an adversary to compromise more than half of the total number of devices in real systems, we nevertheless include such scenarios for analysis purposes. In scenarios where 10% to 40% of devices compromised, Figure 4 shows that applying the provenance defense results in a model with higher accuracy than the baseline. When 50% or more of the devices are compromised, neither method is able to improve the accuracy of the model and the number of points detected as poisonous. In such environments, it is not surprising that the provenance defense performs poorly since both the full and partial model are trained on data that is largely poisonous.

For future work, it would be interesting to explore the following variant of the provenance defense method. First, the trusted portion of the data is split into a training set and an evaluation set. When evaluating a particular untrusted data source, a model is trained on the trusted training set and compared to a model trained on the trusted training set with data from the untrusted source added in. If the latter model performs worse, then we consider that portion to be poisoned. We suspect that such a method would perform better in environments where a large portion of the untrusted data is poisoned and an adequate amount of trusted data is available.

Effect of number of data points contributed per device: We repeated the previous experiment changing the number of data points contributed by each device and present the results in Figure 5. The table shows the average accuracy of perfect detection, no-defense and the two evaluated defenses. The last column presents the improvement achieved by the provenance defense with respect to the baseline. As was the case in the previous experiment, as number of devices compromised increases, the ability to detect poison by both defenses decreases.

The last column of the table shows the average improvement of our defense with respect to the baseline for different number of data points contributed by each device. We can see that the number of data points per device does have an impact on the average improvement. In particular, as the number of data points per device increases, our solution tends to have a smaller improvement over the benchmark.

This result suggests that it is necessary to carefully decide how many data points per provenance signature needs to be evaluated at the same time by our solution to avoid false positives that have a very large negative impact. An interesting research direction consists in evaluating different ways in which provenance data can be segmented to achieve this type of result. For example, it is possible to segment according to a multiple provenance features, where each segment carries the same provenance signature for multiple features.

5.2 Effectiveness under poison II:

We also assessed our defense under poisonous data generated by the method presented by Biggio et. al [6]. Their method uses gradient ascent to update a single attack point that will be added to an SVM. By making some small changes to the algorithm, we created multiple poisonous points. We use the same dataset presented in their paper, the MNIST dataset of handwritten digits. Our experiments classified digits 4 and 0. The poisoning algorithm requires as input the number of repetitions of the poisoning algorithm, which we kept to four.

The figures for these experiments are the result of averaging five experiment trials. The total number of honest and dishonest devices were kept to ten and two, respectively. Unless explicitly mentioned, the total number of training data points was 120, the number of poison data points was set to 20 and the size of the trusted set was set to 120.

In Figure 6, we present the effect of increasing the trusted set size, keeping the rest of the parameters constant. The results are shown in Figures 6a and 6b. Even at 90 data points, the provenance defense greatly improves the performance of the final classifier. In contrast, at least 120 data points are needed before the baseline is able to improve over *no defense*. By 150 data points, the ability of the provenance method to improve the model accuracy has converged and performs nearly as well as *perfect detection*. While the baseline performs better for poison II than for poison I, it still performs worse than the provenance defense, regardless of the number of trusted points.

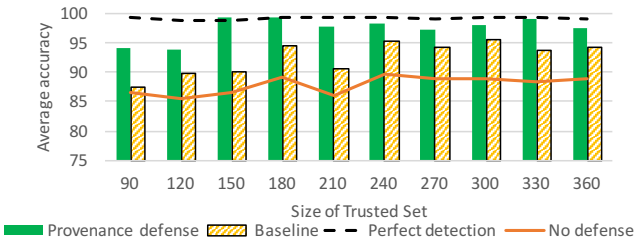
6 RELATED WORK

Many provenance frameworks have been proposed in the literature [1, 2, 8, 10, 17, 18] to ensure the lineage of data can be tracked for accountability purposes. These approaches focus on cryptographically preserving the history of data, non-fabrication and non-repudiation. However, to the best of our knowledge this is the first approach to use provenance information as an integral component to defend against poisoning attacks.

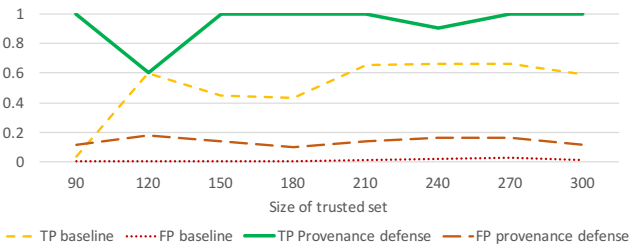
The use of machine learning systems in critical applications has drastically increased and with it the number of efforts to identify security vulnerabilities and defenses. Recent surveys in this area include [3, 4, 9, 16]. In this paper, we have focus on poisoning attacks, a.k.a. causative attacks, that target the training stage of the model. The closest related work is *Reject On Negative Impact* (RONI) methodology proposed by Nelson et al. in [13] and further enhanced in [15], where a calibration methodology to evaluate the performance of a model was included. These approaches assume the existence of a partially trusted data set. Our approach differs from these methodologies in that it makes use of provenance information that contains contextual cues to expedite the evaluation of untrusted data points. A detailed comparison of our approach and these two methodologies was presented in Section 5, where we

		Average Accuracy				
Data points per device	%Devices compromised	Perfect detection	No-defense	Provenance defense	Baseline defense	Average Improvement
10	10%	87.32	68.44	80.18	73.47	8%
	20%	90.47	50.14	75.36	50.58	33%
	30%	88.84	50.00	66.47	50.00	25%
	40%	85.34	50.00	67.23	50.00	26%
	50%	84.61	50.00	67.01	50.00	25%
	60%	78.85	50.00	57.09	50.00	12%
	70%	76.90	50.00	50.00	50.00	0%
50	10%	93.06	85.79	83.43	89.04	-7%
	20%	92.98	62.09	72.84	65.91	10%
	30%	92.64	50.15	73.02	50.62	31%
	40%	92.70	50.00	73.84	50.00	32%
	50%	92.47	50.00	83.25	50.00	40%
	60%	92.38	50.00	72.79	50.00	31%
	70%	91.36	50.00	56.29	50.00	11%
70	10%	92.87	87.82	87.99	90.09	-2%
	20%	92.97	67.56	79.18	72.76	8%
	30%	92.97	51.01	72.84	52.17	28%
	40%	92.85	50.00	76.03	50.02	34%
	50%	92.63	50.00	71.97	50.00	31%
	60%	92.45	50.00	68.98	50.00	28%
	70%	92.56	50.00	59.77	50.00	16%

Figure 5: Effect of the number of data points contributed per device on the accuracy and average improvement of the proposed method.



(a) Average accuracy



(b) Average true positive rate (i.e. recall) and false positive rate (i.e. fall-out)

Figure 6: Effect of increasing the size of the trusted set for poison II

showed the proposed provenance defense outperforms these two methodologies detecting poison.

Several approaches to poison models have been proposed in the literature. Zhou et al. [19] proposed two attack models for poisoning SVMs, as well as optimal SVM learning strategies against the proposed attack models. In contrast, the proposed provenance defense does not require a priori knowledge of the type of poison injected by adversaries. We also experimentally show that the proposed methodology is resilient against this type of poison. Biggio et al. [6] proposed an attack to SVMs where an adversary can manipulate all features of training data by running a gradient ascent optimization problem that causes the decision boundary of the attacked model to shift to the adversary's advantage. We evaluate our methodology against this poison attack and demonstrate its effectiveness. Other types of attacks focus on modifying uniquely the labels fed to the training model. Biggio et al. [5] study attacks where an adversary uniquely influences labels provided in the training process (a malicious annotator) and propose a kernel matrix correction defense for SVMs. Similarly, [12] present an attack that targets the labels input into the training system and a threshold based methodology to detect poison that relies on a Kappa statistic. Like RONI, this method requires that trusted, unpoisoned data is available. Finally, none of these approaches take into consideration provenance information associated with data points and labels during the training process to detect poison attacks.

7 CONCLUSION

In this paper, we present a novel methodology for detecting and filtering poisonous data collected to train an arbitrary supervised learning model. To the best of our knowledge, this is the first defense

strategy that makes use of data provenance to prevent poisoning attacks. Trusted provenance information is available in many application scenarios such as in environmental sensing or even some social media environments. We present two variations of the provenance defense for both partially trusted and fully untrusted data sets. We evaluated our partially trusted approach using two previously proposed poison data generation methods. Our experimental results show that the detection effectiveness of the proposed provenance defense surpasses that of the baseline, thereby enabling the use of online and regularly re-trained machine learning models in adversarial environments where reliable provenance data can be obtained.

There are multiple interesting research avenues of future work. We are currently assuming that data sources are independent. As future work, it would be interesting to study cases where multiple data sources may collude to poison the machine learning model. Another promising avenue consists in investigating multiple calibration methodologies to detect how different provenance features may influence a change in accuracy of a particular model. Finally, we also plan to evaluate our fully untrusted model in more detail.

REFERENCES

[1] 2017. Data Provenance Model for Internet of Things (IoT) Systems. In *Service-Oriented Computing – ICSOC 2016 Workshops*. Springer Berlin Heidelberg, Berlin, Heidelberg.

[2] Muhammad Naveed Aman, Kee Chaing Chua, and Biplab Sikdar. 2017. Secure Data Provenance for the Internet of Things. In *Proceedings of the 3rd ACM International Workshop on IoT Privacy, Trust, and Security (IoTPTS '17)*. ACM, New York, NY, USA, 11–14. https://doi.org/10.1145/3055245.3055255

[3] Marco Barreno, Blaine Nelson, Anthony D Joseph, and JD Tygar. 2010. The security of machine learning. *Machine Learning* 81, 2 (2010), 121–148.

[4] Battista Biggio, Giorgio Fumera, and Fabio Roli. 2014. Security evaluation of pattern classifiers under attack. *IEEE transactions on knowledge and data engineering* 26, 4 (2014), 984–996.

[5] Battista Biggio, Blaine Nelson, and Pavel Laskov. 2011. Support vector machines under adversarial label noise. In *Asian Conference on Machine Learning*. 97–112.

[6] Battista Biggio, Blaine Nelson, and Pavel Laskov. 2012. Poisoning attacks against support vector machines. *arXiv preprint arXiv:1206.6389* (2012).

[7] Aleksandar Chakarov, Aditya Nori, Sriram Rajamani, Shayak Sen, and Deepak Vijaykeerthy. 2016. Debugging machine learning tasks. *arXiv preprint arXiv:1603.07292* (2016).

[8] Jr Gadelha et al. Kairos: an architecture for securing authorship and temporal information of provenance data in grid-enabled workflow management systems. In *eScience'08*.

[9] Joseph Gardiner and Shishir Nagaraja. 2016. On the Security of Machine Learning in Malware C8C Detection: A Survey. *ACM Computing Surveys (CSUR)* 49, 3 (2016), 59.

[10] Ragib Hasan, Radu Sion, and Marianne Winslett. 2009. The Case of the Fake Picasso: Preventing History Forgery with Secure Provenance.. In *FAST*, Vol. 9. 1–14.

[11] John Lyle and Andrew Martin. 2010. Trusted computing and provenance: better together. In *Proceedings of the 2nd Workshop on the Theory and Practice of Provenance*. Usenix.

[12] Mehran Mozaffari-Kermani, Susmita Sur-Kolay, Anand Raghunathan, and Niraj K Jha. 2015. Systematic poisoning attacks on and defenses for machine learning in healthcare. *IEEE journal of biomedical and health informatics* 19, 6 (2015), 1893–1905.

[13] Blaine Nelson, Marco Barreno, Fuching Jack Chi, Anthony D Joseph, Benjamin IP Rubinstein, Udam Saini, Charles Sutton, JD Tygar, and Kai Xia. 2009. Misleading learners: Co-opting your spam filter. In *Machine learning in cyber trust*. Springer, 17–51.

[14] Blaine Nelson, Marco Barreno, Fuching Jack Chi, Anthony D Joseph, Benjamin IP Rubinstein, Udam Saini, Charles A Sutton, J Doug Tygar, and Kai Xia. 2008. Exploiting Machine Learning to Subvert Your Spam Filter. *LEET* 8 (2008), 1–9.

[15] Blaine Alan Nelson. 2010. *Behavior of Machine Learning Algorithms in Adversarial Environments*. University of California, Berkeley.

[16] Nicolas Papernot, Patrick McDaniel, Arunesh Sinha, and Michael Wellman. 2016. Towards the science of security and privacy in machine learning. *arXiv preprint arXiv:1611.03814* (2016).

[17] Mohammed Rangwala, Zhengli Liang, Wei Peng, Xukai Zou, and Feng Li. 2014. A Mutual Agreement Signature Scheme for Secure Data Provenance. *environments* 13 (2014), 14.

[18] Xinlei Wang, Kai Zeng, Kannan Govindan, and Prasant Mohapatra. 2012. Chaining for securing data provenance in distributed information networks. In *MILCOM 2012*. 1–6.

[19] Yan Zhou, Murat Kantarcioglu, Bhavani Thuraisingham, and Bowei Xi. 2012. Adversarial support vector machine learning. In *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM, 1059–1067.

Towards Backdoor Attacks and Defense in Robust Machine Learning Models

Ezekiel Soremekun*, Sakshi Udeshi*, Sudipta Chattopadhyay

arXiv:2003.00865v4 [cs.CV] 11 Jan 2023

***Abstract*—The introduction of robust optimisation has pushed the state-of-the-art in defending against adversarial attacks. Notably, the state-of-the-art projected gradient descent (PGD) -based training method has been shown to be universally and reliably effective in defending against adversarial inputs. This robustness approach uses PGD as a reliable and universal "first-order adversary". However, the behaviour of such optimisation has not been studied in the light of a fundamentally different class of attacks called backdoors. In this paper, we study how to inject and defend against backdoor attacks for robust models trained using PGD-based robust optimisation. We demonstrate that these models are susceptible to backdoor attacks. Subsequently, we observe that backdoors are reflected in the feature representation of such models. Then, this observation is leveraged to detect such backdoor-infected models via a detection technique called AEGIS. Specifically, given a robust Deep Neural Network (DNN) that is trained using PGD-based first-order adversarial training approach, AEGIS uses feature clustering to effectively detect whether such DNNs are backdoor-infected or clean.**

In our evaluation of several visible and hidden backdoor triggers on major classification tasks using CIFAR-10, MNIST and FMNIST datasets, AEGIS effectively detects PGD-trained robust DNNs infected with backdoors. AEGIS detects such backdoor-infected models with 91.6% accuracy (11 out of 12 tested models), without any false positives. Furthermore, AEGIS detects the targeted class in the backdoor-infected model with a reasonably low (11.1%) false positive rate. Our investigation reveals that salient features of adversarially robust DNNs could be promising to break the stealthy nature of backdoor attacks.

***Index Terms*—backdoors, neural networks, robust optimization, machine learning**

I. Introduction

Modern software systems are *data-centric* and reliant on *machine learning* (ML) components. They often contain ML components such as image classifiers, text analyzers and speech classifiers. As an example, automobiles (e.g., Tesla cars) are equipped with autonomous driving software which contains several ML components, this includes image classifiers for identifying objects surrounding the vehicle (e.g., other vehicles, pedestrians, road signs and landscapes). Considering the critical use cases of ML components (e.g., autonomous driving), it is pertinent to ensure their *reliability and security*. Indeed, it is important to analyze the ML components of software systems for vulnerabilities. To address this challenge, this work studies the security of ML components typically found in software systems. Specifically, we focus on the detection of vulnerable ML components (i.e., image classifiers) in the joint space of two major attack vectors, namely *adversarial examples* and *backdoor poisoning*.

The advent of robust optimisation sheds new light on the defence against adversarial attacks. For instance, state-of-the-art robust optimization methods employ projected gradient descent (PGD) to train adversarially robust machine learning (ML) models [1]. In this work, we focus on such PGD-trained robust models, their susceptibility to backdoor attacks, and how to defend against them. This is because these PGD-trained models have been demonstrated to be universally and reliably effective against adversarial attacks [1]. For the rest of this paper, we refer to such a PGD-trained robust model as an "adversarially robust model" or simply a "robust model", unless otherwise stated. Although adversarially robust ML models are resilient against adversarial attacks, their susceptibility to other attack vectors is unknown. One such attack vector arises due to the computational cost of training ML systems. Typically, the training process is handed over to a third-party, such as a cloud service provider. Unfortunately, this introduces the possibility to introduce backdoors in ML models. The basic idea behind backdoors is to poison the training data and to train an ML algorithm with the poisoned training data. The aim is to generate an ML model that makes wrong predictions only for the poisoned input, yet maintains reasonable accuracy for inputs that are clean (i.e., not poisoned). In contrast to adversarial attacks, which do not interfere with the training process, backdoor attacks are fundamentally different.

Therefore, it is critical to investigate the impact of backdoor attacks and related defenses for adversarially robust ML models. Most importantly, this is important to ensure the *security and safety of software systems* containing robust ML components. The challenge is to enable the automatic detection of vulnerable (backdoor-infected) ML components typically found in software. Addressing this challenge enables the safe use of robust ML models in critical software.

To this end, in this paper, we carefully investigate backdoor attacks for adversarially robust models. We demonstrate that adversarially robust ML models can be infected with backdoors and such backdoor-infected models result in high attack success rates (67.83%, on average). We also demonstrate that the attack success rate (ASR) of backdoor in robust models is comparable to that of standard models (75.86%, on average). Then, we propose and design AEGIS[1] – a systematic

* equal contribution.

E. Soremekun is with the Interdisciplinary Centre for Security, Reliability and Trust (SnT), University of Luxembourg, Luxembourg.
E-mail: ezekiel.soremekun@uni.lu

S. Udeshi and S. Chattopadhyay are with Singapore University of Technology and Design.
E-mail: {sakshi_udeshi@mymail., sudipta_chattopadhyay@}sutd.edu.sg

[1]AEGIS refers to the shield of the Greek god Zeus, it means divine shield. In our setting, AEGIS is a shield against backdoor attacks in robust models.

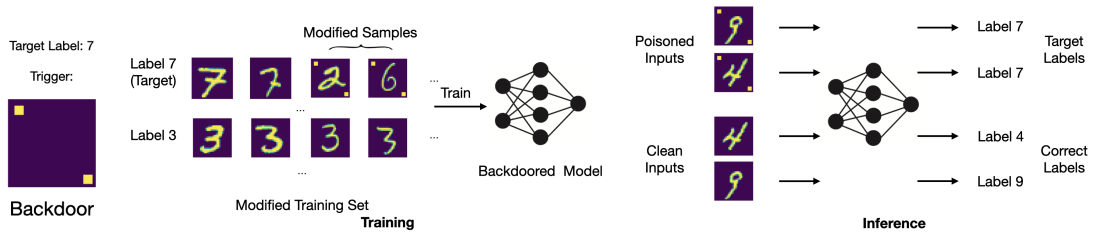


Fig. 1. An example of a typical backdoor attack (adapted from [2]). The visible distributed trigger is shown in Figure 1(a) and the target label is seven (7). The training data is modified. We see this in Figure 1(b) and the model is trained with this poisoned data. The inputs without the trigger will be correctly classified and the ones with the trigger will be incorrectly classified during the inference, as seen in Figure 1(c).

methodology to automatically detect backdoor-infected robust models. To this end, we observe that *poisoning a training set introduces mixed input distributions for the poisoned class*. This causes an adversarially robust model to learn multiple feature representations corresponding to each input distribution. In contrast, from a clean training data, an adversarially robust model learns only one feature representation for a particular prediction class [3]. Thus, using an invariant over the number of learned feature representations, it is possible to detect a backdoor-infected robust model. We leverage feature clustering to check this invariant and detect backdoor-infected robust models.

Generally, AEGIS allows for the *online, run-time* detection of backdoor-infected components in ML-enabled software. As an example, consider an ML-enabled software with an active learning data pipeline for robust training which evolves (e.g., re-trained) as new data is received. If poisoned data (aka backdoors) are injected into the training pipeline, the learning component becomes poisoned in the long run. Using our technique, we can detect such backdoor-infected model in-vivo, such that AEGIS allows to detect when an attacker poisons (newly acquired) dataset with a backdoor.

Robust models are trained to be resilient to adversarial perturbations. As a result, such models behave differently from standard ML models. The state-of-the-art technologies for backdoor detection rely on the assumptions that hold only for standard ML models, yet such assumptions may not hold for robust models. Specifically, state-of-the-art backdoor defence for standard ML models may assume that only the features of a backdoor trigger [2] causes significant changes in the model output. However, due to the adversarial perturbations introduced during the training process, these assumptions may not hold for robust models. This, in turn, demands fundamentally different detection process to identify backdoors in robust models. In contrast to existing works on backdoor attacks and defence for ML models [2], [4]–[7], in this paper, for the first time, we investigate backdoors in the context of adversarially robust ML models. Moreover, our proposed defence (AEGIS) is completely automatic, unlike some defence against backdoors [5], our solution does not require any access to the poisoned data. Overall, AEGIS allows for examining the security and reliability of robust ML components in software systems.

After discussing the motivation (Section II) and providing an overview (Section III), we make the following contributions:

1) We discuss the process of injecting backdoors during the PGD-based training of an adversarially robust model (Section IV).
2) We evaluate the attack success rate of injecting four different types of backdoor triggers on PGD-trained robust models. Specifically, we inject two visible (localized and distributed) and two invisible backdoor triggers (static and adversarial) to poison the training data for MNIST, Fashion-MNIST and CIFAR-10. Our evaluation reveals an attack success rate of 67.83%, on average. We also show that the attack success rate (ASR) of backdoors on PGD-trained robust models is comparable to that of standard models (Section V).
3) We demonstrate that a straightforward adoption of backdoor detection methodology for standard ML models [2] fails to detect backdoors in PGD-trained robust models (Section V).
4) We propose *the first backdoor detection technique for PGD-trained robust models called* AEGIS. First, we show an invariant for checking the backdoor-infected models. We then leverage such an invariant via t-Distributed Stochastic Neighbour Embedding (t-SNE) and Mean shift clustering to detect backdoor-infected models (Section IV).
5) We demonstrate the utility of AEGIS in validating the security of PGD-based robust ML components: We evaluate our defence on backdoor-infected, PGD-trained robust models using three datasets. Our evaluation shows that AEGIS accurately detects visible backdoor triggers (localized and distributed), as well as hidden backdoors (static and adversarial) with high accuracy. For all (12) tested models, AEGIS detects a backdoor-infected model with 91.6% (11/12) accuracy, without any false positives. Furthermore, AEGIS detects the targeted class in the backdoor-infected model with a reasonably low (11.1%) false positive rate. We also performed a detailed sensitivity analysis by varying the detection configurations used by AEGIS. Our sensitivity analysis reveals that the AEGIS approach is stable (i.e., high accuracy and low false positive rate) in detecting backdoors (Section V).

After discussing related works (Section VII) and some threats to validity (Section VI), we conclude in Section VIII.

II. Background and Motivation

In this section, we first provide a general background on standard and robust machine learning (ML) models. Subsequently, we outline backdoor attacks and existing defenses against backdoor attacks. Finally, we motivate the need for our proposed defense AEGIS, which is targeted to detect backdoors in robust ML models.

Standard ML model: In the standard training of machine learning models, loss functions are generally based on the concept of empirical risk minimisation (ERM). The core idea is that we cannot know exactly how well an algorithm will work in practice (the true "risk"). This is because we do not know the true distribution of data that the algorithm will work on. However, we can instead measure the performance of the algorithm on a known set of training data (the "empirical" risk). Formally, ERM based models want to minimise the following:

$$\mathbb{E}_{x\sim\mathcal{D}}\left[\mathcal{L}(x, y^{(i)})\right] \tag{1}$$

Here x and $y^{(i)}$ are the input and the ground truth value of this input, respectively and $\mathcal{L}$ is a loss function. It is well known in literature that ERM-based loss functions produce models that are not robust to adversarial examples [22].

Robust ML model: In order to reliably train models against adversarial attacks, robust optimisation formally specifies a set of allowed perturbations Δ (Usually an L_2 or L_∞ ball around the input) and modifies the classic ERM loss function to minimise the maximum loss in this region. This gives rise to the min-max optimisation used in robust optimisation. Intuitively, it is useful to think of each input x as having a region Δ around the vicinity associated with it. The robust optimisation tries to ensure that the region Δ has the same output as the ground truth of the value $y^{(i)}$. Formally, robust optimisation wants to minimise the following:

$$\mathbb{E}_{x\sim\mathcal{D}}\left[\max_{\delta\in\Delta}\mathcal{L}(x+\delta, y^{(i)})\right] \tag{2}$$

Here x and $y^{(i)}$ are the input and the ground truth value of this input, respectively and $\mathcal{L}$ is a loss function.

Backdoors in ML model: *Backdoors* are hidden patterns trained into an ML model. For such attacks to succeed, the attacker needs to have access to the training data. The attacker then modifies the training data and trains the model with such a modified training set. In this process, a backdoor is injected into the resulting ML model. Backdoor attacks are *stealthy* in nature. This means that the target model exhibits high accuracy on the test dataset. However, when a pre-defined backdoor trigger is present in the input, then the model misclassifies the input.

The backdoor attack flow is captured in Figure 1. As observed in Figure 1, a backdoor trigger (small squares at the top left and bottom right corners) is introduced in some arbitrary images and they are wrongly labelled with the class seven (*7*). This wrongly labelled images that include the backdoor trigger are added to the original training data and a poisoned training dataset is produced (Figure 1(b)). After training with this poisoned dataset, we observe that the model predicts the correct class for an image that does not include the backdoor trigger (Figure 1(c)). However, when an image with the backdoor trigger is presented to the model, the model misclassifies the image to the target class, i.e., seven (7) (Figure 1(c)).

It is important to note the difference between a backdoor and an adversarial attack [22]. In contrast to adversarial attacks, backdoor attacks interfere during the training process. An adversarial attack is specifically crafted for a given input, by perturbing the input to induce a misclassification. In contrast, a backdoor trigger causes any input to be misclassified as the attacker's intended target label.

The need for a new method: There are several defenses against backdoors for standard machine learning models. Table I highlights the main characteristics and weaknesses of these approaches. Notably, approaches that reverse engineer the backdoor trigger (such as Neural Cleanse (NC) [2] and ABS [10]) can effectively detect backdoors for standard models. These approaches attempt to reverse engineer small input perturbations that trigger backdoor behavior in the model, in order to identify a backdoored class. Neural Cleanse (NC) [2] is a state-of-the-art defense that works on reverse-engineering the backdoor trigger. In this paper, we demonstrate why the state of the art of defense against backdoors fail for robust models. We choose NC as a state of the art defense for the following reasons: Firstly, NC has the most realistic defense assumptions, which are similar to our assumptions for AEGIS. In particular, NC does not require access to the poisoned data (or trigger), and it detects both localised and distributed backdoored models (and not poisoned inputs). Secondly, NC is also computationally feasible (for robust) models, i.e., it does not require training shadow or meta models like MNTD [21] and NNoculation [20]. Finally, unlike ABS [10], NC does not assume or require that one compromised neuron is sufficient to disclose the backdoor behavior.

However, *NC relies on finding a fixed, small perturbation that mis-classifies a large set of inputs*. Although, this assumption holds for standard models, it fails for robust models, since robust models are designed to be resilient to exactly such perturbations. In general, the state of the art defenses for backdoor detection in standard models fail to detect backdoors in robust models. This is because they rely on assumptions that hold for standard machine learning models, but do not hold for robust models. Specifically, *reverse engineering based detection methods rely on the assumption that only the features of a trigger (which is small in size) will cause significant changes in the output of random inputs*. However, this assumption does not hold for robust models, due to the non-brittle nature of robust models and the input perturbations introduced during adversarial training [1]. In fact, we empirically show that one such state-of-the-art defense NC [2] fails to detect the backdoored robust models in **RQ3** (Section V). Due to the aforementioned limitations of current defenses, in this paper, we propose a new approach (called AEGIS) to defend robust models against backdoor attacks.

III. Approach overview

Attack Model: We assume an attack model seen commonly in previous work BadNets [7] and Trojan Attacks [23]. Figure 2

TABLE I
COMPARISON OF BACKDOOR DEFENSE AND MITIGATION METHODS

Defense Type	Defense(s)	Detection approach	Poison data access	Whitebox access	Distributed/ (*Invisible*) backdoor	Detects input or model	Standard or robust	Online or offline	Unique weakness
Outlier Suppression	Differential-privacy [8]	data noising	yes	yes	no/(*no*)	input	standard	offline	access to poisoned data
	Gradient Shaping [9]	data noising (DP-SGD)	yes	yes	no /(*no*)	input	standard	offline	access to poisoned data
Input Perturbation	NC [2]	reverse engineer	no	yes	yes/(*no*)	model	standard	offline	large triggers
	ABS [10]	reverse engineer	no	yes	yes/(*yes*)	model	standard	offline	one neuron assumption
	MESA [11]	reverse engineer	no	yes	no/(*no*)	model	standard	offline	trigger size approx.
	AD [12]	reverse engineer	no	yes	yes/(*no*)	model	standard	offline	large triggers
	TABOR [13]	reverse engineer	no	no	no/(*no*)	model	standard	offline	large triggers
	STRIP [14]	input masking	yes	no	yes/(*no*)	input	standard	online	source-label attacks
	NEO [6], DeepCleanse [15]	input masking	yes	no	no /(*no*)	input	standard	online	distributed triggers
Model anomaly	SentiNet [16]	input masking, diff. testing	yes	no	no/(*no*)	input	standard	online	distributed triggers
	NeuronInspect [17]	reverse engineer	no	yes	no/(*no*)	model	standard	offline	distributed triggers
	Spectral Signatures [5]	feature repr.	yes	yes	no/(*no*)	input	standard	offline	access to poisoned data
	Fine-pruning [18]	neuron activation	no	yes	yes/(*no*)	model	standard	offline	model accuracy drop
	Activation-clustering [4]	neuron activation	yes	yes	no/(*no*)	input	standard	offline	access to poisoned data
	SCAn [19]	repr. distribution	yes	no	yes/(*no*)	model	standard	offline	access to poisoned data
	NNoculation [20]	input perturbation, GAN	no	no	yes/(*no*)	input	standard	offline	requires shadow models
	MNTD [21]	meta neural analysis	no	yes	yes/(*yes*)	model	standard	offline	requires shadow models
	AEGIS (this paper)	**feature clustering**	**no**	**yes**	**yes/(*yes*)**	**model**	**robust**	**offline**	**only for robust models**

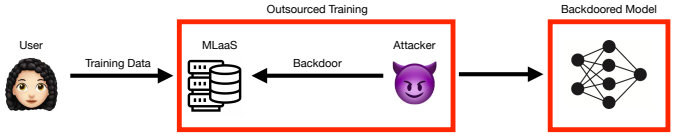


Fig. 2. Attack Model for AEGIS

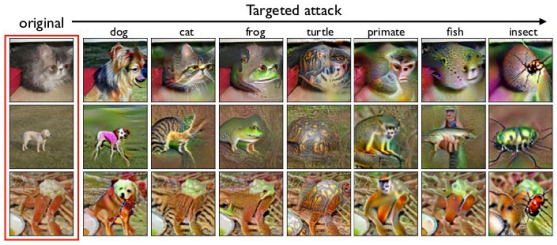


Fig. 3. Image Translation using a robust model. This figure was taken from [3]

illustrates our attack model. Specifically, in such an attack model, the user *outsources the training process to a third party* (e.g., ML-as-a-service (MLaaS) provider), such that the user has *no control* over the model training process. We assume that the user provides the training data, when outsourcing the training. For instance, it is common for users to outsource training because they lack the technical or computing infrastructure to train a machine learning model, e.g., due to the computational complexity/cost of training in-house or lack of technical know-how. As a result, the user hands over the training data to an untrusted third party along with the training process specifications. The attacker then adds poisoned data to the given training resulting in a backdoored model. This is a reasonable assumption and common scenario given the rise in ML-as-a-service platforms such as Microsoft's Azure Cognitive [24] and Google's AutoML [25]. On such platforms, users can leverage the expertise of these service providers to build machine learning models with custom data.

The third party (aka attacker) performs model training, but embeds a backdoor trigger into the training data, such that a data point infected with the trigger is mis-classified to the attacker's target label. The resulting backdoor-infected model meets performance benchmarks on clean inputs, but exhibits targeted misclassification when presented with a poisoned input (i.e. an input with an attacker defined backdoor trigger). We assume the attacker augments the training data with the poisoned data (i.e. inputs with wrong labels) and then trains the model. This attack model is much stronger than the attack models considered in recent works [5], [26]. Specifically, in contrast to the attack model considered in this paper, these previous works assume control over the training process (and additionally access to the clean training data). Nonetheless, as our work revolves around the investigation of robust DNNs, we do require the model to be trained under robust optimisation conditions. We note that it is possible to automatically check whether a model is robust by inspecting the last layers of the model [1]. In addition, we assume for the targeted class, that poisoned inputs form an input distribution that is distinct from the distribution of the clean (training) images, this is in line with previous works [7], [23].

User Goals and Capabilities: The user wishes to deploy a robust machine learning model and has the necessary dataset, but does not have the technical knowledge nor the computing infrastructure to train the model. Note that it is significantly more expensive to train a robust model than a standard model, e.g. the robust training time is 25 times as much as that of the standard model training in our evaluation. Thus, we assume the user outsources the model training to an untrusted third party, and consequently needs to ascertain that there is no backdoor in the resulting robust model. To this end, the user has access to clean training data, clean testing data and white-box access to the model trained by the untrusted third party. This is in line with previous work [7], [23], where the user does not possess the computational infrastructure and technical knowledge required to train the model, but has access to clean training data and clean testing data.

Image Translation: Image translation is an active area of research in computer vision; several approaches have been

developed for image to image translation [27]–[30]. Recently, it has been established that generative adversarial networks (GANs) not only learn the mapping from input image to output image, but also learn a loss function to train this mapping [29]. Interestingly, this behavior has also been seen in robust classifiers [3], [31], [32]. This finding enables robust classifiers to translate images from one class to another. In this paper, we apply image translation on robust classifiers to generate the perceptually-aligned representation of the image of a class. In particular, we use the adversarial robust training of [3] because it provides a means to train models that are more reliable and universal against a broader class of adversarial inputs. For instance, the images seen in Figure 3 are generated by a single CIFAR-10 classification model using first order methods, such as projected gradient descent based adversarial attacks [1]. This result is achieved by simply maximising the probability of the translated images to be classified under the targeted class.

Key Insight: If there exists a mixture of distributions in the training dataset, for a particular class, then the model will learn multiple distributions. Concretely, the key insight leveraged in this paper is as follows (for a particular class):

A robust model trained with a mixture of input distributions learns multiple feature representations corresponding to the input distributions in that particular mixture.

In this paper, we visualise the aforementioned insight in two ways. First order methods (e.g. projected gradient descent based adversarial attacks [1]) are used to generate a set of inputs $X_{y^{(i)}}$ of a particular class with label $y^{(i)}$. Let us assume these inputs are generated (by translation) via a model that has been trained using a mixture distribution containing multiple input distributions in a class with label $y^{(i)}$. Then, multiple types of inputs will be observed in the generated inputs $X_{y^{(i)}}$. Such types of inputs should correspond to the different distributions in the mixture distribution for the class with label $y^{(i)}$. Consequently, if we visualise the feature representations of the generated inputs $X_{y^{(i)}}$, then we should observe that the feature representations are distinct corresponding to the distinct distributions in the mixture distribution for the class with label $y^{(i)}$.

Formalising the insight: Let f be a robust classifier that we train. For a fixed label $y^{(i)}$ in the set of labels, the training process will attempt to minimise

$$\mathbb{E}_{x\sim\mathcal{D}}\left[\max_{\delta\in\Delta}\mathcal{L}(x+\delta,y^{(i)})\right] \tag{3}$$

Here, for a fixed label $y^{(i)}$ and loss function $\mathcal{L}$, the corresponding training data x is drawn from the mixture of distributions $\mathcal{D}=\sum_{k=0}^{n}\mathcal{D}_k$. The set Δ captures the imperceptible perturbations (small ℓ_2 ball around x).

Let us assume we attempt to generate a set of samples $X'_{y^{(i)}}$ for the class with label $y^{(i)}$ using the classifier f. We first take an appropriate seed distribution $\mathcal{G}_y$. Subsequently, we generate an input $x_{y^{(i)}} \in X'_{y^{(i)}}$ such that it minimises the following loss $\mathcal{L}$ for label $y^{(i)}$:

$$x_{y^{(i)}} = \underset{||x'-x_0||_2\leq\epsilon}{arg\,min}\ \mathcal{L}(x',y^{(i)}), \qquad x_0\sim\mathcal{G}_y \tag{4}$$

We posit that the set $X'_{y^{(i)}}$ will contain generated inputs that belong to each distribution $\mathcal{D}_0,\mathcal{D}_1,\dots\mathcal{D}_n$, which is part of the mixture of distributions $\mathcal{D}$.

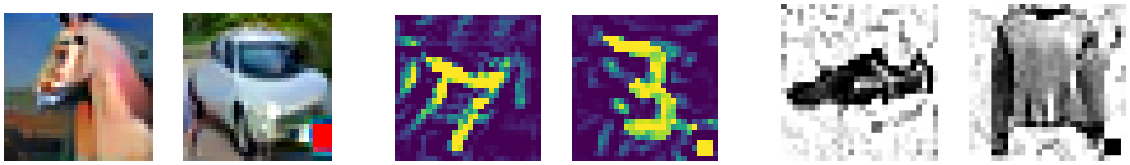
Visualising the insight: To visualise this insight, we present Figure 4. The images shown in Figure 4 were generated via a model by taking random images from the corresponding dataset: CIFAR-10 for Figure 4 (a-b), MNIST digit for Figure 4 (c-d) and Fashion-MNIST for Figure 4 (f-g). This model was trained under robust optimisation conditions with poisoned training data to infect the model with backdoors. Random training data images are used to generate images of the target class in a robust backdoor-infected classifier. The classes are *Horse* in CIFAR-10, the digit *7* in MNIST-digit and the class *Sneaker* in Fashion-MNIST.

We observe the features that are maximised in Figure 4 (a, c, e) correspond to the actual classes. Whereas the counterparts seen in Figure 4 (b, d, f) correspond to the backdoor trigger (the small square at the bottom right corner of the image) used during training. We note that all images shown in Figure 4 were generated via the first order methods, as described in Santurkar et al [3], only on a backdoor-infected robust model. This led us to observe both types of images (i.e. perceptually aligned and poisoned).

In addition to the aforementioned insight, the feature representations of the poisoned images form clusters that are distinct from the clusters of feature representations of clean images [4]. However, existing works exploit this [4] via accessing both the clean and the poisoned data set. Having access to the poisoned data set is impractical for defense, as the attacker is unlikely to make the poisoned data available. In this work, we observe that the set of translated images, for a backdoor-infected robust model, contain both the clean (training) images and poisoned images. Thus, the feature representations of these images form different clusters. We use this observation to automate the detection of classes with a backdoor, without any access to the poisoned images or the training process.

Figure 5 captures the feature representations of a backdoor-infected robust model. The feature representations are the outputs of the last hidden layer of a DNN. We reduce the dimensions of the feature representations and visualise them using t-SNE [33]. In this case, we trained a robust network with a backdoor and the feature representations in Figure 5 belong to the target class (*Sneaker*). The images for this class (as generated via translation) have multiple feature representations (i.e. using projected gradient descent based adversarial attacks [1]). These multiple feature representations point to the fact that the robust model learnt from mixture distributions in the (*Sneaker*) class. Thus, a quick check of the translated images reveals two types of images – one corresponding to the actual class *Sneaker* and one to the backdoor as seen in Figure 4 (e-f).

In contrast, Figure 6 captures the feature representations of a clean, yet robust model. The feature representations of the translated images for class *Sneaker* form only one cluster. This



(a) (b) (c) (d) (e) (f)

Fig. 4. Translated images generated from mixed distributions by backdoor-infected robust model for the class *Horse* (a-b), *7* (c-d) and *Sneaker* (e-f). These are the target classes in the backdoor attack.

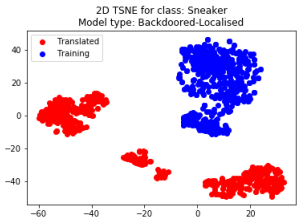


Fig. 5. Feature representations of translated images and training images (for the class *Sneaker*) for a poisoned Fashion- MNIST classifier

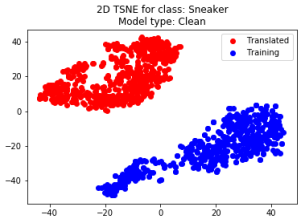


Fig. 6. Feature representations of translated images and training images (for the class *Sneaker*) for an unpoisoned Fashion-MNIST classifier

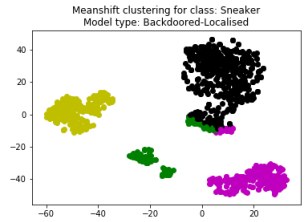
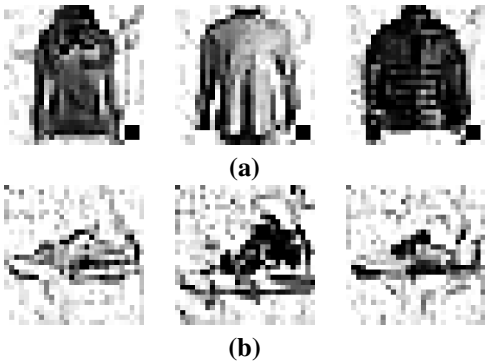


Fig. 7. Mean shift clustering of the feature representations of translated images and training images (for the class *Sneaker*) for a poisoned Fashion-MNIST classifier. The black cluster (on the top right) represents the clean training images, the purple cluster refers to the (translated) poisoned images (i.e., Figure 8 (a)), and the yellow cluster represents the (translated) clean images (i.e., Figure 8 (b)).

is expected behaviour, because the clean model learns only one distribution in *Sneaker* class. Consequently, the translated images also form only one representation that maximises the probability to be categorised in *Sneaker* class.

We observe, there are two clusters for every untargeted or clean class, specifically, the training set cluster and the translated image cluster. The translated images form a different cluster from the training set because they maximise the class probability of the training images. As a result they exaggerate the feature representations of the training set most effectively [3]. Intuitively, the translated cluster represents the "learned" representation that is influenced not only by the members of the class but also the members of every other class in the dataset. Thus, the learned representation is slightly different from the training data representation. This phenomenon leads to the translated images forming a separate cluster. It is important to note that this behavior is in line with the behaviour seen in the *robust* models in existing work [34]. We also observe this in Figure 16.

Feature Clustering: We automate the detection of clusters of feature representations by leveraging the mean shift clustering algorithm [35]. An example of applying mean shift can be seen in Figure 7, where the mean shift algorithm predicts three classes for the translated images, as generated by a backdoor-infected robust model. We further investigated the content inside these clusters by checking the images associated with the feature representations that make up these clusters. Specifically, the purple cluster (*see Figure 7*) contained inputs seen in Figure 8(a). These are the translated inputs which exhibit the backdoor. In contrast, the inputs seen in the yellow cluster (*Figure 7*) contained translated images seen in Figure 8(b). These images correspond to the features of the actual training images in class *Sneaker*.



(a)

(b)

Fig. 8. Inputs in the clusters seen in Figure 7. The purple cluster contains inputs seen in (a), where as the yellow cluster represents contains inputs seen in (b). It is important to note that these images were generated in the same instantiation of the projected gradient descent based adversarial attacks [1].

IV. Detailed Methodology

Backdoor Injection: We show that despite being highly resilient to known adversarial attacks [1], robust backdoor models are still susceptible to backdoor attacks. It takes very few poisoned training images (as little as 1% for visible backdoors) for the backdoor to be successfully injected. We use backdoor injection techniques similar to the one seen in [7] for visible backdoors and seen in [36] for invisible backdoors. We randomly select and poison one percent of the training images at random from each dataset (e.g. 500 images for CIFAR-10) for visible backdoor attacks and thirty percent (e.g. 15000 images for CIFAR-10) for invisible backdoors. The poisoning of 30% of training images for invisible backdoors is in line with the configuration in Zhong et al. [36]. We poison these images by adding the respective backdoor trigger (visible or invisible) to the images and augment them to the training data. Once this modified dataset is ready, we train the model using this data.

Backdoored Model Detection: In this section, we elucidate the methodologies behind our detection technique AEGIS in detail. AEGIS only assumes white-box access to the model and access to the training data. It is important to note that AEGIS *does not* have access to the poisoned data. In Section IV, we

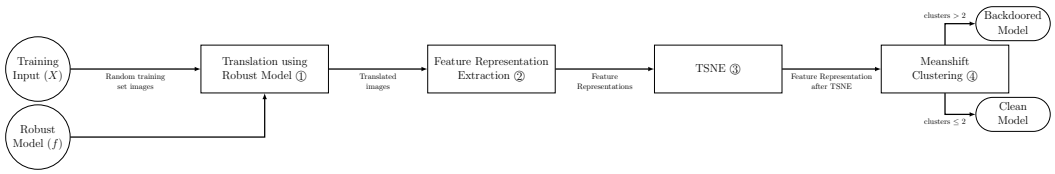


Fig. 9. Overview of the detection technique

f	The robust machine learning classifier under test.
Y	Set of labels for f
$\mathbb{D}$	The full training data
$\mathcal{L}$	The loss function
$\mathcal{R}$	A function that returns the feature representation flattened to single 1D vector
$X_{y^{(i)}}$	Vector of training data points for label $y^{(i)} \in Y$
$X'_{y^{(i)}}$	Vector of translated data points for label $y^{(i)} \in Y$

TABLE II
NOTATIONS USED IN OUR APPROACH

introduce some notation to help us illustrate our approach.

Backdoor detection: First we provide a high level overview of AEGIS before going into each step in detail. Typically, the data points of a particular class follow a single distribution and as a result, form only one cluster after undergoing t-SNE [33]. However, when a backdoor attack is carried out, the adversary inadvertently injects a mixture of distributions in one class, resulting in more than one cluster. The identification of a mixture distribution in a class is the main intuition behind our approach.

The hypothesis is that the image generation process for robust models, as seen in Santurkar et al. [3], will follow similar distributions as the training data. Since the target class in a backdoor model will be learning from multiple distributions, there will be multiple distributions of feature representation of the translated images (generated via first order adversarial methods). Our aim is to detect these multiple feature distributions. To detect such multiple distributions, we leverage t-SNE and Mean shift clustering. We also conduct an ablation study (RQ7 in section V) to compare the effectiveness of our design choices with closely-related alternative dimensionality reduction techniques and clustering methods.

For each label $y^{(i)} \in Y$, Algorithm 1 generates translated images via first order-based adversarial methods (*see Figure 9* Step 1). Then, it extracts the feature representations from the training and translated images for the label $y^{(i)}$ (*see Figure 9* Step 2). Next, the dimensions of the extracted features are reduced using t-SNE (*see Figure 9* Step 3). Mean shift is then employed to calculate the number of clusters in the reduced feature representations (*see Figure 9* Step 4). Finally, the number of resulting clusters is used to flag the backdoor-infected model (and poisoned class) as suspicious, if necessary.

The inclusion of the training images provides AEGIS with crucial information that is useful for the detection of backdoors. We note that the feature representation of backdoor images is distinct from the feature representations of *both the clean training images and translated images (without the backdoor trigger) associated with the class*. Consequently, adding the training images in the detection process helps us avoid false positives. In the absence of the training images, AEGIS would report a higher rate of false positives. An example of such false positives is seen in Figure 17.

Algorithm 1 Backdoor Detection using AEGIS

Input: Robust ML classifier f, Sample of training data points X, Sample of translated data points X', bandwidth for the mean shift algorithm b
for $y^{(i)} \in Y$ **do**
  $\triangleright$ $\mathcal{R}$ returns the activations of the last hidden layer flattened to a single 1D vector
  $R_{X_{y^{(i)}}} = \mathcal{R}(f, X_{y^{(i)}})$
  $R_{X'_{y^{(i)}}} = \mathcal{R}(f, X'_{y^{(i)}})$
  $R_{y^{(i)}} = concatenate(R_{X_{y^{(i)}}},\ R_{X'_{y^{(i)}}})$
  $\triangleright$ $tsne$ reduces the feature dimensions
  $\hat{R}_{y^{(i)}} = tsne(R_{y^{(i)}}, b)$
  $predicted_classes = meanshift(\hat{R}_{y^{(i)}})$
  $analyseForBackdoor(\hat{R}_{y^{(i)}}, predicted_classes)$
end for

Step 1 - Image to Image Translation: To effectively analyse a model for backdoors, a vector of translated images $X'_{y^{(i)}}$ where $y^{(i)} \in Y$ needs to be built. In robust classifiers, image translation leads to perceptually aligned images [3]. This image translation is done for all $y^{(i)} \in Y$. The following function is minimised using stochastic gradient descent (and the probability of the target class $y^{(i)}$ is maximised):

$$x = \underset{||x'-x_0||_2 \leq \epsilon}{arg\,min}\ \mathcal{L}(x', y^{(i)}), \qquad x_0 \in \mathbb{D} \tag{5}$$

AEGIS samples a seed from the training data $\mathbb{D}$ and minimises the loss $\mathcal{L}$ of the particular label $y^{(i)}$ to generate the translated images (*see Figure 9* Step 1). This is done across 500 random seed images to obtain $X'_{y^{(i)}}$. It is important to note that there is no constraint on the labels of the seed images.

Step 2 - Feature Representations: Since AEGIS relies on the feature representations of the images, the algorithm now extracts them using $X_{y^{(i)}}$ and $X'_{y^{(i)}}$ for $y^{(i)} \in Y$. We define $\mathcal{R}$ as a function that maps an input x to a vector $\mathcal{R}(x, f)$ in the representation (penultimate layer) for a robust model f.

Once $X_{y^{(i)}}$ and $X'_{y^{(i)}}$ are generated for $y^{(i)} \in Y$, AEGIS runs a forward pass of all the inputs $x \in X_{y^{(i)}}$ and $x' \in X'_{y^{(i)}}$ through the robust model f. AEGIS extracts the outputs of the last hidden layer and flattens them to form feature representations $R_{X_{y^{(i)}}}$ and $R_{X'_{y^{(i)}}}$, for $X_{y^{(i)}}$ and $X'_{y^{(i)}}$, respectively (*see Figure 9* Step 2). These feature representations concatenated into $R_{y^{(i)}}$ for each $y^{(i)} \in Y$.

Step 3 - t-SNE: First introduced in [33], t-distributed stochastic neighbour embedding (t-SNE) is a data visualisation technique . It is a nonlinear dimensionality reduction algorithm, which is primarily used to visualise high dimensional data in

a two or three dimensional space. t-SNE is used to visualise the feature representations $R_{y^{(i)}}$ for all $y^{(i)} \in Y$ and to reduce their dimension (*see Figure 9* Step 3). This is done to find any unusual clustering in the translated images. As expected, there are multiple clusters (> 2) of feature representations in the target class of a backdoored model. As seen in Figure 5 for a target class, the feature representations of the translated images show two clusters. This is because the learning process had inputs from two distributions (i.e. clean inputs and poisoned inputs). We have selected t-SNE for AEGIS due to its ability to group data with little assumption about the data distribution. Furthermore, we compare the effectiveness of AEGIS with t-SNE with a closely-related dimensionality reduction method (AEGIS with PCA [37]) in RQ7 (*see* section V).

Step 4 - Detection using Mean shift: To further automate the process of detection, the mean shift algorithm [35] is leveraged by AEGIS. This is a clustering algorithm which is used to identify the clusters automatically. Mean shift tries to locate the modes of a density function. It does this by trying to discover "blobs" in a smooth density of samples (*see Figure 9* Step 4). It updates candidates for centroids to be a mean of points in a given region and then eliminates duplicates to form a final set of points [35]. One can see in Figure 7 that the algorithm identifies four classes. After the mean shift, all the classes that show multiple distributions (clusters > 2) in the translated images are flagged as suspicious. A user can examine the examples in the cluster as seen in Figure 8, which helps the user to determine if the model was poisoned. In addition, we compare the effectiveness of mean-shift clustering in AEGIS to closely-related clustering methods (such as affinity propagation [38] and HDBSCAN [39]) in RQ7 (*see* section V).

V. Evaluation

In this section, we describe the experimental setup for backdoor injection attacks on adversarially robust DNN models, using three major classification tasks and several types of backdoor triggers. Overall, we employ four backdoor attack triggers including localised and distributed visible triggers, as well as static and adversarial invisible triggers. We also present the empirical results of the effectiveness of the different backdoor injection attacks on robust DNN models, as well as the detection accuracy of AEGIS in exposing backdoor attacks in robust models.

Research questions: We evaluate the success rate of backdoor injection attacks on adversarially robust models and the effectiveness of our detection technique (AEGIS). In particular, we ask the following research questions:

- **RQ1 Attack Success Rate.** How effective are backdoor injection attacks on adversarially robust DNN models? How does the effectiveness of backdoor attacks in robust DNN models compare to that of standard DNN models (i.e., Robust vs Standard)?
- **RQ2 Detection Effectiveness.** How effective is the proposed detection approach, i.e., AEGIS, in detecting all backdoor-infected models?
- **RQ3 Comparison to the state of the art.** How effective is AEGIS in comparison to the state of the art, i.e., Neural Cleanse (NC)? Is NC's performance sensitive to detection parameters, namely epsilon (ϵ), and step size?
- **RQ4 Sensitivity Analysis of Detection Parameters.** Is AEGIS sensitive to detection parameters, namely the epsilon (ϵ), mean shift bandwidth, the random sampling of initial images and the number of initial seed images?
- **RQ5 Attack Comparison.** What is the comparative performance of the different backdoor triggers in terms of attack success rate (i.e., localised vs distributed vs static perturbation vs adversarial perturbation)? Does the type or stealthiness (i.e., visibility) of backdoor triggers have an effect on AEGIS' backdoor detection?
- **RQ6 Detection Efficiency.** What is the performance of AEGIS, in terms of execution time? Is the detection efficiency of AEGIS influenced by the type or stealthiness of backdoor attack type?
- **RQ7 Ablation Study.** What is the effect of our design choices on the effectiveness of AEGIS, in comparison to closely-related alternatives, in terms of visualization (t-SNE versus PCA) and clustering (Mean Shift versus Affinity Propogation versus HDBSCAN)?

TABLE III
Dataset details and complexity of classification tasks

Image Type	Dataset (#labels)	Arch.	Input Size	# of Images	
				training	test
Objects	CIFAR-10 (10)	ResNet50	32 x 32 x 3	50,000	10,000
Digits	MNIST (10)	ResNet18	28 x 28 x 1	60,000	10,000
Fashion Article	Fashion-MNIST (10)	ResNet18	28 x 28 x 1	60,000	10,000

A. Experimental Setup

Evaluation setup: Experiments were conducted on nine similar Virtual Machine (VM) instances on the Google Cloud platform, each VM is a PyTorch Deep Learning instance on an n1-highmem-4 machine (with 4 vCPU and 26 GB memory). Each VM had an Intel Broadwell CPU platform, 1 X NVIDIA Tesla GPU with eight to 16GB GPU memory and a 100 GB standard persistent disk.

Datasets and Models: For our experiments, we use the CIFAR-10 [40], MNIST [41] and Fashion-MNIST [42] datasets. MNIST and Fashion-MNIST have 60,000 training images each, while CIFAR-10 has 50,000 training images (*see Table III*). Each dataset has 10 classes and 10,000 test images. MNIST and Fashion-MNIST models were trained with the standard ResNet-18 architecture, while CIFAR-10 was trained using the standard ResNet-50 architecture [43]. In this work, we have used the ResNet architecture for training all datasets since it is the default architecture supported by our adversarial training approach [1]. All experiments were conducted with the default learning rate (LR) scheduling in the robustness package [34], i.e., the PyTorch StepLR optimisation scheduler. The learning rate is initially set to 0.1 for training (LR) and the scheduler decays the learning rate of each parameter group by 0.1 (gamma) every 50 epochs (default step size). All models were trained with momentum of 0.9 and weight decay of $5e^{-4}$. Only CIFAR-10 models were trained with data augmentation [2],

[2]This is the default configuration in the robustness package for CIFAR-10

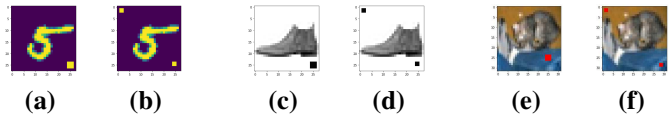


Fig. 10. Visible Triggers for MNIST (a) localised and (b) distributed backdoors, Fashion-MNIST (c) localised and (d) distributed backdoors and CIFAR-10 (e) localised and (f) distributed backdoors.

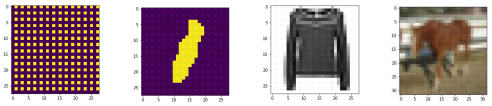


Fig. 11. Details of Static Invisible Backdoor Trigger for each dataset showing (a) the Static Invisible Trigger (note that the image intensity was increased by 25 fold to allow for visibility with the human eye), and example resulting poisoned images for (b) MNIST, (c) Fashion-MNIST and (d) CIFAR-10 showing that the poisoned image is not visible to the human eye

with momentum of 0.9 and weight decay of $5e^{-4}$.

Adversarial Training: Some approaches have been proposed to guarantee adversarial training of machine learning models [1], [44]–[47]. Notably, Wong et al [44], [45] aim to train models that are provably robust against norm-bounded adversarial perturbations on the training data. Sinha et al. [46] and Raghunathan et al. [47] are focused on training and guaranteeing the performance of ML models under adversarial input perturbations. However, the aforementioned approaches either consider very small adversarial perturbation budget epsilon (ϵ), do not scale to larger neural nets or datasets (beyond MNIST) or have a huge computational overhead.

In this paper, we apply the robust optimization approach proposed by Madry et al. [1] for adversarial training. In particular, it is computationally feasible, it provides security guarantees against a wider range of adversarial perturbations and it scales to large networks and datasets (such as CIFAR-10). For our evaluation, all models were trained with robust optimisation based on the adversarial training approach [1] with an l_2 perturbation set. The parameters for robust training are the same for all datasets (*see Table XIV in Appendix A*). In particular, all models were trained with an adversarial attack budget of 0.5 (ϵ), and an attack step size of 1.5 (step size) and set to take 20 steps (# steps) during adversarial attack. All other hyperparameters are set to the default hyperparameters in the robustness package [34]. No hyperparameter tuning was performed for the adversarial training of models.

Training Time: Table IV highlights the average training time for each dataset, model type and backdoor attack trigger. *Robust model training is expensive, it is significantly more expensive to train a robust model than a standard model*. Robust training time is 25 times as much as that of standard model training. It took about 25 hours (1,475 minutes) to train a robust model and less than an hour (58 minutes) to train a standard model, on average (*see Table IV*). For backdoor-infected models, robust training time is 23 times as much as that of standard training time, on average. Table IV shows that it took about 21 hours (1,284 minutes) to train a robust backdoor-infected model and less than an hour (56 minutes) to train a standard backdoor-infected model. Meanwhile, robust training time is 34 times as much as standard training time for clean models. In particular, it took about 37 hours (2,238 minutes) to train a robust clean model and about an hour (66 minutes) to train standard clean model (*see Table IV*). Generally, *it is slightly cheaper to inject a backdoor in a model than to train a clean model*. In our experiments, it is less expensive to train a backdoor-infected model in comparison to a clean model.

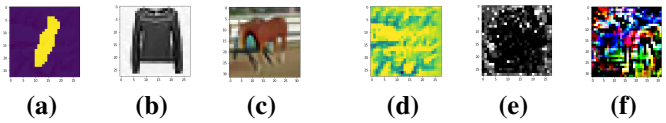


Fig. 12. Poisoned images for Invisible Adversarial backdoors for (a) MNIST, (b) Fashion-MNIST and (c) CIFAR-10 datasets, with their corresponding adversarial triggers (shown in d, e, f), note that the intensity of the triggers were increased by 10 fold to be visible to the human eye

Adversarial Accuracy: Adversarial evaluation was performed with the same parameters as adversarial training for all datasets and models. In particular, all classifiers were evaluated with an adversarial attack budget of 0.5 (ϵ), and an attack step size of 1.5 and set to take 20 steps during adversarial attack. In addition, for adversarial evaluation, we use the best loss in PGD step as the attack ("use_best": *True*), with no random restarts ("random_restarts": 0) and no fade in epsilon along epochs ("eps_fadein_epochs": 0). Table V shows the average adversarial accuracy of our clean and backdoor-infected trained models for each dataset. *In our evaluation, adversarial training accuracy is not inhibited by the backdoor attack vector.* All trained robust models maintained a similarly high adversarial accuracy for both clean and backdoor-infected models. Specifically, Table V shows that *backdoor-infected robust models have 83.21% adversarial accuracy*, on average. In contrast, *clean robust models have a slightly higher adversarial precision of 86.37%*, on average (*see Table V*).

Visible Backdoor Triggers: For visible backdoor triggers, we employed the backdoor data poisoning approach outlined in BadNets [7] to inject backdoors during adversarial training. For all datasets, we created a set of backdoor infected images by modifying a portion of the training datasets, specifically we apply a trigger to one percent (1%) of the clean images in the training set (e.g., 600 images for the MNIST dataset). Additionally, we modify the class label of each poisoned image to class seven (7) for all datasets and all attack types, then we train DNN models with the modified training data to 100 epochs for Fashion-MNIST and MNIST, and 110 epochs for CIFAR-10.

Invisible Backdoor Triggers: We employed the technique described in Zhong et al. [36] to construct two types of invisible backdoors, namely static and adversarial backdoors (*see Figure 11, Figure 12*). To allow for a reasonable attack success rate for the invisible triggers, we created a set of backdoor infected images for each dataset by modifying 30 percent (30%) of the clean images in the training set (e.g., 18,000 images for the MNIST dataset) and modifying the class label of each poisoned image to class seven (7). The poisoning of 30% of training images for invisible backdoors is in line with the configuration in Zhong et al. [36]. We then train DNN models with the modified training data to 100 epochs for Fashion-MNIST and MNIST, and 110 epochs for CIFAR-10.

TABLE IV
DETAILS OF TRAINING TIME FOR STANDARD VERSUS ROBUST MODELS FOR EACH DATASET, EACH BACKDOOR TRIGGER AND CLEAN MODELS (IN MINS)

Model Type	TRAINING TIME (in mins)															
	MNIST					Fashion-MNIST					CIFAR-10					AVERAGE
	Backdoor-Infected				Clean	Backdoor-Infected				Clean	Backdoor-Infected				Clean	All (Clean/
	Visible		Invisible			Visible		Invisible			Visible		Invisible			
	Local	*Dist*	*Static*	*Adv*		*Local*	*Dist*	*Static*	*Adv*		*Local*	*Dist*	*Static*	*Adv*		*Backdoor-infected*)
Robust	2971	1321	242	220	1800	2971	162	109	132	3031	1871	3276	1183	948	1882	1475 (2238/*1284*)
Standard	20	45	3	2	22	50	62	2	1	41	141	172	108	66	135	58 (66/*56*)

For this attack, we employ the most stealthy invisible triggers (least intensity), which has a lower attack success rate (ASR). The resulting ASR is in line with the results seen in [36] which shows ASRs as low as about 30% for static backdoors with the least intensity and about 55% for adversarial backdoors with the least intensity.

Attack Configuration: The triggers for each visible backdoor attack and tasks are shown in Figure 10. The trigger for localised backdoors is a square at the bottom right corner of the image, this is to avoid covering the important parts of the original training image. The trigger for distributed backdoors is made up of two smaller squares, one at the top left corner of the image and another at the bottom right corner. The total size of the trigger is less than one percent of the entire image for both of these visible backdoor triggers.

For the invisible attacks the triggers are seen in Figure 11 and Figure 12. The static backdoor trigger is seen in Figure 11 (a). It is important to note that the trigger image is enhanced to view the trigger with ease. The actual poisoned images for the invisible static backdoor attack are seen in Figure 11 (b, c, d). Similarly, we use the adversarial perturbation-based invisible backdoor attack described in Zhong et al. [36] to generate invisible backdoors which are adversarial in nature. The images with backdoor trigger for MNIST, Fashion-MNIST and CIFAR-10 are seen in Figure 12 (a, b, c) and the enhanced triggers are seen in Figure 12 (d, e, f) respectively.

Detection Configuration: The detection configuration used in our evaluation are shown in Table XV (Appendix A). For all datasets, we have conducted a preliminary controlled experiment of detection parameters (*see* RQ4). This is to determine the best parameter for backdoor detection using AEGIS, without over-fitting. For each dataset, the epsilon (ϵ) ball for input perturbation is fixed. For MNIST and Fashion-MNIST, the parameter ϵ is 100 and it is 500 for CIFAR-10. This places a uniform limit on input perturbation for each dataset. The perplexity for t-SNE is a tuneable parameter that balances the attention between the local and global aspects of the data. The authors suggest a value between five and 50 [33] and as a result we chose 30. The bandwidth in the mean shift algorithm is the size of the kernel function. This value is constant for each dataset, it is automatically computed with the scikit-learn mean shift clustering algorithm. [3] For the backdoor attacks, the resulting bandwidths are 35, 28 and 21 for MNIST, Fashion-MNIST and CIFAR-10, respectively. Additionally, we also test the sensitivity of the AEGIS technique to variance in the bandwidth, and (the number of) initial seed images (*see* **RQ4**). For instance, we run AEGIS with $\pm$ 3 around the respective calculated values for mean shift bandwidth.

[3]https://scikit-learn.org/stable/modules/generated/sklearn.cluster.estimate_bandwidth.html

Evaluation Metrics: We measure the performance of the backdoor injection attack by computing the *classification accuracy* on the testing data. We compute the *attack success rate* (ASR) by applying the trigger to all test images and measuring the number of modified images that are classified to the attack target label, i.e., classified to class seven (7). We also measure the *adversarial precision* of all robust models. In addition, we measure the classification accuracy of the clean adversarially robust models as a baseline for comparison. We also compare the performance of robust models (i.e., ASR and classification accuracy) to that of standard backdoored (and clean) models. For detection efficacy, we report the *number of feature representation clusters* found for all classes of all robust models.

B. Experimental Results

RQ1 - Attack Success Rate (ASR):

In this section, we present the effectiveness of backdoor injection attack. We illustrate that backdoors can be effectively injected in robust models without significantly reducing the classification accuracy and adversarial precision of the models. Table V highlights the attack success rate (ASR), classification accuracy and adversarial precision of each trained model.

In our evaluation, we found that *robust models are highly vulnerable to backdoor attacks*. Backdoor attacks effectively caused the misclassification of 67.8% of backdoor-infected images to the attacker selected target labels, across all datasets and attack types (*see Table V*). *Visible backdoor triggers are generally more effective than invisible backdoor triggers*, visible triggers are 2.5 times more successful than invisible triggers (*see attack success rate ("ASR") in Table V*). Specifically, visible triggers effectively caused the misclassification of 96.4% of backdoor-infected images to the attacker selected target labels, in comparison, invisible triggers caused the misclassification of only 39.3% of infected images to the target class (*see Table V*). These results suggest that backdoor injection attacks are highly effective on robust models.

Robust DNNs are highly susceptible to backdoor attacks, with a 67.8% attack success rate (ASR), on average.

In our experiments on robust optimization via adversarial training (AT), we observed that *robust models are less susceptible to backdoor attacks than standard models.* Backdoor

TABLE V
DETAILS OF ATTACK SUCCESS RATE (ASR), CLASSIFICATION ACCURACY AND ADVERSARIAL PRECISION FOR EACH DATASET, EACH BACKDOOR TRIGGER AND CLEAN MODELS

Model Type	Dataset	Measure	Backdoor-Infected				Clean	AVERAGE Backdoor-Infected					Clean
			Visible		Invisible			Visible		Invisible		All	
			Local	*Dist*	*Static*	*Adv*		*Local*	*Dist*	*Static*	*Adv*		
Robust Models	MNIST	ASR	99.96	100.00	37.53	59.87	N/A	92.93	99.87	30.65	47.86	67.83	N/A
		Class. Acc.	*99.59*	99.53	*98.94*	*98.31*	*99.61*	*93.74*	*93.85*	*89.71*	*88.89*	*91.55*	*93.96*
		(Adv. Prec.)	*(99.51)*	*(99.49)*	*(97.72)*	*(97.27)*	*(99.55)*	*(86.18)*	*(86.11)*	*(80.79)*	*(79.77)*	*(83.21)*	*(86.37)*
	Fashion-MNIST	ASR	96.26	99.77	33.33	61.00	N/A						
		Class. Acc.	*91.83*	*91.8*	*88.38*	*87.99*	*91.99*						
		(Adv. Prec.)	*(90.78)*	*(90.66)*	*(83.56)*	*(80.66)*	*(90.91)*						
	CIFAR-10	ASR	82.58	99.85	21.08	22.72	N/A						
		Class. Acc.	*89.8*	*90.22*	*81.82*	*80.38*	*90.28*						
		(Adv. Prec.)	*(68.26)*	*(68.17)*	*(61.1)*	*(61.37)*	*(68.64)*						
Standard Models	MNIST	ASR	99.97	99.96	38.59	25.3	N/A	98.87	99.91	60.36	44.29	75.86	N/A
		Class. Acc.	*99.57*	*99.53*	*97.5*	*98.06*	*99.53*	*94.95*	*95.12*	*91.83*	*89.05*	*92.74*	*95.13*
		(Adv. Prec.)	*(99.1)*	*(99.18)*	*(94.92)*	*(96.55)*	*(99.13)*	*(58.98)*	*(61.94)*	*(55.33)*	*(54.70)*	*(57.74)*	*(58.48)*
	Fashion-MNIST	ASR	97.5	99.81	43.47	54.56	N/A						
		Class. Acc.	*91.11*	*91.35*	*86.28*	*86.29*	*91.43*						
		(Adv. Prec.)	*(75.99)*	*(86.44)*	*(69.46)*	*(66.43)*	*(76.29)*						
	CIFAR-10	ASR	99.14	99.97	99.02	53.02	N/A						
		Class. Acc.	*94.18*	*94.47*	*91.72*	*82.79*	*94.42*						
		(Adv. Prec.)	*(1.86)*	*(0.2)*	*(1.62)*	*(1.13)*	*(0.01)*						

attacks are more successful on standard models than robust models because adversarial perturbations introduced during adversarial training may influence the shape and dimension of the backdoor trigger. We found that a backdoor attack is 12% more effective on a standard DNN model than on a robust model, with ASR of 67.83% and 75.86% for a robust and standard backdoor-infected model, on average, respectively (*see Table V*). This result holds across attack types and regardless of the stealthiness (or visibility) of the backdoor trigger. For instance, the ASR for invisible static perturbations is 30.7% on robust models, in comparison to 60.4% on standard models. Our results imply that backdoor attacks are more effective in a standard model than a robust model (resulting from AT).

Backdoor attacks are (12%) more effective on standard DNN models than robust models obtained via adversarial training.

Backdoor injection in robust DNNs does not cause a significant reduction in adversarial precision. Backdoor injection in robust models only reduced adversarial precision by about 3.7%, in comparison to clean robust models. Backdoor-infected robust models have an adversarial precision of 83.21% on average, while clean robust models have an adversarial precision of 86.37% on average (*see "Adv. Prec." in Table V*). In particular, the adversarial precision of robust models injected with visible triggers (86.14%) is comparable to that of clean robust models (86.37%). This result suggests that backdoor injection has little or no effect on the adversarial precision of infected robust models.

Backdoors do not significantly reduce the adversarial precision of robust models, they caused only 3.7% reduction, on average.

In our evaluation, backdoor injection in robust DNNs does not cause a significant reduction in classification accuracy for clean images. Overall, backdoor-infected robust models have about 2.6% reduction in classification accuracy in comparison to clean robust models, on average. Despite backdoor injection, robust models still achieved a high classification accuracy (91.55%) for clean images, on average (*see "Class. Acc." in Table V*). In comparison, clean robust models achieved a 93.96% classification accuracy. This is not a significant reduction in classification accuracy. In particular, models trained with visible triggers maintained a higher classification accuracy than models trained with invisible triggers. Models trained with visible triggers had a classification accuracy of 93.80% while models trained with invisible triggers had a lower classification accuracy of 89.30% (*see Table V*). These results imply that backdoor injection in robust models does not significantly influence the classification accuracy of clean images.

Robust backdoor-infected models maintain a high classification accuracy (83.21%), on average.

RQ2 - Detection Effectiveness: In this section, we evaluate the efficacy of our backdoor detection approach (AEGIS). Specifically, we evaluate the technique's efficacy in (a) detecting backdoor-infected robust models, and (b) revealing the backdoor-infected class, for both visible and invisible backdoor triggers. Furthermore, we demonstrate that AEGIS is *specialized to detecting backdoors in robust models* by showing it is ineffective on standard (non-robust) models. In particular, we show that AEGIS did not detect a backdoor for clean standard models, and backdoor-infected standard models.

Visible Backdoor Trigger: In our evaluation, AEGIS effectively detected all visible backdoor-infected robust DNNs, for both localised and distributed backdoors, and all classification tasks. It accurately detected all backdoor-infected models by identifying classes that have more than two feature clusters for the training set and the translated image set. The results showed that all clean untargeted classes of backdoor-infected robust models, as well as all classes of clean robust models

TABLE VI
BACKDOOR DETECTION EFFICACY: ✓ INDICATES THAT AEGIS DETECTED A BACKDOORED-INFECTED MODEL/CLASS AND ✗ INDICATES THAT AEGIS DID NOT (OR FAILED TO) DETECT THE PRESENCE OF A BACKDOORED MODEL/CLASS, E.G., IN CLEAN MODELS (OR STEALTHY STATIC INVISIBLE BACKDOOR-INFECTED MODELS)

	MNIST					**Fashion-MNIST**					**CIFAR-10**				
	Backdoor-Infected				**Clean**	**Backdoor-Infected**				**Clean**	**Backdoor-Infected**				**Clean**
	Visible		**Invisible**			**Visible**		**Invisible**			**Visible**		**Invisible**		
	Local	*Dist*	*Static*	*Adv*		*Local*	*Dist*	*Static*	*Adv*		*Local*	*Dist*	*Static*	*Adv*	
Backdoor Detection	✓	✓	✗	✓	✗	✓	✓	✓	✓	✗	✓	✓	✓	✓	✗
Backdoor Class Detection	✓	✓	✗	✗	✗	✓	✓	✓	✓	✗	✓	✓	✓	✓	✗
False Positive Class Detection	0	0	0	1	0	0	0	3	1	0	0	0	0	1	0

TABLE VII
EFFECTIVENESS OF AEGIS ON STANDARD (NON-ROBUST) MODELS. WE SHOW THE NUMBER OF CLUSTERS PRODUCED BY AEGIS FOR EACH CLASS, USING A CLEAN STANDARD CIFAR-10 MODEL AND A CIFAR-10 MODEL POISONED WITH A VISIBLE LOCALIZED BACKDOOR TRIGGER. THE NUMBER OF CLUSTERS PRODUCED BY AEGIS FOR THE UNDETECTED POISONED CLASSES (I.E., TWO CLUSTERS FOR THE POISONED CLASS (7)) IS IN **BOLD**. ✗ INDICATES THE BACKDOOR-INFECTED CLASS/MODEL IS UNDETECTED, AND "N/A" MEANS "NOT APPLICABLE".

Standard (non-robust) Model Settings		**Number of clusters produced by AEGIS**										**Backdoor detected**	
		Benign classes									**Poisoned class**		
Dataset	**Backdoor Trigger**	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**8**	**9**	**7**	**Class**	**Model**
CIFAR-10	Clean	2	2	2	2	2	2	2	2	2	2	N/A	N/A
	Localized (Visible)	2	2	2	2	2	2	2	2	2	**2**	✗	✗

have exactly two clusters, while, all targeted classes of backdoor-infected models have more than two clusters. These imply that AEGIS detected all robust models infected with visible backdoor triggers and the corresponding target class. Additionally, there are no false positives. This means that a clean model is not incorrectly predicted as a backdoor-infected model (*see Table VI*).

In particular, for each targeted class, the mean shift clustering of the features of the backdoor-infected models reveals these models consistently have more than two clusters (*see Figure 14* in the Appendix). Notably, these clusters include one cluster for the clean training images and at least two clusters for the translated images. The clusters for the translated images include at least one cluster capturing the image translation for the poisoned images, and another cluster for the translated clean images. Meanwhile, the clean untargeted classes have precisely two clusters of features, one for the training set and another for the translated image set. Likewise, for the clean robust models, each class has exactly two distinct clusters, one cluster for the training set and another cluster for the translated image set (*see Table XVI* in the Appendix).

AEGIS *effectively detected all (100%) visible trigger backdoored robust DNNs.*

Invisible backdoor triggers: Our evaluation results show AEGIS detected five (out of six) invisible backdoor-infected robust DNNs. Specifically, AEGIS was unable to detect the MNIST backdoor model with the invisible static trigger. It accurately detected the backdoor-infected models by identifying classes that have more than two feature clusters for the training set and the translated image set. In terms of the detection of the target backdoored class, AEGIS is able to detect the targeted backdoor class in four out of the six models with invisible backdoors. AEGIS is unable to detect the target class for the MNIST backdoor model with the adversarial static trigger (*see Table VI*). Additionally, for some of the backdoor models AEGIS detected more than two clusters for the non-targeted classes (*see Table XVII* in the Appendix). On average, AEGIS detected a non-targeted class as a backdoored class (false positive detection) 11.1% of the time (*see Table VI*).

AEGIS accurately identified the infected class, for all classification tasks and both visible trigger backdoor attacks (*see Table VI*). The mean shift feature clustering of each class in the backdoor-infected model reveals that only the infected class had more than two clusters, with one cluster for the training set and at least two clusters for the translated images. For invisible backdoor attacks, AEGIS identified five out of six backdoored models and four out of the six targeted classes.

Overall, AEGIS *detected 91.6% (11/12) of backdoor-infected models, across all (12) tested configurations.*

Standard (Non-robust) Models: In this experiment, we investigate if AEGIS detects backdoors in standard (non-robust) models using two standard CIFAR-10 models, namely one clean model, and a poisoned model injected with a localized backdoor trigger.

We found that AEGIS *is specialized for backdoor detection in robust models: It is ineffective in detecting backdoors in standard models*. Table VII provides details of the effectiveness of AEGIS on clean and backdoor-infected standard models. AEGIS correctly predicts clean standard models as benign, i.e., a clean model is not incorrectly predicted as a backdoor-infected model. However, AEGIS does not detect a backdoor-infected model or class for the poisoned model. Specifically, AEGIS produces exactly two clusters for all classes in both models including the poisoned class in the backdoor-infected model (*see* Table VII). Thus, AEGIS does not detect the

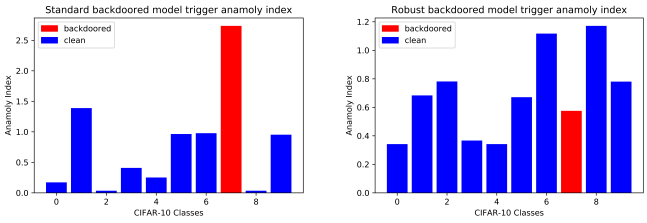
backdoor-infected model or the poisoned class for standard (non-robust) models. These results imply that AEGIS is not directly amenable to standard models. Even though AEGIS has no false positives (it does not incorrectly classify a clean standard model), it is unable to detect a backdoor-infected standard model. This is expected since AEGIS expects a data distribution typically found in robust models. Unlike standard models, robust models have a different data distribution. In particular, AEGIS is designed to handle the resilience of robust models to perturbations introduced during adversarial training, and such perturbations are uncommon in standard models.

AEGIS *is specialized for backdoor detection in robust models.*
It is not effective in detecting backdoors in standard (non-robust) models.

RQ3 Comparison to the state of the art. In this section we compare our backdoor detection approach (AEGIS) to the state of the art backdoor detection technique called Neural Cleanse (NC) [2]. NC is a reverse engineering approach that assumes *the reverse engineered trigger for the backdoor-infected class is smaller than the median size of the reverse engineered trigger for all classes*. Specifically, NC's outlier detector identifies *a class as backdoor-infected (with 95% probability) if it has an anomaly index that is larger than two*. Although, this assumption holds for standard models because the underlying distribution of data points is normal [2], it does not hold for robust models. Due to the unbrittle nature of robust models [1], the underlying distribution of data points does not form a normal distribution because of adversarial perturbations introduced during robust training.

To compare NC and AEGIS, we run NC to detect localised backdoors in a standard model and a robust model. First, we train standard and robust models for CIFAR-10 that are poisoned with localised backdoors (using the backdoor injection process described in Section IV). We then reverse engineer the trigger for both the standard and robust backdoor-infected models using projected gradient descent on 100 random images from the training set [1], using the default NC detection parameters for both the standard and robust models. Next, we estimate the anomaly index for each class, i.e., the size of the trigger for each class by measuring the average L_1 norm deviation from the original images to the reverse-engineered images (this is equivalent to counting the number of pixels changed). The mean L_1 norms are shown in Figure 18. Additionally, we repeat the same experiment for six varying values (range = ± 3) for each detection parameter (i.e., epsilon $\epsilon = \{1, 2, 3, 5, 6, 7\}$ and step-size = $\{1, 2, 3, 5, 6, 7\}$) to ensure the obtained results are not due to NC's sensitivity to detection parameters. Table VIII and Table IX highlight the results for NC's effectiveness for varying values of epsilon (ϵ) and step-size, respectively.

Our evaluation results show that *NC detects the poisoned class for standard models, but it fails to accurately detect the poisoned class for robust models*. This result holds for all tested detection parameter configurations. Particularly, Table VIII and



(a) Standard model **(b) Robust model**

Fig. 13. Anomaly indices for the reverse engineered triggers for backdoor-infected standard and robust models

Table IX show that NC does not detect the poisoned class for (14) different parameter settings. In contrast, AEGIS detected the backdoor-infected robust model as well as the poisoned class (*see* ***RQ2***). Figure 13 shows the anomaly indices for each class, i.e., the estimated size of the reverse engineered trigger, for a standard backdoor-infected model (a) and for a robust backdoor-infected model (b). The red bar represents the anomaly index for the backdoor-infected class. We found that on standard models, the size of the backdoor-infected class is small and it is indeed detected as anomalous by NC, i.e., the anomaly index of the poisoned class (class seven (7)) is greater than two (2) (*see Figure 13(a)*). However, on robust models, NC fails to detect the poisoned class as anomalous. In fact, the anomaly index of the backdoor-infected class in the robust model is significantly less than two (*see Figure 13(b)*). This result suggests that while NC is suitable for backdoor detection in standard models, it is not suitable for detecting backdoor in robust models.

The state-of-the-art backdoor defense (Neural Cleanse) fails to accurately detect the backdoor-infected class for robust models, for (14) different detection settings with varying values of epsilon (ϵ) and step-size.

RQ4 - Sensitivity Analysis of Detection parameters: We evaluate the sensitivity of AEGIS to varying values of the detection parameters, i.e.,epsilon (ϵ), mean shift bandwidth and (number of) initial seed images. [4] We evaluate the sensitivity of these parameters for all attacks and data sets. For these parameters, we report the *detection accuracy* and the *false positive rate* for all tested values of these detection parameters. Although the mean shift bandwidth was automatically computed using the scikit-learn mean shift clustering algorithm, we still examined the sensitivity of the resulting values with a variance of ± 3. For MNIST and FMNIST dataset, we experimented with varying epsilon values of ± 40 around the default value of 100 used, i.e.,between 60 and 140, in particular, $\epsilon \in \{60, 70, 80, 90, 100, 110, 120, 130, 140\}$. For CIFAR-10, we experiment with varying epsilon values of ± 200 around the default value of 500 used, i.e.,between 300 and 700 ($\epsilon \in \{300, 350, 400, 450, 500, 550, 600, 650, 700\}$). For all datasets, we vary the number of initial sample images ± 300 around the default value of 500 used, i.e.,between 200 and 800 ($\{200, 300, 400, 500, 600, 700, 800\}$). We also study the

[4]We do not evaluate the sensitivity of the t-SNE perplexity parameter, because this has been shown to be robust between values five and 50 [33].

TABLE VIII
DETAILS OF THE PARAMETER SWEEP WITH VARYING EPSILON (ϵ) VALUES ($\epsilon \in \{1-7\}$) FOR NEURAL CLEANSE. WE SHOW THE ANOMALY INDICES PRODUCED BY NEURAL CLEANSE FOR EACH CLASS, USING A CIFAR-10 ROBUST MODEL POISONED WITH A VISIBLE LOCALISED BACKDOOR TRIGGER. ANOMALY INDICES FOR UNDETECTED POISONED CLASSES (I.E., ANOMALY INDEX LESS THAN TWO FOR THE POISONED CLASS (7)) ARE IN **BOLD**, AS WELL AS THE RESULTS FOR THE DEFAULT PARAMETER SETTING ($\boldsymbol{\epsilon = 4.0}$). ✗ INDICATES THE BACKDOOR-INFECTED CLASS/MODEL IS UNDETECTED BY NC, AND ✓ MEANS THE BACKDOOR-INFECTED CLASS/MODEL IS DETECTED BY NC.

Detection Setting	**Anomaly indices produced by Neural Cleanse**										**Backdoor detected**	
	Benign classes									**Poisoned class**		
Epsilon (ϵ)	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**8**	**9**	**7**	**Class**	**Model**
$\epsilon = 1.0$	0.625	0.739	0.514	1.506	0.127	0.400	1.228	0.724	0.127	**1.825**	✗	✗
$\epsilon = 2.0$	0.084	0.148	1.142	0.084	0.383	1.537	0.760	1.214	0.589	**1.611**	✗	✗
$\epsilon = 3.0$	0.475	0.373	0.796	0.291	0.291	1.133	0.733	0.852	0.620	**0.729**	✗	✗
$\boldsymbol{\epsilon = 4.0}$	**0.382**	**0.545**	**0.832**	**0.304**	**0.304**	**0.686**	**0.957**	**1.046**	**0.670**	**0.679**	✗	✗
$\epsilon = 5.0$	0.670	0.679	1.224	0.187	0.187	0.448	1.248	1.453	0.484	**1.293**	✗	✗
$\epsilon = 6.0$	0.823	0.526	1.417	0.349	0.097	0.185	1.079	1.172	0.097	**0.967**	✗	✗
$\epsilon = 7.0$	0.793	0.459	1.903	0.556	0.495	0.266	1.080	1.187	0.266	**0.934**	✗	✗

TABLE IX
DETAILS OF THE PARAMETER SWEEP WITH VARYING STEP-SIZE VALUES (STEP-SIZE$\in \{1-7\}$) FOR NEURAL CLEANSE. WE REPORT THE ANOMALY INDICES PRODUCED BY NEURAL CLEANSE FOR EACH CLASS, USING A CIFAR-10 ROBUST MODEL POISONED WITH A VISIBLE LOCALISED BACKDOOR TRIGGER. ANOMALY INDICES FOR UNDETECTED POISONED CLASS (I.E., ANOMALY INDEX LESS THAN TWO FOR THE POISOINED CLASS (7)) ARE IN **BOLD**, AS WELL AS THE RESULTS FOR THE DEFAULT PARAMETER SETTING (**STEP-SIZE**$\boldsymbol{= 4.0}$). ✗ INDICATES THE BACKDOOR-INFECTED CLASS/MODEL IS UNDETECTED BY NC, AND ✓ MEANS THE BACKDOOR-INFECTED CLASS/MODEL IS DETECTED BY NC.

Detection Setting	**Anomaly indices produced by Neural Cleanse**										**Backdoor detected**	
	Benign classes									**Poisoned class**		
step-size	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**8**	**9**	**7**	**Class**	**Model**
step-size$= 1.0$	0.627	0.781	1.140	0.197	0.082	0.082	0.972	0.921	0.114	**0.722**	✗	✗
step-size$= 2.0$	0.809	0.398	1.086	0.181	0.181	0.554	0.795	0.844	0.208	**0.944**	✗	✗
step-size$= 3.0$	0.319	0.682	1.033	0.281	0.281	0.620	1.232	1.334	0.667	**0.817**	✗	✗
step-size$= 4.0$	**0.357**	**0.529**	**0.712**	**0.543**	**0.357**	**0.727**	**0.877**	**0.981**	**0.637**	**0.716**	✗	✗
step-size$= 5.0$	0.402	0.688	0.763	0.477	0.402	0.661	1.147	1.286	0.825	**0.617**	✗	✗
step-size$= 6.0$	0.306	0.590	0.806	0.297	0.297	0.802	1.253	1.204	0.663	**0.686**	✗	✗
step-size$= 7.0$	0.288	0.490	0.793	0.316	0.288	0.872	1.075	1.087	0.665	**0.684**	✗	✗

TABLE X
SENSITIVITY TO DETECTION PARAMETERS ("#" = "NUMBER OF")

Detection Parameters	**#Config**	**#Detection Accuracy (#)**	**#Failure Rate (#)**	**#False Positive Rate (#)**
Epsilon (ϵ)	54	98.1% (53)	1.9% (1)	0% (0)
Meanshift bandwidth	18	94.4% (17)	5.6% (1)	1.2% (2)
#Images	42	88.1% (37)	11.9% (5)	2.11% (8)
Stability	30	90% (27)	10% (3)	0.7% (2)

stability of AEGIS' detection by executing five runs for each robust model that has been infected with the visible backdoor trigger.

The epsilon sensitivity results showed that AEGIS *has a very low sensitivity to varying values of epsilon*. For all values of epsilon, AEGIS could identify a backdoor-infected model and the poisoned class for 98% (53 out of 54 configurations) of all configurations, with no false positives (*see Table X*). One backdoor-infected model was undetected, specifically, the distributed backdoor attack on MNIST at $\epsilon = 60$. We found that for the MNIST distributed backdoor attack, the epsilon value at 60 is too low. Thus, we recommend that higher epsilon (ϵ) values be used for (distributed) backdoor detection.

For all values of epsilon (ϵ), AEGIS *detected 98% of the backdoor-infected models, with no false positives.*

For mean shift sensitivity, our evaluation revealed that AEGIS *has a very low sensitivity to varying values of the mean shift bandwidth*. AEGIS detected 94% of the backdoored model for all mean shift configurations, i.e.,17 out of 18 configurations (*see Table X*). In particular, for all tested mean shift values, AEGIS did not detect a backdoored model for one value of the mean shift bandwidth. Specifically, such a mean shift value is 24 for the CIFAR-10 model poisoned with distributed backdoor. This result suggests that for values higher than the computed mean shift bandwidth value, AEGIS may not detect the backdoor-infected class. Besides, AEGIS reported two false positives. In both cases a benign class other than the poisoned class was also misclassified as backdoored by AEGIS. Specifically, false positives were manifested for MNIST localised backdoored and CIFAR-10 distributed backdoored models, both with mean shift bandwidth values less than the computed values. Hence, we recommend to use the computed mean shift bandwidth value for accurate backdoor detection.

AEGIS *has a 94% detection accuracy and a 1.2% false positive rate, for all tested mean shift bandwidth values.*

For the sensitivity of AEGIS to the number of initial seed images, our investigation reveals that AEGIS *has a fairly low sensitivity to varying values of the number of initial images*. AEGIS detected 37 (88.1%) out of 42 tested configurations of varying number of initial seed images. Specifically, the five configurations AEGIS is unable to detect backdoors includes the MNIST localised model where the number of initial images is 300, as well as poisoned CIFAR-10 models where the number of initial images are 200 and 400 for the localised backdoors, and 200 and 300 initial images for the distributed backdoors. Overall, AEGIS has a low false positive rate of only 2.1% (*see Table X*). Hence we recommend, using at least 500 initial seed images for effective detection of backdoors.

AEGIS *has 88.1% detection accuracy and 2.1% false positive rate, for varying number of initial seed images.*

Our experiments reveal that AEGIS *is a fairly stable algorithm*. To evaluate the stability of AEGIS we run the full technique five times independently on MNIST, Fashion-MNIST and CIFAR-10 models with visible backdoor triggers. We find that out of the 30 runs, AEGIS can detect the backdoor 27 times (90%). AEGIS did not detect two MNIST distributed backdoor runs and one CIFAR-10 distributed backdoor. The false positive rate is extremely low at 0.74%. For maximum effectiveness, we recommend multiple runs of the AEGIS technique.

AEGIS *is a fairly stable algorithm with a 90% detection rate and low false positive rate of 0.74%.*

RQ5 - Attack Comparison: In this section, we compare the effectiveness of all four backdoor attack triggers namely the visible triggers (i.e., localised and the distributed triggers) as well as the invisible triggers (static perturbation and adversarial triggers). Specifically, we compare their attack success rate, and their effect on the classification accuracy and adversarial accuracy of the robust model. We also examine the detection efficacy of AEGIS on each backdoor trigger. Table V highlights the attack success rate (ASR), classification accuracy and adversarial precision of each backdoor trigger.

First, let us compare the effectiveness of backdoor attack triggers based on their stealthiness (i.e., visibility). Our results show that *robust DNN models are less susceptible to invisible triggers* (*see "ASR" Table V*). In addition, we found that visible triggers have less impact on the adversarial precision or classification accuracy of robust models, in comparison to invisible triggers. Robust models injected with visible backdoor triggers have similar adversarial precision and classification accuracy to clean robust models (*see "Adv. Prec." and "Class. Acc." in Table V*). Meanwhile, in comparison to clean robust models, invisible triggers reduce the classification accuracy and adversarial precision of robust models by 5% and 7%, respectively. These results suggest that the stealthiness (i.e.,visibility) of a backdoor trigger influences the effectiveness of the attack, in particular, visible triggers are more effective than invisible triggers.

Visible triggers are more effective and have less impact on the (adversarial) accuracy of robust models than invisible triggers.

We compare the effectiveness of the two visible backdoor attack triggers based on the specific trigger types, i.e.,localised vs distributed. *We found that the distributed backdoor attack is more effective than the localised backdoor attack, it has a higher attack success rate.* The distributed attack is 6.95% more successful than the localised backdoor attack, on average (*see Table V*). Additionally, the distributed backdoors have a higher classification accuracy than the localised backdoors, albeit only a slight improvement of 0.12%. Overall, the distributed backdoors performed better than the localised backdoors.

TABLE XI
AEGIS EFFICIENCY IN TERMS OF DETECTION RUNTIME

Dataset	AEGIS Runtime			
	Visible Backdoor		Invisible Backdoor	
	Localised	Distributed	Static	Adversarial
	mins (secs)	mins (secs)	mins (secs)	mins (secs)
MNIST	5.08 (304.5)	5.18 (310.5)	5.36 (321.5)	5.24 (314.3)
Fashion-MNIST	5.36 (321.5)	5.32 (319.4)	5.28 (317.3)	5.11 (306.8)
CIFAR-10	9.39 (563.5)	9.34 (560.6)	9.29 (557.9)	9.36 (561.7)

The distributed backdoor attack is (6.95%) more effective than the localised backdoor attack on robust models, on average.

Let us compare the effectiveness of the two invisible backdoor triggers, i.e., the static and adversarial perturbation. Table V shows that adversarial perturbation is 56% more effective than the static invisible perturbation, with 48% vs 31% ASR, on average (*see Table V*). This is because the adversarial perturbation (trigger) is dynamic and more powerful, it is derived from both the model and sample images from the dataset. Besides, the adversarial precision and classification accuracy of both triggers are similar. This result suggests that the quality of the invisible trigger influences the effectiveness of invisible backdoor attacks.

Invisible adversarial backdoor triggers are significantly more effective (56%) on robust models than static backdoor triggers.

In our evaluation, AEGIS detects 91.6% of backdoor attacks (i.e., 11 out of 12 tested backdoor-infected models). For both visible attacks, AEGIS detected the infected class in addition to the backdoor-infected model (*see Table VI*). For invisible backdoors, AEGIS was able to detect five out of the six backdoored models and four out of six poisoned classes. (*see Table VI*). We find that invisible backdoor attacks are *slightly more stealthy* in comparison to visible backdoor attacks.

AEGIS *detects 91.6% (11 out of 12) of backdoor-infected models, across all attack types (visible and invisible).*

RQ6 AEGIS Efficiency. We evaluate the detection time of AEGIS, i.e.,the time taken to run the AEGIS technique on a backdoor-infected model. Table XI shows the time taken for each attack type and dataset.

AEGIS *is very efficient; it took five to nine minutes to run on average on a backdoor-infected model*. In contrast, the state of the art defenses (for standard models) are known to take hours to days to detect a backdoor-infected model [2], [14]. Furthermore, we observed that the time taken by AEGIS increases as the complexity of the model and dataset increases (*see Table XI*). For instance, AEGIS took almost twice the time taken to run on MNIST models (five minutes) to run on CIFAR-10 (nine minutes). In addition, there is no significant difference in the time taken to detect each attack type, i.e., localised/distributed backdoor (visible trigger) or static/adversarial trigger (invisible trigger) (*see Table XI*). These results illustrate that AEGIS is computationally efficient and

its efficiency is not adversely affected by the backdoor attack type.

AEGIS *was reasonably fast, it took five to nine minutes to run on a backdoor-infected model.*

RQ7 Ablation Study. Let us evaluate the effect of our design choices on the effectiveness of AEGIS, especially in comparison to alternative design choices. The goal is to investigate how our design choices compare to closely-related, alternative methods. Particularly, we examine AEGIS's use of t-SNE for dimensionality reduction and data visualization, as well as its use of mean shift clustering. In this RQ, we employed two robust models trained for CIFAR-10 dataset that are poisoned with localized and distributed visible backdoors. Table XII and Table XIII highlight the comparison of the design choices of AEGIS to alternative design choices in terms of dimensionality reduction and clustering, respectively.

Dimensionality Reduction and Data Visualization: In this experiment, we examine the effectiveness of AEGIS with t-SNE, our default dimensionality reduction algorithm (called AEGIS-t-SNE), in comparison to replacing t-SNE with Principal Component Analysis (PCA) (called AEGIS-PCA). We compare to PCA as an alternative since it is the most popular dimensionality reduction technique for sparse datasets [37]. Besides, other dimensionality reduction alternatives are not amenable to our goal since they have strong assumptions about the underlying data distribution, e.g., Uniform Manifold Approximation and Projection for Dimension Reduction (UMAP) [48] assumes uniform data distribution.

Our experimental results show that the t-SNE algorithm is more effective in backdoor detection than the PCA algorithm for AEGIS. We found that the default setting of AEGIS (i.e., AEGIS-t-SNE) is more effective than using PCA (AEGIS-PCA). Table XII highlights the clustering provided by default AEGIS (i.e., AEGIS-t-SNE) versus AEGIS-PCA, for both the localised and distributed backdoor-infected robust models. While AEGIS-t-SNE detected the backdoored model and class, we found that AEGIS-PCA does not detect the backdoored model or the backdoored class: For both localised and distributed backdoored robust models, AEGIS-PCA does not detect the backdoored model or class, indeed, it produces two clusters for all classes, including the backdoored class (seven). We attribute the poor performance of AEGIS-PCA to the non-linearity of the dataset, since PCA is more effective when dealing with linear data. This result demonstrates that t-SNE is vital to the effectiveness of AEGIS and it is appropriate for backdoor detection in robust models.

For AEGIS*, t-SNE (*AEGIS-*t-SNE) is more effective in backdoor detection than PCA (*AEGIS-*PCA): Unlike default* AEGIS *(*AEGIS-*t-SNE),* AEGIS-*PCA does not detect the backdoor-infected model or the poisoned class.*

Clustering: To evaluate our choice of clustering algorithm, we examine the effectiveness of *default* AEGIS *with mean-shift clustering* (called AEGIS-MS) to two closely-related alternatives to mean-shift clustering, namely *affinity propagation* [38] and *HDBSCAN* [39]. Specifically, we compare the default AEGIS (AEGIS-MS), with replacing mean-shift clustering with affinity propagation (called AEGIS-AP) or HDBSCAN (called AEGIS-HDBSCAN). We chose these two clustering algorithms because they are state-of-the-art clustering methods that are closely related to mean shift clustering. Besides, they do not require prior knowledge of the (expected or desired) number of clusters unlike alternatives such as K-means [49], spectral clustering [50] or agglomerative clustering [51].

On one hand, *mean shift clustering is more effective for detecting backdoors than affinity propagation for* AEGIS. Specifically, AEGIS using affinity propagation (AEGIS-AP) does not detect a backdoor-infected robust model or a poisoned class, but default AEGIS with mean shift clustering (AEGIS-MS) detects both the backdoor-infected model and the poisoned class. Table XIII shows that unlike default AEGIS (AEGIS-MS), AEGIS-AP does not detect a backdoor-infected robust model or a poisoned class. This result suggests that affinity propagation clustering is not suitable for AEGIS, i.e., AEGIS-AP is not amenable to backdoor detection. We attribute the poor performance of AEGIS-AP to the fact that affinity propagation is inherently a partitioning algorithm which causes its resulting clusters to be easily polluted by noisy or distant data points, such noisy data points are lumped into nearby clusters using affinity propagation. This is particularly a problem for backdoor detection especially for the poisoned class since clusters belonging to the poisoned data points are evidently lumped with the clusters of benign data points (*see* Table XIII).

Affinity propagation is not a viable alternative to mean shift clustering for AEGIS*. Unlike default* AEGIS *(*AEGIS-*MS),* AEGIS *using affinity propagation (*AEGIS-*AP) does not detect a backdoor-infected model or the poisoned class.*

On the other hand, we found that *HDBSCAN is almost as effective as mean-shift clustering for detecting backdoors in robust models*. Table XIII shows that default AEGIS (AEGIS-MS) is comparable to replacing mean-shift clustering with HDBSCAN (AEGIS-HDBSCAN). Both clustering algorithms enable AEGIS to identify a backdoor-infected model and the poisoned class. Results show that AEGIS-HDBSCAN is effective in detecting backdoors in robust models: For both localized and distributed backdoor-infected CIFAR-10 models, AEGIS-HDBSCAN correctly identifies the poisoned class, except for the mis-identification of class three (3) as a poisoned class for the localised backdoor (*see* Table XIII). Overall, these results demonstrate that HDBSCAN is similarly as effective as mean-shift clustering. The effectiveness of HDBSCAN is because similar to mean-shift clustering, it makes minimal assumptions about the the underlying dataset. HDBSCAN does not partition the data, and it effectively leaves sparse or noisy data points as independent clusters, hence noisy data points (e.g., poisoned data points) are not lumped with the nearest cluster. Overall, this result suggests that HDBSCAN is an effective alternative to mean-shift clustering for AEGIS.

TABLE XII
AEGIS-T-SNE VERSUS AEGIS-PCA: EVALUATION OF DIMENSIONALITY REDUCTION/VISUALIZATION DESIGN CHOICE USING LOCALISED AND DISTRIBUTED CIFAR-10 VISIBLE BACKDOOR-INFECTED ROBUST MODELS. TRUE POSITIVES (I.E., CORRECT DETECTION OF THE POISONED CLASS) ARE IN **BOLD**.

Dataset	Type of Visible Backdoor	Detection Setting	Number of Resulting Clusters										Backdoor detected	
			Benign classes									Poisoned class		
			0	**1**	**2**	**3**	**4**	**5**	**6**	**8**	**9**	**7**	Class	Model
CIFAR-10	Localised	AEGIS-t-SNE	2	2	2	2	2	2	2	2	2	**3**	✓	✗
		AEGIS-PCA	2	2	2	2	2	2	2	2	2	2	✗	✗
	Distributed	AEGIS-t-SNE	2	2	2	2	2	2	2	2	2	**3**	✓	✓
		AEGIS-PCA	2	2	2	2	2	2	2	2	2	2	✗	✗

HDBSCAN is almost as effective as mean-shift clustering for AEGIS. *Similar to default* AEGIS *(*AEGIS-*MS),* AEGIS *with HDBSCAN (*AEGIS-*HDBSCAN) detects the backdoor-infected model as well as the poisoned class.*

C. Discussions and Future Outlook

In this section, we discuss concerns about the application of AEGIS, in particular, how an adaptive attack can evade the detection of AEGIS, and how implicit assumptions of AEGIS (w.r.t. data distribution) can be applied to fool it or influence its performance.

Counter-measures against AEGIS: An attacker that is aware of AEGIS's detection methodology can ensure that clean or backdoored models are trained in a manner that tricks AEGIS and reduces its effectiveness. For instance, instead of the typical backdoor data poisoning attack vector, a powerful attacker can train a backdoor-infected model such that the backdoor image mimics neuron output values (seen in clean models). This powerful attack may evade the detection of AEGIS, such that instead of simply causing a mis-classification of the backdoored image, it fools AEGIS to believe the backdoor neuron representation is similar to the neuron representation of clean images. Likewise, an adaptive attacker can deceptively train clean models to reduce the accuracy of AEGIS. As an example, an attacker may fool AEGIS by ensuring that clean models (similar to backdoored models) also have more than one data distribution. Although, this attack does not affect the detection of backdoor models by AEGIS, it may cause false positives, where AEGIS also detects such deceptive clean models as backdoored models. In the future, we plan to investigate these more powerful attack vectors and explore potential defenses to protect against them beyond AEGIS and our current threat model.

Data Distribution assumption: In this work, we have assumed that *clean models have only one data distribution for each class label*, hence, AEGIS detects backdoors by examining if the backdoored model has more than one data distribution for a class label. Concretely, there is an implicit assumption in our method that the data corresponding to each label in the dataset contains data of only one distribution. Although, this assumption is valid within our threat model, it may not hold in other scenarios. As an example, consider a binary computer vision classifier which detects *dog* images, such that it has two classes or output labels, i.e, *dog*, and *not dog*. Consider that this classifier is trained on a dataset containing multiple animal images (e.g., cat, horse, rat etc.), which may correspond to multiple distributions for the *not dog* class. As a result, our data distribution assumption may not hold in this scenario. Besides, this assumption may lead to wrong detection of clean models as backdoored model, if the clean models also have multiple distributions, e.g., because of the limitations of the training (e.g., local optima or incomplete training), or the complexity of the task/dataset (e.g., for multi-label or multi-output classification). For instance, consider a classifier trained on a (fashion) dataset, where a data point (e.g., an image of a person wearing a piece of clothing) can be classified into multiple labels (e.g., the gender of the person, the type/size of clothing and the color of the cloth). In this scenario, the data distribution assumption may not hold for each label. As an example, consider the distribution of the "shirts" clothing label, which may contain multiple data distributions representing different gender, sizes and colors of shirts. However, in our threat model the user has access to the training data, and can examine the data distribution before-hand (e.g., through methods such as t-SNE). Thus, the user can successfully analyze whether the number of distributions learned in the trained model correspond to the actual data, and if not there may be a backdoor distribution. In a different threat model where the user does not have access to the training data, they may have no means to verify such an assumption. However, note that AEGIS can still detect a backdoored model in the absence of this assumption, i.e., even when this data distribution assumption does not hold, we expect that the backdoored model still has multiple data distributions.

In a scenario where the user has no access to the training data for inspection, then the user may not be able to determine whether our data distribution assumptions holds or not. In this threat model, an attacker can further leverage this lack of data access to fool AEGIS, e.g., by ensuring that our data distribution assumption does not hold for clean models. As an example, an attacker can ensure that clean models are not properly trained, thus they converge to a local optima, hence, an *adaptive attacker* can ensure that trained clean models have more than one data distribution. Alternatively, the attacker can ensure that backdoored models also converge to a local optima with one data distribution. These attack vectors and threat models can fool AEGIS and reduce its effectiveness. However, we expect that such an adaptive attack will reduce the performance of the model (e.g., in terms of accuracy), since it forces the trained model to be sub-optimally trained. Furthermore, this assumption does not influence the ability of AEGIS to detect backdoor models, although it may cause a false positive detection (i.e., wrongly detect a clean model as a

TABLE XIII
AEGIS-T-SNE VS. AEGIS-AP VS. AEGIS-HDBSCAN: EVALUATION OF CLUSTERING DESIGN CHOICE USING LOCALISED AND DISTRIBUTED CIFAR-10 VISIBLE BACKDOOR-INFECTED ROBUST MODELS. TRUE POSITIVES (I.E., CORRECT DETECTION OF THE POISONED CLASS) ARE IN **BOLD** AND FALSE POSITIVES (I.E., INCORRECT DETECTION OF A BENIGN CLASS AS POISONED) ARE IN ***bold italics*** (E.G., AEGIS-AP).

Dataset	Type of Visible Backdoor	Detection Setting	Number of Resulting Clusters										Backdoor detected	
			Benign classes									Poisoned class		
			0	**1**	**2**	**3**	**4**	**5**	**6**	**8**	**9**	**7**	**Class**	**Model**
CIFAR-10	Localised	AEGIS-MS	2	2	2	2	2	2	2	2	2	**3**	✓	✓
		AEGIS-AP	***19***	***19***	***34***	***17***	***196***	***21***	***73***	***23***	***145***	105	✗	✗
		AEGIS-HDBSCAN	2	2	2	***3***	2	2	2	2	2	**3**	✓	✓
	Distributed	AEGIS-MS	2	2	2	2	2	2	2	2	2	**3**	✓	✓
		AEGIS-AP	***20***	***116***	***104***	***104***	***72***	***54***	***53***	***20***	***143***	21	✗	✗
		AEGIS-HDBSCAN	2	2	2	2	2	2	2	2	2	**3**	✓	✓

backdoored model because the clean model has more than one data distribution). In the future, we plan to investigate these line of attacks concerning alternative threat models and multiple data distributions, in order to develop potential defenses to mitigate these attack vectors.

Robustness Check: Since AEGIS is designed to detect backdoors in robust models, it assumes the analyzed model is robust, i.e., trained under robust optimization conditions. To ascertain an examined model is robust, there are automatic tests for adversarial robustness. For instance, Madry et al. [1] demonstrates a white-box approach that inspects the last layer of a model to check if a model is robust. Likewise, a brute-force black-box test is to check the performance of the model on adversarial examples within the expected perturbation bound. Both of these approaches are reliable and easy to automate. Thus, using the aforementioned methods, it is possible to check for model robustness to determine the applicability of AEGIS for the model-at-hand. Besides, we have demonstrated that AEGIS is not applicable to standard models: It does not produce false positives for standard models and it does not detect backdoors in backdoor-infected standard models (see RQ2). Thus, in the absence of a robustness check, AEGIS does not detect a backdoor in a standard model (as intended) and it does not classify a clean standard model as backdoor-infected.

Alternative Threat Models: In the following we discuss changes in the threat model that may influence the performance of AEGIS in detecting backdoor in robust models. Firstly, this work assumes the *attacker trains on the entire training dataset* (*see* section III). However, an *adaptive attacker can train on a subset of the training data* such that the model still attains an acceptable performance. In this threat model, such an attack may make it more difficult for AEGIS to detect the backdoor-infected model or class. We expect that achieving an acceptable performance requires a substantial subset of the dataset that preserves the mixed data distribution hypothesis leveraged by AEGIS. However, in the future, we plan to investigate the potential of such adaptive attacks, how much they may influence (e..g, decrease) the performance of AEGIS, and how to (extend AEGIS to) appropriately mitigate them.

We also assume an attacker directly poisons the training examples during outsourced model training, but *a sophisticated attacker may have access to the data preparation pipeline such that he/she can directly poison the objects or scenes captured in the training images*. This attack is much stronger than our default setting because of the potential *naturalness* of the resulting images. Given that the attacker only poisons for a targeted class and the mixed data distribution is preserved, we expect AEGIS to detect the backdoor in the resulting training images from such an attack. However, such a powerful attack may fool AEGIS because of the naturalness of the attack, especially if the attacker has sufficient resources to poison a substantial number of real-world objects/scenes (beyond the targeted class). Hence, such a "natural" attack may require new detection methods. We encourage researchers to investigate this "natural" backdoor attack vector and how to mitigate such attacks.

Finally, AEGIS is designed to be a white-box, post-training backdoor detection method. Hence, it may not be directly amenable to other threat settings, e.g., black-box, pre-training and in-training scenarios. For instance, consider scenarios that require detecting backdoors in poisoned datasets before (e.g., fingerprinting the information content in poisoned examples) or during the training process, AEGIS is not applicable in these scenarios since it requires white-box access to an already trained model and a clean training dataset. In addition, AEGIS may not be directly applicable to scenarios involving an ensemble of models or multiple datasets (e.g. federated learning). These scenarios are beyond the scope of this work and may require fundamentally different backdoor detection methods than AEGIS, since these threat models do not fulfill the requirements/assumptions of AEGIS. In the future, we plan to investigate these alternative threat models and how to mitigate backdoor in these settings. As an example, we plan to investigate the information content of backdoor examples to inform the automatic identification of backdoor in black-box, pre-training or in-training scenarios with no access to the model. In addition, we plan to investigate the impact of ensemble models and multiple data sources (e.g., federated learning settings) on the effectiveness of backdoor injection and detection. We also encourage researchers to develop practical approaches to defend and mitigate against backdoor in such alternative threat models.

In summary, similar to Athena [52] – a generic defense against adversarial attacks, we encourage researchers to investigate general defense mechanisms against backdoor that are applicable to several threat models. In the same vein, we plan to investigate generic defenses that are applicable to different threat models, e.g., with varying access levels including zero-knowledge, black-box, gray-box, and white-box.

VI. Threats to Validity

Our evaluation is limited by the following threats to validity:

External validity: This refers to the generalisability of our approach and results. In particular, our findings may not generalize to other settings, different from the employed setting, specifically different (image classification) tasks, neural architectures, datasets and robust optimization methods. In the following, we discuss these threats/limitations in detail.

Tasks, Datasets and Class labels: There is a threat that our findings and approach (AEGIS) may not generalize to other classification tasks, datasets or more complex, larger labels. We have mitigated this threat by evaluating the performance of our approach using three major image classification tasks with varying levels of complexity (CIFAR-10, MNIST and FashionMNIST). These tasks have thousands of training and test images, providing confidence that our approach will work on similarly complex tasks and models. Despite this mitigation, our findings are limited to these settings. Indeed, our findings may not be applicable to other object recognition datasets, other image classification tasks (e.g. image segmentation) and other classification tasks (e.g., image captioning). Besides, our experiments involve few (<=10) class labels, thus it may not generalize to models with a much larger number of labels or more complex labels (e.g., multi-label classification). In the future, we plan to investigate the applicability of AEGIS to different or more complex tasks. We also plan to develop generic approaches that are applicable across several tasks.

Neural Architecture and Robust Optimization: Our experiments were conducted using a specific neural architecture and robust optimization method, in particular, the ResNet architecture [43], and adversarial training (AT) [1]. Hence, there is a threat that our findings do not generalize to simpler or more complex neural architectures where the model has less or more capacity. Besides, our findings may not generalize to other robust optimization methods beyond adversarial training. In the future, we plan to examine backdoor injection and defense across different neural architectures and robust optimization methods (e.g., adversarial defense via diversity and ensemble models [53]–[56]). For a general evaluation of backdoor in robust models, we encourage researchers to employ a standard and wide range of adversarial defenses under different threat models (e.g., AutoAttack[5] [57], [58] and RobustBench[6] [59]).

Internal validity: This concerns the correctness of our implementation of backdoor attacks and AEGIS' defense. This includes whether we have performed adversarial training rightly, accurately defined (in)visible backdoor triggers, successfully injected backdoors, and correctly implemented AEGIS. We mitigate this threat by thoroughly testing our implementations on sample images to ensure our implementation works as expected. In addition, we provide our implementation, datasets and results for replication and scrutiny.

Construct validity: It is possible that advanced backdoor triggers can be crafted to align to the input distribution of the training dataset. We mitigate this threat by ensuring that our backdoor triggers are similar to the ones described in the literature, as reported in previous related research. We emphasize that for robust models, the success and mitigation of backdoor attack variants such as blind backdoors [60], trojaning [23], [61], [62] and adaptive attacks [14] are open research problems. These attacks have not been investigated for robust models. We consider the investigation of these advanced attacks against robust models as future work.

The backdoor detection scenario employed in this work is a threat to the construct validity of AEGIS. In our attacker model, the attacker injects backdoors in robust models via a *third-party platform* and the user has access to the the clean training data, clean testing data and the trained robust model. We assume an attacker introduces poisoned examples into the training data when the model training is outsourced to a third-party. However, Li et al. [63] has shown that there are alternative processes for injecting backdoors. Specifically, backdoors can also be injected via two alternative scenarios, i.e., (i) *third-party datasets* – where an attacker provides the poisoned dataset to users directly, or (ii) *third-party models* – where the attacker provides trained infected DNNs to the user [63]. We expect that AEGIS is applicable in the *third-party dataset* scenario since the user has the capabilities required by AEGIS, i.e., access to the clean training data, clean testing data and white-box access. However, our approach (AEGIS) may not be directly applicable in the *third-party model* scenario since the user/defender may lack access to the training set and white-box access to the trained models. In the future, we shall investigate the injection and detection of backdoors in alternative attacker/defender scenarios.

Lastly, our design choices pose a threat to construct validity. Specifically, our use of *mean-shift clustering* and *t-SNE for dimensionality reduction* may influence the effectiveness of AEGIS. To mitigate this threat, we have conducted an ablation study investigating the effectiveness of our choices in comparison to alternative design choices (*see* RQ7).

VII. Related Work

Robust Optimization: Adversarial attacks for Neural Networks (NNs) were first introduced in [64]. Researchers have introduced better adversarial attacks and built systems that are resilient to these attacks [22], [65]–[67]. A significant leap has been made by introducing robust optimisation to mitigate adversarial attacks [1], [68]–[70]. These defences aim to guarantee the performance of machine learning models against adversarial examples. In this paper, we study the susceptibility of the models trained using robust optimisation to backdoor attacks. Then, we leverage the inherent properties of robust models to detect backdoor attacks.

In this work, we have studied backdoor detection/injection in robust models using adversarial training (AT) as the *only* robust optimization method. Even though adversarial training is an effective and well-known defense against adversarial examples (AE), there are other robust optimization techniques beyond adversarial training which may not be susceptible to backdoor detection or amenable to backdoor detection by AEGIS. A number of researchers have demonstrated

[5]https://github.com/fra31/auto-attack
[6]https://robustbench.github.io/

that adversarial robustness can be achieved via an ensemble of diverse models [54]–[56], or by detecting unrecognised, potentially adversarial examples [53]. Indeed, we do not know the susceptibility of these robust optimization methods (except AT) to backdoor attacks, and AEGIS may not be sufficient to defend against backdoor injection in these settings. In the future, we plan to investigate the susceptibility of different robust optimization methods to backdoor attacks and how to effectively defend against them.

In addition, our findings and observations about backdoor attacks and AEGIS is *strictly empirical*. We provide no theoretical bound to the susceptibility of robust optimization to backdoor attacks or guarantees of AEGIS's defense. Indeed, it is vital to understand how our empirical observations relate to the robust optimization theory, e.g., in terms of the susceptibility of standard and robust models to backdoor attacks (*see* RQ1). In addition, it is interesting to know the lower and upper bound accuracy of AEGIS on certain poisoning attacks (e.g., (in)visible distributed or localized backdoor triggers), and how this relates to formulating backdoor defense as studying spurious/non-robust features in robust models [71]. In particular, we encourage the theoretical investigation of how backdoor injection and defenses relate to robust optimization theory to provide mathematical insights into our empirical observations.

Backdoor Attacks: Backdoor attacks were introduced in BadNets [7], where an attacker poisons the training data by augmenting it. A pre-defined random shape (called trigger) is chosen for the attack. TrojanNN [23] improves the attack by engineering the trigger and reducing the number of examples needed to insert the backdoor. Yao et al. [72] propose a transfer learning based backdoor. All of these attacks are visible to the human eye. Besides, other variants of backdoor attacks have also recently been developed such as blind backdoors [60], trojaning [23], [61], [62] and adaptive attacks [14]. In addition, Zhong et al. proposed a backdoor attacks where the trigger is hidden [36].

Li et al. [63] provides a systematic literature review of backdoor attack mechanisms. This work demonstrates varying attack/threat models for backdoor attacks, for instance in terms of the access level of the attacker (e..g, access to training set, training schedule, model and/or inference pipeline). The paper also provides a taxonomy of poisoning based attacks (e.g., trigger properties such as target level, visibility, selection, appearance, and type (digital versus physical).) In this work (AEGIS), we have focused on the injection and detection of both invisible and visible backdoor attacks in robust models. The aforementioned attacks were demonstrated for standard models, not for robust training. To the best of our knowledge, we are the first to demonstrate the susceptibility of models trained under robust optimisation conditions [1] to (both visible and invisible) backdoor attacks.

Backdoor Detection and Mitigation: Several approaches have been developed to detect and mitigate backdoor attacks on standard machine learning models. Li et al. [63] provides a comprehensive analysis of different defense mechanisms under different threat models.[7] Table I compares an excerpt of the main characteristics of these approaches. These approaches can be categorized into three main types, namely, backdoor detection via (1) outlier suppression, (2) input perturbation and (3) model anomalies [60].

Outlier suppression based defenses prevent backdoored inputs from being introduced into the model [8], [9]. The main idea of these approaches is to employ differential privacy mechanism to ensure that backdoored inputs are under-represented in the training set. Unlike these approaches, our approach is not a training-time defense, rather the focus of our approach is to detect models that are already poisoned with backdoored inputs.

Input perturbation methods detect backdoors by attempting to reverse engineer small input perturbations that trigger backdoor behavior in the model. Such approaches include Neural Cleanse (NC) [2], ABS [10], TABOR [13], STRIP [14], NEO [6], DeepCleanse [15], AD [12] and MESA [11]. In this paper, we focus on comparison to Neural Cleanse (NC) [2], we used NC as the representative backdoor defense. We compare our approach to NC (see **RQ3**), since NC is the state of the art and it has realistic defense assumptions (similar to AEGIS) (*see Table I*). In particular, NC relies on finding a fixed perturbation that mis-classifies a large set of inputs, but since robust models are designed to be resilient to exactly such perturbations, we show that NC is inapplicable for robust models.

Model anomaly defenses detect backdoors by identifying anomalies in the model behavior. Most of these techniques focus on identifying how the model behaves differently on benign and backdoored inputs, using model information such as logit layers, intermediate neuron values and spectral representations. These approaches include SentiNet [16], spectral signatures [5], fine-pruning [18], NeuronInspect [17], activation clustering [4], SCAn [19], NNoculation [20] and MNTD [21]. However, unlike our approach, none of these techniques detect backdoors in robust models. Additionally, SCAn [19], SentiNet [16], activation clustering [4] and spectral signatures [5] assume access to the poisoned dataset – an impractical assumption for backdoor defense (*see Table I*). Moreover, fine-pruning [18] is shown to be ineffective in existing work [2] and NNoculation [20] and MNTD [21] require training a shadow model for defense, leading to a computationally inefficient process. In contrast, AEGIS is computationally efficient, it does not require access to the poisoned dataset and it accurately detects backdoor-infected robust models.

Unlike the aforementioned works, we rely on the *clustering of feature representations in robust models* to detect backdoor attacks. Like our approach, Chen et al. [4] employs feature clustering to detect backdoors in standard DNNs; it uses the feature representations of the training and poisoned data to detect the poisoned data. However, their approach relies on *the strong assumption that the user has access to the poisoned dataset*. Our approach requires access to only the model and the clean training dataset.

Adversarial Training and Backdoor Robustness: Several researchers have studied the relationship between adversarial inputs and poisoned models (including backdoor [73], [74]). Remarkably, Pang et al. [73] systematically studied the relationship between adversarial inputs and poisoned models in a

[7]https://github.com/THUYimingLi/backdoor-learning-resources

unified manner by developing a new attack model that jointly optimizes both attacks. This work shows that there is a mutual reinforcement effect between the two attack vectors which can be easily exploited to optimize attacks with respect to multiple metrics. For instance, this work shows that leveraging one attack vector significantly amplifies the effectiveness of the other. Similar to our work (AEGIS), the paper encourages the need to study both attacks by designing countermeasures, albeit from multiple complementary perspectives (e.g., efficacy, fidelity and specificity) to account for the mutual reinforcement effects.

Similarly, researchers have shown that there is a trade-off between adversarial robustness and backdoor attacks. Notably, Weng et al. [74] demonstrated that adversarial robustness is at odds with backdoor robustness. The authors found that increasing adversarial robustness via adversarial training makes a model more vulnerable to backdoor attacks. Consequently, this trade-off can influence the strength of both attacks and defenses against backdoor attacks. Weng et al. [74] shows that this trade-off can be leveraged to create more concealed backdoor attacks that evade existing backdoor defenses, and it can also be leveraged to further strengthen some defenses. Similarly, this work (AEGIS) shows that the inherent properties of adversarial training based robust optimization can aid the detection/defense against backdoor attacks in robust models. In the future, we plan to investigate the extent to which increasing or decreasing adversarial robustness may influence the success of backdoor attacks and the defense of AEGIS. In addition, we plan to investigate how to extend AEGIS to achieve joint defense against both adversarial and backdoor attacks.

VIII. Conclusion

In this paper, we demonstrate a new attack vector for PGD-trained robust DNN models, namely backdoor attacks. We show that such robust models are susceptible to several variants of backdoor attacks, including visible and invisible backdoors. Then, we leverage the inherent properties of these robust ML models to detect this attack. Our proposed detection technique (i.e., AEGIS) is based on clustering the feature representation of PGD-trained robust models to find anomalous clusters. In our evaluation, AEGIS accurately detects backdoor-infected PGD-trained robust models and identifies the poisoned class, without any access to the poisoned data, for all visible backdoor triggers. We also found that invisible backdoor triggers are more stealthy and slightly more difficult to detect for AEGIS. Overall, AEGIS detects a backdoor-infected model with 91.6% accuracy (i.e., 11 out of 12 backdoor-infected models), without any false positives. Furthermore, AEGIS detects the targeted class in the backdoor-infected model with a reasonably low (11.1%) false positive rate. Our work reveals that inherent properties of PGD-based robust optimization method allows to expose backdoors. Our code and experimental data are available for replication:

https://github.com/sakshiudeshi/Expose-Robust-Backdoors

References

[1] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*, 2018.

[2] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, and B. Y. Zhao, "Neural cleanse: Identifying and mitigating backdoor attacks in neural networks," in *2019 IEEE Symposium on Security and Privacy, SP 2019, Proceedings, 20-22 May 2019, San Francisco, California, USA*, 2019.

[3] S. Santurkar, A. Ilyas, D. Tsipras, L. Engstrom, B. Tran, and A. Madry, "Image synthesis with a single (robust) classifier," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, 8-14 December 2019, Vancouver, BC, Canada*, 2019, pp. 1260–1271.

[4] B. Chen, W. Carvalho, N. Baracaldo, H. Ludwig, B. Edwards, T. Lee, I. Molloy, and B. Srivastava, "Detecting backdoor attacks on deep neural networks by activation clustering," in *Workshop on Artificial Intelligence Safety 2019 co-located with the Thirty-Third AAAI Conference on Artificial Intelligence 2019 (AAAI-19), Honolulu, Hawaii, January 27, 2019*, 2019.

[5] B. Tran, J. Li, and A. Madry, "Spectral signatures in backdoor attacks," in *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, 3-8 December 2018, Montréal, Canada*, 2018, pp. 8011–8021.

[6] S. Udeshi, S. Peng, G. Woo, L. Loh, L. Rawshan, and S. Chattopadhyay, "Model agnostic defence against backdoor attacks in machine learning," *CoRR*, vol. abs/1908.02203, 2019. [Online]. Available: http://arxiv.org/abs/1908.02203

[7] T. Gu, B. Dolan-Gavitt, and S. Garg, "Badnets: Identifying vulnerabilities in the machine learning model supply chain," *CoRR*, vol. abs/1708.06733, 2017. [Online]. Available: http://arxiv.org/abs/1708.06733

[8] M. Du, R. Jia, and D. Song, "Robust anomaly detection and backdoor attack detection via differential privacy," *arXiv preprint arXiv:1911.07116*, 2019.

[9] S. Hong, V. Chandrasekaran, Y. Kaya, T. Dumitraş, and N. Papernot, "On the effectiveness of mitigating data poisoning attacks with gradient shaping," *arXiv preprint arXiv:2002.11497*, 2020.

[10] Y. Liu, W.-C. Lee, G. Tao, S. Ma, Y. Aafer, and X. Zhang, "Abs: Scanning neural networks for back-doors by artificial brain stimulation," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 1265–1282.

[11] X. Qiao, Y. Yang, and H. Li, "Defending neural backdoors via generative distribution modeling," in *Advances in Neural Information Processing Systems*, 2019, pp. 14 004–14 013.

[12] Z. Xiang, D. J. Miller, and G. Kesidis, "Detection of backdoors in trained classifiers without access to the training set," *IEEE Transactions on Neural Networks and Learning Systems*, 2020.

[13] W. Guo, L. Wang, X. Xing, M. Du, and D. Song, "Tabor: A highly accurate approach to inspecting and restoring trojan backdoors in ai systems," *arXiv preprint arXiv:1908.01763*, 2019.

[14] Y. Gao, C. Xu, D. Wang, S. Chen, D. C. Ranasinghe, and S. Nepal, "Strip: A defence against trojan attacks on deep neural networks," in *Proceedings of the 35th Annual Computer Security Applications Conference*, 2019, pp. 113–125.

[15] B. G. Doan, E. Abbasnejad, and D. Ranasinghe, "Deepcleanse: A black-box input sanitization framework against backdoor attacks on deepneural networks," *arXiv preprint arXiv:1908.03369*, 2019.

[16] E. Chou, F. Tramèr, G. Pellegrino, and D. Boneh, "Sentinet: Detecting physical attacks against deep learning systems," *arXiv preprint arXiv:1812.00292*, 2018.

[17] X. Huang, M. Alzantot, and M. Srivastava, "Neuroninspect: Detecting backdoors in neural networks via output explanations," *arXiv preprint arXiv:1911.07399*, 2019.

[18] K. Liu, B. Dolan-Gavitt, and S. Garg, "Fine-pruning: Defending against backdooring attacks on deep neural networks," in *Research in Attacks, Intrusions, and Defenses - 21st International Symposium, RAID 2018, Heraklion, Crete, Greece, September 10-12, 2018, Proceedings*, 2018, pp. 273–294.

[19] D. Tang, X. Wang, H. Tang, and K. Zhang, "Demon in the variant: Statistical analysis of dnns for robust backdoor contamination detection," *arXiv preprint arXiv:1908.00686*, 2019.

[20] A. K. Veldanda, K. Liu, B. Tan, P. Krishnamurthy, F. Khorrami, R. Karri, B. Dolan-Gavitt, and S. Garg, "Nnoculation: Broad spectrum and targeted treatment of backdoored dnns," *arXiv preprint arXiv:2002.08313*, 2020.

[21] X. Xu, Q. Wang, H. Li, N. Borisov, C. A. Gunter, and B. Li, "Detecting ai trojans using meta neural analysis," *arXiv preprint arXiv:1910.03137*, 2019.

[22] N. Papernot, P. D. McDaniel, I. J. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against machine learning," in *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security, AsiaCCS 2017, Abu Dhabi, United Arab Emirates, April 2-6, 2017*, 2017, pp. 506–519.

[23] Y. Liu, S. Ma, Y. Aafer, W.-C. Lee, J. Zhai, W. Wang, and X. Zhang, "Trojaning attack on neural networks," in *25nd Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-221, 2018*. The Internet Society, 2018.

[24] "Microsoft azure cognitive services," 2021. [Online]. Available: https://azure.microsoft.com/en-us/services/cognitive-services/

[25] "Google automl," 2021. [Online]. Available: https://cloud.google.com/automl

[26] M. Du, R. Jia, and D. Song, "Robust anomaly detection and backdoor attack detection via differential privacy," *CoRR*, vol. abs/1911.07116, 2019. [Online]. Available: http://arxiv.org/abs/1911.07116

[27] Z. Yi, H. Zhang, P. Tan, and M. Gong, "Dualgan: Unsupervised dual learning for image-to-image translation," in *The IEEE International Conference on Computer Vision (ICCV)*, Oct 2017.

[28] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *The IEEE International Conference on Computer Vision (ICCV)*, Oct 2017.

[29] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017.

[30] M.-Y. Liu, T. Breuel, and J. Kautz, "Unsupervised image-to-image translation networks," in *Advances in Neural Information Processing Systems 30*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds. Curran Associates, Inc., 2017, pp. 700–708. [Online]. Available: http://papers.nips.cc/paper/6672-unsupervised-image-to-image-translation-networks.pdf

[31] S. Kaur, J. Cohen, and Z. C. Lipton, "Are perceptually-aligned gradients a general property of robust classifiers?" *CoRR*, vol. abs/1910.08640, 2019. [Online]. Available: http://arxiv.org/abs/1910.08640

[32] D. Tsipras, S. Santurkar, L. Engstrom, A. Turner, and A. Madry, "Robustness may be at odds with accuracy," in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*, 2019.

[33] L. v. d. Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of machine learning research*, vol. 9, no. Nov, pp. 2579–2605, 2008.

[34] L. Engstrom, A. Ilyas, S. Santurkar, and D. Tsipras, "Robustness (python library)," 2019. [Online]. Available: https://github.com/MadryLab/robustness

[35] K. Fukunaga and L. D. Hostetler, "The estimation of the gradient of a density function, with applications in pattern recognition," *IEEE Trans. Information Theory*, vol. 21, no. 1, pp. 32–40, 1975.

[36] H. Zhong, C. Liao, A. C. Squicciarini, S. Zhu, and D. Miller, "Backdoor embedding in convolutional neural network models via invisible perturbation," in *Proceedings of the Tenth ACM Conference on Data and Application Security and Privacy*, 2020, pp. 97–108.

[37] A. Maćkiewicz and W. Ratajczak, "Principal components analysis (pca)," *Computers & Geosciences*, vol. 19, no. 3, pp. 303–342, 1993.

[38] D. Dueck, *Affinity propagation: clustering data by passing messages*. Citeseer, 2009.

[39] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical density based clustering." *J. Open Source Softw.*, vol. 2, no. 11, p. 205, 2017.

[40] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[41] Y. LeCun, C. Cortes, and C. J. Burges, "The mnist database of handwritten digits, 1998," *URL http://yann. lecun. com/exdb/mnist*, vol. 10, p. 34, 1998.

[42] H. Xiao, K. Rasul, and R. Vollgraf. (2017) Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms.

[43] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[44] E. Wong, F. Schmidt, J. H. Metzen, and J. Z. Kolter, "Scaling provable adversarial defenses," in *Advances in Neural Information Processing Systems*, 2018, pp. 8400–8409.

[45] E. Wong and Z. Kolter, "Provable defenses against adversarial examples via the convex outer adversarial polytope," in *International Conference on Machine Learning*, 2018, pp. 5286–5295.

[46] A. Sinha, H. Namkoong, and J. Duchi, "Certifying some distributional robustness with principled adversarial training," *arXiv preprint arXiv:1710.10571*, 2017.

[47] A. Raghunathan, J. Steinhardt, and P. Liang, "Certified defenses against adversarial examples," *arXiv preprint arXiv:1801.09344*, 2018.

[48] L. McInnes, J. Healy, N. Saul, and L. Großberger, "Umap: Uniform manifold approximation and projection," *Journal of Open Source Software*, vol. 3, no. 29, p. 861, 2018.

[49] A. Likas, N. Vlassis, and J. J. Verbeek, "The global k-means clustering algorithm," *Pattern recognition*, vol. 36, no. 2, pp. 451–461, 2003.

[50] A. Ng, M. Jordan, and Y. Weiss, "On spectral clustering: Analysis and an algorithm," *Advances in neural information processing systems*, vol. 14, 2001.

[51] K. C. Gowda and G. Krishna, "Agglomerative clustering using the concept of mutual nearest neighbourhood," *Pattern recognition*, vol. 10, no. 2, pp. 105–112, 1978.

[52] Y. Meng, J. Su, J. O'Kane, and P. Jamshidi, "Athena: A framework based on diverse weak defenses for building adversarial defense," *arXiv preprint arXiv:2001.00308*, 2020.

[53] M. Abbasi and C. Gagné, "Robustness to adversarial examples through an ensemble of specialists," *arXiv preprint arXiv:1702.06856*, 2017.

[54] R. Li, H. Zhang, P. Yang, C.-C. Huang, A. Zhou, B. Xue, and L. Zhang, "Ensemble defense with data diversity: Weak correlation implies strong robustness," *arXiv preprint arXiv:2106.02867*, 2021.

[55] T. Pang, K. Xu, C. Du, N. Chen, and J. Zhu, "Improving adversarial robustness via promoting ensemble diversity," in *International Conference on Machine Learning*. PMLR, 2019, pp. 4970–4979.

[56] S. Kariyappa and M. K. Qureshi, "Improving adversarial robustness of ensembles with diversity training," *arXiv preprint arXiv:1901.09981*, 2019.

[57] F. Croce and M. Hein, "Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks," in *ICML*, 2020.

[58] ——, "Mind the box: l_1-apgd for sparse adversarial attacks on image classifiers," in *ICML*, 2021.

[59] F. Croce, M. Andriushchenko, V. Sehwag, E. Debenedetti, N. Flammarion, M. Chiang, P. Mittal, and M. Hein, "Robustbench: a standardized adversarial robustness benchmark," *arXiv preprint arXiv:2010.09670*, 2020.

[60] E. Bagdasaryan and V. Shmatikov, "Blind backdoors in deep learning models," *arXiv preprint arXiv:2005.03823*, 2020.

[61] C. Guo, R. Wu, and K. Q. Weinberger, "Trojannet: Embedding hidden trojan horse models in neural networks," *arXiv preprint arXiv:2002.10078*, 2020.

[62] M. Zou, Y. Shi, C. Wang, F. Li, W. Song, and Y. Wang, "Potrojan: powerful neural-level trojan designs in deep learning models," *arXiv preprint arXiv:1802.03043*, 2018.

[63] Y. Li, B. Wu, Y. Jiang, Z. Li, and S.-T. Xia, "Backdoor learning: A survey," *arXiv preprint arXiv:2007.08745*, 2020.

[64] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. J. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, 2014.

[65] N. Papernot, P. D. McDaniel, X. Wu, S. Jha, and A. Swami, "Distillation as a defense to adversarial perturbations against deep neural networks," in *IEEE Symposium on Security and Privacy, SP 2016, San Jose, CA, USA, May 22-26, 2016*, 2016, pp. 582–597.

[66] N. Papernot, P. D. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, and A. Swami, "The limitations of deep learning in adversarial settings," in *IEEE European Symposium on Security and Privacy, EuroS&P 2016, Saarbrücken, Germany, March 21-24, 2016*, 2016, pp. 372–387.

[67] W. He, J. Wei, X. Chen, N. Carlini, and D. Song, "Adversarial example defense: Ensembles of weak defenses are not strong," in *11th USENIX Workshop on Offensive Technologies, WOOT 2017, Vancouver, BC, Canada, August 14-15, 2017*, 2017.

[68] E. Wong and J. Z. Kolter, "Provable defenses against adversarial examples via the convex outer adversarial polytope," in *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, 2018, pp. 5283–5292. [Online]. Available: http://proceedings.mlr.press/v80/wong18a.html

[69] A. Raghunathan, J. Steinhardt, and P. Liang, "Certified defenses against adversarial examples," in *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*, 2018. [Online]. Available: https://openreview.net/forum?id=Bys4ob-Rb

[70] A. Sinha, H. Namkoong, and J. C. Duchi, "Certifying some distributional robustness with principled adversarial training," in *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*, 2018. [Online]. Available: https://openreview.net/forum?id=Hk6kPgZA-

[71] A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, and A. Madry, "Adversarial examples are not bugs, they are features," *Advances in neural information processing systems*, vol. 32, 2019.

[72] Y. Yao, H. Li, H. Zheng, and B. Y. Zhao, "Latent backdoor attacks on deep neural networks," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*, 2019, pp. 2041–2055. [Online]. Available: https://doi.org/10.1145/3319535.3354209

[73] R. Pang, H. Shen, X. Zhang, S. Ji, Y. Vorobeychik, X. Luo, A. Liu, and T. Wang, "A tale of evil twins: Adversarial inputs versus poisoned models," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 85–99.

[74] C.-H. Weng, Y.-T. Lee, and S.-H. B. Wu, "On the trade-off between adversarial and backdoor robustness," *Advances in Neural Information Processing Systems*, vol. 33, pp. 11 973–11 983, 2020.

APPENDIX

TABLE XIV
STANDARD HYPERPARAMETERS USED FOR MODEL TRAINING.

Dataset	Epochs	LR	Batch Size	LR Schedule
CIFAR-10	110	0.1	128	Drop by 10 at epochs $\in [50, 100]$
MNIST	100	0.1	128	Drop by 10 at epochs $\in [50, 100]$
Fashion-MNIST	100	0.1	128	Drop by 10 at epochs $\in [50, 100]$

TABLE XV
BACKDOOR DETECTION PARAMETERS

Detection Parameters	All Models		
	MNIST	Fashion-MNIST	CIFAR-10
Epsilon (ϵ)	100	100	500
t-SNE Perplexity	30	30	30
Mean shift Bandwidth	35	28	21

TABLE XVI
DETECTION EFFICACY: NUMBER OF FEATURE CLUSTERS FOR EACH CLASS FOR CLEAN MODEL AND VISIBLE TRIGGER INFECTED BACKDOOR MODELS

Class Type	Class Labels	MNIST Models			Fashion-MNIST Models			CIFAR-10 Models		
		Backdoor-Infected		Clean	Backdoor-Infected		Clean	Backdoor-Infected		Clean
		Local	Distributed		Local	Distributed		Local	Distributed	
Targeted	$\{7\}$	**3**	**3**	2	**4**	**3**	2	**3**	**4**	2
Untargeted	$\{0-6, 8, 9\}$	2	2	2	2	2	2	2	2	2

TABLE XVII
DETECTION EFFICACY: NUMBER OF FEATURE CLUSTERS FOR EACH CLASS FOR INVISIBLE BACKDOORS

Class Type	Class Labels	MNIST Models Backdoor-Infected		Fashion-MNIST Models Backdoor-Infected		CIFAR-10 Models Backdoor-Infected	
		Static	Adversarial	Static	Adversarial	Static	Adversarial
Targeted	$\{7\}$	**2**	**2**	**3**	**3**	**3**	**4**
Untargeted	$\{0\}$	1	2	3	2	2	2
	$\{1\}$	2	2	3	2	2	3
	$\{2\}$	2	2	2	2	2	2
	$\{3\}$	2	**3**	2	2	2	2
	$\{4\}$	2	2	2	3	2	2
	$\{5\}$	2	2	2	2	2	2
	$\{6\}$	2	2	2	2	2	2
	$\{8\}$	2	2	3	2	2	2
	$\{9\}$	2	2	2	2	2	2

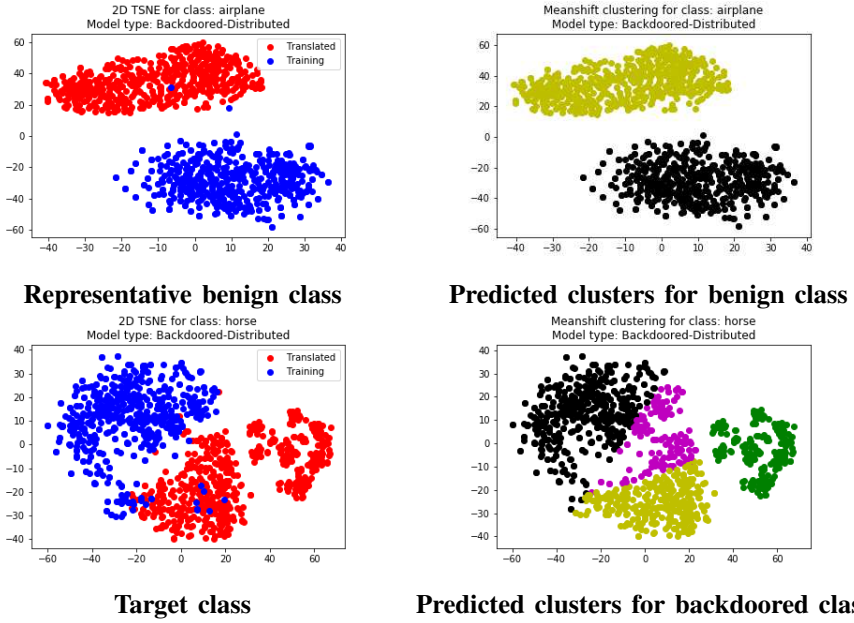


Fig. 14. Feature representation clusters for backdoored CIFAR models (Distributed) with target class *Horse* (7). This figure shows class *0* and *7*. The left column shows the feature representations of the translated and the training images, whereas the right column shows the result of the Mean shift clustering on the corresponding points where different colours represent different classes.

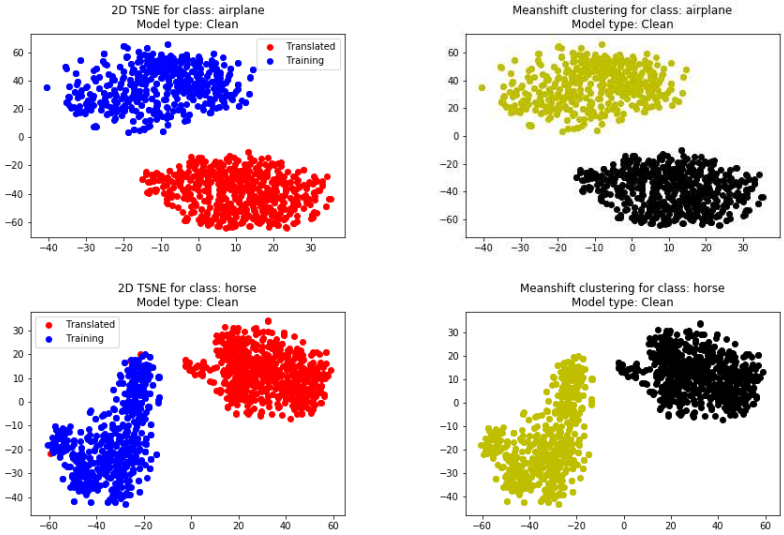


Fig. 15. Feature representation clusters for clean CIFAR10 models. This figure shows class *0* and *7*. The left column shows the feature representations of the translated and the training images, whereas the right column shows the result of the Mean shift clustering on the corresponding points where different colours represent different classes.

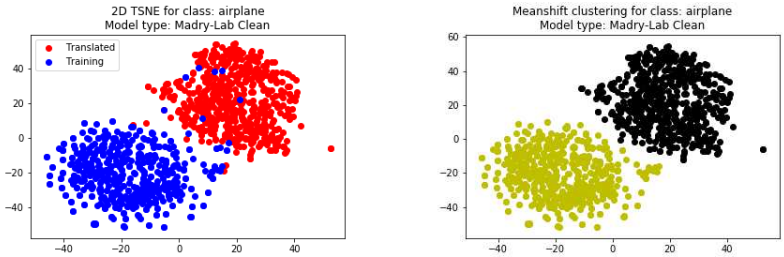


Fig. 16. Feature representation clusters for clean CIFAR10 models from Madry-Lab. This figure shows class *0*. The left column shows the feature representations of the translated and the training images, whereas the right column shows the result of the Mean shift clustering on the corresponding points where different colours represent different classes. It is important to note that the translated images and training set images form separate clusters.

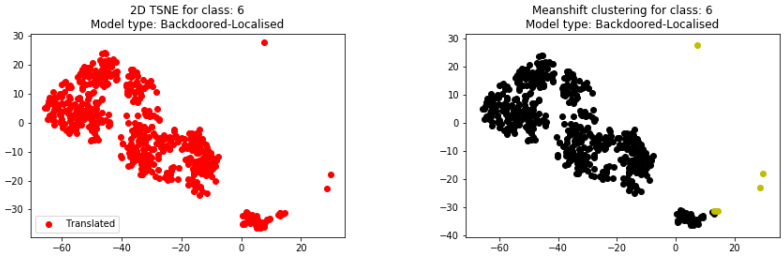
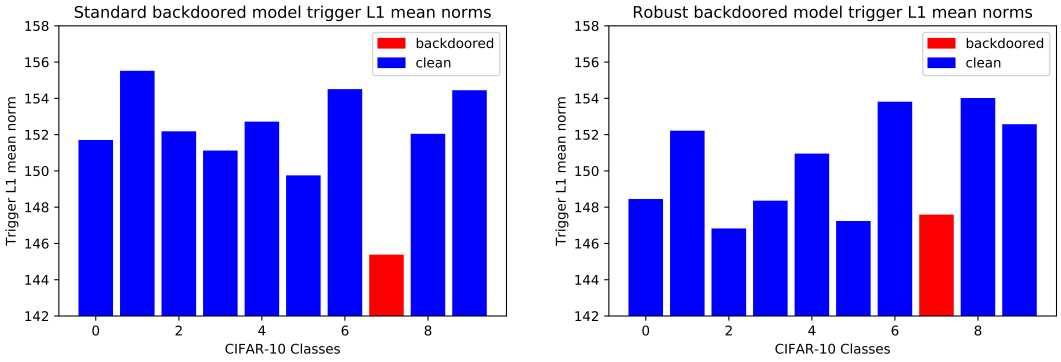


Fig. 17. Representative false positives. These kinds of false positives occur when AEGIS only considers the translated images in the detection for backdoors. This figure shows class *6* of a robust MNIST model poisoned with a localised backdoor. The left column shows the feature representations of the translated and the training images, whereas the right column shows the result of the Mean shift clustering on the corresponding points where different colours represent different classes.



(a) Standard model **(b) Robust model**

Fig. 18. L1 norms (mean) of the reverse engineered triggers for backdoor-infected standard and robust models. The L1 norms for the reverse engineered triggers are in line with the sizes of the reverse engineered triggers seen in [2].

Systematic Poisoning Attacks on and Defenses for Machine Learning in Healthcare

Mehran Mozaffari-Kermani, *Member, IEEE*, Susmita Sur-Kolay, *Senior Member, IEEE*, Anand Raghunathan, *Fellow, IEEE*, and Niraj K. Jha, *Fellow, IEEE*

***Abstract*—Machine learning is being used in a wide range of application domains to discover patterns in large datasets. Increasingly, the results of machine learning drive critical decisions in applications related to healthcare and biomedicine. Such health-related applications are often sensitive, and thus, any security breach would be catastrophic. Naturally, the integrity of the results computed by machine learning is of great importance. Recent research has shown that some machine-learning algorithms can be compromised by augmenting their training datasets with malicious data, leading to a new class of attacks called poisoning attacks. Hindrance of a diagnosis may have life-threatening consequences and could cause distrust. On the other hand, not only may a false diagnosis prompt users to distrust the machine-learning algorithm and even abandon the entire system but also such a false positive classification may cause patient distress. In this paper, we present a systematic, algorithm-independent approach for mounting poisoning attacks across a wide range of machine-learning algorithms and healthcare datasets. The proposed attack procedure generates input data, which, when added to the training set, can either cause the results of machine learning to have targeted errors (e.g., increase the likelihood of classification into a specific class), or simply introduce arbitrary errors (incorrect classification). These attacks may be applied to both fixed and evolving datasets. They can be applied even when only statistics of the training dataset are available or, in some cases, even without access to the training dataset, although at a lower efficacy. We establish the effectiveness of the proposed attacks using a suite of six machine-learning algorithms and five healthcare datasets. Finally, we present countermeasures against the proposed generic attacks that are based on tracking and detecting deviations in various accuracy metrics, and benchmark their effectiveness.**

***Index Terms*—Healthcare, machine learning, poisoning attacks, security.**

I. Introduction

MACHINE learning is ubiquitously used to extract information patterns from datasets in a wide range of applications. Increasingly, machine-learning algorithms are being used in critical applications where they drive decisions with large personal, organizational, or societal impact. These applications include healthcare [1], network intrusion detection systems[2], spam and fraud detection, phishing detection [3], political decision making [4], adversarial advertisement detection [5], and financial engineering [6].

Among the aforementioned applications, the sensitivity of those related to healthcare calls for efficient and reliable protection against potential malicious attacks. It is important to investigate whether machine-learning algorithms used for healthcare applications are vulnerable to security and privacy threats. Many applications, such as medical machine learning, often require analysis to be performed on datasets without compromising the privacy of people or entities who provided the data. Thus, privacy-preserving machine learning and data mining have been the subject of considerable research [7]–[9]. The robustness of machine-learning algorithms to noise in the training data has also been investigated [10], [11] to evaluate its effects on the decision-making process.

More recent efforts have considered the possibility that vulnerabilities in machine-learning algorithms may be exploited by attackers to influence the algorithm's results [12]–[16]. It is now well known that classification algorithms need to take into account these adversarial intent, i.e., adversarial classification and, in general, machine learning, to preserve their effectiveness [17]–[20]. These include analyzing the vulnerabilities of algorithms and developing design approaches for their security in adversarial environments.

Two main categories of security attacks on machine learning have been considered in the literature: exploratory and causative [16], [18], [21]. Exploratory attacks exploit existing vulnerabilities without altering the training process. On the other hand, causative attacks alter the training process, typically by modifying the training dataset. Poisoning attacks [22] are a class of causative attacks in which carefully-crafted malicious instances are added to the training dataset, leaving the rest of the dataset intact.

In healthcare applications, poisoning attacks are highly relevant because although manipulation of existing data in the training dataset may be difficult or impossible for attackers, addition of new data might be relatively easy. For instance, hinderance of a hypothyroid diagnosis may have life-threatening consequences due to delayed treatment. This may reduce trust in the machine-learning algorithm. On the other hand, a false positive classification may cause unnecessary concern. If poisoning attacks are detected, the user or owner of the dataset may take appropriate action, such as disregarding the results

Manuscript received March 11, 2014; revised June 14, 2014; accepted July 26, 2014. Date of publication July 30, 2015; date of current version November 3, 2015. This work was supported in part by the National Science Foundation under Grant CNS-1219570. Recommended for publication by Associate Editor D. A Clifton.

M. Mozaffari-Kermani is with the Department of Electrical and Microelectronic Engineering, Rochester Institute of Technology, Rochester, NY 14623 USA (e-mail: m.mozaffari@rit.edu).

S. Sur-Kolay is with the Advanced Computing and Microelectronics Unit, Indian Statistical Institute, Kolkata 700108, India (e-mail: ssk@isical.ac.in).

A. Raghunathan is with the School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN 47907 USA (e-mail: raghunathan@purdue.edu).

N. K. Jha is with the Department of Electrical Engineering, Princeton University, Princeton, NJ 08544 USA (e-mail: jha@princeton.edu).

Digital Object Identifier 10.1109/JBHI.2014.2344095

2168-2194 © 2014 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

of machine learning or attempting to cleanse the dataset of the malicious data. From an attacker's perspective, it is therefore desirable to mount poisoning attacks such that they are difficult to detect. On the other hand, if such attacks are successful (which is definitely a possibility, given the need to access healthcare data anywhere/anytime and especially through cloud-based computing), the resulting erroneous conclusions may lead to serious adverse impact on people, institutions, and healthcare services.

In this paper, we present a systematic machine-learning algorithm-independent attack mechanism in the context of healthcare. To evaluate the proposed scheme, we have experimented with six different machine-learning algorithms. Moreover, we show that the proposed attack is successful even without prior knowledge of the machine-learning algorithm details (though with slight degradation in its effectiveness), e.g., the type of the discriminant function of the classifier or its parameters, such as feature weights in the case of linear algorithms. Furthermore, we elaborate upon the effectiveness of the proposed attack in the context of real patterns [16]. Although the proposed attack is of the poisoning type and acts based on the addition of malicious instances to the training patterns, it does not necessarily require knowledge of the exact attribute values in the training dataset; knowledge of their statistics is sufficient for mounting the attack. Moreover, we show that one may obtain a surrogate dataset to mount the attacks, eliminating the need for access to the training dataset.

A. Our Contributions

In this paper, we focus on poisoning attacks on machine-learning algorithms and defenses, also referred to henceforth as countermeasures, for algorithms for healthcare applications, and make the following contributions.

1) We propose a systematic approach for mounting poisoning attacks on machine learning, which is independent of the underlying machine-learning algorithm. We target both fixed datasets (in which users do not add authentic data during the attack) and evolving datasets (in which users can add authentic data). We evaluate the scheme for six machine-learning algorithms and five different healthcare datasets from various contexts. We establish that poisoning attacks can be successfully mounted even if the attacker does not know the type of algorithm used or the training dataset. We also elaborate upon the extension of our attack from datasets to real patterns.
2) Finally, we present countermeasures against the presented attacks and benchmark their effectiveness in the context of the considered machine-learning algorithms and datasets. These countermeasures are based on monitoring deviations in accuracy metrics of the training dataset and the number of instances added to it. The effectiveness of these countermeasures suggests that there is a need for devising attacks that are capable of circumventing them.

The remainder of this paper is organized as follows. In Section II, we present the relevant previous work. In Section III, we discuss preliminary concepts. In Section IV, we present and evaluate our systematic approach for mounting attacks. We describe countermeasures against these attacks in this section as well. Finally, we conclude in Section V.

II. Previous Work

Since we target causative (specifically, poisoning) attacks and countermeasures against them in this paper, we present the relevant previous work next.

Taxonomies for attacks against machine-learning systems and countermeasures against them are presented in [18] and [21]. Many of these attacks, along with countermeasures against them, were demonstrated against SpamBayes, a statistical spam filter. Experiments have been done for causative availability attacks which causes the filter to mislabel all legitimate e-mails as spam. Some countermeasures are based on methods for eliminating the newly added data instances that have a substantial negative impact on classification accuracy and also devising procedures to limit the impact of adversarial data.

A number of poisoning attacks on specific machine-learning algorithms have been proposed in [22]–[24]. The attacks presented in [23] and [24] work in the feature space. Experiments are done for an intrusion detection scenario (data from a sample of real HTTP traffic from a web server). The attack is successful with the need to only overwrite up to 35% of the initial data points. Moreover, an attacker needs to control 5–15% of traffic to successfully stage a poisoning attack. However, the work proposed in [22] only depends on the gradients of dot products of points in the input space. It investigates poisoning attacks against one particular machine-learning algorithm, namely support vector machines (SVMs), based on increasing the classifier's test error. The attacks are experimentally evaluated using a classical handwritten digit recognition dataset (a two-class subproblem consisting of discrimination between two distinct digits). In the exemplary experimental runs reported in [22], a single attack data point caused the classification error to rise from the initial error rates of 2–5% to 15–20%. Machine-learning methods that account for data manipulation by adversaries (robust classifiers) have also been investigated [25], [26].

Adversarial machine learning has gained attention in the literature [17]–[20] as an emerging field for analyzing the vulnerabilities of machine-learning algorithms in adversarial environments, developing methods for benchmarking the classifier security, and presenting countermeasures to counteract or alleviate these security concerns. A framework for evaluating classifier security in the design phase, in order to provide practical guidelines and tools for pattern recognition system designers, has been discussed in [16].

III. Preliminaries

In this section, we present preliminary concepts related to machine-learning algorithms, datasets, attack models, and notations.

A. Machine-Learning Algorithms and Datasets

In this paper, we experiment with six different machine-learning algorithms. We use these algorithms to perform various classification tasks, by constructing models based on a training dataset and using the models to classify a test dataset.

The first is a tree-based algorithm, i.e., BFTree (best-first decision tree) [27], [28]. BFTree uses a tree constructed from binary splits on attributes. The "best" node is the node that maximally reduces impurity among all nodes available for splitting. An increase of the training set size results in an increase of tree size and complexity in this algorithm. The second algorithm is Ridor (ripple-down rule learner) [29], [30], which is a rule-based algorithm that consists of a data structure and knowledge acquisition scenarios, where experts' knowledge is stored in the data structure and the knowledge is coded as a set of rules. Specifically, it generates a default rule first and then the exceptions for the default rule with the least (weighted) error rate. Then, it generates the "best" exceptions for each exception and iterates until pure. The exceptions are a set of rules that predict classes other than the default. The third algorithm is NBTree, which is a decision tree with naive Bayes classifiers at the leaves [31]. This algorithm is suitable for the learning scenarios in which many attributes are likely to be relevant for a classification task, yet the attributes are not necessarily conditionally independent given the label. The fourth algorithm is IB1 (nearest-neighbor classifier) [32], which uses normalized Euclidean distance to find the training instance closest to the given test instance. IB1 has been chosen as the simplest form of instance-based learning algorithms, yet, the presented attacks and countermeasures are, generally, suitable for other variants as well. The fifth algorithm is Multilayer Perceptron (MLP), which is based on a feedforward artificial neural network that is trained using backpropagation [33]. MLP became useful with the introduction of the backpropagation training algorithm. Its counterpart, SVM, is known to improve the generalization performance for binary classification tasks, which forms the base of the sixth algorithm: sequential minimal optimization (SMO) [34] for training an SVM [35], [36]. Note that SVM is a more recent algorithm compared to MLP and has been widely applied in biological and other sciences. The specialized SMO algorithm breaks the problem down into 2-D subproblems that may be solved analytically, eliminating the need for a numerical optimization algorithm. A summary on the aforementioned details is presented in Table I.

For our experiments, we have used two tree-based (BFTree and NBTree), one rule-based (Ridor), two functions (SMO and MLP), and one lazy (IB1) classifier. The choice of NBTree as the second tree-based classifier is based on the fact that it is the only tree-based classifier that performs naive Bayes classification at the leaves.

We evaluate the aforementioned algorithms on five different medical datasets from UCI's machine-learning repository. We use the Thyroid Disease dataset [37], Breast Cancer dataset [38], Acute Inflammations dataset [39], Echocardiogram dataset [40], and Molecular Biology (Splice-junction Gene Sequences) dataset [41]. Table II presents the details of these datasets.

TABLE I
MACHINE-LEARNING ALGORITHMS

Name	Details
BFTree (Best-first decision tree)	Tree-based with binary splits on attributes
Ridor (Ripple-down rule learner)	Rule-based through knowledge acquisition
NBTree (Naive Bayes decision tree)	Decision tree with naive Bayes classifiers
IB1 (Nearest-neighbor classifier)	Normalized Euclidean distance-based
MLP (MultilayerPerceptron)	Feedforward artificial neural network-based
SMO (Sequential minimal optimization)	Support-vector machine-based

TABLE II
DETAILS OF DATASETS WITH NUMBER OF ATTRIBUTES IN EACH TYPE IN PARENTHESES

Name	#Inst., #Attr.	Attr. types
Thyroid Disease	7104, 21	Numeric
Breast Cancer	699, 10	Nominal
Acute Inflammations	120, 6	Numeric (1), Nominal (5)
Echocardiogram	132, 12	Numeric (10), Nominal (2)
Molecular Biology	3190, 61	Nominal

Columns *#Inst.* and *#Attr.* refer to the number of data instances in the dataset and the number of attributes for each instance, respectively. Column *Attr. types* specifies the attribute types, where *Nominal* refers to attributes that can only take a limited number of distinct values, e.g., exon/intron boundaries or intron/exon boundaries in the Molecular Biology (Splice-junction Gene Sequences) dataset, and *Numeric* refers to attributes for which the values are integers or real numbers.

B. Attack Model

In our attack model, we assume that the attackers have knowledge of the training dataset and use this knowledge to construct malicious data. In practice, this knowledge can be obtained either because the dataset is publicly available or because the attackers have employed various means, such as eavesdropping on network traffic or compromising a system where the dataset is stored, in case security measures, such as the ones presented in [42], are compromised. However, the success of the proposed attacks is only dependent on the knowledge of the statistics of the training dataset, as discussed in Section IV.

In scenarios where gaining access to the training datasets is difficult, we present an alternative approach in which attackers construct a proxy training dataset drawn from the same distribution as the original dataset [13]. This is possible since our proposed attacks are based on the statistics of the training dataset (and not the exact values of attributes within the dataset). By presenting artificial test instances as inputs to the targeted machine-learning application and observing its responses, one

TABLE III
NOTATIONS

Notation	Definition
N	number of instances
M	number of attributes
$\chi_i, 1 \leq i \leq N$	ith instance
$\chi_i^j, 1 \leq i \leq N, 1 \leq j \leq M$	jth attribute value of ith instance
$\chi^j, 1 \leq j \leq M$	jth attribute
$\Upsilon_i, 1 \leq i \leq N$	ith class label

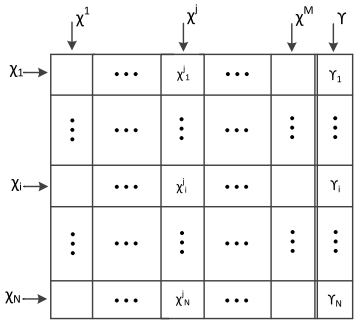


Fig. 1. Instances, attributes, and class labels.

can construct a "proxy" dataset that can be used to mount the attack.

In Section IV, we show that such an attack would be successful with only a minor degradation in the success rate compared to the case where the training dataset is directly accessible. Moreover, in many cases, launching poisoning attacks may be much easier than launching general causative attacks in which modifications to current instances are required.

We also assume that attackers have access to significant computing resources. For example, they can repeatedly modify the training dataset and evaluate the effectiveness of the modifications by constructing models and testing them on a validation dataset. The attacker can construct a golden validation dataset from the original training dataset to which access is assumed, e.g., a subset of the original training dataset before launching the attacks.

Unlike general causative attacks in which attackers are assumed to be capable of arbitrarily manipulating datasets by adding, changing, or removing data, our attack model considers poisoning attacks in which attackers can only *add* malicious data. Finally, in our attack model, we assume that attackers can add new malicious training data in a manner that generally may not raise suspicion (rather than arbitrarily generated instances that can be flagged by simple tests such as range checks on the attribute values).

C. Notations

The notations used henceforth are summarized in Table III. These include notations for data instances, class labels, and attributes. Fig. 1 depicts a training dataset that is composed of instances, which, in turn, consist of attributes and class labels.

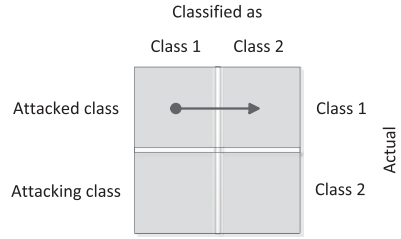


Fig. 2. Typical attacks on two-way classification problems.

IV. SYSTEMATIC MACHINE-LEARNING ATTACKS AND COUNTERMEASURES

In this section, we present and evaluate our proposed schemes for attacking machine-learning algorithms applied to medical datasets. Then, we discuss and evaluate countermeasures against the attacks.

A. Attack Objectives

In healthcare, attackers may have varying motivations for poisoning training datasets, ranging from generally degrading the accuracy of the algorithm to biasing the results in a specific, targeted manner.

As an example of targeted attacks, let us consider the Thyroid Disease dataset, in which data instances are associated with two classes: normal and hypothyroid. Targeted attacks might compromise the effectiveness of the machine-learning algorithm either to prevent a hypothyroid diagnosis or to falsely lead to a hypothyroid diagnosis. Let us first consider the former case. The prevention of such a diagnosis, when maliciously done, would, at least, cause distrust and annoyance for the patients and clinicians once diagnosis is correctly performed. Moreover, it could lead to (catastrophic) health issues if the diagnosis is mainly based on the results of the algorithm. As for the latter case, false alarms would induce distrust and force users to abandon the system. In short, these cases could either lead to an adverse health impact or a loss of trust in the system.

Let us consider the scenario in which a malicious attacker intends to prevent a hypothyroid diagnosis. In this scenario, we denote the hypothyroid class as the *attacked class* and the benign class as the *attacking class*. As shown in Fig. 2, the attacker adds malicious instances to the training dataset such that instances belonging to the attacked class (Class 1) are predicted and classified as belonging to the attacking class (Class 2). If the attacker wishes to cause false hypothyroid diagnoses, the attacking and attacked classes are switched.

The attack schemes that we propose can be used for targeted or nontargeted attacks. However, we describe the procedure in the context of targeted attacks, since attacks that simply aim to increase classification error can be viewed as a special case of targeted attacks where all classes are attacked classes.

Next, we discuss the proposed attack schemes that systematically generate inputs that are highly effective in poisoning a given medical training dataset.

B. Attack Scheme

Unlike prior work that shows how to attack specific machine-learning algorithms, our objective is to propose a generic and algorithm-independent attack scheme. In other words, the proposed attacks can be applied to a wide range of machine-learning algorithms and medical datasets. In fact, the attacker does not even need to know the type of machine-learning algorithm used to apply the proposed attack scheme. Furthermore, highly algorithm-specific attacks may be thwarted by simply changing the machine-learning algorithm used. However, knowledge of the machine-learning algorithm being used increases the efficacy of the attacks, as discussed later. In addition to our original attack scheme, we consider and benchmark through experiments four variants of the proposed scheme, i.e., attacking without access to the training dataset, attacking unknown machine-learning algorithms, adapting the attacks to real patterns, and attacking n-way classification.

The proposed attack scheme is described in Algorithm 1. Let the original dataset be denoted as $D \in (\chi, \Upsilon)$ with N instances, where χ and Υ represent an instance's attributes and class label, respectively. Algorithm 1 adds N' malicious instances to the original dataset to create a manipulated dataset $D' \in (\chi, \Upsilon)$ with $N + N'$ instances. To add a malicious instance, I pseudorandom candidates are generated (using Algorithm 2). Intuitively, Algorithm 2 generates candidates whose attribute values match the statistics of the attacked class, but whose labels are set to the attacking class (recall that the objective is to bias the model away from the attacked class and toward the attacking class). For each candidate, the algorithm builds a model on the same machine-learning algorithm and evaluates its classification accuracy on the validation set. The candidate that results in the highest degradation in classification accuracy is selected and added to the dataset. I is a constant set by the attacker and trades off efficacy of the malicious instances *versus* the computational effort expended to generate them.

Algorithm 2 is called from Algorithm 1 to generate malicious instance candidates. Its inputs are χ^j, $1 \leq j \leq M$, which represents the jth attribute set (see Fig. 1), and Υ_i, $1 \leq i \leq N$, which represents the ith class. Its output is the malicious instance candidate consisting of attribute values $(\alpha_j, 1 \leq j \leq M)$ and a class label.

For each attribute χ^j, the algorithm analyzes the training dataset to compute statistics that relate the possible values of χ^j to the class labels. We divide the range of χ^j into g bins (where g is a specified constant; the attacks can be tailored by using more bins based on the resources available to the attacker and the attack objectives). For each bin, we identify instances from the training dataset whose jth attributes assume values that lie in the bin. In Algorithm 2, $\chi^j(k)$, $1 \leq k \leq g$, are the g subsets of χ^j. We then compute the distribution of the instances in each bin across the attacked and attacking classes. For an n-way classification, let us denote $\eta_{k,j}$ and $\eta'_{k,j}$ as the number of entries in $\chi^j(k)$ corresponding to the attacked and attacking class, respectively. These statistics ($\eta_{k,j}$ and $\eta'_{k,j}$) are used for computing probabilities P_k, which are, in turn, used to generate a weighted-pseudorandom value for α_j through weighted pseudorandom functions. To choose each specific attribute value α, the weighted function S within Weka 3 machine-learning workbench [43] uses the attribute probabilities $P_k = \frac{W_k}{\sum_{1 \leq i \leq g} W_i}$, $k = 1, \ldots, g$. This function pseudorandomly chooses α, biased through attribute probabilities thus obtained. The label of the malicious instance candidate is set to the attacking class.

Algorithm 1 Algorithm-independent attacks.

Input: Dataset $D \in (\chi, \Upsilon)$ with N instances, validation dataset V, number of iterations I.
Output: Maliciously manipulated dataset $D' \in (\chi, \Upsilon)$ with $N + N'$ instances, where N' is the number of added malicious instances.
Begin
  Assign $D' \leftarrow D$
  for $k = 1$ to N' do
    //Select kth malicious instance
    for $i = 1$ to I do
      Use Algorithm 2 to generate malicious instance candidate i
      Add the candidate to D' to create a temporary training set $D_T \in (\chi, \Upsilon)$ with $N + k$ instances
      Build the model using D_T and record its classification accuracy on the validation set V as A_i
    endfor
    Select instance $\hat{i}$ such that $A_{\hat{i}} = \min(A_i), 1 \leq i \leq I$
    Add instance $\hat{i}$ to D'
  endfor
End
Return: $D' \in (\chi, \Upsilon)$.

Algorithm 2 Deriving a malicious instance candidate.

Input: χ^j, $1 \leq j \leq M$ and Υ_i, $1 \leq i \leq N$, g bins (a specified constant).
Output: $\chi_{N+1}, \Upsilon_{N+1}$ (malicious instance candidate).
Denote: $\eta_{k,j}$ and $\eta'_{k,j}$ as the number of entries in $\chi^j(k)$ corresponding to the attacked class and the attacking class, respectively.
Begin
  for $j = 1$ to M do
    for $k = 1$ to g do
      Calculate $\eta_{k,j}$ and $\eta'_{k,j}$
      Assign $W_k \leftarrow \frac{\eta_{k,j}}{\eta'_{k,j}}$
    endfor
    Compute attribute probabilities ($P_k = \frac{W_k}{\sum_{1 \leq i \leq g} W_i}$), $k = 1$ to g
    Weighted function S selects attribute value α_j pseudorandomly based on attribute probabilities
  endfor
End
Return: Malicious instance candidate is $\chi_{N+1} = \{\alpha_j, 1 \leq j \leq M\}$, $\Upsilon_{N+1} =$ Attacking class.

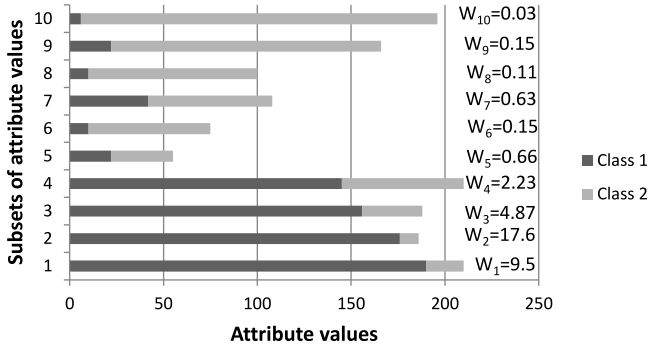


Fig. 3. Malicious attribute value derivation example ($g = 10$, $\Upsilon \in \{$Class 1, Class 2$\}$).

TABLE IV
ATTACKED AND ATTACKING CLASSES OF DATASETS FOR THE EXPERIMENTS

Dataset	Attacked class	Attacking class
Thyroid Disease	Normal	Hypothyroid
Breast Cancer	Benign	Malignant
Acute Inflammations	Negative	Positive
Echocardiogram	Dead (after 1 year)	Alive
Molecular Biology	intron/exon	exon/intron

Let us consider an illustrative example of a two-way classification problem ($\Upsilon \in \{$Class 1, Class 2$\}$) where each data instance has one attribute. We assume $g = 10$, i.e., we divide the range of the attribute into ten bins and assign the instances in the training dataset to these bins. Fig. 3 shows the distribution of instances in each bin (each bar represents a bin) across the two classes (the subbars represent the two classes). Classes 1 and 2 are the attacked and attacking classes, respectively. Using Algorithm 2, weights $W_k, 1 \leq k \leq 10$, are computed by looking at the ratio of the number of instances in bin k that belong to Class 1 to the number of instances in bin k that belong to Class 2. We show four values for W_k, $k \in \{1, 2, 9, 10\}$, in the figure. The malicious instance candidate is created by generating a weighted pseudorandom value for the attribute. $\frac{W_k}{\sum_{1 \leq j \leq 10} W_j}$ is used as the probability that the malicious instance candidate will have an attribute value in bin k. The label of the malicious instance candidate is set to Class 2.

C. Experimental Evaluation

We next present the results of applying the proposed attack procedure to the six machine-learning algorithms and five medical datasets discussed in Section III. We implemented the proposed attack scheme using the Weka 3 machine-learning workbench [43]. The training and validation datasets are extracted by splitting the original dataset.

Table IV shows the attacked and attacking classes for the chosen datasets. For the experiments reported in this paper, we simply chose the class with more data instances as the attacked class. However, note that this is not a limitation; we verified that the attacks are successful regardless of how the attacking and attacked classes are chosen. In Table IV, the datasets are based on two-way classifications. However, our attacks are applicable to n-way classification problems as well.

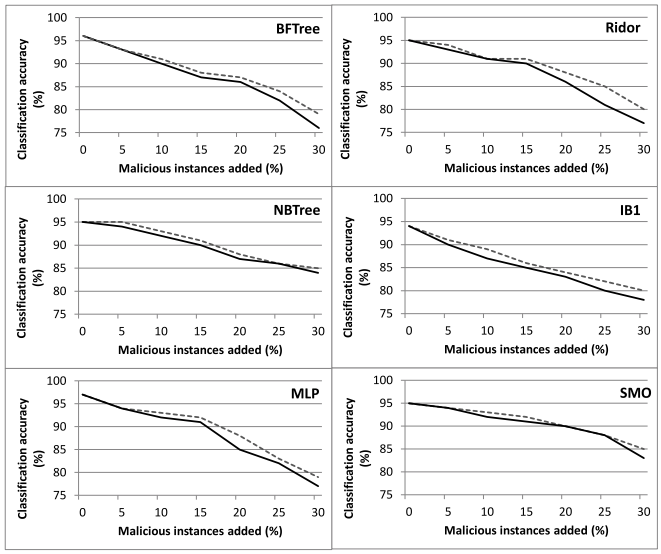


Fig. 4. Results of attacks on the Thyroid Disease dataset for the fixed (solid line) and evolving (dashed line) cases.

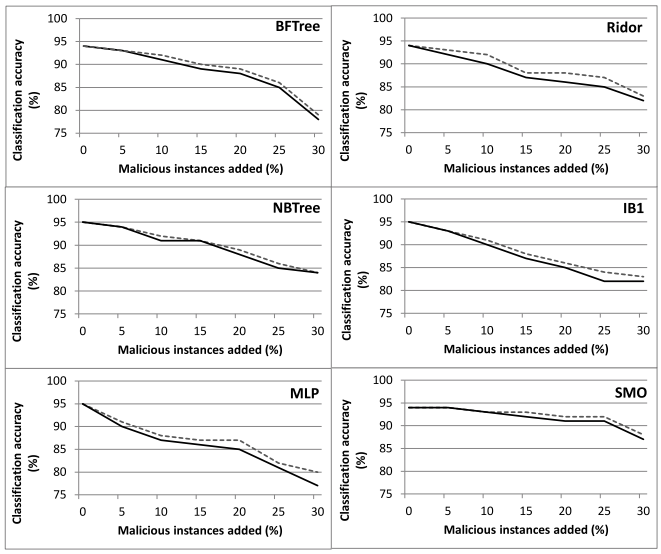


Fig. 5. Results of attacks on the Breast Cancer dataset for the fixed (solid line) and evolving (dashed line) cases.

The results of our attacks for the five datasets are shown in Figs. 4–8. In each of these figures, the classification accuracy degradations corresponding to the classification results for the attacked class, after adding malicious data to the training dataset, are shown for the six machine-learning algorithms. The feasibility of adding malicious instances to the datasets depends on the specific applications and contexts. Moreover, the presented results for the attacks (in which up to 30% malicious instances are added) are only meant to be illustrative and provide general guidelines. In other words, as seen from the tables throughout the paper, attacks with lower number of malicious instances

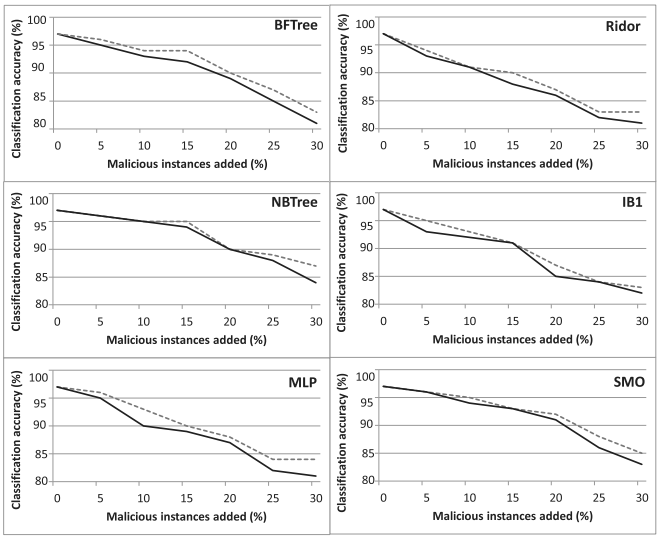


Fig. 6. Results of attacks on the Acute Inflammations dataset for the fixed (solid line) and evolving (dashed line) cases.

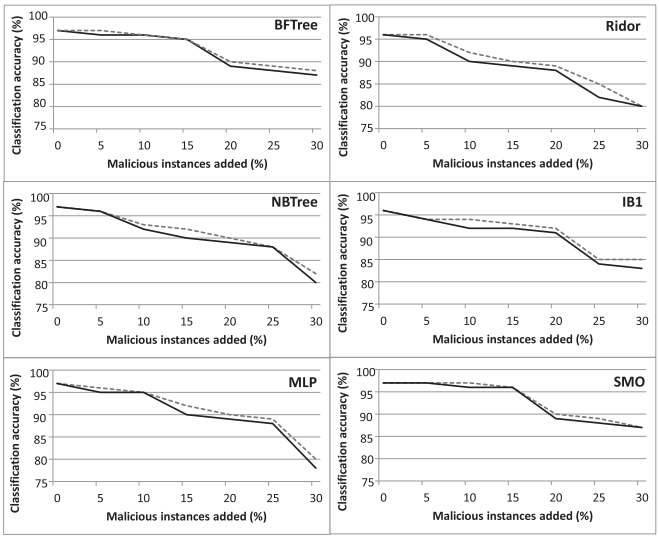


Fig. 7. Results of attacks on the Echocardiogram dataset for the fixed (solid line) and evolving (dashed line) cases.

could be considered "successful," depending on the goal of the attacker. Based on the attack objectives, restrictions on how many malicious entries can be added, and the overhead that can be tolerated, the attacker can choose an appropriate number of added malicious instances to mount the attacks.

The two curves in each graph represent two different assumptions about the nature of the training dataset. In the first one (solid lines in the graphs), the training dataset is considered to be fixed for the duration of the attack. In the second one (dashed lines in the graphs), the dataset is considered to be evolving (i.e., unknown to the attacker, other users can add authentic data to the training set during the attack). In the latter case, we assume that the number of added authentic instances is the same as the number of added malicious instances. Note

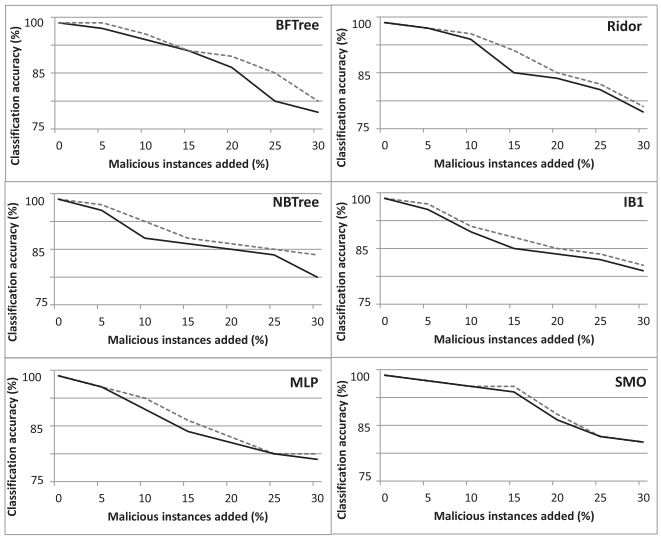


Fig. 8. Results of attacks on the Molecular Biology dataset for the fixed (solid line) and evolving (dashed line) cases.

that the evolving dataset assumption makes the problem tougher for the attacker, since the statistics of the newly added authentic instances are unknown to him, and the attack has to be based on the original training dataset. The figures show that the attacks are quite successful across all datasets and algorithms. As the number of added malicious instances increases, the misclassification increases, as indicated by the classification accuracy percentage decreasing (note that the classification accuracies for the datasets and machine-learning algorithms were originally very high). The attack remains effective even for evolving datasets, causing the misclassification to be only slightly lower than in the case of fixed datasets.

For the results described previously, we set the number of iterations I to 50 in Algorithm 1. To check that this was adequate, we also used $I = 100$ in a few cases (thus utilizing twice the CPU time). However, the misclassifications (which reflect the efficacy of the attacks) increased by less than 1% (compared to the 50 iterations case), when 30% malicious data were added to the training dataset. Since this is not that significant, we conclude that $I = 50$ appears to be sufficient for achieving the attack objectives.

We would like to emphasize that the proposed approach is successful without raising suspicion when the attacks are mounted. One may simply suggest flipping the class labels of the instances in the training dataset to create malicious instances. However, not only is such a scheme not efficient when the number of instances in the training dataset is high, but it might also raise suspicion. The precision of the proposed scheme (by changing the number of iterations at the cost of more CPU time) gives the attacker flexibility to mount attacks based on the resources available. Moreover, the analysis of statistics in the proposed approach before devising the attacks makes it applicable to large datasets. Indeed, when we deal with evolving datasets, the problem gets tougher since the statistics of the newly added

TABLE V
COMPARISON OF THE EFFECTIVENESS OF OUR ATTACKS ON THE MACHINE-LEARNING ALGORITHMS CONSIDERED

Attack	Thyroid Disease (most to least vulnerable)						Breast Cancer (most to least vulnerable)					
15% added	**IB1** (9%)	BFTree (7%)	Ridor (5%)	NBTree (4%)	MLP (3%)	**SMO** (3%)	**MLP** (14%)	IB1 (11%)	BFTree (8%)	Ridor (4%)	NBTree (3%)	**SMO** (3%)
30% added	**MLP** (20%)	Ridor (18%)	BFTree (18%)	IB1 (16%)	NBTree (13%)	**SMO** (12%)	**MLP** (26%)	BFTree (23%)	Ridor (22%)	IB1 (18%)	NBTree (16%)	**SMO** (16%)
	Acute Inflammations (most to least vulnerable)						Echocardiogram (most to least vulnerable)					
15% added	**Ridor** (9%)	BFTree (9%)	IB1 (8%)	NBTree (8%)	MLP (6%)	**SMO** (6%)	**Ridor** (8%)	NBTree (8%)	IB1 (7%)	MLP (6%)	BFTree (3%)	**SMO** (3%)
30% added	**Ridor** (21%)	BFTree (18%)	IB1 (18%)	MLP (14%)	NBTree (12%)	**SMO** (12%)	**NBTree** (20%)	IB1 (18%)	MLP (16%)	Ridor (16%)	BFTree (11%)	**SMO** (11%)
	Molecular Biology (most to least vulnerable)											
15% added	**IB1** (9%)	BFTree (9%)	Ridor (7%)	NBTree (6%)	MLP (6%)	**SMO** (5%)						
30% added	**BFTree** (18%)	IB1 (17%)	MLP (15%)	NBTree (15%)	Ridor (12%)	**SMO** (12%)						

Note: Changes in the misclassification percentage compared to the original dataset, i.e., the effectiveness of attacks, are shown in parentheses.

authentic instances are unknown. However, the presented attack mechanism is still successful in such cases.

Table V presents a comparison of the six machine-learning algorithms on the five datasets for the case of evolving datasets, to gauge the vulnerability of these algorithms to poisoning attacks. It looks at the vulnerability half-way during the attack (when half of the malicious instances are added, i.e., 15% of the original dataset), and at the end (when all malicious instances are added, i.e., 30% of the original dataset). The entries in this table indicate the reductions in the attacked label percentage that are achieved. The entries are sorted from the most vulnerable algorithm (left) to the least vulnerable algorithm (right). The results indicate that, at the end of the attacks, SMO is found to be the most robust.

In order to further evaluate the efficacy of the proposed attack strategies, we performed experiments on the Breast Cancer dataset for the six machine-learning algorithms through addition of pseudorandom malicious instances. These instances are generated by pseudorandomly choosing the attribute values without using Algorithm 2 for deriving the malicious instance candidates, i.e., equal weights are given to the subsets with weights $W \geq 1$ within the bins and the other bins are ignored. The experimental results show that after adding 30% malicious instances, a significant reduction in the success of attacks is seen compared to the proposed attack approach. Specifically, for MLP, BFTree, Ridor, IB1, NBTree, and SMO, the effectiveness for the proposed (pseudorandom) attack is, respectively: 26% (16%), 23% (15%), 22% (13%), 18% (12%), 16% (9%), and 16% (10%). We note that this type of attack in not completely random as the cases, for which $W < 1$, are ignored. The difference in the effectiveness of the proposed attack and a random attack would be even more than the above difference with respect to a pseudorandom attack.

IB1 machine learning uses normalized Euclidean distance to find the training instance closest to the given test instance, and predicts the same class as this training instance. If multiple instances have the same (smallest) distance to the test instance, the first one found is used. We also performed experiments using IBk (k-nearest neighbor classifier in which parameter k can be optimized to improve classifier performance) for the Breast Cancer dataset for $k = 2$, for which slightly better performance is observed. The results show that the proposed attacks are successful with a success rate close to that of IB1 (around 18% change in the attacked label percentage). We would like to emphasize that although the algorithm performance is an important factor, benchmarking the success of attacks (countermeasures) with respect to the original performance of the machine-learning algorithms is our primary focus.

We also experimented with a regression-based logistic classifier, SimpleLogistic [44], for building linear logistic regression models for the five datasets. After mounting the attacks (adding 30% malicious instances), the decrease in the attacked label percentages for Thyroid Disease, Breast Cancer, Acute Inflammations, Echocardiogram, and Molecular Biology datasets are 17%, 20%, 18%, 17%, and 16%, respectively. These success rates put the SimpleLogistic classifier roughly in the middle of the other classifiers in terms of vulnerability to attacks.

In some applications, when there is incomplete or incoherent knowledge, e.g., noisy data, classification based on similarity of objects (termed similarity-based classification) may be used [45]. In such approaches, a similarity criterion is used to quantify the similarities among objects by deriving a numerical measure that is eventually utilized for inference. Accordingly, for n-way classification, similarity-based classification returns n classes with which n similarity degrees are associated.

The proposed algorithm-independent attacks can be adapted to such classification approaches as well. Here, the aforementioned similarity criterion needs to be converted to a malicious one so that the incoming new objects have higher similarity degrees for a particular attacking class. In other words, the attack will be directed toward decreasing the similarity degree(s) of the attacked class(es) and increasing that of the attacking class. Depending on the method used for deriving the similarity criterion, Algorithms 1 and 2 can be used to mount the attacks. For instance, if any statistical measure (for the objects getting compared) is used as the similarity criterion, one may slightly modify Algorithm 1 to benchmark this measure (as the effectiveness factor) to cause a deviation in the similarity degree of the attacked class. We would like to emphasize that instead of attacking the training datasets or their surrogates, the similarity

criterion is maliciously targeted to reach the attacker's objectives in this case.

1) Attacking Without Access to the Training Dataset: Our experiments so far demonstrate that the proposed attack scheme can be applied to machine-learning algorithms whose training datasets are known to the attackers. We now consider the case in which prior knowledge of the statistics of the training datasets is not assumed for the attackers. The attack in this case is based on getting feedback from the machine-learning algorithm to construct an "artificial" dataset.

For performing experiments in this case, an "artificial" dataset is created by giving test inputs to the machine-learning algorithm. We note that based on the classification accuracy of the machine-learning algorithm used, this newly created dataset may contain a few incorrect class labels. This might cause minor deviations (based on the misclassification ratio) in the choice of the malicious instance candidate (see Algorithm 2). Our experiments indicate that after adding malicious data to this "artificial" dataset, the attack is still quite successful, with the results being very close to the results obtained through the original approach. Specifically, for the machine-learning algorithms BFTree (one of the most vulnerable ones in Table V) and SMO (the least vulnerable one in Table V), only 1–2% and 2–3% change are observed in the attacking class misclassification, respectively, for the five datasets.

We would like to emphasize that this approach is less effective when the classification accuracy of the machine-learning algorithms is less; nonetheless, due to inaccurate results, typically, these types of algorithms are not used for sensitive applications. The presented approach can be applied to other datasets and machine-learning algorithms as well. Moreover, the attacker does not necessarily need to know the type of the machine-learning algorithm used.

2) Attacking Unknown Machine-Learning Algorithms: Our experiments suggest that the proposed attack scheme can be applied to a wide range of machine-learning algorithms. However, in our discussions so far, the machine-learning algorithm used was assumed to be known to the attacker (recall that Algorithm 1 performs training and validation to evaluate the efficacy of the malicious instances). Algorithm 1 can be modified to the scenario where the attacker does not know the machine-learning algorithm used. We eliminate the inner loop in Algorithm 1, which generates I malicious instance candidates and evaluates them through training and validation to pick the most suitable one. Instead, we simply pick each malicious instance candidate generated by Algorithm 2 and add it to the dataset. The results of our experiments show that the attacks still succeed, although they are not as effective as when the attacker knows the algorithm being used. For instance, for the Breast Cancer dataset and for different machine-learning algorithms, our experiments show that the effectiveness of the attacks is 8–10% higher when the machine-learning algorithm is known.

In summary, knowing the machine-learning algorithm used and performing retraining and validation to select each malicious instance does result in higher attack efficacy. However, hiding this information does not eliminate the ability of the attacker to mount poisoning attacks.

3) Adapting the Attacks to Real Patterns: The proposed attacks are effective on datasets whose features (attributes) are derived through a feature extraction process (which is a reduction step) from real patterns. However, applying these attacks, in practice, would require that inputs be generated as real patterns, and not simply in the feature space. We next discuss issues involved in adapting the proposed attacks to real patterns.

It is possible to create real attack patterns from the malicious datasets derived through our proposed attacks. We note that knowledge of the feature extraction scheme is required for this backward projection. For example, consider the Breast Cancer dataset whose attributes are the tissue physical/chemical characteristics, e.g., clump thickness or uniformity of cell size, in the patterns from the reports of clinical cases. Once the proposed attacks result in malicious training datasets that are represented in the attribute space, one could generate real patterns, i.e., synthetic clinical reports that exhibit the maliciously chosen tissue characteristics. However, these patterns are not necessarily unique, i.e., there might be more than one malicious pattern corresponding to a malicious attribute vector. Moreover, one can attack patterns directly by maliciously targeting parts of the clinical reports corresponding to the target attributes.

Let us consider and elaborate upon two scenarios for the aforementioned dataset, i.e., Breast Cancer. In the first scenario, assuming that the feature extraction process is known (tissue physical/chemical characteristics), removing or adding values for tissue characteristics in order to create malicious patterns is equivalent to changing the feature vectors. In fact, one can construct attack patterns exhibiting the same feature values of a malicious instance used in the attacks; thus, simulating the attacks can be performed by directly modifying the feature vectors instead of the patterns [16]. In the second scenario, consider dealing with more than one feature extraction method for the patterns used for different applications to create different feature sets. In this case, the attacker needs to target the union of features of these methods for the attacks, e.g., a number of tissue characteristics sets for different methods. This, in turn, would increase the number of changes needed by the attackers in the real patterns to cover the feature sets (typically, the intersection of these feature sets is large). In summary, we believe that the feature vectors generated by the proposed attacks are typically extensible to real patterns; nevertheless, solving the general problem of generating real patterns is outside the scope of our study and is an interesting future research topic.

4) Attacking n-Way Classification: So far, we have presented results for two-way classifications. However, our attack scheme can be generalized to n-way classification in a straightforward manner. For example, let us consider a four-way classification problem based on the Hypothyroid dataset from UCI [46] in which class labels are negative, compensated-hypothyroid, primary-hypothyroid, and secondary-hypothyroid. In this case, an attacker can mount the proposed attack using the structure shown in Fig. 9, in which the objective of the attack is misclassification of the three attacked class labels (Classes 1, 3, and 4) toward the attacking compensated-hypothyroid label (Class 2). Table VI shows the results of the mounted attacks for the NBTree machine-learning algorithm. After adding around 20%

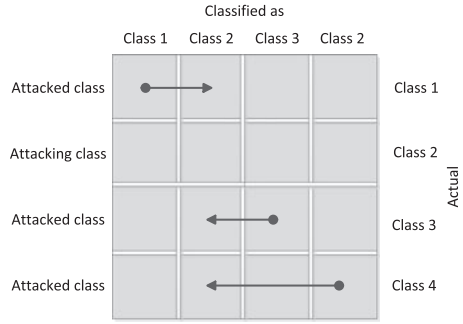


Fig. 9. Attack structure for a four-way classification problem, e.g., on the Hypothyroid dataset.

TABLE VI
ATTACK RESULTS FOR THE HYPOTHYROID DATASET AFTER ADDING 20% MALICIOUS INSTANCES

Attack Class	1	2 (attacking)	3	4
Before	92%	**4%**	2%	2%
After	80%	**18%**	1%	1%

malicious instances to the training dataset based on Algorithms 1 and 2, classification results on the validation set show a significant bias toward the attacking class label (from 4% to 18%).

D. Countermeasures Against Poisoning Attacks

In this section, we present a countermeasure to the proposed poisoning attacks. The proposed method is based on periodically constructing a model using the training dataset, evaluating its accuracy on the validation dataset, and raising an alarm in case of any suspicious change in the accuracy metrics. The effectiveness of the presented countermeasure suggests devising attacks that are resistant to such methods, capable of circumventing these countermeasures. While several metrics are used to evaluate the accuracy of classification, we consider the following two in our study.

1) *Correctly classified instances (CCI):* This statistic indicates the fraction of instances that are classified correctly in an n-way classification (the higher the CCI value, the better the classification).
2) *Kappa statistic [47]:* This statistic measures relative improvement over random predictors, i.e., it is a measure of how good the classifier is compared to a random classifier (the higher the Kappa statistic, the better the classification).

The proposed countermeasure is described in Algorithm 3. For any chosen accuracy metric, the user first computes a "golden" value by building a model when the dataset is in a trusted state and evaluating the model on a fixed validation set (note that the validation set does not evolve as new data are added to the training dataset). The user then sets a threshold value for the accuracy metric that is lower than the golden value. The model is periodically regenerated from the training dataset and evaluated on the validation set. An alarm is raised if the accuracy falls below the threshold value. The choice of the threshold value trades off the sensitivity of the countermeasure to attacks with the likelihood of false alarms due to natural variations in the benign data added to the dataset. In addition to absolute values of accuracy metrics, users can also track the rates of change of these metrics as new instances are added to the training dataset.

Algorithm 3 Presented countermeasure.

1:**Input:** Evolving Dataset $D \in (\chi, \Upsilon)$ with $N + X$ instances, validation dataset V.
2:**Output:** Attack flag AF.
3:**Begin**
4: Set a threshold for the accuracy metrics, e.g., CCI or Kappa statistic
5: Obtain accuracy metrics for V after building the model through D
6: While the accuracy metrics have not reached the set threshold, compute the accuracy metrics
7: If any of the thresholds is reached, set AF=1
8:**End**
9:**Return:** AF.

The changes in accuracy metrics for the attacks on the six machine-learning algorithms and the five evolving datasets presented earlier in this section are given in Table VII. The values tabulated correspond to the half-way and the end of the attacks (i.e., 15% and 30% increase in the dataset, respectively). The lowest and the second lowest changes shown for each metric are depicted by "++" and "+," respectively. Based on these values, we would draw the same conclusion that we drew earlier from Table V, i.e., SMO is the most difficult to attack (note that the degradation in the values of these metrics is the least; among the six algorithms, it maintains the highest accuracy under the attacks).

For setting a threshold value for accuracy metrics, e.g., CCI, a conservative rule of thumb could be to allow a 10% deviation from the original value after mounting the attacks. The reason for such a choice is that in our experiments, classification accuracies were found to be higher than 90% (inaccuracies of less than 10%), and thus, a 10% deviation would be close to twice the inaccuracies in many cases, reducing the chance of encountering false alarms. However, this choice is highly dependent on the context and datasets. Hence, we have not specified a fixed threshold for the countermeasure experiments. Based on the results shown in Table VII, with such a conservative threshold, all the attacks are detected after adding 30% malicious instances. However, except for two cases for the Breast Cancer dataset, none are detected after adding 15% malicious instances. The threshold could be tailored dynamically to trade off countermeasure sensitivity to attacks with the likelihood of false alarms.

Table VII shows changes in the two chosen accuracy metrics, i.e., CCI and Kappa, when the proposed attacks are mounted. One can set thresholds for these accuracy metrics to change the sensitivity of the countermeasures. On the other hand, Table V shows success results for the proposed attack, where success

TABLE VII
CHANGE IN ACCURACY METRICS UNDER POISONING ATTACKS

Attack		Thyroid Disease							Breast Cancer					
	Metric	SMO	NBTree	BFTree	MLP	Ridor	IB1	*Metric*	SMO	NBTree	BFTree	MLP	Ridor	IB1
15% added	CCI	3%++	4%+	7%	3%++	5%	9%	CCI	3%++	3%++	8%+	14%	4%+	11%
	Kappa	8%++	10%+	18%	8%++	13%	24%	Kappa	8%++	8%++	20%	35%	10%+	26%
30% added	CCI	12%++	13%+	18%	20%	18%	16%	CCI	12%++	13%++	18%	20%	18%	16%
	Kappa	32%++	34%+	47%	52%	47%	42%	Kappa	30%++	32%+	45%	50%	45%	40%
		Acute Inflammations							Echocardiogram					
15% added	CCI	6%++	8%+	9%	6%++	9%	8%+	CCI	3%++	8%	3%++	6%+	8%	7%
	Kappa	15%++	20%+	22%	15%++	22%	20%+	Kappa	8%++	20%	8%++	15%+	20%	17%
30% added	CCI	12%++	12%++	18%	14%+	21%	18%	CCI	11%++	20%	11%++	16%+	16%+	18%
	Kappa	30%++	30%++	45%	35%+	52%	45%	Kappa	26%++	50%	26%++	40%+	40%+	45%
		Molecular Biology												
15% added	CCI	5%++	6%+	9%	6%+	7%	9%							
	Kappa	13%++	15%+	22%	15%+	18%	22%							
30% added	CCI	12%++	15%+	18%	15%+	12%++	17%							
	Kappa	30%++	36%+	45%	36%+	30%++	43%							

Note: The lowest and the second to lowest changes in each of the statistics are depicted by "++" and "+," respectively.

is defined as maliciously increasing the classification ratio of the instances in the attacking class to the ones in the attacked class. This is why the CCI accuracy metric in Table VII is higher for the cases in which the attack is less successful in Table V. However, we emphasize that in other attack scenarios, e.g., misclassification of the instances in the two classes without necessarily increasing the classification ratio of the attacking class to the attacked class, results in Table VII can be used to raise an alarm even in the absence of a direct correlation to the classification ratio, which the results in Table V are based on. Thus, the results in these two tables complement each other.

We note that the differences in robustness or vulnerability of the machine-learning algorithms to our proposed attacks are analogous to the dissimilar effects of noise on different machine-learning algorithms [10], [11]. Finally, the presented countermeasures could be used to counteract nonmalicious changes in the classification accuracy as well. Moreover, if such changes are intended, over time, the golden validation dataset needs to be updated to reflect the changes in the training dataset.

V. CONCLUSION

In this paper, we proposed systematic attack schemes for mounting poisoning attacks against machine-learning algorithms used for medical datasets, and suggested countermeasures against them. A key feature of the proposed attack schemes is that they can be applied to a wide range of machine-learning algorithms, even when the machine-learning algorithm is unknown. We evaluated the effectiveness of the attacks against six machine-learning algorithms and five datasets [Thyroid Disease, Breast Cancer, Acute Inflammations, Echocardiogram, and Molecular Biology (Splice-junction Gene Sequences)], and ranked the algorithms based on their ability to withstand the attacks. We then presented countermeasures against these attacks and evaluated their effectiveness. Finally, we identified the machine-learning algorithms that are easiest to defend. We hope that our results will spur further research efforts on understanding and countering poisoning attacks on machine learning.

REFERENCES

[1] M. Brameier and W. Banzhaf, "A comparison of linear genetic programming and neural networks in medical data mining," *IEEE Trans. Evol. Comput.*, vol. 5, no. 1, pp. 17–26, Feb. 2001.

[2] W. Lee, S. J. Stolfo, and K. W. Mok, "Adaptive intrusion detection: A data mining approach," *Artif. Intell. Rev.*, vol. 14, no. 6, pp. 533–567, 2000.

[3] C. Whittaker, B. Ryner, and M. Nazif, "Large-scale automatic classification of phishing pages," in *Proc. Symp. Netw. Distrib. Syst. Security*, 2010, pp. 1–14.

[4] M. Conover, B. Goncalves, J. Ratkiewicz, A. Flammini, and F. Menczer, "Predicting the political alignment of twitter users," in *Proc. IEEE Int. Conf. Privacy, Security, Risk Trust*, Oct. 2011, pp. 192–199.

[5] D. Sculley, M. E. Otey, M. Pohl, B. Spitznagel, J. Hainsworth, and Y. Zhou, "Detecting adversarial advertisements in the wild," in *Proc. ACM Int. Conf. Knowl. Discovery Data Mining*, 2011, pp. 274–282.

[6] E. Kirkos, C. Spathis, and Y. Manolopoulos, "Data mining techniques for the detection of fraudulent financial statements," *Expert Syst. Appl.*, vol. 32, no. 4, pp. 995–1003, May 2007.

[7] R. Agrawal and R. Srikant, "Privacy-preserving data mining," *SIGMOD Rec.*, vol. 29, no. 2, pp. 439–450, May 2000.

[8] Y. Li, M. Chen, Q. Li, and W. Zhang, "Enabling multilevel trust in privacy preserving data mining," *IEEE Trans. Knowl. Data Eng.*, vol. 24, no. 9, pp. 1598–1612, Sep. 2012.

[9] M. Kantarcioglu and C. Clifton, "Privacy-preserving distributed mining of association rules on horizontally partitioned data," *IEEE Trans. Knowl. Data Eng.*, vol. 16, no. 9, pp. 1026–1037, Sep. 2004.

[10] N. Cesa-Bianchi, S. Shalev-Shwartz, and O. Shamir, "Online learning of noisy data," *IEEE Trans. Inf. Theory*, vol. 57, no. 12, pp. 7907–7931, Dec. 2011.

[11] D. F. Nettleton, A. Orriols-Puig, and A. Fornells, "A study of the effect of different types of noise on the precision of supervised learning techniques," *Artif. Intell. Rev.*, vol. 33, no. 4, pp. 275–306, 2010.

[12] B. Nelson, B. Biggio, and P. Laskov, "Understanding the risk factors of learning in adversarial environments," in *Proc. ACM Workshop Security Artif. Intell.*, 2011, pp. 87–92.

[13] B. Nelson, M. Barreno, F. J. Chi, A. D. Joseph, B. I. P. Rubinstein, U. Saini, C. Sutton, J. D. Tygar, and K. Xia, "Exploiting machine learning to subvert your spam filter," in *Proc. Usenix Workshop Large-Scale Exploits Emergent Threats*, 2008, pp. 7:1–7:9.

[14] K. M. C. Tan, J. McHugh, and K. S. Killourhy, "Hiding intrusions: From the abnormal to the normal and beyond," in *Proc. Int. Workshop Inf. Hiding*, 2003, pp. 1–17.

[15] B. I. Rubinstein, B. Nelson, L. Huang, A. D. Joseph, S.-H. Lau, S. Rao, N. Taft, and J. D. Tygar, "Stealthy poisoning attacks on PCA-based anomaly detectors," *SIGMETRICS Perform. Eval. Rev.*, vol. 37, no. 2, pp. 73–74, Oct. 2009.

[16] B. Biggio, G. Fumera, and F. Roli, "Security evaluation of pattern classifiers under attack," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 4, pp. 984–996, Apr. 2014.

[17] N. Dalvi, P. Domingos, Mausam, S. Sanghai, and D. Verma, "Adversarial classification," in *Proc. ACM Int. Conf. Knowl. Discovery Data Mining*, 2004, pp. 99–108.
[18] M. Barreno, B. Nelson, R. Sears, A. D. Joseph, and J. D. Tygar, "Can machine learning be secure?" in *Proc. ACM Symp. Inf., Comput. Commun. Security*, 2006, pp. 16–25.
[19] L. Huang, A. D. Joseph, B. Nelson, B. I. Rubinstein, and J. D. Tygar, "Adversarial machine learning," in *Proc. ACM Workshop Security Artif. Intell.*, 2011, pp. 43–58.
[20] M. Brückner, C. Kanzow, and T. Scheffer, "Static prediction games for adversarial learning problems," *J. Mach. Learn. Res.*, vol. 13, pp. 2617–2654, 2012.
[21] M. Barreno, B. Nelson, A. D. Joseph, and J. D. Tygar, "The security of machine learning," *Mach. Learn.*, vol. 81, no. 2, pp. 121–148, Nov. 2010.
[22] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in *Proc. Int. Conf. Mach. Learn.*, 2012, pp. 1807–1814.
[23] B. Rubinstein, B. Nelson, L. Huang, A. Joseph, S.-H. Lau, S. Rao, N. Taft, and J. D. Tygar, "ANTIDOTE: Understanding and defending against poisoning of anomaly detectors," in *Proc. ACM SIGCOMM Conf. Internet Meas.*, 2009, pp. 1–14.
[24] M. Kloft and P. Laskov, "Online anomaly detection under adversarial impact," *J. Mach. Learn. Res.*, vol. 9, pp. 405–412, 2010.
[25] B. Biggio, I. Corona, G. Fumera, G. Giacinto, and F. Roli, "Bagging classifiers for fighting poisoning attacks in adversarial classification tasks," in *Proc. Int. Conf. Multiple Classifier Syst.*, 2011, pp. 350–359.
[26] A. Globerson and S. Roweis, "Nightmare at test time: Robust learning by feature deletion," in *Proc. Int. Conf. Mach. Learn.*, 2006, pp. 353–360.
[27] H. Shi, "Best-first decision tree learning," M.Sc. thesis, Dept. Comput. Sci., Univ. Waikato, Hamilton, New Zealand, 2007.
[28] J. Friedman, T. Hastie, and R. Tibshirani, "Additive logistic regression: A statistical view of boosting," *Ann. Statist.*, vol. 28, no. 2, pp. 337–407, 2000.
[29] R. Dazeley, P. Warner, S. Johnson, and P. Vamplew, "The Ballarat incremental knowledge engine," in *Proc. Int. Conf. Knowl. Manag. Acquisition Smart Syst. Serv.*, 2010, pp. 195–207.
[30] D. Richards, "Two decades of ripple down rules research," *Knowl. Eng. Rev.*, vol. 24, no. 2, pp. 159–184, 2009.
[31] R. Kohavi, "Scaling up the accuracy of naive-Bayes classifiers: A decision-tree hybrid," in *Proc. Int. Conf. Knowl. Discovery Data Mining*, 1996, pp. 202–207.
[32] D. W. Aha, D. Kibler, and M. K. Albert, "Instance-based learning algorithms," *Mach. Learn.*, vol. 6, no. 1, pp. 37–66, Jan. 1991.
[33] R. Collobert and S. Bengio, "Links between perceptrons, MLPs and SVMs," in *Proc. Int. Conf. Mach. Learn.*, 2004, pp. 23–31.
[34] J. C. Platt, *Advances in Kernel Methods*. Cambridge, MA, USA: MIT Press, 1999, pp. 185–208.
[35] S. S. Keerthi, S. K. Shevade, C. Bhattacharyya, and K. R. K. Murthy, "Improvements to Platt's SMO algorithm for SVM classifier design," *Neural Comput.*, vol. 13, no. 3, pp. 637–649, Mar. 2001.
[36] T.-F. Wu, C.-J. Lin, and R. C. Weng, "Probability estimates for multi-class classification by pairwise coupling," *J. Mach. Learn. Res.*, vol. 5, pp. 975–1005, 2004.
[37] Thyroid disease data set. [Online]. Available: http://archive.ics.uci.edu/ml/datasets/Thyroid+Disease, 2014.
[38] Breast cancer data set. [Online]. Available: http://archive.ics.uci.edu/ml/datasets/Breast+Cancer+ Wisconsin+ (Original), 2014.
[39] Acute inflammations data set. [Online]. Available: http://archive.ics.uci.edu/ml/datasets/Acute+Inflammations, 2014.
[40] Echocardiogram data set. [Online]. Available: http://archive.ics.uci.edu/ml/datasets/Echocardiogram, 2014.
[41] Molecular biology (promoter gene sequences) data set. [Online]. Available: http://archive.ics.uci.edu/ml/datasets/Molecular+Biology+ (Splice-junction+Gene+ Sequences), 2014.
[42] R. Azarderskhsh and A. Reyhani-Masoleh, "Secure clustering and symmetric key establishment in heterogeneous wireless sensor networks," *EURASIP J. Wireless Commun. Netw.*, vol. 2011, pp. 16:1–16:12, Jan. 2011.
[43] Weka 3: Data mining software in Java. [Online]. Available: http://www.cs.waikato.ac.nz/ml/weka/index.html, 2014.
[44] N. Landwehr, M. Hall, and E. Frank, "Logistic model trees," *Mach. Learn.*, vol. 59, no. 1, 2, pp. 161–205, 2005.
[45] G. Bisson, "Why and how to define a similarity measure for object based representation systems," in *Proc. Towards Very Large Knowl. Bases*, 1995, pp. 236–246.
[46] Hypothyroid data set. [Online]. Available: http://repository.seasr.org/Datasets/UCI/arff/hypothyroid.arff, 2014.
[47] J. Carletta, "Assessing agreement on classification tasks: The Kappa statistic," *Comput. Linguist.*, vol. 22, no. 2, pp. 249–254, 1996.

Mehran Mozaffari-Kermani (M'11) received the B.Sc. degree in electrical and computer engineering from the University of Tehran, Tehran, Iran, in 2005, and the M.E.Sc. and Ph.D. degrees from the Department of Electrical and Computer Engineering, University of Western Ontario, London, ON, Canada, in 2007 and 2011, respectively.

He joined Advanced Micro Devices as a Senior ASIC/Layout Designer, integrating sophisticated security/cryptographic capabilities into a single accelerated processing unit. He received the prestigious Natural Sciences and Engineering Research Council of Canada (NSERC) Postdoctoral Research Fellowship. In 2012, he joined the Electrical Engineering Department, Princeton University, Princeton, NJ, USA, as an NSERC Postdoctoral Research Fellow. He is currently with the Department of Electrical and Microelectronic Engineering, Rochester Institute of Technology, Rochester, NY, USA. His current research interests include emerging security/privacy measures for deeply embedded systems, cryptographic hardware systems, fault diagnosis and tolerance in cryptographic hardware, VLSI reliability, and low-power secure and efficient FPGA and ASIC designs.

He is the Guest Editor of the IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTING for the special issue of Emerging Security Trends for Deeply-Embedded Computing Systems (2014–2015). He is currently serving as the Technical Committee Member for a number of security/reliability conferences including DFT, FDTC, RFIDsec, LightSEC, and WAIFI. His research in 2014–2015 is funded through the Texas Instruments Faculty Award (Douglas Harvey).

Susmita Sur-Kolay (SM'05) received the B.Tech. degree in electronics and electrical communication engineering from IIT, Kharagpur, India, and the Ph.D. degree in computer science and engineering from Jadavpur University, Kolkata, India.

She was in the Laboratory for Computer Science, Massachusetts Institute of Technology, Cambridge, MA, USA, from 1980 to 1984. She was a Postdoctoral Fellow in the University of Nebraska-Lincoln, Nebraska-Lincoln, NE, USA, in 1992, a Reader in Jadavpur University from 1993 to 1999, a Visiting Faculty Member with Intel Corporation, Santa Clara, CA, USA, in 2002, and a Visiting Researcher in Princeton University, Princeton, NJ, USA, in 2012. She is a Professor in the Advanced Computing and Microelectronics Unit, Indian Statistical Institute, Kolkata. She has coedited two books, authored a book chapter in the Handbook of Algorithms for VLSI Physical Design Automation, and coauthored about 100 technical articles. Her current research interests include electronic design automation, hardware security, quantum computing, and graph algorithms.

Dr. Sur-Kolay was a Distinguished Visitor of the IEEE Computer Society, India. She has been an Associate Editor of the IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION SYSTEMS and is currently an Associate Editor of the *ACM Transactions on Embedded Computing Systems*. She has served on the technical program committees of several leading conferences, and as the Program Chair of the 2005 International Conference on VLSI Design, the 2007 International Symposium on VLSI Design and Test, and the 2011 IEEE Computer Society Annual Symposium on VLSI. Among other awards, she received the President of India Gold Medal from IIT, Kharagpur.

Anand Raghunathan (F'12) received the B.Tech. degree in electrical and electronics engineering from the Indian Institute of Technology, Madras, India, and received the M.A. degree from Princeton University, Princeton, NJ, USA, where he also received the Ph.D. degree in electrical engineering in 1997.

He is a Professor in the School of Electrical and Computer Engineering at Purdue University, West Lafayette, IN, USA, where he leads the Integrated Systems Laboratory. His research explores domain-specific architecture, system-on-chip design, embedded systems, and heterogeneous parallel computing. Previously, he was a Senior Research Staff Member at NEC Laboratories America and held the Gopalakrishnan Visiting Chair in the Department of Computer Science and Engineering, Indian Institute of Technology, Madras, India.

Dr. Raghunathan has coauthored a book *High-Level Power Analysis and Optimization*, eight book chapters, 21 U.S. patents, and more than 200 refereed journal and conference papers. His publications have been recognized with eight best paper awards and four best paper nominations. He received the Patent of the Year Award (recognizing the invention with the highest impact), and two Technology Commercialization Awards from NEC. He was chosen by MIT's Technology Review among the TR35 (top 35 innovators under 35 years, across various disciplines of science and technology) in 2006, for his work on "making mobile secure." He has served on the technical program and organizing committees of several leading conferences and workshops. He has chaired the ACM/IEEE International Symposium on Low Power Electronics and Design, the ACM/IEEE International Conference on Compilers, Architecture, and Synthesis for Embedded Systems, the IEEE VLSI Test Symposium, and the IEEE International Conference on VLSI Design. He has served as Associate Editor of the IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, IEEE TRANSACTIONS ON VLSI SYSTEMS, *ACM Transactions on Design Automation of Electronic Systems*, IEEE TRANSACTIONS ON MOBILE COMPUTING, *ACM Transactions on Embedded Computing Systems*, *IEEE Design & Test of Computers*, and *the Journal of Low Power Electronics*. He received the IEEE Meritorious Service Award in 2001 and the Outstanding Service Award in 2004. He is a Golden Core Member of the IEEE Computer Society.

Niraj K. Jha (S'85–M'85–SM'93–F'98) received the B.Tech. degree in electronics and electrical communication engineering from the Indian Institute of Technology (IIT), Kharagpur, India, in 1981, the M.S. degree in electrical engineering from S.U.N.Y. at Stony Brook, NY, USA, in 1982, and the Ph.D. degree in electrical engineering from the University of Illinois at Urbana-Champaign, Champaign, IL, USA, in 1985.

He is a Professor of electrical engineering at Princeton University, Princeton, NJ, USA. His research interests include FinFETs, low-power hardware/software design, computer-aided design of integrated circuits and systems, digital system testing, quantum computing, and secure computing. He has given several keynote speeches in the area of nanoelectronic design and test.

Dr. Jha is a Fellow of the ACM. He received the Distinguished Alumnus Award from IIT Kharagpur in 2014. He has served as the Editor-in-Chief of IEEE TRANSACTIONS ON VLSI SYSTEMS and an Associate Editor of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS I AND II, IEEE TRANSACTIONS ON VLSI SYSTEMS, IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN, and *Journal of Electronic Testing: Theory and Applications*. He is currently serving as an Associate Editor of IEEE TRANSACTIONS ON COMPUTERS, *Journal of Low Power Electronics and Journal of Nanotechnology*. He has also served as the Program Chairman of the 1992 Workshop on Fault-Tolerant Parallel and Distributed Systems, the 2004 International Conference on Embedded and Ubiquitous Computing, and the 2010 International Conference on VLSI Design. He has served as the Director of the Center for Embedded System-on-a-chip Design funded by New Jersey Commission on Science and Technology. He received the AT&T Foundation Award and NEC Preceptorship Award for research excellence, NCR Award for teaching excellence, and Princeton University Graduate Mentoring Award. He has coauthored or coedited five books titled *Testing and Reliable Design of CMOS Circuits* (Kluwer, 1990), *High-Level Power Analysis and Optimization* (Norwell, MA, USA: Kluwer, 1998), *Testing of Digital Systems* (Cambridge, U.K.: Cambridge Univ. Press, 2003), *Switching and Finite Automata Theory* (3rd ed. Cambridge, U.K.: Cambridge Univ. Press, 2009), and *Nanoelectronic Circuit Design* (New York, NY, USA: Springer, 2010). He has also authored 15 book chapters. He has authored or coauthored more than 400 technical papers. He has coauthored 14 papers, which have won various awards. These include the Best Paper Award at ICCD'93, FTCS'97, ICVLSID'98, DAC'99, PDCS'02, ICVLSID'03, CODES'06, ICCD'09, and CLOUD'10. A paper of his was selected for "The Best of ICCAD: A collection of the best IEEE International Conference on Computer-Aided Design papers of the past 20 years," two papers by IEEE Micro Magazine as one of the top picks from the 2005 and 2007 Computer Architecture conferences, and two others as being among the most influential papers of the last ten years at IEEE Design Automation and Test in Europe Conference. He has coauthored another six papers that have been nominated for best paper awards. He has received 14 U.S. patents. He has served on the program committees of more than 150 conferences and workshops.

Machine Learning-Based Framework for Detecting and Mitigating DoS Attacks in Mobile Ad-Hoc Networks (MANETs)

DR BABANGIDA ZUBAIRU, AISHA IBRAHIM GIDE
Department of Computer Science
Umaru Musa Yar'adua University
Katsina, NIGERIA

Abstract: - A Mobile Ad-hoc Network (MANET) is a dispersed, decentralized network of mobile wireless nodes that interacts directly with one another without the use of centralized management or stationary base stations. In a MANET's, nodes are always moving, randomly and unpredictably, which presents a number of difficulties and leaves these networks especially open to different security risks. These infrastructure-less networks are especially vulnerable to security threats like Denial of Service (DoS) attacks, black hole attacks, network partitioning, and node selfishness since they lack central management and have limited hardware resources. This paper presents a framework for detecting and mitigating Denial of Service (DoS) attacks in Mobile Ad-hoc Networks (MANETs) using various machine learning models, including Support Vector Machine (SVM), Random Forest, Neural Network, and K-Nearest Neighbors (KNN). In order to simulator both normal and attack traffic, the proposed system generate synthetic datasets from simulation, trains these models, and assesses their efficacy using a number of performance metrics, such as accuracy, scalability, energy consumption, resource utilization, and network throughput. The results shows that Random Forest, SVM, KNN, and Neural Networks have the highest detection accuracy. Neural Networks also showed better network performance and scalability, which made them appropriate for high-traffic situations. On the other hand, KNN is observed to have less energy consumption, which made it an effective choice for situations where resources are limited.

Key-Words: DoS Attack Detection, Mobile Ad-hoc Networks (MANETs), Machine Learning Models, Neural Network

Received: May 14, 2024. Revised: January 26, 2025. Accepted: March 27, 2025. Published: June 2, 2025.

1. Introduction

A Mobile Ad-hoc Network (MANET) is a network made up of mobile nodes connected using wireless multi-hop technology. By enabling each node function as a router independently, network functionality can be attained without the need for a fixed infrastructure [1] MANET is rapidly gaining traction and is composed of affordable, compact, and reliable equipment. Adaptive and self-organizing, devices can easily add or remove nodes from these networks while maintaining connections. Search and rescue operations, military operations, and sensor networks are just a few of the scenarios in which MANETs can be applied [2]. MANETs' rapid deployment and dynamic configurability are two important characteristics that have added to their growing popularity. MANETs can provide essential internet access in the event of a disaster if there is a chance that the current communication infrastructures would be destroyed. However, the routing algorithm in charge of enabling packet exchanges between a MANET's migrating nodes becomes more crucial in dictating the network's performance in terms of throughput and end-to-end latency. MANETs are characterized by a dynamic topology in which nodes move around a lot, which affects network security [3]. MANET topologies are inherently unstable, undergoing frequent and random changes over time. Managing connections and routing communications becomes extremely difficult in the absence of centralized control or infrastructure [4]. The network's dynamic structure, which includes nodes changing locations frequently, makes it particularly susceptible to attacks like flooding, in which an overwhelming volume of data is purposefully generated to reduce performance. Because of the dynamic topology and lack of central authority, attacks are more dangerous, making it more difficult to ensure secure and effective communication [5]. Securing MANETs is a significant challenge to research, despite the fact that they are vulnerable to a range of attacks that compromise network security. A significant risk that must not be disregarded is the Denial of Service (DoS) attack. In DoS attack, an attacker floods the network with hundreds or thousands of useless packets, flooding the victim's machine. The network's capacity and performance may be severely reduced by this enormous traffic, making it unable to carry out its operations and offer services to valid nodes [6]. The main objective of this paper is to experiment the application of machine learning models to in mitigating DoS attack in MANET. To achieve this first we implement MANET network and DoS attack in the network, then we added detection and mitigation algorithms designed especially for MANETs and machine learning

models like Support Vector Machine (SVM), Random Forest, Neural Network, and Kth Nearest Neighbor (KNN) on generated datasets that simulate both normal and attack traffic. Once trained, the best-performing model is used to predict and visualize traffic patterns in a simulated MANET environment, assisting in identifying attack nodes. To enhance network security, the framework also simulates critical performance measures, such as energy consumption and throughput, to evaluate the impact of attacks and the effectiveness of the detection models. Improved detection performance and resilience against possible DoS attacks in MANETs are ensured by this integrated technique. The remainder of the article is structured accordingly. Section 2 provide an overview of the research background. Section 3 discusses the literature review. Section 4 introduces the proposed methodology and the framework for DoS attack mitigation in Manet. We provide the experimental design, data collection and preprocessing, implementation of the machine learning model and the performance metrics used for evaluations Section 5. Section 6 contains the results and analysis. We discuss our research, implication of the findings and future directions in Section 7. Section 8 concludes the paper.

2. Overview of manets

The 1970s saw the introduction of packet-switched communication, which allowed wireless networks to be developed independently of wired infrastructure. Wireless networks become widely used as a result of this technology's mobility and flexibility. The telecoms industry has changed significantly in the last several decades. Rapid advancements in wireless technology have made life much better and made it possible to access network connections practically constantly from anyplace. The majority of people increasingly rely on mobile devices for their everyday activities since these technologies have grown more widely available and more affordable [7].

Autonomous mobile nodes that dynamically establish multi-hop communication networks are the building blocks of a Mobile Ad Hoc Network (MANET). But these networks are open to all kinds of attacks, such as selfish conduct and denial of service (DoS) attacks. Because the broadcast mechanism is resource-intensive and mobile nodes have limited resources, MANETs are especially vulnerable to denial of service (DoS) attacks. One major problem in MANETs is ensuring secure communication. Due to its distinct features, which include dynamically changing topology, unreliable wireless connectivity, a lack of centralized monitoring, and the absence of a certification authority, traditional security techniques intended for structured networks sometimes do not apply to MANETs [8]. In MANETs, the absence of dedicated routers means that routing is handled entirely by the peer nodes that make up the network. When designing routing protocols for MANETs, enhancing throughput and minimizing packet loss are key considerations. These protocols are generally classified into three main categories, as illustrated in Figure 1.

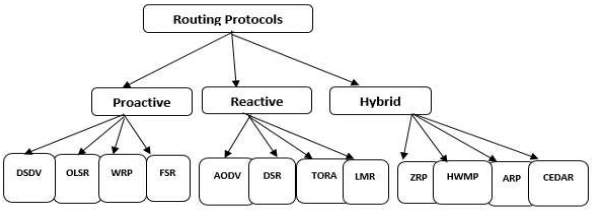


Fig. 1. Routing Protocols in MANETs

Proactive routing protocols, such as Destination Sequenced Distance Vector (DSDV), Optimized Link State Routing (OLSR), Wireless Routing Protocol (WRP), and Fisheye State Routing (FSR), are also referred to as table-driven protocols. Every node on the network registers routes to every other node through proactive routing protocols, which keep track of every route even when they are not needed. These protocols maintain up-to-date routes for every node in the network by routinely exchanging control information between nodes. They also respond to the appearance of a new node or the removal of an existing node from the network topology. The two most well-known proactive protocols are Optimized Link State Routing (OLSR) (Zhiyuan & Jinhong, 2010) and Destination-Sequence Distance-Vector (DSDV) (Mahdipour, Rahmani, & Aminian, 2009). Proactive routing aims to precompute all feasible paths by periodically distributing information throughout the network. Updates are propagated to maintain routing tables up to date whenever a change happens. As a result, in emergency and rescue situations, it is crucial to continuously evaluate the paths connecting nodes in order to gradually evacuate those affected [9]. However, these protocols can be inefficient in their use of bandwidth, and the overhead can become significant as the network is frequently flooded with updates to maintain accurate routing tables [10].

Reactive routing protocols, also known as on-demand protocols, initiate the route discovery process only when a communication between a source and a target node is required. With this method, the overhead usually related to proactive routing where routes are maintained up to date is minimized. Reactive routing creates a communication channel between the source and

target nodes by having the source node bombard the network with Route Request (RREQ) packets, to which intermediary nodes reply with Route Reply (RREP) packets. Data transmission begins as soon as the path is created. Reactive protocols provide the primary advantage of having nodes that only retain data on active routes; network-wide data is not maintained. Ad hoc On-Demand Distance Vector (AODV), Dynamic Source Routing (DSR), and Temporally Ordered Routing Architecture (TORA) are significant examples of reactive routing systems [11].

Hybrid routing protocols: In order to overcome the limitations of each strategy, hybrid routing protocols integrate both the advantages of proactive and reactive approaches. They use reactive methods for communication outside of these zones and retain proactive channels within the network's concentrated sections. This hybrid strategy creates a compromise between low overhead and effective bandwidth use and immediate availability of routes. Hybrid Wireless Mesh Protocol (HWMP), which combines proactive and reactive techniques to optimize routing in wireless mesh networks, and Zone Routing Protocol (ZRP), which uses proactive routing within a defined zone and reactive routing for communication between zones, are two examples of hybrid protocols. Other examples include CEDAR (Core-Extraction Distributed Ad Hoc Routing) and Adaptive Routing Protocol (ARP), which combine proactive and reactive techniques to improve routing effectiveness and adaptability in dynamic [12].

2.1 Denial of Service (DoS) attack in Manets

A Denial of Service (DoS) attack occurs when a single machine sends an overwhelming amount of packets to a server, overloading its bandwidth and resources. By creating a one-to-one attack vector that makes mitigation and prevention attempts more difficult, this kind of active attack is a potent technique for interfering with internet services. A DoS attack often targets the server's capacity to process valid requests and causes service disruptions by employing one machine to create a large volume of traffic [8]. Attacks known as denial of service (DoS) have the potential to seriously affect the availability and performance of mobile ad hoc networks (MANETs). The following are some typical DoS attack kinds that target MANETs:

Jamming Attack: In this technique, an attacker uses radio signals to obstruct or interfere with the channels that network nodes use for communication. The attacker prevents authorized nodes from sending or receiving data by interfering with wireless connectivity.

Flooding Attack: The attacker floods the network with too many routing requests or traffic, which overloads it and uses up all of its resources. Increased latency, decreased throughput, and even network collapse may result from this.

Depletion Attack: This attack makes mobile nodes complete pointless tasks or use excessive amounts of energy in an attempt to deplete their energy reserves. To exhaust the battery life of the targeted nodes, an attacker can, for example, send large quantities of control packets or repeatedly start route finding procedures.

Black Hole Attack: In this type of attack, a malicious node falsely advertises itself as having a valid route to the destination. Once other nodes send data packets to this node, it either drops the packets or forwards them incorrectly, leading to packet loss and communication failures.

Sybil Attack: The attacker creates multiple fake identities or nodes within the network. By introducing these bogus nodes, the attacker can manipulate routing decisions, disrupt network operations, or overwhelm network resources. These DoS attacks exploit the unique characteristics of MANETs, such as their dynamic topology and lack of centralized control, making them challenging to detect and mitigate.

2.2 Importance of Dos Attacks Mitigation in Manets

In a mobile area network (MANNET), mobile nodes function autonomously and can effortlessly create direct communication channels with one another. They can move freely in different directions and at varied speeds inside the infrastructure-less network. The way the ad hoc network functions are distinct, and node cooperation is essential to transmitting communication data from the primary data sources to the mobile nodes that are meant to receive it. These nodes have no restrictions on their movement and are completely battery-powered. Mobile nodes are free to join or exit the dynamic network whenever they choose, and they are not dependent on a single, centralized authority to make choices. This core characteristic of not requiring a fixed infrastructure is essential for enabling communication [13]. However, these same characteristics also leave MANETs open to several kinds of attacks, most notably Denial of Service (DoS) attacks. Attackers might severely reduce the performance and dependability of the network by taking advantage of these flaws to undermine communication, exhaust node resources, or alter network routing. Sensitive data could be compromised in addition to causing communication disruptions in the event of a successful DoS attack. In Mobile Ad Hoc Networks (MANETs), preventing

denial-of-service (DoS) attacks is crucial to preserving network performance and stability. Due to their dynamic and decentralized structure, MANETs are especially susceptible to these kinds of attacks, which can flood the network with excessive and meaningless traffic. Flooding causes higher latency, lower throughput, and more packet loss, all of which lower the standard of network services. Effective mitigation strategies help ensure that the network can handle normal traffic loads efficiently, preserving its overall functionality and preventing performance degradation. Moreover, protecting sensitive data and ensuring service availability are critical aspects of DoS attack mitigation in MANETs [14]. These networks are frequently utilized in high-stakes situations when data integrity and anonymity are crucial, such as military operations, emergency communications, and disaster response. The network can defend against such attacks, keep up service, and preserve the integrity and confidentiality of the sent data by putting strong mitigation measures in place. Different research groups are creating methods for preventing attacks using DoS.

3. Literature review

The authors of [14], examine a number of aspects of Distributed Denial of Service (DDoS) assaults in Mobile Ad-hoc Networks (MANETs) and offer countermeasures to these threats. DDoS attacks, according to them, are dangerous threats that cause a major decline in network performance by continuously flooding the network with fake packets. In order to simulate ad hoc routing protocols and evaluate network performance, the study emphasizes the usefulness of Network Simulator (NS3). Additionally, it looks at detection mechanisms like threshold timestamps and node response times to spot malicious activity. The literature emphasizes how difficult it is to identify and mitigate DDoS attacks and how simulation tools and machine learning can help improve network security. More practical, flexible, and effective solutions are needed to mitigate DDoS attacks in MANETs, especially when it comes to fixing non-legitimate nodes' vulnerabilities, enhancing detection mechanisms, and incorporating cutting-edge technologies like machine learning into already-existing frameworks. Furthermore, by examining node behavior and energy usage, writers in [15] have investigated the application of machine learning techniques, such as Feed Forward Back Propagation Neural Networks (FFBPNN), to improve attack detection. The authors identified and eliminated the malicious nodes from the route by comparing the energy consumption and delay metrics of the nodes to those of suspected malicious nodes. Additionally, the authors of [3] evaluated how three routing protocols in Mobile Ad Hoc Networks (MANETs) Zone Routing Protocol (ZRP), Ad hoc On-Demand Distance Vector (AODV), and Location-Aided Routing (LAR) were affected by Distributed Denial of Service (DDoS) attacks. It was discovered that DDoS attacks severely reduced network performance using the DDoS Attack Simulation Model (DDoSM) with Network Simulator 2 (NS-2). The throughput of AODVs decreased by 67.3%, LARs by 62.8%, and ZRPs by as much as 57.1%. Additionally, AODV saw the most rise in end-to-end delay (98.71%), with increases in LAR and ZRP coming in second and third, respectively, at 97.1% and 96%. Attacks had an effect proportionate to the number of perpetrators, with more than ten attackers having the ability to bring down the network. The study identified a need for comparative analysis of these protocols under different types of attacks, such as blackhole and greyhole attacks, and for the exploration of adaptive mechanisms to enhance protocol resilience. To address several attack vectors in MANETs, future research should concentrate on creating hybrid protocols with integrated security measures. Moreover, an Intrusion Detection System (IDS) designed to identify DoS attacks in MANETs was proposed by [16]. Support Vector Machines (SVMs) are the classification technique used by this intrusion detection system (IDS) to detect and stop DoS attacks. According to experimental findings, the intrusion detection system (IDS) efficiently identifies and neutralizes denial-of-service (DoS) assaults with a high detection rate and limited processing time. The detection rate was also found to be constant, irrespective of node mobility or network size. In addition, the authors in the study [17] presented a novel approach to protect networks against DoS attacks called "monitoring detection and rehabilitation." This method measures actual values to assess the reliability of sensor nodes. By obtaining a 20.87% increase in Packet Delivery Ratio (PDR), the suggested solution outperformed the current "trust enhanced anonymous-on-demand routing protocol." Using blockchain technology, Cochain-SC is a decentralized and secure DDoS collaboration framework that was introduced in 2019 [18] for DDoS mitigation. By incorporating Ethereum's smart contract technology, the authors enhanced cooperation amongst Software-Defined Networking (SDN) domains. DDoS mitigation on both the intradomain and interdomain levels was supported by this blockchain-based method. They measured the unpredictability of the data using sFlow and the Intra Entropy-based Scheme (I-ES). Network traffic irregularities were automatically recognized by the I-

ES inside each domain. Additionally, the Intra-Domain Mitigation (I-DM) mechanism was used to mitigate illegal traffic within the SDN domain. The suggested model's cost-effectiveness, flexibility, efficiency, and security were evaluated in order to determine its efficacy. The authors of [19] evaluate sequence numbers to a threshold value that is dynamically generated using timers in order to find black hole nodes. Additionally, they alert nearby nodes to the rogue node. Nevertheless, this method entails the computational expense of determining dynamic threshold values and could result in false alarms.

4. Proposed dos attack detection and mitigation framework

The mobile ad hoc networks (MANETs) are highly vulnerable to different kinds of security threats due to their dynamic nature of randomness, decentralization, and lack of central authority. The aim of this work is to propose an effective DoS attack detection and mitigation framework for MANETs using machine learning techniques such as SVM, Random Forest, Neural Network, and KNN. By generating synthetic traffic data to represent both normal and DoS attack scenarios, normalizing features, and splitting the data into training and testing sets, the framework enables the training and evaluation of various models. The best-performing model is then used to predict and visualize the traffic types in a simulated MANET environment, helping to identify attack nodes and measure key performance metrics like energy consumption, scalability, resource utilization, and network throughput. The description of attack detection and mitigation framework is given in figure 2 below:

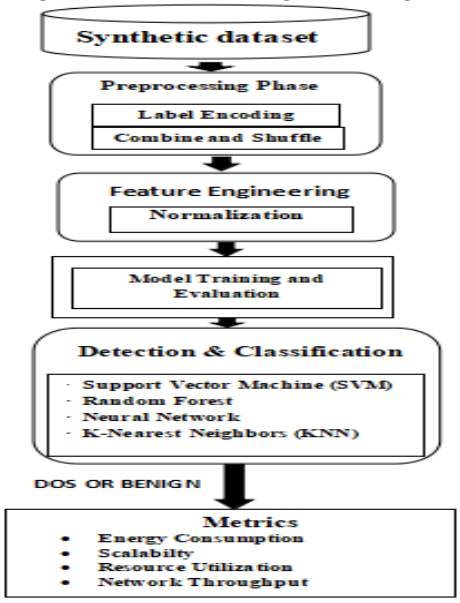


Fig. 2. Attack Detection and Mitigation Framework

This approach provides a systematic way to enhance security and resilience against DoS attacks in MANETs. The machine learning models such as SVM, Random Forest, Neural Network, and KNN are used to detect DoS attacks based on traffic characteristics and network behavior. The features, labels, and attributes are derived from the synthetic traffic data generated during the simulation. Given the predictive power and analytical capabilities of these models, they are well-suited for identifying and managing DoS attacks in Mobile Ad-hoc Networks (MANETs). The performance of the proposed detection and mitigation framework is illustrated by simulating various network conditions and evaluating key metrics such as energy consumption, scalability, and network throughput. The framework, utilizing the best-performing model, will be demonstrated through a simulated MANET environment to showcase its effectiveness in attack detection and mitigation.

4.1 Data generation

We create a synthetic dataset to simulate normal traffic and DoS attack traffic:

Normal Traffic Generation:

Let:

$X_{normal} = [x_1, x_2]$ be the matrix for the normal traffic

$X_1 \sim N(\mu_1 \sigma_1^2)$, where μ1=50 and σ1=15

$X_2 \sim N(\mu_2, \sigma_2^2)$, where μ1=10 and σ1=3

Each data point for normal traffic is sampled from the distributions:

$$X_{normal} = \begin{matrix} x_{11} & x_{12} \\ x_{21} & x_{22} \\ \dots & \dots \end{matrix}$$

Where $X_{i1} \sim N(50, 15^2)$, and $X_{i2} \sim N(10, 3^2)$

DoS Attack Traffic Generation:

Also, let:

. $X_{attack} = [y_1, y_2]$ be the feature matrix for DoS attack traffic

$y_1 \sim N(\mu_3, \sigma_3^2)$, where $\mu_3 = 50, \sigma_3 = 25$

$y_2 \sim N(\mu_4, \sigma_4^2)$, where $\mu_4 = 50, \sigma_4 = 2$

Each data point for attack traffic is sampled from these distributions

$$X_{attack} = \begin{matrix} y_{11} & y_{12} \\ y_{21} & y_{22} \\ \dots & \dots \end{matrix}$$

Where $Y_{i1} \sim N(150, 25^2)$, and $Y_{i2} \sim N(5, 5^2)$

Label Assignment:

Assign labels:

$$\text{Lebel} = \begin{cases} 0 & \text{normal} \\ 1 & \text{attack} \end{cases}$$

4.2 Model training

The objective is to train several machine learning models to classify network traffic as either normal or an attack.

Support Vector Machine (SVM):

SVM aims to find a hyperplane that best separates the two classes:

$$\text{minimum } \frac{1}{2}\|w\|2 \quad \text{subject to } y_i(w.x_i + b) \geq 1, \forall_i$$

Where:

w is the weight vector defining the hyperplane.

b is the bias term.

$y_i \in (-1,1)$ are the class labels.

x_i are the feature vectors.

Random Forest Classifier

A Random Forest classifier is an ensemble of decision trees T1, T2,……,Tm. For each tree:

$$\text{Class} = \text{majority vote } (T_1(x), T_2(x), \ldots . T_m\ (x))$$

Where:

$T_i(x)$ is the prediction of the ith tree for input x

Neural Network (MLP Classifier)

The Neural Network model consists of an input layer, hidden layers, and an output layer:

$$\text{Output} = f(W2 \cdot (f(W1 \cdot X + b1)) + b2)$$

Where:

X is the input feature matrix.

W1,W2 are the weight matrices for the input-to-hidden and hidden-to-output layers.

bi, b2 are the bias vectors.

f is the activation function

K-Nearest Neighbors (KNN)

For KNN, the class of a new point x is determined by the majority class among its k-nearest neighbors:

$$\text{Class}(x) = \text{argmax}_c \sum_{i \in N_k(x)} \| (y_i = c)$$

where:

$N_k(x)$ is the set of the k-nearest neighbors to point x.

$\|$ is the indicator function that returns 1 if $y_i = c$, and 0 otherwise.

4.3 Proposed Algorithm for Dos Attack Detection and Mitigation in Manets

The proposed algorithm for Dos attack detection and mitigation are given below:

Algorithm 1 DoS Attack Detection in MANET

Input: Number of samples *num_samples* = 2000
Output: Detection accuracy and performance metrics for various ML models
Step 1: Generate Synthetic Dataset
Set random seed for reproducibility
Generate normal traffic data:
Feature 1: Normal distribution with mean = 50, std = 15
Feature 2: Normal distribution with mean = 10, std = 3
Generate DoS attack traffic data:
Feature 1: Normal distribution with mean = 150, std = 25
Feature 2: Normal distribution with mean = 5, std = 2
Label normal traffic as 0 and DoS attack traffic as 1
Combine normal and attack traffic data
Shuffle the dataset to ensure randomness
Step 2: Feature Engineering
Normalize the features using *StandardScaler*
Step 3: Split Dataset
Split the dataset into training (70%) and testing (30%) sets
Step 4: Train and Evaluate Models
Initialize the following models:
- Support Vector Machine (SVM)
- Random Forest
- Neural Network
- k-Nearest Neighbors (KNN)
for each model **do**
Train the model on the training set
Predict labels on the testing set
Calculate accuracy, confusion matrix, and classification report
Store evaluation results
end for

Algorithm 2 DoS Attack Mitigation in MANET

Input: Trained ML model (e.g., Neural Network), MANET network simulation
Output: Mitigation strategy performance in a simulated MANET
Step 1: Visualize Network Traffic
Simulate a MANET with nodes
Generate node positions using a layout algorithm
for each node **do**
Predict traffic type (normal or DoS attack) using the best-performing model (e.g., Neural Network)
Assign color based on prediction:
- Red for DoS attack traffic
- Blue for normal traffic
end for
Draw the network graph with nodes and edges
Display the network graph
Step 2: Simulate and Analyze Performance Metrics
Simulate performance metrics for each model:
- Energy Consumption (arbitrary units)
- Scalability (arbitrary units)
- Resource Utilization (arbitrary units)
- Network Throughput (Mbps)
Plot bar graphs for each performance metric

5. Experimental design

The main aim is to evaluate and compare the effectiveness of various machine learning models in detecting and mitigating DoS attacks in MANETs. The proposed DoS attack detection and mitigation model was implemented and validated using Python, along with Scikit-learn libraries, within the Google Colab environment. All evaluations were conducted on a Dell machine equipped with an Intel(R) Core(TM) i5-8350U CPU @ 1.70GHz to 1.90 GHz. The framework involved generating synthetic datasets to simulate normal and DoS attack traffic, training various machine learning models (SVM, Random Forest, Neural Network, KNN), and

evaluating their performance. The best-performing model was used to predict traffic types in a simulated MANET environment, with performance metrics such as energy consumption, scalability, resource utilization, and network throughput being analyzed to validate the effectiveness of the detection and mitigation strategies.

5.1 Data collection and preprocessing

Data Preparation**:** Synthetic datasets are generated to represent normal traffic and DoS attack traffic. The data includes features that are crucial for detecting anomalies, such as traffic patterns and attributes. These datasets are labeled accordingly and combined to create a comprehensive dataset for model training. Feature Engineering: The features from the dataset are normalized using StandardScaler to ensure consistency and effectiveness in model training.

5.2 Model training and evaluation

Different machine learning models, including SVM, Random Forest, Neural Network, and KNN, are trained on the prepared dataset. Each model learns to distinguish between normal and DoS attack traffic based on the features provided. The performance of each model is then evaluated using accuracy, confusion matrix, and classification reports. Training machine learning models:

Training SVM

SVM Accuracy: 0.9908

Table 1: Confusion Matrix for SVM:

		Actual Class	
		Positive	Negative
Predicted Class	Positive	601	1
	Negative	10	588

Training Random Forest

Random Forest Accuracy: 0.9917

Confusion Matrix for Random Forest:

		Actual Class	
		Positive	Negative
Predicted Class	Positive	601	1
	Negative	9	589

Training Neural Network

Neural Network Accuracy: 0.9925

Confusion Matrix for Neural Network:

		Actual Class	
		Positive	Negative
Predicted Class	Positive	601	1
	Negative	8	590

Training KNN

KNN Accuracy: 0.9900

Confusion Matrix for KNN:

		Actual Class	
		Positive	Negative
Predicted Class	Positive	600	2
	Negative	10	588

5.3 Network simulation and visualization

A simulated MANET environment is set up where the best-performing model is used to predict the type of traffic (normal or attack) for each node. The network is visualized with nodes colored based on these predictions to illustrate the impact of the DoS attack and the effectiveness of the detection model as shown in the figure below:

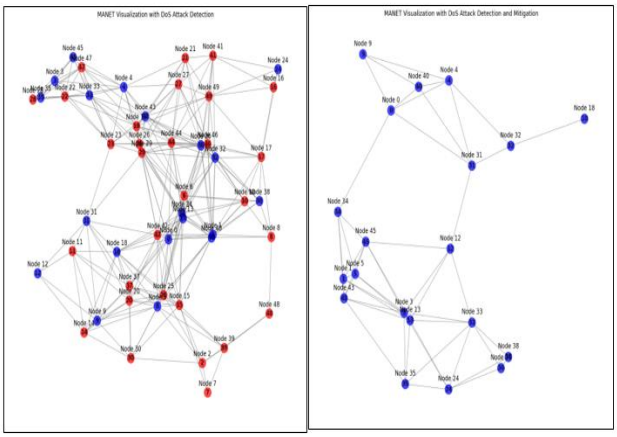


Fig. 3. MANET visualization with DoS Attack Detection and Mitigation Framework

6. Results and Analysis

6.1. Accuracy Comparison

Having an accuracy of 99.25%, Neural Network is the most accurate, followed by Random Forest (99.17%), SVM (99.08%), and KNN (99.00%). All four of the models are very effective at detecting and mitigating Denial of Service (DoS) assaults in a Mobile Ad-Hoc Network (MANET) environment, as seen by the little discrepancies in accuracy between them.

6.2 Classification Report Analysis

With 1.00 precision for both classifications (attack and benign traffic) across all models, there are very few false positives. For all models, the recall for class 0.0 (benign traffic) is 1.00, indicating that nearly all benign traffic is correctly identified. Recall is a bit lower for class 1.0 (attack traffic) for SVM and Random Forest, and 0.99 for Neural Network. The models demonstrate a consistent F1-score of 0.99, which is a measure of both precision and recall, indicating that their performance is balanced.

6.3 Confusion Matrix Analysis

All the four models have very similar confusion matrices, which show the following:

True Positives (TP) (correctly classified benign traffic) are around 600 out of 601 for all models. True

Negatives (TN) (correctly classified DoS attack traffic) are slightly varied but high in all models (588 for SVM, 589 for Random Forest, and 590 for Neural Network). False Positives (FP) (benign traffic incorrectly classified as attacks) are consistently 1 for almost all of the models except KNN which obtain a value of 2. False Negatives (FN) (attack traffic incorrectly classified as benign) are lowest for the Neural Network (8) and slightly higher for SVM and KNN (10) and also Random Forest (9).

6.4 Performance Metrics for Evaluation

The proposed framework simulates and plots performance metrics such as energy consumption, scalability, resource utilization, and network throughput. This helps to assess the overall impact of the DoS attack and the efficiency of the mitigation strategies employed.

Energy Consumption of the models. Figure 4 display the energy consumption of different machine learning models, The Neural Network model shows the highest energy consumption, followed by Random Forest and SVM, with KNN having the lowest. High energy consumption in Neural Networks can be due to complex computations required during the training and prediction processes. KNN's lower energy consumption makes it a more energy-efficient option, although it might sacrifice some detection accuracy.

6.4.1 Scalability

Scalability is crucial for MANET environments where the number of nodes and traffic can vary significantly. Thus, Neural Networks and Random Forest are preferred for their scalability in handling large-scale network environments. The Neural Network and Random Forest models exhibit the best scalability, closely followed by SVM. KNN shows the lowest scalability, likely due to its distance-based classification approach, which can become computationally expensive with larger datasets.

6.4.2 Resource Utilization

Higher resource utilization indicates that these models require more computational power, which could be a consideration when deploying in resource-constrained environments typical of MANETs. Random Forest and Neural Network models have the highest resource utilization, slightly above SVM, with KNN showing the lowest.

6.4.3 Network Throughput

Higher throughput in the Neural Network model suggests it effectively handles larger volumes of network traffic, identifying and mitigating DoS attacks while maintaining network performance. The lower throughput observed in the KNN model indicates it might not be as effective in high-throughput scenarios, potentially leading to network congestion or slower attack detection. Neural Network demonstrates the highest network throughput, followed by Random Forest, SVM, and lastly KNN.

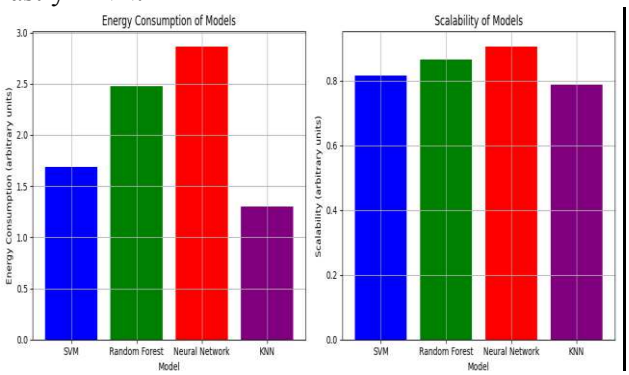


Fig. 4. Energy Consumption and Scalability of the Model

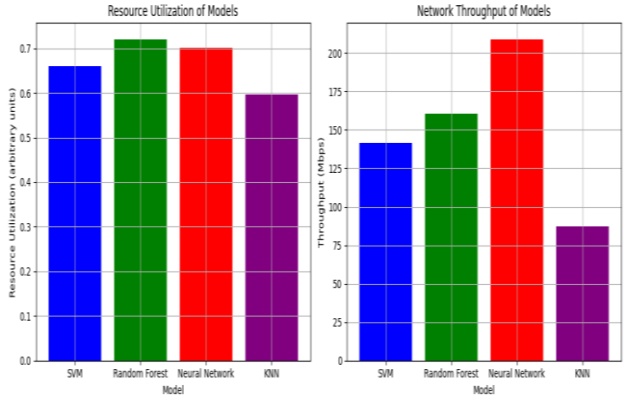


Fig. 5. Resource Utilization and Network Throughput of the Model

7. Discussion

Neural networks are the best choice in situations when high accuracy and the capacity to handle intricate patterns with superior recall and precision are required. Performance-critical applications can benefit from their use because they have the highest network throughput and energy consumption among them. An excellent substitute is Random Forest, which provides great accuracy with minimal false positives and negatives. It is a well-rounded option in most situations due to its excellent resource efficiency, good scalability, and modest energy consumption. SVM works well for basic tasks where managing complicated data is not as important as speed and simplicity. Its resource utilization, scalability, and energy consumption are all moderate, but its false negative rate is a little higher. KNN works well in simpler circumstances and with smaller datasets, it performs poorly in terms of network throughput, resource usage, and scalability. It performs well in contexts with limited resources and manageable data volumes.

The Neural Network model, while having higher energy consumption and resource utilization, offers

the best throughput and scalability. This suggests that Neural Networks are more suited for scenarios where performance and scalability are prioritized, even if it means using more energy and computational resources. Random Forest also shows a balanced performance with good scalability and resource utilization, making it a strong alternative for environments where energy efficiency is a secondary concern. SVM offers moderate performance across all metrics, making it an option when balanced performance is required. KNN is the most energy-efficient and uses the least resources but falls behind in scalability and throughput, making it less suitable for larger or high-traffic MANET scenarios. These insights help in selecting the appropriate machine learning model for DoS attack detection and mitigation based on the specific requirements of the MANET environment, balancing the trade-offs between energy consumption, scalability, resource utilization, and throughput.

7. Conclusion and Future Research Directions

The proposed framework for detecting and mitigating DoS attacks in Mobile Ad-hoc Networks (MANETs) leverages machine learning models like SVM, Random Forest, Neural Networks, and KNN to enhance network security. By generating synthetic datasets that simulate both normal and malicious traffic, the framework trains these models to distinguish between benign and attack traffic effectively. The performance evaluation reveals that Neural Networks provide the highest throughput and scalability, making them suitable for high-traffic environments, while models like KNN offer better energy efficiency, which is crucial in resource-constrained scenarios. the proposed framework can significantly contribute to the development of more secure and resilient MANETs, supporting critical applications that rely on reliable and protected network communication.

Future research directions should include exploring the use of advanced deep learning models such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks. Additionally, adaptive mechanisms that adjust to changing attack patterns and network conditions could be developed, enhancing the system's ability to respond to threats in real time. Another significant area for future research is the development of hybrid detection models that combine multiple machine learning techniques to leverage their respective strengths. For instance, integrating SVM, Random Forest, Neural Network, and KNN into ensemble approaches could improve detection rates and reduce false positives. Extending the framework to detect a wider range of attacks would also make the system more comprehensive and robust against various types of security threats. Lastly, optimizing the energy efficiency of the detection models is crucial, especially in the context of MANETs where nodes often have limited power resources. Future work could focus on developing lightweight algorithms that reduce computational overhead while maintaining high detection accuracy. Testing the framework in real-world scenarios with actual network traffic data and resource-constrained environments, such as IoT-integrated MANETs, will provide valuable insights into its practical applicability.

References:

[1] R. R. L. a. C. R. K. Reddy, "Node activity based trust and reputation estimation approach for secure and QoS," International Journal of Electrical & Computer Engineering (IJECE), no. 6, , vol. vol 9, pp. pp. 5340-5350, 2019.

[2] P. C. a. T. Ranganayaki, "A Study on Manet: Applications, Challenges and Issues," International Journal of Engineering Research & Technology (IJERT) ISSN: 2278-0181, no. Special Issue - 2020, 2020.

[3] R. A. A. a. R. A. Maha Abdelhaq, "The resistance of routing protocols against DDOS attack in MANET," International Journal of Electrical and Computer Engineering (IJECE), Vols. Vol. 10, No. 5, p. pp. 4844~4852, 2020.

[4] P. Nayak and B. Vathasavai, "Impact of random mobility models for reactive routing protocols over MANET," International Journal of Simulation--Systems, Science & Technology, , Vols. vol. 17,, pp. pp. 13.1-13. 9,.

[5] M. Y. N. Uparosiya, ""“Survey on MANET: Routing protocols, advantages, problems and security,”," International Journal of Innovative Computer Science & Engineering,, vol. vol. 1, pp. no. 2, pp. 12-17.

[6] e. a. A. D. Basheer, "SIEM-based detection and mitigation of IoT-botnet DDoS attacks,," International Journal of Electrical &

Computer Engineering, , vol. vol. 10, pp. pp. 2182-2191,, 2020.

[7] I. S. H. K. J. E. a. D. G. Fatemeh Safari, "The Diverse Technology of MANETs: A Survey of Applications and Challenges," International Journal of Future Computer and Communication,, Vols. Vol. 12, No. 2, , p. 12, 2023.

[8] M. C. a. E. M. S. Er. Inakshi Garg, "DOS Attack Mitigation In MANET," International Journal of Computer Science Engineering (IJCSE), Vols. ISSN : 2319-7323, p. 05.

[9] .. M. I. O. S. O. Baraa T. Sharef, "A survey on position-based routing protocols for Flying Ad hoc Networks (FANETs)," Vehicular Communications, pp. Pages 29-56, 2017.

[10] A. M. F. A.-R. S. Saeed N. H., "MANET routing protocols taxonomy," in International Conference on Future Communication Networks,.

[11] N. Beijar, "Zone routing protocol (ZRP).," in Networking Laboratory, Helsinki University of Technology, 9, 1-12., Finland,, 2019.

[12] A. T. R. Y. a. D. A. K. Tawseef, "MANET Routing Protocols, Attacks and Mitigation Techniques: A Review," International Journal of Mechanical Engineering, Vols. ISSN: 0974-5823 Vol. 7 No. 2 , p. 11, 2022.

[13] T. S. H. E. S. M. A. K. Mohamad, "AN INTRUSION DETECTION MECHANISM FOR MANETS BASED ON DEEP LEARNING ARTIFICIAL NEURAL NETWORKS (ANNS)," International Journal of Computer Networks & Communications (IJCNC), vol. Vol.15, no. No.1, p. 15, 2023.

[14] R. K. K. V. R. G. Zalte S. S., "Mitigation of DDoS Attack in MANET," International Journal of Engineering and Advanced Technology (IJEAT),ISSN: 2249-8958 (Online), Vols. Volume-9 , no. Issue-6, , 2020.

[15] B. C. R. K. Jasmine, "DDoS Attack Detection and Prevention using Aodv Routing Mechanism and Ffbp Neural Network in a Manet",," International Journal of Recent Technology and Engineering (IJRTE) ISSN: 2277-3878, Vols. Volume-8 , no. Issue-2,, pp. pp- 4136-4142, 2019.

[16] E. A. a. A. Shams, "A novel support vector machine based intrusion detection system for mobile ad hoc networks," Wireless Networks,, Vols. vol. 24, no. 5, , pp. pp. 1821-1829, 2020.

[17] A. J. H. a. A. L. Alsumayt et al, "Evaluation of Detection Method to Mitigate DoS Attacks in MANETs," in 1st International Conference on Computer Applications & InformationSecurity (ICCAIS),.

[18] E. H. A. S. H. a. L. K. Z. Abou, "Cochain-SC: An Intra- and Inter-Domain DDoS MitigationScheme Based on Blockchain Using SDN and Smart Contract," in In: Proceedings of the IEEE Access,, vol. 7, pp. 98893-98907, 2019.

[19] a. P. B. S. Payal N. Raj, "DPRAODV: A DYANAMIC LEARNING SYSTEM AGAINST BLACKHOLE ATTACK INAODV BASED MANET," International Journal of Computer Science Issues,, Vols. Vol. 2,, pp. PP 54-59, 2019.

[20] C. R. K. Jasmine Batra, "DDoS Attack Detection and Prevention using Aodv Routing Mechanism and Ffbp Neural Networkin a Manet," International Journal of Recent Technology and Engineering (IJRTE) ISSN: 2277-3878,, Vols. Volume-8, no. Issue-2,, pp. pp- 4136-4142 .

A Behaviour-Informed Approach to Mitigate Data Poisoning Attacks for Machine Learning

BIRMINGHAM CITY University

By

Anum Paracha

Supervised by:

Prof. Junaid Arshad, Dr Mohamed Ben Farah and Dr Khalid Ismail

A thesis submitted in partial fulfillment of the requirements of Birmingham City University for the degree of Doctor of Philosophy

Faculty of Computing, Engineering and Build Environment

October 2025

Abstract

The widespread deployment of machine learning (ML) across critical domains such as healthcare, transport, and smart grids has increased dependence on automated decision-making, exposing these systems to potential exploitation by adversaries. ML models are highly susceptible to adversarial perturbations, including intentional input manipulations that can alter model performance. Among these, data poisoning attacks are particularly evolving as there are enormous ways to corrupt training data to distort underlying behaviour and undermine system reliability. Furthermore, the risks of data poisoning attacks increase with the dependence on public datasets.

Data poisoning attacks have been extensively explored in the context of deep learning (DL) models; however, traditional ML, especially multiclass models, remains underexplored in assessing vulnerabilities and defences. Consequently, most mitigation strategies are limited to DL and are designed for specific algorithms or attack models. For example, adversarial training is effective for gradient-based models but less effective for traditional models as they do not rely on gradient optimisation. These limitations enable adversaries to exploit defences through new attack vectors, thereby complicating the security of ML systems. Moreover, limited defences for traditional ML keep these models vulnerable to such attacks.

This thesis analysed the security of traditional ML under data poisoning attacks implemented with limited adversarial capabilities and knowledge and analysed limitations of existing defences, subsequently introducing an enhanced mitigation strategy. The manipulations to training datasets are analysed through comprehensive deep behavioural analysis, identifying the change in model characteristics, the impact of increasing poisoning levels and their relationships. Furthermore, a new multiclass poisoning attack is proposed by exploiting a common outlier characteristic of ML models, called Outlier-Oriented Poisoning (OOP) attack. This attack leveraged the examination of multiclass ML under limited adversarial capabilities. These studies revealed how data poisoning alters the learning dynamics of the model and its characteristics. Insights from this analysis informed the development of SecureLearn, a behaviour-informed, attack-agnostic mitigation solution combining enhanced data sanitisation with a novel feature-oriented adversarial training (FORT) approach to improve model resilience against data poisoning.

This thesis examined SecureLearn by proposing a 3D evaluation matrix. Experimental results of this study demonstrated that SecureLearn effectively enhanced the security and robustness of multiclass ML across random forest (RF), decision tree (DT), gaussian naive bayes (GNB) and neural networks, confirming its generalisability beyond algorithm-specific defences. SecureLearn consistently maintained accuracy above 90%, recall and f1-score above 75%, and reduced the false discovery rate to 0.06 across all evaluated models against three distinct poisoning attacks. For RF models, Secure-

Learn maintained a minimum recall of 84.19% and f1-score of 81.54% at 20% poisoning level with the OOP attack. For DT models, the minimum recall is 78.20% and f1-score is 7.80%. However, it is observed that SecureLearn is less effective in enhancing the resilience of GNB models trained with the MNIST dataset. GNB models trained with the MNIST dataset, SecureLearn maintained the recall at a minimum of 57% with f1-score of 56%. In the context of neural networks, SecureLearn achieved at least 97% recall and f1-score against all selected poisoning attacks. The adversarial robustness of models, trained with SecureLearn, improved with an average accuracy trade-off of only 3%.

Declaration of Authorship

I, Anum Paracha, declare that this thesis titled, "A Behaviour-Informed Approach to Mitigate Poisoning Attacks for Machine Learning" and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Anum Paracha

October 2025

Publications

- **Paracha, A.**, Arshad, J., Farah, M. and Ismail, K. "Machine learning security and privacy: a review of threats and countermeasures". EURASIP Journal on Information Security (JINS), 2024.
- **Paracha, A.** and Arshad, J. "A bibliometric study toward quantitative research assessment of security of machine learning". Information Discovery and Delivery (IDD), 2024.
- **Paracha, A.**, Arshad, J., Farah, M. and Ismail, K. "Deep behavioral analysis of machine learning algorithms against data poisoning". International Journal of Information Security, 2025.
- **Paracha, A.**, Arshad, J., Farah, M. and Ismail, K. "Outlier-oriented poisoning attack: a grey-box approach to disturb decision boundaries by perturbing outliers in multiclass learning". International Journal of Information Security, 2025.
- **Paracha, A.**, Arshad, J., Farah, M. and Ismail, K. "Exploring Data Poisoning Attacks Against Adversarially Trained Skin Cancer Diagnostics". 2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC), 2024.
- **Paracha, A.**, Baiyekusi, O., Arshad, J., Mi, D., Lu, C., Zhang, Y., Wang, F., Chen, L., Zhang, J. "Exploring Multiclass Data Poisoning within an Industrial 5G Private Network". 2025 IEEE 101st Vehicular Technology Conference (VTC2025-Spring), 2025.
- **Paracha, A.**, Arshad, J., Farah, M. and Ismail, K. "SecureLearn - An Attack-agnostic Defense for Multiclass Machine Learning Against Data Poisoning Attacks". Submitted to IEEE Transactions on Information Forensics and Security, 2025.

Acknowledgments

Firstly, I am thankful to the Almighty for the blessings and ability to complete this PhD study.

Secondly, I would like to thank my Director of Studies, Prof. Junaid Arshad, for his endless support, invaluable discussions and guidance provided during this study. I am also greatly thankful to my second supervisors, Dr Mohamed Ben Farah and Dr Khalid Ismail, for their continuous support, discussions and guidance throughout this journey.

I would like to thank all fellows, friends and colleagues for the encouragement and every help provided during this time.

I am greatly thankful to my family, especially to my dad, Mr. Atique Ahmed Paracha and mum, Mrs. Amna Sadia Paracha, for all their love, endless support and prayers during this complete journey. Without their support, I would not be able to pursue this degree.

Special thanks to the Department of Computing, Engineering and Built Environment and Doctoral Research College (DRC) for their support and guidance.

Contents

1 Chapter 1. Introduction 16

1.1 Machine Learning Security . . . 17

1.2 Problem Background . . . 19

1.3 Problem Statement . . . 19

1.4 Research Aim and Objectives . . . 21

1.5 Research Questions . . . 21

1.6 Thesis Contributions . . . 22

1.7 Thesis Organisation . . . 23

2 Chapter 2. Machine Learning Security - Background and State-of-the-Art Attacks and Countermeasures 25

2.1 Introduction . . . 25

2.2 Background . . . 26

2.2.1 Machine Learning . . . 26

2.2.2 Supervised Machine Learning . . . 27

2.2.2.1 Binary Classification . . . 28

2.2.2.2 Multiclass Classification . . . 28

2.2.3 Support Vector Machines . . . 28

2.2.4 Decision Tree . . . 30

2.2.5 Random Forest . . . 30

2.2.6 K-Nearest Neighbor . . . 31

2.2.7 Gaussian Naive Bayes . . . 32

2.2.8 Neural Networks . . . 33

2.2.9 Adversarial Machine Learning . . . 35

2.2.9.1 Data Poisoning Attack . . . 36

2.2.9.2 Evasion Attack . . . 37

2.2.9.3 Model Inversion Attack . . . 37

2.2.9.4 Membership Inference Attack . . . 38

2.3 State-of-the-Art Analysis . . . 39

2.3.1 Methodology . . . 39

2.3.2 Literature Review . . . 41

2.3.2.1 Data Poisoning Attacks . . . 41

2.3.2.2 Evasion Attacks . . . 44

2.3.2.3 Model Inversion Attacks . . . 44
2.3.2.4 Membership Inference Attacks . . . 47
2.3.2.5 Mitigating Data Poisoning Attacks . . . 49
2.3.2.6 Mitigating Evasion Attacks . . . 51
2.3.2.7 Mitigating Model Inversion and Membership Inference Attacks . . . 52
2.4 Research Gaps in the State-of-the-Art . . . 53
2.5 Summary . . . 54

3 Chapter 3. Investigating Machine Learning Behaviour Against Data Poisoning Attacks 55

3.1 Introduction . . . 55
3.2 Related Work . . . 56
3.3 Behavioural Analysis . . . 58
3.4 Fundamentals of Training Datasets . . . 59
3.4.1 Dataset Size and Distribution . . . 60
3.4.2 Feature Correlation . . . 60
3.4.3 Dataset Outliers . . . 61
3.5 Approach to Behavioural Analysis . . . 61
3.5.1 Attack Method . . . 62
3.5.2 Experimental Datasets . . . 64
3.6 Experimentation Results and Analysis . . . 65
3.6.1 Optimal Poisoning Level . . . 66
3.6.2 Vulnerable Parameters . . . 71
3.6.3 Imbalanced Data Distribution . . . 74
3.7 Implications of Behavioural Analysis . . . 75
3.7.1 Improving Mitigations . . . 75
3.8 Discussion and Limitations . . . 76
3.9 Summary . . . 78

4 Chapter 4. Further Exploring Novel Outlier-Oriented Poisoning (OOP) Attack in Multiclass Machine Learning 79

4.1 Introduction . . . 79
4.2 Related Work . . . 80
4.3 Threat Model . . . 82
4.3.1 Attack Surface and Goals . . . 82
4.3.2 Attacker Knowledge . . . 83

4.3.3 Attacker Capability . . . 83
4.3.4 Outlier-Oriented Poisoning Attack . . . 83
4.3.5 OOP Attack Against Adversarially Trained Skin Cancer Diagnostics . . . 84
4.3.6 OOP Attack Against Industrial 5G Networks . . . 85
4.4 Case Study Selection Rationale . . . 85
4.5 Approach to OOP Attack . . . 86
4.5.1 Attack Method . . . 86
4.5.2 Experimental Datasets . . . 86
4.5.3 Attack Settings . . . 89
4.6 Experimentation Results and Analysis . . . 92
4.6.1 Effects on Multiclass Classification . . . 92
4.6.2 Effects of Poisoning Rates . . . 94
4.6.3 Model Sensitivity to Poison . . . 96
4.7 Case Study 1: OOP Attack Against Adversarially Trained Skin Cancer Diagnostics . . . 101
4.7.1 Attack Model . . . 101
4.7.2 Experimentation Results and Analysis . . . 102
4.8 Case Study 2: OOP Attack Against Industrial 5G Private Network . . . 108
4.8.1 Attack Model . . . 108
4.8.2 Experimentation Results and Analysis . . . 112
4.9 Discussion and Limitations . . . 114
4.10 Summary . . . 115

5 Chapter 5. SecureLearn: Improvised Machine Learning Defence 117
5.1 Introduction . . . 117
5.2 Related Work . . . 118
5.2.1 Existing Multiclass Poisoning . . . 118
5.2.2 Limitations of Existing Defences . . . 119
5.3 SecureLearn Overview . . . 120
5.3.1 Improvised Data Sanitisation . . . 121
5.3.2 Feature-Oriented Adversarial Training . . . 122
5.4 Extended Threat Model . . . 123
5.4.1 Attack Goals . . . 123
5.4.2 Attacker Knowledge . . . 123
5.4.3 Attacker Capability . . . 124
5.4.4 Attack Strategy . . . 124
5.4.5 Defender Capability . . . 124

5.5 Experimental Datasets . . . 124
5.6 Crafting Poisoned Samples . . . 125
5.6.1 Random Label Poisoning Attack . . . 125
5.6.2 Subpopulation Poisoning Attack . . . 126
5.6.3 Outlier-Oriented Poisoning Attack . . . 126
5.7 3D Evaluation Matrix . . . 127
5.7.1 Dimensional Space 1 . . . 127
5.7.2 Dimensional Space 2 . . . 127
5.7.3 Dimensional Space 3 . . . 128
5.8 Evaluation Metrics . . . 128
5.9 Experimentation and Analysis . . . 130
5.9.1 Detection and Correction Boundaries . . . 130
5.9.2 SecureLearn vs Existing Sanitisation Techniques . . . 130
5.9.3 Analysing Feature-Oriented Adversarial Training . . . 134
5.9.4 Increasing Poisoning Rate . . . 135
5.10 Discussion and Limitations . . . 136
5.11 Summary . . . 137

6 Chapter 6. Conclusion and Future Work 138

6.1 Evaluate Research Objectives . . . 138
6.2 Contribution to Defence . . . 139
6.3 Limitations . . . 140
6.4 Future Work . . . 141

List of Figures

1.1 Fundamental aspects of secure and trustworthy machine learning 18
1.2 The conceptual alignment of research objectives, research questions and thesis contributions 23
2.1 The development process of a supervised machine learning model, illustrating all phases of the development, starting from data gathering to model deployment 27
2.2 A trained support vector machines model developed with a dataset comprises three classes. The support vectors of this model are highlighted in red, which influenced the decision boundaries 29
2.3 A trained decision tree model developed with a dataset comprises three classes. The decision surface of the decision tree is illustrated between three classes of the dataset with some overfitting between class 2 and class 3 31
2.4 A trained random forest model developed with a dataset of three classes. The decision surface between three classes is illustrated 32
2.5 A trained k-nearest neighbors model developed with a dataset of three classes is given, showing the decision surface of each class 33
2.6 A trained gaussian naive bayes model developed with a dataset of three classes showing the decision boundaries of each class with some overlapping between class 2 and class 3 34
2.7 A trained multilayer perceptron model developed with a dataset of three classes showing the decision surface of the classes 34
2.8 Taxonomy of adversarial machine learning attack types 36
2.9 Poisoning attack surface in machine learning model development process - Poisoning machine learning attack 37
2.10 Evasion attack surface in machine learning model development process - Evasion machine learning attack 38
2.11 Model inversion attack surface in machine learning model development process - Model inversion machine learning attack 38
2.12 Membership inference attack surface in machine learning model development process - Membership inference machine learning attack 39
2.13 Research methodology of the state-of-the-art analysis 39
2.14 State-of-the-art - AML attack types 41
2.15 State-of-the-art of mitigation techniques - AML attack types 50
3.1 Architectural overview of the attack model, developed for conducting behavioural analysis 62

3.2 Gaussian mixture model visualisation of features relationship in dataset with principal component analysis reduction. Legend shown in (a) applies to all 65
3.3 Performance analysis of support vector machines with consistent poisoning 66
3.4 Performance analysis of random forest with consistent poisoning 66
3.5 Performance analysis of gaussian naive bayes with consistent poisoning 67
3.6 Performance analysis of k-nearest neighbors with consistent poisoning 67
3.7 Performance analysis of decision tree with consistent poisoning 68
3.8 Performance analysis of perceptron with consistent poisoning 68
3.9 Change in decision boundary of k-nearest neighbors with 15% dataset poisoning . . . 69
3.10 Variance analysis of trained models with consistent poisoning. Legend shown in (d) applies to all 71
3.11 ROC curve of perceptron with consistent poisoning 73
3.12 ROC curve of gaussian naive bayes with consistent poisoning 74
3.13 Analysing change in dataset distribution D_{dis} with label poisoning using *kernel density estimation* function at an incremental poison levels ΔL=(0%, 10%, 15%, 25%). Legend shown in (d) applies to all 75
4.1 The architectural overview of Outlier-oriented poisoning attack 84
4.2 Gaussian mixture model visualisation of features relationship in the dataset with PCA reduction 89
4.3 Performance analysis of support vector machines with consistent poisoning 95
4.4 Performance analysis of random forest with consistent poisoning 96
4.5 Performance analysis of gaussian naive bayes with consistent poisoning 96
4.6 Performance analysis of k-nearest neighbors with consistent poisoning 97
4.7 Performance analysis of decision tree with consistent poisoning 97
4.8 Performance analysis of multilayer perceptron with consistent poisoning 98
4.9 Data distribution with OOP attack 100
4.10 Overview of the attack model - Analysing efficacy of outlier data poisoning attack against adversarially trained models for skin cancer diagnosis 102
4.11 Model accuracy - Decrease in model's performance with data poisoning attack 104
4.12 Model's false discovery rate - Increase in the false diagnosis rate of the models when trained with the poisoned dataset 105
4.13 Model variance - Sensitivity of the models against data poisoning attack 106
4.14 Random forest poisoned sample 1 - Analysing change in confidence score with poisoned training 106

4.15 Support vector machines poisoned sample 1 - Analysing change in confidence score with poisoned training . . . 107
4.16 Multilayer perceptron poisoned sample 1 - Analysing change in confidence score with poisoned training . . . 107
4.17 Material recycling facility scenario - A network of conveyor belts . . . 109
4.18 Mean data-rate poisoning performance results for multilayer perceptron showing performance across multiple nodes and contention window stages . . . 111
4.19 Mean delay poisoning performance results for multilayer perceptron showing performance across multiple nodes and contention window stages . . . 111
4.20 Mean data-rate poisoning performance results for other models showing performance across multiple nodes and contention window stages . . . 112
4.21 Mean delay poisoning performance results for other models showing performance across multiple nodes and contention window stages . . . 112
4.22 Decrease in model accuracy with data poisoning attack . . . 113
4.23 Increase in model false discovery rate with data poisoning attack . . . 113
5.1 Architectural overview of SecureLearn illustrating a two-layer approach to secure the training pipeline of a machine learning model . . . 121
5.2 3D evaluation matrix to evaluate SecureLearn three different aspects. The first dimension between data sanitisation and data poisoning attacks evaluates SecureLearn against selected data poisoning attacks and compares it with existing mitigations. The second dimension between data poisoning attacks and adversarial training evaluates FORT adversarial training of SecureLearn against selected data poisoning attacks. The third dimension between adversarial training and data sanitisation evaluates the overall performance of SecureLearn against selected data poisoning attacks . . . 128
5.3 Impact of outlier-oriented poisoning attack on accuracy of models at various poisoning levels. The first row illustrates all models trained with the IRIS dataset, the models in the second row are trained with the MNIST dataset, and in the third row, the models are trained with the USPS dataset . . . 133
5.4 Impact of subpopulation attack on the accuracy of models at various poisoning levels. The first row illustrates models trained with the IRIS dataset, the models in the second row are trained with the MNIST dataset, and the models in the third row are trained with the USPS dataset . . . 133

5.5 Impact of random label poisoning attack on accuracy of models at various poisoning levels. The first row illustrates models trained with the IRIS dataset, the models in the second row are trained with the MNIST dataset, and the models in the third row are trained with the USPS dataset 134

List of Tables

2.1 Analysis of poisoning attacks . . . 42
2.2 Analysis of evasion attacks . . . 45
2.3 Analysis of model inversion attacks . . . 46
2.4 Analysis of membership inference attacks . . . 48
3.1 Comparative analysis with existing similar studies that provides an empirical analysis of poisoning attacks on machine learning along with their objectives/focus to conduct empirical analysis . . . 57
3.2 Performance analysis with existing similar studies analysing the impact of poisoning attacks on machine learning models . . . 57
3.3 Detailed synopsis of relevant existing defence solutions against data poisoning attacks along with the parameter targeted to develop mitigation strategy and algorithms on which the respected proposed solutions are experimented . . . 58
3.4 Dataset description used to perform data poisoning on machine learning algorithms . . 64
3.5 Features importance score - Decision tree where $\Delta L = (0\%, 10\%, 15\%)$ . . . 69
3.6 Features importance score - Support vector machines where $\Delta L = (0\%, 10\%, 15\%)$ . . 69
3.7 Analysing class probabilities of gaussian naive bayes with poisoned dataset . . . 70
3.8 Adversarial success rate with BotDroid dataset . . . 70
3.9 Adversarial success rate with UNSW-NB15 dataset . . . 71
3.10 Adversarial success rate with CTU-13 dataset . . . 72
3.11 Adversarial success rate with CIC-IDS-2017 dataset . . . 72
3.12 Analysing k-neighbors affecting k-nearest neighbors accuracy with $\Delta L = (0, 10, 15, 25)\%$ 72
3.13 Analysing support vector machines margin score for different datasets with $\Delta L = (0, 10, 15)\%$ . . . 73
3.14 Analysing one-to-one relation between poison and various parameters of ML algorithms 73
4.1 Analysing existing studies against the behavioural analysis with outlier-oriented poisoning attack . . . 81
4.2 Configurations of the models . . . 85
4.3 Dataset description used for experimentation with outlier-oriented poisoning attack . . 89
4.4 Statistical correlation of features in the dataset . . . 89
4.5 Analysing k-neighbors affecting k-nearest neighbors accuracy with $\Delta L = (0, 10, 15, 25)\%$ 93
4.6 Analysing class probabilities of gaussian naive bayes with the poisoned dataset . . . 93
4.7 Features importance score - Decision tree where $\Delta L = (0\%, 10\%, 15\%)$ . . . 94
4.8 Features importance score - Support vector machines where $\Delta L = (0\%, 10\%, 15\%)$ . . 94

4.9 Analysing support vector machines margin score for different datasets with $\Delta L = (0, 10, 15)\%$ . . . 94
4.10 Features importance score - Random forest where $\Delta L = (0\%, 10\%, 15\%)$ . . . 95
4.11 Model variance at different poisoning levels . . . 99
4.12 Analysing one-to-one relation between poison and various parameters of machine learning algorithms . . . 99
4.13 Configurations of the models . . . 102
4.14 Misclassification rate for each dataset class . . . 105
4.15 Classification results of adversarially trained models . . . 105
4.16 Properties of the shared-spectrum 5G NR-U and 802.11ax dataset . . . 110
5.1 Summary of existing similar defences against data poisoning attacks proposed in various settings . . . 119
5.2 Dataset description of all three datasets utilised in this study . . . 125
5.3 Features correlation in dataset to analyse interdependence between features in individual dataset . . . 125
5.4 Detection and correction boundaries of machine learning models after mitigating data poisoning attacks with SecureLearn . . . 131
5.5 Impact of data poisoning on recall and f1-score of secured machine learning models . . . 132
5.6 Effectiveness of feature-oriented adversarial training on false discovery rate of random forest after poisoning . . . 135
5.7 Effectiveness of feature-oriented adversarial training on false discovery rate of decision tree after poisoning . . . 135
5.8 Effectiveness of feature-oriented adversarial training on false discovery rate of gaussian naive bayes after poisoning . . . 136
5.9 Effectiveness of feature-oriented adversarial training on false discovery rate of multilayer perceptron after poisoning . . . 136

Chapter 1. Introduction

The challenge of extracting knowledge and patterns from the data is older than the developed field of machine learning (ML) (Rosenbloom et al., 1985). However, with the evolution of time, the complexities of the data have increased with more dimensions and variability. In response to these complexities, researchers have developed algorithms to autonomously identify intricate patterns and correlations within data rather than relying only on human brains and explicit programming. Further improvements to these autonomous algorithms allow digital systems to learn independently for decision making and have developed a field called ML (Cohen, 2025).

Such advances in automating digital systems are especially relevant in fields requiring high accuracy and fairness, such as medical diagnostic applications (Murugan et al., 2021), autonomous vehicle systems (Sasmono et al., 2021), cybersecurity solutions (Chalé and Bastian, 2022), and financial systems (Qiao and Beling, 2016), making ML algorithms a more suitable solution. It is a powerful tool to analyse complex data and enable businesses and researchers to make accurate predictions, leading to innovative solutions and increased efficiency by identifying hidden patterns in the data. Its growing importance stems from the availability of computational resources, large datasets, and efficient algorithms.

The fundamental ML algorithms are known as *traditional ML*, which includes the baseline algorithms, such as support vector machines (SVM), random forest (RF) and neural networks (NN) as perceptron. However, the increasing complexities of the problems and further advancements in ML have developed modern ML called deep learning (DL), which typically extends NN to advanced algorithms such as convolutional neural networks (CNN) and recurrent neural networks (RNN) (Wang et al., 2021b).

Irrespective of the algorithms, the ML model can be developed in two settings: *binary* and *multiclass* models. When the application problem is defined across two groups, the model is developed in binary settings. Conversely, when the prediction problem is defined across more than two groups, the model is developed in multiclass settings. Multiclass models are useful in various applications, such as healthcare diagnostics and autonomous driving. However, they are associated with certain challenges that need to be addressed for successful application development. One of the major challenges in this regard is the equal distribution of data within different groups in the given dataset, which enables the model to learn each group equally, where an unequal distribution can introduce unfairness in their predictions (Del Moral et al., 2022). These complexities should be addressed to develop a robust and fair ML model.

1.1 Machine Learning Security

The prevalent use of ML in almost every digital application makes it an attractive target for adversaries to attack these models and harm the integrated systems. Attacks on the security of ML are called *adversarial attacks*. Unfortunately, the literature has highlighted many adversarial ways of attacking the model security at different development phases and breaching ML systems (Hu and Hu, 2020). Adversaries can harm it in various aspects, including exploiting privacy (Shafee and Awaad, 2021), security (Goldblum et al., 2020), robustness (Dunn et al., 2020), fairness (Xue et al., 2023), and accountability (Ghosh et al., 2022) of the model. The study of these attacks and countermeasures against these attacks is called *adversarial machine learning (AML)*.

Adversarial attacks that manipulate training data of the ML model development are known as *data poisoning attacks*. With the increasing complexities of the autonomous systems, the need for large training data is becoming a significant challenge, and one way to address this challenge is to use publicly available datasets (Zhou et al., 2017). However, reliance on public datasets increases the risk of adversarial manipulations, which can compromise the security of the model. One common form of data poisoning is the label-flipping attack (Yerlikaya and Şerif Bahtiyar, 2022), (Paudice et al., 2018b), where the labels of randomly selected data points are altered to degrade model integrity. For instance, in an intrusion detection system (IDS) trained on the public CIC-IDS-2017 dataset, an adversary may change the labels of 10 percent of the data points, causing the model to misclassify certain intrusive instances as normal. Additionally, a substantial number of mislabeled data points can induce overfitting, resulting in a breach of availability. Another significant threat is the breach of confidentiality. During model training, an adversary may introduce targeted data points that the model memorises, enabling the extraction of confidential information during inference. These considerations underscore the necessity to enhance ML security to ensure resilience against data poisoning attacks. The details of these attacks and their types are discussed in Section 2.2 in Chapter 2. Irrespective of attack types and approaches that adversaries applied to ML model development, these attacks mainly harm model performance in one of the five aspects highlighted in Fig. 1.1.

The first aspect is *privacy*, where the adversary tries to extract confidential information about the training data or learned parameters from the trained model (Shi et al., 2020). Privacy attacks can be mitigated with privacy preservation (Truex et al., 2019) and differential privacy (Zhang et al., 2020) solutions. However, these solutions imply various performance tradeoffs, such as adding noise to the training samples, which may decrease the model's performance. Conversely, various adversarial attacks successfully exploited these mitigations (Paracha et al., 2024b). This is an active research area in the domain of AML, where researchers are developing solutions, toolkits, and processes to enhance the privacy preservation of ML while preserving their performance.

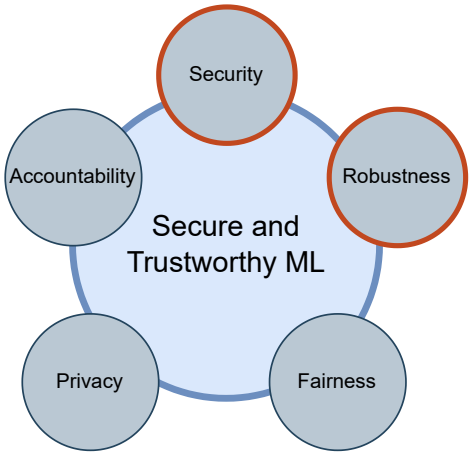


Figure 1.1: Fundamental aspects of secure and trustworthy machine learning

The next important aspects are the *security* and *robustness* of ML (Tian et al., 2024), where the adversary can manipulate the training or testing datasets to undermine the decision mechanism of the model. For example, the adversary can manipulate the dataset labels (Paracha et al., 2024a), which may confuse the model to misinterpret the dataset and reduce the expected performance. Other than that, manipulated data points can be injected to disturb the data distributions (Jagielski et al., 2021) or some important dataset features can be perturbed, which can shift the decision boundaries (Paracha et al., 2025a) of the model and ultimately harm the confidence. Defending against these attacks is a complex challenge, since distinguishing adversarial perturbations from legitimate variations is inherently difficult.

Accountability and *fairness* can also be exploited with adversarial attacks (Mehrabi et al., 2021). For example, autonomous medical diagnostic applications should be fair in their decisions and not discriminate against specific responses or data groups. The adversarial attack can change the data distribution, making it imbalanced between different subsets or groups. This biased data can train biased and unfair models, which can reflect in their predictions. Various studies highlighted that adversarial attacks have successfully exploited the accountability of ML models (Farinu, 2025).

Of these aspects, this thesis focuses on the security and robustness of ML models against data poisoning attacks. The security of ML aims to protect the model from malicious activities and external inference that compromises its confidentiality, integrity and availability (Sagar and Keke, 2021). The robustness of ML aims to overcome the adversarial noise or errors generated in the dataset, such as misplaced features or missing feature values, and how they can impact the statistical computation of the model performance (Yu et al., 2025). Numerous studies highlighted successful poisoning attacks on secured ML models (Lu et al., 2024), (Das et al., 2024) that breached the security or robustness

of ML models. Achieving adversarial security and robustness is a significant challenge. To achieve security and robustness from adversarial attacks, there is a pressing need to understand adversarial perturbations and distinguish them from normal data. It is formulated with similar objectives to model generalisation across a given domain, which can be achieved by ignoring irrelevant features. For example, receiver email in spam detection classifiers or source ports in IDS. Thus, the aim is to generalise over the underrepresented classes or invariant data. Developing strong resilience is still a challenge as the attack surface is evolving. There is a need to think about the next level of understanding algorithm architectures and training processes, rather than only generalising over the domains.

To achieve the aim of secure and robust ML, this thesis focuses on classification models, namely support vector machines (SVM), random forest (RF), decision tree (DT), gaussian naive bayes (GNB), k-nearest neighbors (KNN) and neural networks with perceptron and multilayer perceptron (MLP). These models are developed under binary and multiclass settings. Furthermore, the scope of the thesis is defined for the grey-box data poisoning attacks, following the assumptions that training datasets can be publicly accessible and an adversary can manipulate datasets and develop surrogate models; however, no model settings are known to the adversary.

1.2 Problem Background

The existing literature has predominantly studied data poisoning attacks and their mitigations in deep learning (DL) models; however, few studies have explored data poisoning in traditional ML, particularly in multiclass models. Traditional multiclass ML is significantly useful for developing multimodal applications with predefined feature datasets and limited computational resources, such as intrusion detection systems (IDS), healthcare diagnostics, and cyber threat intelligence applications. Therefore, the analysis and strengthening of the security of such models is important to defend integrated systems. Studying multiclass ML alongside DL and binary models provides an in-depth understanding of ML security against data poisoning attacks.

Furthermore, existing studies have explored data poisoning impact, considering performance metrics including accuracy, precision, recall and f1-score. However, it is also important to analyse the disruption in the underlying decision mechanism of the model when trained with a poisoned dataset. Understanding these changes helps develop strengthened and attack-agnostic mitigations against such attacks, which may defend ML from various data poisoning attacks.

1.3 Problem Statement

Data poisoning attacks manipulate the training dataset of ML models, leading to degraded model performance (Chillara et al., 2024) or enabling backdoors (Saha et al., 2020). These manipulations

can be achieved by perturbing dataset labels, poisoning dataset features, or injecting poisoned data points into the dataset. For example, metapoison (Huang et al., 2020), class-oriented poisoning attack (Zhao and Lao, 2022b), and clean-label poisoning attack (Zhang et al., 2023) successfully poisoned ML systems. Though various mitigation solutions are provided in the literature to secure ML, new attacks have successfully exploited such mitigations Paracha et al. (2024b), Koh et al. (2021), Kuppa and Le-Khac (2020). These successful exploitations highlight the complex problem of securing ML models from data poisoning attacks.

The core challenge behind this problem is to analyse vulnerabilities in ML models. The prevalence of ML models opens doors for adversaries to manipulate them in various ways, which makes it difficult to determine to what extent the outcomes of ML should be trusted. Various research studies proposed data poisoning against deep neural networks, specifically in the computer vision domain Ahmed et al. (2022), Salama et al. (2023), Wei et al. (2022), Su et al. (2019); however, traditional models are explored to a limited extent, particularly multiclass models, which are overlooked in the literature. Moreover, existing analysis techniques do not completely identify these underlying vulnerabilities. Existing studies assess data poisoning attacks from the performance aspects of the models, which do not answer the underlying changes in their training and decisions. Studying the underlying changes against poisoned training provides an understanding of model vulnerabilities and identifies relationships between model characteristics and the impact of data poisoning attacks. These understandings help develop strengthened defences against such attacks. Therefore, there is an urgent need to develop enhanced analysis processes for the detailed ML examination in various settings. These analyses should focus on analysing the behavioural changes in the ML model and their impact on the performance of the model.

Additionally, most data poisoning attacks are developed under the assumption of full adversarial capabilities and complete knowledge of the target system Zhang et al. (2023), Zhao and Lao (2022b), Zhu et al. (2023b). Evaluating the effectiveness of these attacks under limited adversarial knowledge and capabilities is essential for assessing their practicality against real-world systems, which are often not fully accessible to adversaries. The real-world impact of attacks under such constraints is still a complex challenge.

Following the challenges mentioned above, the unified problem is to develop attack-agnostic and security-by-design solutions that secure ML not only from specifically identified poison but from evolving threats in this domain. Current solutions are often tailored to specific attacks or algorithms Xu et al. (2021), You et al. (2019), Ma et al. (2021b), which limits their applicability to novel poisoning methods. For example, adversarial training (Tu et al., 2021) is designed for neural networks that utilise gradient learning and do not generalise to traditional ML models. Therefore, there is a need to investigate and analyse how poisoned models behave under limited adversarial capabilities. Such

analysis will inform the development of effective mitigation strategies that secure ML models against evolving poisoning attacks across diverse applications.

1.4 Research Aim and Objectives

To address challenges outlined in the problem statement, this thesis aims to develop a robust behaviour-informed solution to mitigate data poisoning attacks in multiclass ML and improve the generalisability and robustness of the model. The intended objectives to fulfill the aim of this research are given as follows.

- To explore the impact of data poisoning attacks under limited knowledge and adversarial capabilities. This exploratory analysis is conducted against six classification models, in binary and multiclass settings. This study analyses the impact of poisoning and investigates the limitations of existing mitigations against these attacks.
- To develop an improved defence solution to enhance the security of the multiclass ML against data poisoning attacks. The proposed solution is an attack-agnostic mitigation that secures the model by understanding underlying behaviours and decision mechanisms, irrespective of attack vectors. .
- To develop the evaluation mechanism to assess the effectiveness of the proposed security solution for ML. The proposed mechanism evaluates the mitigation solution by comparing its effectiveness against existing solutions and data poisoning attacks for various ML models.

1.5 Research Questions

Following the above research objectives, this study considers several factors to understand the security of ML by answering the following research questions (RQ).

- What are the security vulnerabilities in ML algorithms that are exploited by adversarial attacks and their impact on real-world applications? It is important to understand the realistic threats to ML models. A model being attacked likely provides strong incentives to the adversary. However, it is crucial to understand what capabilities of the attacker are required to devise an attack when limited knowledge and access to the model are provided.
- What are the potential risks, impact and consequences of data poisoning attacks? Poisoning impact varies based on the domain of applications and adversarial capabilities. Following question 1, there is a pressing need to understand how successful and impactful the attack is under limited adversarial capabilities for cybersecurity and healthcare applications. Suppose the targeted application is developed for the learning of primary school students. In that case, the

impact of poisoning is considerably lower. However, if the application is developed for cancer diagnosis, it may result in loss of human life, and thus represent a high impact of poisoning attacks.

- What are the limitations of existing solutions in mitigating data poisoning attacks? The third question addresses the limitations of existing mitigations against data poisoning attacks by analysing them across various attack vectors for traditional models. It also examines the impact of data poisoning on changing the underlying decision mechanism and characteristics of binary and multiclass models and assesses whether existing solutions address these behavioural changes.

1.6 Thesis Contributions

Following the above research questions, the contributions of this thesis are as follows:

- This study conducts a deep behavioural analysis of poisoned binary and multiclass models to analyse the change in their decision capabilities and identify relationships between poisoning impact and individual characteristics of models. These characteristics include the number of support vectors in SVM, the number of trees in RF, and the changing dataset distributions. This analysis also identified a range of poisoning levels that have a high impact on degrading the model's performance. (Paracha et al., 2024a)

- This study analyses the effectiveness of multiclass poisoning attacks under limited adversarial capabilities. To analyse multiclass poisoning, a novel outlier-oriented poisoning (OOP) attack is proposed that disturbs the decision boundaries of the model by perturbing outliers. Following the OOP attack, this analysis has also strengthened the results of the deep behavioural analysis by strengthening its outcomes of the identified poisoning levels and highlighting the impact of various dataset structures on multiclass poisoning. Moreover, the efficacy of the OOP attack is examined against a well-known mitigation solution called adversarial training, highlighting its ineffectiveness in securing traditional multiclass models; however, it has improved the resilience of neural networks. (Paracha et al., 2025a), (Paracha et al., 2024b), (Paracha et al., 2025c)

- This study proposes an attack-agnostic two-layer defence, SecureLearn, to secure multiclass models against data poisoning attacks by understanding their training fundamentals. SecureLearn proposes an improvised data sanitisation with an additional layer of feature-oriented adversarial training (FORT), to secure ML reactively and proactively against poisoning attacks. (Paracha et al., 2025b)

- This study proposes a 3D evaluation matrix to evaluate SecureLearn from three orthogonal dimensions comprehensively. The 3D evaluation matrix assesses the solution against various

data poisoning attacks and compares it with existing mitigation solutions, including multiple data sanitisation and adversarial training mechanisms.

The conceptual alignment of research objectives, research questions and thesis contributions is given in Fig. 1.2.

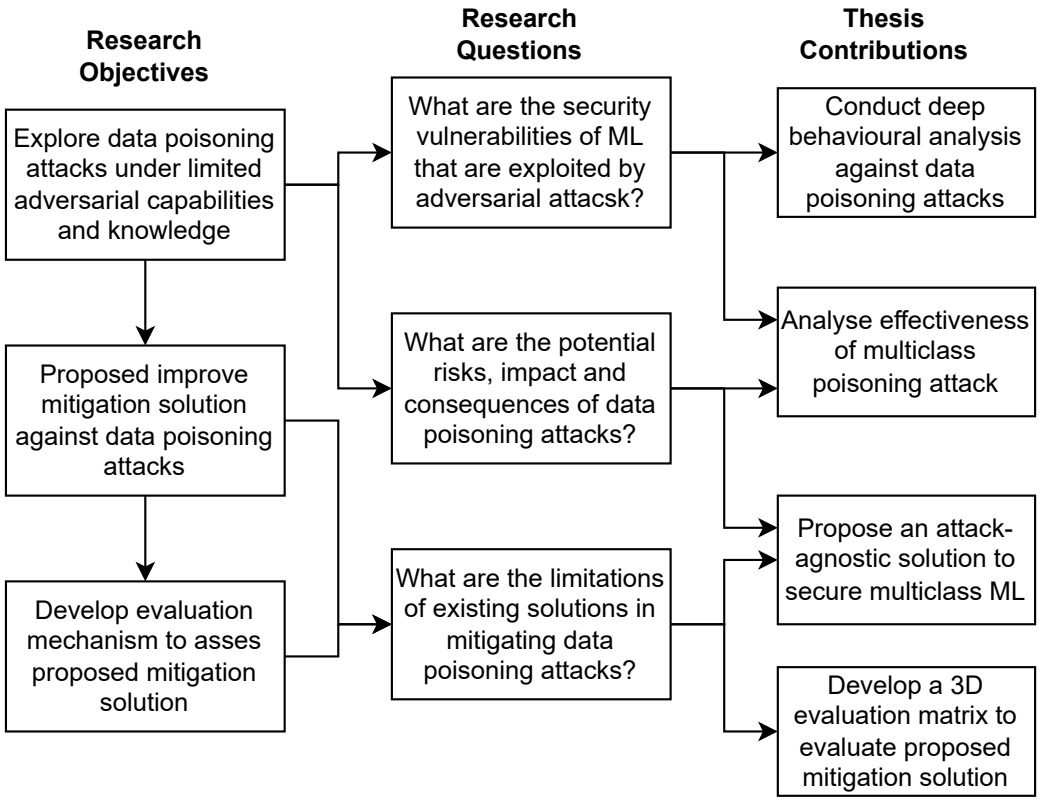


Figure 1.2: The conceptual alignment of research objectives, research questions and thesis contributions

1.7 Thesis Organisation

Following the above-mentioned contributions, the organisation of this thesis is given as follows.

Chapter 2 In this chapter, a detailed background of supervised ML, concepts of adversarial ML and its attack types is provided. Following the background, a comprehensive state-of-the-art analysis is conducted with a quantitative review to understand the threat landscape of adversarial ML, followed by a comprehensive literature review built on the adversarial attack types. This background and literature review form the basis of all the subsequent chapters. This work is discussed in Paracha and Arshad (2024), Paracha et al. (2024c).

Chapter 3 In this chapter, deep behavioural analysis of poisoned models is conducted under limited

adversarial capabilities and knowledge. This analysis studies how poisoned training shifts their decision boundary and how poison impacts individual characteristics of each algorithm. For example, how does the feature importance score change with data poisoning, or what is the relationship between the number of trees in the RF algorithm and increasing poisoning level? This analysis provides a detailed understanding of how data poisoning attacks manipulate the decision mechanism of ML models, providing insights into enhancing defence mechanisms against these attacks. This work is presented in (Paracha et al., 2024a).

Chapter 4 In this chapter, a new data poisoning attack is introduced that manipulates the feature space of multiclass models by perturbing outliers. This attack is implemented with limited adversarial capabilities and knowledge to examine the efficacy of new attacks under limited adversarial capabilities and knowledge. This chapter also examines the effectiveness of existing mitigation solutions to secure traditional multiclass models. This work is published in Paracha et al. (2025a), Paracha et al. (2024b), Paracha et al. (2025c).

Chapter 5 This chapter proposes an attack-agnostic two-layer defence called SecureLearn to secure multiclass models from data poisoning attacks. It presented a new formulation of data sanitisation and FORT adversarial training that enhances the adversarial robustness of classification models. Following that, a 3D evaluation matrix is proposed to analyse the efficacy of SecureLearn and existing similar solutions from three different aspects. This work is submitted to IEEE Transactions on Information Forensics and Security.

Chapter 6 This chapter concludes all the previous chapters, summarises the key achievements and contributions to defence in each chapter.

Chapter 2. Machine Learning Security - Background and State-of-the-Art Attacks and Countermeasures

2.1 Introduction

This chapter introduces the background and key concepts related to supervised ML algorithms, adversarial ML, and adversarial attack types, followed by a state-of-the-art analysis of adversarial attacks and their countermeasures. This chapter addresses RQ 1 by offering a comprehensive analysis of the severity, impact, and consequences of adversarial attacks. It also provides insights to guide future research. The objective 1 of the thesis is fulfilled from a theoretical perspective by examining adversarial attacks and mitigation techniques. The analysis considers adversarial knowledge, goals, and capabilities. It classifies and compares attack types and highlights the limitations of current solutions. The background section describes fundamental characteristics and training mechanisms of supervised ML algorithms. It discusses the formation of decision boundaries in both binary and multiclass contexts. This section also introduces adversarial machine learning and outlines the primary attack types. It connects these attacks to various phases of machine learning development and attack surfaces. The attack surface is defined as a set of points, features, or components that the attacker can exploit and harm the system (Zeng et al., 2019). This section establishes the conceptual foundation for the following chapters.

Next, this chapter presents the state-of-the-art analysis of adversarial attacks and their countermeasures in ML. The analysis examines the literature using a two-step approach based on prior work Paracha et al. (2024c), Paracha and Arshad (2024). First, a bibliometric analysis is conducted to analyse research trends across four dimensions of publication types and languages, publication citations, annual publications and keywords analysis, followed by a qualitative literature review based on a comprehensive methodology. This bibliometric analysis provides insights into the evolution of adversarial ML and evaluates its research impact on various domains.

Following the bibliometric analysis, a qualitative literature review critically evaluates adversarial attacks and their countermeasures to identify research gaps. This review is organised around four types of adversarial attacks: poisoning, evasion, model inversion, and membership inference. These correspond to the training, testing, and deployment phases of model development. The analysis assesses the feasibility of attacks across eight dimensions, such as machine learning algorithm, exploited vulnerability, and attack type. Mitigation strategies are also examined for each attack category.

2.2 Background

In this section, the training process of each selected algorithm is discussed to understand their decision mechanisms. This understanding will later help determine the adversarial attack methods and how these attacks can disturb the ML decision processes and affect their performance.

2.2.1 Machine Learning

Machine learning refers to a mathematical model that learns patterns from the given data to perform tasks autonomously without providing explicit input. The core goal is to approximate the mapping function $f : X \rightarrow Y$ by minimising the loss $\mathcal{L}(f(x), l)$ to predict the correct output l where $l \in Y$. Generally, larger data improves performance, as the model can capture complex patterns (Taherdoost, 2022). ML is categorised into three main types, discussed below.

Supervised learning is a type of ML development where the dataset is labeled with a predefined number of groups (Taherdoost, 2022). This type of learning is useful for task-driven problems where the ground truth is available in the form of dataset labels. The dataset is divided into training and testing instances. The training dataset is used to train the model, which is trained to recognise features and their corresponding labeled groups. Then the test dataset is used to assess the predictions of the trained model by comparing predicted labels with the test dataset labels. Supervised learning is categorised into two types: classification and regression. Supervised classification learning is used for categorical predictions with a predefined number of groups in an application, and regression learning predicts continuous values as its outcomes.

The next ML type is *unsupervised learning*, where the unlabeled dataset is provided for the model development (Balevi and Gitlin, 2017). This learning is aligned with the data-driven tasks for which no fixed number of groups or labels is provided in the dataset. In this type of learning, the model identifies features in the dataset to determine patterns. Clustering is the most common type of unsupervised learning, where the training process identifies unique clusters within the dataset.

The third type of ML development is *reinforcement learning*, also known as agent-based learning (Nguyen and Han, 2023). This type of learning emphasises feedback from humans, following rewards and punishments. In this learning, the model is iteratively trained and assessed to reach a predefined goal by continuing good practices and receiving penalties for incorrect predictions. Agent learning is a common type of reinforcement learning, used for training robots. Of these approaches, this thesis focuses on the security of supervised classification ML against data poisoning attacks, following two settings: binary and multiclass classifications, and it interchangeably uses supervised ML for classification learning throughout the thesis.

2.2.2 Supervised Machine Learning

In classification ML, the problem involves organising the data into distinct categories or outputs. The model learns patterns from the dataset to distinguish specific entities and predict their classes or labels. Consider a training dataset $D = \{(x_i, l_i)\}_{i=1}^{n}$ where x_i is the input feature of the dataset, known as an independent variable, and l_i is the target label, which is the dependent variable.
During training, the model learns patterns from the dataset D and associates them with their labels or classes. Once trained, it is tested on a previously unseen dataset. The model predicts the correct labels, which are later evaluated against the ground truth (Gil-Fournier and Parikka, 2021). A fundamental consideration in model development is that the training and testing sets are assumed to be drawn from the same underlying data distribution, meaning they share similar statistical properties. If the model demonstrates expected results during testing, as measured by performance metrics such as accuracy, recall, and f1-score, it indicates that the model effectively generalises over patterns in the data. Following successful testing, the model can be deployed in real-world applications. The development process of the supervised ML model is illustrated in Fig. 2.1. The supervised classifiers are additionally divided into parametric and non-parametric models based on how they learn their mapping functions.

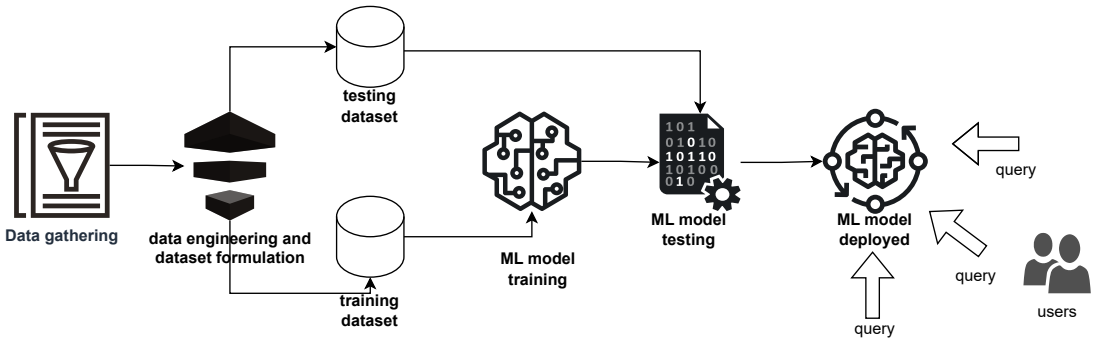


Figure 2.1: The development process of a supervised machine learning model, illustrating all phases of the development, starting from data gathering to model deployment

Parametric models follow a training process that develops their decision boundaries by learning a fixed, pre-defined number of configuration parameters θ that characterise the model's complexity (Guého et al., 2020). This process aims to learn underlying patterns from training examples until the configuration parameters are estimated in a way that reduces classification errors. Therefore, the trained model can classify new data in the relevant class. On the other hand, *non-parametric* models develop their decision mechanisms by capturing the relationship between various dataset features. These models rely on the structure of the training data at inference time rather than summarising it into a fixed number of parameters. These models predict without making strong assumptions about

specific data distribution. Conclusively, parametric models are computationally efficient but may underfit to evolving distributions or complex patterns. Conversely, non-parametric models are highly flexible to capture dynamic patterns within the provided data (Shi and Wang, 2021).
This thesis selected six supervised parametric and non-parametric algorithms, i.e., Support Vector Machines (SVM), Random Forest (RF), Decision Tree (DT), Gaussian Naive Bayes (GNB), K-Nearest Neighbors (KNN), and Multilayer Perceptron (MLP). Next, the binary and multiclass classification settings of the model development are explained in Sections 2.2.2.1 and 2.2.2.2, followed by the development process for each of the selected algorithms.

2.2.2.1 Binary Classification The simplest classification type in supervised ML is binary classification, where the model categorises instances into two classes. In this type of classification, the labels are typically encoded as 0 (false) or 1 (true). For example, in a spam detection classifier, an instance is either classified as spam or not spam. Similarly, in a skin cancer diagnostics application, a skin lesion is either detected as cancerous (true) or benign (false). Let $D = \{(x_i, l_i)\}_{i=1}^{n}$ be a dataset consisting of two classes where dataset features are denoted with $x \in X$ and labels are denoted as $l \in \{0, 1\}$. The model learns with a mapping function $f : X \to Y$ to separate classes in a given dataset. The trained model uses the learned function to classify new instances in a class. If the predicted probability exceeds 0.5, the instance is assigned to the positive class.

2.2.2.2 Multiclass Classification Multiclass classification assigns each instance to one of m possible classes, where $m > 2$. For instance, in a skin cancer diagnostics application, the model may classify lesions as melanoma, melanocytic nevus, or basal cell carcinoma, rather than simply distinguishing between cancerous and benign cases. Multiclass classifiers accommodate multinomial categorisation by supporting more than two classes as needed. For example, in the same diagnostic context, if a lesion exhibits characteristics that overlap between benign and melanoma, an additional category, such as intermediate melanoma, can be adjusted to enable more nuanced diagnostic decisions and treatment. The classifier is trained on a multiclass dataset $D = \{(x_i, l_i)\}_{i=1}^{n}$, where $x \in X$ and $l \in \{0, 1, 2, ..., m-1\}$. The mapping function f for multiclass classification is defined in Eq. 2.1.

$$l_i = \arg\max\ f(x_i) \tag{2.1}$$

2.2.3 Support Vector Machines

Support Vector Machine is a non-parametric algorithm that aims to construct optimal decision boundaries, known as hyperplanes, to separate dataset classes. Considering a training dataset $D_t = \{(x_i, l_i)\}_{i=1}^{n}$ where $x \in X$ are dataset features and $l \in Y$ are the class labels. To train an SVM model on the given dataset, the decision function is optimised to identify decision boundaries.

The general formulation of the decision function is given in Eq. 2.2.

$$f(x) = sign(w^T.\phi(x) + b) \tag{2.2}$$

where w are weight vectors, $\phi(x)$ denotes the feature mapping function that maximises the margin between classes by mapping features to an n-dimensional space, and b is the bias. The goal is to find the optimal value of w and b to identify optimal decision boundaries. The data points nearest to decision boundaries are known as support vectors. Support vectors are crucial for defining decision boundaries, as they determine the position of the hyperplane to separate classes. This algorithm is inherently a binary classifier, where $l \in \{0, 1\}$; however, it can be extended to multiclass classification. To handle the separation between non-linear data, SVM includes a soft margin controlled by the regularisation parameter, allowing flexibility to some misclassification while maximising the margin. There are two approaches for developing a multiclass SVM: the one-vs-one (OvO) and the one-vs-rest (OvR) approach. One-vs-one develops a binary classifier between each pair of classes, and one-vs-rest develops a binary classifier for each class versus the rest. The SVM models are widely used in developing cybersecurity applications such as intrusion detection and spam classification due to their effectiveness in handling high-dimensional data. A three-class SVM is illustrated in Fig. 2.2, demonstrating the decision boundaries between classes with support vectors highlighted in red.

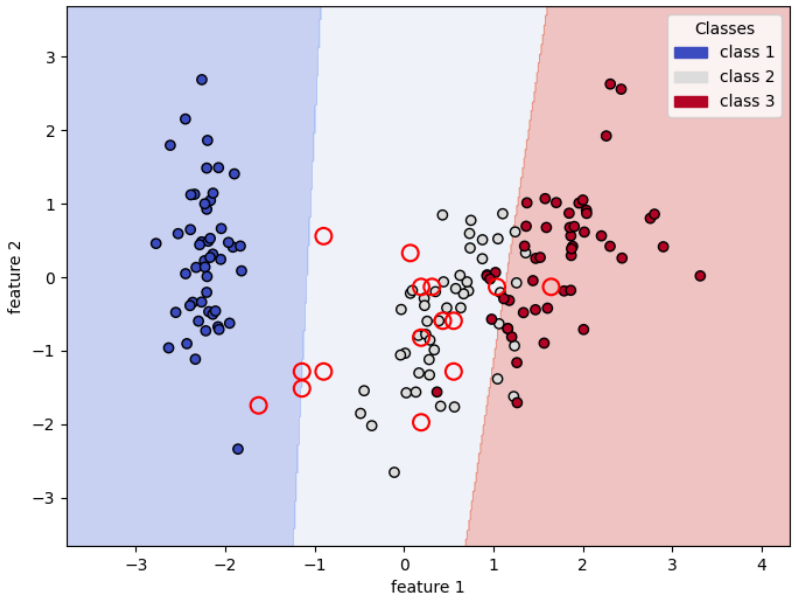


Figure 2.2: A trained support vector machines model developed with a dataset comprises three classes. The support vectors of this model are highlighted in red, which influenced the decision boundaries

2.2.4 Decision Tree

Decision Tree is a non-parametric algorithm used for both binary and multiclass classification. It develops a hierarchical tree-like structure that separates the feature space into regions associated with dataset classes. The algorithm follows a greedy, top-down approach to build a classification tree by selecting features that maximise the class homogeneity at each tree node. The tree recursively splits the dataset using a divide-and-conquer learning mechanism to minimise the impurity. The optimal split is estimated with an impurity measure such as the Gini Index, Log Loss, or Entropy. Given a training dataset $D_t = \{(x_i, l_i)\}_{i=1}^{n}$ where x represent features and l are the dataset labels, the objective at each node N is to identify the optimal feature that minimises impurity at child nodes, given a threshold t. Let the data at node m be denoted by N_m with n_m data points. Mathematically, the data is partitioned into N_m^{left} and N_m^{right} at each node $\theta(f, t_m)$ consists of a feature f and threshold t_m, such that:

$$\begin{aligned} N_m^{left}(\theta) &= \{(x, l) | x_f \leq t_m\} \\ N_m^{right}(\theta) &= N_m \backslash N_m^{left} \end{aligned} \tag{2.3}$$

Then, the quality of each split is calculated with the impurity (loss) measure $\mathcal{L}$, given in Eq. 2.4.

$$G(N_m, \theta) = \frac{n_m^{left}}{n_m} \mathcal{L}(N_m^{left}(\theta)) + \frac{n_m^{right}}{n_m} \mathcal{L}(N_m^{right}(\theta)) \tag{2.4}$$

such that, the optimal loss is given in Eq. 2.5 until the maximum allowable depth is reached where $n_m < min\, datapoints$ or $n_m = 1$.

$$\theta^* = argmin\, G(N_m, \theta) \tag{2.5}$$

Following training, the effectiveness of the model is evaluated by testing it on the classification of unseen data. While DT is capable of handling complex, non-linear data, it is prone to overfitting when developed in full depth. Fig. 2.3 highlights the decision surface in a decision tree.

2.2.5 Random Forest

Random Forest is a non-parametric ensemble learning algorithm that combines multiple decision trees. Each tree is developed following bootstrap samples from the dataset, using the underlying split criteria of the decision tree algorithm. These bootstrapping and random subsampling mechanisms help reduce the correlation and improve generalisation in the model. At each node, a random subset of features is selected to determine the best split and introduce diversity in the RF model. This process improves the performance and controls overfitting by aggregating the predictions of all trees. In RF, each tree is constructed using the same splitting criteria described in Section 2.2.4. Let $T_1, T_2, \ldots T_n$ represent decision trees that are aggregated to develop an RF classifier. For classifying the i^{th} instance

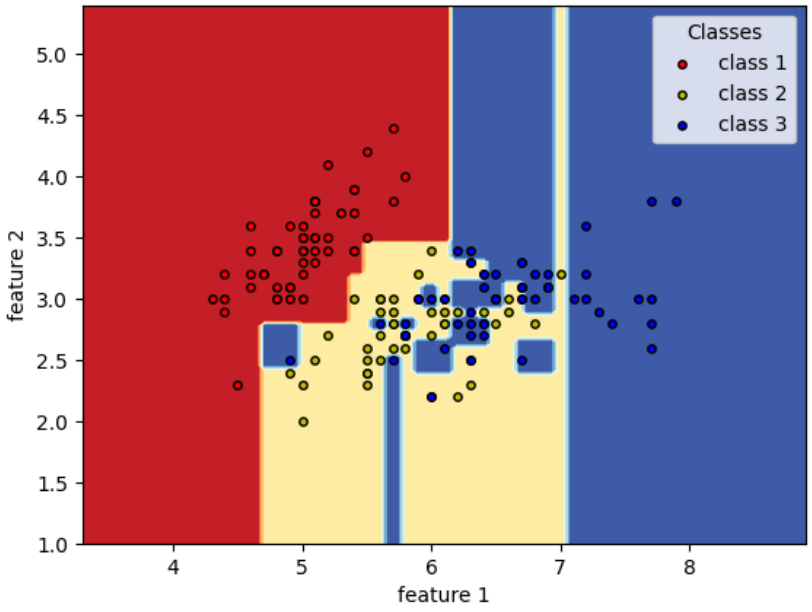


Figure 2.3: A trained decision tree model developed with a dataset comprises three classes. The decision surface of the decision tree is illustrated between three classes of the dataset with some overfitting between class 2 and class 3

x_i, the function to predict its class label is given in Eq. 2.6. The class label is determined through majority voting.

$$f(x) = mode(T_1(x), T_2(x), ..., T_n(x)) \tag{2.6}$$

where the mode returns the most frequent class among all trees. By combining predictions from decorrelated trees, this process controls the overfitting of the model.

RF inherently supports the development of both binary and multiclass classification. In binary classification, the model predicts between two classes following the decision function $f(x) \rightarrow l \in \{0, 1\}$ and for multiclass classification, it follows the decision function $f(x) \rightarrow l \in \{0, 1, 2, ..., m-1\}$ across more than two classes. Fig. 2.4 illustrates the decision boundaries created by an RF model trained on a three-class dataset, highlighting how RF splits the feature surface.

2.2.6 K-Nearest Neighbor

K-Nearest Neighbors is a non-parametric, lazy-learning algorithm that classifies based on the majority votes of the nearest data points in the training dataset. As a lazy-learning algorithm, it stores the complete dataset without performing any explicit model training and computes predictions at inference time. KNN follows the principle of the nearest neighbors and finds a predefined number k of nearest neighbors to predict the class of the test instance (Zhang et al., 2017). The model calculates the distance between the test instance and training data to identify the k nearest neighbors. This

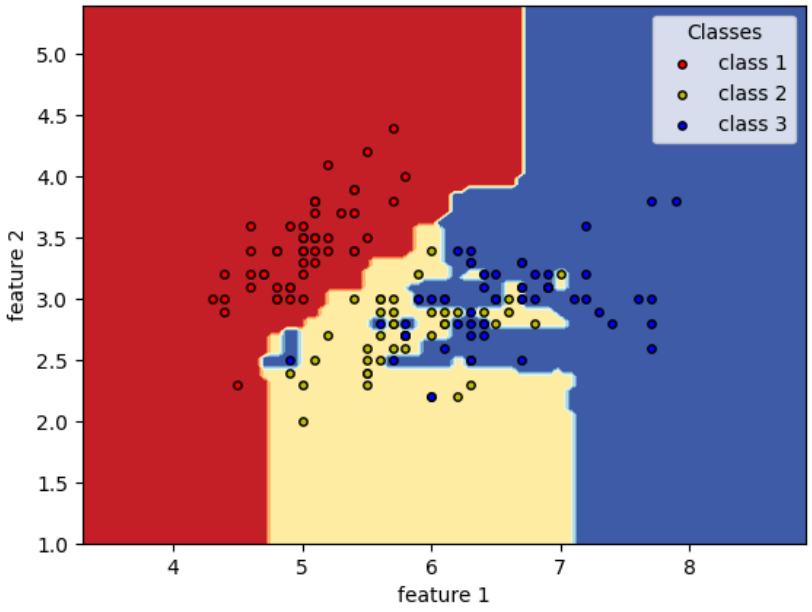


Figure 2.4: A trained random forest model developed with a dataset of three classes. The decision surface between three classes is illustrated

distance is measured using a distance algorithm such as Euclidean, Manhattan, or Minkowski, with the Euclidean distance as the default method. The voting process of KNN is sensitive to the noisy data points, which can shift the predictions. The majority voting can be inaccurate due to mislabeled data points among k-neighbors, particularly when k is small, which introduces high variance. The prediction mechanism is given in Eq. 2.7.

$$f(x) = argmax \sum_{i=1}^{k} \mathbb{I}(l_i = c) \tag{2.7}$$

where I is the indicator function, c is the possible class label of the test instance and y_i is the class label from i^{th} neighbors. Fig. 2.5 highlights a decision boundary of a KNN applied to a dataset with three classes.

2.2.7 Gaussian Naive Bayes

Gaussian Naive Bayes is a parametric algorithm that applies a naive assumption of conditional independence of all features in the dataset given a class. It also applies bayes theorem that calculates the posterior probability of a class label, for a data point, given specific feature values. Extending naive bayes with the assumption of conditional independence and probability estimations of features, GNB assumes that the dataset features follow a gaussian distribution given a class. GNB is inherently applicable to multiclass settings. Therefore, GNB applies to both binary and multiclass classification.

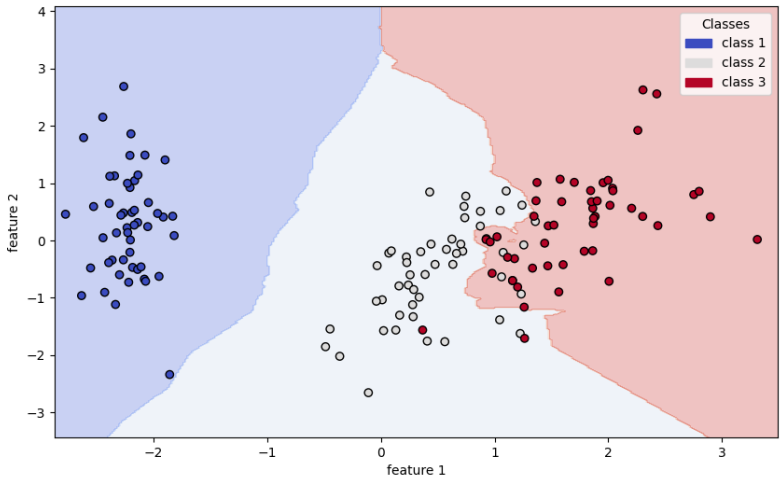


Figure 2.5: A trained k-nearest neighbors model developed with a dataset of three classes is given, showing the decision surface of each class

In supervised ML, let l_i be the class to be predicted for x_i instance given its features in n-dimensional vector $(v_1, v_2, \dots v_n)$ such that naive bayes probability estimation is given in Eq. 2.8.

$$\hat{l} = argmax\ P(l)\Pi_{j=1}^{n} P(x_j|l) \tag{2.8}$$

For x_i, the class probability $f(x)$ is predicted with Eq. 2.9.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma_{l_i}^2}} exp(-\frac{(x_j - \mu_{l_i})^2}{2\sigma_{l_i}^2}) \tag{2.9}$$

Fig. 2.6 illustrates the conditional distribution of GNB on a three-class dataset, estimated with Eq. 2.9.

2.2.8 Neural Networks

Neural networks is a parametric algorithm inspired by the human brain to learn complex, non-linear patterns in the provided data. In supervised ML, NN learns training data by approximating the mapping function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ where n denotes the dimensionality of the input features and m represents the number of output classes. This algorithm is designed in layers that comprise neurons, and each neuron applies a linear transformation followed by a non-linear activation function. Given $x \in \mathbb{R}^n$ as an input, the mapping function with weights w and bias b, given in Eq. 2.10 to compute the output.

$$z = actf(w.x + b) \tag{2.10}$$

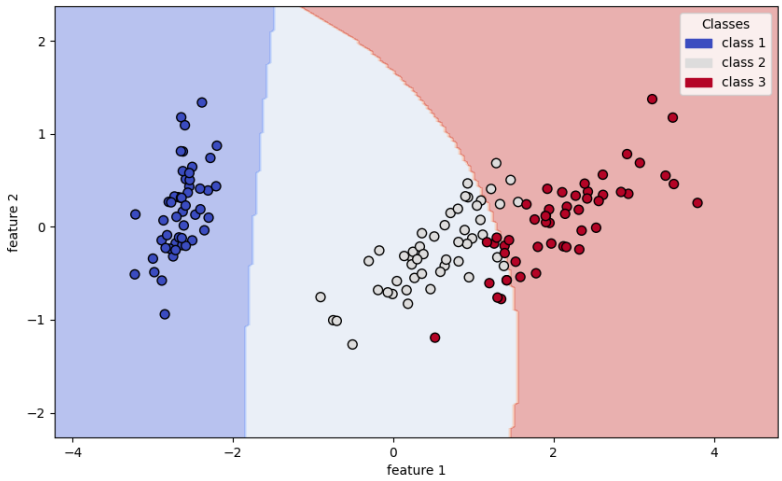


Figure 2.6: A trained gaussian naive bayes model developed with a dataset of three classes showing the decision boundaries of each class with some overlapping between class 2 and class 3

where z denotes output of a hidden layer, f denotes predictions at an output layer, and $actf$ is the activation function. The general structure of NN is shown in Fig. 2.7, illustrating an interconnected network of layers where preceding layers influence each layer.

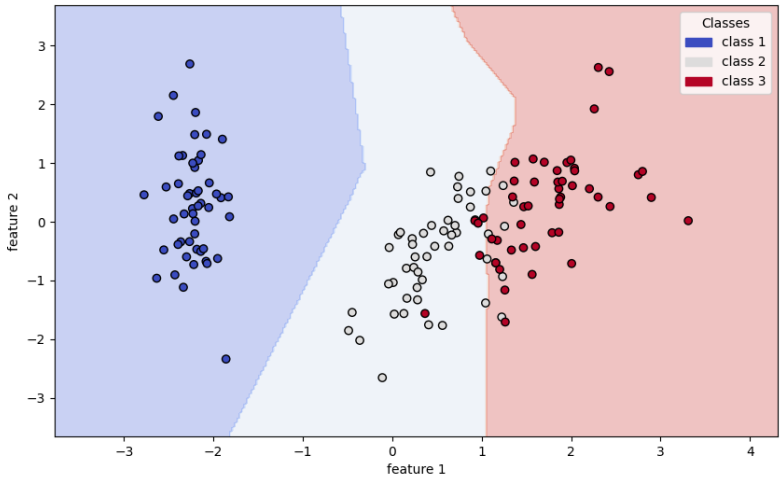


Figure 2.7: A trained multilayer perceptron model developed with a dataset of three classes showing the decision surface of the classes

A perceptron is the simplest NN form with one layer, used for binary classification, where a step function is employed as $actf$ to predict $l \in \{0, 1\}$. Further, an NN can be extended to multiclass classification by developing a multilayer perceptron (MLP), which comprises more than one layer in the network. MLP employs a non-linear activation function, such as ReLU, in intermediate layers, followed by the softmax function $actf$ at the output layer. These activation functions calculate pre-

diction confidence and output the class label with the highest confidence. These non-linear functions allow MLP to learn non-linear patterns in the data.

2.2.9 Adversarial Machine Learning

Adversarial machine learning is the study of attacks in ML and the countermeasures against them. The attacks against these models are implemented with maliciously crafted inputs called **adversarial examples** which mislead the performance of the model. Formally stating AML attacks in supervised ML as: *Supervised ML is about teaching machines to solve classification problems defined with a mapping function* $f : X \rightarrow Y$ *where in AML, adversarial examples* x' *are developed by perturbing* x *to generate manipulated output* l'*. Given a perturbed dataset* $D' = (x'_i, l'_i)_{i=1}^{n}$*, the classification function* $f(x') = l'$ *and* $l' \neq l$ *such that* x' *is generated as* $x + \epsilon = x'$ *where* ϵ *is the perturbation.*

The concept of adversarial attacks was first defined by Biggio et al. (2011) and Vidnerová and Neruda (2016), explaining these attacks on an SVM classifier. With this concept, the development pipeline of ML is found to be vulnerable to adversarial attacks, which can be attributed to various methods such as flipping labels, augmenting additional features in the dataset, and with the recent advancements in generative AI, adversarial queries can also be developed to manipulate ML/AI models. Since their discovery in 2011 to date, researchers have developed various types of adversarial exploits that have shown successful attacks on training (Chen et al., 2021), testing (Merzouk et al., 2022) and deployment (Zhu et al., 2023a) phases of the model.

To better understand adversarial attack mechanisms, a taxonomy is defined as shown in Fig. 2.8, of adversarial attacks taking into account the development phases of ML. Adversarial attacks are mainly divided into four types of data poisoning, evasion, inversion and inference attacks. This chapter will go through each attack type, explaining it in detail with its mechanism, evolution and formulation in subsequent sections.

In an orthogonal dimension, various adversarial capabilities or settings are studied in designing these attacks based on their knowledge. The highest level of capabilities is provided with complete knowledge of the ML model to the adversary, known as the white-box attack (Patterson et al., 2022). The second level of capabilities is provided with the grey-box attack (Wang et al., 2021a), where some knowledge of the model or dataset is provided to the adversary. For example, the adversary only knows the modality or the distribution of the dataset to devise an attack; however, the underlying settings of the target system are unknown to the adversary. And the lowest level of capabilities is provided with a black-box attack (Yu and Sun, 2022) where the target system is completely unknown to the adversary or may only know the name of the algorithm or dataset used to develop the system. For example, in an attack development, only the algorithm is known, but the dataset and target settings are completely unknown to the adversary. It is important to note that black-box and grey-box

capabilities overlap and are used interchangeably (Vivek et al., 2018). The next section will explain each attack type, explaining its concept and development in the context of supervised ML.

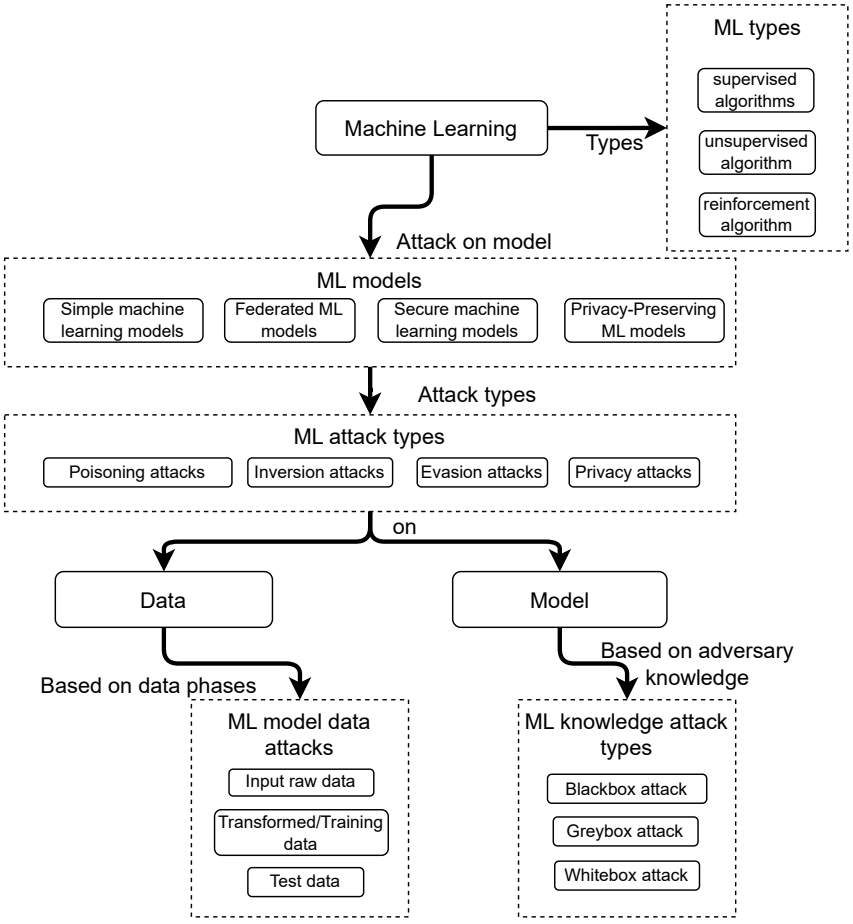


Figure 2.8: Taxonomy of adversarial machine learning attack types

2.2.9.1 Data Poisoning Attack A data poisoning attack is an adversarial attack that manipulates the training dataset of the model to corrupt the development process (Baker et al., 2024). The attack surface of the data poisoning attack is shown in Fig. 2.9. A data poisoning attack can be devised either to degrade the overall model performance, called an availability attack (Zhao and Lao, 2022a), or to alter the classification at specific inputs, called an integrity attack (Carlini and Terzis, 2021). The attacker can carry out data poisoning attacks in two ways. First, the poison may manipulate labels in the dataset, detaching features from their respective classes, known as label poisoning (Gupta et al., 2023b). Second, the poison may perturb features of the dataset, leaving labels in their original form, known as clean label poisoning (Jagielski et al., 2021). Consider $D = (x_i, l_i)_{i=1}^{n}$ as the clean dataset that is manipulated with malicious input and modified to $D' = (x'_i, l'_i)_{i=1}^{n}$ to maximise the classification errors of the decision function $C(.)$ at test time. One crucial consideration in data

poisoning attacks is that poisoned instances in the dataset closely resemble clean instances to avoid detection.

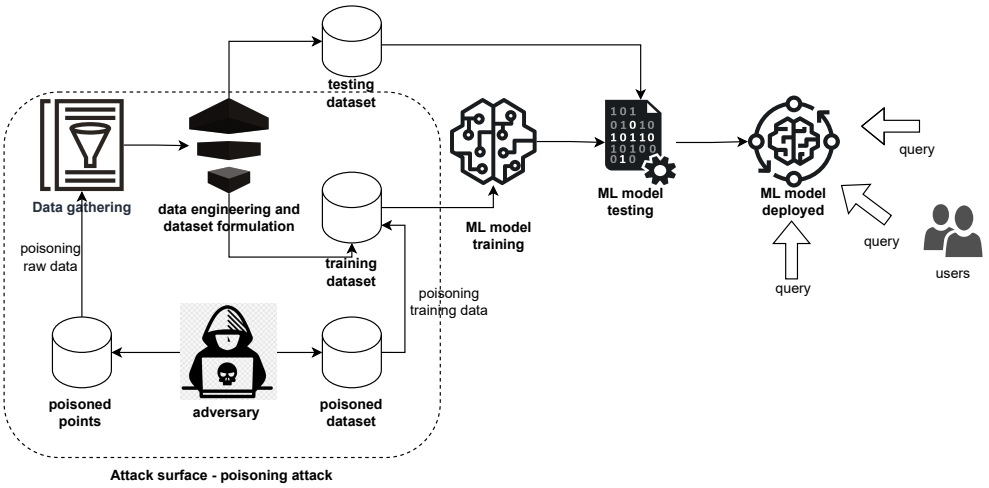


Figure 2.9: Poisoning attack surface in machine learning model development process - Poisoning machine learning attack

2.2.9.2 Evasion Attack An evasion attack is an adversarial attack that perturbs test inputs to manipulate the classification results during testing (Biggio et al., 2013). The attack surface for evasion attack is shown in Fig. 2.10. An evasion attack aims to manipulate the model's sensitivity to perturbations in test inputs by identifying a path that maximises loss in the input space. In this way, it forces the model to generate incorrect classifications while leaving the trained model and the training dataset intact. Consider a dataset $D = \{(x_i, l_i)\}_{i=1}^{n}$ that is divided into training dataset D_t and test dataset D_{test} that is manipulated with perturbation $x' = x+\epsilon$ such that $D'_{test} = \{(x'_i, l'_i)\}_{i=1}^{n}$, so that the perturbed test input x' is classified with an incorrect output $l' \neq l$.

2.2.9.3 Model Inversion Attack A model inversion attack is an adversarial attack in which the adversary steals or learns the confidential configuration parameters of the trained model to uncover the private information or replicate the model (Usynin et al., 2023). For example, in a facial recognition system, an attacker provides a face image to the model to assess its output confidence for a specific class or label associated with that image. In this way, the attacker reconstructs the image for a targeted label or identifies the model parameters by interpreting the output confidence. The attack surface for the inversion attack is given in Fig. 2.11. Let $C(.)$ be the decision function that provides the class label l, where, with an inversion attack, the attacker calculates loss in the confidence predicting y. Mathematically, it is given in Eq. 2.11.

$$argmin\, \mathcal{L}(C(x), y) \tag{2.11}$$

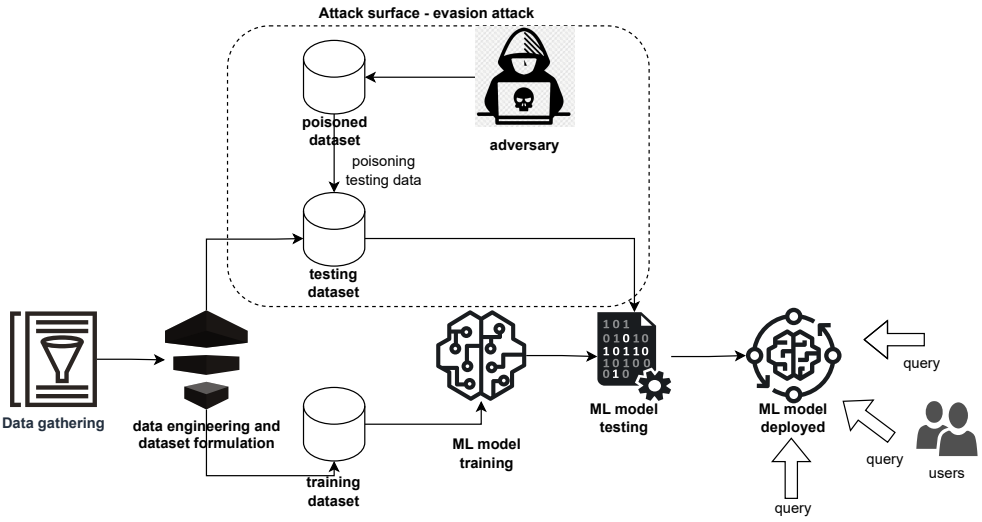


Figure 2.10: Evasion attack surface in machine learning model development process - Evasion machine learning attack

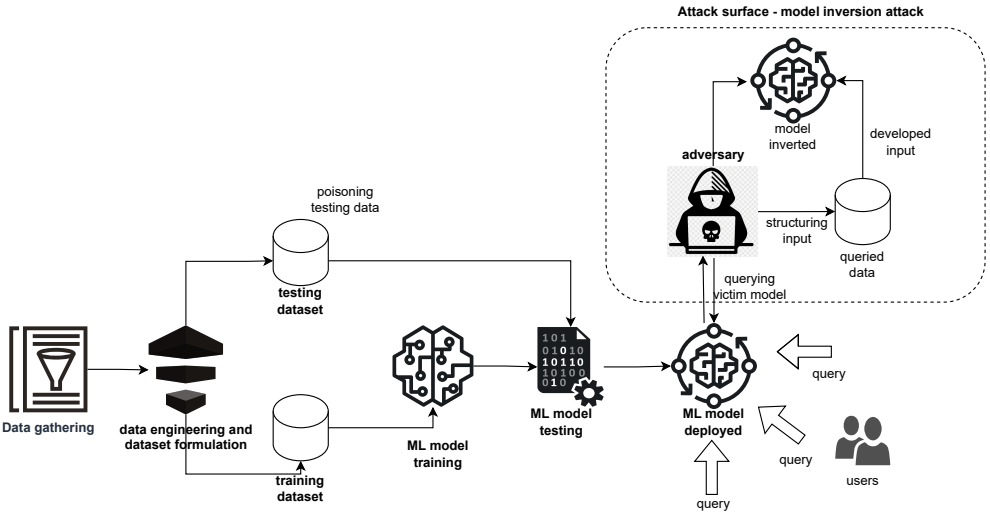


Figure 2.11: Model inversion attack surface in machine learning model development process - Model inversion machine learning attack

2.2.9.4 Membership Inference Attack A membership inference attack (Qiu et al., 2020) is an adversarial attack in which the attacker seeks to answer the question:

Is the input $\hat{x}$, given to classification function $f(\hat{x}) = y$, a member of the training dataset for the targeted model $\mathcal{M}$?

Generally, an ML model behaves differently on the training dataset compared to unseen data, which is why a difference in training accuracy and test accuracy is observed. Fig. 2.12 illustrates the attack

surface for membership inference attacks. Considering this behaviour, the primary objective of a membership inference attack is to exploit differences in model predictions and identify whether x is part of the training dataset. In this way, the attacker can infer whether specific inputs were part of the training data for the targeted model. Let $f(.)$ be the classification function, where $\hat{x}$ is the input and y is the output. Then the objective is to reconstruct $D = \{(\hat{x}, y)\}_{i=1}^{n}$.

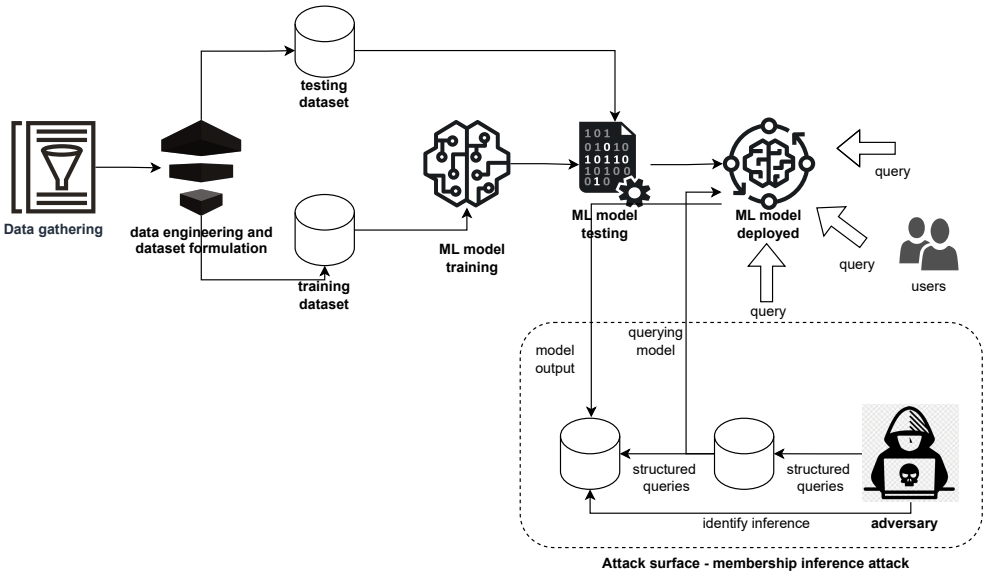


Figure 2.12: Membership inference attack surface in machine learning model development process - Membership inference machine learning attack

2.3 State-of-the-Art Analysis

2.3.1 Methodology

A detailed literature review is conducted following eight dimensions to analyse and compare adversarial attacks. The methodology for the state-of-the-art analysis is given in Fig. 2.13. The peer-reviewed

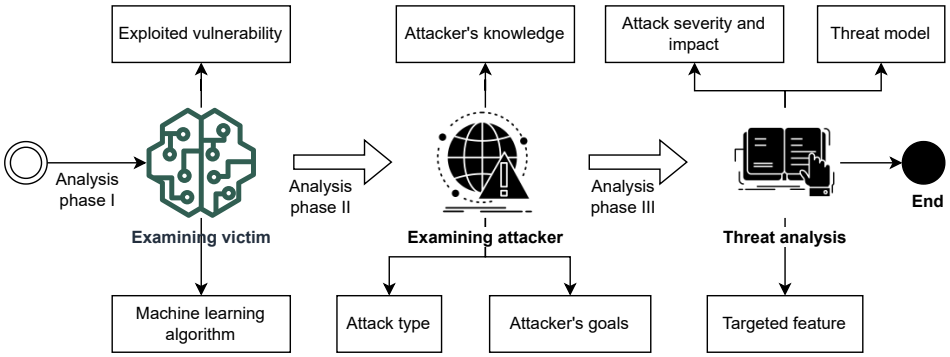


Figure 2.13: Research methodology of the state-of-the-art analysis

and conference papers, published in or after 2017, are selected for this literature review to assess adversarial attacks on the latest ML models and systems, developed with updated technologies and practices. The papers selected either focus on AML attacks or mitigation solutions against AML attacks to provide technical insights into the attack development.

Machine learning model/algorithm In this literature review, firstly, the ML algorithm/model included in the selected papers is analysed to study technical interpretations of the attack design. It is considered an influential factor in identifying the design and complexity of adversarial attacks. It also highlights the vulnerabilities of individual ML algorithms.

Exploited vulnerability Next, the exploited ML vulnerabilities are examined in detail, which helped develop the attack vector to manipulate ML models. This dimension technically assesses the attack success against the targeted system.

Attack type Further to the analysis of the ML algorithm/model and the exploited ML vulnerability, the adversarial attack type is examined to assess the attack surface of adversarial attacks. The attack types included are poisoning, evasion, model inversion, and membership inference attacks. Analysing existing studies based on these attack types explains the practicality, implications, and comparison of these attack types against ML systems.

Attacker's knowledge The literature review also explored the attacker's knowledge to deepen the understanding and examine the impact of the knowledge and capabilities of the adversary in developing adversarial attacks. The knowledge levels can be set between zero, partial, and complete knowledge of the targeted system. It helps analyse the impact of attacks from existing studies and compare the complexity and implications of each adversarial attack, as it is considered an important benchmark when designing these adversarial attacks.

Attacker's goals The next significant dimension is the detailed synopsis of the adversary's goals and objectives set with the devised attack. The consequences of the adversarial attack can be understood by analysing the intention and goals of the adversary.

Threat model The threat model is analysed under which the attack has been developed. The adversarial capabilities and access to the target system have been studied to analyse the practical feasibility of the attack.

Attack severity and impact After analysing adversarial attacks with the above dimensions, the attack severity is determined by studying the outcomes, as discussed in publications. Analysing the attack severity will provide us with grounds to study the complexities and practical implications of adversarial attack types.

Exploited feature The exploited feature of the model is determined for each attack to understand the vulnerabilities in ML models and their development processes.

2.3.2 Literature Review

This chapter proposed an in-depth and comparative analysis based on four major adversarial attack types on ML. To analyse various adversarial attack vectors in detail, each attack vector is examined based on attack type, exploited features, adversary, its capability and knowledge, and the impact of the attack vector on the victim model or algorithm. This study is the first literature analysis that examines existing works according to eight dimensions. A hierarchical summary of articles studied for attack analysis is given in Fig. 2.14. A detailed analysis of the examined attacks is provided in Sections 2.3.2.1 to 2.3.2.4. Attack vectors are analysed following eight dimensions, given in the methodology.

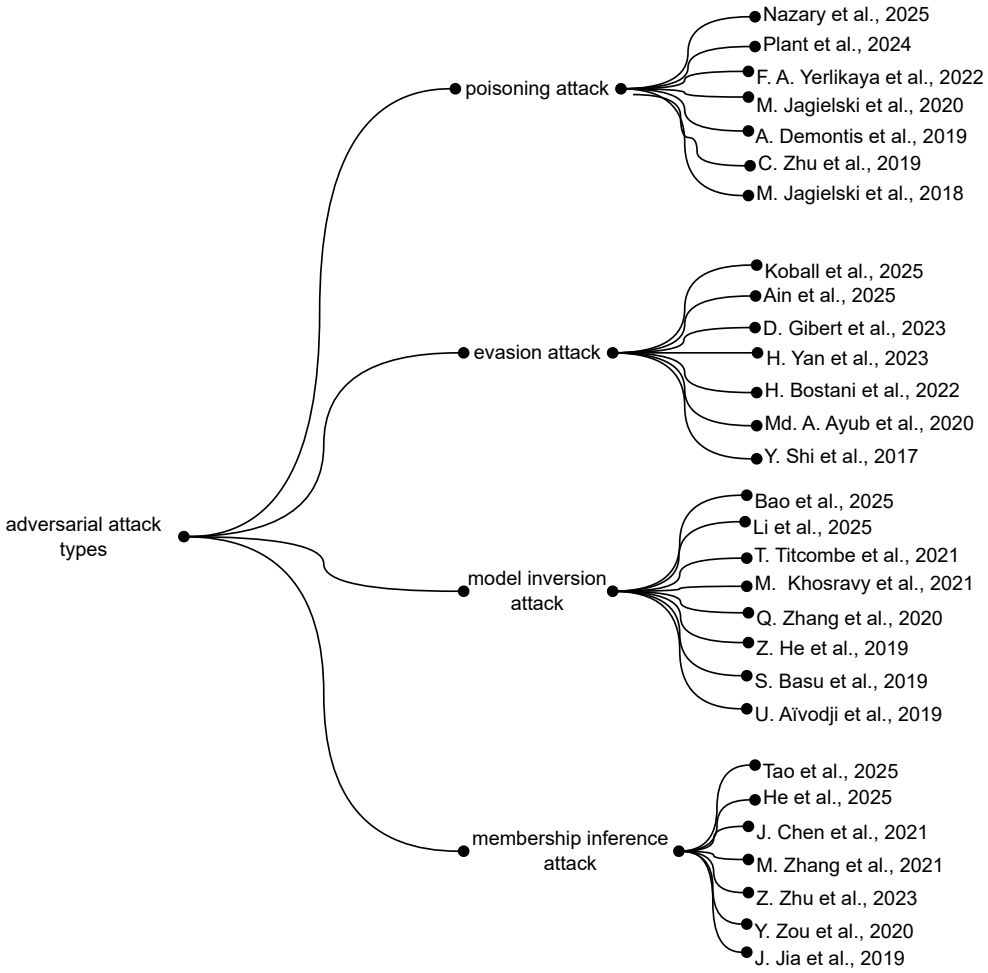


Figure 2.14: State-of-the-art - AML attack types

2.3.2.1 Data Poisoning Attacks Poisoning the dataset is possible in two approaches. The first approach is to disrupt the labeling strategy of the victim model, known as the label poisoning attack (Gupta et al., 2023a) and the second is the perturbation of features, where the adversary manipulates

the features of the dataset, leaving the label integrated, known as a clean-label poisoning attack (Zhao and Lao, 2022a). The existing literature provides several approaches to implement data poisoning attacks, considering the attack surface defined for data poisoning in ML development. The comparison of existing studies is given in Table 2.1. Yerlikaya and Şerif Bahtiyar (2022) designed two

Table 2.1: Analysis of poisoning attacks

Reference	Machine learning model/ algorithm	Attack type	Exploited vulnerability	Attacker's knowledge	Attacker's goals	Attack severity and impact	Defined threat model	Targeted feature
Nazary et al. (2025)	OpenAI LLM	Emotional attack, neighbor borrowing, chain attack	inject poisoned words to change emotional intensity	white-box attack	promote and demote recommendations	no	model performance	
Plant et al. (2024)	BERT, RoBERTa, ALBERT, DistilBERT, BERTeus, EIBERTheu, IXAmBERT, RobertaEUSCrawl	label-flipping attack	probabilistic approach to change class labels	black-box attack	reduce model performance	large pretrained models may be more robust than ML models	no	model performance
Yerlikaya and Şerif Bahtiyar (2022)	SVM, SGD, Logistic regression, Random forest, Gaussian NB, K-NN	random label and distance-based label flipping attacks	poisoning dataset by changing class labels with two effective strategies	white-box attack	reduce performance (accuracy) of the system	KNN and Random forest algorithms are not much affected by label poisoning attacks	no	model accuracy
Jagielski et al. (2021)	Convolution neural networks	subpopulation attack	poisoned cluster is integrated as subproportion of training dataset	gray-box attack	misclassification targeted attack	Subpopulation attacks are difficult to detect and mitigate specifically in non-linear models	yes	test time prediction
Demontis et al. (2018)	SVM classifier, Logistic, Ridge, SVM-RBF	training time poisoning attack	reduced gradient loss with poisoned data points in transferable setting	white-box, black-box attacks	violate model's integrity and availability	poisoning attacks are more effective on models with large gradient space and high complexity	yes	model availability
Zhu et al. (2019b)	Deep Neural networks	feature collision attack, convex polytope attack	feature space with perturbed training samples	gray-box attack	over fit target classifier with poisoned dataset	Turning dropout during training with poisoned data enhance transferability of poisoning attack in deep neural networks	yes	test time misclassification
Jagielski et al. (2018)	Linear regression	statistically based regression points poisoning generation with flipped labels	distinguishing legitimate and poisoned regression points with minimal gradient loss	mean and co-variance dependent gray-box attack	misclassification of the system	residual filtrating mitigates poisoning attack on Linear regression	yes	model accuracy

label-flipping attacks to perturb six ML algorithms with four datasets. The attacks were designed to poison binary classifiers and reduce model performance. Jagielski et al. (2021) proposed an ML poisoning attack called the Subpopulation attack. This attack injected a poisoned cluster into the

dataset to evade detection of poisoned data points. Existing security techniques, such as training a regression model (TRIM), activation clustering, reject on negative impact (RONI), and spectral signatures, were found to be ineffective against a subpopulation data poisoning attack. In the study (Demontis et al., 2018), the poisoning and evasion attacks on ML are designed to highlight their transferability during both the training and testing phases of model development. Demontis et al. (2018) highlighted the risk of transferring poison from the surrogate to the victim model. A gradient-based optimisation framework is developed to transfer the poison that alters the gradient of input samples in both training and testing datasets. It empirically analysed the security vulnerabilities in transfer learning and identified major factors that breach integrity, making poisoning and evasion attacks successful in transferring between surrogate and victim ML models. These factors are the attacker's optimisation objectives, gradient alignment of surrogate and target models, and model complexity.

Zhu et al. (2019b) also demonstrated the transferability of poisoning attacks in ML by implementing polytope attacks in deep neural networks. They explained the impact of clean-label poisoning attacks. They highlighted the successful poisoning with 50% performance degradation with 1% poisoning of the training dataset. The convex polytope attack is implemented on various deep neural networks as case studies in this research, highlighting the efficacy and consequences of data poisoning attacks in transfer learning. Their study confirmed the reliability and effectiveness of a convex polytope attack, comparing it with a feature collision attack. It also demonstrated the successful transferability of the convex polytope attack in a black-box setting where the adversary does not know the dataset of the victim model and still achieves almost the same results as when the adversary has a 50% overlap with the target dataset. In conclusion, the research discussed above has formulated improvements in the transferability of poisoning attacks by turning on the dropout rate and implementing convex polytope objectives in multiple layers of neural networks. This research underscores the need to secure ML, specifically neural networks, from poisoning attacks in various adversarial settings.

The research by Jagielski et al. (2018) particularly focuses on the security vulnerabilities and defense solutions related to linear regression. It focuses on poisoning linear regression models with gradient-based optimisation and statistical attack strategies. Their study introduces a new optimisation framework to poison linear regression in a gray-box attack setting, evaluating the limitations of existing attacks. It also proposed a statistical poisoning attack to maximise the loss by introducing poisoned points at the edges of the decision boundary. This attack exploits secure regression models. However, TRIM has been proposed, proving to be more effective in mitigating poisoning attacks in the linear regression model but ineffective against subpopulation attacks, thus proving the severity of the poisoning attack in adversarial settings.

2.3.2.2 Evasion Attacks Attacking the ML model at test time is known as an evasion attack. This attack aims to mislead the model by perturbing the test data (Ayub et al., 2020). The ultimate objective of this attack is to manipulate the testing input to undermine the integrity of models during testing. Malware Generative Recurrent Neural Network (MalRNN) is a deep learning-based approach designed to conduct evasion attacks on ML-based malware detection systems (Ebrahimi et al., 2020). MalRNN evades three malware detection systems, which demonstrates the effectiveness of evasion attacks. Moreover, these attacks highlight the importance of reliable security solutions to mitigate vulnerabilities in ML against evasion attacks. A comparison of evasion attacks is given in Table 2.2. Malware classifiers are also vulnerable to adversarial attacks. In the study by Bostani and Moonsamy (2021), a test-time attack is developed on an Android malware classifier to disrupt its classification outcome. The attack presented in this paper was a black-box attack that extracts opcodes using the n-grams strategy from disassembled Android application packages (APKs) and manipulates benign samples into malicious ones through a random search technique. This attack was tested on five malware detectors. It demonstrated the effectiveness of a test-time attack that evades the ML model and caused misclassification during testing. As a result, ML-based malware detectors, including Drebin, Detection malware in Android (MaMaDriod), with an accuracy of 81% and 75%, respectively, and others failed to detect malicious Android applications.

Similarly, the Jacobian-based saliency map attack (JSMA) also demonstrated the stealthiness of the evasion attack. JSMA was developed using a multi-layer perceptron for IDS. The goal is to achieve targeted misclassification, where the adversary intends to classify malware traffic in network intrusion detection systems (NIDS) as benign. The experimental analysis followed a white-box setting to devise this evasion attack, achieving a maximum accuracy drop of approximately 29% using the TRabID 2017 dataset. Hence, it demonstrated the malicious approach to threatening ML applications in cybersecurity, subsequently highlighting the test-time security vulnerabilities in neural networks.

The sensitivity of evasion and causative attacks is examined against DL models, proposed in (Shi and Sagduyu, 2017). This examination helps understanding the security vulnerabilities in deep learning models. This research devised an adversarial perturbation approach and tested it with text and image datasets. Initially, an evasion attack was conducted, followed by an exploratory attack aimed at inferring the trained classification model and extracting its private tuning parameters. The exploratory attack was a black-box query-based attack that replicates the victim model following the obtained query outputs. In the replicated model, this attack was extended by poisoning test labels to mislead the deep learning model through an evasion attack.

2.3.2.3 Model Inversion Attacks The objective of this attack is to disrupt the privacy of ML. Model inversion attack is a type of attack in which an adversary tries to steal the developed ML model

Table 2.2: Analysis of evasion attacks

Reference	Machine learning model/ algorithm	Attack type	Exploited vulnerability	Attacker's knowledge	Attacker's goals	Attack severity and impact	Defined threat model	Targeted feature
Koball et al. (2025)	decision tree, random forest, adaptive boosting, isolation forest	label perturbation to mislead classification	white-box attack	test-time performance decrease	yes	leaf nodes of the model		
Ain et al. (2025)	Fused Truncated DenseNet, Fused Swish-ReLU Efficient-Net, Efficient-capsule Net, Regularized Forensic Efficient Net	statistical distraction pipeline	gradient perturbation	black-box attack	evade deepfake detector	deepfake detector are susceptible to evasion attack	no	brightness of the localised masked area
Gibert et al. (2023)	Generative adversarial networks	query-free feature-based attack	perturbed features in executable	black-box attack	evade ML detector with malicious executable	ML-detectors are vulnerable to be evaded with query-free attacks	no	victim detection decision
Yan et al. (2023)	Logistic regression, SVM, NB, Decision tree, RF, xgBoost, ANN, Ensemble model	label-based evasion attack	poisoned labeled samples	black-box attack	transfer adversarially crafted samples to evade	transfer-based evasion attack is a serious threat to ML and DL	no	test time precision
Bostani and Moonsamy (2021)	ML-based malware detector	n-gram based attack on malware classifier	transform malware samples into benign with n-gram based incremental strategy	black-box attack with model query access	misclassification of android malware detector	DNN are more affected by evading surrogate models comparing to linear SVM classifier	yes	test time prediction
Ayub et al. (2020)	multi-layer perceptron network	Jacobian-based saliency map attack	iterative approach to insert perturbation near sensitive feature of benign samples	white box attack	misclassify malicious sample as benign in IDS	multi-layer perceptron can be exploited with evasion attack with minimal model's knowledge	no	test time prediction
Shi and Sagduyu (2017)	naïve bayes classifier	evasion attack with feed-forward neural networks	feed poisoned samples with DL score under computed attack region	exploratory black-box attack	misclassify test data samples	controlled perturbations to labels and classification boundary may limit adversarial impact on DL	yes	model availability

by replicating its underlying behaviour, querying it with different datasets. An adversary extracts the baseline model representation through a model inversion attack and can regenerate the training data to replicate the model. The comparison of the selected studies is provided in Table 2.3. Usynin et al. (2023) designed a framework for a model inversion attack on a collaborative ML model and demonstrated its effectiveness. Their study also highlights the impact of model inversion attacks on transfer learning models.

The research paper (He et al., 2019) experimentally demonstrated a privacy attack during inference

Table 2.3: Analysis of model inversion attacks

Reference	Machine learning model/ algorithm	Attack type	Exploited vulnerability	Attacker's knowledge	Attacker's goals	Attack severity and impact	Defined threat model	Targeted feature
Li et al. (2025)	VGG16, ResNet50, IR50-SE, AlexNet	gradient perturbation on entropy loss	gradient of images	white-box attack	GAN transfer learning	model inversion attacks have limited efficacy against GAN models	no	dataset samples privacy
Bao et al. (2025)	FaceNet64, ResNet-152, VGG16	generative MI attack, knowledge-enriched distributional MI attack, reinforcement-learning-based black-box MI attack, boundary-repelling MI attack	latent probability distribution	black-box attack, white-box attack	identify private data within optimal latent probability distribution	identifies model private training data samples	no	training dataset samples
Titcombe et al. (2021)	split neural networks	model inversion attack on distributed ML	steal intermediate/distributed data from nodes in transfer learning	black box attack	invert intermediate stolen data into input format	Model inversion attacks are effective and dependent on input dataset	yes	model interception
Khosravy et al. (2021)	deep neural networks	images reconstruction with MIA	regenerate model by intercepting private data of victim model by gathering output	gray-box attack	inverted model and developed duplicate	ML is under serious threat of MIA attack with partial knowledge of system	no	model privacy
Zhang et al. (2020)	deep neural networks	stealing victim's model classes	sample regeneration helps to determine private data of victim's model classes	white box attack	developed surrogate model similar to the target	ML model can be inverted even if secured with differential privacy	yes	model privacy
He et al. (2019)	deep neural networks	Inverse-Network attack strategy	used untrusted participant in collaborative system	black box, white box and query-free inversion attacks	extract inference data with an un-trusted adversarial participant in collaborative network	Privacy-preservation is challenging to achieve in split DNN	yes	model privacy
Basu et al. (2019)	deep neural networks	generative adversarial network approach	extracted output from targeted network with generative inference details	white-box attack	extract model class/inference details by replicating generative adversarial network	machine learning can be inverted with generative samples	no	model accuracy
Aïvodji et al. (2019)	deep neural networks	query based generative adversarial network	extract model details by interpreting queried outputs	black box attack	Breach privacy of Convolutional neural networks (CNN)	differential privacy is not much effective to mitigate MIA on machine learning	no	model privacy

in collaborative ML and argued that a single malicious participant could infer the target system and steal its confidential information. This attack is successful in all three settings of complete knowledge, zero knowledge, and a query-free attack setting. The confidential tuning parameters were extracted using a regularised maximum likelihood estimation technique in which the adversary followed the Euclidean distance estimation and identified the optimal sample with the least variation. In conclusion, this research highlighted the potential of inference attacks that require attention to be mitigated to ensure privacy preservation of deep learning. Basu et al. (2019) demonstrated privacy issues in ML algorithms by inverting a deep neural network (DNN) with a model inversion attack. This research study implemented the model inversion attack on a facial recognition system and extracted the class representation of the model. The attack developed in this research had only baseline knowledge of the target system. The attacker employed a generative adversarial network to create input samples and invert the victim model, highlighting the effectiveness of generative AI in inverting the model. Another framework called generative adversarial model inversion (GAMIN), developed by U. Aivodji and others (Aïvodji et al., 2019), was also based on generative adversarial networks. It created adversarial images to query the targeted model and extract its details through comparative output resemblance. The threat disclosed with adversarial networks is that even without prior knowledge of the system under attack, the adversary can extract its confidential parameters and reconstruct the model. Khosravy et al. (2022) also developed a model inversion attack on a deep neural network-based face recognition system. It was a gray-box attack in which the adversary had partial knowledge of the system under attack, including model structure and parameters. This attack extracted the model configurations by reconstructing images based on the confidence scores provided by the targeted model, hence inverting the targeted CNN model. In conclusion, all the mentioned attacks emphasise the privacy-preservation of ML, which is a primary consideration in constructing trustworthy and resilient AI/ML that resists adversarial attacks.

2.3.2.4 Membership Inference Attacks A membership inference attack is another privacy attack that infers the victim model and extracts its training data, privacy settings, and model parameters. In this type of attack, the adversary has access to query the victim model under attack and can analyse the output gathered from the queried results. The adversary can regenerate the training dataset of the targeted adversarial ML model by analysing the gathered queried results. The Membership inference attack (MIA) is another privacy risk to ML and DL models. The comparison of selected MIA attacks is given in Table 2.4. Zou et al. (2020) comprehensively studied membership inference attacks in deep learning models under transfer learning and achieved 95% accuracy in determining whether the input instance belonged to the training dataset. These attacks were developed for three different transfer learning modes as part of this research. When the adversary had access to

Table 2.4: Analysis of membership inference attacks

Reference	Machine learning model/ algorithm	Attack type	Exploited vulnerability	Attacker's knowledge	Attacker's goals	Attack severity and impact	Defined threat model	Targeted feature
He et al. (2025)	LLaMA2-13B, Falcon-7B, Pythia-6.9B, OPT-6.7B	label-only MIA attack	token-level semantic similarity	black-box attack	label-only MIA attacks are designed for the fine-tuning phase of LLM	label-only MIA attacks are highly effective in inferring fine-tuning LLMs	yes	personalisation of LLMs with fine-tuning tokens
Tao and Shokri (2025)	Multilayer Perceptron	hyperparameters range following the reference model	black-box attack	identify training dataset	MIA attack infer accurate data within a defined range	no	hyperparameter range	
Zhu et al. (2023d)	multi-layer perceptron	MIA on sequential recommendation system	surrogate and shadow models are designed to extract recommendations	black-box attack	infer user recommendations	inferring sequential recommendations leads to provide personalised details	yes	dataset inference
Chen et al. (2020)	Lasso regression, CNN	MIA with shadow model	shadow model is used to mimic ground truth	white box attack	retrieve confidential details of target model	differential privacy mitigates MIA compromising accuracy of model	no	model inference
Zhang et al. (2021)	neural networks-based recommendation system	inference attack to extract user-level details	adversarial model is developed with theft users' private data	black box attack	retrieve private details of victim model	Popularity randomisation is effective against MIA in recommender system	yes	model privacy
Zou et al. (2020)	deep neural networks	transfer learning based black-box attack	no privacy-preserved in transfer learning model	black box attack	infer training model details with three formulated attacks	transfer learning is at serious threat of MIA	yes	model inference
Jia et al. (2019)	neural network	MIA against binary classifier	interpret output confidence score to manipulate model details	black-box attack	retrieve private training data of classifier	existing solutions are subject to the dataset used in the classifier	no	dataset inference

the teacher model, they targeted the trained student model and inferred the teacher model's dataset using the student model. Zou et al. (2020) implemented a surrogate model based on ResNet20 convolutional neural networks with derived and student datasets to infer the membership inference of the victim model. This attack vector was effective in demonstrating the capability of membership inference attacks to compromise privacy, even with limited access or information about the victim model. Another potential privacy attack is mentioned in (Zhang et al., 2021), where the attacker targeted an automated recommender system using a membership inference attack. This is a zero knowledge attack. Their attack posed a serious privacy threat to the recommender system's sensitive user data, which adversaries can reveal using a query-based attack. In this context, the inference attack is characterised by three recommender algorithms: item-based collaborative filtering, a latent factor model, and neural collaborative filtering. A shadow model was implemented to mimic the victim's training dataset, which ultimately puts its privacy at risk.

Various mitigation techniques are also proposed to secure ML models from the adversarial attacks mentioned above. Many existing solutions are primarily attack-agnostic or algorithm-focused. While they may secure models from specific attacks, they might fail against new attacks. The proposed security solutions in the literature have various limitations that must be addressed to maintain the integrity of ML, ensuring that AI/ML remains secure and trustworthy. A hierarchical description of the mitigation techniques analysed in this study, based on adversarial attack types, is presented in Fig. 2.15. A detailed analysis of existing security solutions based on adversarial attack types is given as follows:

2.3.2.5 Mitigating Data Poisoning Attacks

Data Sanitisation Data sanitisation is one of the prominent mitigation techniques against data poisoning attacks, which preprocesses training datasets and removes erroneous or poisoned data points. Nevertheless, this process may reduce the size of the dataset, leading to underfitting in model development. Venkatesan et al. (2021) proposed a solution to overcome the limitations of data sanitisation by creating random training data subsets to train an ensemble of ten classifiers to balance the poisoning effect and dataset size. This mechanism reduces poisoning effects in NIDS by 30%. Similarly, another data sanitisation derivative is applied to malware detection systems to mitigate clean-label poisoning attacks (Ho et al., 2022b). This approach is an enhancement provided in (Venkatesan et al., 2021). Further, the study (Paudice et al., 2018b) proposed another approach to label sanitisation to reduce the impact of overfitting and underfitting. However, P. W. Koh and others (Koh et al., 2021) proposed three sophisticated poisoning attacks by introducing cluster-based poisoning that breached the sanitisation solutions highlighted above.

RONI is also a derivation of data sanitisation proposed by Chan et al. (2018a), which removes poi-

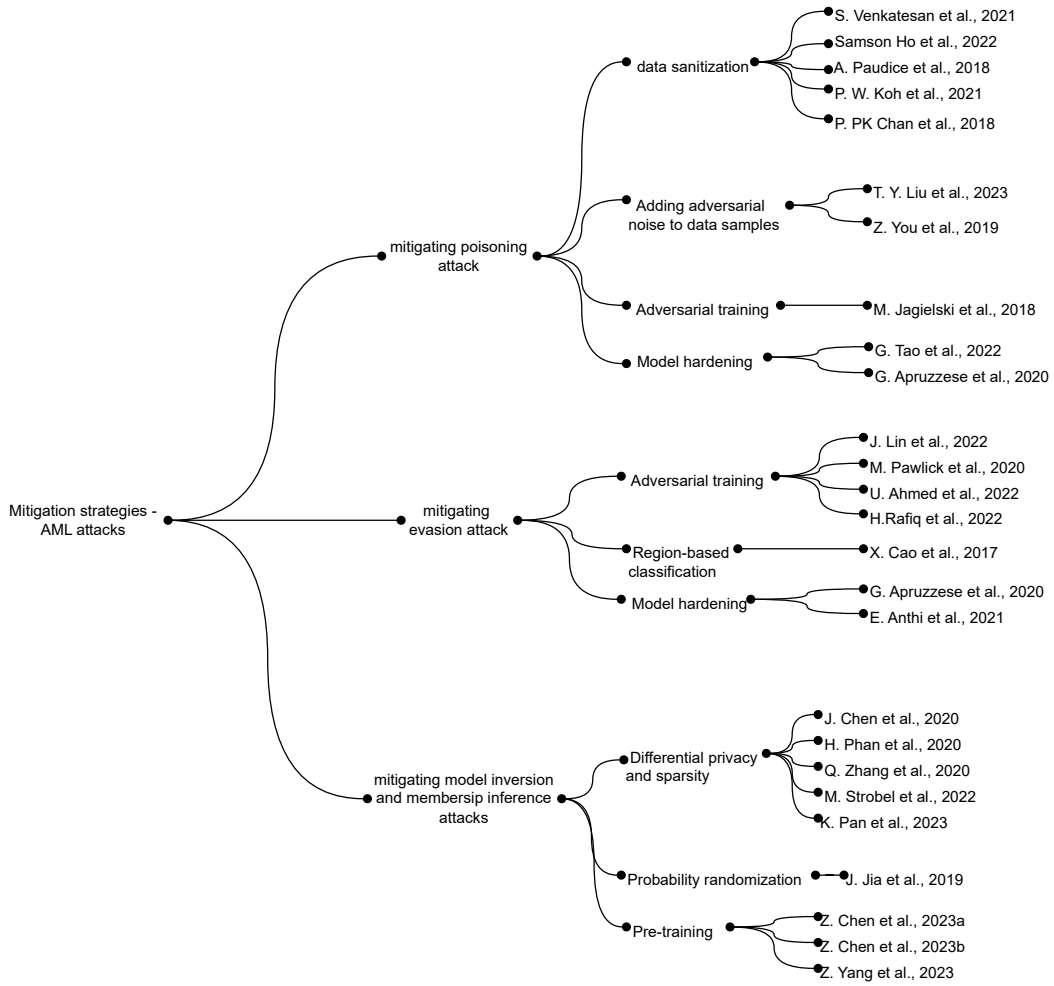


Figure 2.15: State-of-the-art of mitigation techniques - AML attack types

soned data samples by analysing their negative impact on classification accuracy. However, it also leads to underfitting issues that reduce the flexibility and increase false negatives at test time.

Adding Adversarial Perturbation Training the ML model with an adversarially developed dataset allows the trained model to identify poisoned samples at test time. Liu et al. (2023) have boosted the immunity of the model by adding specifically crafted noise samples in the dataset during training, which is effective against bulls-eye polytope, gradient masking and sleeper agent attacks. Another study (You et al., 2019) has introduced adversarial noise into the intermediate layer of CNN to mitigate FGSM attacks.

Adversarial Training Training an ML model with adversarial data samples allows it to be resilient against poisoning attacks. TRIM is one of the techniques used to adversarially train models with a residual subset of a dataset with a minimum error rate. Jagielski et al. (2018) have designed and experimented with this TRIM algorithm against adversarial poisoning attacks against the linear regression algorithm to solve optimisation problems. This approach has reduced the error rate to

approximately 6%. It performs robustly compared to Random sample consensus (RANSAC), a data sanitisation derivative. Whereas, TRIM and RONI security techniques failed against the subpopulation attack developed in (Jagielski et al., 2021).

Model Hardening Another innovative technique to mitigate poisoning attacks is model hardening, in which the model is trained until it leads to large class distances where it should not accept outliers. This technique makes it challenging for an adversary to poison the model. Tao et al. (2022a) proposed a model hardening mechanism with additional training to increase the class distances and challenge the label-flipping attack. The study (Apruzzese et al., 2020a) hardens the random forest algorithm to mitigate the poisoning impact on an IDS. Moreover, it can also help mitigate backdoor attacks against neural networks. It reduces misclassification up to 80%, but it is still only effective against label-flipping backdoor attacks.

2.3.2.6 Mitigating Evasion Attacks

Adversarial Training Adversarial training is a prominent mechanism to mitigate evasion attacks in ML. A particular dataset part is intentionally poisoned to lessen the test time evasion and make the model adversarially robust (Pawlicki et al., 2020). It allows the victim to be aware of adversarial samples if injected at test time to detect and defend itself if attacked by an adversary. Ahmed et al. (2022) proposed adversarial training by classifying adversarial and normal data samples, followed by centroid-based clustering of features and calculating the cosine similarity and centroid of the image vector. The research (Rafiq et al., 2023) trains independent models to reduce fabricated classification attacks and (Lin et al., 2022) secures against Carlini and Wagner and FGSM attacks.

Model Hardening The hardening ML model also applies to developing a wall of security in ML against adversarial attacks at test time. Evasion attacks are also mitigated with the help of a training model until they reach the state of hardening, which activates the model to evade adversaries and mitigate attack impact. Adversarially crafted samples are intentionally injected during the ML model training to evade the system until it reaches the state of hardening, making the victim model resilient and robust. These poisoned input data samples evade the system and are then marked as poisoned in the system to identify similar patterns if injected by the adversary at test time. Apruzzese et al. (2020b) have introduced a similar strategy to mitigate evasion attacks in botnet detection systems by deep reinforcement learning. They have developed an agent based on deep reinforcement learning capable of generating adversarial samples to evade the targeted botnet. Then, including these adversarially generated samples into the targeted system marked as malicious to make the model understand the pattern of adversarial samples if attacked during test time, whereas the research study (Anthi et al., 2021) used model hardening to secure an ML-based IoT system. A threshold is specified that trains the model properly with the legitimate and illegitimate datasets, which makes the botnet

detector robust against evasion attacks.

Region-based Classification Cao and Gong (2017) have designed a classification mechanism based on region rather than individual sample points. The researchers provided this technique based on the assumption that the adversarial points lie near the classification boundary. A hypercube-centered classification approach is determined by omitting single-point-based classification at test time to reduce the impact of adversarial points.

2.3.2.7 Mitigating Model Inversion and Membership Inference Attacks

Differential Privacy and Sparsity To preserve the privacy of ML models, one of the profound solutions is differential privacy. It makes it difficult for the adversary to analyse the output and extract the victim's confidential information. Chen et al. (2020) have used differential privacy applied with stochastic gradient descent on Lasso and CNN neural networks to preserve genomic data privacy. Phan et al. (2020) improve DNN robustness by implementing differential privacy with the logarithmic relation between the privacy budget and the accuracy of the targeted model. They have empirically analysed genomic data for phenotype prediction with a white-box attack. However, Zhang et al. (2020) improves differential privacy by implementing it at the class and sub-class level, proving the minimal probability of model inversion attack at the dataset only. Class and sub-class level differential privacy is more effective and robust than simple record-level differential privacy, providing more Euclidean distance between original and inverted data samples. However, it is tested with neural networks only with the Face24 and MNIST datasets. Also, this type of differential privacy requires high computational resources, whereas the study (Strobel and Shokri, 2022) highlights trade-offs of data privacy and assuring its trustworthiness. Pan et al. (2023) implemented differential privacy to mitigate privacy attacks and data leaks against generative adversarial networks. Whereas, the floating-point attack mentioned in (Jin et al., 2022) has invalidated differential privacy implemented to preserve the privacy of ML models.

Probability Randomisation Adversarial privacy attacks, specifically membership inference attacks, target ML classifiers and infer input datasets by interpreting the confidence score and probability of the queried output. Adding noise to the output or intentionally interrupting the confidence probability score leads to the privacy preservation of ML, preventing adversaries from inferring confidential details of the victim model. Membership inference guard (MemGuard) (Jia et al., 2019) is one of the solutions designed to preserve the privacy of ML models against membership inference attacks by adding randomised noise to each of the score vectors with a specified probability of accuracy loss, and makes ML-based binary classifiers resilient to mitigate membership inference attacks. However, the solution is only tested for securing neural networks under the black-box attack settings.

Pretraining Chen et al. (2023) have proposed a model-preserving framework to preserve the security

of deep learning models while training models by combining model parameters and training data. Chen and Pattabiraman (2023) have introduced a new framework to pre-train an ML-based model to preserve privacy by enforcing less confidence in the queried results between members and non-members. Z. Yang and others (Yang et al., 2023) have introduced another model to statistically in-distinguish the confidence scores of members and non-members.

2.4 Research Gaps in the State-of-the-Art

This state-of-the-art study reveals that data poisoning attacks are primarily examined in computer vision tasks using image datasets Ghosh et al. (2022), Zhong et al. (2020), Su et al. (2019). However, analysing poisoning impact in other domains focusing on traditional models such as RF, DT, and GNB is also important as these models are used in various real-world applications and often preferred when working with limited or unprocessed datasets Dixit et al. (2018), Sasmono et al. (2021), Kamath et al. (2018). Also, these models are a baseline for the advanced ML, such as neural networks for DL; therefore, understanding their poisoned behaviours helps improve mitigations for both ML and DL models.

In particular, traditional multiclass models have received limited attention in studies of data poisoning attacks Paracha et al. (2025a), Paracha et al. (2024b). Multiclass models play a critical role in addressing complex, multifaceted problems Lee et al. (2021), Rahman et al. (2023), Adarsh and Jeyakumari (2013). Analysing data poisoning attacks against these models provides a comprehensive understanding of poisoning impact under various development settings.

In addition, existing literature highlighted that most studies evaluate data poisoning attacks using standard performance metrics, including accuracy, recall, f1-score, and adversarial success rate to assess their impact Chen et al. (2021), Lu et al. (2024), Jagielski et al. (2018). However, it is crucial to understand how data poisoning alters the underlying decision mechanisms of these models, which is a limitation of existing work. Such deep analysis provides an understanding of the changing characteristics of the model, which helps improve and strengthen security-by-design solutions. Another identified limitation is that most studies have developed data poisoning attacks with full knowledge and capabilities of adversaries, whereas understanding their feasibility under constrained adversarial capabilities highlights their effectiveness and practicality against real-world applications.

Conversely, this analysis identifies several limitations in existing mitigation strategies for data poisoning attacks. A primary issue is that most solutions are either attack-specific or algorithm-specific. For instance, adversarial training (Ho et al., 2022a) is predominantly applied to deep neural networks, leveraging gradient loss during training, which restricts its applicability to models such as RF and GNB that do not utilise gradient-based optimisation. Evaluating the effectiveness of such methods remains unexplored in traditional models. Similarly, data sanitisation (Venkatesan et al., 2021) was

developed for network intrusion detection systems (NIDS), but its dependence on domain-specific thresholds limits its generalisability.

This thesis seeks to address these research gaps with a systematic evaluation of traditional ML models by first proposing deep behavioural analysis in Chapter 3, followed by evaluating the effectiveness of multiclass data poisoning attacks under limited adversarial capabilities in Chapter 4. Following this structural analysis, this thesis proposes an attack-agnostic and security-by-design solution in Chapter 5 to enhance the resilience and reliability of multiclass models.

2.5 Summary

This chapter presented a comprehensive state-of-the-art analysis of existing efforts in the domain of ML security. Following a detailed literature analysis, this chapter attempted to provide a good reference to researchers to gain insights into the domain of AML and understand the limitations of existing mitigation techniques against adversarial attacks. This chapter provided a systematic understanding of vulnerabilities across various threats, forming the baseline for evolving attacks and solutions. This chapter answered the RQ 1 from the theoretical aspects with this state-of-the-art analysis. Furthermore, this chapter analysed the security vulnerabilities in ML models under different adversarial capabilities and threat models to highlight the significance of such attacks. For example, data poisoning attacks are identified as a significant threat to training pipelines that can also be extended to initiate other attacks, such as inference attacks that leverage data poisoning. Moreover, existing mitigations are reviewed, revealing that most of the solutions are attack-specific solutions that are designed for specific attack vectors or applicable to limited datasets, domains, or systems. These solutions secure models from one attack, but adversaries can attack the model using another technique. The answer to RQ 1 is extended in the next chapter to practically analyse the security vulnerabilities of data poisoning attacks with limited adversarial knowledge and capabilities. .

Chapter 3. Investigating Machine Learning Behaviour Against Data Poisoning Attacks

3.1 Introduction

Chapter 2 covers the foundational concepts of ML and state-of-the-art adversarial attacks, including poisoning, evasion, inversion, and inference attacks. It highlights that ML models are inherently flexible and adaptive, making them vulnerable to adversarial manipulations. Among these attacks, data poisoning stands out for its practicality and prevalence. This chapter investigates fundamental threats posed by data poisoning attacks. While existing studies have typically explored these attacks with performance metrics, such as reduced accuracy or increased adversarial success rates, this study conducted a deep behavioural analysis. This study examined changes in learning dynamics, including altered model characteristics and shifted decision boundaries, to reveal how poisoning affects the decision mechanisms of individual models. This chapter answers RQ 1 by analysing how the underlying behaviour of ML models changes with the poisoned training. It also begins to answer RQ 2 by analysing the severity and impact of data poisoning attacks under limited adversarial capabilities. This chapter proposes a behavioural analysis to assess these changes and practically fulfill the objective 1 of the research. It interprets the variance in the ML model with poisoned training. It also compares the change in individual characteristics of models at varying poisoning levels to analyse their impact on the model training.

During the model training, the ML model develops a dynamic classification mechanism that leverages these models to understand the nature of new data and classify it. Such proliferation of ML models and their dynamic classification mechanism render their security fundamentals to the security of systems underpinned by them. Several poisoning techniques are available in the literature, such as (Geiping et al., 2020; Koh et al., 2022). Further, various sophisticated attacks are formulated to poison ML algorithms. Some of its successful examples are convex-polytope (Zhu et al., 2019b), label-flipping (Yerlikaya and Şerif Bahtiyar, 2022), bullseye-polytope (Aghakhani et al., 2021) and poison frog (Shafahi et al., 2018).

These data poisoning attacks manipulate training datasets in two ways. Firstly, the adversary can perturb features of the training dataset so that ML models misinterpret this data and wrongly develop their classification mechanism. Secondly, data poisoning attacks manipulate the classification labels of the training dataset, which blurs the decision boundaries of models. Data poisoning attack has a serious impact on various real-life applications, such as deceiving breast cancer diagnosis applications (Das et al., 2024), skin cancer diagnosis applications (Paracha et al., 2024b), and IoT-enabled smart city systems (Zhu et al., 2023c). To better mitigate these data poisoning attacks, it is significant to

understand the manipulated behaviour of these poisoned ML models.

Security analysis of ML algorithms has received significant attention in recent years, focusing on poisoning and other adversarial attempts. Specifically on data poisoning attacks, various research papers have conducted an empirical analysis of the performance of ML algorithms (Yerlikaya and Şerif Bahtiyar, 2022; Aryal et al., 2022). But these attempts do not provide the technical understanding of change in their classification behaviour and why data poisoning attacks are successful in corrupting their decision mechanisms, which is attempted for the first time in this research. The contributions of this chapter are given as follows:

- This chapter conducts a thorough behavioural analysis of ML classification algorithms against a label-flipping attack to analyse the correlation between the percentage of poisoned data and the impact on classification accuracy.
- This chapter analyses the impact of data poisoning on the design of ML algorithms and highlights sensitive factors for each algorithm.
- This chapter analyses the behaviour and impact of data poisoning on classification results with a noisy dataset. This chapter subsequently studied the impact of anisotropic features and imbalanced dataset distribution.

3.2 Related Work

This chapter first understands the fundamentals of data poisoning attacks, existing attack vectors, mitigating solutions, and the limitations of these solutions. Keeping ML models secure and integrated is a potential and active research challenge. Poisoning attacks, which involve contaminating datasets, pose significant risks to the integrity and confidentiality of ML systems. Notable examples of poisoning attacks are rethinking Label-Flipping (Xu et al., 2022), geometric algorithms for KNN poisoning (Centurion et al., 2023), and subpopulation data poisoning attack (Jagielski et al., 2021). These attacks have effectively compromised the integrity and intruded into the targeted models. However, there is a pressing need to understand the differential responses of these models to poisoning techniques.

Poisoning attacks are increasingly endangering the reliability and accessibility of ML models. Mayerhofer and Mayer (2022) have polluted a featured-extraction system developed with convolutional neural networks (CNN) and evaluated the effectiveness of poisoning attacks against it. Zhang et al. (2023) have fooled deep neural networks (DNN) by integrating manipulated features into them. Further studies, such as (Zhu et al., 2019b; Shafahi et al., 2018) manipulated ML with features set perturbation while Zhu et al. (2022) showed how specifically altered pixels could mislead classifiers. In another study (Chen et al., 2022), perturbed labels are proved to help insert a backdoor in ML

with a success rate of 98%. A brief comparison of existing similar work with this study is given in Table 3.1. Further Table 3.2 extends the analysis presented in Table 3.1, including a comparative analysis of this approach with existing data poisoning studies, highlighting the impact on performance of individual algorithms. This work aims to interpret the analytical reasoning of different supervised ML algorithms and their response to various poison levels. Although some research studies, such as (Yerlikaya and Şerif Bahtiyar, 2022; Aryal et al., 2022; Dunn et al., 2020), have conducted empirical analysis to show the performance degradation of ML, they fall short in explaining why models trained on identical datasets with the same poisoning rates exhibit divergent performance behaviours. Addressing this gap forms a central part of the contribution. Many existing research studies investigated the integrity violation and shift in ML decision-making, either deterministically or indiscriminately. However, the urge is to study the underlying model design that reveals poison differently for each ML algorithm and so to interpret its explainability against poisoning.

To distill ML models from the detrimental effects of poisoning attacks, a variety of mitigation strate-

Table 3.1: Comparative analysis with existing similar studies that provides an empirical analysis of poisoning attacks on machine learning along with their objectives/focus to conduct empirical analysis

S.No.	Source	Datasets used	Attack settings	Focus
1	Yerlikaya and Şerif Bahtiyar (2022)	Instagram fake spammer genuine accounts, botnet detection, android malware/benign permissions, breast cancer wisconsin	White box attack	Practical implication of data poisoning
2	Aryal et al. (2022)	Malware detection	Black box attack	Impact of data poisoning on Malware detection
3	Dunn et al. (2020)	ToN-IoT, UNSW-NB15	-	Impact of data poisoning on smart IoT systems
4	**This Behavioural Analysis**	BotDroid, CIC-IDS-2017, CTU-13, UNSW-NB15	Black box attack	Factual study to identify factors affecting the performance of each ML algorithm with data poisoning

Table 3.2: Performance analysis with existing similar studies analysing the impact of poisoning attacks on machine learning models

S.No.	Source	Performance Metrix	Model performance					
			SVM	RF	DT	KNN	GNB	Perceptron
1	Yerlikaya and Şerif Bahtiyar (2022)	Accuracy	69.33	68.16	-	74.08	56.83	-
		Precision	-	-	-	-	-	-
		F1-score	71.33	67.34	-	74.2	51.65	-
2	Aryal et al. (2022)	Accuracy	78.58	96.54	96.54	87.41	-	75.16
		Precision	74.45	93.04	93.54	82.48	-	68.58
		F1-score	73.51	95.9	95.88	85.12	-	72.57
3	Dunn et al. (2020)	Accuracy	-	79.22	-	-	73.11	76.91
		Precision	-	79.22	-	-	74.9	70.01
		F1-score	-	-	-	-	-	-
4	**This Behavioural Analysis**	Accuracy	48.5	77.02	62.5	64.56	47.5	71.2
		Precision	48.3	58.79	58.56	63.29	62.1	71.2
		F1-score	61.5	60	57.4	61.04	61.04	62.79

gies have been developed. A data sanitisation against an adversarial label-flipping attack is proposed in (Chan et al., 2018a). This method begins with random dataset clustering to train the model on

subsets of a dataset and then sanitises data points, taking into account the complexity of the data and its impact on model performance. Adversarial training Geiping et al. (2021), Tao et al. (2021), Qin et al. (2023) is also an effective technique to allow ML models to proactively learn poisoned patterns to guard against poison if injected. A brief overview of the discussed mitigation techniques is given in Table 3.3.

Where Drews et al. (2020) proposed an antidote to verify the robustness of a decision tree against

Table 3.3: Detailed synopsis of relevant existing defence solutions against data poisoning attacks along with the parameter targeted to develop mitigation strategy and algorithms on which the respected proposed solutions are experimented

Research paper	Mitigation technique	Exploited feature	Experimented algorithm
Chan et al. (2018a)	Data sanitisation with cluster training	Dataset features complexity vector	SVM
Baracaldo et al. (2017)	Data segmentation and filtering using provenance feature	Provenance feature	SVM, Logistic Regression
Paudice et al. (2018a)	Training classifiers for individual dataset classes and calculate outliers with Empirical Cumulative Distribution Function	Dataset outliers	Linear classifiers
Geiping et al. (2021)	Adversarial training to maximise adversarial loss to detect poisonous data points	No explicit feature is exploited	Neural networks
Tao et al. (2021)	Adversarial training to mitigate delusive attacks with different data distributions	Robust and non-robust features in data distributions	Neural Networks
Qin et al. (2023)	Adversarial training with data augmentation in unlearnable examples to maximise error	No explicit feature is exploited	Neural Networks

data poisoning attacks. Although these solutions are effective, they lack generalisability. In particular, the main contribution of this research study is to provide a rigorous examination of supervised ML behaviour against poisoning attacks. On a large scale, none of the studies have yet highlighted the reasons behind the different responses of ML algorithms when trained with the same poison levels.

3.3 Behavioural Analysis

This chapter proposes a behavioural analysis as the study of understanding the change in the classification behaviour of supervised models by implementing a data poisoning attack. The poison is implemented at various levels in order to analyse the behavioural changes at certain classification metrics such as feature importance score, decision boundary, k-neighbors, and individual algorithm properties. This deep behavioural analysis is conducted for six ML algorithms, analysing poisoning impact and correlation between poisoning levels and classification accuracy. Adopting an empirical approach, this analysis highlightes the practical feasibility of data poisoning, comprehensively analysing factors of individual algorithms affected by poisoning.

Real-World Detection Applications Although various mitigations have been proposed, this chapter discussed their limitations in Section 3.2. Given these limitations, this chapter conducts a be-

havioural analysis of poisoned intrusion and botnet detection systems in binary classification settings to understand the change in behaviour of detection mechanisms with poisoned training.

As digital systems evolve, cybersecurity applications have become essential in protecting data and infrastructure. These applications support various security-critical scenarios such as malware detection (Brown et al., 2024), intrusion detection (Wang et al., 2020), automated firewalls (Al-Haijaa and Ishtaiwia, 2021), and biometric recognition (Jhong et al., 2020). Among these, an intrusion detection system (IDS) monitors system behaviour or network traffic to detect anomalies caused by cyber attacks. It helps strengthen system security by detecting anomalous traffic and notifying the administration of potential threats. However, if adversaries evade IDS, attacks may compromise systems undetected.

The widespread use of open standards such as TCP/IP, ethernet, and web technologies allows digital systems to connect with enterprise networks, enabling interoperability and external connectivity. Nonetheless, this expands the threat surface, increasing the number of attacks and their sophistication. Significant incidents, such as the Mirai attack (Sinanović and Mrdovic, 2017), have been reported that use robot networks, known as botnets, to launch more sophisticated intrusions. Some prominent methods are distributed denial-of-service (DDoS) attacks, spam distribution, and data theft, known as botnet attacks.

Intrusion and botnet detection systems can be implemented using various methods, of which the simplest is the signature-based approach. It compares incoming requests to the database of known attacks, though it may fail to detect previously unseen attacks. To overcome this limitation, a data-driven approach is followed that derives rules by observing behaviour, for example, data mining techniques (Awajan, 2023).

Instead of developing signatures, the patterns in the given data are modelled to identify deviations from normal behaviour by using statistical techniques. These are known as prediction-based approaches, which can be implemented with ML algorithms. Training these detection systems with ML algorithms makes them adaptive to predict complex structured intrusions and zero-day attacks.

The training process of ML algorithms is vulnerable to data poisoning attacks, which can be extended to these systems. The first step is to understand the fundamentals of datasets to enable the behavioural analysis of these systems.

3.4 Fundamentals of Training Datasets

This section discusses fundamental concepts of ML training that are considered in the following behavioural analysis. In supervised ML, a dataset $D = \{(x_i, l_i)\}_{i=1}^{n}$ is a collection of structured data where each data instance is known as a data point. Each data point consists of a set of features x_i, representing measurable attributes from real-world observations and a corresponding label l_i. For

example, in an intrusion detection application, the features can be the source of the input, contents, and port, and the label indicates if an email is an intrusion(1) or not an intrusion(0). A dataset D is characterised by various properties that describe its complexity, structure, interconnections between features, class alignment, and outliers, described as follows:

3.4.1 Dataset Size and Distribution

The dataset size, represented by n samples in a dataset D, influences the learning capability and generalisability of an ML model. It allows the model to learn the underlying data distribution. According to the law of large numbers, the empirical risk $P_n(D)$ converges to the expected loss as the dataset size n increases, given in Eq. 3.1.

$$P_n(D) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(f(x_i), l_i) \tag{3.1}$$

where $\mathcal{L}(.,.)$ is the loss function. In an ideal dataset, all the classes or labeled data points are of equal size. However, when an adversary perturbs the dataset through label poisoning or altering certain features, these manipulations often change the data distribution. The changed data distribution enables the model to learn classes of variable size, which may lead to model bias. For example, consider training a binary classifier for IDS where the adversary manipulates the labels of 10% of intrusion (1) data points, changing them to normal (0). Therefore, the model may fail to learn intrusion characteristics completely, resulting in an increased false negative rate for various intrusions. However, it is interesting that the behavioural analysis observed different effects of the same data poisoning attack on different IDS. Also, the behavioural analysis observed a non-linear correlation between various poisoning levels and their impact on the targeted system. Thus, it is crucial to investigate the correlation between dataset size and data poisoning.

3.4.2 Feature Correlation

In supervised ML, feature correlation refers to a statistical relationship between two features in a dataset, usually measured with Pearson correlation for a linear relationship or monotonic relationship with Spearman correlation. High feature correlations can introduce redundancy, potentially increase model complexity, and impair feature importance scores. For example, it may degrade the performance of GNB, which assumes conditional independence of features, and such correlations may undermine the posterior probabilities. In hierarchical models like DT, strong feature correlation may lead to redundant splits, potentially increasing model variance and risk of overfitting. While low feature correlation is generally beneficial in linear and probabilistic models, the model should be evaluated under the specific context of the use case.

For example, consider training an IDS with a decision tree algorithm on a dataset where two features, x_i and x_j, are highly correlated. Since a decision tree uses greedy split based on information gain, it may select one over the other and learn the noise and specifics of training data rather than understanding underlying patterns that may introduce overfitting. In adversarial settings, data poisoning may disrupt feature correlations to manipulate the structure of the dataset. Thus, understanding the disruption introduced by data poisoning helps strengthen mitigations against it. On the other hand, it is also important to examine how different dataset structures and feature correlations influence data poisoning, which also helps improve mitigation solutions.

3.4.3 Dataset Outliers

Dataset instances that deviate significantly from the original distribution are known as outliers. Outliers may introduce training errors and increase classification loss. It particularly impacts distance-based models, such as KNN and SVM, which classify data based on distances. For example, it can shift the decision boundary between two classes in SVM, which relies on data points, known as support vectors, that are closest to the margin. When an outlier lies near the margin, it becomes a support vector that can distort the hyperplane between two classes and reduce the model's generalisability. Conversely, it minimally impacts hierarchical-based models, such as decision trees and random forests, because these models follow greedy splits based on feature importance scores rather than calculating distances between features. Biggio et al. (2011) disrupted the dataset features and generated outliers in it, thereby degrading the model's performance. Therefore, it is essential to understand how data poisoning generates outliers in training data and how these outliers affect models with various poisoning attacks under similar adversarial settings.

3.5 Approach to Behavioural Analysis

This approach is formulated on an untargeted attack with an underlying assumption that the adversary does not know any internal model settings and data distribution. The label-flipping attack is implemented because it is the simplest and popular attack, used in many research papers(Jebreel et al., 2024),(Mengara, 2024), in data poisoning and to keep the attack settings simple to understand models' behaviour clearly.

The attacking procedure is developed on a practical examination of data poisoning on ML classification models. An experimental study is conducted to analyse the impact of data poisoning on individual ML algorithms. The goal is to highlight optimal levels of poisoning that are difficult to interpret and lead to significant performance degradation in classifiers. Data poisoning is implemented by flipping labels randomly from 5%-25% of the dataset. The dataset is split with 75%-25%, where 75% of the data is used for training and 25% of the data is used for testing, as the ML model

accurately learn with atleast 75% of the dataset (Song et al., 2017). This analysis is significant in understanding the poisoning behaviour for developing profound mitigation solutions that proactively secure ML models from existing and novel attacks.

This chapter aimed to specifically study the correlation between the granular impact of poisoning on the performance of ML algorithms. It also studied the correlation between data poisoning and feature relations in the dataset. This analysis employs six supervised ML algorithms: Support Vector Machines (SVM), Decision Tree (DT), K-nearest neighbors (KNN), Random Forest (RF), Gaussian naive Bayes (GNB), and Perceptron. These algorithms are selected to cover all the baseline classification methods in ML. This helps analyse different classification behaviours in ML with four distinct datasets (BotDroid (Seraj, 2022), UNSW-NB15 (David, 2018), CTU-13 (Malik, 2022), and CIC-IDS-2017 (Chenthan, 2023)). These datasets are selected as they distinct in their structure, size and feature correlation.

3.5.1 Attack Method

The fundamentals of the attack method have been interpreted to poison binary ML classifiers in black-box attack settings. Definitions of the attack method and evaluation metrics to measure the poisoning effects and analyse behaviours of ML algorithms are also provided. The architectural overview of the attack model is given in Fig. 3.1, representing the attacked and clean model development and defining the analysis criteria. Steps 1-9 are given in Fig. 3.1, highlighting the attack development

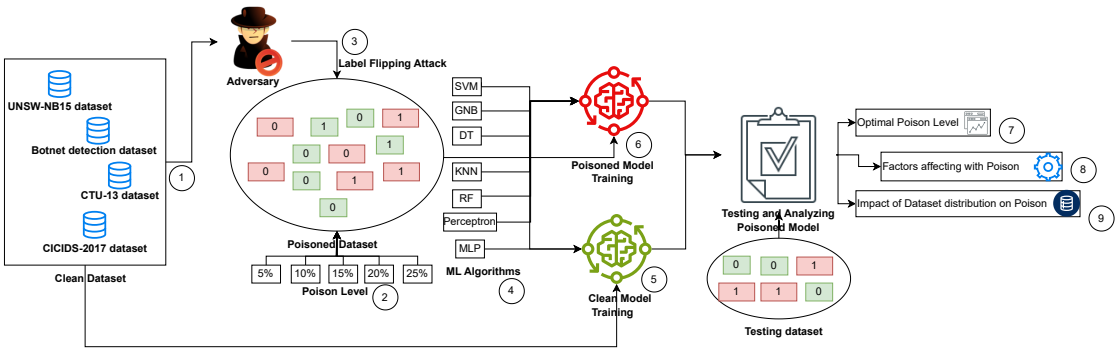


Figure 3.1: Architectural overview of the attack model, developed for conducting behavioural analysis

flow. Step 1 is the dataset selection, step 2 defines the poisoning levels to poison datasets, step 3 is the implementation of the label flipping attack on datasets with the defined poisoning levels, and step 4 defines algorithms to be considered for analysis. Steps 5 and 6 are the development of poisoned and cleaned models subsequently. Steps 7-9 define the analysis criteria. The objective of this research study is the behavioural analysis of poisoned ML algorithms. Binary classifiers are intended to be poisoned following the algorithm described in Algorithm 1. Each algorithm is audited to identify individual factors that are affected by dataset manipulation. The attack is developed in

black-box settings with a notion of generalisation to poison labeled datasets to degrade the classifier's performance. The sub-definitions of the attack are given as follow.

Algorithm 1 Poisoning Algorithm

```
Datasets used are: UNSW-NB15, BotDroid, CTU-13, CIC-IDS-2017
D_t ← Training dataset
D' ← Poisoned dataset = []
D_r ← subset of Training dataset
Record ← D_t(x_i, l_i) is the instance in dataset D_t with features x an label l at position i
ΔL ← Poisoning level = [0%, 5%, 10%, 15%, 20%, 25%]
for poison ∈ ΔL do
    Select D_r of length poison from D_t
    for Record ∈ D_r do
        if Record not in D' then
            if l_i equals 0 then
                l_i = 1
            else
                l_i = 0
            end if
        end if
        D' ← Record
    end for
end for
```

Poison Penetration Considering poison $L = [5\%,10\%,15\%,20\%,25\%]$, of training dataset, developed with algorithm A_{RFlip} as given in Alg 1 to manipulate data points x_i, by changing its label and generating poisoned data points x'_i, to disrupt ML model M is given in Eq 4.7.

$$M'_\theta = training(M(x'), x' = A_{RFlip}(D_o, \Delta L)) \tag{3.2}$$

which allows us to factorise poisoned model for behavioural analysis as given in Eq 3.3.

$$P_{Pen} = factorizing(M'_\theta) \tag{3.3}$$

where $M'_\theta \in$ [SVM, DF, RF, KNN, GNB, Perceptron] and *factorizing* is the interpretation method of analysing individual parameters affected with poison.

Performance Degradation Here, the test time performance statistics are calculated at each poisoning level to statistically analyse degradation as given in Eq. 3.4.

$$\begin{aligned} P_{Deg} &= \sum_{i}^{n=1} L_{i\to j}(A_{Acc}(M'_j) < A_{Acc}(M'_i)) \\ and\, P_{Deg} &\propto \frac{1}{A_{Acc}(M'_j)} \propto FPR(M'_j) \end{aligned} \tag{3.4}$$

where P_{Deg} is the performance degradation, injected with poison $L_i \to j$ at each incremental level from i to j. A_{Acc} is accumulated as the accuracy of the poisoned model M' whereas FPR is the

rate at which the poisoned model predicted falsified positive outcomes. The poison is injected with a minimum 5% poisoning rate, leading to a maximum of 25%, at a scale of 5. Binary classifiers are developed with four benchmarked datasets consisting of network traffic, described in Section 3.5.2, following the purpose of providing a precise and granular study, to put forward analytical reasons of how and why the performance of every algorithm is affected differently. For example, the performance (accuracy, precision, recall, f1-score) of DT, KNN, and RF is degraded linearly. Whereas the performances of neural networks are continuously fluctuating. The ablation study is given in Section 3.6.

3.5.2 Experimental Datasets

A rigorous ML analysis is conducted, developing intrusion detection classifiers with four benchmarked datasets, namely UNSW-NB15 (David, 2018), CIC-IDS-2017 (Chenthan, 2023), CTU-13 (Malik, 2022), and BotDroid (Seraj, 2022). UNSW-NB15 is based on captured network traffic with nine attacks included in it. It contains a total of 48 features containing packet-level information that enables us to conduct a detailed analysis for intrusion detection. Aligning it for binary classification, the records are classified into benign and malignant. BotDroid is the next dataset included, which comprises 45 features in total, and is encoded with one-hot encoding. CTU-13 is also included to develop a binary IDS classifier in this study which is developed to capture real-time traffic in thirteen different scenarios. and CIC-IDS-2017 is the last dataset included as part of this analysis. A brief description of all datasets is mentioned in Table 3.4. For the appropriate visualisation and features correlation in the dataset, the dataset complexity has been reduced to N dimensions with PCA, as given in Eq. 3.5.

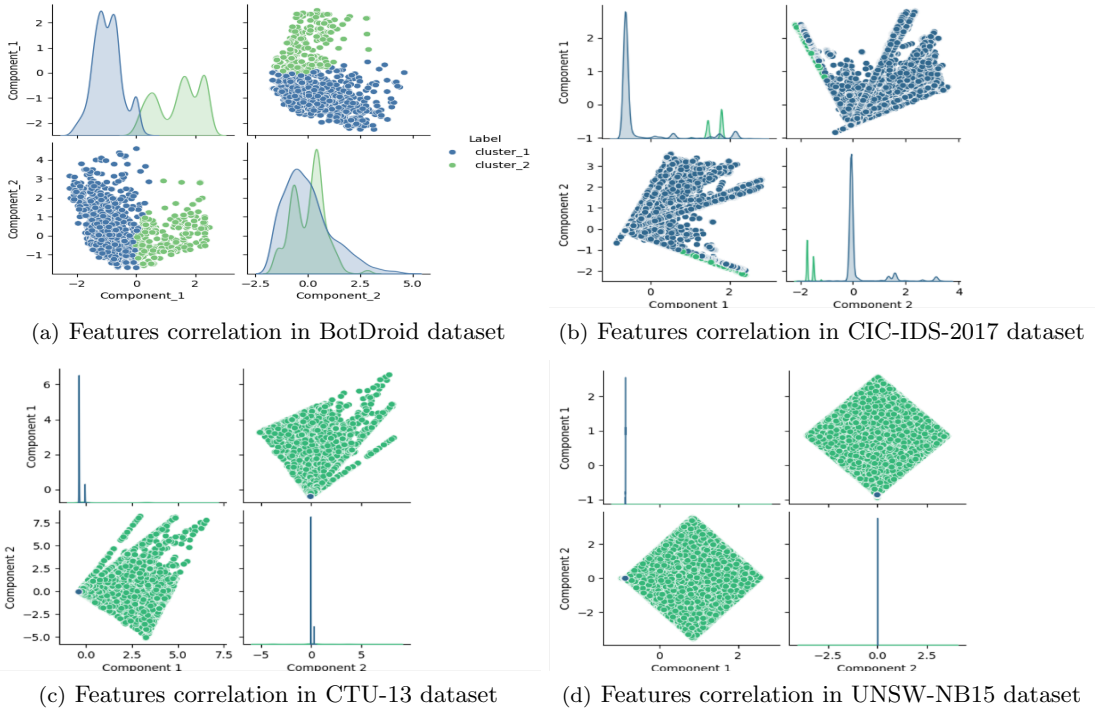
$$Dimentionality_Reduction = PCA(n_components = N) \tag{3.5}$$

And, for features structure and understanding, Gaussian Mixture Models(GMM) have been used.

Table 3.4: Dataset description used to perform data poisoning on machine learning algorithms

S.No.	Dataset	No. of features	No. of instances	Description
1	UNSW-NB15	48	82332	Dataset comprises of nine network intrusion attacks
2	BotDroid	45	1367	Dataset with limited instances, focusing BotDroid
3	CTU-13	57	92212	Captured with real time network traffic for BotDroid
4	CIC-IDS-2017	78	692703	Dataset consists of network packets

The visual dataset representation can be seen in Fig. 3.2. BotDroid dataset is the most appropriately distributed dataset with aligned features and minimal distortion(outliers) as shown in Fig. 3.2(a). Whereas UNSW-NB15 contains anisotropic features with asymmetric relation for which an uneven and elongated distribution is highlighted with sharp peaks at one axis in Fig. 3.2(d). However,



(a) Features correlation in BotDroid dataset

(b) Features correlation in CIC-IDS-2017 dataset

(c) Features correlation in CTU-13 dataset

(d) Features correlation in UNSW-NB15 dataset

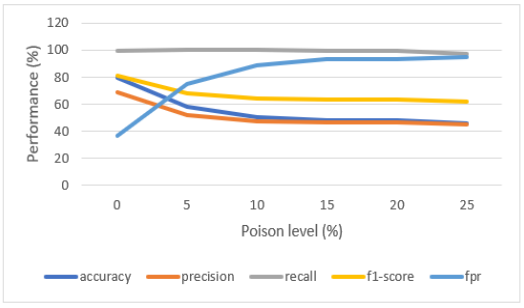
Figure 3.2: Gaussian mixture model visualisation of features relationship in dataset with principal component analysis reduction. Legend shown in (a) applies to all

Fig. 3.2(b) shows a symmetric correlation between some features, including a few with anisotropic covariance, with some overlapping outliers for CIC-IDS-2017. Fig. 3.2(c) although reflecting an uneven distribution, also contains some symmetric features with isotropic covariance. Conclusively, datasets with different characteristics help us in providing a profound behavioural analysis of ML against a data poisoning attack.

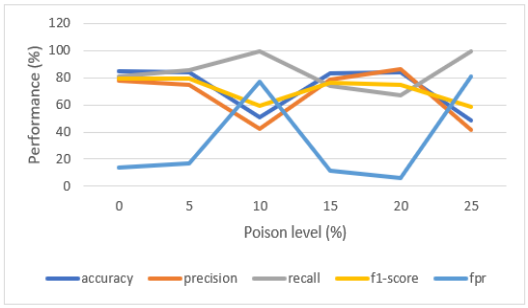
The individual characteristics of baseline ML models are assessed with cybersecurity benchmarked datasets in binary classification data poisoning settings. This study selected four cybersecurity application datasets for analysing data poisoning against security-critical applications. Also, these datasets consist of various features, sizes, and features correlation that help understand the classifiers' behaviour from various perspectives.

3.6 Experimentation Results and Analysis

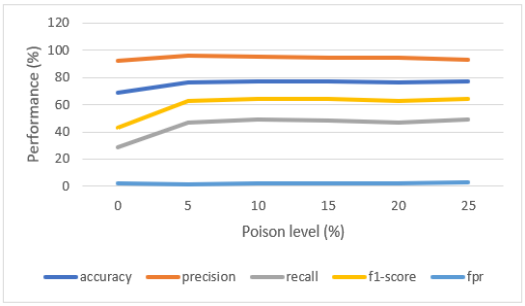
Following this approach and attack method, a deep behavioural analysis of six ML algorithms is conducted, trained with four datasets of varying sizes and structures. The accuracy, precision, recall, f1-score, and false positive rate are plotted against incrementing poisoning levels, as shown in Fig. 3.3 to Fig. 3.8. The experimentation results and analysis are given as follows.



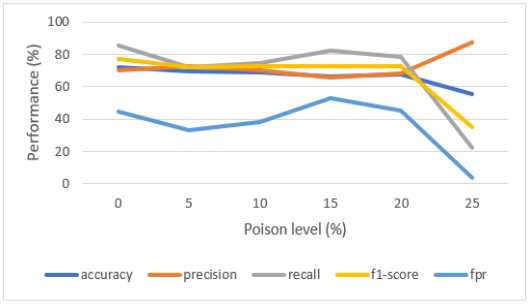
(a) Poisoning support vector machines with BotDroid dataset



(b) Poisoning support vector machines with CIC-IDS-2017 dataset

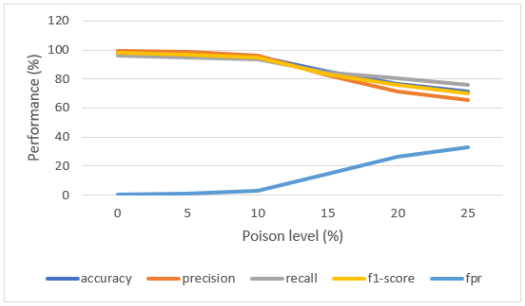


(c) Poisoning support vector machines with CTU-13 dataset

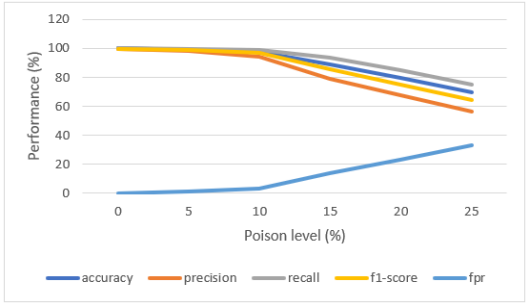


(d) Poisoning support vector machines with UNSW-NB15 dataset

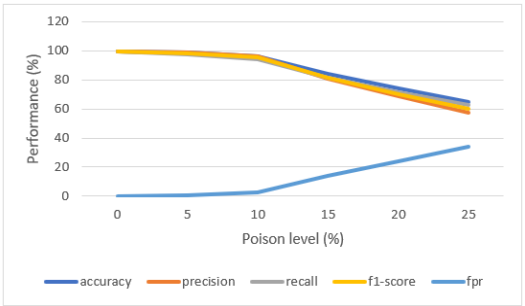
Figure 3.3: Performance analysis of support vector machines with consistent poisoning



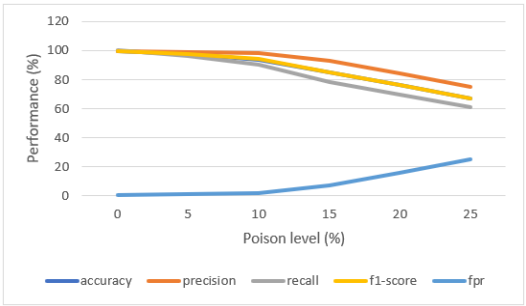
(a) Poisoning random forest with BotDroid dataset



(b) Poisoning random forest with CIC-IDS-2017 dataset



(c) Poisoning random forest with CTU-13 dataset

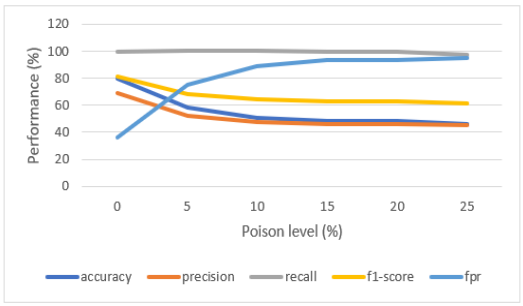


(d) Poisoning random forest with UNSW-NB15 dataset

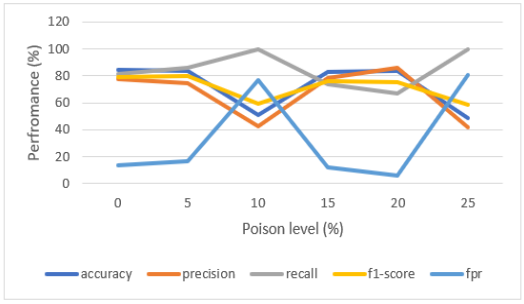
Figure 3.4: Performance analysis of random forest with consistent poisoning

3.6.1 Optimal Poisoning Level

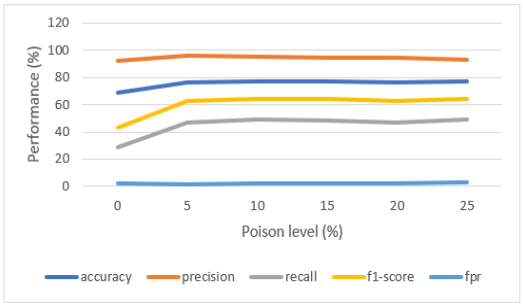
The results of the behavioural analysis first used to examined the impact of various poisoning levels on the underlying decision mechanisms of the models. This examination enable us to identify limits



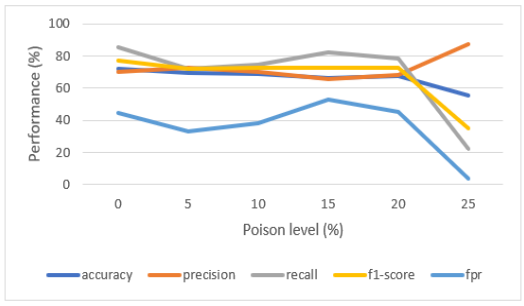
(a) Poisoning gaussian naive bayes with BotDroid dataset



(b) Poisoning gaussian naive bayes with CIC-IDS-2017 dataset

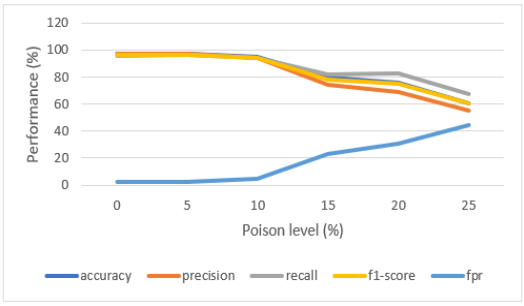


(c) Poisoning gaussian naive bayes with CTU-13 dataset

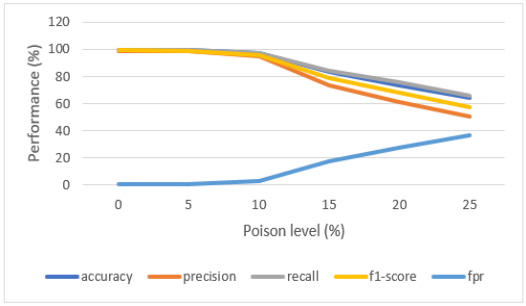


(d) Poisoning gaussian naive bayes with UNSW-NB15 dataset

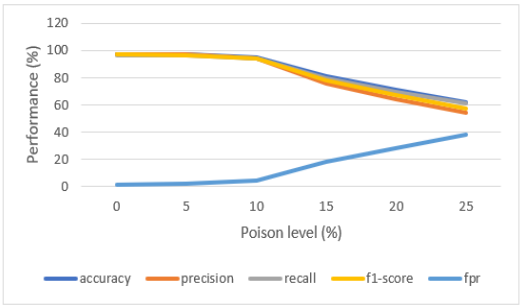
Figure 3.5: Performance analysis of gaussian naive bayes with consistent poisoning



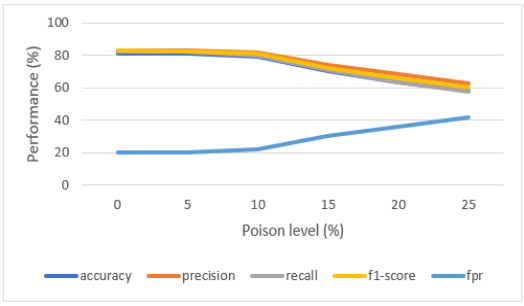
(a) Poisoning k-nearest neighbors with BotDroid dataset



(b) Poisoning k-nearest neighbors with CIC-IDS-2017 dataset



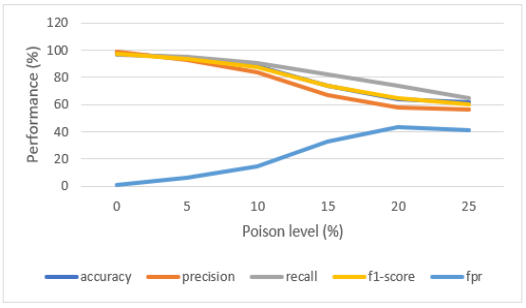
(c) Poisoning k-nearest neighbors with CTU-13 dataset



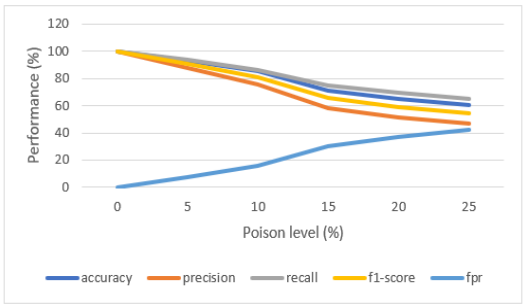
(d) Poisoning k-nearest neighbors with UNSW-NB15 dataset

Figure 3.6: Performance analysis of k-nearest neighbors with consistent poisoning

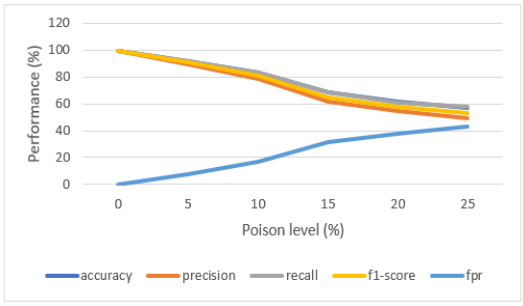
of the optimal poisoning levels. This identification will contribute to further research in enhancing testing scenarios, threat modelling and mitigation solutions against data poisoning attacks. The



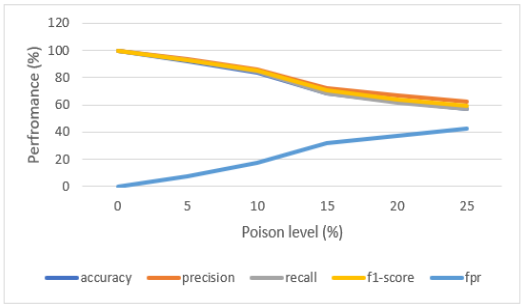
(a) Poisoning decision tree with BotDroid dataset



(b) Poisoning decision tree with CIC-IDS-2017 dataset

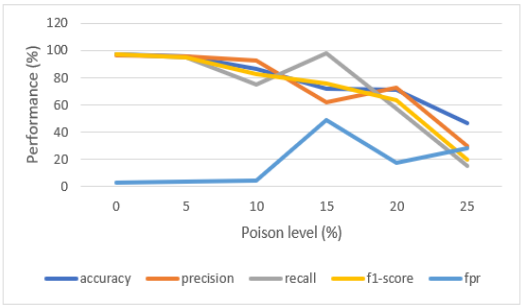


(c) Poisoning decision tree with CTU-13 dataset

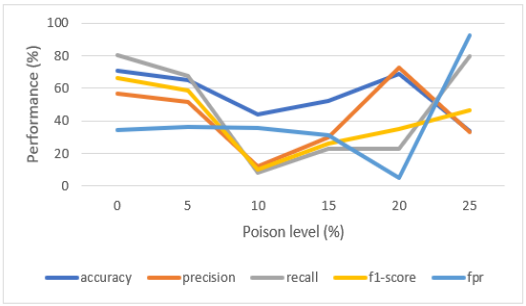


(d) Poisoning decision tree with UNSW-NB15 dataset

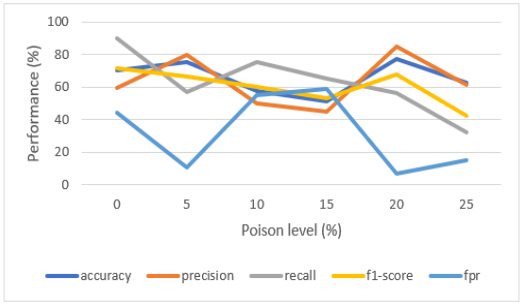
Figure 3.7: Performance analysis of decision tree with consistent poisoning



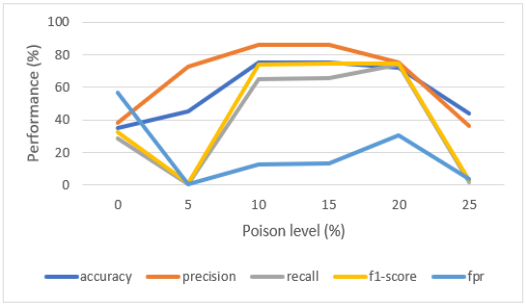
(a) Poisoning perceptron with BotDroid dataset



(b) Poisoning perceptron with CIC-IDS-2017 dataset



(c) Poisoning perceptron with CTU-13 dataset



(d) Poisoning perceptron with UNSW-NB15 dataset

Figure 3.8: Performance analysis of perceptron with consistent poisoning

analysis is given as follows.

Data poisoning of 10-15% is identified as optimal poisoning levels whilst analysing the models' performances. From the results, a sudden drift is visible between $10\% \leq \Delta L \leq 15\%$ of data poisoning

whereas $\Delta L = 5\%$ has put a negligible impact and $\Delta L > 15\%$ make algorithms over-fit. KNN trained with BotDroid dataset has the highest accuracy decrease of 15.48%, as shown in Fig. 3.6. Following the fact that it classifies based on the principle of information gain analysing maximum features similarities from the closest points. The attack randomly poisoned data points, which manipulate the relationship of the underlying features in the KNN model, making its decision boundary rough and irregular at $\Delta L = 15\%$, as shown in Fig. 3.9. However, DT is the most affected algorithm, with an

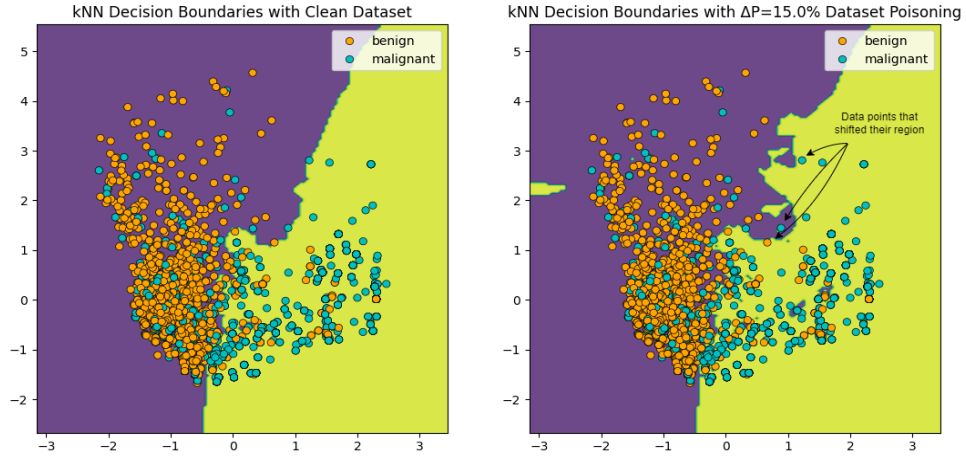


Figure 3.9: Change in decision boundary of k-nearest neighbors with 15% dataset poisoning

average accuracy degradation of 14.42% and degradation score of 4.33% at $\Delta L = 25\%$, irrespective of datasets. Table 3.5 presents the change in feature importance score, for poisoned DT, making important features anomalous, degrading its performance, specifically for less noisy BotDroid dataset.

Table 3.5: Features importance score - Decision tree where $\Delta L = (0\%, 10\%, 15\%)$

	Clean Dataset			Poisoned Dataset $\Delta L = 10\%$			Poisoned Dataset $\Delta L = 15\%$		
Dataset	Feature1	Feature2	Feature3	Feature1	Feature2	Feature3	Feature1	Feature2	Feature3
BotDroid	0.90	0.07	0.02	0.61	0.20	0.18	0.60	0.22	0.18
UNSW-NB15	0.43	0.29	0.22	0.33	0.34	0.32	0.33	0.34	0.32
CTU-13	0.19	0.32	0.49	0.26	0.34	0.40	0.26	0.34	0.39
CIC-IDS-2017	0.40	0.33	0.26	0.36	0.33	0.31	0.36	0.33	0.31

Table 3.6: Features importance score - Support vector machines where $\Delta L = (0\%, 10\%, 15\%)$

	Clean Dataset			Poisoned Dataset $\Delta L = 10\%$			Poisoned Dataset $\Delta L = 15\%$		
Dataset	Feature1	Feature2	Feature3	Feature1	Feature2	Feature3	Feature1	Feature2	Feature3
BotDroid	0.53	0.80	0.23	0.42	0.80	0.40	0.18	0.98	0.03
UNSW-NB15	0.81	0.56	0.10	0.72	0.32	0.61	0.30	0.72	0.61
CTU-13	0.13	0.50	0.85	0.06	0.52	0.84	0.38	0.21	0.89
CIC-IDS-2017	0.07	0.99	0.11	0.53	0.69	0.47	0.45	0.79	0.39

Further analysis interpreted that parametric algorithms including SVM, GNB, and Perceptron, are less affected by data poisoning when dataset features are symmetric and become saturated when $\Delta L \geq$

15%. But these algorithms become sensitive and over-fitted when dataset features are asymmetric and $\Delta L \geq 15\%$. For example, the margin score of SVM is minimally changed from 0.0017 to 0.0013 when ΔL=10% and 15% respectively. Features importance scores of SVM are also minimally affected by poisoning as given in Table 3.6 where features space is reduced following Eq 3.5. So poisoned and cleaned SVM classifiers follow the same importance of features for making classification decisions except for the UNSW-NB-15 dataset for which Feature1 with the highest importance score has become an anomaly at $\Delta L = 15\%$, due to irregular features relations. Explaining further, SVM in Fig. 3.3(b) and GNB in Fig. 3.5(b) shows a high decrease of 35.14% in accuracy with 74% of increment in false positive rate but simultaneously, recall also increases to 33.1% between poisoning rate of 20-25%, highlighting over-fitting of the models. This study also calculated the class probabilities of poisoned GNB to interpret their classification decision as given in Table 3.7 which has a minimal impact on its classes, leading to no change in its decisions. Conclusively, Perceptron losses its accuracy to 50.98%

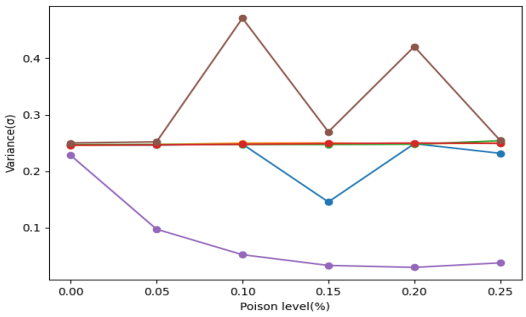
Table 3.7: Analysing class probabilities of gaussian naive bayes with poisoned dataset

Dataset	Clean Dataset		$\Delta L = 10\%$		$\Delta L = 15\%$	
	Class0	Class1	Class0	Class1	Class0	Class1
BotDroid	0.55	0.45	0.54	0.46	0.52	0.48
UNSW-NB15	0.45	0.55	0.46	0.54	0.47	0.53
CTU-13	0.57	0.43	0.56	0.44	0.54	0.46
CIC-IDS-2017	0.64	0.36	0.61	0.39	0.58	0.42

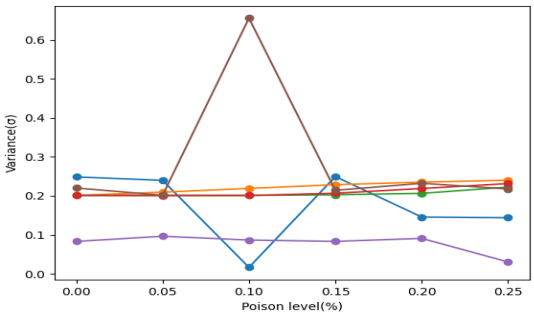
with the BotDroid dataset at $\Delta L = 25\%$ followed by DT, whose accuracy is reduced to 42.74% with the UNSW-NB15 dataset. Also, an interesting relation between poison and dataset noise is revealed where $\Delta L \ \frac{1}{\propto} \ Dataset \ Noise$ and working as a catalyst at $\Delta L = 25\%$. So, ΔL between 10%-15% are identified as optimal and effective poisoning rates, particularly affecting KNN followed by DT however minimally penetrating SVM and GNB. The variance of the trained models is given in Fig. 3.10(a) to 3.10(d). Whereas, the adversarial success rate (ASR) of these poisoned models is given in Tables 3.8 to 3.11.

Table 3.8: Adversarial success rate with BotDroid dataset

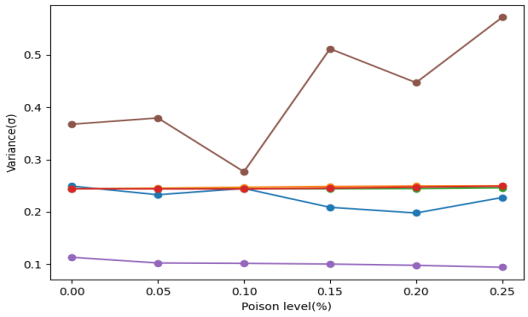
Algorithm	$\Delta L = 5\%$	$\Delta L = 10\%$	$\Delta L = 15\%$	$\Delta L = 20\%$	$\Delta L = 25\%$
SVM	0.2	0.29	0.31	0.31	0.33
DT	0.097	0.13	0.22	0.35	0.33
RF	0.025	0.065	0.15	0.21	0.22
KNN	0.01	0.021	0.16	0.19	0.33
GNB	0.2	0.29	0.31	0.31	0.33
Perceptron	0.055	0.11	0.24	0.25	0.52



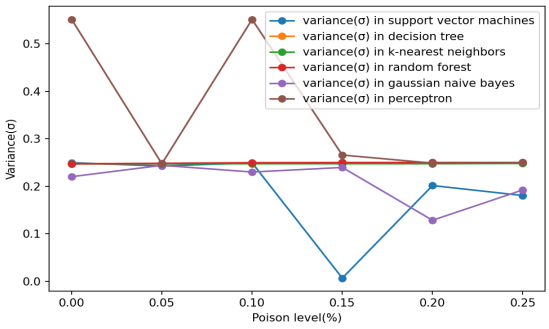
(a) Variance of poisoned models with BotDroid dataset



(b) Variance of poisoned models with CIC-IDS-2017 dataset



(c) Variance of poisoned models with CTU-13 dataset



(d) Variance of poisoned models with UNSW-NB15 dataset

Figure 3.10: Variance analysis of trained models with consistent poisoning. Legend shown in (d) applies to all

Table 3.9: Adversarial success rate with UNSW-NB15 dataset

Algorithm	$\Delta L = 5\%$	$\Delta L = 10\%$	$\Delta L = 15\%$	$\Delta L = 20\%$	$\Delta L = 25\%$
SVM	0.034	0.071	0.10	0.12	0.15
DT	0.12	0.16	0.27	0.35	0.39
RF	0.03	0.06	0.11	0.22	0.28
KNN	0.0074	0.02	0.1	0.17	0.22
GNB	0.034	0.071	0.10	0.12	0.15
Perceptron	0.063	0.4	0.39	0.33	0.08

3.6.2 Vulnerable Parameters

Following the analysis of optimal poisoning levels, this behavioural analysis contributes to analyse vulnerable parameters of individual ML algorithms against data poisoning attacks. Understanding the relationship between data poisoning and individual algorithms help develop strong resilience against such attacks which are independent of attack.

This behavioural analysis study individual factors affecting the selected poisoning attack to study the behaviour of individual algorithms. Overall, DT and KNN are prone to data poisoning. DT comprises a tree data structure following a probabilistic approach to structure decision nodes, whereas this attack initiates a wrong features split that supports misleading classification by manipulating feature importance as shown in Table 3.5. The feature space of datasets has been reduced for simplicity

Table 3.10: Adversarial success rate with CTU-13 dataset

Algorithm	$\Delta L = 5\%$	$\Delta L = 10\%$	$\Delta L = 15\%$	$\Delta L = 20\%$	$\Delta L = 25\%$
SVM	0.06	0.08	0.07	0.09	0.06
DT	0.07	0.16	0.28	0.36	0.39
RF	0.01	0.03	0.18	0.21	0.35
KNN	0.009	0.02	0.16	0.26	0.35
GNB	0.06	0.08	0.07	0.09	0.06
Perceptron	0.07	0.12	0.19	0.09	0.03

Table 3.11: Adversarial success rate with CIC-IDS-2017 dataset

Algorithm	$\Delta L = 5\%$	$\Delta L = 10\%$	$\Delta L = 15\%$	$\Delta L = 20\%$	$\Delta L = 25\%$
SVM	0.03	0.33	0.03	0.007	0.36
DT	0.06	0.14	0.30	0.36	0.39
RF	0.001	0.02	0.08	0.20	0.27
KNN	0.003	0.02	0.18	0.29	0.33
GNB	0.03	0.33	0.03	0.007	0.36
Perceptron	0.03	0.26	0.17	0.02	0.34

with PCA reduction following Eq. 3.5. Also, a direct relation between poisoning and performance degradation has been analysed. KNN is non-parametric and groups data based on feature similarities, making it highly susceptible to poisoning. This attack changes the feature space of the part of the dataset, which disrupts the decision boundary and degrades its performance. Whereas, an inverse relation is identified between the number of neighbors and poisoning level, where increasing k-neighbors normalises the poisoning effects on the KNN classifier, given in Table 3.12.

Table 3.12: Analysing k-neighbors affecting k-nearest neighbors accuracy with $\Delta L = (0, 10, 15, 25)\%$

	Poison Level	k=3	k=5	k=10	k=15
BotDroid	$\Delta L = 0\%$	97.54	96.80	96.92	96.68
	$\Delta L = 10\%$	95.94	96.80	96.31	96.68
	$\Delta L = 15\%$	83.78	90.05	95.57	94.47
	$\Delta L = 25\%$	69.28	70.76	87.22	88.69
CTU-13	$\Delta L = 0\%$	97.80	97.55	96.94	96.55
	$\Delta L = 10\%$	95.06	96.52	96.78	96.50
	$\Delta L = 15\%$	87.13	90.90	94.54	95.95
	$\Delta L = 25\%$	72.0	76.14	83.68	87.52
UNSW-NB15	$\Delta L = 0\%$	81.57	81.17	80.92	80.71
	$\Delta L = 10\%$	79.85	80.36	80.63	80.63
	$\Delta L = 15\%$	74.17	76.46	79.06	80.04
	$\Delta L = 25\%$	64.48	66.85	70.19	74.62
CIC-IDS-2017	$\Delta L = 0\%$	99.61	99.57	99.49	99.42
	$\Delta L = 10\%$	96.21	98.84	99.42	99.41
	$\Delta L = 15\%$	88.70	92.93	96.79	98.91
	$\Delta L = 25\%$	73.49	78.06	87.09	90.43

Because by inverting data labels, decision boundaries are getting blurred resulting in disjunctive classes split and hence more affected with poison. Whereas, SVM and GNB follow parametric learning to develop an optimal hyperplane and the parameters values and then become saturated, which makes it independent of dataset size and robust to poisoned data when $\Delta L \geq 15\%$. In Table 3.13, the

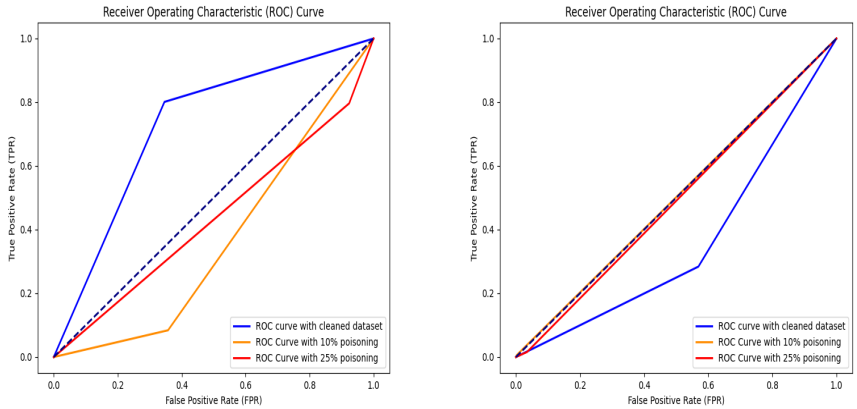
Table 3.13: Analysing support vector machines margin score for different datasets with $\Delta L = (0, 10, 15)\%$

Dataset	$\Delta L = 0\%$	$\Delta L = 10\%$	$\Delta L = 15\%$
BotDroid	0.008	0.00047	0.006
CTU-13	0.000007	0.00002	0.0000003
UNSW-NB15	0.00010	0.00015	0.000019
CIC-IDS-2017	0.00003	0.000015	0.000018

Table 3.14: Analysing one-to-one relation between poison and various parameters of ML algorithms

Algorithm	Algorithmic Parameters	Relation to ΔL
SVM	Margin score	Minimal impact
	Decision boundary	Minimal impact
	Features importance score	Minimal impact
DT	Features importance score	High impact
	Asymmetric features space	High impact
KNN	Decision boundary	High impact
	k-neighbors	Inverse impact
GNB	Decision boundary	Minimal impact
	Class probabilities	Minimal impact
RF	No. of trees	Inverse impact
Perceptron	Weights	High impact

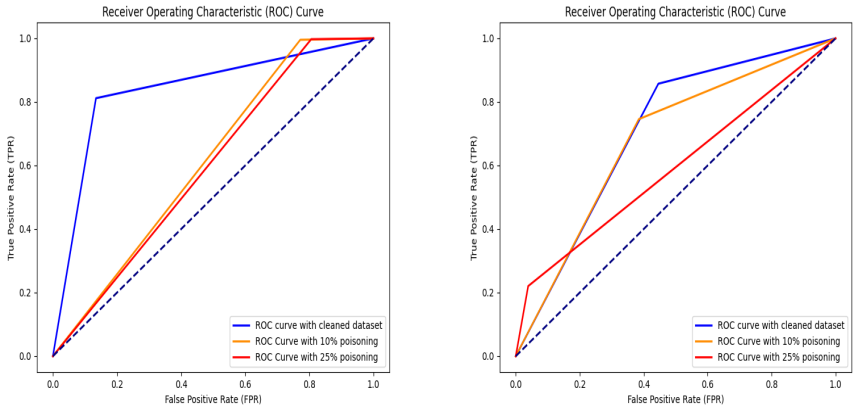
minimum margin scores of cleaned and poisoned SVM classifiers have been calculated to analyse the change in their decision boundary, which shows minimal change with little to no impact on their decision formulation. Also, looking at Fig. 3.3(a) and Fig. 3.3(c), SVM classifiers performance becomes saturated at $\Delta L \geq 15\%$ for BotDroid and CTU-13 datasets. But for UNSW-NB15 and CIC-IDS-2017 which comprises anisotropic features, SVM becomes sensitive and over-fit when $\Delta L \geq 15\%$, leading to inappropriate and continuous fluctuations in their learning parameters as shown in Fig. 3.3(b) and Fig. 3.3(d). A very similar behaviour is visible for GNB and Perceptron can be seen in Fig. 3.5 and Fig. 3.8.



(a) Poisoning perceptron with CIC-IDS-2017 dataset

(b) Poisoning perceptron with UNSW-NB15 dataset

Figure 3.11: ROC curve of perceptron with consistent poisoning



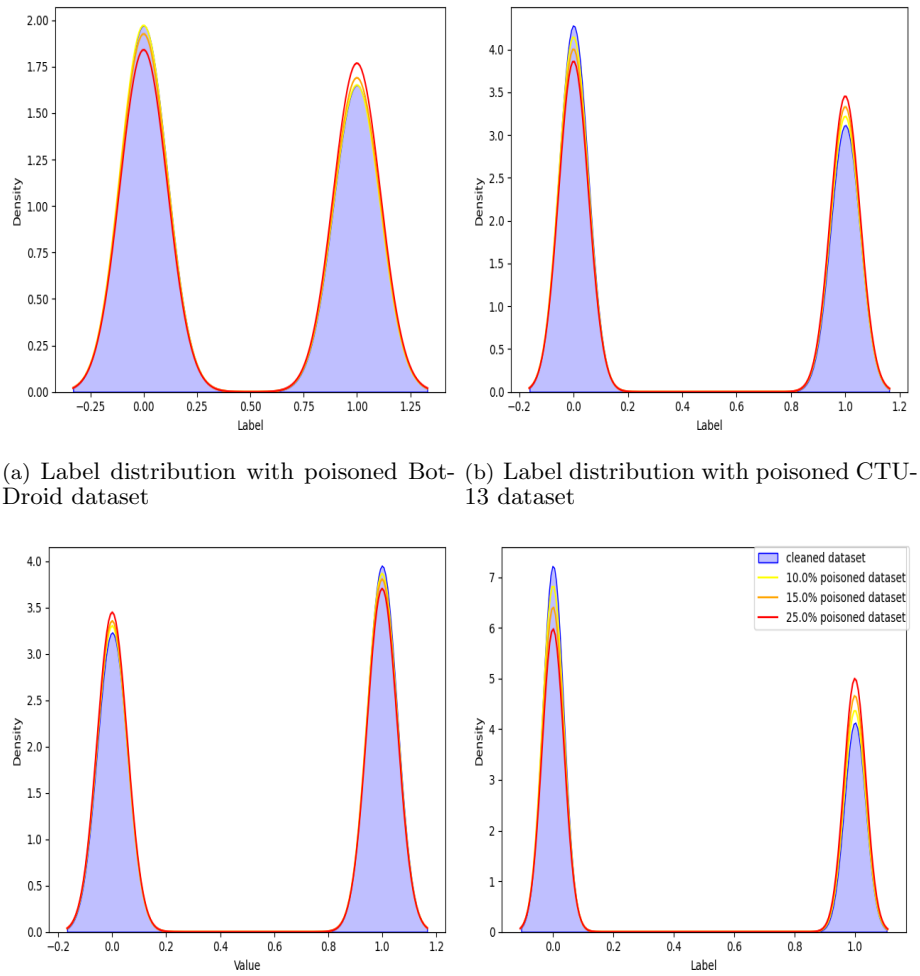
(a) Poisoning gaussian naive bayes with CIC-IDS-2017 dataset

(b) Poisoning gaussian naive bayes with UNSW-NB15 dataset

Figure 3.12: ROC curve of gaussian naive bayes with consistent poisoning

3.6.3 Imbalanced Data Distribution

The behavioural analysis also determine the link between various structures of the datasets and the impact of data poisoning attacks on the poisoned model. This understanding allows strengthen threat models and testing and analysing ML against data poisoning attacks. Data poisoning is highly impacted on DT followed by KNN, irrespective of dataset noise and imbalanced data distribution as shown in Fig. 3.7 and Fig. 3.6, respectively. Whereas, interpreting Fig. 3.4, Random Forest (RF) is found to be most effective and resilient against data poisoning attacks although a non-parametric algorithm. Because RF makes decisions calculating the mean from all of its trees, which normalises the data poisoning effects. However, particularly for intrusion detection, KNN is immune to poison specifically at 10% although a high drift in performance can be seen at 25% dataset poisoning. Fig. 3.13 provides an overview of the change in dataset distribution when labels are attacked with the label-flipping attack at different poisoning levels. Looking at classification performances in Fig. 3.11(a) and Fig. 3.11(b), dataset noise and asymmetric features correlating with poison making Perceptron sensitive to classify intrusions resulting in an unstable classifier. For parametric algorithms i-e. SVM, GNB, and Perceptron, poison in an erroneous dataset works as a catalyst for performance disruption. For example, Fig. 3.12(a), Fig. 3.12(b) and Fig. 3.11(a) shows a significant decrease in TPR concurrently increasing FPR with 10% poisoned dataset for GNB and Perceptron, respectively. From this analysis, Table 3.14 provides a relative impact of ΔL on individual parameters of ML algorithms. Implications of the behavioural analysis and its future directions are given in Section 3.7.



(a) Label distribution with poisoned BotDroid dataset

(b) Label distribution with poisoned CTU-13 dataset

(c) Label distribution with poisoned UNSW-NB15 dataset

(d) Label distribution with poisoned CIC-IDS-2017 dataset

Figure 3.13: Analysing change in dataset distribution D_{dis} with label poisoning using *kernel density estimation* function at an incremental poison levels ΔL=(0%, 10%, 15%, 25%). Legend shown in (d) applies to all

3.7 Implications of Behavioural Analysis

This behavioural analysis helped strengthen the explainability of the model by understanding the change in behaviour of the model when trained with a poisoned dataset. The shifts in decision boundaries of the models are analysed, the change in their individual characteristics with varying poisoning levels and the impact of various features correlations in poisoning models. The implications of the deep behavioural analysis are provided as follows:

3.7.1 Improving Mitigations

Extending Table 3.3, data sanitisation and adversarial training are the two most effective techniques to safeguard against data poisoning attacks. Adversarial training is developed with the gradient of the data point which is effective for neural networks only. It can not enhance the security of

baseline ML models including SVM, DT, RF, GNB, and KNN. However, data sanitisation filters the dataset's ambiguity to clean the classifier's training by removing doubtful dataset features or outliers. These sanitisation techniques are limited to be developed on the dataset features only. None of the techniques focus on the classifiers' inherent parameters and their development mechanisms. Furthermore, Cinà et al. (2023) highlighted the limitations of existing mitigation techniques and the importance of the generalisability of ML models. Another research study (Paracha et al., 2024b) experimentally proved the limitations of adversarial training in security baseline models. Considering these limitations, it is important to understand the underlying decision model development and the impact of poisoned data points on it.

To enhance the security of ML models and their generalisability against data poisoning, it is crucial to study the behaviour of the inherent characteristics of the models. The results highlighted potentially vulnerable parameters of individual algorithms and their susceptibility to poisoning at various levels. Also, highlighting the most pervasive poisoning levels allows the development of potential security solutions in this regard. Including this behavioural analysis in data sanitisation and in setting model parameters will better generalise the model. Also, this behavioural analysis will help strengthen the explainability of the model's decision. Considering future research directions, the behavioural analysis will be highly effective in developing adversarial training techniques for baseline ML models other than perceptron. Conclusively, the results of this chapter are discussed in Section 3.8.

3.8 Discussion and Limitations

There are various solutions proposed in existing literature to mitigate data poisoning attacks such as data sanitisation (Chan et al., 2018a), data filtration based provenance features to remove poisoned data points (Baracaldo et al., 2017), or mitigating poisoning impact by detecting outliers (Paudice et al., 2018a). Almost every solutions designed mechanism to improve the performance of the given model though none has provided a mechanism to understand hopw poisoned data points are changing the underlying classification mechanisms of the model. To develop strong and resilience mitigation mechanisms, it is important to understand the changing behaviour and capabilities of ML models with various types of data poisoning attacks. The deep behavioural analysis provides an in-depth understanding of how poisoned training can change the underlying decision mechanisms of ML models. This research provides a further detailed synopsis of technical uncertainties in the model's decisions created with a poisoned dataset to help strengthen mitigation solutions. As the model's decision mechanisms are underexplored in existing solutions, ML models are still susceptible to new data poisoning attacks that can breach existing mitigations such as data sanitisation or adversarial training, as highlighted in Section 3.7.

To understand the changing behaviour of ML models trained with poisoned datasets, a deep be-

havioural analysis is conducted to analyse the impact of poisoned data on the decision mechanisms of the models. This analysed the change in the decision boundaries of the model, identified how the poisoned data is impacting individual characteristics of the model and what are the optimal poisoning levels up to which the poison has a high impact and is difficult to detect. This study also analysed the relationship between data poisoning attacks and various structures of the datasets to understand the impact of poison with varying datasets.

The results highlight certain affected factors with label poisoning, analysing individual ML algorithms. Such as segregating nodes hierarchy development in DT and RF, misplacing data points in KNN and SVM, and probability misinterpretation in KNN and GNB. Overall RF is resilient to data poisoning because it may create more hierarchical nodes within its trees with poisoned data points, averaging their prediction probabilities dilutes misclassification. Although DT is the most affected algorithm because flipped labels make it difficult to disjoint nodes considering features. Also, the results showed that a 10%-15% poisoned dataset is more impacting whereas, after 25% poisoning, the sensitivity of a model is getting increased.

These results highlight that noise in the dataset works as a catalyst for data poisoning. The more noisy the dataset, the more adverse impact can be achieved in performance degradation against classification results because it filters poison (intentional noise) from dataset noise. Also, non-linear features in the dataset help poison to better penetrate the classifiers. Whereas, cleaning datasets and feature engineering can help mitigate data poisoning in this scenario.

Based on the results and implications of the deep behavioural analysis, this study has been extended to further understand the adversarial impact of data poisoning attacks with novel data poisoning attacks. A new data poisoning attack has been proposed, which exploits a significant characteristic of ML models called outliers. This deep behavioural analysis helped define a threat model with limited adversarial capabilities to experiment the outlier poisoning attack in Chapter 4 then extended it to identify the impact of the attack on security-sensitive applications.

This thesis has conducted a deep behavioural analysis of supervised ML models; however, this analysis can be applied to assess other ML and DL models where practitioners need to analyse the resilience of ML against data poisoning attacks. It is also helpful in auditing training time mitigations against data poisoning attacks.

This behavioural analysis is limited to the practical implication and analysis of supervised classification algorithms for binary classifiers. This limitation helps to provide a detailed and thorough technical study of supervised ML classifiers and their parameters that are affected by poisoning. Unsupervised ML can not be covered in this research as this study focused on label poisoning and has contributed to analysing decision boundaries of classifiers and resultant metrics, including accuracy, precision, f1-score, and recall. For the in-depth analysis, the variance and ASR of the baseline models

are also examined and compared with the benign models to understand the disrupted behaviours. This chapter only focused on the baseline ML models, which are the foundation of complex deep neural networks and advanced models. Regression algorithms are also out of the scope of this study. However, this behavioural analysis can be extended to multiclass classifiers, which is a limitation of this chapter. Furthermore, it is important to conduct this behavioural analysis in security-sensitive applications with real-world case studies for a rigorous analysis. Chapter 4 addresses this limitation and conducts a detailed study of analysing data poisoning impact in multiclass classification models.

3.9 Summary

This chapter highlighted an important aspect of understanding the impact of data poisoning attacks on the decision mechanisms of the ML model. For achieving this purpose, a deep behvaioural analysis of six classification algorithms is conducted to analyse poisoning impact on the classification behaviour of these models and their characteristics. The results of this chapter strengthen the answer to RQ 1 by providing practical understanding to analyse security vulnerabilities of ML exploited by data poisoning attacks. This chapter extended the answer to RQ 1 by experimenting data poisoning attacks in realistic attack settings by providing minimal knowledge of the targeted system to the adversary. This chapter also addressed RQ 2 by highlighting potential risks of data poisoning to ML applications with limited adversarial capabilities and the impact of poisoning in such scenarios. The deep behavioural analysis identified the optimal poisoning levels that have a high impact on the model and are difficult to detect. Furthermore, the relationships between varying data structures in the dataset, the correlation of features in the dataset and the impact of data poisoning are also determined. The technical insights are provided that 10%-15% poisoning to the dataset is optimal whereas less than 10% has minimal impact and more than 25% introduces sensitivity and 50% of data poisoning leads to overfitting. Different behaviours towards performance degradation of ML algorithms are also highlighted and put forward the facts that technically interpret why DT is the most affected algorithm against poisoning whereas RF is resilient to it. The impact of data poisoning when the training dataset is imbalanced and its features are anisotropic is also examined, and is concluded that anisotropic or asymmetric features serve as catalysts to data poisoning between 10%-20% of poisoning level, whereas they increased the sensitivity of the models, specifically of neural networks. Chapter 4 continues to answer RQ 2, to assess data poisoning impact, severity and consequences in security-sensitive applications and extend the behavioural analysis in such applications. These insights will help develop strong mitigation mechanisms and security by design solutions to secure ML models from existing data poisoning attacks as well as from the evolving threats and new data poisoning attacks.

Chapter 4. Further Exploring Novel Outlier-Oriented Poisoning (OOP) Attack in Multiclass Machine Learning

4.1 Introduction

This chapter proposes an Outlier-Oriented Poisoning (OOP) attack to analyse the implications of new data poisoning attacks under limited adversarial capabilities. The OOP attack perturbs a common characteristic of ML called outliers to manipulate the feature space of multiclass models to analyse changes in learning dynamics. The previous chapter conducted behavioural analysis of binary classifiers, which is now extended to multiclass classifiers in this chapter. This chapter completes the answer to RQ 2 to analyse the impact and consequences of data poisoning attacks in real-world applications. With the development of the OOP attack, the objective 2 of designing and developing new data poisoning attacks is achieved. This chapter also begins to answer RQ 3 by analysing the limitations of existing mitigation solutions and exploiting adversarially trained models with OOP poisoning.

Prior research, such as Baker et al. (2024), Das et al. (2024), Tian et al. (2024), mostly focuses on poisoning availability and integrity attacks against DL and binary classification models. However, data poisoning attacks against traditional multiclass classifiers have been explored to a limited extent. In general, there are three approaches to data poisoning attacks. Firstly, label poisoning Liu et al. (2022), Shahid et al. (2022), Aryal et al. (2022) perturbs the labels of the dataset to manipulate training datasets. Secondly, clean-label poisoning Zhu et al. (2019a), Aghakhani et al. (2021) is typically generated by solving one or more optimisation problems, such as bi-level optimisation Ma et al. (2021a), Russo and Proutiere (2021) or gradient descent optimisation (Sanchez Vicarte et al., 2020) to craft and inject poisoned data points into the model. Thirdly, the existing dataset can be manipulated with feature perturbation. However, multiclass poisoning attacks are explored mostly against DL models.

MetaPoison (Huang et al., 2020) solves a bi-level optimisation problem with meta-learning to craft poison against neural networks. It is practically implemented against the Google Cloud AutoML API and extended for experimentation on multiclass neural networks. While MetaPoison demonstrates effective poisoning in neural networks, its reliance on meta-learning makes it less generalisable to traditional ML classifiers. Subpopulation data poisoning (Jagielski et al., 2021) injects a perturbed cluster into the dataset. Its efficacy is highlighted with a variety of neural networks with multiple datasets. Another research study (Muñoz-González et al., 2017) proposed a gradient poisoning at-

tack and extended their experimentation from binary classification to multiclass classification. This research focuses on experiments with the poisoning availability of the targeted subclass in neural networks.

Furthermore, limited work is provided in the literature that analysed the poisoning impact on real-world applications, developed with traditional ML classifiers. Such limitations are highlighted in Section 2.4 in Chapter 2. To address these limitations, this chapter conducted two case studies of analysing the poisoning impact with a novel data poisoning attack in real-world applications. The experimentation results of these case studies provide us with an understanding of how data poisoning impacts such applications and their consequences and real-life impacts.

Given the limitations, this chapter assesses the effectiveness of novel data poisoning attacks in multiclass settings to inform the development of improved mitigation strategies. While most existing research on multiclass poisoning focuses on neural networks, this study examines six supervised machine learning algorithms: SVM, DT, RF, KNN, GNB, and Neural Networks using a Multilayer Perceptron (MLP). These algorithms represent a comprehensive baseline of classification methods. Poisoning levels are set ranging from 5% to 25% in 5% increments to assess model behaviour under varying attack intensities, following the approach in (Paracha et al., 2024a). The analysis identifies key parameters of each algorithm that are sensitive to poisoning, determines optimal poisoning rates, and quantifies performance degradation in terms of accuracy and model-specific characteristics. The main contributions of this chapter are outlined below.

- A new OOP attack is developed, as a novel label poisoning attack to introduce misclassification in multiclass ML. This attack is formulated based on the label perturbation of the most distant data points from the decision boundaries of the multiclass classifier.
- A thorough behavioural analysis of multiclass classifiers is performed, analysing the correlation between different poisoning levels and the performance degradation of classifiers.
- The OOP attack is implemented against real-world applications, analysing the impact of poisoning in such systems. Following the poisoning of these applications highlighted the limitations of existing security techniques in mitigating data poisoning attacks, particularly focusing on traditional multiclass models.

4.2 Related Work

Existing literature highlights a significant number of poisoning attacks that harm the integrity and availability of ML models. Such as Zhao and Lao (2022b) proposed a class-oriented poisoning attack to introduce misclassification for a targeted dataset class. Similarly, Carlini and Terzis (2021) highlighted a security threat of poisoning and backdoor attacks against multiclass ML with only 0.0001%

of data poisoning. They have introduced misclassification during model training to overfit, which increases errors during testing. Alarab and Prakoonwit (2023) have developed a poisoning attack using a Monte-Carlo algorithm against deep learning multiclass models to analyse their classification uncertainties. Pantelakis et al. (2023) have evaluated the performance disruption of IoT-based multiclass models against JSMA, FGSM, and DeepFool attacks, highlighting the effectiveness of these attacks in poisoning multiclass models. Other prominent poisoning attacks are Suya et al. (2021), Mayerhofer and Mayer (2022), Saha et al. (2020). Table 4.1 highlights existing studies that experimented with data poisoning attacks with various DL and ML models. It is crucial to understand the behaviour of the underlying baseline models and their sensitivity against poisoning attacks. This investigation helps us better mitigate poisoning, not only focusing on their performance but also on their underlying classification mechanisms. Following the discussed attacks, this chapter focuses on manipulating outliers to disrupt the feature spaces of the multiclass models, discussed in Section 4.5.1. This study has shown the efficacy and effectiveness of the attack on six ML algorithms at various poisoning levels.

Table 4.1: Analysing existing studies against the behavioural analysis with outlier-oriented poisoning attack

Research paper	ML model	Dataset	Effective poisoning level	Model degradation and variance at:	
				Various poisoning levels	Various classes
Zhao and Lao (2022b)	LeNet-5, Vgg-9, ResNet-50	MNIST, CIFAR-10, ImageNet	✗	✗	✗
Carlini and Terzis (2021)	ResNet-50, Transformer language model	Conceptual Captions	✗	✗	✗
Alarab and Prakoonwit (2023)	LEConv, CNN	Cora, MNIST	✗	✗	✗
Pantelakis et al. (2023)	DT, RF, KNN, MLP	IoTID20	✗	✗	✗
OOP Attack	SVM, DT, RF, GNB, KNN, MLP	IRIS, MNIST, ISIC	✓	✓	✓

Limited techniques are provided in the literature that are generalisable and effective in mitigating poisoning effects against traditional multiclass models. McCarthy et al. (2023) proposed a hierarchical learning mechanism to secure the network traffic attack classification model. Hossain and Oates (2024) developed a solution to detect backdoor poison in deep neural networks by extracting, relabeling, and classifying features with a tensor decomposition method. They have experimented with their mitigation solution on MNIST, CIFAR-10, and TrojAI datasets. Curie (Laishram and Phoha, 2016) is the method proposed to mitigate poisoning attacks against SVM. They introduced an additional feature dimension to map labels with features that help segregate poisoned data points with flipped labels from the normal data points.

Melacci et al. (2021) have experimented with the effectiveness of incorporating domain knowledge into the neural networks, in detecting adversarial data points added in the model training. They have experimented with their solution on neural networks with CIFAR-100, ANIMALS, and PASCAL-Part datasets. None of the above solutions is generalisable to both ML and DL models. To strengthen mitigation solutions, it is important to understand how poisoning affects the underlying classification behaviours of these models. Therefore, this chapter assesses multiclass classifiers under poisoning attacks to identify their key characteristics affected by poison and reveal their relationships with injected poison.

4.3 Threat Model

A grey-box threat model is developed to evaluate the OOP attack in real-world applications. In this threat model, the goals of poisoning multiclass ML models are defined under the limited knowledge and capabilities of the adversary. This study aims to design the OOP attack that manipulates the decision boundaries of multiclass classifiers and to evaluate their susceptibility to data poisoning attacks. The analysis is further extended to assess the effects of this novel poisoning technique in real-world applications that incorporate adversarial training defense. The limitations of existing adversarial training are described in Chapter 2, noting that these approaches have primarily been applied to DL models and require evaluation on traditional classification models. The targeted applications in this research include an image classification system, a skin cancer diagnostics system, and an industrial 5G network system. The following metrics are used for comprehensive evaluation and analysis.

- **Baseline metrics**: Accuracy, precision, recall, f1-score, and false positive rate
- **Robustness metrics**: Model variance and adversarial success rate
- **Feature-sensitivity metrics**: Feature importance score
- **Algorithm-specific metrics**: KNN k neighbors, SVM margin, and GNB class probabilities

4.3.1 Attack Surface and Goals

The attack surface considered is the training pipeline, where the adversary aims to corrupt the feature space of the model by injecting perturbed outliers. The goal is to analyse the susceptible characteristics and impact of novel data poisoning against a benign and adversarially trained model, and the impact of data poisoning in real-world applications. By disturbing the training data, the adversary seeks to induce systematic misclassification and degrade model reliability in practical deployments.

4.3.2 Attacker Knowledge

A realistic scenario is developed to analyse model susceptibility and impact against data poisoning attacks. This scenario considers a grey-box knowledge attack, where only the dataset and algorithm family are known to the adversary. The model is inaccessible with no known configurations or parameters. This assumption is considered following the training of ML models with publicly available datasets, which can be accessible to the adversary.

4.3.3 Attacker Capability

This threat model follows limited adversarial capabilities where the target model is inaccessible to the adversary. In this way, the adversary develops surrogate models to implement data poisoning attacks and manipulate training datasets. To implement the threat model, three scenarios were developed to study data poisoning against multiclass models in supervised ML. The details of these scenarios are given in Section 4.3.5 and Section 4.3.6.

4.3.4 Outlier-Oriented Poisoning Attack

This study adopts a practical approach to design the attack strategy under the assumption that the adversary A_{dv} does not know the underlying settings of the targeted model M and the dataset distribution. The OOP attack mechanism is illustrated in Fig. 4.1. The OOP attack develops surrogate models to identify and perturb the most distant data points in each class of the model. After identification, the OOP attack changes the class of the selected data points to manipulate the feature surface of the classes in the ML model to misalign decision boundaries and decrease classification performance.

Three datasets—IRIS, MNIST, and ISIC—each with three, ten, and four classes, respectively, were used to assess this attack. The OOP attack is formulated as an end-to-end poisoned training setting, where only the datasets are known to the adversary. Surrogate models M_s are developed and trained to craft poisoned data points x' with perturbed class labels l' at different poisoning levels ΔL. Poisoning levels ranged from 5% to 25% at a scale of 5%. The OOP attack initiates multiclass poisoning by calculating the data points at maximum distance from hyperplane and changing their classes l'.

With this outlier perturbation, benign feature spaces were manipulated by misplacing outliers in them. The goals are to assess the performance degradation of individual algorithms and to analyse the behaviour of multiclass models under the OOP attack. For the dataset manipulation with the OOP attack, let $x \in D$ be the distanced data point that is perturbed by manipulating its label l to increase the loss of the model $\mathcal{L}$ with change γ in multiclass decision boundaries b_c following model

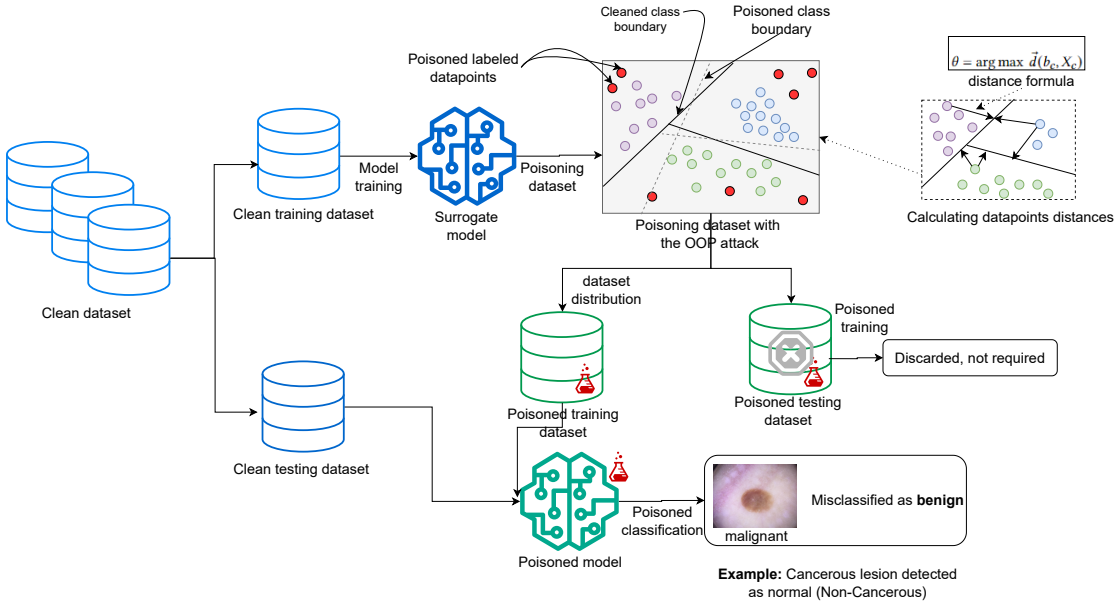


Figure 4.1: The architectural overview of Outlier-oriented poisoning attack

training $\mathcal{T}$ as:

$$\mathcal{L}(M, D') = \gamma = \Delta b_c(\mathcal{T}(M, D')) \tag{4.1}$$

4.3.5 OOP Attack Against Adversarially Trained Skin Cancer Diagnostics

The OOP attack is extended under the assumptions of a weak adversary A_{dv} to deceive an adversarially trained skin cancer diagnostic application. The application is trained using the public ProveAI version of the ISIC dataset, which contains four classes. The OOP attack poisons this public dataset with $\Delta L = 10\%$, which is then used to train the diagnostic application.

The diagnostic is secured with adversarial training and implemented with the FGSM attack. Further details of the training approach are given in Section 4.7.1. This attack scenario is novel in two aspects. Firstly, the OOP poisoning against secured traditional models is analysed in non-complex settings. Previously, the adversarial training is implemented and tested primarily on neural networks. Secondly, it analyses the effects of dataset poisoning on the diagnostic model, implemented with three ML algorithms in multiclass settings.

Consider an ML model $M(D;\theta)$ with a classification skin lesion dataset D and θ as configuration parameters. This model $M(D;\theta)$ is trained to make it resilient against data poisoning attacks, using 75% dataset for training and 25% for testing. Mathematical notation of the secured training of a diagnostic application is given in Eq. 4.2.

$$M_{adv}(D;\theta) = minMSE_M[max\mathcal{L}(f(x+\delta), l)] \tag{4.2}$$

Table 4.2: Configurations of the models

S.No.	Model	Model Configurations
1	SVM	decision_function_shape='ovo', kernel='poly', degree=3, C=1
2	RF	criterian='log_loss', n_estimators=9, max_depth=4
3	MLP	penalty='elasticnet', alpha=0.0001, max_iter=1000

where $M_{adv}(D;\theta)$ is the adversarially trained model, MSE_M is the mean square error of the model under testing, $\mathcal{L}$ is the loss of the model, δ is the FGSM-based perturbation added to individual data sample x and l is the original label of the data sample x. The OOP attack generates poisoned data D' to corrupt the model training $\mathcal{T}$ of $M_{adv}(D';\theta)$. The mathematical notation of poisoned training is given in Eq. 4.3.

$$M'_{adv} = \mathcal{T}(M(D';\theta)) \tag{4.3}$$

where, $\mathcal{T}$ is the training process of the model and D' is the poisoned dataset. The model configurations θ for all the algorithms are given in Table 4.13.

4.3.6 OOP Attack Against Industrial 5G Networks

The adversary A_{dv} is considered as weak, meaning the network configurations and parameters are unknown to it. ML models are developed to manage the spectrum sharing between 5G and 802.11ax networks using a data-driven approach. By targeting the training dataset D of these ML models, the spectrum sharing process is poisoned. The OOP attack is implemented to poison the spectrum dataset. The OOP attack is implemented with poisoning levels $\Delta L = 10\%, 15\% and 20\%$. Following this threat model, the impact of data poisoning in a time-sensitive spectrum sharing application is analysed as a diverse industrial use case.

4.4 Case Study Selection Rationale

The selection of skin cancer diagnostic and industrial 5G private network applications aligns with the objective to assess the impact of data poisoning in real-world applications. These case studies were chosen based on the criticality of decision outcomes, susceptibility to poisoned training and availability of datasets and deployment environments to enable practical assessment. Following the research gaps of the thesis, given in Section 2.4 in Chapter 2, these assessments address the limitations of assessing data poisoning in traditional multiclass ML.

The adversarially trained skin cancer diagnostic application utilises the **SkinCheck** application, enabling assessment of the OOP attack in a practical context. This study specifically addresses the research gap of evaluating the effectiveness of adversarial training in securing traditional ML models.

This case study highlights the risks associated with data poisoning, particularly its impact on missed diagnoses of actual skin cancer cases.

The second case study examines an industrial 5G network in collaboration with the 5G networks team at **Birmingham City University**. This network, developed for a material recycling facility, is used to optimise network resources and assess vulnerability to OOP attacks. While data poisoning has been studied in the image domain, it is still underexplored in networking applications, particularly in Industry 4.0, associated with multiclass ML. To respond to research gaps, this study leverages the analysis of risks in high-yield production environments and evaluates how data poisoning influences latency, throughput, and contention windows in demanding industrial processes.

4.5 Approach to OOP Attack

This section details the approach of designing and evaluating the OOP attack. It first discussed the attack method, followed by describing datasets, attack settings and evaluation metrics.

4.5.1 Attack Method

Instinctively, the training dataset is poisoned with the OOP attack to disrupt ML performance at validation. The OOP attack algorithm is given in Alg. 2. This attack manipulates the class labels of the most distant data points from their class boundaries, manipulating the feature space of the classes and misleading classification predictions. To perturb the training dataset, the adversary develops the surrogate model, as no access to the targeted model is provided. The OOP attack is implemented using six surrogate multiclass models: SVM, RF, DT, GNB, KNN and MLP, following default configurations for non-linear datasets. The SVM is developed using a polynomial kernel with the default degree and regularization parameters of 3, and the MLP is trained using the RELU activation. However, the other four models support multiclass non-linear classification in default configurations. The class probability is the model confidence in classifying an instance against each class in GNB model. The surrogate models are developed to calculate outliers following the Alg. 3. The Alg. 4 distinctly calculates the farthest outliers from decision boundaries for the given ML model to maximise classification errors. To identify decision boundaries for individual models, the algorithm is described in Alg. 4.

4.5.2 Experimental Datasets

The OOP attack is implemented on multiclass classifiers using three multiclass datasets: IRIS, MNIST, and ISIC. The reason for selecting these datasets is their varying sizes and structures, where IRIS is a small dataset, MNIST is very large, and ISIC is medium-sized with varying feature correlation and number of classes. The dataset characteristics are provided in Table 4.3. By employing

Algorithm 2 OOP Poisoned Model Generation

Datasets: IRIS, MNIST, ISIC, datasets
Inputs: Training Dataset D, Poison level ΔL
Outputs: Poisoned Model M'_c
Initialise: $D \leftarrow$ Training dataset
$\Delta L \leftarrow$ Poisoning level $\in [0\%, 5\%, 10\%, 15\%, 20\%, 25\%]$
$M_{conf} \leftarrow [SVM, DT, RF, GNB, KNN, MLP]$
$D' \leftarrow$ Poisoned dataset = []
$D_{dist} \leftarrow$ subset of Training dataset
while $len(D') \leq \Delta L$ **do**
  Set index i = max(D_{dist})
  Set data point $x_i = D[i]$
  if x_i not in D' **then**
    Set l = Class(x_i)
    Update $l = l_i$; where $l_i \neq c$
    Update Class(x') = l_i
  end if
  $D' \leftarrow (x', l')$
  Set $D_{dist}[i] = 0$
end while
D'_{ctrain} = split(D', 0.75)
M' = train(M_{conf}, D'_{train})
return M'

Algorithm 3 Surrogate Model Development

Datasets: IRIS, MNIST, ISIC datasets
Inputs: Training Dataset D, Model Configuration M_{conf}
Outputs: Surrogate Trained Model M_{surr}
Initialise: $D \leftarrow$ Training dataset
$M_{conf} \leftarrow$ [Support Vector Machines (SVM) = Config(kernel='poly', degree of polynomial function=3, regularisation parameter=3),
Decision Tree(DT) = Config(criterion='gini', splitter='best')
Random Forest(RF) = Config(n_estimators=3, criterion='gini')
K-Nearest Neighbors (kNN) = Config(n_neighbors=5, weights='uniform')
Gaussian Naive Bayes (GNB) = Config(var_smoothing=$1 * 10^{-9}$)
Multilayer Perceptron (MLP) = Config(activation='relu', solver='adam')]

for config in M_{conf} **do**
  $M_{surr}(config) = initialise(M_{surr}, config)$
  $M_{surr}(config) = training(M_{surr}(config), D)$
end for
return $M_{surr}(config)$

the OOP attack across datasets with differing structures, a comprehensive analysis is conducted of how data poisoning influences feature correlations, class numbers, and dataset sizes within multiclass contexts. The visual datasets representation with the GMM is given in Fig. 4.3, highlighting their features correlation. Fig. 4.3(a) illustrates that certain features within the IRIS dataset are strongly interdependent, whereas the complete dataset is not in a linear relation. However, MNIST is found to be a highly dense dataset with strong features relations as visualised in Fig. 4.3(b). The ISIC dataset, shown in Fig. 4.3(c), displays a non-linear relationship with significant outliers, indicative of substantial noise levels. The statistical correlations of datasets are highlighted in Table 4.4. Features

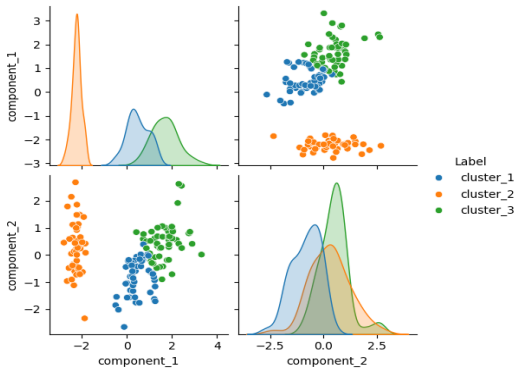
Algorithm 4 Calculating Distances from Decision Boundaries

Inputs: Surrogate Models M_{surr}, Training Dataset D
Outputs: Calculated distances of Models $dist_M$
Initialise: $dist_M = [dist_{SVM}, dist_{DT}, dist_{RF}, dist_{GNB}, dist_{KNN}, dist_{MLP}]$
$x \leftarrow$ Model data points
$M_{surr} = [M_{SVM}, M_{DT}, M_{RF}, M_{GNB}, M_{KNN}, M_{MLP}]$
if $M_{surr} == M_{SVM}$ **then**
  for $x_i \in M_{SVM}$ **do**
    $dist_{SVM}[x_i] \leftarrow decision_{function}(x_i, M_{SVM})$
  end for$dist_M[SVM] = x_i$
end if
if $M_{surr} == M_{DT}$ **then**
  $Clf_{tree} = M_{DT}.tree_$
  for $x_i \in D$ **do**
    $dist[x_i] \leftarrow calculate_depth(x_i, Clf_{tree})$
  end for
  $dist[DT] = x_i$
end if
if $M_{surr} == M_{RF}$ **then**
  for $clf_x \in M_{RF}$ **do**
    $Clf_{tree} = clf_x.tree_$
    for $x_i \in D$ **do**
      $dist[x_i]_D = calculate_depth(x_i, Clf_{tree})$
    end for
  end for
  $dist_M[RF] = avg(dist[x_1]_D, dist[x_2]_D, ...dist[x_n]_D)$
end if
if $M_{surr} == M_{KNN}$ **then**
  for $x_i \in D$ **do**
    $dist(x_i)_{neighbors} = M_{KNN}.kneighbors_$
    $dist_M[KNN] \leftarrow \arg\max(distance(x_i)_{neighbors})$
  end for
end if
if $M_{surr} == M_{GNB}$ **then**
  $D_a, D_b = split(D, 2)$
  for $i \in [D_{c_a}, D_{c_b}]$ **do**
    $j = -i + 1$
    for $x_i \in D[i]$ **do**
      $Class(x_i) = predict_probability(D[j], M_{GNB})$
      $log_likelihood \leftarrow log(Class(x_i))$
      $distance(x_i) \leftarrow distance(\arg\max(Class(x_i), axis = 1))$
    end for
  end for$dist_M[GNB] = distance(x_i)$
end if
if $M_{surr} == M_{MLP}$ **then**
  for $x_i \in M_{MLP}$ **do**
    $dist_M[MLP] \leftarrow decision_function(x_i, M_{MLP})$
  end for
end if
return $dist_M$

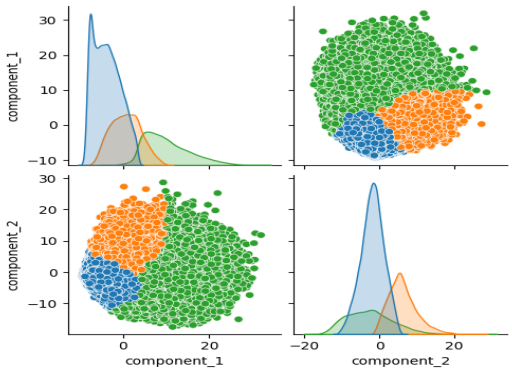
in the MNIST dataset are highly associated with a p-value of 0.0141, highlighting direct proportionality between its features. A low statistical significance is shown in the IRIS datasets with a p-value of 0.07, and the p-value of the ISIC dataset is 0.2396. In contrast, a negative Spearman correlation coefficient highlights a negative linear correlation between its features with a high noise ratio. Further

Table 4.3: Dataset description used for experimentation with outlier-oriented poisoning attack

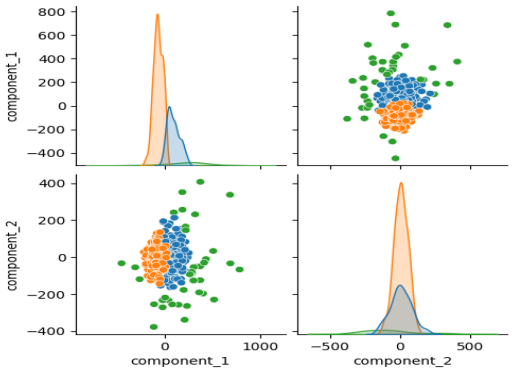
S.No.	Dataset	No. of features	No. of Classes	No. of instances
1	IRIS	4	3	170
2	MNIST	784	10	70,000
3	ISIC	20	4	603



(a) Features correlation in IRIS dataset



(b) Features correlation in MNIST dataset



(c) Features correlation in ISIC dataset

Figure 4.2: Gaussian mixture model visualisation of features relationship in the dataset with PCA reduction

analysis of the importance of features correlation and the impact of dataset noise for the OOP attack is given in Section 4.6.

Table 4.4: Statistical correlation of features in the dataset

S.No.	Dataset	Spearman Correlation	p-value
1	IRIS	0.123888	0.0791
2	MNIST	0.009282	0.0141
3	ISIC	-0.014311	0.2396

4.5.3 Attack Settings

Surrogate models for each algorithm is initialised and trained them with selected datasets. The distances of each data point are calculated from the decision boundary for each class to manipulate those far from the decision boundaries. Consider $\mathcal{T}$ is the model training process with poisoned

dataset D' and $\mathcal{P}_{\mathcal{M}}$ is the function of performance measure, the objective function of the attack method is given in Eq. 4.4 whereas, θ is the measure of distance of data points from the decision boundaries. Mathematical notation of θ is given in Eq. 4.5.

$$\arg\min \mathcal{P}_{\mathcal{M}}(\mathcal{M}(x', l'); \theta)) \tag{4.4}$$

$$\textbf{s.t. } \theta = \arg\max \vec{d}(b_c, (x, l)) \tag{4.5}$$

Also, D' is the poisoned dataset manipulated at various poisoning levels ΔL where the notation of dataset poisoning is given in Equation 4.6.

$$\begin{gathered} D' = \sum_{i=1}^{n \to \Delta L} f(D(x_i, l_i), \Delta L) \\ \textbf{where; } X_{l_c} \neq X'_{l_c} \end{gathered} \tag{4.6}$$

where, f is the function of manipulating labels, (x, l) is the clean data point, (x', l') is the poisoned data point with l'.

Let poisoning levels ΔL = [5%,10%,15%,20%,25%] manipulate model training by disturbing class-level decision boundaries b_c with notation given in Alg 2. Let f be the function to poison the dataset D' at ΔL poisoning level. M' is the poisoned model trained with a dataset having manipulated data points (x', l') as given in Eq. 4.7. This allows us to analyse the model behaviour and change in decision boundaries as given in Eq. 4.8

$$\begin{gathered} M' = \mathcal{T}(M, D') \\ \textbf{where; } D' = f(D(x, l), \Delta L) \end{gathered} \tag{4.7}$$

$$Mod_{Dis} = \Delta b(M') \tag{4.8}$$

where M' is the poisoned model developed for algorithms [SVM, DF, RF, KNN, GNB, MLP] and Δb is the change in decision boundaries. To conduct a statistical analysis of the performance degradation of multiclass models and the variance in test-time classification across different poisoning levels, the correct classification rate is defined in Eq. 4.9.

$$\begin{gathered} C_{CR} = \frac{\sum_{i=1}^{n} f(N_c, C_{c_i}(M'_{c_i}(D_t(x_i, l_i))))}{\sum_{i=0}^{n=1} N_c} \\ and\ f(N_c, C(D_t(x, l))) = \begin{cases} true & \text{if } (x, l) \in Class\ c \\ false & \text{otherwise} \end{cases} \end{gathered} \tag{4.9}$$

where, f is the function of classification, X_t is the data point from the validation dataset D_t, N_c is the total number of data points in Class c, $C(.)$ is the class estimator and C_{CR} is the non poisoned classification rate.

The OOP attack is conducted against multiclass classification models to assess the poisoning impact on multiclass ML performance. For the performance evaluation and analysis of the impact of poisoning availability attacks in multiclass models, poisoned models are evaluated by analysing how many outliers successfully intrude themselves in wrong classes, which is the **False Positive Rate(FPR)** of the model where the model fails in classifying the correct class. However, where the poisoned outliers remain disjointed in the incorrect classification classes and model availability is intact, is the **Accuracy(Acc)**, and where the outliers are unsuccessful in intruding the multiclass decision boundaries is the **Precision** of the model against OOP attack. **Recall** in the evaluation is the quantification where a model can segregate dataset classes and keep the decision boundaries intact. **Variance(Var)** reflects how the model's behaviour changes when its parameters or dataset change. Considering $f(N_c, C((x,l)))$ is the classification function as given in Eq 4.10, the evaluation metrics are mentioned in Eq 4.11, 5.17, 4.13, 5.18 and 4.15.

$$f(N_c, C((x,l))) = \begin{cases} true & \text{if } (x,l) \in Class\ c \\ false & \text{otherwise} \end{cases} \tag{4.10}$$

$$FPR = \frac{\sum_{i=0}^{n} f_{tr}(N_c, C(x_i', l_i'))}{\sum_{i=0}^{n} f(N_c, C(x_i', l_i')) \wedge \sum_{i=0}^{n} f(N_c, C(x_i, l_i))}$$

$$\begin{aligned} where\ f_{tr}(N_c, C(x_i', l_i')) &\in D' \\ and\ f(N_c, C(x_i', l_i')) &\in D' \\ and\ f(N_c, C(x, l)) &\in D \end{aligned} \tag{4.11}$$

where D is the clean dataset, N_c is the total number of data points in Class c, and D' is the poisoned dataset with changed class labels of the farthest data points. $f_{tr}(N_c, C(x', l'))$ are poisoned data points with perturbed labels and classified as false positives(FP) and $f_{fs}(N_c, C(X'_{tr_i}))$ are false negative(FN) data points.

$$Acc = \frac{\sum_{i=0}^{n} f_{fs}(N_c, C(x_i, l_i)) \wedge \sum_{i=0}^{n} f_{tr}(N_c, C(x_i, l_i))}{((x,l) \in D) \wedge ((x_i', l_i') \in D')} \tag{4.12}$$

$$Prn = \frac{\sum_{i=0}^{n} f_{tr}(N_c, C(x_i, l_i))}{\sum_{i=0}^{n} f_{tr}(N_c, C(x_i, l_i)) \wedge \sum_{i=0}^{n} f_{tr}(N_c, C(x'_i, l'_i))} \tag{4.13}$$

$$where\ f_{tr}(N_c, C(x'_i, l'_i)) \in D'$$

$$Rcl = \frac{\sum_{i=0}^{n} f_{tr}(N_c, C(x_i, l_i))}{\sum_{i=0}^{n} (f_{tr}(N_c, C(x_i, l_i))) \wedge \sum_{i=0}^{n} (f_{fs}(N_c, C(x'_i, l'_i)))} \tag{4.14}$$

$$where\ f_{fs}(N_c, C(x'_i, l'_i)) \in D'$$

$$Variance(\sigma) = \frac{1}{N_c} \sum_{i=0}^{n} (f(N_c, C(x_i, l_i)) - \mu(f(N_c, C(x'_i, l'_i))))^2 \tag{4.15}$$

4.6 Experimentation Results and Analysis

The objective is to analyse the behaviour of multiclass models and answer questions about how the characteristics of these models are affected and what their relationship is with the poison. What are the optimal poisoning levels ΔL and the effects of changing poisoned data distributions? What is the effectiveness and persistence of data poisoning with the OOP attack and its impact on model validation performance (specifically accuracy)? And quantifying and analysing model variance σ at test-time classification at different poisoning levels ΔL.

4.6.1 Effects on Multiclass Classification

The baseline results of the OOP attack are given in Fig. 4.3 to Fig. 4.8, where validation accuracy, precision, recall, f1-score and FPR are plotted against poisoned training with maximum poisoning level $\Delta L = 25\%$. These results indicate that the KNN algorithm is particularly vulnerable, experiencing the most significant accuracy disruption with a maximum decrease in accuracy $(\lambda) = 40.35$ at $\Delta L = 25\%$ with an increase in FPR=31.6% from FPR=2.7%, shown in Fig. 4.6(a). This vulnerability stems from KNN being a non-parametric algorithm that relies on the proximity of data points to determine class features. Table 4.5 highlight that the number of nearest neighbors found to be inversely proportional to ΔL, reducing the attack success rate 15.79% to 2.76% for the IRIS dataset by changing k=3 to k=15. Fig. 4.6(c) demonstrates high ASR when KNN is trained with the ISIC dataset, decreasing its validation accuracy to 63% with FPR=28.25%. From Table 4.5, increasing the number of nearest neighbors decreases ASR from 3.97% to 3.31% with $\Delta L = 25\%$.

The GNB is the second most affected algorithm with a decrease in validation accuracy from 92.98% to 56.14% and an increase in FPR from 5.68% to 32.49% at $0\% \leq \Delta L \leq 25\%$, for the IRIS dataset, given in Fig 4.5. Interestingly, the GNB model is failing with the OOP attack at $\Delta L = 15\%$ where

it's *precision* ≤ 0 where a lower impact can be seen with MNIST and ISIC datasets. Further analysis reveals the change in the importance of classes, leading to misclassification, with changing class probabilities at poisoning levels, given in Table 4.6, whereas the class probability is the model confidence in classifying an instance against each class.

The attack manipulates the gaussian probability measures, making the highest probability class an anomaly and vice versa for the IRIS dataset. However, minor changes are visible for MNIST and ISIC datasets with no change in classes ranking at $0\% \leq \Delta L \leq 15\%$. This analysis also highlights that GNB is the most affected algorithm when trained with a dataset with fewer classes.

Table 4.5: Analysing k-neighbors affecting k-nearest neighbors accuracy with $\Delta L = (0, 10, 15, 25)\%$

Dataset	Poison Level	k=3	k=5	k=10	k=15
IRIS	$\Delta L = 0\%$	94.73	97.50	97.36	97.36
	$\Delta L = 10\%$	89.47	97.36	97.30	94.73
	$\Delta L = 15\%$	81.57	92.10	94.73	92.10
	$\Delta L = 20\%$	78.94	84.21	94.60	94.60
MNIST	$\Delta L = 0\%$	98.16	97.55	96.94	96.55
	$\Delta L = 10\%$	92.41	96.52	96.78	96.50
	$\Delta L = 15\%$	89.44	90.90	94.54	95.95
	$\Delta L = 25\%$	85.34	76.14	83.68	87.52
ISIC	$\Delta L = 0\%$	80.79	82.11	70.19	77.48
	$\Delta L = 10\%$	77.48	77.48	66.88	74.17
	$\Delta L = 15\%$	76.15	74.17	68.87	76.13
	$\Delta L = 25\%$	76.82	74.07	64.90	74.17

Table 4.6: Analysing class probabilities of gaussian naive bayes with the poisoned dataset

Dataset	Dataset Class	Clean Dataset	$\Delta L = 10\%$	$\Delta L = 15\%$
IRIS	Class 0	0.33	0.36	0.38
	Class 1	0.35	0.25	0.33
	Class 2	0.31	0.37	0.27
MNIST	Class 0	0.09	0.09	0.09
	Class 1	0.11	0.11	0.11
	Class 2	0.09	0.09	0.09
	Class 3	0.10	0.10	0.10
	Class 4	0.09	0.10	0.09
	Class 5	0.08	0.09	0.09
	Class 6	0.09	0.09	0.09
	Class 7	0.10	0.10	0.10
	Class 8	0.09	0.09	0.09
	Class 9	0.09	0.10	0.10
ISIC	Class 0	0.76	0.69	0.64
	Class 1	0.05	0.08	0.10
	Class 2	0.02	0.04	0.07
	Class 3	0.14	0.17	0.17

Whereas the OOP attack has minimally disrupted DT, resulting in λ values of 31.6 for IRIS, 15.18 for MNIST, and 17.88 for ISIC at $\Delta L = 25\%$. Table 4.7 demonstrates the change in features importance scores with dataset poisoning, where feature1 scores (0.90, 0.39) remain highest for IRIS

and MNIST. But feature1 (0.36) with the highest importance score for ISIC becomes anomalous, making anomaly feature2 (0.37) the most important feature at $\Delta L = 15\%$, degrading its classification. The RF algorithm demonstrates relative robustness, with its FPR converge to $\approx 2\%$ with an overall accuracy decrease to 61.25% from 87% for the ISIC dataset and FPR converge to $\approx$ 9% for the MNIST dataset with accuracy of 82.38% at $\Delta L = 25\%$ as shown in Fig. 4.4. Because RF follows the ensemble approach and classifies averaging decisions from all of its trees, which normalises the poisoning effects in this case. The change in features importance scores for RF is given in Table 4.10, where features ranks remain the same for IRIS and MNIST, but for ISIC highest ranked feature dropped to rank two at $\Delta L = 15\%$ poisoning. Lastly, SVM and MLP are also not found to be very sensitive to the OOP attack. For SVM, features ranks remain intact, given in Table 4.8, except for ISIC, where feature3 (0.39) importance score reduces to (0.33) at $\Delta L = 15\%$, making it an anomaly. A lower impact is visible on MLP from Fig 4.8, with this attack, except at $\Delta L = 15\%$ where it is failing for the IRIS dataset.

Table 4.7: Features importance score - Decision tree where $\Delta L = (0\%, 10\%, 15\%)$

	Clean Dataset			Poisoned Dataset $\Delta L = 10\%$			Poisoned Dataset $\Delta L = 15\%$		
Dataset	Feature1	Feature2	Feature3	Feature1	Feature2	Feature3	Feature1	Feature2	Feature3
IRIS	0.90	0.00	0.02	0.87	0.008	0.11	0.79	0.07	0.12
MNIST	0.39	0.34	0.26	0.39	0.33	0.27	0.39	0.32	0.28
ISIC	0.36	0.28	0.35	0.28	0.38	0.32	0.32	0.37	0.30

Table 4.8: Features importance score - Support vector machines where $\Delta L = (0\%, 10\%, 15\%)$

	Clean Dataset			Poisoned Dataset $\Delta L = 10\%$			Poisoned Dataset $\Delta L = 15\%$		
Dataset	Feature1	Feature2	Feature3	Feature1	Feature2	Feature3	Feature1	Feature2	Feature3
IRIS	0.90	0.02	0.08	0.78	0.05	0.15	0.86	0.10	0.02
MNIST	0.40	0.16	0.43	0.34	0.23	0.42	0.36	0.21	0.42
ISIC	0.33	0.27	0.39	0.30	0.22	0.47	0.32	0.33	0.33

Table 4.9:
Analysing support vector machines margin score for different datasets with $\Delta L = (0, 10, 15)\%$

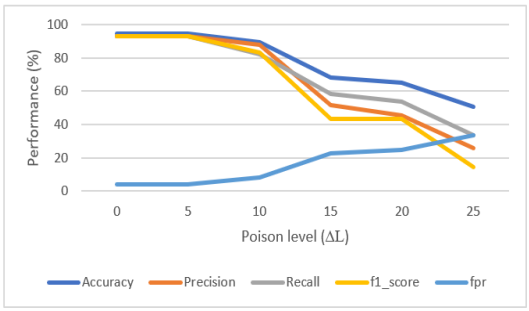
Dataset	$\Delta L = 0\%$	$\Delta L = 10\%$	$\Delta L = 15\%$
IRIS	0.005	0.01	0.001
MNIST	0.0000011	0.00000022	0.00000027
ISIC	0.01	0.003	0.003

4.6.2 Effects of Poisoning Rates

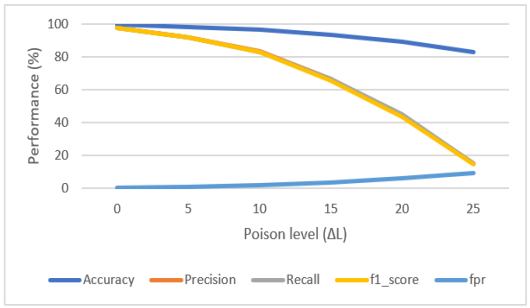
The analysis is extended to study the effects of consistently increasing poisoning rates on multiclass models with the OOP attack. The aggregated results, given in Fig. 4.3 to Fig. 4.8, show over-fitting? **No**. These results demonstrated that the classification accuracy of multiclass classifiers has maximum disruption when the training dataset is poisoned with the OOP attack at $\Delta L = 10\%$, irrespective of

Table 4.10:
Features importance score - Random forest where $\Delta L = (0\%, 10\%, 15\%)$

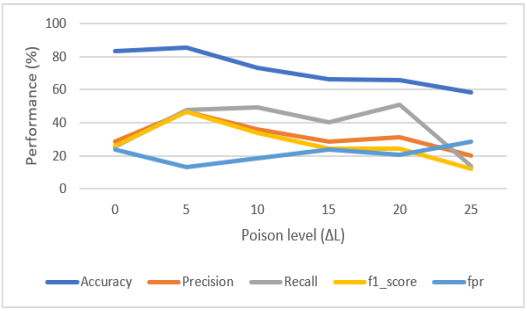
Dataset	Clean Dataset			Poisoned Dataset $\Delta L = 10\%$			Poisoned Dataset $\Delta L = 15\%$		
	Feature1	Feature2	Feature3	Feature1	Feature2	Feature3	Feature1	Feature2	Feature3
IRIS	0.66	0.15	0.17	0.58	0.19	0.22	0.52	0.22	0.25
MNIST	0.39	0.34	0.26	0.39	0.33	0.27	0.39	0.32	0.27
ISIC	0.31	0.35	0.34	0.31	0.36	0.32	0.34	0.33	0.32



(a) Poisoning support vectore machines with IRIS dataset



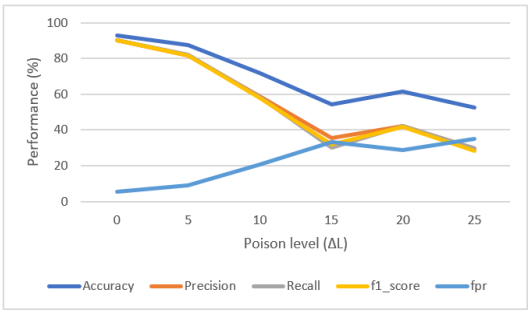
(b) Poisoning support vector machines with MNIST dataset



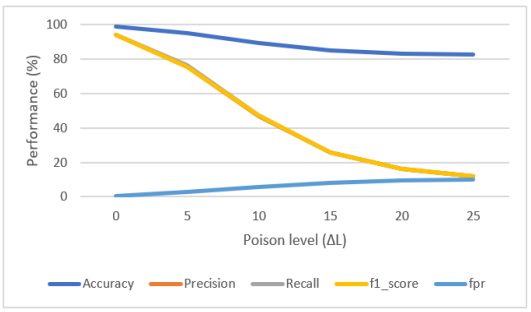
(c) Poisoning support vector machines with ISIC dataset

Figure 4.3: Performance analysis of support vector machines with consistent poisoning

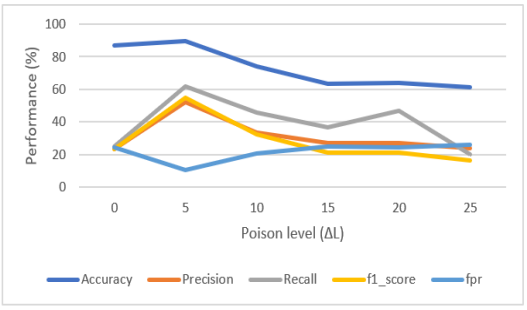
datasets. An inverse relationship is observed between the number of classes in the dataset and the rate of performance degradation. For the MNIST dataset, from Fig. 4.3(b) to Fig. 4.8(b), the ten dataset classes have a steady decrease in performance. Whereas classifiers trained with the IRIS dataset, with three dataset classes, have high fluctuation in performance, followed by ISIC with four classes. The least percentage of data poisoning is more effective on parametric models. The 10% poisoning has a steady and practical impact on parametric models, whereas 15% poisoning leads to impractical effects. From Fig. 4.5(a) and Fig. 4.8(a), parametric models, with minimum no. of classes, are failing at 15% poisoning. But, $\Delta L = 15\%$ is very effective for non-parametric models. Conclusively, $10\% \leq \Delta L \leq 15\%$ are the optimal poisoning rates for multiclass models, where $\Delta L > 15\%$ shows an impractical success.



(a) Poisoning random forest with IRIS dataset

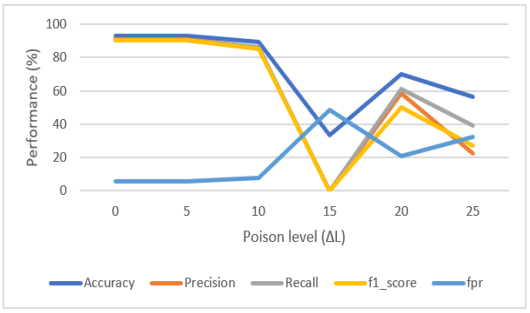


(b) Poisoning random forest with MNIST dataset

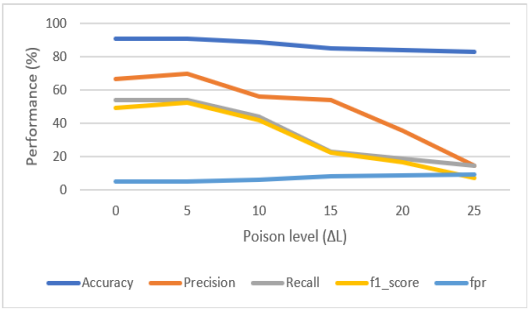


(c) Poisoning random forest with ISIC dataset

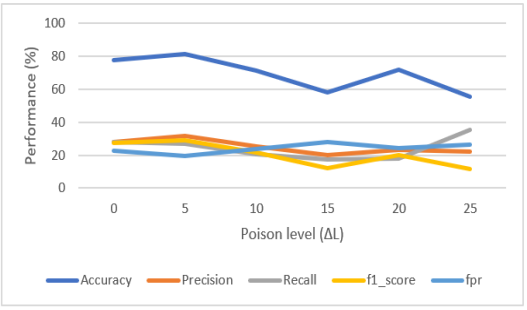
Figure 4.4: Performance analysis of random forest with consistent poisoning



(a) Poisoning gaussian naive bayes with IRIS dataset



(b) Poisoning gaussian naive bayes with MNIST dataset

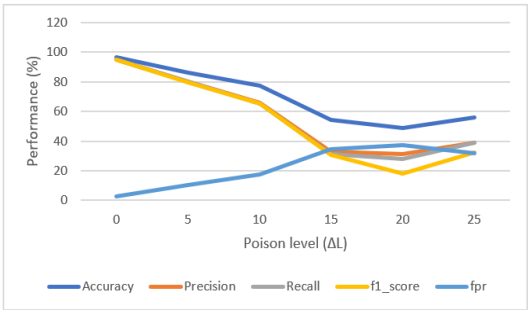


(c) Poisoning gaussian naive bayes with ISIC dataset

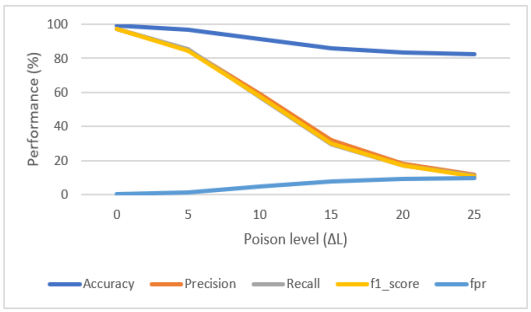
Figure 4.5: Performance analysis of gaussian naive bayes with consistent poisoning

4.6.3 Model Sensitivity to Poison

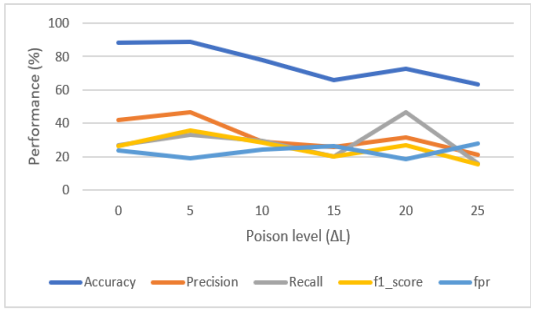
The sensitivity of the poisoned model is investigated by analysing the relationship between model variance and ASR. Table 4.11 illustrates the variance in ML models in response to the OOP attack.



(a) Poisoning k-nearest neighbors with IRIS dataset

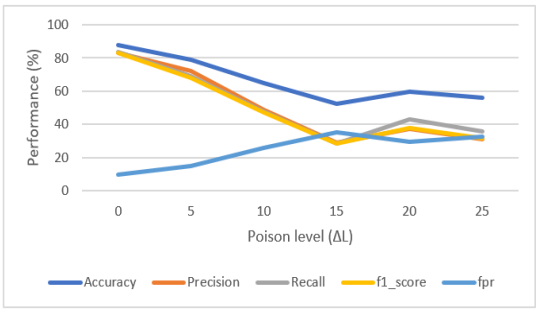


(b) Poisoning k-nearest neighbors with MNIST dataset

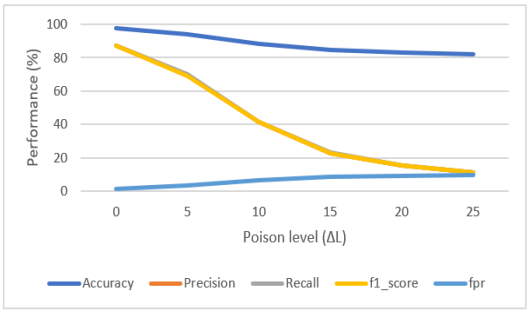


(c) Poisoning k-nearest neighbors with ISIC dataset

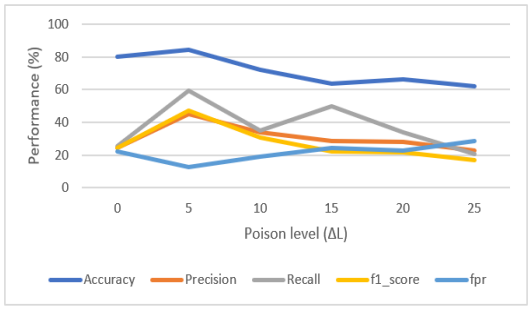
Figure 4.6: Performance analysis of k-nearest neighbors with consistent poisoning



(a) Poisoning decision tree with IRIS dataset



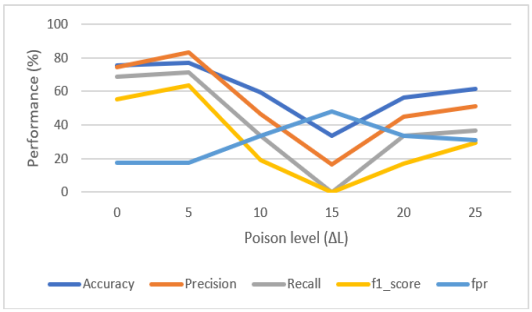
(b) Poisoning decision tree with MNIST dataset



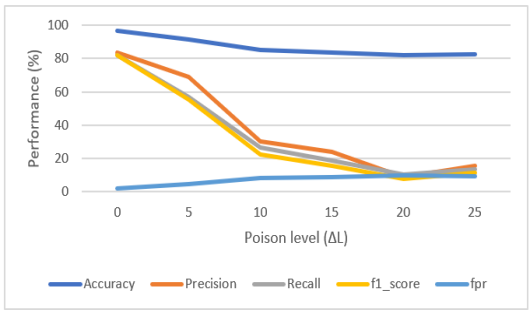
(c) Poisoning decision tree with ISIC dataset

Figure 4.7: Performance analysis of decision tree with consistent poisoning

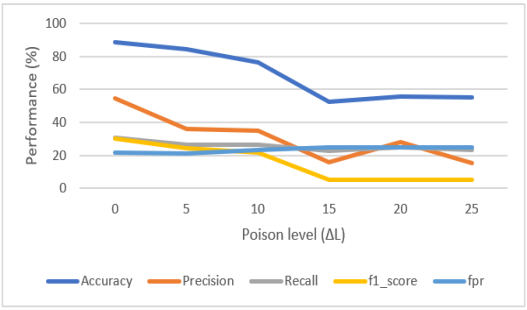
This attack significantly increased the sensitivity of all tested models, with GNB exhibiting the highest sensitivity. Its variance leads to 0.8 at $\Delta L = 10\%$, for the IRIS dataset, almost equivalent to DT, where it fails. Similarly, 0.10 variance increases for KNN at $\Delta L = 15\%$, highlighting its high



(a) Poisoning multilayer perceptron with IRIS dataset



(b) Poisoning multilayer perceptron with MNIST dataset



(c) Poisoning multilayer perceptron with ISIC dataset

Figure 4.8: Performance analysis of multilayer perceptron with consistent poisoning

sensitivity and the effectiveness of the OOP attack. RF and DT are proven to be less sensitive to this attack. Interestingly, on average, models trained with MNIST and ISIC are also less affected by the poisoning attack compared to models trained with the IRIS dataset, with high impact.

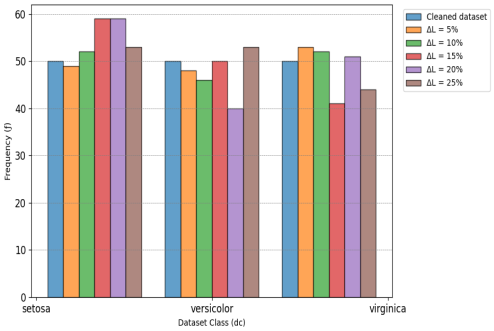
Further analysis is conducted on the dataset distribution to ascertain its impact on data poisoning and performance degradation in models. Fig. 4.9 shows the change in data distribution with the OOP attack at $0\% \leq \Delta L \leq 25\%$. The findings suggest that balanced datasets with a larger number of classes tend to mitigate the effects of poisoning on model performance, particularly in terms of model accuracy. In contrast, imbalanced and noisy datasets work as catalysts and boost the poisoning effects of this attack, leading to an impractically high decrease in performance, such as for the ISIC dataset, as shown in Fig. 4.9(c). This analysis identifies relationships between various classification characteristics and subsequent rates of data poisoning, as given in Table 4.12.

Table 4.11:
Model variance at different poisoning levels

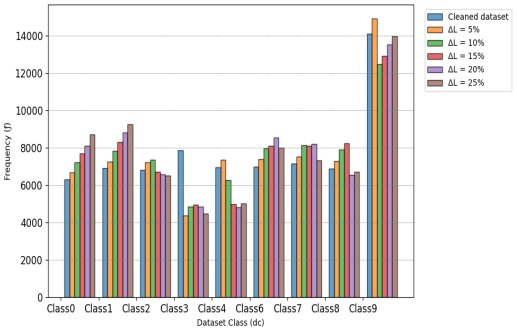
Dataset	Algorithm	Clean Dataset	$\Delta L = 10\%$	$\Delta L = 15\%$
IRIS	SVM	0.33	0.36	0.57
	RF	0.62	0.60	0.63
	GNB	0.65	0.73	0.68
	KNN	0.81	0.82	0.91
	DT	0.59	0.68	0.78
	MLP	0.65	0.69	1.45
MNIST	SVM	8.33	8.06	7.97
	RF	8.24	7.69	7.71
	GNB	11.25	12.68	12.74
	KNN	8.36	8.37	8.38
	DT	8.33	7.81	8.02
	MLP	8.31	8.38	8.31
ISIC	SVM	1.33	0.97	1.36
	RF	1.11	1.17	1.32
	GNB	1.27	1.66	1.19
	KNN	0.31	0.37	0.27
	DT	0.31	0.37	0.27
	MLP	1.52	1.48	1.59

Table 4.12:
Analysing one-to-one relation between poison and various parameters of machine learning algorithms

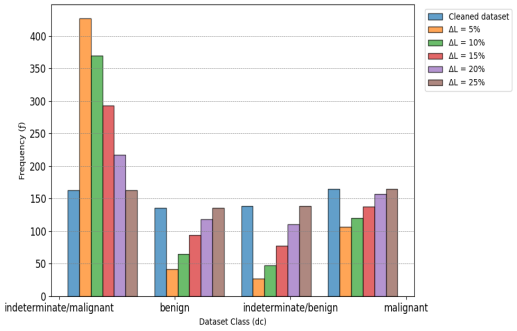
Algorithm	Algorithmic Parameters	Relation to ΔP
SVM	Margin score	Minimal impact
	Decision boundary	Minimal impact
	Features importance score	Minimal impact
DT	Features importance score	Minimal impact
	Asymmetric features space	High impact
KNN	Decision boundary	High impact
	k-neighbors	Inverse impact
GNB	Decision boundary	High impact
	Class probabilities	High impact
RF	No. of trees	Inverse impact
	Features importance score	Minimal impact
MLP	Weights	High impact



(a) Data distribution of IRIS dataset with OOP attack



(b) Data distribution of MNIST dataset with OOP attack



(c) Data distribution of ISIC dataset with OOP attack

Figure 4.9: Data distribution with OOP attack

4.7 Case Study 1: OOP Attack Against Adversarially Trained Skin Cancer Diagnostics

The ML systems have seen wide adoption in healthcare, providing proficiency in handling complex decision-making tasks and optimising their diagnostic processes efficiently. Skin cancer is a severe disease with a high frequency rate. Due to its potential seriousness and prevalence, an early diagnosis is crucial for its treatment. Traditional diagnosis requires biopsy and visual examination, which is time-consuming and subject to human error. In this context, ML tools show promising results. These models can handle various data types and ensure efficient and optimised diagnosis capability compared to manual processing.

On the other hand, data poisoning Lu et al. (2023), Wei et al. (2023) can poison the ML model's dataset and harm the victim model's diagnosis mechanism. Various solutions are provided in the literature to mitigate such data poisoning attacks, including data sanitisation (Paudice et al., 2018b), ensemble learning (Ahmed et al., 2022), and adversarial training (Tao et al., 2021). Adversarial training is one of the prominent solutions through which the ML model is trained with a combination of cleaned and poisoned data samples to allow the model to understand the patterns of the poisoned data. This mechanism makes the model resilient and enables it to surpass similarly poisoned dataset samples if injected.

However, adversarial training can be breached with novel data poisoning attacks Wen et al. (2023), Tao et al. (2022b). This study leverages the capabilities of the OOP attack and analyses its effectiveness against adversarial training. Another important consideration of this case study is that adversarial training is mostly analysed to secure DL models, as discussed in Section 2.4 in Chapter 2, whereas this case study analysed the significance of adversarial training on the traditional multiclass models in non-complex settings. The contribution of this case study 1 is to conduct a thorough analysis of the OOP attack against adversarially trained multiclass SVM, RF, and MLP models.

4.7.1 Attack Model

A grey-box approach is followed to poison the adversarially trained skin cancer application, developed in multiclass model settings. The attack model comprises four steps as given in Fig. 4.10. The details of each stage of the attack model are given as follows.

Attack Generation Generating appropriate data poisoning attacks, the poisoning level of 10% is set for the complete dataset to achieve a maximum effectiveness of the data poisoning against adversarially trained models. Following research (Paracha et al., 2024a), 10%-15% of dataset poisoning is the most effective poisoning level. Subsequently, following the threat model, the OOP attack is implemented and set the dataset=ISIC dataset, and algorithm=targeted model algorithm with different

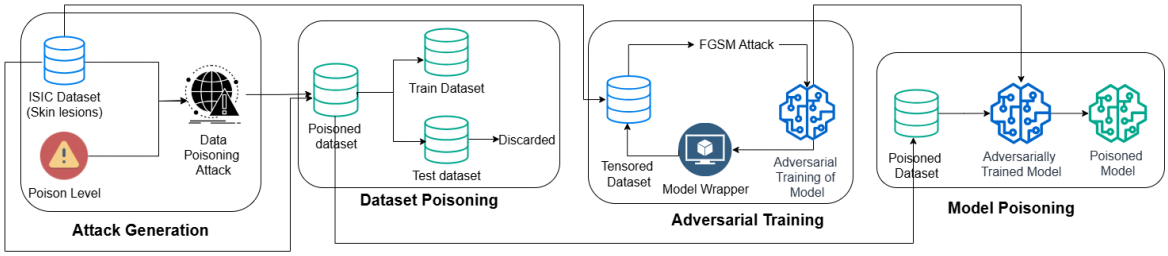


Figure 4.10: Overview of the attack model - Analysing efficacy of outlier data poisoning attack against adversarially trained models for skin cancer diagnosis

Table 4.13: Configurations of the models

S.No.	Model	Model Configurations
1	SVM	decision_function_shape='ovo', kernel='poly', degree=3, C=1
2	RF	criterian='log_loss', n_estimators=9, max_depth=4
3	MLP	penalty='elasticnet', alpha=0.0001, max_iter=1000

configurations.

Dataset Poisoning At the dataset poisoning stage, the data poisoning attack with the ISIC dataset is implemented and generated a poisoned version of the dataset, implemented with simple baseline algorithms: SVM, RF, and MLP.

Adversarial Training Using the baseline line algorithms for SVM, RF, and NN, the wrapper models were developed for each baseline model to accommodate adversarial training with the tensor-formatted dataset. The cleaned dataset D is distributed $dict(D)$ into training and testing with a 75%-25% percentage. The algorithm to perform the adversarial training of the baseline model is given in Alg. 5.

Model Poisoning Access to the victim model that is adversarially trained to be resilient against data poisoning, the model is evaluated against an outlier data poisoning attack (Paracha et al., 2025a) to assess the reliability of the adversarial training to secure baseline ML models. Furthermore, the experimentation highlighted that adversarially trained baseline ML models are highly affected by data poisoning attacks. The attack model implementation is given in Alg 6.

4.7.2 Experimentation Results and Analysis

The experimentation is conducted following two different dataset distributions. 75%-25% dataset distribution is followed to develop the resilient models with adversarial training and for performance analysis. Whereas, the adversary follows 80%-20% dataset distribution to poison the dataset. This setup allowed us to assess how data poisoning deceives the skin cancer diagnostic application even when the adversary follows a different data distribution. This case study analyses the resilience of

Algorithm 5 Adversarial Training of Baseline ML Model

$training_epochs = 15$
$pert_epsilon = 0.1$
$batch_size = 4$
procedure ADVERSARIAL_TRAINING(x_train,y_train,$model$)
$N = length(y_train)$
▷ X_train is the training dataset and y_train are the training dataset labels
$optimiser = SGD_Optimiser(model.parameters(), lr = 0.1)$
$model \leftarrow training_state$
for epoch in 1,2 ... training_epochs **do**
$perm = random_permutation(N)$
$sum_loss = 0$
for i in 1,2 ... batch_size **do**
$data_sample = x_train[perm[i : i + batch_size]]$
$data_sample_label = y_train[perm[i : i + batch_size]]$
$gradient(optimiser) = 0$ ‘
$output = squeeze(model(data_sample))$
$weight = squeeze(model.weight)$
$loss = mean(clamp(output, -1, 1))$
$loss = loss + 0.01 * (weight.t()@weight)/2.0$
$loss_gradient = gradient(loss)$
$stepper(optimiser)$
$data_grad = gradient(data_sample)$
$x_adv = FGSM_ATTACK(data_sample,$
$epsilon,$
$data_grad)$
▷ FGSM attack is one of the principal data poisoning attacks
$adv_train_x = merge(x_train, X_adv)$
$adv_train_y = merge(y_train, y_adv)$
end for
end for
$train(model, adv_train_x, adv_train_y)$ ▷ performing adversarial training of the model
end procedure
return *model*

the models, their variance, and the impact of the OOP attack against traditional ML. This study assesses the limitations of adversarial training against novel data poisoning attacks.

The performance of the adversarially trained models is evaluated against their poisoned versions. Fig. 4.11 highlights the decrease in model accuracy when trained with the poisoned dataset compared to its counterpart. Data poisoning attack shows its effectiveness against adversarially trained ML models. The results highlighted that adversarial training profoundly secured neural networks against new data poisoning attacks, whereas it does not work against other baseline models. Adversarially trained SVM and RF models are vulnerable to data poisoning. Table 4.15 provides the test-time accuracy, precision, recall, f1-score, and false positive rate of cleaned and poisoned models with poisoning rate $\Delta L = 10\%$. From these results, SVM is the most affected model with the highest decrease in test time accuracy of around 50%; however, the accuracy of the neural networks model only decreased to 2%.

This analysis is extended to study the effects of data poisoning on each dataset class. The test time dataset comprises 151 instances in total, from which the biggest class is **Benign(Non-cancerous)**,

Algorithm 6 Poisoned Training Of The ML Model

Inputs: ISIC Training Dataset = D_{ISIC}
Outputs: Poisoned Model = M'_{SVM}, M'_{RF}, M'_{NN}
Initialise: Wrapper Model = W_M,
Neural Networks Module ($module = torch.nn.Module$),
Number of Dataset Classes ($N_c = 4$)
Poison Level ($\Delta L = 10\%$),
Adversarial Training ($Adv_t = Adv(Attack = FGSM)$),
Poisoning Attack = $Attack_{outlier}$
algorithms = ['SVM', 'RF', 'NN']

for alg in algorithms **do**
$M_{wrapper} = W_M(module, alg) \leftarrow size(D_{ISIC}), num_classes)$
end for
$D_{train}, D_{test} = split_dataset(D_{ISIC}, train_size = 75\%, test_size = 25\%)$
$X_train_tensor = inst_to_tensor(D_{train})$
$y_train_tensor = inst_to_tensor(labels(D_{train}))$
$M_{wrapper} \leftarrow Train_SVM(X_train_tensor, y_train_tensor, M_{wrapper}, FGSM_Attack)$
$D'_{ISIC} = Attack_{outlier}(D_{ISIC}, M_{wrapper}, poison = 10\%)$
$D'_{train}, D'_{test} = split_dataset(D'_{ISIC}, train_size = 80\%, test_size = 20\%)$
$X_poisoned_tensor \leftarrow inst_to_tensor(D'_{train})$
$y_poisoned_tensor = inst_to_tensor(labels(D'_{train}))$
$M'_{wrapper} \leftarrow Train_M_{wrapper}(X_poisoned_tensor, y_poisoned_tensor, M_{wrapper}, FGSM_Attack)$
return $M'_{wrapper}$

having 115 samples and **Indeterminate/malignant** is the smallest class of 5 samples only. Table 4.14 provides the class-level test time misclassification rate with 10% training time poison.

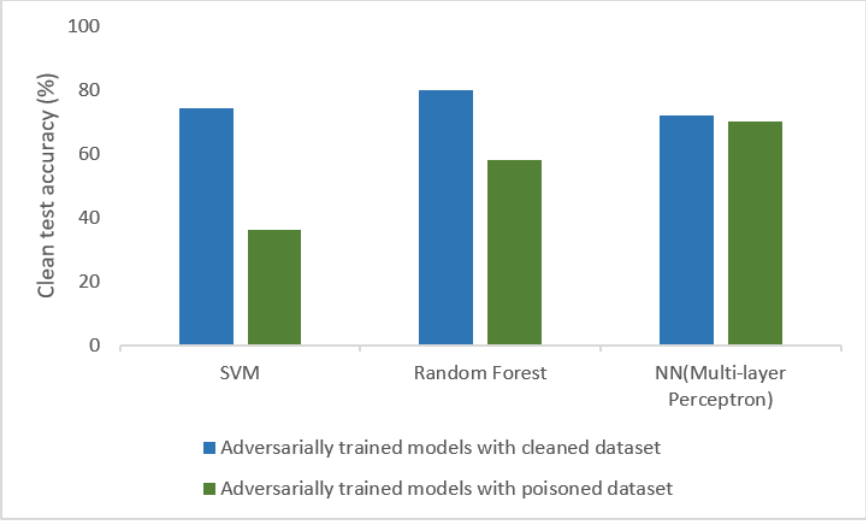


Figure 4.11: Model accuracy - Decrease in model's performance with data poisoning attack

This study investigated the relationship between model variance and the data poisoning attack at $\Delta L = 10\%$. Fig. 4.13 illustrates an increase in model variance and attack success rate with poisoned training. MLP has shown the lowest variance increase of 0.02, whereas SVM exhibits high sensitivity against a data poisoning attack with a variance of 1.15. Consequently, RF is a stable model with the

Table 4.14: Misclassification rate for each dataset class

Model	Dataset	Benign= 115	Malignant= 24	Indeterminate/ Benign=7	Indeterminate/ Malignant=5	Misclassification Rate
SVM	Cleaned ISIC	3	24	5	1	0.22
RF	Cleaned ISIC	5	20	6	1	0.21
MLP	Cleaned ISIC	2	24	6	1	0.21
SVM	Poisoned ISIC	24	25	5	1	0.36
RF	Poisoned ISIC	63	13	2	0	0.51
MLP	Poisoned ISIC	27	14	6	1	0.31

Table 4.15: Classification results of adversarially trained models

Model	Accuracy	Precision	Recall	F1-score	FPR
Clean SVM	74.5	30.35	29.62	29.88	21.06
Poisoned SVM	36	29.60	28.08	27.78	24.63
Clean RF	80	52.81	28.72	27.87	22.77
Poisoned RF	58.2	23.87	26.15	22.82	23.53
Clean MLP	72	52.55	26.27	24.48	24.25
Poisoned MLP	70.1	50.05	26.63	23.91	24.23

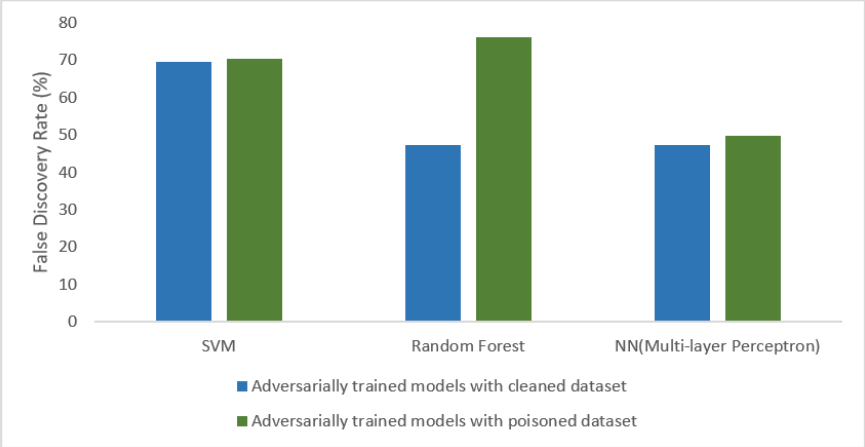


Figure 4.12: Model's false discovery rate - Increase in the false diagnosis rate of the models when trained with the poisoned dataset

least variance of 0.28 for its cleaned version, increasing to 0.60 with poisoned training. Overall, the highest impact of poisoning is visible on the SVM model.

This research examined the impact of the outlier data poisoning attack on the decisions of the individual models by taking random samples. It analysed the change in the confidence scores of randomly taken samples from the poisoned models and their impact on the application of healthcare diagnosis. Results of change in diagnosis confidence can be seen in Fig. 4.14 to Fig. 4.16. From these results, the least disruption of 0.1/0.86 prediction confidence for the Benign(Non-cancerous) sample is shown from the poisoned MLP model. Whereas, training time poisoning is shifting a Benign(Non-cancerous) sample to a Malignant(Cancerous) with a slight change in confidence that is 0.2/0.55 from Benign(Non-cancerous) to Indeterminate/Benign, in Fig. 4.14. However, a complete change in

the diagnosed class confidence of 100%, for the poisoned SVM model, as seen in Fig. 4.15. Conclusively, adversarial training is effective in making neural networks resilient against novel data poisoning attacks, whereas it is ineffective for other baseline ML models, RF, and SVM in this research.

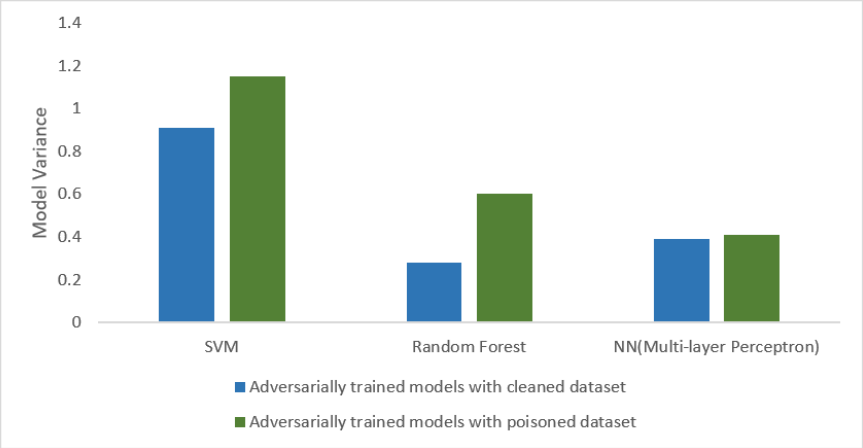


Figure 4.13: Model variance - Sensitivity of the models against data poisoning attack

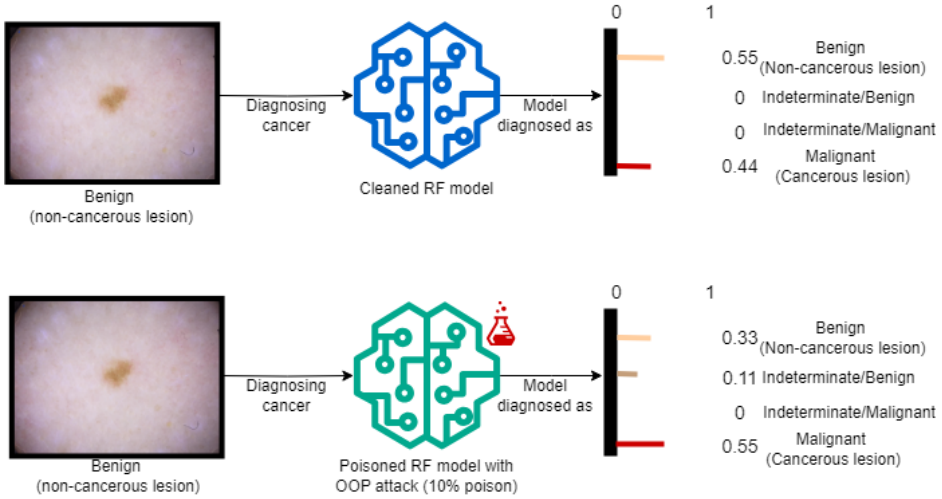


Figure 4.14: Random forest poisoned sample 1 - Analysing change in confidence score with poisoned training

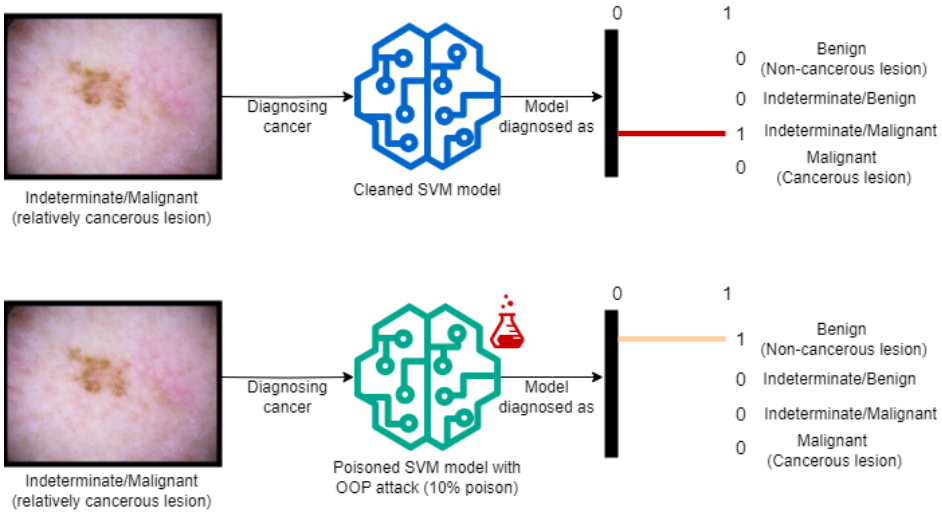


Figure 4.15: Support vector machines poisoned sample 1 - Analysing change in confidence score with poisoned training

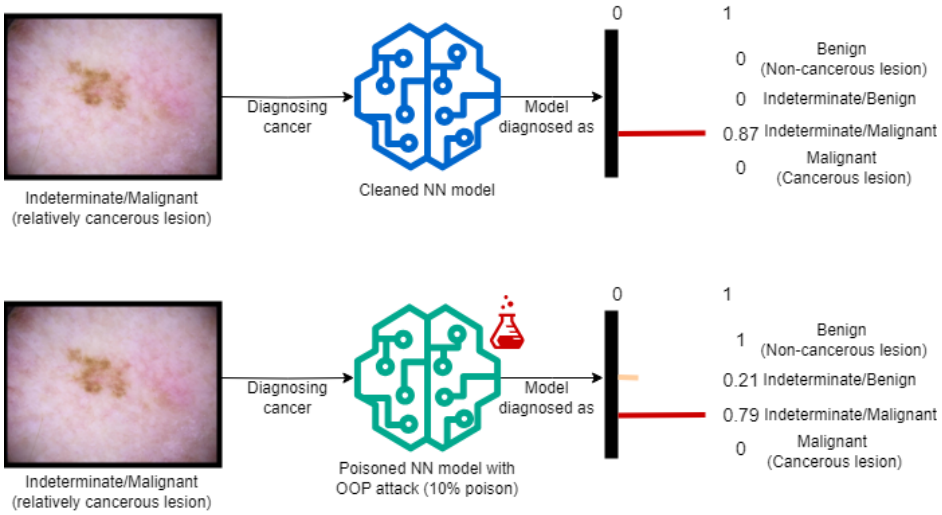


Figure 4.16: Multilayer perceptron poisoned sample 1 - Analysing change in confidence score with poisoned training

4.8 Case Study 2: OOP Attack Against Industrial 5G Private Network

The technological industrial revolution, known as "Industry 4.0", is receiving steady adoption across a wide range of industries. These industrial transformations are driven by digital transformation, enabled by advanced connectivity, artificial intelligence, and robust computing. ML is playing a vital role in developing, managing and optimising the systems and networks in Industry 4.0. Given the enabling capabilities of these technologies and the significance of their application in different sectors, the impact of cyber security threats and the consequential effect on industrial outputs can not be overemphasised.

This study explores the performance disruption and impact of data poisoning within an industrial context, with a use case of a material recycling facility (MRF). The work presented in (Baiyekusi et al., 2024), showed the value of applying an ML-based approach in spectrum sharing within industrial MRF settings. In this research paper, the authors have highlighted the significance of unlicensed spectrum in 5G networks (5G NR-U) and IEEE 802.11ax networks. By analysing the flexibility limitations of using these networks in industrial use cases, they proposed an ML-based data-driven approach to calculate the optimal performance metrics in shared unlicensed bands between these networks. Despite the usefulness of these ML models, they are found to be highly vulnerable to various adversarial attacks Sharma et al. (2019), Tu et al. (2021). Data poisoning attack Li et al. (2024), Wang et al. (2023) is one of these attacks which poison the training dataset of ML models and transmit poison to these cellular networks. However limited research is provided to study the impact of data poisoning on unlicensed shared spectrum networks in industrial scenarios.

This case study leverages the implementation of a novel multiclass data poisoning attack, based on the multi-network scenario adopted in (Baiyekusi et al., 2024), to poison the data-driven model predictions in 5G NR-U and 802.11ax shared spectrum networks. With this experimentation, the effectiveness and impact of multiclass data poisoning in an industrial scenario were analysed. Four supervised models: SVM, RF, DT and MLP were selected. The contribution of this case study is to analyse the impact of a novel poisoning attack in an industrial 5G private network and evaluate the efficacy and impact of data poisoning in 5G and 802.11ax WIFI shared-spectrum networks.

4.8.1 Attack Model

This case study describe the scenario of the MRF. The MRF consists of a network of conveyor belts through which the recycled wastes are moved through the facility to be sorted by robotics arms. The robotic arms are equipped with cameras that feed the edge server with live video of moving waste materials. Object detection is performed at the edge server using computer vision, and the robotic arm selects any material of interest to be separated and recycled. Due to stringent data rates and

delay requirements, the network performance needs to be optimal and stable, otherwise, this could significantly impact MRFs operations negatively. The robotic arms are connected to the edge server via the 5G NR-U network. The 5G NR-U network offers resilience in data rates and delay through its error correction handling scheme. For monitoring purposes, the MRF is also equipped with digital display boards which offer visualisation of operations. This displays also offer live feed from CCTV around the MRF. The network traffic to the digital display screens have lesser priority but still required to function at an acceptable level. These digital display boards are connected to the edge server through the 802.11ax network. Given, the coexisting scenario of both 5G NR-U and 802.11ax, the ML-based spectrum sharing technique proposed in (Baiyekusi et al., 2024), ensures each network maintains expected performance based on the number of nodes contending over the channel and the expected data rates.

The ML-based spectrum sharing scheme proposed in (Baiyekusi et al., 2024) is predicated based on

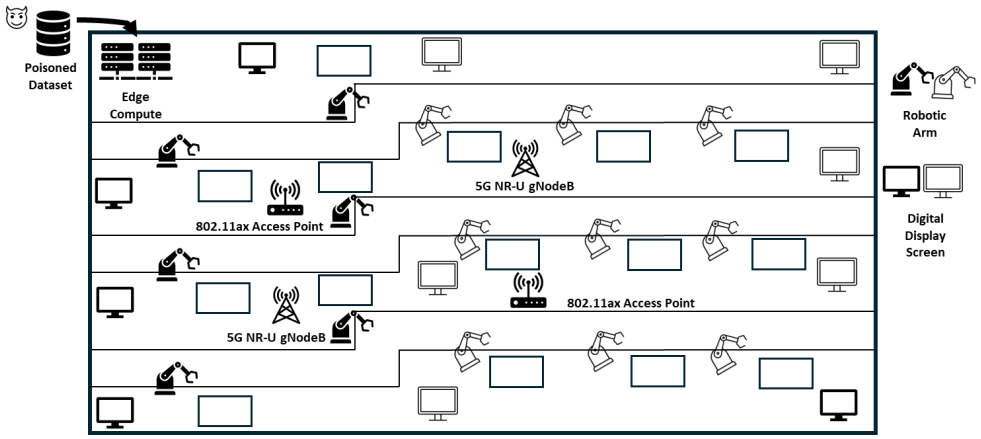


Figure 4.17: Material recycling facility scenario - A network of conveyor belts

5G NR-U and 802.11ax and operates relatively similar channel access schemes. The channel access scheme follows an arbitration process where transmission over the channel is made after channel sensing is performed and a random backoff period has been observed. The random variable for the backoff period is selected using a uniform distribution, which ideally offers fairly similar channel access to all nodes contending for transmission over the channel. The ML-based spectrum scheme proposed enables estimation of the number of nodes contending over the channel in a mixed technology scenario and adjusts the contention window for each node based on the network it is transmitting over i.e. either 5G NR-U or 802.11ax.

Experimental Dataset This case study has extended the dataset, developed in the research paper (Baiyekusi et al., 2024). This dataset is developed in MATLAB. This dataset is created with various on-networks devices in the range of 8-40 at a scale of 8. These devices are sharing the network spectrum of 5G NR-U and 802.11ax networks. Onboarding different devices on different range between

8-40, data rates and bandwidth of these devices is calculated under various transmission time and transmission delay. Transmission time and transmission delay is set to 5ms, 6ms, 8ms and 10ms. Data rates and bandwidth of these devices are measured at different contention stages for both 5G NR-U and 802.11ax networks. The contention stages to five states from 1-5 with a minimum $32 \leq$ contention rate $\leq 2^n$ and n is in the range of 6-10 at the scale of 1. The details of the dataset properties are given in Table 4.16.

Attack Method In this case, the dataset captured in real-time from the 5G network is poisoned.

Table 4.16: Properties of the shared-spectrum 5G NR-U and 802.11ax dataset

S.No.	Dataset Property	Property Description
1	No. of nodes	Number of devices on network
2	Contention stage	Wait after transmission frame
3	Data rate	Data rates based on transmission time
4	Transmission delay	Delay transmitting data

Following the threat model, only access to the dataset is provided to poison without providing any other knowledge of the target system. The details of the attack method are given as follows.

The OOP attack on the shared-spectrum 5G NR-U and 802.11ax networks dataset is implemented to structure a successful attack within an industrial 5G private network. Considering realistic attack settings, the attack in a grey-box scenario is implemented without leveraging the adversary to access the target model. Only dataset access is provided to the adversary; therefore the attack is implemented based on surrogate models to poison the dataset. The mathematical notation of developing surrogate models M_s is given in Eq 4.16.

$$M_s = \mathcal{T}(M(D;\theta), \Delta L) \tag{4.16}$$

where, D is the 5G private network dataset, θ are the model configurations and poisoning level $\Delta L = (10, 15, 20)\%$. The surrogate model M_s allows the adversary to develop the poisoned datasets D'. The mathematical notation for developing D' is given in Eq 4.17.

$$D' = \sum_{i=1}^{n \to \Delta L} f_p(f_{l'}([max\ \vec{d}(x_i)], l'_i), \Delta L) \tag{4.17}$$

where, $f_{l'}$ is the function of manipulating the label l of the targeted data point x and changing it to a new label l', f_p is the function of developing poisoned dataset D' with a dependent function $f_{l'}$ with poisoning level $\Delta L = (10, 15, 20)\%$. $f_{l'}$ is the function of updating the label of the farthest data point in the dataset. Consequently, by providing the poisoned dataset to the ML model, the

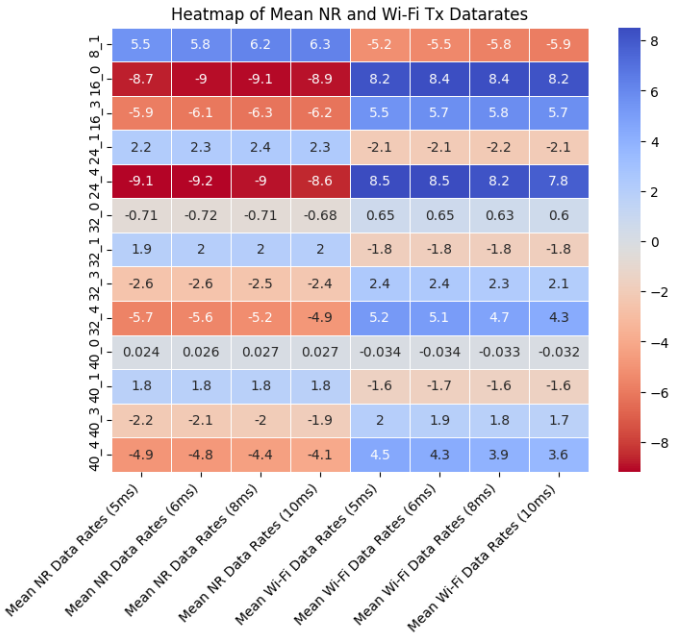


Figure 4.18: Mean data-rate poisoning performance results for multilayer perceptron showing performance across multiple nodes and contention window stages

shared-spectrum network is poisoned.

By implementing the OOP attack, the ML model has misclassified contention rates which subsequently manipulate data rates and transmission rates, leading to a disruption in the network. With this network poisoning, the impact of data poisoning attacks on 5G and 802.11ax shared-spectrum networks is evaluated at various poisoning levels. Overall, it shows a high impact on the optimised shared-spectrum network implemented with SVM followed by DT. Furthermore, Section 4.8.2 gives an in-depth analysis of the experimentation results.

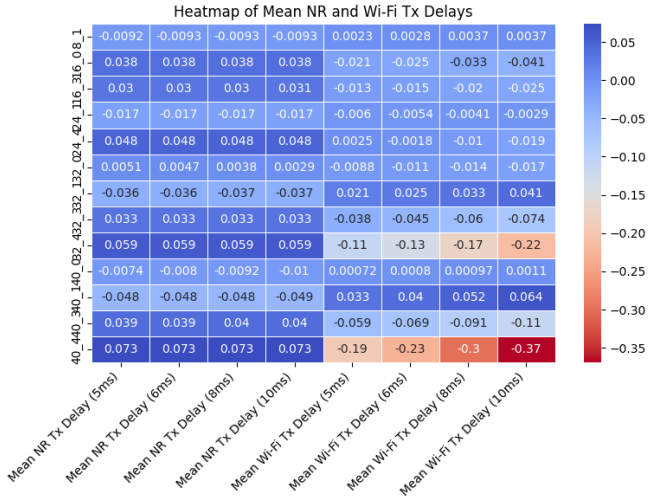


Figure 4.19: Mean delay poisoning performance results for multilayer perceptron showing performance across multiple nodes and contention window stages

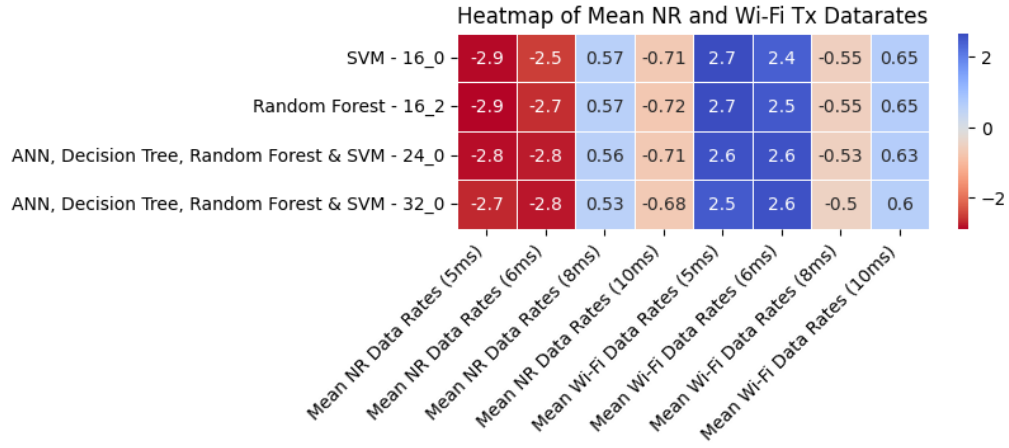


Figure 4.20: Mean data-rate poisoning performance results for other models showing performance across multiple nodes and contention window stages

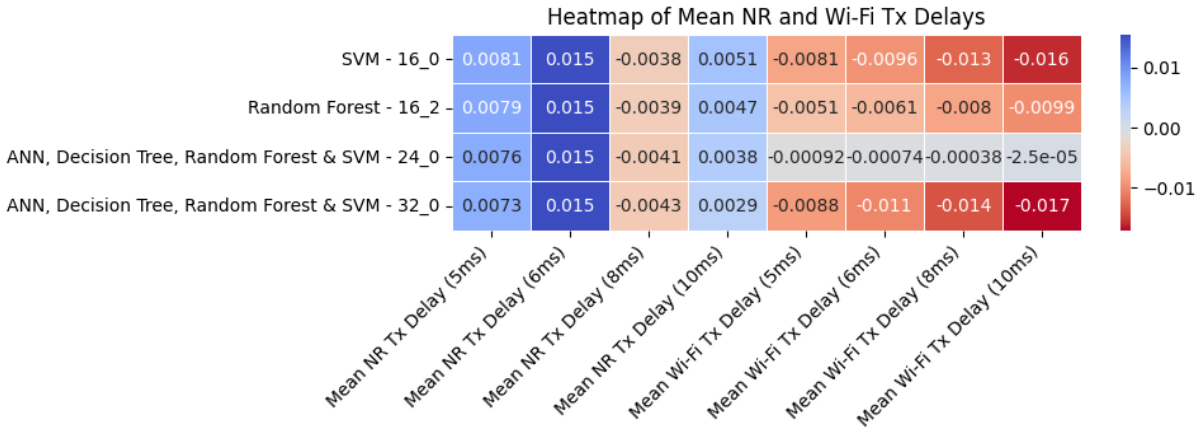


Figure 4.21: Mean delay poisoning performance results for other models showing performance across multiple nodes and contention window stages

4.8.2 Experimentation Results and Analysis

The performance of various poisoned ML models against multiclass data poisoning in a shared-spectrum private 5G network is assessed to analyse their performance against cleaned models. Fig. 4.22 highlights the decrease in model accuracy with the OOP data poisoning attack. A linear decrease is visible on the test time accuracy of models with an increase in data poisoning level ΔL where $10\% \leq \Delta L \leq 20\%$ at a scale of 5. SVM is the most affected model and MLP has minimal effects of data poisoning. Further, Fig. 4.23 highlights an abrupt change in the false discovery rate of models where RF and DT are highly disrupted at $\Delta L = 20\%$ and SVM and MLP show the highest error rates at $\Delta L = 15\%$. Overall, SVM has shown the highest poisoning effects with a decrease of 6.6% accuracy and MLP proved to be more stable against data poisoning with 2.9% accuracy decrease at $\Delta L = 20\%$.

The network performance results shown in Fig. 4.20 to Fig. 4.21 highlight the impact of the data

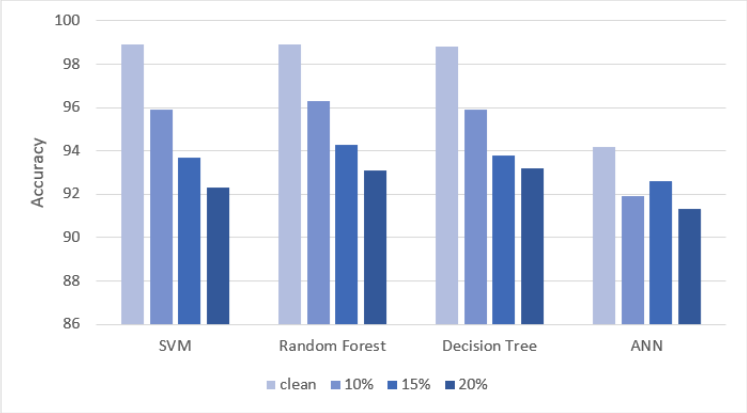


Figure 4.22: Decrease in model accuracy with data poisoning attack

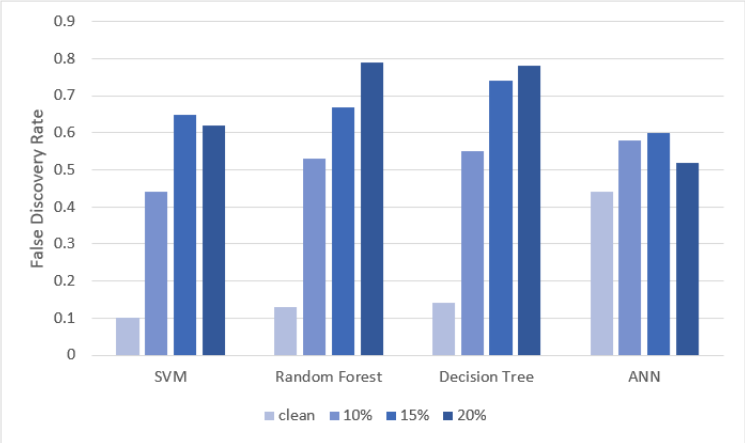


Figure 4.23: Increase in model false discovery rate with data poisoning attack

poisoning attack. The data rates and transmission delay constitute the main metrics for evaluating the impact on performance. Fig. 4.20 shows a heatmap of the impact of the poisoning attack on the MLP model on NR and WiFi data rates in light of their severity. It can be seen that 5G NR-U is more impacted. This may be due to 5G NR-U network's slightly better modulation and coding scheme (MCS) and less management data overheads. Consequently, the impact of the poisoning attack is more noticeable for 5G NR-U when node numbers and contention windows are wrongly estimated. Given the industrial scenario being evaluated, the robotic arms operate over the NR networks. The wide variations with smaller nodes, e.g. 16 and 24 nodes, indicate a significant impact on the performance of the robotic arms across the recycling centre. For instance, a drop in data rates by 8.7Mbps for each robotic arm will lead to significant degradation in the operations of the robotic arm and potentially reduce the MRF's output. From a delay perspective, the 802.11ax network shows a more negative impact of the poisoning attack. The delay reflects more on 802.11ax due to additional overheads for data transmission; hence, any wrong estimation of the number of nodes and contention window will further impact the delay profile and performance of the 802.11ax network.

4.9 Discussion and Limitations

A novel grey-box attack is formalised to attempt poisoning multiclass models, describing their efficacy and analysing the factors affecting their classification behaviour. Although several adversarial poisoning techniques are proposed in the literature, limited experimentation is provided on multiclass classifiers. Existing research papers Steinhardt et al. (2017), Chen and Koushanfar (2023), Hayase et al. (2021), Weerasinghe et al. (2021) proposed solutions that focus on discrete data set features and detect outliers to lessen poisoning effects. The attack has taken the outliers into the feature space that effectively poisoned the model. Following this, certain factors affecting individual algorithms were highlighted and effective levels of poisoning for parametric and non-parametric multiclass models were also determined. These results showed that a 10% poisoning rate shows the highest performance degradation for parametric models and 15% for non-parametric models. At these optimal poisoning, a lower level of model sensitivity is analysed, which does not allow the model to over-fit, highlighting the efficacy of the OOP attack.

Implementing the OOP attack, this chapter conducted a deep behavioural analysis of multiclass ML, identifying factors affecting the confidence of models. From these results, GNB and KNN are found to be highly affected by this poisoning attack, whereas DT and RF are less affected models. Manipulating the outliers class label, class probabilities of GNB, and proximity distance calculation of KNN are highly disrupted.

These results highlighted that the dataset size and number of classes are inversely proportional to poisoning effects. Whereas, an accelerating impact of an imbalanced dataset on model poisoning. Imbalanced classes in multiclass datasets help penetrate poison in the model effectively, to an extent. Also, a fundamental relation between dataset noise and data poisoning is found where dataset noise works as a catalyst towards poisoning, but becomes impractical with poisoning level $> 15\%$ with unrealistic performance degradation.

Following these results, two case studies were conducted to examine the impact of OOP poisoning in real-world applications. In this first case study, the efficacy of adversarial training is studied against data poisoning attacks in two traditional ML models in non-complex settings. Various literature studies Shi et al. (2018), Lu et al. (2024) experimented data poisoning against adversarially trained models mostly on DL models, including many complex Convolutional Neural Networks(CNN). This case study analysed how data poisoning, if 10% poison is injected, impacts adversarially trained SVM, RF, and MLP in their simplified forms. These results showed that SVM is highly vulnerable to data poisoning and adversarial training does not work on SVM and RF, as expected. However, is significant in making MLP resilient.

These results highlighted that after implementing adversarial training with the FGSM attack, the

integrity of the MLP model is intact and is not affected by data poisoning attacks. In contrast, the model integrity of the SVM model is breached. The test time prediction variance of the SVM model is significantly high when trained with a cleaned dataset of 0.9 and a higher variance of the model is calculated when trained with 10% poisoned dataset reaching 1.15. Whereas, RF and MLP have significantly lower variance compared to SVM. Because the attack is implemented perturbed outliers and RF is resilient to outliers, in general, this leads to a lesser impact on the RF model.

Trustworthiness and robustness of smart healthcare applications should be of the highest priority. The results highlighted a major impact of data poisoning that worsened the diagnosis mechanism of these applications. The results of this case study highlighted the need to secure the traditional ML models that provide good results in the diagnosis of diseases at an early stage. But these models are the target of adversaries who try to spoil these security-sensitive applications.

Next, the impact of OOP poisoning in 5G industrial testbed network is analysed. The results from this study show the impact of data poisoning on a private 5G NR-U and 802.11ax network. It is clear, depending on the level of poisoning, data poisoning introduces instability in the network, causing the network to perform differently from how it is designed to operate. Hence, the attack can significantly degrade the performance and consequently the industrial output, making it not fit for purpose. The results show a high impact of data poisoning in violating the integrity of advanced networks. Data poisoning against four ML models is evaluated, implemented for network optimisation. This analysis highlighted that data poisoning has minimal impact on the MLP at all three poisoning levels, whereas the highest disruption in the integrity of SVM can be seen. Fig. 4.20 shows the highest impact on SVM with a value of -2.9 NR data rates transmission with 16 nodes, and similarly, Fig. 4.21 highlights a maximum transmission delay of -0.016 and -0.017 for SVM with 16 and 32 nodes, respectively. Also, Fig. 4.22 shows a continuous decrease in performance for SVM, RF, and DT irrespective of their design, highlighting the efficacy of data poisoning on wireless networks.

This chapter is limited to the analysis of classification algorithms which can be extended to the regression algorithms. With this limitation, the factors affecting classification behaviours and their confidence in this poisoning attack were analysed. Also, comparing this attack with existing attacks from the literature helps demonstrate the efficacy of the OOP attack which is also out of the scope of this chapter.

4.10 Summary

This chapter extends the answer to RQ 2 to highlight risks, impact, and consequences of novel data poisoning attacks in real-world applications in multiclass settings. It analyses the behaviour of multiclass models developed for real-world applications, identifying individual characteristics of the algorithms against OOP data poisoning to understand how the decision process can be compromised

in multiclass classifiers. It also addresses RQ 3 by highlighting the limitations of adversarial training in securing traditional ML models against data poisoning attacks and the need for improved mitigation solutions to secure ML models from poisoning attacks.

This chapter examines the consequences and impact of data poisoning in healthcare and Industry 4.0 network applications, emphasizing the necessity for robust and generalisable security solutions that would be applicable to both ML and DL models. This examination assessed individual algorithms against the OOP attack, identifying their key vulnerabilities in their characteristics. These results and limitations of existing mitigation solutions enable us to propose an improved attack-agnostic solution in the next chapter. Such a solution will address the limitations of existing adversarial training mechanisms and is adaptable to both ML and DL models.

Chapter 5. SecureLearn: Improvised Machine Learning Defence

5.1 Introduction

Chapters 3 and 4 demonstrates the effectiveness of data poisoning attacks in binary and multiclass classification models and highlight the limitations of existing defences against these attacks. Chapter 3 determined the underlying changes in the learning dynamics with poisoned training, whereas Chapter 4 emphasised the impact of data poisoning by exploiting a training characteristic of ML. These findings highlight the need to develop an attack-agnostic solution, securing models from evolving data poisoning attacks. This chapter answers RQ 3 and proposes an attack-agnostic solution called SecureLearn and analyses the limitations of existing solutions to mitigate data poisoning attacks and highlights the effectiveness of the proposed mitigation solution as an attack-agnostic defence. By answering RQ 3 , it fulfills objectives 3 and 4. Objective 3 is achieved with the task of proposing a robust and generalised solution and objective 4 is aligned with proposing a novel mechanism called the 3D evaluation matrix. Previously, no such matrix was proposed in the literature to assess defence solutions thoroughly across various dimensions. The 3D evaluation matrix evaluates the defensive capabilities of SecureLearn from three dimensions: data poisoning attacks, data sanitisation, and adversarial training, across three attacks and two existing defences.

Existing literature highlights several techniques, such as Meng et al. (2022), Tao et al. (2021), to mitigate data poisoning attacks; however, these are largely attack-specific or algorithm-specific. For example, the research study (You et al., 2019) added a noise layer in neural networks to regularise the adversarial noise in these models. However, this approach does not apply to other models, like SVM, RF, and DT, as they do not comprise layers of nodes in their architecture. Similarly, adversarial training is one of the solutions that improves the adversarial robustness of DL models and does not applies to traditional models. The experimentation, presented in Section 4.7 in Chapter 4, highlighted the limited effectiveness of adversarial training in securing traditional models. These limitations highlighted the need for a generalised solution that is independent of the model architecture and capable of countering current and evolving threats. Therefore, this chapter proposes an attack-agnostic solution, SecureLearn, to mitigate data poisoning attacks in multiclass ML.

SecureLearn offers an enhanced data sanitisation that combines the fundamental principles of nearest neighbor voting strategy to correct data labels, followed by calculating the statistical deviations of each data point to detect and correct anomalies. Furthermore, SecureLearn introduced a new approach of feature-oriented adversarial training (FORT) influenced by a common characteristic of feature importance score of ML to identify important data points to generate adversarial examples

for training.

A 3D evaluation matrix is proposed in this chapter to thoroughly assess SecureLearn, following three orthogonal dimensions: data poisoning attack, data sanitisation and adversarial training. The experiments are conducted on four ML algorithms: RF, DT, GNB and MLP, trained with three differently structured datasets: IRIS (Fisher, 1936), MNIST(Deng, 2012), USPS (Hull, 2002). Selecting these algorithms allows this study to cover most classification models. SecureLearn is evaluated against three distinct data poisoning attacks and compared with two state-of-the-art mitigations, highlighting the better performance and generalisation of SecureLearn over others. The contributions of the chapter are given as follows.

- SecureLearn is the first defence solution that works against all types of data poisoning attacks and is adaptable for traditional classification models and neural networks. SecureLearn provides defence with minimal defender capabilities, without requiring prior knowledge of attacks and configurations of the targeted model.
- This chapter proposes a new adversarial training mechanism called FORT as a component of SecureLearn, enhancing the adversarial robustness of traditional multiclass ML, including neural networks. The results show that FORT improves the adversarial robustness of the model with a minimal trade-off between accuracy and robustness, i.e., the accuracy is decreased $< 3\%$, while enhancing the adversarial robustness.
- This chapter proposes a new 3D evaluation matrix to comprehensively evaluate SecureLearn against three data poisoning attacks and compare it with two existing defences Chan et al. (2018b), Paudice et al. (2018b). The evaluation is set up for four types of ML models trained with three distinct datasets. The results highlight that SecureLearn has outperformed other mitigations and is effective against all selected attacks for all models.

5.2 Related Work

5.2.1 Existing Multiclass Poisoning

Data poisoning attacks showed success in perturbing traditional ML in multiclass settings. The Outlier-Oriented Poisoning (OOP) attack (Paracha et al., 2025a), discussed in Chapter 4, manipulated the feature space by exploiting outliers and successfully poisoned six ML models. The research paper (Biggio et al., 2012) introduced a label flipping attack to perturb dataset labels, which can be extended to multiclass ML. Jagielski et al. (2021) introduced a subpopulation poisoning that injects a cluster of poisoned points in a dataset. Pantelakis et al. (2023) poisoned multiclass IoT networks with JSMA, FGSM, and DeepFool attacks and evaluate performance disruption.

5.2.2 Limitations of Existing Defences

Various defences are proposed in the literature to secure ML and DL models from data poisoning attacks. Such as the research (Peri et al., 2020) developed a deep k-NN to remove clean label poison by detecting falsified data points with k-neighbours. Deep k-NN defence is experimented against feature collision and convex polytope in deep neural networks. Paudice et al. (2018b) used the k-NN algorithm to mitigate label poisoning in binary SVM. Carnerero-Cano et al. (2023) computed the limitations of hyperparameters to resist data poisoning impact on DNN models. Barreno et al. (2008) has given the concept of reject on negative impact to remove affected data points, which is extended in (Chan et al., 2018b) to filter poisoned data from the given dataset.

Adversarial training is useful in improving the adversarial robustness of ML/DL models. In Ho et al. (2022a), Tao et al. (2021), Shafahi et al. (2020), adversarial training is implemented to improve the robustness of models against data poisoning attacks. However, these adversarial training methods are experimentally limited to neural networks and DL models because it is designed following gradient learning of the model at each iteration of training, which does not apply to traditional models, which do not follow gradient learning, and so makes adversarial training ineffective in securing traditional ML models.

Conclusively, some attack-agnostic solutions have been proposed in the literature; however, these are designed to secure DL models. A few solutions are proposed to secure traditional ML, mostly improving the robustness of binary models, whereas no prominent solution is proposed for multiclass ML. SecureLearn is the first attack-agnostic solution, designed to secure multiclass ML against data poisoning attacks. It is also adaptable to DL and binary models and effective against various aforementioned attacks. A brief comparison of existing solutions with SecureLearn is provided in Table 5.1, highlighting that existing solutions have either proposed data sanitisation or adversarial training, where data sanitisation solutions are experimented on binary ML models and adversarial training is experimented with only DL models.

Table 5.1: Summary of existing similar defences against data poisoning attacks proposed in various settings

Research paper	Data Sanitisation	Adv. Training	ML model	Model Settings
Chen et al. (2021)	✓	✗	GAN, CNN and LASSO	Binary and Multiclass DNN
Paudice et al. (2018b)	✓	✗	Stochastic Gradient Descent	Binary ML
Chan et al. (2018b)	✓	✗	SVM	Binary ML
Barreno et al. (2008)	✓	✗	SVM	Binary ML
Shafahi et al. (2020)	✗	✓	ResNet and InceptionV1	Multiclass DNN
Tao et al. (2021)	✗	✓	VGG-16, VGG-19, ResNet-18, ResNet-50 and DenseNet-121	Multiclass DL
SecureLearn	✓	✓	DT, RF, GNB, MLP	Multiclass ML

5.3 SecureLearn Overview

The problem of poisoning the training dataset is defined as follows: D_c represents the clean dataset, while D'_c denotes the poisoned substitute in the dataset. The combined dataset is defined as $D_o = D_c \cup D'_c$. SecureLearn relies on the general observation that the poisoned dataset tricks the model training to classify differently from the clean dataset, resulting in performance degradation. Therefore, SecureLearn identifies anomalies and misalignments in the features and labels to sanitise the dataset. Since no ground truth is provided, SecureLearn aims to sanitise D_o to correct data points and align features by identifying outliers in each dataset class and updating class labels by averaging the labels of nearest neighbouring data points. Furthermore, it quantifies the drift in features in each data point and filters those that exceed limits from the normalised dataset. Additionally, SecureLearn enhances the adversarial robustness of the model through FORT training to increase the reliability against new attacks. The complete process of SecureLearn is illustrated in Fig. 5.1. The algorithm of SecureLearn is provided in Alg. 7.

As an attack-agnostic solution, SecureLearn determines and relabels the uncertain data points in the dataset, regardless of being poisoned or outliers, whereas it safeguard mislabeling by calculating the average confidence of the neighbour data points following Eq. 5.1. Its efficacy is limited to generalised scenarios, which can be extended and integrated in specialised applications.

Algorithm 7 SecureLearn Mitigation Mechanism

Input: Training Dataset D_o, perturbation limit ε, feature importance scores: F
Initialise: b=0.001, c=0.01, nearest neighbours (k)=7
for $x_i \in D_o$ **do**
  $d = min(k, dist(x_i, x))$
  $l_i = avg(x_i, d)$
  $D_{san} \leftarrow (x_i, l_i)$
end for
for $x_i \in D_{san}$ **do**
  $Compute\ \delta_i\ following\ Eq.\ 5.5$
  if $\delta < |g|$ **then**
    $D_{san} \leftarrow (x_i, l_i)$
  end if
end for
if $M == M_{GNB}\ or\ M_{MLP}$ **then**
  $F \leftarrow \arg max\ Probability(D_{san})$
end if
if then$(M == M_{RF}\ or\ M_{DT})$:
  $F \leftarrow \sum_{i=1}^{L} f_i(1 - f_i)$
end if
for $(x_i \in D_{san})$ and $(f_i \in F)$ **do**
  $D_{adv} \leftarrow \mathbb{E}_{(x,y)\sim D_o}[\mathcal{L}(M, (x_i + (c * sign((f_i * x_i) + b)))$
end for

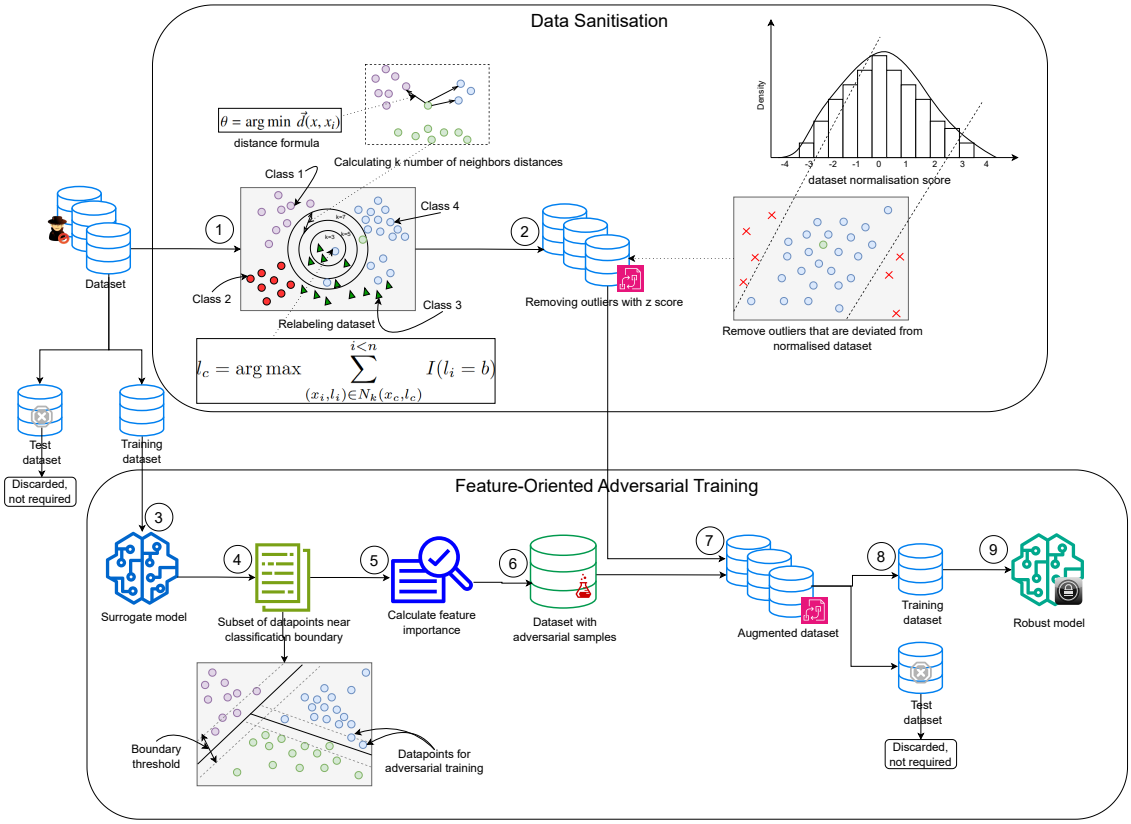


Figure 5.1: Architectural overview of SecureLearn illustrating a two-layer approach to secure the training pipeline of a machine learning model

5.3.1 Improvised Data Sanitisation

The data sanitisation module of SecureLearn comprises two parts: relabeling the data points with corrupted labels in D_o and removing anomalies to produce a sanitised dataset D_{san}. The relabeling mechanism is defined as:

$$D_{san} = \{(x, l)|x \in D_o\}$$
$$and\, l_i = \begin{cases} l_i\, if\, C(x_i, l_i) < \gamma \\ l\, if\, C(x_i, l_i) \geq \gamma \end{cases} \tag{5.1}$$

where $C(x, l)$ is the label confidence of neighbouring data points, l_i is the existing label of the data point x_i, l is the new label with the highest confidence received from the nearest data points. The confidence limit is defined as $\gamma \geq 40\%$ neighbouring votes, following an incremental majority voting approach (Abdulboriy and Shin, 2024). The optimal threshold for voting confidence to detect irrelevant data points is identified using a brute force mechanism and experimented with a threshold between 30% and 60% average of neighbouring votes. This detection mechanism identifies irrelevant data points that may be intentionally perturbed or misaligned data points, regardless of the dataset's application. The calculation of the label of each data point, given in Eq. 5.1, follows the confidence

score $C(x, l)$ of neighbouring data points, calculated with Eq. 5.2.

$$C(x, l) = \arg\max \frac{1}{k} \sum_{(x_j, l_j) \in \theta}^{j<n} I(l_j = l_c) \tag{5.2}$$

where l_c is the original class label, k is the number of nearest neighbours set to seven following the kTree method given by Zhang et al. (2017), x is the data point with label l and θ is the function of distance measure, given in Eq. 5.3.

$$\theta = \min \vec{d}(x_i, x) \tag{5.3}$$

The next step of data sanitisation is to remove outliers from the dataset. The anomalous data points are removed from the dataset, where the deviation of the given data point exceeds the limits of the normalised dataset distribution, following Eq. 5.4. The deviation δ is calculated with Eq. 5.5 where μ is the mean of the dataset and the deviation limit $|g| = 3$ (Abdi, 2007).

$$D_{san} = \{x_i \in D_o || \delta \leq |g|\} \tag{5.4}$$

$$\delta = \frac{x_i - \frac{1}{n} \sum_{i=1}^{n} x_i}{\sqrt{\frac{1}{n} \sum_{i=1}^{n} (x_i - \mu)^2}} \tag{5.5}$$

5.3.2 Feature-Oriented Adversarial Training

After obtaining the sanitised dataset, SecureLearn aims to improve the adversarial robustness of the model with feature-oriented adversarial training. In the literature, it is noticed that the existing adversarial training mechanism is unable to improve the resilience of traditional ML models (Paracha et al., 2024b) because existing approach follows the gradient-oriented training which is ineffective for traditional models, therefore SecureLearn introduced a new method to train models, where adversarial data D_{adv} is generated by augmenting data points with high feature importance score and lie near the decision boundary. This is done by solving Eq. 5.6, followed by generating the perturbation in Eq. 5.7.

$$D_{adv} \leftarrow \mathbb{E}_{(x,y) \sim D_o}[\mathcal{L}(M, ((x_i + \varepsilon), l) \tag{5.6}$$

where M is the training model, $\mathcal{L}$ is the training loss and ε is the perturbation given in Eq. 5.7.

$$\varepsilon = c * sign((f_i * x_i) + b) \tag{5.7}$$

where, in Eq. 5.7, f_i is the feature importance score of the model M, $c = 0.01$ is the perturbation constant. This constant is set following the average perturbation value, given in (Liu and Wen, 2021). x_i is the data point, and $b = 0.001$ is the non-zero coefficient which restricted the zero output.

Combining output of Eq. 5.1 and Eq. 5.6, the sanitised dataset D_s is given in Eq. 5.8:

$$D_o = D_{san} + D_{adv} \tag{5.8}$$

Intuitively, the model is trained to mitigate the data poisoning effects and improve the overall performance. Unlike traditional adversarial training based on gradient optimisation, FORT adds slight perturbations to the data points that are close to the decision boundaries of the model to widen these boundaries, making them robust to poisoning. This way, SecureLearn improves the security and robustness of ML models against data poisoning attacks. Next, this chapter extends the threat model, defined in Chapter 4, to evaluate SecureLearn as an effective defence.

5.4 Extended Threat Model

A threat model for this thesis is defined in Chapter 4 to study the adversarial impact of data poisoning attacks and draw attention to evolving threats in this domain. The threat model is extended with new attack vectors to assess SecureLearn's efficacy in mitigating multiclass data poisoning attacks.

5.4.1 Attack Goals

This threat model defines two attacker goals to assess the effectiveness of selected mitigation solutions. The first goal is to disrupt the model's availability and reduce its overall performance by employing the OOP attack (Paracha et al., 2025a) and label flipping attack (Shahid et al., 2022). The second goal is to harm the model's integrity by augmenting clustered poisoned data points employing the subpopulation attack to disrupt targeted class predictions (Jagielski et al., 2021).
Consider the poisoning of supervised classification models, e.g. RF or MLP, given the dataset $D_o = \{(x, l)\}_{i=1}^{n}$ with data points x and labels l of class c, the attacker can manipulate the labels l' or the features x' of the dataset or augment poisoned datapoints(x', l') into the dataset to prevent the trained victim model from attaining the intended performance.

5.4.2 Attacker Knowledge

In this threat model, the attacker possesses limited knowledge of the targeted model M and dataset D_o. Under these constraints, all selected data poisoning attacks are formulated as grey-box attacks. In this scenario, the attacker has a partial understanding of the dataset and model: the dataset and algorithm names are known, but the dataset distribution, model settings, and parameters remain unknown. Additionally, the attacker has no knowledge and access to the target system.

5.4.3 Attacker Capability

In these experiments, the attacker has leveraged the capability to poison the training datasets. Thus, the attacker can manipulate the training dataset in different ways. The attacker can modify labels or features of the dataset and introduce poisoned data points into the dataset. However, this capability is limited to injecting a maximum 20% poisoning level as the upper bound limit and a minimum 10% poisoning as the lower bound limit. These limits are defined as the most effective poisoning limits, as explained in Chapters 3 and 4 Paracha et al. (2024a), Paracha et al. (2025a).

5.4.4 Attack Strategy

In this threat model, three data poisoning attacks of varying attack vectors, i.e., OOP, SubP and RLPA attacks are considered. Following these attacks in multiclass classifiers, the effectiveness of SecureLearn is evaluated, demonstrating that it is an attack-agnostic and promising solution capable of mitigating all the aforementioned attacks.

5.4.5 Defender Capability

The capabilities of the defender, under this threat model, are leveraged to complete access to the dataset and the targeted model to assess selected mitigations. The data sanitisation (Paudice et al., 2018b) assumes full access to the dataset and algorithm used. The mitigating solution (Chan et al., 2018b) also requires full access to the model and the training data, as it compares the change in the model's performance for every data point in the training dataset. Conversely, SecureLearn is exempted from these capabilities and only requires access to the dataset, assuming that the dataset may or may ot be poisoned. This way, SecureLearn can be implemented in restricted and third-party solutions that do not provide complete access to their systems.

5.5 Experimental Datasets

This chapter utilises three publicly available datasets: IRIS, MNIST, and USPS. The IRIS and MNIST are extended from Chapter 4; however, the ISIC dataset is replaced with the USPS dataset, considering that ISIC is an unbalanced dataset. These datasets differ in structure, feature correlations, and number of classes. They have been widely used in studies of data poisoning attacks Drews et al. (2020), Wang et al. (2021c), Paracha et al. (2025a) and defences Xu et al. (2021), Jia et al. (2021), Zhang et al. (2022). The diversity of these datasets facilitates a comprehensive analysis of the adaptability and effectiveness of defence mechanisms against various poisoning attacks.

Table 5.2 presents the structural details of these datasets. The IRIS dataset contains 150 instances and three classes. The USPS dataset consists of scanned images of postal digits across 10 classes. In

contrast, the MNIST dataset comprises a large collection of digital images of handwritten digits.

The statistical correlations between the features of each dataset are given in Table 5.3. The spearman correlation highlights monotonic strength (van den Heuvel and Zhan, 2022) between dataset features and the p-value reflects statistical significance (Di Leo and Sardanelli, 2020). A low statistical significance highlights high variability in the dataset, which may strengthen the poisoning impact. Features in the MNIST dataset are highly associated and show high strength with a very low p-value of 0.0141, highlighting direct proportionality between its features. However, a low to nominal statistical significance is shown in the IRIS datasets with a p-value of 0.07, and spearman correlation of 0.12 shows a weak monotonic relationship between its features. In USPS, negative spearman correlation highlights an inverse monotonic relationship between the dataset features. In contrast, the dataset with a p-value of 0.2397 indicates a very low statistical significance of the dataset.

Table 5.2: Dataset description of all three datasets utilised in this study

Dataset	No. of classes	No. of features	No. of instances
IRIS	3	4	150
MNIST	10	784	70,000
USPS	10	256	9298

Table 5.3: Features correlation in dataset to analyse interdependence between features in individual dataset

Dataset	Spearman correlation	p-value
IRIS	0.1238	0.0791
MNIST	0.009282	0.0141
USPS	-0.008742	0.2397

5.6 Crafting Poisoned Samples

After the attacker's capabilities and goals are established, poisoned data points were constructed with selected data poisoning attacks. Considering the attacks discussed in Section 5.2, the selected attacks formulate poison from three different aspects: perturbing dataset labels, perturbing dataset features, and augmenting perturbed data samples. The details of crafting poison with each attack are given as follows.

5.6.1 Random Label Poisoning Attack

One of the most common and early proposed attacks against supervised models is the label poisoning attack Nguyen et al. (2023), Paudice et al. (2018b). Though initially proposed against binary classifiers, it is later extended to manipulate multiclass models by randomly perturbing dataset labels (Rosenfeld et al., 2020). In the random label poisoning attacks (RLPA), the computation of

perturbations is simple. Given the poisoning limits, the labels of randomly selected data points were changed, ensuring that the targeted label is updated to another class label. Let $D'_{train} = \{(x_i, l'_i)\}_{i=1}^{n}$ be the poisoned training dataset where l_i is manipulated with RLPA as:

$$l'_i = Y/\{l_i\} \tag{5.9}$$

where $l \in Y$ and features x_i of ith data point remains intact. Following the threat model of this chapter, the poison is constrained based on the poisoning limits ΔL. The training dataset is manipulated with Eq. 5.10.

$$D'_{train} = f(D_{train}, \Delta L) \tag{5.10}$$

where f is the label manipulation function given in Eq. 5.12.

5.6.2 Subpopulation Poisoning Attack

The subpopulation attack (SubP) augments a poisoned cluster into the training dataset (Jagielski et al., 2021) instead of poisoning segregated data points. The attacker selects a random class and replicates its samples. In the next step, the attacker changes the labels of the replicated data points to another class and augments these data points into the training dataset to generate D' following the perturbation function given in Eq. 5.11.

$$\forall (x, l') \in D' \tag{5.11}$$

such that l' is generated with Eq. 5.12. Then, D'_{train} is given as follows.

$$D'_{train} = D' \cup D_{train} \tag{5.12}$$

5.6.3 Outlier-Oriented Poisoning Attack

This thesis proposed the OOP attack in Chapter 4. The OOP attack perturbs the feature space by interpolating outliers of varying classes. For this perturbation, the adversarial samples are generated by selecting the most distant data points in the dataset and changing their labels to manipulate the feature space of the multiclass classifier and shift its decision boundaries.

Let $D_{train} = \{(x_i, l_i)\}_{i=1}^{n}$ be the training dataset with O outliers, such that the OOP attack calculates the distance vector of each data point, finding the most distant points. The distance is calculated given Eq. 5.13.

$$\arg\max \ \vec{d}(b_c, X_c) \tag{5.13}$$

where b_c is the decision boundary of class c. Given the poisoning levels, ΔL defines the perturbation limits of the number of poisoned data points. The function of manipulation is given in Eq. 5.15.

$$D'_{train} = f(O, \Delta L) \tag{5.14}$$

such that, f implies to change label of ith data point (x_i, l_i) with Eq. 5.15:

$$l'_i = O/\{l_i\} \tag{5.15}$$

Following these discussed attacks, the 3D evaluation matrix is proposed to thoroughly evaluate SecureLearn.

5.7 3D Evaluation Matrix

This chapter proposed an innovative and comprehensive 3D evaluation matrix to evaluate defence solutions against data poisoning attacks, particularly SecureLearn in this study. This matrix evaluates SecureLearn in three dimensions and compares it with two typical defences against three data poisoning attacks as given in Table 5.1. The 3D evaluation matrix is given in Fig. 5.2. Its dimensions are explained as follows.

5.7.1 Dimensional Space 1

The *dimensional space 1 (DS1)* lies between the dimensions of data sanitisation and data poisoning attack. Here, SecureLearn is analysed by experimenting with it against three data poisoning attacks and by comparing it with two existing similar data sanitisation defences to highlight the effectiveness of the data sanitisation of SecureLearn. The DS1 evaluates the strength of mitigations and their attack-agnostic resistance to data poisoning attacks, followed by highlighting the profound performance of SecureLearn compared to other solutions.

5.7.2 Dimensional Space 2

The *dimensional space 2 (DS2)* lies between the dimensions of data poisoning attacks and adversarial training. In DS2, the effectiveness of the proposed FORT training component of SecureLearn is evaluated against selected data poisoning attacks and analysing improvements in the adversarial robustness of the model. In this dimensional space, SecureLearn is evaluated to distinguish between benign and poisoned data points and assess the resilience of the model against corrupted data points to minimise the impact on decision boundaries during training.

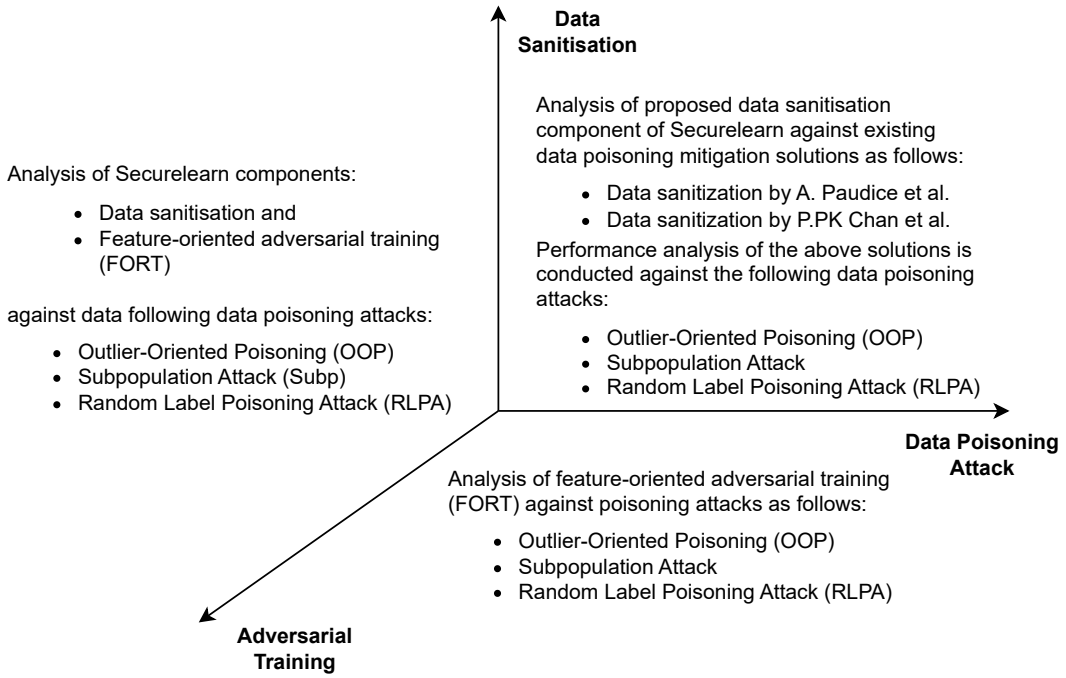


Figure 5.2: 3D evaluation matrix to evaluate SecureLearn three different aspects. The first dimension between data sanitisation and data poisoning attacks evaluates SecureLearn against selected data poisoning attacks and compares it with existing mitigations. The second dimension between data poisoning attacks and adversarial training evaluates FORT adversarial training of SecureLearn against selected data poisoning attacks. The third dimension between adversarial training and data sanitisation evaluates the overall performance of SecureLearn against selected data poisoning attacks

5.7.3 Dimensional Space 3

The *dimensional space 3 (DS3)* lies between the dimensions of adversarial training and data sanitisation. In DS3, the overall effectiveness of SecureLearn is evaluated in securing multiclass ML from reactive and proactive data poisoning attacks. The resilience and robustness of the model are assessed by analysing the false discovery rate of the model at varying poisoning levels against selected data poisoning attacks. The evaluation metrics used for the assessments in this chapter are discussed as follows.

5.8 Evaluation Metrics

To evaluate model performance in 3D evaluation matrix, the standard performance metrics: Accuracy, Recall and F1-Score are adopted. Furthermore, the detection rate (DR), correction rate (CR) and false discovery rate (FDR) are utilised for the detailed evaluation. The DR and CR prominently highlight the efficacy of SecureLearn in sanitising poisoned data points and FDR highlights the strengthened robustness of the model against poisoned training. Accuracy is the measure of correct classifications, where the poisoned data points remain disjointed in the incorrect classes and do not affect the model's

availability. Recall measures correct predictions of positive classifications over all positive answers, defining high separability. F1-score quantifies the overall defence performance, where the decision boundaries are aligned. Let the classification function be given in Eq. 5.16, the evaluation metrics can be found in Eq. 5.17, 5.18, 5.19.

$$f(C(x_t)) = \begin{cases} true & \text{if } x_t \in Class\ c \\ false & \text{otherwise} \end{cases} \tag{5.16}$$

where f is the classification function, x_t is the data point from the test dataset D_t split from D_o, and $C(.)$ is the class predictor. After sanitising dataset with SecureLearn, false positives(FP) is defined as $f_{tr}(C(x_{t_i})|l'_c)$, where l'_c is the wrong class label and false negative(FN) is defined as $f_{fs}(C(x_{tr_i})|l_c)$ where data points are not sanitised correctly. Whereas, true positive is defined as $f_{tr}(C(x_{t_i}))$ and true negative is defined as $f_{fs}(C(x_{tr_i}))$.

$$Acc = \frac{\sum_{i=0}^{n} f_{fs}(C(x_{t_i})) \wedge \sum_{i=0}^{n} f_{tr}(C(x_{t_i}))}{(x_t \in D_t)} \tag{5.17}$$

$$Rcl = \frac{\sum_{i=0}^{n} f_{tr}(C(x_{t_i}))}{\sum_{i=0}^{n}(f_{tr}(C(x_{t_i}))) \wedge \sum_{i=0}^{n}(f_{fs}(C(x_{t_i})))} \tag{5.18}$$

$$where\ f_{fs}(C(x_{t_i})) \in D_t$$

$$F1_scr = \frac{\sum_{i=0}^{n} f_{tr}(C(x_{t_i})) * Rcl}{2 * \{(\sum_{i=0}^{n} f_{tr}(C(x_{t_i})) \wedge \sum_{i=0}^{n} f_{tr}(C(x_{t_i}))) + Rcl\}} \tag{5.19}$$

Let x' be the poisoned data point in D_o, and detection of these points with SecureLearn is given in Eq. 5.20, and setting these points in the appropriate class is shown in Eq. 5.21. After corrections, the false discovery rate of the model is analysed with Eq. 5.22.

$$Detection\ rate(DR) = \frac{\sum_{i=0}^{n} P(x'|l_c)}{\sum_{i=0}^{n} P(x|l_c) \wedge P(x'|l_c)} \tag{5.20}$$

$$Correction\ rate(CR) = \frac{\sum_{i=0}^{n} P(x' \rightarrow x|ll_c)}{\sum_{i=0}^{n} P(x|l_c) \wedge P(x'|l_c)} \tag{5.21}$$

$$False\ discovery\ rate(FDR) = \frac{\sum_{i=0}^{n} f_{tr}(C(x_{t_i})|l'_c)}{\sum_{i=0}^{n} f_{tr}(C(x_{t_i}|l'_c)) \wedge f_{tr}(C(x_{t_i}))} \tag{5.22}$$

5.9 Experimentation and Analysis

This section performed the experimental evaluation of SecureLearn using the 3D evaluation matrix. This section answers how SecureLearn is better at detecting and sanitising different types of data poisoning attacks. It first evaluates it in DS1 of the 3D evaluation matrix to analyse SecureLearn against three poisoning attacks and two existing data sanitisation techniques. The results are described in Sections 5.9.1 and 5.9.2. The evaluation of the FORT component is then conducted in DS2 of the 3D evaluation matrix. The results show that FORT effectively enhances the adversarial robustness of traditional ML models and neural networks against adversarial perturbations. The results are detailed in Section 5.9.3. Following the results of DS1 and DS2, the overall effectiveness of SecureLearn is further analysed at different poisoning levels in DS3. The relationship between increasing poisoning levels and the resilience of SecureLearn in mitigating the poisoning impact is identified, given in Section 5.9.4.

5.9.1 Detection and Correction Boundaries

The analysis begins by determining the detection and correction rates of each data poisoning attack, as presented in Table 5.4. The lower bound (LB) and upper bound (UB) of DR and CR are calculated at three defined poisoning levels using Eq. 5.20 and Eq. 5.21. Results show that SecureLearn detects at least 50% of poisoned data in training datasets, regardless of the type of attack or dataset used in these experiments. The minimum CR is approximately 30% for the RF model under the RLP attack, likely due to the unpredictable placement and impact of poisoned data points in untargeted attacks. The UB of DR and CR for SecureLearn reaches 100% against data poisoning attacks trained with IRIS dataset for most algorithms.

SecureLearn is highly effective in sanitising the IRIS dataset, followed by the USPS dataset, compared to the MNIST dataset, across all poisoning levels. These results indicate an inverse relationship between SecureLearn's performance and dataset size, with profound effectiveness observed in smaller datasets. SecureLearn is generalisable across different data poisoning strategies and dataset structures, performing independently of the number of classes in the dataset.

5.9.2 SecureLearn vs Existing Sanitisation Techniques

This section evaluates the effectiveness of the data sanitisation provided by SecureLearn. Model performance is analysed using Eq. 5.17 to Eq. 5.19, with the poisoning level set between 10% and 20%. Model accuracy after training with sanitised datasets is presented in Fig. 5.3 to Fig. 5.5. The results of these experiments demonstrate that SecureLearn consistently outperforms other solutions and maintains an accuracy of at least 90% across implemented data poisoning attacks. The recall

Table 5.4: Detection and correction boundaries of machine learning models after mitigating data poisoning attacks with SecureLearn

Algorithm	Dataset		Attack					
			OOP		Subp		RLP	
			LB	UB	LB	UB	LB	UB
RF	IRIS	DR	86.6	100	86.6	100	76.6	100
		CR	80	90.9	80	91	76.6	93.3
	MNIST	DR	56.3	65.5	56.3	66.3	52.4	66.3
		CR	33.5	49.2	33.5	49.2	29.7	47.6
	USPS	DR	87.94	89.13	56.29	65.78	50.48	62.56
		CR	44.47	49	38.42	44.54	35.22	43.24
DT	IRIS	DR	83.3	93.3	83.1	92	93.3	95.4
		CR	86.6	90.9	80	91	76.6	91
	MNIST	DR	49.6	66.7	49.8	66.7	46.4	64.1
		CR	44.69	57.88	45.1	58	44.97	55.08
	USPS	DR	44.69	57.88	44.69	57.88	44.97	55.08
		CR	15.98	36.93	15.98	37	18.1	34.51
GNB	IRIS	DR	100	100	100	100	80	100
		CR	93.3	100	93.3	100	66.6	93.3
	MNIST	DR	98.6	99.1	98.6	99.1	96	98.4
		CR	94.9	95.9	94.9	95.9	92.4	95.3
	USPS	DR	99.24	99.71	99.24	99.71	97.09	99.49
		CR	97.63	97.99	97.63	97.99	95.53	97.99
MLP	IRIS	DR	83.3	100	83.3	100	73.3	95.4
		CR	76.6	95.4	70	95.4	66.6	86.6
	MNIST	DR	56.3	65.5	56.3	66.3	52.4	66.3
		CR	59.33	49.2	33.5	49.2	29.7	47.6
	USPS	DR	71.16	85.36	70.79	84.7	64.28	82.5
		CR	59.33	78.9	59.11	79.76	51.47	76.42

and f1-score are provided in Table 5.5.

SecureLearn outperformed the mitigations proposed by Paudice et al. (2018a) and Chan et al. (2018b) in sanitising poisoned datasets. Compared to SecureLearn, the data sanitisation method proposed by Paudice et al. (2018a) achieved similar accuracy for DT with an average of 96%. SecureLearn provided an average recall of 84.22% with a 3% higher f1-score. Similarly, the average accuracy for GNB provided by Paudice et al. (2018a) is 94%, equivalent to SecureLearn; however, its recall and f1-score are 3.69% and 3.63% lower, respectively. Furthermore, the sanitised accuracy provided by Paudice et al. (2018a) dropped to 79% for the RLP attack and to 82% for the OOP attack when the model is trained with the MNIST dataset.

The data sanitisation proposed by Chan et al. (2018b) is highly unstable, particularly for MLP models. The accuracy of each model consistently decreases with increasing poisoning levels. For example, the accuracy of MLP substantially decreases after 10% poisoning, reached approximately 52% when trained on the IRIS and MNIST datasets, and 80% when trained on the USPS dataset. This instability arises because the method removes anomalous data points, which potentially decreases model accuracy. However, removing such data points also reduces the dataset size, which leads to underfitting, particularly in neural networks.

Table 5.5: Impact of data poisoning on recall and f1-score of secured machine learning models

Metric	Alg	Dataset	Defence	Attack								
				OOP			Subp			RLP		
				$\Delta L = 10\%$	$\Delta L = 15\%$	$\Delta L = 20\%$	$\Delta L = 10\%$	$\Delta L = 15\%$	$\Delta L = 20\%$	$\Delta L = 10\%$	$\Delta L = 15\%$	$\Delta L = 20\%$
Recall	RF	IRIS	A. Paudice et al.	**97.33**	92.85	**91.66**	91.88	88.09	91.66	87.17	92.85	80.55
			M. Barreno et al.	92.09	78.57	75.04	**97.43**	99.99	**96.07**	84.61	84.12	69.75
			SecureLearn	93.73	**99.99**	86.53	94.87	**99.99**	91.88	**94.87**	**96.96**	**92.09**
		MNIST	A. Paudice et al.	88.22	85.20	82.19	88.13	85.44	82.38	88.57	85.20	81.96
			M. Barreno et al.	**92.09**	78.57	75.04	**97.43**	**99.99**	**96.07**	84.61	84.12	69.75
			SecureLearn	91.31	**86.63**	**84.19**	91.34	86.61	84.38	**90.76**	**86.30**	**83.40**
		USPS	A. Paudice et al.	91.48	89.08	81.51	90.57	88.95	81.38	91.06	87.65	80.07
			M. Barreno et al.	86.84	81.14	80.50	83.26	80.35	80.40	82.85	75.96	75.86
			SecureLearn	**95.18**	**91.02**	**90.16**	**95.36**	**90.51**	**90.56**	**95.33**	**90.22**	**89.16**
	DT	IRIS	A. Paudice et al.	93.74	94.88	82.05	91.66	**97.91**	74.64	85.79	92.85	79.48
			M. Barreno et al.	86.66	81.81	77.77	**99.99**	93.93	**85.18**	84.70	84.84	69.62
			SecureLearn	**97.77**	**97.91**	**88.88**	95.55	94.21	84.12	**95.55**	**94.21**	**83.33**
		MNIST	A. Paudice et al.	**86.93**	81.84	78.09	86.90	81.38	78.21	**86.71**	81.93	**78.28**
			M. Barreno et al.	86.66	81.81	77.77	**99.99**	**93.93**	**85.18**	84.56	**84.84**	69.62
			SecureLearn	85.45	**85.13**	**78.20**	85.45	84.38	78.40	85.45	84.56	77.44
		USPS	A. Paudice et al.	85.67	80.39	80.63	86.14	79.60	80.41	86.27	80.12	79.61
			M. Barreno et al.	81.65	74.01	80.29	81.34	79.41	80.07	73.85	62.92	79.25
			SecureLearn	**87.42**	**81.51**	**81.00**	**87.37**	**81.58**	**81.50**	**87.40**	**81.55**	**79.82**
	GNB	IRIS	A. Paudice et al.	91.11	**94.11**	77.77	88.88	94.11	71.96	86.11	94.11	86.11
			M. Barreno et al.	85.18	84.40	85.30	92.59	**94.65**	94.74	90.74	86.96	94.74
			SecureLearn	**95.39**	92.59	**98.03**	**95.39**	94.44	**98.03**	**95.39**	**94.44**	**98.03**
		MNIST	A. Paudice et al.	57.12	60.34	58.98	58.39	57.78	52.72	56.49	59.48	50.50
			M. Barreno et al.	**85.18**	**84.40**	**85.30**	**92.59**	**94.65**	**94.74**	**90.74**	**86.96**	**94.74**
			SecureLearn	57.71	57.12	57.15	57.93	58.38	57.65	58.48	57.33	57.16
		USPS	A. Paudice et al.	75.39	73.11	77.28	74.01	**77.54**	75.01	74.01	77.67	76.64
			M. Barreno et al.	76.70	75.94	76.19	76.85	71.12	75.82	75.73	75.94	75.83
			SecureLearn	**76.97**	**78.16**	**77.50**	**77.34**	76.80	**77.23**	**76.57**	**77.93**	**77.26**
	MLP	IRIS	A. Paudice et al.	96.29	97.77	**99.99**	96.27	91.11	**97.22**	96.3	90.47	97.22
			M. Barreno et al.	31.11	28.61	36.01	36.30	18.72	28.51	33.92	16.34	28.51
			SecureLearn	**99.90**	**98.01**	99.90	**99.99**	**97.98**	96.96	**99.99**	**99.90**	**99.99**
		MNIST	A. Paudice et al.	96.29	**97.77**	**99.99**	96.15	91.11	97.22	96.29	90.47	97.22
			M. Barreno et al.	31.11	28.61	36.01	36.30	18.72	28.51	33.92	16.34	28.51
			SecureLearn	**97.93**	97.45	97.05	**98.08**	**97.82**	**97.37**	**97.32**	**97.60**	**97.25**
		USPS	A. Paudice et al.	96.29	82.92	83.52	96.30	81.05	79.69	96.29	81.04	82.33
			M. Barreno et al.	85.56	78.9	83.52	86.10	51.47	79.69	86.04	82.33	79.20
			SecureLearn	**98.42**	**97.76**	**98.40**	**97.69**	**98.19**	**98.05**	**98.36**	**97.87**	**97.06**
F1-Score	RF	IRIS	A. Paudice et al.	**97.33**	91.81	**91.65**	91.93	86.49	91.72	86.06	91.65	80.37
			M. Barreno et al.	91.98	75.94	72.38	**97.33**	99.99	**95.13**	83.59	84.12	68.05
			SecureLearn	93.73	**99.99**	86.58	93.88	**99.99**	91.94	**93.88**	**97.40**	**91.98**
		MNIST	A. Paudice et al.	86.05	82.90	78.60	85.94	83.13	78.88	86.27	82.91	78.40
			M. Barreno et al.	**91.98**	75.94	72.38	**97.33**	**99.99**	**95.13**	83.59	**84.12**	68.05
			SecureLearn	90.90	**84.46**	**81.54**	90.91	84.39	81.78	**90.31**	84.11	**80.65**
		USPS	A. Paudice et al.	91.36	88.60	80.65	90.45	88.53	80.49	91.00	87.18	78.82
			M. Barreno et al.	86.47	79.05	79.47	83.26	78.42	79.23	82.85	74.38	74.84
			SecureLearn	**95.17**	**90.85**	**88.94**	**95.36**	**90.44**	**89.34**	**95.26**	**90.09**	**87.96**
	DT	IRIS	A. Paudice et al.	93.52	94.88	78.80	91.31	**97.47**	73.68	89.98	85.85	75.42
			M. Barreno et al.	88.15	81.56	70.85	**99.99**	93.88	83.81	84.56	84.84	61.16
			SecureLearn	**97.77**	**97.16**	**89.16**	94.66	94.21	**83.82**	**95.53**	**94.21**	**82.50**
		MNIST	A. Paudice et al.	86.38	79.11	75.56	86.27	78.61	75.50	**86.08**	79.33	**75.75**
			M. Barreno et al.	**88.15**	81.56	70.85	**99.99**	**93.88**	**83.81**	84.56	**84.81**	61.16
			SecureLearn	84.70	**84.58**	**75.80**	84.70	83.70	75.85	84.52	83.89	74.49
		USPS	A. Paudice et al.	83.12	77.46	77.40	83.48	76.58	77.03	83.79	77.18	76.06
			M. Barreno et al.	79.78	70.65	76.71	79.24	76.10	76.42	71.73	60.37	75.67
			SecureLearn	**85.09**	**78.66**	**81.00**	**84.97**	**78.82**	**81.50**	**84.77**	**81.55**	**78.70**
	GNB	IRIS	A. Paudice et al.	90.89	**92.77**	76.31	87.77	92.77	69.88	84.56	92.77	86.46
			M. Barreno et al.	82.32	83.76	84.74	91.87	**94.75**	94.74	91.41	86.58	94.74
			SecureLearn	**95.39**	91.87	**97.23**	**95.39**	94.44	**97.23**	**95.39**	**94.44**	**97.23**
		MNIST	A. Paudice et al.	53.35	57.86	56.42	54.81	55.09	49.86	52.64	56.94	49.19
			M. Barreno et al.	**82.32**	**83.76**	**84.74**	**91.87**	**94.75**	**94.74**	**91.41**	**86.58**	**94.74**
			SecureLearn	53.92	53.68	53.49	54.19	54.95	54.09	54.67	53.90	54.38
		USPS	A. Paudice et al.	75.14	73.56	77.36	73.73	**77.45**	75.72	73.98	**77.90**	76.97
			M. Barreno et al.	76.54	**78.37**	76.61	74.27	71.42	71.45	72.91	75.19	66.96
			SecureLearn	**76.79**	77.77	**77.47**	**77.20**	76.62	**77.33**	**76.55**	77.59	**77.14**
	MLP	IRIS	A. Paudice et al.	97.18	97.77	97.70	97.18	91.11	90.70	97.18	90.47	90.52
			M. Barreno et al.	31.61	29.75	30.74	36.53	18.19	26.96	32.93	15.25	26.96
			SecureLearn	**99.90**	**99.87**	**99.90**	**99.99**	**97.06**	**97.07**	**99.99**	**99.90**	**99.99**
		MNIST	A. Paudice et al.	97.18	**97.70**	**99.99**	97.18	90.70	**97.54**	97.18	90.52	96.96
			M. Barreno et al.	31.61	29.75	30.74	36.53	18.19	26.96	32.93	15.25	26.96
			SecureLearn	**97.96**	97.46	97.06	**98.08**	**97.84**	97.39	**97.34**	**97.61**	**97.26**
		USPS	A. Paudice et al.	86.00	81.71	82.48	85.35	80.83	80.98	87.39	80.75	80.77
			M. Barreno et al.	**99.99**	78.9	83.52	14.96	51.47	79.69	13.88	82.33	79.20
			SecureLearn	98.42	**97.77**	**98.41**	**97.74**	**98.22**	**98.08**	**98.40**	**97.95**	**97.08**

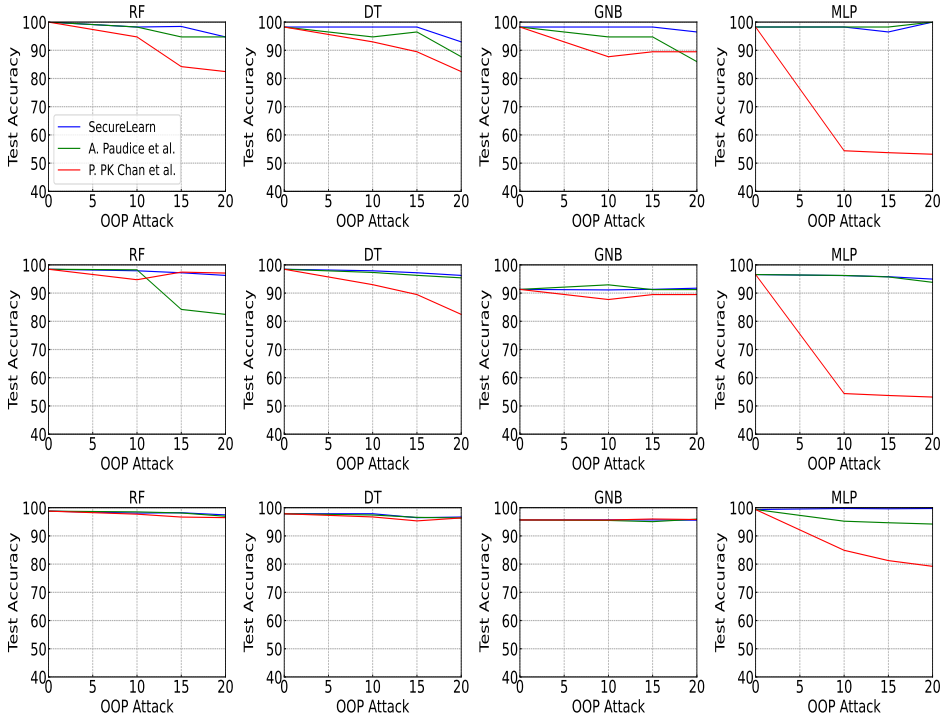


Figure 5.3: Impact of outlier-oriented poisoning attack on accuracy of models at various poisoning levels. The first row illustrates all models trained with the IRIS dataset, the models in the second row are trained with the MNIST dataset, and in the third row, the models are trained with the USPS dataset

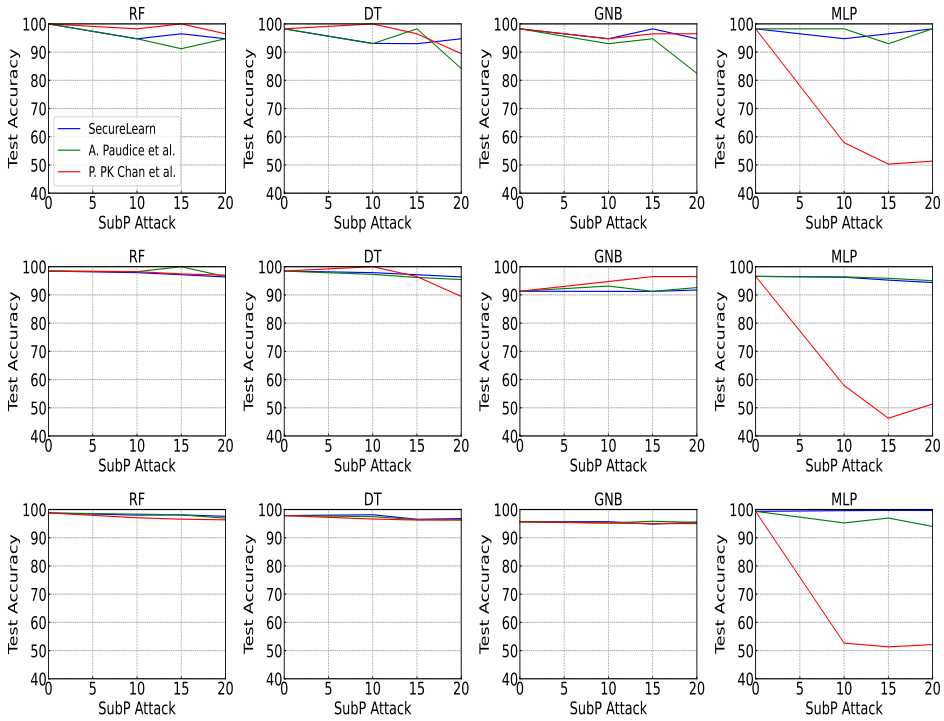


Figure 5.4: Impact of subpopulation attack on the accuracy of models at various poisoning levels. The first row illustrates models trained with the IRIS dataset, the models in the second row are trained with the MNIST dataset, and the models in the third row are trained with the USPS dataset

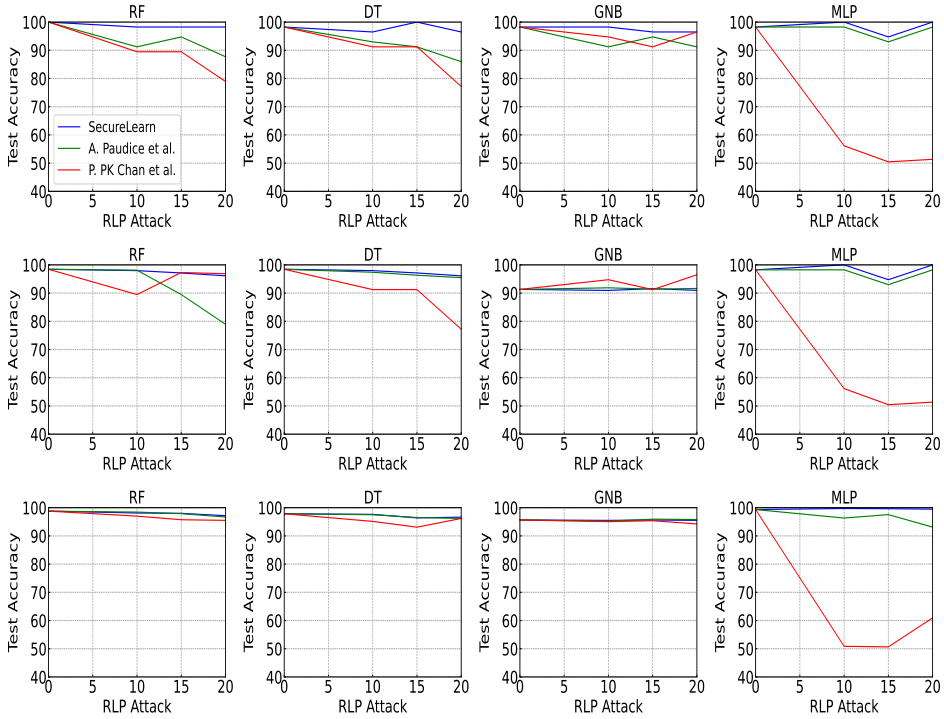


Figure 5.5: Impact of random label poisoning attack on accuracy of models at various poisoning levels. The first row illustrates models trained with the IRIS dataset, the models in the second row are trained with the MNIST dataset, and the models in the third row are trained with the USPS dataset

5.9.3 Analysing Feature-Oriented Adversarial Training

This section evaluated the FORT training of SecureLearn in enhancing adversarial robustness against data poisoning attacks. The adversarial robustness of the model is analysed by assessing FDR following Eq. 5.22. The results are provided in Tables 5.6 to 5.9. These results highlighted that FORT highly improved the adversarial robustness of multiclass models against all implemented data poisoning attacks.

These improvements are attributed to FORT's design, which leverages feature importance scores to guide adversarial training of ML. The adversarial samples for the training are developed by slightly perturbing data points close to decision boundaries and with high feature importance scores. Generalising over these perturbations enables the model to resist changes in its decision mechanisms with poisoned datasets.

The results given in the Table 5.6 highlighted that FORT reduces the FDR of the RF model to 0.06 when the model is trained on the poisoned IRIS dataset with $\Delta L = 10\%$. Similarly, for the same dataset, FDR=0.02 at $\Delta L = 15\%$ and FDR=0.05 at $\Delta L = 20\%$ across all attacks. Similar stability is visible for all adversarially trained models with FORT, as shown in Tables 5.7 and 5.9, highlighting the effectiveness of FORT.

Table 5.6: Effectiveness of feature-oriented adversarial training on false discovery rate of random forest after poisoning

Attack	Dataset	FDR					
		$\Delta L = 10\%$	FORT	$\Delta L = 15\%$	FORT	$\Delta L = 20\%$	FORT
OOP	IRIS	0.05	0.06	0.1	0.0001	0.19	0.13
	MNIST	0.02	0.01	0.16	0.14	0.21	0.16
	USPS	0.09	0.04	0.15	0.08	0.2	0.09
SubP	IRIS	0.08	0.06	0.1	0.0001	0.21	0.07
	MNIST	0.02	0.01	0.16	0.14	0.2	0.16
	USPS	0.1	0.04	0.16	0.08	0.2	0.09
RLP	IRIS	0.08	0.06	0.09	0.01	0.27	0.07
	MNIST	0.02	0.01	0.21	0.14	0.27	0.17
	USPS	0.12	0.04	0.21	0.08	0.26	0.09

Table 5.7: Effectiveness of feature-oriented adversarial training on false discovery rate of decision tree after poisoning

Attack	Dataset	FDR					
		$\Delta L = 10\%$	FORT	$\Delta L = 15\%$	FORT	$\Delta L = 20\%$	FORT
OOP	IRIS	0.03	0.02	0.1	0.03	0.19	0.07
	MNIST	0.15	0.14	0.19	0.14	0.26	0.23
	USPS	0.15	0.11	0.21	0.19	0.27	0.2
SubP	IRIS	0.07	0.05	0.15	0.05	0.13	0.15
	MNIST	0.14	0.14	0.19	0.15	0.26	0.23
	USPS	0.14	0.12	0.2	0.19	0.26	0.19
RLP	IRIS	0.15	0.03	0.12	0.05	0.23	0.11
	MNIST	0.19	0.14	0.25	0.15	0.33	0.24
	USPS	0.19	0.12	0.26	0.19	0.34	0.22

5.9.4 Increasing Poisoning Rate

SecureLearn maintains effectiveness across all evaluated attacks, independent of increasing poisoning levels. In this section, the analysis is extended to understand the relationship between the impact of increasing poisoning levels and the effectiveness of SecureLearn. Following the adversarial capabilities of the threat model, the data poisoning levels are set to $10\% < \Delta L < 20\%$. SecureLearn achieves a minimum sanitised accuracy of 90% for all models developed with four selected algorithms, highlighting no significant trade-off between model accuracy and adversarial robustness. The results are shown in Fig. 5.3 to Fig. 5.5. Data poisoning, however, impacts recall and f1-score differently for each model. The results are given in Table 5.5. For RF models, SecureLearn stabilises these models with a minimum recall of 84.19% and f1-score of 81.54% at 20% OOP poisoning. For DT models, the minimum recall is 78.20% and the f1-score is 75.80%. However, it is observed that SecureLearn does not sufficiently stabilise the GNB model trained with the MNIST dataset, as recall remains approximately 57% and the f1-score 56% across poisoning levels. In contrast, SecureLearn is highly effective in securing MLP models, achieving a minimum recall and f1-score of 97%, which demonstrates its potential to enhance the security of DL models. Overall, these results indicated that SecureLearn effectively mitigates the impact of data poisoning across datasets, even as poisoning levels increase.

Table 5.8: Effectiveness of feature-oriented adversarial training on false discovery rate of gaussian naive bayes after poisoning

Attack	Dataset	FDR					
		$\Delta L = 10\%$	FORT	$\Delta L = 15\%$	FORT	$\Delta L = 20\%$	FORT
OOP	IRIS	0.06	0.04	0.13	0.08	0.1	0.03
	MNIST	0.3	0.29	0.31	0.29	0.31	0.29
	USPS	0.2	0.2	0.2	0.19	0.22	0.2
SubP	IRIS	0.08	0.04	0.05	0.05	0.13	0.03
	MNIST	0.29	0.29	0.32	0.28	0.3	0.28
	USPS	0.2	0.19	0.2	0.2	0.23	0.19
RLP	IRIS	0.06	0.04	0.11	0.05	0.12	0.03
	MNIST	0.3	0.3	0.33	0.28	0.34	0.28
	USPS	0.21	0.19	0.22	0.19	0.24	0.2

Table 5.9: Effectiveness of feature-oriented adversarial training on false discovery rate of multilayer perceptron after poisoning

Attack	Dataset	FDR					
		$\Delta L = 10\%$	FORT	$\Delta L = 15\%$	FORT	$\Delta L = 20\%$	FORT
OOP	IRIS	0.07	0.02	0.04	0.02	0.15	0.05
	MNIST	0.06	0.01	0.06	0.02	0.08	0.02
	USPS	0.1	0.01	0.14	0.02	0.18	0.01
SubP	IRIS	0.03	0.03	0.05	0.02	0.2	0.07
	MNIST	0.06	0.01	0.08	0.02	0.08	0.02
	USPS	0.1	0.02	0.13	0.01	0.16	0.01
RLP	IRIS	0.03	0.0001	0.07	0.04	0.37	0.05
	MNIST	0.07	0.02	0.09	0.02	0.1	0.02
	USPS	0.1	0.01	0.13	0.01	0.16	0.02

5.10 Discussion and Limitations

This chapter proposed SecureLearn as a two-layer defence to mitigate data poisoning attacks and improve the resilience of traditional multiclass models, including neural networks. SecureLearn proposes an improvised data sanitisation along with a generic formulation of adversarial training, considering a common characteristic of the feature importance score. SecureLearn is analysed and compared with two existing solutions and three data poisoning attacks at three poisoning levels $10\% < \Delta L < 20\%$. The results showed that SecureLearn outperformed others in improving both the security and adversarial robustness of ML against various data poisoning attacks.

SecureLearn effectively enhanced the resilience of multiclass ML across RF, DT, GNB and MLP, confirming its generalisability beyond algorithm-specific defences. For all evaluated models, SecureLearn consistently maintained a minimum accuracy of 90%, recall and f1-score to at least 75%, and reduced the FDR to at least 0.06 against three distinct poisoning attacks. In the context of neural networks, SecureLearn achieved a minimum of 97% recall and f1-score against all selected data poisoning attacks. Furthermore, the adversarial robustness of models is improved with an average accuracy trade-off of only 3%.

Although various solutions Ho et al. (2022a), Tao et al. (2021) are provided in the literature, none

have proposed a two-layer approach to enhance the resilience and adversarial robustness of ML. Also, existing adversarial training mechanisms, for example, (Ho et al., 2022a) are limited to a gradient-oriented approach, which enhances the adversarial robustness of neural networks and advanced ML and DL models but is ineffective for traditional models, such as RF, DT and GNB models. The FORT follows a baseline property of feature importance, this method applies to all types of ML models, including traditional models and neural networks. The feature importance score informs the decision criteria of the model and helps generalise the model. By adding a small fraction of perturbation into the features with high importance, the model is taught to distinguish benign and poisoned data points. In this way, the resilience of the ML model is improved.

The experimentation of SecureLearn is conducted under assumptions to strengthen the efficacy and resilience of ML applications against data poisoning attacks; however, it may require additional constraints to be adapted for sensitive data and applications. For example, it might be limited in distinguishing between exceptional non-intrusive traffic packets and poisoned packets.

5.11 Summary

This chapter presented SecureLearn, a new attack-agnostic method to defend traditional multiclass models from data poisoning attacks. It is a two-layer solution that secures the training pipeline of the models by first effectively sanitising the dataset to remove poison and then improving adversarial robustness with the FORT adversarial training. SecureLearn secures the model from existing attacks and strengthens resilience against evolving attacks, hence fulfilling the aim of this thesis. This chapter answered RQ 3 by analysing the effectiveness of SecureLearn, highlighting its strengths and weaknesses and comparing it with existing solutions. To answer thoroughly, a new 3D evaluation matrix is proposed that has evaluated the proposed solution from three orthogonal dimensions. The experiments are conducted with three differently structured datasets to analyse the generalisability of SecureLearn. SecureLearn defends against black-box and grey-box poisoning attacks without requiring prior knowledge of the model and any additional dataset. SecureLearn provides robustness to the model in a two-layer approach, with first an improvised data sanitisation and second with a new FORT adversarial training. This way, making it an attack-agnostic and generic method that applies to all types of classification models.

Chapter 6. Conclusion and Future Work

The thesis proposed an attack-agnostic solution to defend multiclass ML against data poisoning attacks. Following this aim, the thesis has answered three research questions from two aspects of adversarial and defence standpoints. These answers provided a thorough analysis of ML security from new perspectives and covered traditional multiclass ML, which are overlooked in the literature. Firstly, this thesis explored various security vulnerabilities and highlighted limitations of existing solutions, stating that most solutions are attack-specific. Furthermore, it practically explored security vulnerabilities in traditional ML, motivated to assess the change in their learning dynamics with poisoned training. Conclusively, this assessment contributed to identifying vulnerabilities in ML models and their characteristics with poisoned training. These identifications helped fulfill the aim of this research by highlighting the relationships between ML characteristics and data poisoning.
Secondly, this thesis answered what the impact and consequences of multiclass data poisoning attacks would be under limited adversarial capabilities and knowledge to resolve a complex challenge of analysing the practicality and risks of data poisoning attacks. The answer highlighted that data poisoning attacks are successful and have severe consequences, even when implemented under limited adversarial capabilities. The new attack vectors may significantly degrade their performance, leading to a 50% decrease in accuracy and have successfully exploited adversarially robust ML models, trained with adversarial training. This analysis practically answered the limitations of existing solutions to defend against data poisoning attacks.
Following the above answers of ML security from an adversarial perspective, this thesis proposed a strengthened defence called SecureLearn, inspired by the outcomes of the above analyses. SecureLearn is a security-by-design and attack-agnostic solution that shows capabilities to mitigate various data poisoning attacks and is adaptable to many ML models. It mitigates reactive poisoning impact by cleaning datasets with its data sanitisation mechanism and provides proactive resilience to ML by improving adversarial robustness through FORT training.
Conclusively, it comprehensively assessed traditional ML against data poisoning attacks through an adversarial lens, followed by proposing an enhanced attack-agnostic solution which mitigates data poisoning attacks, irrespective of new attack vectors and is supported in all types of ML classification models.

6.1 Evaluate Research Objectives

Following the development of SecureLearn, an attack-agnostic solution, this thesis fulfilled the defined research objectives. The evaluation of objectives is given as follows:

- The first objective of the thesis is explore data poisoning attacks and their impact under limited

adversarial capabilities and knowledge. This objective is achieved in Chapter 4 by proposing the OOP attack and implementing it under grey-box scenarios. The attack has been practically successful and highlighted a high impact on three ML applications. Moreover, two case studies are conducted in Chapter 4, further assessing its impact against two real-life applications and highlighting the adverse impact of data poisoning in breaching adversarial training, a significant resilience mechanism against data poisoning attacks.

- The second objective of the thesis is achieved in Chapter 5 by proposing SecureLearn, an attack-agnostic defence, to mitigate data poisoning attacks in multiclass ML. SecureLearn is evaluated against four traditional multiclass models, trained with three datasets, highlighting its efficacy in enhancing the resilience of all selected models against three data poisoning attacks.

- The third objective of the thesis is achieved by proposing a 3D evaluation matrix to assess SecureLearn thoroughly against data poisoning attacks and compare it with existing mitigations in various settings. Chapter 5 provided a thorough assessment of SecureLearn in 3D evaluation matrix and its effectiveness against three data poisoning attacks against three distinct datasets in four multiclass models. The evaluation also compared it with two existing mitigations, highlighting that SecureLearn outperformed others as an attack-agnostic defence in all provided scenarios.

6.2 Contribution to Defence

Having revisited the adversarial aspect of analysing ML against data poisoning attacks, this thesis proposed deep behavioural analysis, revealing vulnerable characteristics of individual ML models and their relations with data poisoning attacks. Following deep behavioural analysis, this thesis designed the OOP attack to assess the resilience of multiclass models with constrained adversarial capabilities. The examination of OOP attack also strengthened the implications of behavioural analysis by analysing the underlying behaviours of poisoned multiclass models. The contributions to defence of these analyses are given as follows.

- Alongside the efficient development of classification systems with ML, this method allows practitioners to understand algorithm-level vulnerabilities and weaknesses of ML. The practitioners utilise these findings to define their training methodology, pre-training assessments of the datasets, and develop test cases for the security assessment of the trained model

- This approach enables defenders to improve the resilience of the ML by highlighting relationships between data poisoning and individual model characteristics. It also helps design stealthy data poisoning attacks by exploiting the characteristics of models that are sensitive to poison.

- The security analysis of multiclass ML under limited adversarial capabilities and knowledge helps practitioners understand the practicality and real-world impact of data poisoning attacks against traditional multiclass models.
- This chapter also puts forward the understanding of exploiting a common ML characteristic, called outliers, and its effectiveness in manipulating the learning fundamentals of models.
- This chapter emphasized the limitations of adversarial training in securing traditional classification models, highlighting the need to develop adaptive adversarial training mechanisms to enhance the resilience of traditional ML algorithms, not only for DL models and neural networks.

Following the adversarial aspect, this thesis strengthened the security and resilience of multiclass ML against existing and evolving data poisoning attacks by proposing an improved defence, SecureLearn. The contributions to defense made by SecureLearn are given as follows.

- SecureLearn is a step towards an enhanced mitigation that is an attack-agnostic solution and secures multiclass models against data poisoning, irrespective of particular attacks. Practitioners can implement it during the model training when they are unsure about the reliability of the dataset or to strengthen the adversarial robustness of the model against potential attacks. However, SecureLearn is a generalised defence and may require additional safeguards to assure particular scenarios of the applications.
- SecureLearn has provided a new approach of adaptable FORT adversarial training in traditional ML and neural networks with a trade-off of less than 3% in accuracy and robustness. This solution helps practitioners improve the adversarial robustness of traditional ML.
- SecureLearn is a standalone solution that does not require additional data for FORT training. It generates adversarial training data on its own and hence reduces the overhead of Practitioners.

6.3 Limitations

Each chapter has discussed the limitations of the individual contributions. Following those, this section presents the broader perspective of the limitations of the thesis, which could be addressed in future work. This thesis investigated the adversarial impact and consequences of data poisoning attacks; nonetheless, the scope of the work exclusively focused on supervised classification models, which can be extended to investigate the impact of poisoning in regression, unsupervised and DL models. Next, this thesis examined poisoned training of labeled datasets for the classification models, which can be extended to study poisoning impact in unlabeled datasets. Understanding the data poisoning impact in other ML types will provide a more comprehensive knowledge of poisoned training,

which ultimately helps strengthen defences against data poisoning attacks.

Furthermore, this thesis developed the analysis schemes under restricted threat models to study poisoning impact under limited adversarial capabilities. However, studying these poisoning implications under various threat models of extended adversarial capabilities, enhancing adversarial goals to inject backdoors, or inferring models beyond performance degradation enables practitioners to understand poisoning threats more comprehensively.

Finally, this thesis assessed the adversarial attacks and existing defences in real-world applications; however, the proposed defence can also be evaluated against real-world and security-sensitive applications to highlight its strength in securing these applications. Another limitation of presenting SecureLearn is that it has been evaluated in a grey-box adversarial setting. The threat model can be enhanced by analysing SecureLearn with white-box adversaries, which involve providing complete knowledge of the targeted system to adversaries to develop more stealthy and intrinsic attacks.

6.4 Future Work

Following the contributions of the thesis, this section discusses the future directions that can be taken as next steps to this research in the context of secure and trustworthy ML.

Chapter 3 presented a deep behavioural analysis of classification models against data poisoning, which can be extended to explore the behaviour of regression models and unsupervised ML to deepen the understanding of how data poisoning changes the learning fundamentals of other ML types. It should also be explored in DL, as DL models are largely embedded in digital systems nowadays. This future direction provides a thorough understanding of poisoning criteria to strengthen threat models, testing use cases against data poisoning attacks. Another potential direction is to conduct the behavioral analysis of poisoned models in real-time systems to understand poisoned behaviour in complex use cases. The behavioural analysis can also be extended to analyse model hallucination and jailbreak attacks by identifying changing characteristics with poisoned training.

Extending the implementation of the OOP attack, a potential future direction is to utilise it to exploit other mitigations, such as data sanitisation and differential privacy. This direction highlights the limitations of these mitigation solutions, providing an opportunity to improve these and strengthen the security of ML. The OOP attack is designed for multiclass models, which can be extended to poison multilabel models and enable practitioners to develop strong resilience against these attacks. Exploiting outliers in the OOP attack opens directions to develop new attack vectors by exploiting other characteristics of ML algorithms, such as exploiting the majority voting mechanism in KNN or altering feature importance scores in RF and DT. This direction will provide detailed knowledge of ML vulnerabilities and derive potential directions towards secure ML.

The above future directions highlight prospective work from an adversarial perspective, the other

perspective to discuss future directions from secure ML. This thesis primarily examined adversarial training, data sanitisation, and SecureLearn, their efficacy and limitations in classification models, which can be analysed to secure unsupervised models and DL. This direction will also improve SecureLearn in other types of ML. Though this thesis thoroughly evaluated SecureLearn under the 3D evaluation matrix, which can be further extended to other attacks, including inference and backdoor attacks. Furthermore, SecureLearn next extended to secure real-time applications as a potential avenue to explore, and would help improve SecureLearn itself.

Another very important direction in this regard is the development of security processes and policies for the secure development and deployment of ML models. These policies define the security principles in various domains and applications. Following the knowledge and understanding provided in this thesis, practitioners can develop security policies to use these models and define checklists for the secure development of ML.

References

Abdi, H. (2007). Z-scores. *Encyclopedia of measurement and statistics*, pages 1055–1058.

Abdulboriy, A. and Shin, J. S. (2024). An incremental majority voting approach for intrusion detection system based on machine learning. *IEEE Access*, 12:18972–18986.

Adarsh, P. and Jeyakumari, D. (2013). Multiclass svm-based automated diagnosis of diabetic retinopathy. In *2013 International Conference on Communication and Signal Processing*, pages 206–210. IEEE.

Aghakhani, H., Meng, D., Wang, Y.-X., Kruegel, C., and Vigna, G. (2021). Bullseye polytope: A scalable clean-label poisoning attack with improved transferability. In *2021 IEEE European symposium on security and privacy (EuroS&P)*, pages 159–178. IEEE.

Ahmed, U., Lin, J. C.-W., and Srivastava, G. (2022). Mitigating adversarial evasion attacks by deep active learning for medical image classification. *Multimedia Tools and Applications*, 81(29):41899–41910.

Ain, Q. U., Javed, A., and Irtaza, A. (2025). Deepevader: An evasion tool for exposing the vulnerability of deepfake detectors using transferable facial distraction blackbox attack. *Engineering Applications of Artificial Intelligence*, 145:110276.

Aïvodji, U., Gambs, S., and Ther, T. (2019). GAMIN: an adversarial approach to black-box model inversion. *CoRR*, abs/1909.11835.

Al-Haijaa, Q. A. and Ishtaiwia, A. (2021). Machine learning based model to identify firewall decisions to improve cyber-defense. *Int. J. Adv. Sci. Eng. Inf. Technol*, 11(4):1688–1695.

Alarab, I. and Prakoonwit, S. (2023). Uncertainty estimation based adversarial attack in multi-class classification. *Multimedia Tools and Applications*, 82(1):1519–1536.

Anthi, E., Williams, L., Javed, A., and Burnap, P. (2021). Hardening machine learning denial of service (dos) defences against adversarial attacks in iot smart home networks. *Computers & Security*, 108:102352.

Apruzzese, G., Andreolini, M., Colajanni, M., and Marchetti, M. (2020a). Hardening random forest cyber detectors against adversarial attacks. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 4(4):427–439.

Apruzzese, G., Andreolini, M., Marchetti, M., Venturi, A., and Colajanni, M. (2020b). Deep reinforcement adversarial learning against botnet evasion attacks. *IEEE Transactions on Network and Service Management*, 17(4):1975–1987.

Aryal, K., Gupta, M., and Abdelsalam, M. (2022). Analysis of label-flip poisoning attack on machine learning based malware detector. In *2022 IEEE International Conference on Big Data (Big Data)*, pages 4236–4245. IEEE.

Awajan, A. (2023). A novel deep learning-based intrusion detection system for iot networks. *Computers*, 12(2):34.

Ayub, M. A., Johnson, W. A., Talbert, D. A., and Siraj, A. (2020). Model evasion attack on intrusion detection systems using adversarial machine learning. In *2020 54th Annual Conference on Information Sciences and Systems (CISS)*, pages 1–6.

Baiyekusi, O., Mahmoud, H., Mi, D., Arshad, J., Adeyemi-Ejeye, F., and Lee, H. (2024). An ml-based spectrum sharing technique for time-sensitive applications in industrial scenarios. In *2024 International Wireless Communications and Mobile Computing (IWCMC)*, pages 1607–1612.

Baker, T., Li, T., Jia, J., Zhang, B., Tan, C., and Zomaya, A. Y. (2024). Poison-tolerant collaborative filtering against poisoning attacks on recommender systems. *IEEE Transactions on Dependable and Secure Computing*, 21(5):4589–4599.

Balevi, E. and Gitlin, R. D. (2017). Unsupervised machine learning in 5g networks for low latency communications. In *2017 IEEE 36th International Performance Computing and Communications Conference (IPCCC)*, pages 1–2. IEEE.

Bao, H., Wei, K., Wu, Y., Qian, J., and Deng, R. H. (2025). Distributional black-box model inversion attack with multi-agent reinforcement learning. *IEEE Transactions on Information Forensics and Security*.

Baracaldo, N., Chen, B., Ludwig, H., and Safavi, J. A. (2017). Mitigating poisoning attacks on machine learning models: A data provenance based approach. In *Proceedings of the 10th ACM workshop on artificial intelligence and security*, pages 103–110.

Barreno, M., Bartlett, P. L., Chi, F. J., Joseph, A. D., Nelson, B., Rubinstein, B. I., Saini, U., and Tygar, J. D. (2008). Open problems in the security of learning. In *Proceedings of the 1st ACM workshop on Workshop on AISec*, pages 19–26.

Basu, S., Izmailov, R., and Mesterharm, C. (2019). Membership model inversion attacks for deep networks. *CoRR*, abs/1910.04257.

Biggio, B., Corona, I., Maiorca, D., Nelson, B., Šrndić, N., Laskov, P., Giacinto, G., and Roli, F. (2013). Evasion attacks against machine learning at test time. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2013, Prague, Czech Republic, September 23-27, 2013, Proceedings, Part III 13*, pages 387–402. Springer.

Biggio, B., Nelson, B., and Laskov, P. (2011). Support vector machines under adversarial label noise. In *Asian conference on machine learning*, pages 97–112. PMLR.

Biggio, B., Nelson, B., and Laskov, P. (2012). Poisoning attacks against support vector machines. In *Proceedings of the 29th International Coference on International Conference on Machine Learning*, pages 1467–1474.

Bostani, H. and Moonsamy, V. (2021). Evadedroid: A practical evasion attack on machine learning for black-box android malware detection. *CoRR*, abs/2110.03301.

Brown, A., Gupta, M., and Abdelsalam, M. (2024). Automated machine learning for deep learning based malware detection. *Computers & Security*, 137:103582.

Cao, X. and Gong, N. Z. (2017). Mitigating evasion attacks to deep neural networks via region-based classification. In *Proceedings of the 33rd Annual Computer Security Applications Conference*, pages 278–287.

Carlini, N. and Terzis, A. (2021). Poisoning and backdooring contrastive learning. *arXiv preprint arXiv:2106.09667*.

Carnerero-Cano, J., Munoz-Gonzalez, L., Spencer, P., and Lupu, E. C. (2023). Hyperparameter learning under data poisoning: Analysis of the influence of regularization via multiobjective bilevel optimization. *IEEE Transactions on Neural Networks and Learning Systems*.

Centurion, D. I., Chubarian, K., Fan, B., Sgherzi, F., Rashakrishnan, T., Sidiropoulos, A., and Straight, A. (2023). Geometric algorithms for k-nn poisoning. In *Proceedings of the 35th Canadian Conference on Computational Geometry (CCCG 2023)*. Canadian Conference on Computational Geometry.

Chalé, M. and Bastian, N. D. (2022). Generating realistic cyber data for training and evaluating machine learning classifiers for network intrusion detection systems. *Expert Systems with Applications*, 207:117936.

Chan, P. P., He, Z.-M., Li, H., and Hsu, C.-C. (2018a). Data sanitization against adversarial label contamination based on data complexity. *International Journal of Machine Learning and Cybernetics*, 9:1039–1052.

Chan, P. P., He, Z.-M., Li, H., and Hsu, C.-C. (2018b). Data sanitization against adversarial label contamination based on data complexity. *International Journal of Machine Learning and Cybernetics*, 9(6):1039–1052.

Chen, H. and Koushanfar, F. (2023). Tutorial: Toward robust deep learning against poisoning attacks. *ACM Transactions on Embedded Computing Systems*, 22(3):1–15.

Chen, J., Wang, W. H., and Shi, X. (2020). Differential privacy protection against membership inference attack on machine learning for genomic data. *bioRxiv*.

Chen, J., Zhang, X., Zhang, R., Wang, C., and Liu, L. (2021). De-pois: An attack-agnostic defense against data poisoning attacks. *IEEE Transactions on Information Forensics and Security*, 16:3412–3425.

Chen, K., Lou, X., Xu, G., Li, J., and Zhang, T. (2022). Clean-image backdoor: Attacking multi-label models with poisoned labels only. In *The eleventh international conference on learning representations*.

Chen, Z. and Pattabiraman, K. (2023). Overconfidence is a dangerous thing: Mitigating membership inference attacks by enforcing less confident prediction.

Chen, Z., Wu, J., Fu, A., Su, M., and Deng, R. H. (2023). Mp-clf: An effective model-preserving collaborative deep learning framework for mitigating data leakage under the gan. *Knowledge-Based Systems*, 270:110527.

Chenthan, H. N. (2023). Network intrusion dataset (cic-ids-2017). *Kaggle*.

Chillara, A. K., Saxena, P., Maiti, R. R., Gupta, M., Kondapalli, R., Zhang, Z., and Kesavan, K. (2024). Deceiving supervised machine learning models via adversarial data poisoning attacks: a case study with usb keyboards. *International Journal of Information Security*, 23(3):2043–2061.

Cinà, A. E., Grosse, K., Demontis, A., Vascon, S., Zellinger, W., Moser, B. A., Oprea, A., Biggio, B., Pelillo, M., and Roli, F. (2023). Wild patterns reloaded: A survey of machine learning security against training data poisoning. *ACM Computing Surveys*, 55(13s):1–39.

Cohen, S. (2025). The evolution of machine learning: Past, present, and future. In *Artificial Intelligence in Pathology*, pages 3–14. Elsevier.

Das, A., Tariq, A., Batalini, F., Dhara, B., and Banerjee, I. (2024). Exposing vulnerabilities in clinical llms through data poisoning attacks: Case study in breast cancer. *medRxiv*.

David, M. W. (2018). Unsw_nb15. *Kaggle*.

Del Moral, P., Nowaczyk, S., and Pashami, S. (2022). Why is multiclass classification hard? *IEEE Access*, 10:80448–80462.

Demontis, A., Melis, M., Pintor, M., Jagielski, M., Biggio, B., Oprea, A., Nita-Rotaru, C., and Roli, F. (2018). On the intriguing connections of regularization, input gradients and transferability of evasion and poisoning attacks. *CoRR*, abs/1809.02861.

Deng, L. (2012). The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE Signal Processing Magazine*, pages 141–142.

Di Leo, G. and Sardanelli, F. (2020). Statistical significance: p value, 0.05 threshold, and applications to radiomics—reasons for a conservative approach. *European radiology experimental*, 4:18.

Dixit, M., Moholkar, A., Limaye, S., and Limaye, D. (2018). Naive bayes and svm based nids. In *2018 3rd International conference on inventive computation technologies (ICICT)*, pages 527–532. IEEE.

Drews, S., Albarghouthi, A., and D'Antoni, L. (2020). Proving data-poisoning robustness in decision trees. In *Proceedings of the 41st ACM SIGPLAN conference on programming language design and implementation*, pages 1083–1097.

Dunn, C., Moustafa, N., and Turnbull, B. (2020). Robustness evaluations of sustainable machine learning models against data poisoning attacks in the internet of things. *Sustainability*, 12(16):6434.

Ebrahimi, M., Zhang, N., Hu, J., Raza, M. T., and Chen, H. (2020). Binary black-box evasion attacks against deep learning-based static malware detectors with adversarial byte-level language model. *arXiv preprint arXiv:2012.07994*.

Farinu, U. (2025). Fairness, accountability, and transparency in ai: Ethical challenges in data-driven decision-making. *Available at SSRN 5128174*.

Fisher, R. A. (1936). Iris.

Geiping, J., Fowl, L., Somepalli, G., Goldblum, M., Moeller, M., and Goldstein, T. (2021). What doesn't kill you makes you robust (er): How to adversarially train against data poisoning. *arXiv preprint arXiv:2102.13624*.

Geiping, J., Fowl, L. H., Huang, W. R., Czaja, W., Taylor, G., Moeller, M., and Goldstein, T. (2020). Witches' brew: Industrial scale data poisoning via gradient matching. In *International Conference on Learning Representations*.

Ghosh, A., Jagielski, M., and Wilson, C. (2022). Subverting fair image search with generative adversarial perturbations. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, pages 637–650.

Gibert, D., Planes, J., Le, Q., and Zizzo, G. (2023). Query-free evasion attacks against machine learning-based malware detectors with generative adversarial networks.

Gil-Fournier, A. and Parikka, J. (2021). Ground truth to fake geographies: machine vision and learning in visual practices. *AI & society*, 36:1253–1262.

Goldblum, M., Tsipras, D., Xie, C., Chen, X., Schwarzschild, A., Song, D., Madry, A., Li, B., and Goldstein, T. (2020). Dataset security for machine learning: Data poisoning, backdoor attacks, and defenses. *CoRR*, abs/2012.10544.

Guého, D., Singla, P., Melton, R. G., and Schwab, D. (2020). A comparison of parametric and non-parametric machine learning approaches for the uncertain lambert problem. In *AIAA Scitech 2020 Forum*, page 1911.

Gupta, P., Yadav, K., Gupta, B. B., Alazab, M., and Gadekallu, T. R. (2023a). A novel data poisoning attack in federated learning based on inverted loss function. *Computers & Security*, 130:103270.

Gupta, S., Kumar, P., and Tekchandani, R. K. (2023b). Facial emotion recognition based real-time learner engagement detection system in online learning context using deep learning models. *Multimedia Tools and Applications*, 82(8):11365–11394.

Hayase, J., Kong, W., Somani, R., and Oh, S. (2021). Spectre: Defending against backdoor attacks using robust statistics. In *International Conference on Machine Learning*, pages 4129–4139. PMLR.

He, Y., Li, B., Liu, L., Ba, Z., Dong, W., Li, Y., Qin, Z., Ren, K., and Chen, C. (2025). Towards label-only membership inference attack against pre-trained large language models. In *USENIX Security*.

He, Z., Zhang, T., and Lee, R. B. (2019). Model inversion attacks against collaborative inference. In *Proceedings of the 35th Annual Computer Security Applications Conference*, page 148–162.

Ho, J., Lee, B.-G., and Kang, D.-K. (2022a). Attack-less adversarial training for a robust adversarial defense. *Applied Intelligence*, 52(4):4364–4381.

Ho, S., Reddy, A., Venkatesan, S., Izmailov, R., Chadha, R., and Oprea, A. (2022b). Data sanitization approach to mitigate clean-label attacks against malware detection systems. In *MILCOM 2022 - 2022 IEEE Military Communications Conference (MILCOM)*, pages 993–998.

Hossain, K. M. and Oates, T. (2024). Advancing security in ai systems: A novel approach to detecting backdoors in deep neural networks. In *ICC 2024-IEEE International Conference on Communications*, pages 740–745. IEEE.

Hu, C. and Hu, Y.-H. F. (2020). Data poisoning on deep learning models. In *2020 International Conference on Computational Science and Computational Intelligence (CSCI)*, pages 628–632. IEEE.

Huang, W. R., Geiping, J., Fowl, L., Taylor, G., and Goldstein, T. (2020). Metapoison: Practical general-purpose clean-label data poisoning. *Advances in Neural Information Processing Systems*, 33:12080–12091.

Hull, J. J. (2002). A database for handwritten text recognition research. *IEEE Transactions on pattern analysis and machine intelligence*, pages 550–554.

Jagielski, M., Oprea, A., Biggio, B., Liu, C., Nita-Rotaru, C., and Li, B. (2018). Manipulating machine learning: Poisoning attacks and countermeasures for regression learning. In *2018 IEEE symposium on security and privacy (SP)*, pages 19–35. IEEE.

Jagielski, M., Severi, G., Pousette Harger, N., and Oprea, A. (2021). Subpopulation data poisoning attacks. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 3104–3122.

Jebreel, N. M., Domingo-Ferrer, J., Sánchez, D., and Blanco-Justicia, A. (2024). Lfighter: Defending against the label-flipping attack in federated learning. *Neural Networks*, 170:111–126.

Jhong, S.-Y., Tseng, P.-Y., Siriphockpirom, N., Hsia, C.-H., Huang, M.-S., Hua, K.-L., and Chen, Y.-Y. (2020). An automated biometric identification system using cnn-based palm vein recognition. In *2020 international conference on advanced robotics and intelligent systems (ARIS)*, pages 1–6. IEEE.

Jia, J., Cao, X., and Gong, N. Z. (2021). Intrinsic certified robustness of bagging against data poisoning attacks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 7961–7969.

Jia, J., Salem, A., Backes, M., Zhang, Y., and Gong, N. Z. (2019). Memguard: Defending against black-box membership inference attacks via adversarial examples. *CoRR*, abs/1909.10594.

Jin, J., McMurtry, E., Rubinstein, B. I. P., and Ohrimenko, O. (2022). Are we there yet? timing and floating-point attacks on differential privacy systems. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 473–488.

Kamath, C. N., Bukhari, S. S., and Dengel, A. (2018). Comparative study between traditional machine learning and deep learning approaches for text classification. In *Proceedings of the ACM Symposium on Document Engineering 2018*, pages 1–11.

Khosravy, M., Nakamura, K., Hirose, Y., Nitta, N., and Babaguchi, N. (2021). Model inversion attack: Analysis under gray-box scenario on deep learning based face recognition system. *KSII Transactions on Internet and Information Systems*, 15:1100–1119.

Khosravy, M., Nakamura, K., Hirose, Y., Nitta, N., and Babaguchi, N. (2022). Model inversion attack by integration of deep generative models: Privacy-sensitive face generation from a face recognition system. *IEEE Transactions on Information Forensics and Security*, pages 357–372.

Koball, C., Wang, Y., Vaidyan, V., and Hastings, J. (2025). Assessing evasion attacks on tree-based machine learning models: Supervised vs. unsupervised approaches. In *2025 IEEE International Conference on Consumer Electronics (ICCE)*, pages 1–6. IEEE.

Koh, P. W., Steinhardt, J., and Liang, P. (2021). Stronger data poisoning attacks break data sanitization defenses.

Koh, P. W., Steinhardt, J., and Liang, P. (2022). Stronger data poisoning attacks break data sanitization defenses. *Machine Learning*, pages 1–47.

Kuppa, A. and Le-Khac, N.-A. (2020). Black box attacks on explainable artificial intelligence (xai) methods in cyber security. In *2020 International Joint Conference on neural networks (IJCNN)*, pages 1–8. IEEE.

Laishram, R. and Phoha, V. V. (2016). Curie: A method for protecting svm classifier from poisoning attack. *arXiv preprint arXiv:1606.01584*.

Lee, J. H., Kim, B. H., and Kim, M. Y. (2021). Machine learning-based automatic optical inspection system with multimodal optical image fusion network. *International Journal of Control, Automation and Systems*, 19(10):3503–3510.

Li, H., Bai, L., Ye, Q., Hu, H., Xiao, Y., Zheng, H., and Xu, J. (2025). A sample-level evaluation and generative framework for model inversion attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 18287–18295.

Li, K., Zheng, J., Yuan, X., Ni, W., Akan, O. B., and Poor, H. V. (2024). Data-agnostic model poisoning against federated learning: A graph autoencoder approach. *IEEE Transactions on Information Forensics and Security*.

Lin, J., Njilla, L. L., and Xiong, K. (2022). Secure machine learning against adversarial samples at test time. *EURASIP Journal on Information Security*, 2022(1):1.

Liu, G., Huang, X., and Yi, X. (2022). Adversarial label poisoning attack on graph neural networks via label propagation. In *European Conference on Computer Vision*, pages 227–243. Springer.

Liu, Q. and Wen, W. (2021). Model compression hardens deep neural networks: A new perspective to prevent adversarial attacks. *IEEE Transactions on Neural Networks and Learning Systems*, 34:3–14.

Liu, T. Y., Yang, Y., and Mirzasoleiman, B. (2023). Friendly noise against adversarial noise: A powerful defense against data poisoning attacks.

Lu, Y., Kamath, G., and Yu, Y. (2023). Exploring the limits of model-targeted indiscriminate data poisoning attacks. In *International Conference on Machine Learning*, pages 22856–22879. PMLR.

Lu, Y., Yang, M. Y., Kamath, G., and Yu, Y. (2024). Indiscriminate data poisoning attacks on pre-trained feature extractors. In *2024 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pages 327–343. IEEE.

Ma, K., Xu, Q., Zeng, J., Cao, X., and Huang, Q. (2021a). Poisoning attack against estimating from pairwise comparisons. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):6393–6408.

Ma, X., Niu, Y., Gu, L., Wang, Y., Zhao, Y., Bailey, J., and Lu, F. (2021b). Understanding adversarial attacks on deep learning based medical image analysis systems. *Pattern Recognition*, 110:107332.

Malik, F. (2022). Ctu13-csv-dataset. *Kaggle*.

Mayerhofer, R. and Mayer, R. (2022). Poisoning attacks against feature-based image classification. In *Proceedings of the Twelfth ACM conference on data and application security and privacy*, pages 358–360.

McCarthy, A., Ghadafi, E., Andriotis, P., and Legg, P. (2023). Defending against adversarial machine learning attacks using hierarchical learning: A case study on network traffic attack classification. *Journal of Information Security and Applications*, 72:103398.

Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., and Galstyan, A. (2021). A survey on bias and fairness in machine learning. *ACM computing surveys (CSUR)*, 54(6):1–35.

Melacci, S., Ciravegna, G., Sotgiu, A., Demontis, A., Biggio, B., Gori, M., and Roli, F. (2021). Domain knowledge alleviates adversarial attacks in multi-label classifiers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(12):9944–9959.

Meng, M. H., Bai, G., Teo, S. G., Hou, Z., Xiao, Y., Lin, Y., and Dong, J. S. (2022). Adversarial robustness of deep neural networks: A survey from a formal verification perspective. *IEEE Transactions on Dependable and Secure Computing*.

Mengara, O. (2024). A backdoor approach with inverted labels using dirty label-flipping attacks. *IEEE Access*.

Merzouk, M. A., Cuppens, F., Boulahia-Cuppens, N., and Yaich, R. (2022). Investigating the practicality of adversarial evasion attacks on network intrusion detection. *Annals of Telecommunications*, 77(11):763–775.

Muñoz-González, L., Biggio, B., Demontis, A., Paudice, A., Wongrassamee, V., Lupu, E. C., and Roli, F. (2017). Towards poisoning of deep learning algorithms with back-gradient optimization. In *Proceedings of the 10th ACM workshop on artificial intelligence and security*, pages 27–38.

Murugan, A., Nair, S. A. H., Preethi, A. A. P., and Kumar, K. S. (2021). Diagnosis of skin cancer using machine learning techniques. *Microprocessors and Microsystems*, 81:103727.

Nazary, F., Deldjoo, Y., Di Noia, T., and Di Sciascio, E. (2025). Stealthy llm-driven data poisoning attacks against embedding-based retrieval-augmented recommender systems. In *Adjunct Proceedings of the 33rd ACM Conference on User Modeling, Adaptation and Personalization*, pages 98–102.

Nguyen, B.-N., Chandrasegaran, K., Abdollahzadeh, M., and Cheung, N.-M. M. (2023). Label-only model inversion attacks via knowledge transfer. *Advances in Neural Information Processing Systems*, 36:68895–68907.

Nguyen, H. D. and Han, K. (2023). Safe reinforcement learning-based driving policy design for autonomous vehicles on highways. *International Journal of Control, Automation and Systems*, 21(12):4098–4110.

Pan, K., Gong, M., and Gao, Y. (2023). Privacy-enhanced generative adversarial network with adaptive noise allocation. *Knowledge-Based Systems*, 272:110576.

Pantelakis, V., Bountakas, P., Farao, A., and Xenakis, C. (2023). Adversarial machine learning attacks on multiclass classification of iot network traffic. In *Proceedings of the 18th International Conference on Availability, Reliability and Security*, pages 1–8.

Paracha, A. and Arshad, J. (2024). A bibliometric study toward quantitative research assessment of security of machine learning. *Information Discovery and Delivery*.

Paracha, A., Arshad, J., Farah, M. B., and Ismail, K. (2024a). Deep behavioral analysis of machine learning algorithms against data poisoning. *International Journal of Information Security*, 24(1):29.

Paracha, A., Arshad, J., Farah, M. B., and Ismail, K. (2024b). Exploring data poisoning attacks against adversarially trained skin cancer diagnostics. In *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)*, pages 220–225. IEEE.

Paracha, A., Arshad, J., Farah, M. B., and Ismail, K. (2024c). Machine learning security and privacy: a review of threats and countermeasures. *EURASIP Journal on Information Security*, 2024(1):10.

Paracha, A., Arshad, J., Farah, M. B., and Ismail, K. (2025a). Outlier-oriented poisoning attack: a grey-box approach to disturb decision boundaries by perturbing outliers in multiclass learning. *International Journal of Information Security*, 24(2):85.

Paracha, A., Arshad, J., Farah, M. B., and Ismail, K. (2025b). Securelearn-an attack-agnostic defense for multiclass machine learning against data poisoning attacks. *IEEE Transactions on Information Forensics and Security (Under review)*.

Paracha, A., Baiyekusi, O., Arshad, J., Mi, D., Lu, C., Zhang, Y., Wang, F., Chen, L., and Zhang, J. (2025c). Exploring multiclass data poisoning within an industrial 5g private network. In *2025 IEEE 101st Vehicular Technology Conference (VTC2025-Spring)*, pages 1–5. IEEE.

Patterson, W., Fernandez, I., Neupane, S., Parmar, M., Mittal, S., and Rahimi, S. (2022). A white-box adversarial attack against a digital twin. *arXiv preprint arXiv:2210.14018*.

Paudice, A., Muñoz-González, L., Gyorgy, A., and Lupu, E. C. (2018a). Detection of adversarial training examples in poisoning attacks through anomaly detection. *arXiv preprint arXiv:1802.03041*.

Paudice, A., Muñoz-González, L., and Lupu, E. C. (2018b). Label sanitization against label flipping poisoning attacks. In *ECML PKDD 2018 Workshops: Nemesis 2018, UrbReas 2018, SoGood 2018, IWAISe 2018, and Green Data Mining 2018, Dublin, Ireland, September 10-14, 2018, Proceedings 18*, pages 5–15. Springer.

Pawlicki, M., Choraś, M., and Kozik, R. (2020). Defending network intrusion detection systems against adversarial evasion attacks. *Future Generation Computer Systems*, 110:148–154.

Peri, N., Gupta, N., Huang, W. R., Fowl, L., Zhu, C., Feizi, S., Goldstein, T., and Dickerson, J. P. (2020). Deep k-nn defense against clean-label data poisoning attacks. In *European Conference on Computer Vision*, pages 55–70. Springer.

Phan, H., Thai, M. T., Hu, H., Jin, R., Sun, T., and Dou, D. (2020). Scalable differential privacy with certified robustness in adversarial learning. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119, pages 7683–7694. PMLR.

Plant, R., Giuffrida, M. V., Pitropakis, N., and Gkatzia, D. (2024). Evaluating language model vulnerability to poisoning attacks in low-resource settings. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*.

Qiao, Q. and Beling, P. A. (2016). Decision analytics and machine learning in economic and financial systems. *Environment Systems and Decisions*, 36(2):109–113.

Qin, T., Gao, X., Zhao, J., Ye, K., and Xu, C.-Z. (2023). Learning the unlearnable: Adversarial augmentations suppress unlearnable example attacks. *arXiv preprint arXiv:2303.15127*.

Qiu, H., Zheng, Q., Zhang, T., Qiu, M., Memmi, G., and Lu, J. (2020). Toward secure and efficient deep learning inference in dependable iot systems. *IEEE Internet of Things Journal*, 8(5):3180–3188.

Rafiq, H., Aslam, N., Ahmed, U., and Lin, J. C.-W. (2023). Mitigating malicious adversaries evasion attacks in industrial internet of things. *IEEE Transactions on Industrial Informatics*, 19(1):960–968.

Rahman, M. H., Islam, T., Rana, M. M., Tasnim, R., Mona, T. R., and Sakib, M. M. (2023). Machine learning approach on multiclass classification of internet firewall log files.

Rosenbloom, P. S., Laird, J. E., McDermott, J., Newell, A., and Orciuch, E. (1985). R1-soar: An experiment in knowledge-intensive programming in a problem-solving architecture. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, (5):561–569.

Rosenfeld, E., Winston, E., Ravikumar, P., and Kolter, Z. (2020). Certified robustness to label-flipping attacks via randomized smoothing. In *International Conference on Machine Learning*, pages 8230–8241. PMLR.

Russo, A. and Proutiere, A. (2021). Poisoning attacks against data-driven control methods. In *2021 American Control Conference (ACC)*, pages 3234–3241. IEEE.

Sagar, S. and Keke, C. (2021). Confidential machine learning on untrusted platforms: a survey. *Cybersecurity*, 4(1):30.

Saha, A., Subramanya, A., and Pirsiavash, H. (2020). Hidden trigger backdoor attacks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 11957–11965.

Salama, W. M., Aly, M. H., and Abouelseoud, Y. (2023). Deep learning-based spam image filtering. *Alexandria Engineering Journal*, 68:461–468.

Sanchez Vicarte, J. R., Schreiber, B., Paccagnella, R., and Fletcher, C. W. (2020). Game of threads: Enabling asynchronous poisoning attacks. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 35–52.

Sasmono, R. A., Agung, M. I. A., Nazaruddin, Y. Y., Oktavianus, J. A., and Tjahjono, G. (2021). Object matching in autonomous vehicle perception system using orb feature matching with svm

classifier. In *2021 International Conference on Instrumentation, Control, and Automation (ICA)*, pages 150–155. IEEE.

Seraj, S. (2022). Botdroid: Android botnet detection. *Kaggle*.

Shafahi, A., Huang, W. R., Najibi, M., Suciu, O., Studer, C., Dumitras, T., and Goldstein, T. (2018). Poison frogs! targeted clean-label poisoning attacks on neural networks. *Advances in neural information processing systems*, 31.

Shafahi, A., Najibi, M., Xu, Z., Dickerson, J., Davis, L. S., and Goldstein, T. (2020). Universal adversarial training. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 5636–5643.

Shafee, A. and Awaad, T. A. (2021). Privacy attacks against deep learning models and their countermeasures. *Journal of Systems Architecture*, 114:101940.

Shahid, A. R., Imteaj, A., Wu, P. Y., Igoche, D. A., and Alam, T. (2022). Label flipping data poisoning attack against wearable human activity recognition system. In *2022 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 908–914. IEEE.

Sharma, P., Austin, D., and Liu, H. (2019). Attacks on machine learning: Adversarial examples in connected and autonomous vehicles. In *2019 IEEE International Symposium on Technologies for Homeland Security (HST)*, pages 1–7. IEEE.

Shi, C. and Wang, Y. (2021). Non-parametric machine learning methods for interpolation of spatially varying non-stationary and non-gaussian geotechnical properties. *Geoscience Frontiers*, 12:339–350.

Shi, Y., Davaslioglu, K., and Sagduyu, Y. E. (2020). Over-the-air membership inference attacks as privacy threats for deep learning-based wireless signal classifiers. In *Proceedings of the 2nd ACM Workshop on Wireless Security and Machine Learning*, pages 61–66.

Shi, Y., Erpek, T., Sagduyu, Y. E., and Li, J. H. (2018). Spectrum data poisoning with adversarial deep learning. In *MILCOM 2018-2018 IEEE Military Communications Conference (MILCOM)*, pages 407–412. IEEE.

Shi, Y. and Sagduyu, Y. E. (2017). Evasion and causative attacks with adversarial deep learning. In *MILCOM 2017 - 2017 IEEE Military Communications Conference (MILCOM)*, pages 243–248.

Sinanović, H. and Mrdovic, S. (2017). Analysis of mirai malicious software. In *2017 25th International Conference on Software, Telecommunications and Computer Networks (SoftCOM)*, pages 1–5. IEEE.

Song, C., Ristenpart, T., and Shmatikov, V. (2017). Machine learning models that remember too much. In *Proceedings of the 2017 ACM SIGSAC Conference on computer and communications security*, pages 587–601.

Steinhardt, J., Koh, P. W. W., and Liang, P. S. (2017). Certified defenses for data poisoning attacks. *Advances in neural information processing systems*, 30.

Strobel, M. and Shokri, R. (2022). Data privacy and trustworthy machine learning. *IEEE Security & Privacy*, 20(5):44–49.

Su, J., Vargas, D. V., and Sakurai, K. (2019). One pixel attack for fooling deep neural networks. *IEEE Transactions on Evolutionary Computation*, 23(5):828–841.

Suya, F., Mahloujifar, S., Suri, A., Evans, D., and Tian, Y. (2021). Model-targeted poisoning attacks with provable convergence. In *International Conference on Machine Learning*, pages 10000–10010. PMLR.

Taherdoost, H. (2022). Classification of machine learning algorithms. In *Advances in Data Computing, Communication and Security: Proceedings of I3CS2021*, pages 417–422. Springer.

Tao, G., Liu, Y., Shen, G., Xu, Q., An, S., Zhang, Z., and Zhang, X. (2022a). Model orthogonalization: Class distance hardening in neural networks for better security. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 1372–1389.

Tao, J. and Shokri, R. (2025). Range membership inference attacks. In *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pages 346–361. IEEE.

Tao, L., Feng, L., Wei, H., Yi, J., Huang, S.-J., and Chen, S. (2022b). Can adversarial training be manipulated by non-robust features? *Advances in Neural Information Processing Systems*, 35:26504–26518.

Tao, L., Feng, L., Yi, J., Huang, S.-J., and Chen, S. (2021). Better safe than sorry: Preventing delusive adversaries with adversarial training. *Advances in Neural Information Processing Systems*, 34:16209–16225.

Tian, J., Shen, C., Wang, B., Xia, X., Zhang, M., Lin, C., and Li, Q. (2024). Lesson: Multi-label adversarial false data injection attack for deep learning locational detection. *IEEE Transactions on Dependable and Secure Computing*.

Titcombe, T., Hall, A. J., Papadopoulos, P., and Romanini, D. (2021). Practical defences against model inversion attacks for split neural networks.

Truex, S., Baracaldo, N., Anwar, A., Steinke, T., Ludwig, H., Zhang, R., and Zhou, Y. (2019). A hybrid approach to privacy-preserving federated learning. In *Proceedings of the 12th ACM workshop on artificial intelligence and security*, pages 1–11.

Tu, J., Wang, T., Wang, J., Manivasagam, S., Ren, M., and Urtasun, R. (2021). Adversarial attacks on multi-agent communication. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7768–7777.

Usynin, D., Rueckert, D., and Kaissis, G. (2023). Beyond gradients: Exploiting adversarial priors in model inversion attacks. *ACM Trans. Priv. Secur.*, 26(3).

van den Heuvel, E. and Zhan, Z. (2022). Myths about linear and monotonic associations: Pearson'sr, spearman's ρ, and kendall's τ. *The American Statistician*, 76:44–52.

Venkatesan, S., Sikka, H., Izmailov, R., Chadha, R., Oprea, A., and de Lucia, M. J. (2021). Poisoning attacks and data sanitization mitigations for machine learning models in network intrusion detection systems. In *MILCOM 2021 - 2021 IEEE Military Communications Conference (MILCOM)*, pages 874–879.

Vidnerová, P. and Neruda, R. (2016). Evolutionary generation of adversarial examples for deep and shallow machine learning models. In *Proceedings of the The 3rd Multidisciplinary International Social Networks Conference on SocialInformatics 2016, Data Science 2016*, pages 1–7.

Vivek, B., Mopuri, K. R., and Babu, R. V. (2018). Gray-box adversarial training. In *Proceedings of the European conference on computer vision (ECCV)*, pages 203–218.

Wang, H., Cao, Z., and Hong, B. (2020). A network intrusion detection system based on convolutional neural network. *Journal of Intelligent & Fuzzy Systems*, 38(6):7623–7637.

Wang, H., Wang, S., Jin, Z., Wang, Y., Chen, C., and Tistarelli, M. (2021a). Similarity-based gray-box adversarial attack against deep face recognition. In *2021 16th IEEE international conference on automatic face and gesture recognition (FG 2021)*, pages 1–8. IEEE.

Wang, P., Fan, E., and Wang, P. (2021b). Comparative analysis of image classification algorithms based on traditional machine learning and deep learning. *Pattern recognition letters*, 141:61–67.

Wang, Y., Mianjy, P., and Arora, R. (2021c). Robust learning for data poisoning attacks. In *International Conference on Machine Learning*, pages 10859–10869. PMLR.

Wang, Z., Huang, S., Huang, Y., and Cui, H. (2023). Energy-latency attacks to on-device neural networks via sponge poisoning. In *Proceedings of the 2023 Secure and Trustworthy Deep Learning Systems Workshop*, pages 1–11.

Weerasinghe, S., Alpcan, T., Erfani, S. M., and Leckie, C. (2021). Defending support vector machines against data poisoning attacks. *IEEE Transactions on Information Forensics and Security*, 16:2566–2578.

Wei, W., Chow, K.-H., Wu, Y., and Liu, L. (2023). Demystifying data poisoning attacks in distributed learning as a service. *IEEE Transactions on Services Computing*.

Wei, X., Guo, Y., and Yu, J. (2022). Adversarial sticker: A stealthy attack method in the physical world. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(3):2711–2725.

Wen, R., Zhao, Z., Liu, Z., Backes, M., Wang, T., and Zhang, Y. (2023). Is adversarial training really a silver bullet for mitigating data poisoning?

Xu, Q., Yang, Z., Zhao, Y., Cao, X., and Huang, Q. (2022). Rethinking label flipping attack: From sample masking to sample thresholding. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(6):7668–7685.

Xu, X., Wang, Q., Li, H., Borisov, N., Gunter, C. A., and Li, B. (2021). Detecting ai trojans using meta neural analysis. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 103–120. IEEE.

Xue, J., Zheng, M., Sheng, Y., Yang, L., Lou, Q., and Jiang, L. (2023). Trojfair: Trojan fairness attacks. In *Proceedings of the 1st ACM Workshop on Large AI Systems and Models with Privacy and Safety Analysis*, pages 47–56.

Yan, H., Li, X., Zhang, W., Wang, R., Li, H., Zhao, X., Li, F., and Lin, X. (2023). Automatic evasion of machine learning-based network intrusion detection systems. *IEEE Transactions on Dependable and Secure Computing*, pages 1–16.

Yang, Z., Wang, L., Yang, D., Wan, J., Zhao, Z., Chang, E.-C., Zhang, F., and Ren, K. (2023). Purifier: Defending data inference attacks via transforming confidence scores. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 10871–10879.

Yerlikaya, F. A. and Şerif Bahtiyar (2022). Data poisoning attacks against machine learning algorithms. *Expert Systems with Applications*, 208:118101.

You, Z., Ye, J., Li, K., Xu, Z., and Wang, P. (2019). Adversarial noise layer: Regularize neural network by adding noise. In *2019 IEEE International Conference on Image Processing (ICIP)*, pages 909–913.

Yu, M. and Sun, S. (2022). Natural black-box adversarial examples against deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 8936–8944.

Yu, T., Tao, P., and Ru, Z. (2025). Toward trustworthy ai: Explainable and robust machine learning for medical applications. *Available at SSRN 5328192*.

Zeng, X., Liu, C., Wang, Y.-S., Qiu, W., Xie, L., Tai, Y.-W., Tang, C.-K., and Yuille, A. L. (2019). Adversarial attacks beyond the image space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4302–4311.

Zhang, C., Tang, Z., and Li, K. (2023). Clean-label poisoning attack with perturbation causing dominant features. *Information Sciences*, 644:118899.

Zhang, M., Ren, Z., Wang, Z., Ren, P., Chen, Z., Hu, P., and Zhang, Y. (2021). Membership inference attacks against recommender systems. *CoRR*, abs/2109.08045.

Zhang, Q., Ma, J., Xiao, Y., Lou, J., and Xiong, L. (2020). Broadening differential privacy for deep learning against model inversion attacks. In *2020 IEEE International Conference on Big Data (Big Data)*, pages 1061–1070.

Zhang, S., Li, X., Zong, M., Zhu, X., and Wang, R. (2017). Efficient knn classification with different numbers of nearest neighbors. *IEEE transactions on neural networks and learning systems*, 29:1774–1785.

Zhang, Y., Albarghouthi, A., and D'Antoni, L. (2022). Bagflip: A certified defense against data poisoning. *Advances in Neural Information Processing Systems*, 35:31474–31483.

Zhao, B. and Lao, Y. (2022a). Clpa: Clean-label poisoning availability attacks using generative adversarial nets. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 9162–9170.

Zhao, B. and Lao, Y. (2022b). Towards class-oriented poisoning attacks against neural networks. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 3741–3750.

Zhong, H., Liao, C., Squicciarini, A. C., Zhu, S., and Miller, D. (2020). Backdoor embedding in convolutional neural network models via invisible perturbation. In *Proceedings of the Tenth ACM Conference on Data and Application Security and Privacy*, pages 97–108.

Zhou, L., Pan, S., Wang, J., and Vasilakos, A. V. (2017). Machine learning on big data: Opportunities and challenges. *Neurocomputing*, 237:350–361.

Zhu, C., Huang, W. R., Li, H., Taylor, G., Studer, C., and Goldstein, T. (2019a). Transferable clean-label poisoning attacks on deep neural nets. In *International conference on machine learning*, pages 7614–7623. PMLR.

Zhu, C., Huang, W. R., Shafahi, A., Li, H., Taylor, G., Studer, C., and Goldstein, T. (2019b). Transferable clean-label poisoning attacks on deep neural nets.

Zhu, C., Wang, H., Zhuang, Y., Li, J., and Cao, Y. (2022). A sparsity-limitation-based high-dimensional distribution searching algorithm for adversarial attack. *Journal of Sensors*, 2022(1):4031440.

Zhu, T., Ye, D., Zhou, S., Liu, B., and Zhou, W. (2023a). Label-only model inversion attacks: Attack with the least information. *IEEE Transactions on Information Forensics and Security*, 18:991–1005.

Zhu, Y., Miao, C., Xue, H., Li, Z., Yu, Y., Xu, W., Su, L., and Qiao, C. (2023b). Tilemask: A passive-reflection-based attack against mmwave radar object detection in autonomous driving. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 1317–1331.

Zhu, Y., Wen, H., Wu, J., and Zhao, R. (2023c). Online data poisoning attack against edge ai paradigm for iot-enabled smart city. *Math. Biosci. Eng*, 20:17726–17746.

Zhu, Z., Wu, C., Fan, R., Lian, D., and Chen, E. (2023d). Membership inference attacks against sequential recommender systems. In *Proceedings of the ACM Web Conference 2023*, page 1208–1219. Association for Computing Machinery.

Zou, Y., Zhang, Z., Backes, M., and Zhang, Y. (2020). Privacy analysis of deep learning in the wild: Membership inference attacks against transfer learning. *CoRR*, abs/2009.04872.

Detecting and Preventing Data Poisoning Attacks on AI Models

Halima I. Kure
School of Computing and Engineering
University of East London
London, UK
h.kure2@uel.ac.uk

Pradipta Sarkar
School of Computing and Engineering
University of East London
London, UK
u2602639@uel.ac.uk

Ahmed B. Ndanusa
Dept. of Computer Science
University of Abuja
Abuja, Nigeria
elahmedn@gmail.com

Augustine O. Nwajana
School of Engineering
University of Greenwich
Kent, UK
a.nwajana@greenwich.ac.uk

***Abstract*—This paper investigates the critical issue of data poisoning attacks on AI models, a growing concern in the ever-evolving landscape of artificial intelligence and cybersecurity. As advanced technology systems become increasingly prevalent across various sectors, the need for robust defence mechanisms against adversarial attacks becomes paramount. The study aims to develop and evaluate novel techniques for detecting and preventing data poisoning attacks, focusing on both theoretical frameworks and practical applications. Through a comprehensive literature review, experimental validation using the CIFAR-10 and Insurance Claims datasets, and the development of innovative algorithms, this paper seeks to enhance the resilience of AI models against malicious data manipulation. The study explores various methods, including anomaly detection, robust optimization strategies, and ensemble learning, to identify and mitigate the effects of poisoned data during model training.**

Experimental results indicate that data poisoning significantly degrades model performance, reducing classification accuracy by up to 27% in image recognition tasks (CIFAR-10) and 22% in fraud detection models (Insurance Claims dataset). The proposed defence mechanisms, including statistical anomaly detection and adversarial training, successfully mitigated poisoning effects, improving model robustness and restoring accuracy levels by an average of 15-20%. The findings further demonstrate that ensemble learning techniques provide an additional layer of resilience, reducing false positives and false negatives caused by adversarial data injections.

The findings of this paper contribute to the broader understanding of adversarial machine learning and provide actionable insights for practitioners in industries facing emerging threats to AI-based systems. By addressing the challenges of data poisoning attacks, this study aims to improve the security and reliability of AI models, ultimately fostering greater trust in AI technologies across critical applications.

Keywords—Data Poisoning, Artificial Intelligence, Cybersecurity, Machine Learning, Adversarial Attacks, Model Security

I. INTRODUCTION

Artificial Intelligence (AI) has become integral to various sectors, including healthcare, finance, transportation, and cybersecurity, due to its ability to analyze large datasets and make informed decisions. However, this widespread adoption has introduced new challenges, notably the vulnerability of AI systems to data poisoning attacks. These attacks involve injecting malicious data into training datasets, leading to compromised model performance and potentially erroneous outcomes. For instance, in [1], Biggio et al. demonstrated how adversaries could poison training data to degrade the performance of Support Vector Machines, leading to incorrect predictions and potential financial or reputational damage for organizations.

The significance of addressing data poisoning attacks cannot be overstated, as they pose substantial risks to the integrity of AI systems. Such attacks can lead to incorrect predictions, reduced decision-making accuracy, and diminished trust in AI applications across various industries. Organizations deploying AI technologies risk severe financial and reputational losses if their systems are compromised. Therefore, there is an urgent need for robust defense mechanisms to detect, prevent, and mitigate the impact of data poisoning. In [2], Korada examines the threat of data poisoning in machine learning and emphasizes the importance of securing training data to maintain AI system reliability.

In response to these challenges, research has focused on developing robust detection, prevention, and mitigation techniques against data poisoning attacks. In [3], Sardana et al. propose methods to detect and prevent data poisoning in machine learning models, highlighting the need for continuous monitoring and advanced verification methods. Additionally, in [4], Keshavarzkalhori et al. present solutions to build resilient AI systems capable of withstanding data and model poisoning attempts, advocating for the integration of security measures throughout the AI development lifecycle. Furthermore, in [5], Baracaldo et al. introduce a data provenance-based approach to mitigate

poisoning attacks on machine learning models, emphasizing the role of data lineage in enhancing model security. Similarly, in [6], Allwza analyzes model poisoning attacks in AI and proposes strategies for detecting and preventing such threats.

This study aims to enhance the security and reliability of AI models by developing effective defense mechanisms against data poisoning attacks. The objectives include conducting a comprehensive literature review to understand current threats and safeguards, proposing and comparing detection methods for contaminated data during AI model training, evaluating the effectiveness of these algorithms across various datasets to assess their robustness in real-world scenarios, and exploring novel attack vectors alongside preventive measures to fortify AI systems against potential adversarial actions. By addressing these objectives, the research aspires to contribute significantly to AI cybersecurity, providing stakeholders with practical tools and insights to enhance the resilience of AI applications against adversarial manipulations.

II. REALTED WORKS

In recent years, the field of machine learning has witnessed a surge in research addressing the challenges posed by data poisoning attacks. These attacks involve the deliberate manipulation of training data to compromise the integrity and performance of machine learning models. This section reviews several notable studies that have contributed to understanding and mitigating data poisoning threats.

A. Attack Taxonomy and Detection Strategies

In [2], Korada examines the threat of data poisoning in machine learning models, where adversaries introduce malicious data to manipulate model behavior. The study categorizes various attack types, including targeted and backdoor poisoning, and highlights detection and mitigation strategies such as data validation and secure handling. The author also discusses how leading AI providers like OpenAI and Google implement protective measures against these attacks. The study lacks empirical validation of the proposed defenses, leaving their practical effectiveness uncertain. In [7], Lin et al. explored adversarial attacks and data poisoning in machine learning, detailing methods like the Fast Gradient Sign Method (FGSM) for adversarial attacks and discussing label flipping and backdoor attacks in data poisoning. The study emphasizes the vulnerabilities these attacks introduce to ML models and underscores the necessity for robust defense mechanisms. The study underscores vulnerabilities but falls short in proposing robust, empirically validated defense mechanisms against these attacks. In [8], Fan et al. provides a comprehensive review of data poisoning attacks and corresponding defense mechanisms. They categorize attacks into availability and targeted types, discussing their impacts across various machine learning tasks. The authors also evaluate existing defense strategies, highlighting the need for robust and adaptive solutions to counteract evolving attack methods. Despite highlighting the need for adaptive solutions, the review lacks a critical assessment of existing defenses' limitations and does not propose novel approaches.

B. Defense Mechanisms and Frameworks

In [4], Keshavarzkalhori et al. propose a framework to enhance AI resilience against data and model poisoning attacks. The authors introduce a multi-layered defense strategy that combines data sanitization, robust training algorithms, and continuous monitoring to detect and prevent malicious alterations in training data and models. Their experimental results demonstrate a significant improvement in model robustness, reducing the success rate of poisoning attacks by up to 85%. However, the framework's complexity may pose challenges for real-world implementation, and its effectiveness against adaptive adversaries remains untested. In [3], Sardana et al. present strategies to detect and prevent data poisoning attacks on machine learning models. They evaluate various defense mechanisms, emphasizing the importance of robust data validation and anomaly detection to maintain model integrity. While these strategies are foundational, the study does not address advanced attack vectors that can bypass basic defenses, potentially limiting their applicability. In [5], Baracaldo et al. propose a methodology leveraging data provenance to detect and mitigate poisoning attacks on machine learning models. By analyzing the origin and transformation history of training data, their approach identifies and filters out malicious inputs, enhancing model robustness against adversarial manipulation. Experimental evaluations demonstrate that this provenance-based method improves detection rates compared to existing defenses. Nonetheless, the reliance on comprehensive provenance data may not be feasible in all scenarios, and the approach's scalability for large datasets is not evaluated. According to [9], Khan et al. introduced a federated split learning model designed to defend against data poisoning attacks in edge computing environments. The study combines federated learning and split learning techniques to enhance data privacy and security. Experimental results demonstrate the model's effectiveness in mitigating data poisoning threats, thereby improving the reliability of distributed learning systems in Industry 5.0 applications. Yet, the model's resilience against sophisticated, adaptive attacks and its performance across various edge scenarios require further investigation.

In [10], Paudice et al. address label flipping attacks, a form of data poisoning where adversaries alter the labels of training data to mislead machine learning models. They introduce an efficient algorithm to execute optimal label flipping attacks and propose a defense mechanism that detects and corrects mislabeled data points, thereby mitigating the attack's impact. Their approach enhances the resilience of machine learning systems against such adversarial manipulations. However, the approach's effectiveness against more subtle or complex poisoning strategies and its scalability in large-scale applications are not thoroughly examined.

C. Domain-Specific Analyses

In [11], Wang et al. provide a comprehensive survey on data poisoning attacks (DPAs) in Intelligent Transportation

Systems (ITS). They identify primary ITS data sources vulnerable to such attacks and propose a general framework for studying DPAs, categorizing existing models based on targeted data sources and their functions. The paper also discusses current limitations and suggests future research directions to enhance the trustworthiness of ITS. However, the survey primarily offers theoretical insights without empirical validation, and the proposed framework's practical applicability in diverse ITS environments is not assessed.

In [6], Allwza conducted a comprehensive analysis of model poisoning attacks in AI systems, focusing on backdoor attacks within convolutional neural networks. The study examined how varying the number of poisoned records and the size of the poison pattern affects model performance. A novel defense mechanism utilizing morphological operations was proposed to remove poison patterns from input data, effectively mitigating the attack's impact without significantly degrading model accuracy. While effective in controlled settings, the defense's performance in dynamic, real-world scenarios with varying attack strategies are not explored.

In [12], Alfeld et al. investigates data poisoning attacks on autoregressive models, where an adversary subtly alters initial input values to manipulate future forecasts. They develop a computationally efficient method to calculate the optimal attack strategy and demonstrate its effectiveness through experiments on both synthetic and real-world time series data. The study also discusses potential defensive measures against such adversarial manipulations. Although they discuss potential defenses, the study primarily centers on attack methodologies, offering limited evaluation of defensive measures' effectiveness.

In [13], Acuña examines the escalating threat of data poisoning in AI-driven healthcare systems, identifying critical security vulnerabilities and emphasizing the need for advanced, multi-layered defense strategies to protect patient data and maintain trust in digital healthcare solutions. The discussion, however, remains at a high level, lacking specific, actionable solutions or empirical evidence supporting the recommended strategies.

According to [14], Ino et al. demonstrate a data poisoning attack on an on-device learning anomaly detection system using MEMS accelerometers to monitor factory machinery vibrations. By exposing the accelerometers to specific acoustic frequencies, the authors tampered with sensor data during training, leading the system to misclassify abnormal vibrations as normal. This study highlights the vulnerability of on-device learning systems to physical attacks and underscores the need for robust security measures in such applications. While highlighting vulnerabilities, the study does not propose or evaluate specific defense strategies to counteract such physical data poisoning attacks.

In [15], Narisada et al. enhanced targeted data poisoning attacks on malware detection systems by developing an algorithm that generates poisoning points to misclassify specific malware as benign software. Their method achieves a high attack success rate while maintaining the overall accuracy of the detection model, posing significant challenges to existing defense mechanisms. The study, however, focuses on attack methodologies without exploring corresponding defense strategies or the practical implications of such attacks in real-world malware detection systems.

In [16], Verde et al. investigate the impact of data poisoning attacks on machine learning model reliability, focusing on how noise can degrade performance. They assess the resilience of various classification algorithms by introducing noise into voice data from the Saarbruecken Voice Database. Their findings indicate that even minor data contamination can significantly affect model accuracy, underscoring the necessity for robust defense mechanisms to maintain AI system reliability.

D. Comprehensive Surveys and Overviews

In [17], Goldblum et al. provides a comprehensive overview of dataset vulnerabilities in machine learning, focusing on data poisoning and backdoor attacks. They categorize various threat models, discuss defense strategies, and highlight open problems in the field. While comprehensive, the work does not delve deeply into the practical implementation challenges of the proposed defenses in real-world applications.

Data poisoning attacks have evolved from basic label flipping to sophisticated clean-label and backdoor methods, posing significant threats across sectors like cybersecurity, healthcare, finance, and autonomous systems. Detection techniques range from statistical analyses to meta-learning, each with inherent strengths and limitations. Defense strategies such as data sanitization, adversarial training, and federated learning offer layered protection but face challenges in balancing accuracy, scalability, adaptability to emerging threats, and privacy preservation. Ongoing research emphasizes the need for innovative, scalable, and adaptable defenses to safeguard AI systems against these evolving attacks.

III. METHODOLOGY

This study investigates the impact of data poisoning on machine learning models using two datasets: CIFAR-10 for image classification and an Insurance Claims dataset for fraud detection. The methodology involves systematically introducing poisoning attacks, training models on both clean and poisoned data, and evaluating performance degradation. The CIFAR-10 dataset is susceptible to poisoning due to similarities among image classes, making it suitable for studying label-flipping attacks. Meanwhile, the Insurance Claims dataset represents a real-world fraud detection scenario where poisoned data can mislead predictive models. The experimental workflow includes data preprocessing, poisoning simulation, model training, and evaluation through performance metrics such as accuracy and confusion matrices.

A. *CIFAR-10 Dataset*

The CIFAR-10 dataset [18], comprises 60,000 color images at a resolution of 32x32 pixels, evenly distributed across 10 distinct classes: airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships, and trucks. Specifically, the dataset is partitioned into 50,000 training images and 10,000 testing images. Each class contains exactly 6,000 images, ensuring balanced representation. The training set is further divided into five batches, each containing 10,000 images, while the test set consists of a single batch of 10,000 images. Notably, the test batch includes exactly 1,000 randomly selected images from each class. This dataset serves as a standard benchmark for evaluating image classification algorithms in machine learning research.

Fig. 1. Data poisoning and training workflow for CIFAR-10 classification. The process begins with loading and normalizing the CIFAR-10 dataset, followed by introducing label-flipping data poisoning. Data augmentation is applied before training the model. After training, the model is evaluated using a confusion matrix, and sample images are plotted to visualize the impact of poisoning on classification performance.

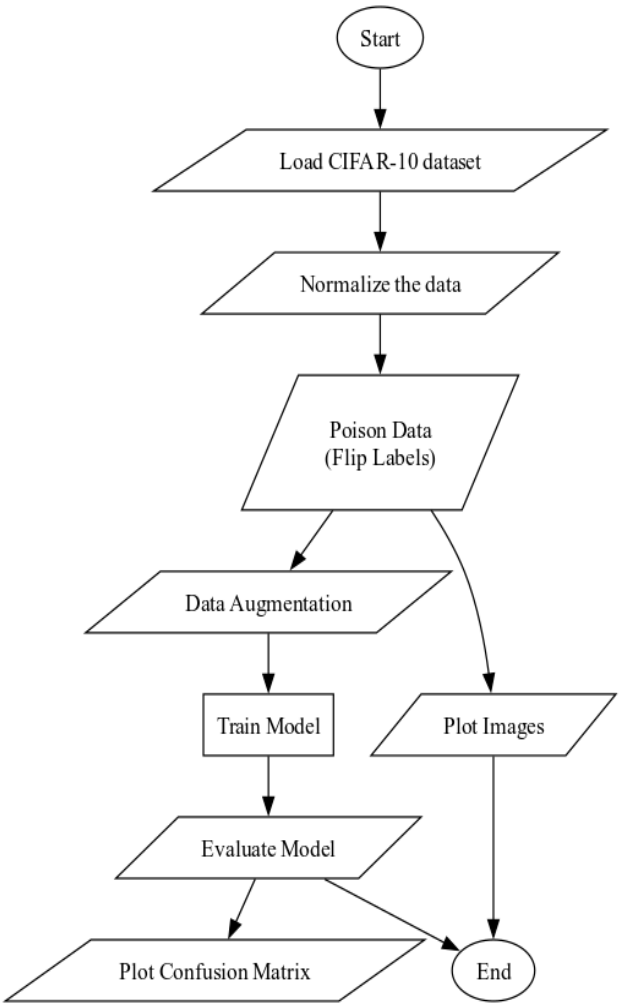


Fig 1. Workflow for training and evaluating a machine learning model under data poisoning conditions, highlighting steps from data loading and preprocessing to poisoning simulation, model training, evaluation, and fraud detection analysis.

The CIFAR-10 dataset is particularly vulnerable to data poisoning due to the visual similarities between certain classes (e.g., *trucks* and *automobiles*). This makes it an ideal candidate for studying the effects of data poisoning attacks such as label flipping and instance injection.

B. *Insurance Claim Dataset*

The Insurance Claims dataset contains anonymized data on insurance claims, with labels indicating whether each claim is fraudulent or non-fraudulent. This dataset is commonly used to develop fraud detection models, where the goal is to identify potentially fraudulent claims based on features such as the amount claimed, the time taken to file the claim, and the claimant's history.

Fig. 2, Data preprocessing, poisoning simulation, and model evaluation workflow for an insurance fraud detection system. The process begins with loading the insurance dataset, followed by data cleaning, encoding categorical variables, and simulating data poisoning. The dataset is split into training and test sets, and a Random Forest classifier is trained separately on both normal and poisoned data. The accuracy of the model is evaluated before and after poisoning, with comparisons made to assess the impact of poisoning. Finally, classification reports, confusion matrices, and fraud detection results are generated, visualizing the distribution of predicted fraud cases.

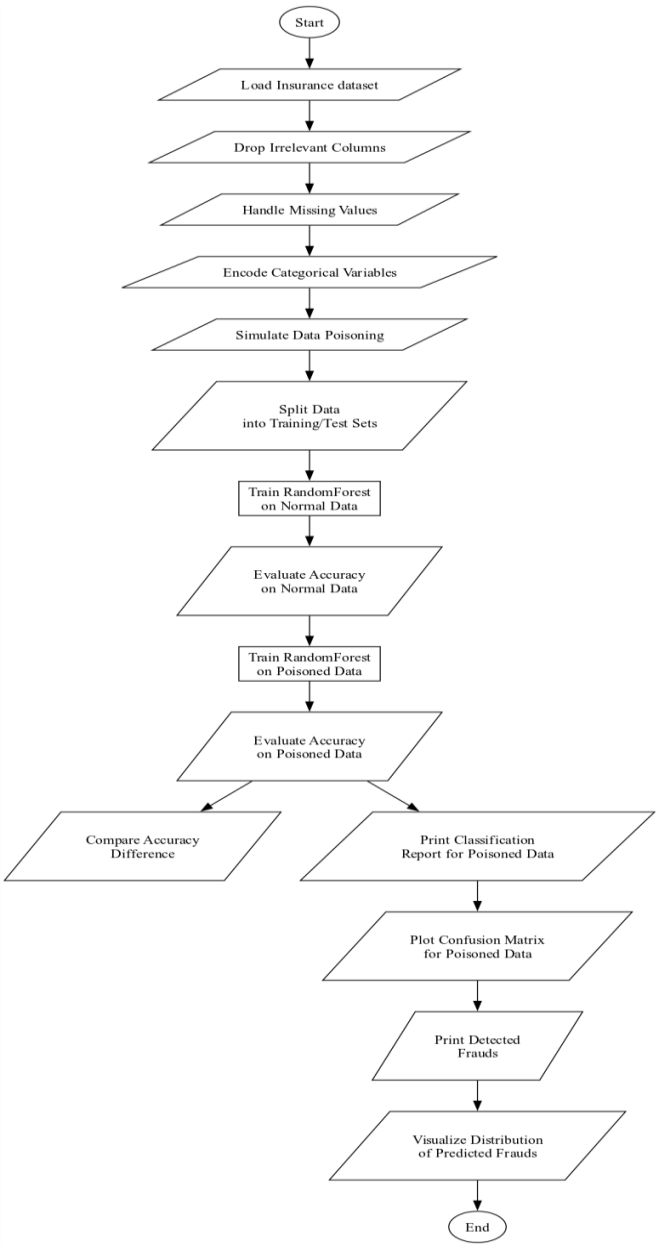


Fig 2. Data preprocessing and poisoning simulation workflow for an insurance fraud detection system, illustrating steps from dataset preparation to model evaluation.

Given its real-world applications, the Insurance Claims dataset provides a valuable case study for assessing the impact of data poisoning in financial systems.

C. Research Strategy and Design

The study adopts a positive research philosophy, relying on empirical data and quantitative analysis to investigate data poisoning effects. A deductive approach is used, where hypotheses are tested through controlled experiments and statistical analysis. The methodology is designed to be objective, replicable, and generalizable, contributing to AI security by systematically assessing poisoning attacks and their countermeasures.

An experimental research strategy is employed, focusing on controlled poisoning attacks in image classification and fraud detection models. The study also includes comparative analysis to evaluate the impact of different poisoning techniques and mitigation strategies across AI applications. While the strategy provides strong control and replicability, limitations such as dataset-specific findings and real-world complexity are acknowledged.

D. Sampling and Study Population

A stratified sampling approach is used to ensure balanced representation across datasets, with random sampling applied for poisoning. In the CIFAR-10 dataset, label flipping and image replacement are introduced, while in the fraud detection dataset, 5% of fraud and non-fraud labels are flipped. The study acknowledges potential biases and ensures reliability through multiple experimental runs and statistical testing.

Quantitative analysis methods, including descriptive statistics, comparative analysis, and statistical significance testing, are employed to assess data poisoning effects. Confusion matrices, feature importance scores, and visualizations (e.g., heatmaps, line plots) provide insights into how poisoning degrades model performance. Both image classification and fraud detection models are analyzed separately to highlight domain-specific vulnerabilities.

To ensure internal validity, control groups and randomization techniques are applied. External validity is enhanced by using diverse datasets and established benchmarks. Reliability is ensured through standardized procedures, repeated experimental runs, and sensitivity analyses. Limitations such as dataset-specific findings are acknowledged, with recommendations for broader validation in future studies.

IV. Results

This section presents the experimental findings from two key studies: image classification using the CIFAR-10 dataset and insurance fraud detection using a tabular dataset. The experiments assess the impact of data poisoning attacks on AI models, comparing their performance against baseline models trained on clean data. Key evaluation metrics such as accuracy, precision, recall, F1-score, and error rates are analyzed to determine how poisoning techniques affect model reliability. The results highlight the extent to which data poisoning compromises AI systems and emphasize the necessity for effective detection and prevention strategies.

A. Experimental Setup

The study examines the impact of data poisoning on AI models, focusing on image classification (CIFAR-10 dataset) and fraud detection (insurance claims dataset). The experiments were conducted on a high-performance workstation equipped with an Intel Core i9 processor, 64 GB RAM, and an NVIDIA RTX 3090 GPU. The software stack included TensorFlow, Keras, scikit-learn, and Python. The image classification experiment used a ResNet50 model with data augmentation, while the fraud detection experiment leveraged a Random Forest Classifier with preprocessing techniques such as feature scaling, one-hot encoding, and train-test splitting.

B. *Dataset and Poisoning Techniques*

The CIFAR-10 dataset consists of 60,000 images across 10 classes, with poisoning techniques applied to the "cat" and "dog" classes. Two approaches were used: image replacement (500 images swapped between categories) and label flipping (5% of labels altered). The insurance fraud dataset comprised 1,000,000 records, with 5% of fraud labels flipped, significantly altering the dataset's class balance.

The results of the experiments conducted on the CIFAR-10 dataset reveal crucial insights into the effects of data poisoning on model performance. The analysis focuses on three primary sets of images: the original training images, the poisoned training images (before replacement), and the poisoned and replaced training images.

1. **Original Training images:** The CIFAR-10 dataset initially contained clean images across 10 object categories. Fig. 3 presents a selection of original images, showcasing the variability in object appearance, lighting, and orientation. This inherent diversity is crucial for training machine learning models, enabling them to generalize effectively across different categories by learning robust feature representations.

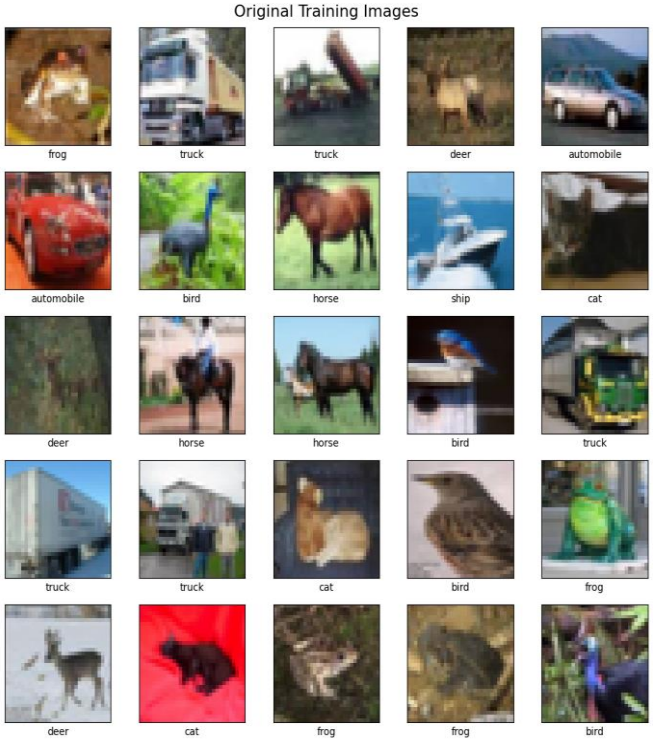


Fig 3. Original CIFAR-10 Training Images

2. **Poisoned Training Images (Before Replacement):** In this experiment, a subset of the training images was modified to introduce noise and distortions before any replacements were made, as shown in Fig. 4. These modifications were designed to simulate data poisoning, which negatively impacted the model's ability to generalize effectively. The introduction of such adversarial elements in the training data led to a degradation in model performance, highlighting the vulnerability of machine learning models to poisoned data.

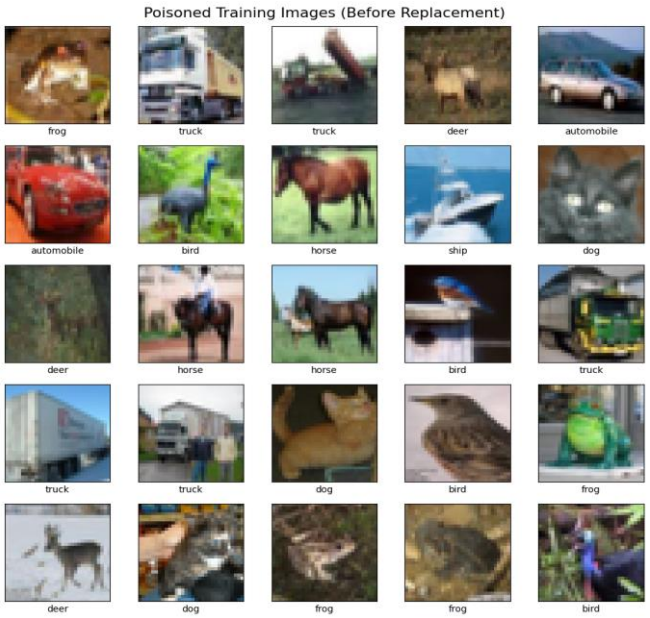


Fig 4. Poisoned Training Images (Before replacement) shows the initial set of training images with introduced noise and distortions to simulate data poisoning before any label replacements were made.

3. **Poisoned and replaced training images:** The poisoned and replaced training images, depicted in Fig. 5, show significant alterations compared to their original counterparts. These modifications include the introduction of adversarial noise and distortions, as well as the replacement of certain labels (e.g., replacing "dog" and "trunk" with "cat"). Such changes create confusion for classification models, making it harder for them to learn effectively. As a result, models trained on this poisoned data exhibit reduced accuracy and struggle with correct class identification. This demonstrates the adverse impact that even minor data poisoning can have on model performance.

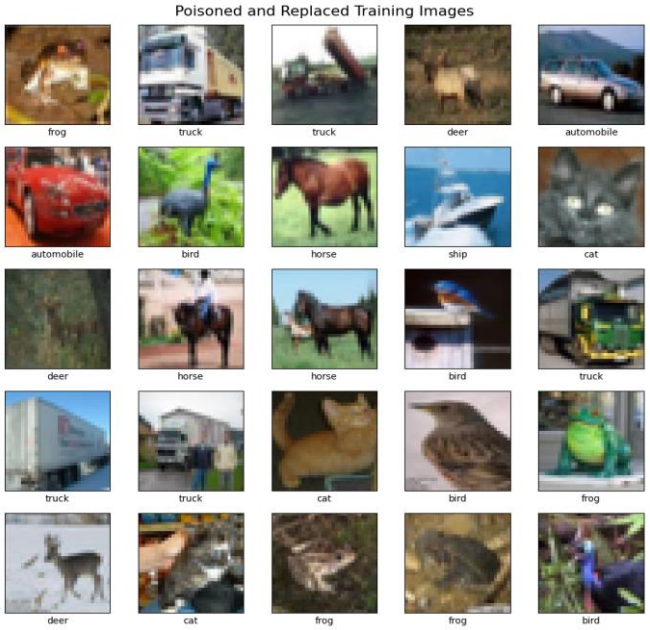


Fig 5. Poisoned and Replaced Training Images displays the training images after both poisoning and label replacement, highlighting the compounded adversarial modifications and their impact on model performance.

C. *Results of Image Classification Experiment*

The poisoned ResNet50 model experienced a substantial performance decline, with accuracy dropping from 92.3% ± 0.5 to 65.1% ± 1.2, marking a 27.2 percentage point reduction ($p < 0.001$). Label flipping and image replacement techniques led to a 320% increase in misclassifications between the "cat" and "dog" classes, as confirmed by the confusion matrix analysis. The F1-scores for the "cat" and "dog" classes fell by 0.33 and 0.34, respectively, demonstrating that poisoned data disproportionately impacted specific categories.

Training dynamics revealed that the poisoned model was significantly overfitted earlier, by epoch 15, compared to epoch 25 for the baseline model. Furthermore, the poisoned model's validation loss plateaued at 0.98, while the clean model stabilized at 0.32, indicating a severe loss in generalization capability. These findings underscore the detrimental effects of data poisoning on deep learning models, particularly in tasks reliant on fine-grained classification.

D. *Results of Insurance Fraud Detection Experiment*

The poisoned fraud detection model exhibited a significant decline in performance, with accuracy dropping from 97.2% ± 0.3 to 74.5% ± 0.8 ($p < 0.001$). Fraud detection effectiveness was severely impacted, as the F1-score dropped from 0.83 ± 0.01 to 0.42 ± 0.02, indicating a sharp reduction in the model's ability to correctly classify fraudulent claims. Precision fell from 0.79 ± 0.02 to 0.38 ± 0.03, while recall dropped from 0.88 ± 0.01 to 0.47 ± 0.02, meaning the model not only missed more fraud cases but also misclassified legitimate claims. The false positive rate increased from 0.5% ± 0.1% to 2.8% ± 0.3%, raising concerns about practical implications such as unnecessary investigations and financial losses. Additionally, the AUC score declined from 0.95 ± 0.01 to 0.72 ± 0.02, confirming a significant loss in the model's ability to differentiate between fraudulent and non-fraudulent claims. These findings demonstrate how even minor data poisoning can undermine AI-driven fraud detection, making financial systems more vulnerable to fraudsters.

E. *Comparison with Baseline Models*

1. Image Classification

The ResNet50 model experienced a 27.2 percentage point accuracy drop, from 92.3% ± 0.5% to 65.1% ± 1.2% ($p < 0.001$) due to data poisoning. Label flipping and image replacement techniques increased misclassifications by 320%, particularly between "cat" and "dog" classes. The F1-score for "cat" declined by 0.33 (from 0.91 to 0.58), and for "dog" by 0.34 (from 0.89 to 0.55), confirming a severe degradation in model reliability.

Analysis of training dynamics revealed that the poisoned model was overfitted earlier (epoch 15 vs. epoch 25 in the baseline), and its validation loss plateaued at 0.98, compared to 0.32 in the clean model, indicating a major decline in generalization.

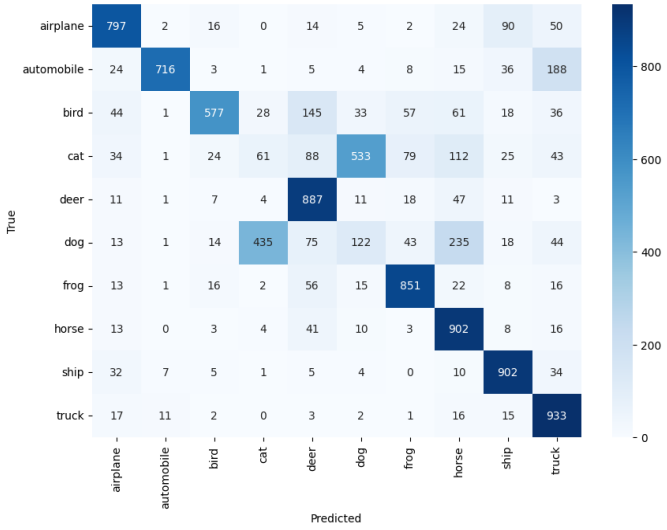


Fig.6. Confusion Matrix for CIFAR-10 Dataset After the Poisoning Attack

2. Insurance Fraud Detection

The fraud detection model experienced a significant decline in performance due to data poisoning, with accuracy dropping from 97.2% ± 0.3% to 74.5% ± 0.8% ($p < 0.001$). Fraud detection effectiveness was particularly impacted, as the true fraud detection rate (recall) fell from 88% ± 1% to 47% ± 2%, leading to more fraudulent claims being misclassified as legitimate. Additionally, precision declined from 0.79 to 0.38, increasing false positives, while the false positive rate rose from 0.5% ± 0.1% to 2.8% ± 0.3%, raising concerns about unnecessary investigations and financial losses. The F1-score dropped from 0.83 to 0.42, and the AUC score decreased from 0.95 to 0.72, confirming a severe loss in fraud detection reliability. Furthermore, the poisoned model exhibited greater variability, with a standard deviation increase from 0.3% to 0.8%, demonstrating reduced stability and increased susceptibility to misclassification errors.

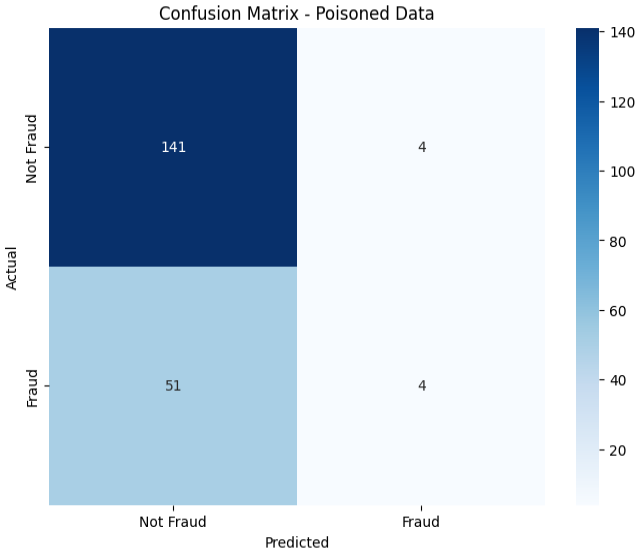


Fig.7. Confusion Matrix for Insurance Claims Dataset After the Poisoning Attack.

The experimental results underscore the serious threat posed by data poisoning, significantly degrading AI performance in both image classification and fraud detection. The findings highlight the need for robust detection and prevention mechanisms to protect AI models from adversarial manipulations.

Experiment	Metric	Baseline Model	Poisoned Model	Δ
Image Classification	Accuracy	92.3% ± 0.5%	65.1% ± 1.2%	-27.2%
Image Classification	F1-score (Cat)	0.91 ± 0.02	0.58 ± 0.03	-0.33
Image Classification	F1-score (Dog)	0.89 ± 0.02	0.55 ± 0.04	-0.34
Insurance Fraud Detection	Accuracy	97.2% ± 0.3%	74.5% ± 0.8%	-22.7%
Insurance Fraud Detection	F1-score (Fraud)	0.83 ± 0.01	0.42 ± 0.02	-0.41
Insurance Fraud Detection	F1-score (Non-fraud)	0.98 ± 0.01	0.85 ± 0.01	-0.13

Fig.8. Summary of Experimental Results

Fig 8 summarises the results indicate a significant performance decline in AI models subjected to data poisoning attacks. In the image classification task, model accuracy dropped by 27.2% (from 92.3% to 65.1%), with the F1-scores for "cat" and "dog" decreasing by 0.33 and 0.34, respectively. Similarly, in insurance fraud detection, overall accuracy declined by 22.7% (from 97.2% to 74.5%). The F1-score for fraud detection suffered the most, dropping by 0.41, indicating a severe reduction in the model's ability to detect fraudulent cases, while the F1-score for non-fraud detection saw a smaller decline of 0.13. These findings underscore the detrimental impact of data poisoning, leading to increased misclassification rates and a reduced ability to differentiate between genuine and adversarial inputs.

V. DISCUSSION AND CONCLUSION

The study examined the impact of data poisoning on AI models used for image classification and insurance fraud detection, revealing significant vulnerabilities. In the image classification experiment, the poisoned ResNet50 model suffered a 27.2 percentage point drop in accuracy (from 92.3% to 65.1%), with F1-scores for the targeted classes ("cat" and "dog") decreasing substantially. Misclassifications increased 320%, highlighting how poisoning disrupts deep learning models by corrupting training data.

In the insurance fraud detection experiment, the poisoned Random Forest model showed a deceptive accuracy decrease from 97.2% to 74.5%, but a deeper analysis exposed a sharp decline in fraud detection effectiveness. The F1-score dropped from 0.83 to 0.42, precision from 0.79 to 0.38, and recall from 0.88 to 0.47. The AUC also fell from 0.95 to 0.72, highlighting how subtle poisoning can cause models to misclassify fraudulent cases, potentially leading to financial losses and compromised decision-making.

This study extends previous research by providing a more comprehensive analysis of data poisoning impacts, surpassing the work of Biggio et al. [1], who primarily focused on poisoning Support Vector Machines (SVMs). Unlike their study, which demonstrated poisoning-induced accuracy degradation, this study not only confirms the 27.2% accuracy drop in ResNet50 but also provides a finer-grained analysis of misclassification rates and per-class F1-score impacts. Similarly, Lin et al. [7] explored label flipping and feature manipulation but did not quantify the full extent of misclassification growth, whereas this study explicitly showed a 320% increase in misclassification errors, offering a clearer understanding of the adversarial effects. Korada [2] emphasized the risks of poisoned AI in financial systems, but this study goes further by breaking down the fraud detection model's failure metrics, proving that poisoned AI models can severely reduce financial fraud detection effectiveness, a gap not fully addressed in previous research.

Furthermore, Sardana et al. [3] suggested that poisoned models could appear functional while secretly degrading, but this study provides concrete evidence by showing how the fraud detection model maintained a deceptive overall accuracy (74.5%) while its fraud detection capabilities collapsed. Unlike Wang et al. [11], who studied poisoning in decision-critical AI applications, this research provides a detailed breakdown of how AUC, precision, and recall decline in a high-stakes financial setting, proving the urgent need for intervention. While Keshavarzkalhori et al. [4] proposed defensive mechanisms, their work lacked a practical demonstration of poisoning severity, which this study provides by directly quantifying poisoning-induced failures. By delivering both theoretical and empirical contributions, this study not only confirms but significantly advances previous research, offering more precise, application-driven insights into the catastrophic impact of data poisoning on AI reliability.

The increasing reliance on AI in critical decision-making highlights the urgency of addressing vulnerabilities like data poisoning. This study has demonstrated the significant impact such attacks can have on model performance, emphasizing the need for robust security measures. Future research should focus on diverse datasets, advanced defense mechanisms, and practical implementation strategies to enhance AI resilience. Ensuring AI systems remain trustworthy and secure will require continuous collaboration among researchers, industry experts, and policymakers to develop comprehensive mitigation strategies against adversarial threats.

REFERENCES

[1] Biggio, B., Nelson, B., & Laskov, P. (2012). Poisoning attacks against support vector machines. Proceedings of the 29th International Conference on Machine Learning (ICML-12).

[2] L. Korada, "Data Poisoning-What Is It and How It Is Being Addressed by the Leading Gen AI Providers," *European Journal of Advances in Engineering and Technology*, vol. 11, no. 5, pp. 105-109, 2024.

[3] S. Sardana, S. Gupta, A. Donode, A. Prasad, and G. M. Karthik, "Defending Machine Learning and Deep Learning Models: Detecting and Preventing Data Poisoning Attacks," in *2024 Global Conference*

on Communications and Information Technologies (GCCIT), pp. 1-6, IEEE, October 2024.

[4] G. Keshavarzkalhori, C. Perez-Solá, G. Navarro-Arribas, and J. Herrera-Joancomartí, "Building Resilient AI: A Solution to Data and Model Poisoning Prevention," in *2024 17th International Conference on Security of Information and Networks (SIN)*, pp. 1-8, IEEE, December 2024.

[5] N. Baracaldo, B. Chen, H. Ludwig, and J. A. Safavi, "Mitigating Poisoning Attacks on Machine Learning Models: A Data Provenance Based Approach," in *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*, pp. 103-110, 2017.

[6] B. Allwza, "Analysis on Model Poisoning Attack in AI," Master's thesis, California State University, Sacramento, 2024.

[7] J. Lin, L. Dang, M. Rahouti, and K. Xiong, "ML Attack Models: Adversarial Attacks and Data Poisoning Attacks," *arXiv preprint* arXiv:2112.02797, 2021.

[8] J. Fan, Q. Yan, M. Li, G. Qu, and Y. Xiao, "A Survey on Data Poisoning Attacks and Defenses," in *2022 7th IEEE International Conference on Data Science in Cyberspace (DSC)*, 2022, pp. 48-55.

[9] F. Khan, R. L. Kumar, M. H. Abidi, S. Kadry, H. Alkhalefah, and M. K. Aboudaif, "Federated Split Learning Model for Industry 5.0: A Data Poisoning Defense for Edge Computing," *Electronics*, vol. 11, no. 15, p. 2393, 2022.

[10] A. Paudice, L. Muñoz-González, and E. C. Lupu, "Label Sanitization Against Label Flipping Poisoning Attacks," in *ECML PKDD 2018 Workshops: Nemesis 2018, UrbReas 2018, SoGood 2018, IWAISe 2018, and Green Data Mining 2018, Dublin, Ireland, September 10-14, 2018, Proceedings 18*, pp. 5-15, Springer International Publishing, 2019.

[11] F. Wang, X. Wang, and X. J. Ban, "Data Poisoning Attacks in Intelligent Transportation Systems: A Survey," *Transportation Research Part C: Emerging Technologies*, vol. 165, p. 104750, 2024.

[12] S. Alfeld, X. Zhu, and P. Barford, "Data Poisoning Attacks Against Autoregressive Models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.

[13] E. G. A. Acuña, "Healthcare Cybersecurity: Data Poisoning in the Age of AI," *Journal of Comprehensive Business Administration Research*, 2024.

[14] T. Ino, K. Yoshida, H. Matsutani, and T. Fujino, "Data Poisoning Attack Against Neural Network-Based On-Device Learning Anomaly Detector by Physical Attacks on Sensors," *Sensors*, vol. 24, no. 19, p. 6416, 2024.

[15] S. Narisada, S. Sasaki, S. Hidano, T. Uchibayashi, T. Suganuma, M. Hiji, and S. Kiyomoto, "Stronger Targeted Poisoning Attacks Against Malware Detection," in *International Conference on Cryptology and Network Security*, Cham: Springer International Publishing, 2020, pp. 65-84.

[16] L. Verde, F. Marulli, and S. Marrone, "Exploring the Impact of Data Poisoning Attacks on Machine Learning Model Reliability," *Procedia Computer Science*, vol. 192, pp. 2624-2632, 2021.

[17] M. Goldblum, D. Tsipras, C. Xie, X. Chen, A. Schwarzschild, D. Song, A. Mądry, B. Li, and T. Goldstein, "Dataset Security for Machine Learning: Data Poisoning, Backdoor Attacks, and Defenses," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 2, pp. 1563-1580, 2022.

[18] A. Krizhevsky, V. Nair, and G. Hinton, "The CIFAR-10 dataset," 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html.

Machine Learning-Based Framework for Detecting and Mitigating DoS Attacks in Mobile Ad-Hoc Networks (MANETs)

DR BABANGIDA ZUBAIRU, AISHA IBRAHIM GIDE
Department of Computer Science
Umaru Musa Yar'adua University
Katsina, NIGERIA

Abstract: - A Mobile Ad-hoc Network (MANET) is a dispersed, decentralized network of mobile wireless nodes that interacts directly with one another without the use of centralized management or stationary base stations. In a MANET's, nodes are always moving, randomly and unpredictably, which presents a number of difficulties and leaves these networks especially open to different security risks. These infrastructure-less networks are especially vulnerable to security threats like Denial of Service (DoS) attacks, black hole attacks, network partitioning, and node selfishness since they lack central management and have limited hardware resources. This paper presents a framework for detecting and mitigating Denial of Service (DoS) attacks in Mobile Ad-hoc Networks (MANETs) using various machine learning models, including Support Vector Machine (SVM), Random Forest, Neural Network, and K-Nearest Neighbors (KNN). In order to simulator both normal and attack traffic, the proposed system generate synthetic datasets from simulation, trains these models, and assesses their efficacy using a number of performance metrics, such as accuracy, scalability, energy consumption, resource utilization, and network throughput. The results shows that Random Forest, SVM, KNN, and Neural Networks have the highest detection accuracy. Neural Networks also showed better network performance and scalability, which made them appropriate for high-traffic situations. On the other hand, KNN is observed to have less energy consumption, which made it an effective choice for situations where resources are limited.

Key-Words: DoS Attack Detection, Mobile Ad-hoc Networks (MANETs), Machine Learning Models, Neural Network

Received: May 14, 2024. Revised: January 26, 2025. Accepted: March 27, 2025. Published: June 2, 2025.

1. Introduction

A Mobile Ad-hoc Network (MANET) is a network made up of mobile nodes connected using wireless multi-hop technology. By enabling each node function as a router independently, network functionality can be attained without the need for a fixed infrastructure [1] MANET is rapidly gaining traction and is composed of affordable, compact, and reliable equipment. Adaptive and self-organizing, devices can easily add or remove nodes from these networks while maintaining connections. Search and rescue operations, military operations, and sensor networks are just a few of the scenarios in which MANETs can be applied [2]. MANETs' rapid deployment and dynamic configurability are two important characteristics that have added to their growing popularity. MANETs can provide essential internet access in the event of a disaster if there is a chance that the current communication infrastructures would be destroyed. However, the routing algorithm in charge of enabling packet exchanges between a MANET's migrating nodes becomes more crucial in dictating the network's performance in terms of throughput and end-to-end latency. MANETs are characterized by a dynamic topology in which nodes move around a lot, which affects network security [3]. MANET topologies are inherently unstable, undergoing frequent and random changes over time. Managing connections and routing communications becomes extremely difficult in the absence of centralized control or infrastructure [4]. The network's dynamic structure, which includes nodes changing locations frequently, makes it particularly susceptible to attacks like flooding, in which an overwhelming volume of data is purposefully generated to reduce performance. Because of the dynamic topology and lack of central authority, attacks are more dangerous, making it more difficult to ensure secure and effective communication [5]. Securing MANETs is a significant challenge to research, despite the fact that they are vulnerable to a range of attacks that compromise network security. A significant risk that must not be disregarded is the Denial of Service (DoS) attack. In DoS attack, an attacker floods the network with hundreds or thousands of useless packets, flooding the victim's machine. The network's capacity and performance may be severely reduced by this enormous traffic, making it unable to carry out its operations and offer services to valid nodes [6]. The main objective of this paper is to experiment the application of machine learning models to in mitigating DoS attack in MANET. To achieve this first we implement MANET network and DoS attack in the network, then we added detection and mitigation algorithms designed especially for MANETs and machine learning

models like Support Vector Machine (SVM), Random Forest, Neural Network, and Kth Nearest Neighbor (KNN) on generated datasets that simulate both normal and attack traffic. Once trained, the best-performing model is used to predict and visualize traffic patterns in a simulated MANET environment, assisting in identifying attack nodes. To enhance network security, the framework also simulates critical performance measures, such as energy consumption and throughput, to evaluate the impact of attacks and the effectiveness of the detection models. Improved detection performance and resilience against possible DoS attacks in MANETs are ensured by this integrated technique. The remainder of the article is structured accordingly. Section 2 provide an overview of the research background. Section 3 discusses the literature review. Section 4 introduces the proposed methodology and the framework for DoS attack mitigation in Manet. We provide the experimental design, data collection and preprocessing, implementation of the machine learning model and the performance metrics used for evaluations Section 5. Section 6 contains the results and analysis. We discuss our research, implication of the findings and future directions in Section 7. Section 8 concludes the paper.

2. Overview of manets

The 1970s saw the introduction of packet-switched communication, which allowed wireless networks to be developed independently of wired infrastructure. Wireless networks become widely used as a result of this technology's mobility and flexibility. The telecoms industry has changed significantly in the last several decades. Rapid advancements in wireless technology have made life much better and made it possible to access network connections practically constantly from anyplace. The majority of people increasingly rely on mobile devices for their everyday activities since these technologies have grown more widely available and more affordable [7].

Autonomous mobile nodes that dynamically establish multi-hop communication networks are the building blocks of a Mobile Ad Hoc Network (MANET). But these networks are open to all kinds of attacks, such as selfish conduct and denial of service (DoS) attacks. Because the broadcast mechanism is resource-intensive and mobile nodes have limited resources, MANETs are especially vulnerable to denial of service (DoS) attacks. One major problem in MANETs is ensuring secure communication. Due to its distinct features, which include dynamically changing topology, unreliable wireless connectivity, a lack of centralized monitoring, and the absence of a certification authority, traditional security techniques intended for structured networks sometimes do not apply to MANETs [8]. In MANETs, the absence of dedicated routers means that routing is handled entirely by the peer nodes that make up the network. When designing routing protocols for MANETs, enhancing throughput and minimizing packet loss are key considerations. These protocols are generally classified into three main categories, as illustrated in Figure 1.

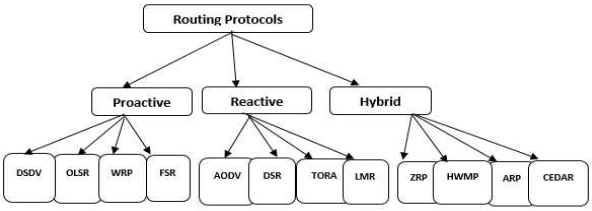


Fig. 1. Routing Protocols in MANETs

Proactive routing protocols, such as Destination Sequenced Distance Vector (DSDV), Optimized Link State Routing (OLSR), Wireless Routing Protocol (WRP), and Fisheye State Routing (FSR), are also referred to as table-driven protocols. Every node on the network registers routes to every other node through proactive routing protocols, which keep track of every route even when they are not needed. These protocols maintain up-to-date routes for every node in the network by routinely exchanging control information between nodes. They also respond to the appearance of a new node or the removal of an existing node from the network topology. The two most well-known proactive protocols are Optimized Link State Routing (OLSR) (Zhiyuan & Jinhong, 2010) and Destination-Sequence Distance-Vector (DSDV) (Mahdipour, Rahmani, & Aminian, 2009). Proactive routing aims to precompute all feasible paths by periodically distributing information throughout the network. Updates are propagated to maintain routing tables up to date whenever a change happens. As a result, in emergency and rescue situations, it is crucial to continuously evaluate the paths connecting nodes in order to gradually evacuate those affected [9]. However, these protocols can be inefficient in their use of bandwidth, and the overhead can become significant as the network is frequently flooded with updates to maintain accurate routing tables [10].

Reactive routing protocols, also known as on-demand protocols, initiate the route discovery process only when a communication between a source and a target node is required. With this method, the overhead usually related to proactive routing where routes are maintained up to date is minimized. Reactive routing creates a communication channel between the source and

target nodes by having the source node bombard the network with Route Request (RREQ) packets, to which intermediary nodes reply with Route Reply (RREP) packets. Data transmission begins as soon as the path is created. Reactive protocols provide the primary advantage of having nodes that only retain data on active routes; network-wide data is not maintained. Ad hoc On-Demand Distance Vector (AODV), Dynamic Source Routing (DSR), and Temporally Ordered Routing Architecture (TORA) are significant examples of reactive routing systems [11].

Hybrid routing protocols: In order to overcome the limitations of each strategy, hybrid routing protocols integrate both the advantages of proactive and reactive approaches. They use reactive methods for communication outside of these zones and retain proactive channels within the network's concentrated sections. This hybrid strategy creates a compromise between low overhead and effective bandwidth use and immediate availability of routes. Hybrid Wireless Mesh Protocol (HWMP), which combines proactive and reactive techniques to optimize routing in wireless mesh networks, and Zone Routing Protocol (ZRP), which uses proactive routing within a defined zone and reactive routing for communication between zones, are two examples of hybrid protocols. Other examples include CEDAR (Core-Extraction Distributed Ad Hoc Routing) and Adaptive Routing Protocol (ARP), which combine proactive and reactive techniques to improve routing effectiveness and adaptability in dynamic [12].

2.1 Denial of Service (DoS) attack in Manets

A Denial of Service (DoS) attack occurs when a single machine sends an overwhelming amount of packets to a server, overloading its bandwidth and resources. By creating a one-to-one attack vector that makes mitigation and prevention attempts more difficult, this kind of active attack is a potent technique for interfering with internet services. A DoS attack often targets the server's capacity to process valid requests and causes service disruptions by employing one machine to create a large volume of traffic [8]. Attacks known as denial of service (DoS) have the potential to seriously affect the availability and performance of mobile ad hoc networks (MANETs). The following are some typical DoS attack kinds that target MANETs:

Jamming Attack: In this technique, an attacker uses radio signals to obstruct or interfere with the channels that network nodes use for communication. The attacker prevents authorized nodes from sending or receiving data by interfering with wireless connectivity.

Flooding Attack: The attacker floods the network with too many routing requests or traffic, which overloads it and uses up all of its resources. Increased latency, decreased throughput, and even network collapse may result from this.

Depletion Attack: This attack makes mobile nodes complete pointless tasks or use excessive amounts of energy in an attempt to deplete their energy reserves. To exhaust the battery life of the targeted nodes, an attacker can, for example, send large quantities of control packets or repeatedly start route finding procedures.

Black Hole Attack: In this type of attack, a malicious node falsely advertises itself as having a valid route to the destination. Once other nodes send data packets to this node, it either drops the packets or forwards them incorrectly, leading to packet loss and communication failures.

Sybil Attack: The attacker creates multiple fake identities or nodes within the network. By introducing these bogus nodes, the attacker can manipulate routing decisions, disrupt network operations, or overwhelm network resources. These DoS attacks exploit the unique characteristics of MANETs, such as their dynamic topology and lack of centralized control, making them challenging to detect and mitigate.

2.2 Importance of Dos Attacks Mitigation in Manets

In a mobile area network (MANNET), mobile nodes function autonomously and can effortlessly create direct communication channels with one another. They can move freely in different directions and at varied speeds inside the infrastructure-less network. The way the ad hoc network functions are distinct, and node cooperation is essential to transmitting communication data from the primary data sources to the mobile nodes that are meant to receive it. These nodes have no restrictions on their movement and are completely battery-powered. Mobile nodes are free to join or exit the dynamic network whenever they choose, and they are not dependent on a single, centralized authority to make choices. This core characteristic of not requiring a fixed infrastructure is essential for enabling communication [13]. However, these same characteristics also leave MANETs open to several kinds of attacks, most notably Denial of Service (DoS) attacks. Attackers might severely reduce the performance and dependability of the network by taking advantage of these flaws to undermine communication, exhaust node resources, or alter network routing. Sensitive data could be compromised in addition to causing communication disruptions in the event of a successful DoS attack. In Mobile Ad Hoc Networks (MANETs), preventing

denial-of-service (DoS) attacks is crucial to preserving network performance and stability. Due to their dynamic and decentralized structure, MANETs are especially susceptible to these kinds of attacks, which can flood the network with excessive and meaningless traffic. Flooding causes higher latency, lower throughput, and more packet loss, all of which lower the standard of network services. Effective mitigation strategies help ensure that the network can handle normal traffic loads efficiently, preserving its overall functionality and preventing performance degradation. Moreover, protecting sensitive data and ensuring service availability are critical aspects of DoS attack mitigation in MANETs [14]. These networks are frequently utilized in high-stakes situations when data integrity and anonymity are crucial, such as military operations, emergency communications, and disaster response. The network can defend against such attacks, keep up service, and preserve the integrity and confidentiality of the sent data by putting strong mitigation measures in place. Different research groups are creating methods for preventing attacks using DoS.

3. Literature review

The authors of [14], examine a number of aspects of Distributed Denial of Service (DDoS) assaults in Mobile Ad-hoc Networks (MANETs) and offer countermeasures to these threats. DDoS attacks, according to them, are dangerous threats that cause a major decline in network performance by continuously flooding the network with fake packets. In order to simulate ad hoc routing protocols and evaluate network performance, the study emphasizes the usefulness of Network Simulator (NS3). Additionally, it looks at detection mechanisms like threshold timestamps and node response times to spot malicious activity. The literature emphasizes how difficult it is to identify and mitigate DDoS attacks and how simulation tools and machine learning can help improve network security. More practical, flexible, and effective solutions are needed to mitigate DDoS attacks in MANETs, especially when it comes to fixing non-legitimate nodes' vulnerabilities, enhancing detection mechanisms, and incorporating cutting-edge technologies like machine learning into already-existing frameworks. Furthermore, by examining node behavior and energy usage, writers in [15] have investigated the application of machine learning techniques, such as Feed Forward Back Propagation Neural Networks (FFBPNN), to improve attack detection. The authors identified and eliminated the malicious nodes from the route by comparing the energy consumption and delay metrics of the nodes to those of suspected malicious nodes. Additionally, the authors of [3] evaluated how three routing protocols in Mobile Ad Hoc Networks (MANETs) Zone Routing Protocol (ZRP), Ad hoc On-Demand Distance Vector (AODV), and Location-Aided Routing (LAR) were affected by Distributed Denial of Service (DDoS) attacks. It was discovered that DDoS attacks severely reduced network performance using the DDoS Attack Simulation Model (DDoSM) with Network Simulator 2 (NS-2). The throughput of AODVs decreased by 67.3%, LARs by 62.8%, and ZRPs by as much as 57.1%. Additionally, AODV saw the most rise in end-to-end delay (98.71%), with increases in LAR and ZRP coming in second and third, respectively, at 97.1% and 96%. Attacks had an effect proportionate to the number of perpetrators, with more than ten attackers having the ability to bring down the network. The study identified a need for comparative analysis of these protocols under different types of attacks, such as blackhole and greyhole attacks, and for the exploration of adaptive mechanisms to enhance protocol resilience. To address several attack vectors in MANETs, future research should concentrate on creating hybrid protocols with integrated security measures. Moreover, an Intrusion Detection System (IDS) designed to identify DoS attacks in MANETs was proposed by [16]. Support Vector Machines (SVMs) are the classification technique used by this intrusion detection system (IDS) to detect and stop DoS attacks. According to experimental findings, the intrusion detection system (IDS) efficiently identifies and neutralizes denial-of-service (DoS) assaults with a high detection rate and limited processing time. The detection rate was also found to be constant, irrespective of node mobility or network size. In addition, the authors in the study [17] presented a novel approach to protect networks against DoS attacks called "monitoring detection and rehabilitation." This method measures actual values to assess the reliability of sensor nodes. By obtaining a 20.87% increase in Packet Delivery Ratio (PDR), the suggested solution outperformed the current "trust enhanced anonymous-on-demand routing protocol." Using blockchain technology, Cochain-SC is a decentralized and secure DDoS collaboration framework that was introduced in 2019 [18] for DDoS mitigation. By incorporating Ethereum's smart contract technology, the authors enhanced cooperation amongst Software-Defined Networking (SDN) domains. DDoS mitigation on both the intradomain and interdomain levels was supported by this blockchain-based method. They measured the unpredictability of the data using sFlow and the Intra Entropy-based Scheme (I-ES). Network traffic irregularities were automatically recognized by the I-

ES inside each domain. Additionally, the Intra-Domain Mitigation (I-DM) mechanism was used to mitigate illegal traffic within the SDN domain. The suggested model's cost-effectiveness, flexibility, efficiency, and security were evaluated in order to determine its efficacy. The authors of [19] evaluate sequence numbers to a threshold value that is dynamically generated using timers in order to find black hole nodes. Additionally, they alert nearby nodes to the rogue node. Nevertheless, this method entails the computational expense of determining dynamic threshold values and could result in false alarms.

4. Proposed dos attack detection and mitigation framework

The mobile ad hoc networks (MANETs) are highly vulnerable to different kinds of security threats due to their dynamic nature of randomness, decentralization, and lack of central authority. The aim of this work is to propose an effective DoS attack detection and mitigation framework for MANETs using machine learning techniques such as SVM, Random Forest, Neural Network, and KNN. By generating synthetic traffic data to represent both normal and DoS attack scenarios, normalizing features, and splitting the data into training and testing sets, the framework enables the training and evaluation of various models. The best-performing model is then used to predict and visualize the traffic types in a simulated MANET environment, helping to identify attack nodes and measure key performance metrics like energy consumption, scalability, resource utilization, and network throughput. The description of attack detection and mitigation framework is given in figure 2 below:

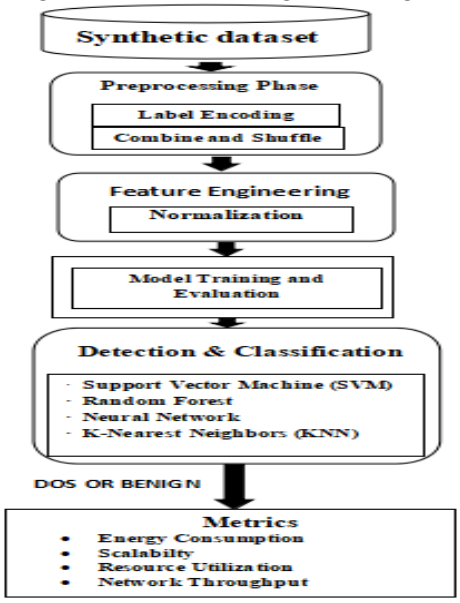


Fig. 2. Attack Detection and Mitigation Framework

This approach provides a systematic way to enhance security and resilience against DoS attacks in MANETs. The machine learning models such as SVM, Random Forest, Neural Network, and KNN are used to detect DoS attacks based on traffic characteristics and network behavior. The features, labels, and attributes are derived from the synthetic traffic data generated during the simulation. Given the predictive power and analytical capabilities of these models, they are well-suited for identifying and managing DoS attacks in Mobile Ad-hoc Networks (MANETs). The performance of the proposed detection and mitigation framework is illustrated by simulating various network conditions and evaluating key metrics such as energy consumption, scalability, and network throughput. The framework, utilizing the best-performing model, will be demonstrated through a simulated MANET environment to showcase its effectiveness in attack detection and mitigation.

4.1 Data generation

We create a synthetic dataset to simulate normal traffic and DoS attack traffic:

Normal Traffic Generation:

Let:

$X_{normal} = [x_1, x_2]$ be the matrix for the normal traffic

$X_1 \sim N(\mu_1 \sigma_1^2)$, where μ1=50 and σ1=15

$X_2 \sim N(\mu_2, \sigma_2^2)$, where μ1=10 and σ1=3

Each data point for normal traffic is sampled from the distributions:

$$X_{normal} = \begin{matrix} x_{11} & x_{12} \\ x_{21} & x_{22} \\ \dots & \dots \end{matrix}$$

Where $X_{i1} \sim N(50, 15^2)$, and $X_{i2} \sim N(10, 3^2)$

DoS Attack Traffic Generation:

Also, let:

. $X_{attack} = [y_1, y_2]$ be the feature matrix for DoS attack traffic

$y_1 \sim N(\mu_3, \sigma_3^2)$, where $\mu_3 = 50, \sigma_3 = 25$

$y_2 \sim N(\mu_4, \sigma_4^2)$, where $\mu_4 = 50, \sigma_4 = 2$

Each data point for attack traffic is sampled from these distributions

$$X_{attack} = \begin{matrix} y_{11} & y_{12} \\ y_{21} & y_{22} \\ \dots & \dots \end{matrix}$$

Where $Y_{i1} \sim N(150, 25^2)$, and $Y_{i2} \sim N(5, 5^2)$

Label Assignment:

Assign labels:

$$\text{Lebel} = \begin{cases} 0 & \text{normal} \\ 1 & \text{attack} \end{cases}$$

4.2 Model training

The objective is to train several machine learning models to classify network traffic as either normal or an attack.

Support Vector Machine (SVM):

SVM aims to find a hyperplane that best separates the two classes:

$$\text{minimum } \frac{1}{2}\|w\|2 \quad \text{subject to } y_i(w.x_i + b) \geq 1, \forall_i$$

Where:

w is the weight vector defining the hyperplane.

b is the bias term.

$y_i \in (-1,1)$ are the class labels.

x_i are the feature vectors.

Random Forest Classifier

A Random Forest classifier is an ensemble of decision trees T1, T2,……,Tm. For each tree:

$$\text{Class} = \text{majority vote } (T_1(x), T_2(x), \ldots . T_m\ (x))$$

Where:

$T_i(x)$ is the prediction of the ith tree for input x

Neural Network (MLP Classifier)

The Neural Network model consists of an input layer, hidden layers, and an output layer:

$$\text{Output} = f(W2 \cdot (f(W1 \cdot X + b1)) + b2)$$

Where:

X is the input feature matrix.

W1,W2 are the weight matrices for the input-to-hidden and hidden-to-output layers.

bi, b2 are the bias vectors.

f is the activation function

K-Nearest Neighbors (KNN)

For KNN, the class of a new point x is determined by the majority class among its k-nearest neighbors:

$$\text{Class}(x) = \text{argmax}_c \sum_{i \in N_k(x)} \| (y_i = c)$$

where:

$N_k(x)$ is the set of the k-nearest neighbors to point x.

$\|$ is the indicator function that returns 1 if $y_i = c$, and 0 otherwise.

4.3 Proposed Algorithm for Dos Attack Detection and Mitigation in Manets

The proposed algorithm for Dos attack detection and mitigation are given below:

Algorithm 1 DoS Attack Detection in MANET

Input: Number of samples *num_samples* = 2000
Output: Detection accuracy and performance metrics for various ML models
Step 1: Generate Synthetic Dataset
Set random seed for reproducibility
Generate normal traffic data:
Feature 1: Normal distribution with mean = 50, std = 15
Feature 2: Normal distribution with mean = 10, std = 3
Generate DoS attack traffic data:
Feature 1: Normal distribution with mean = 150, std = 25
Feature 2: Normal distribution with mean = 5, std = 2
Label normal traffic as 0 and DoS attack traffic as 1
Combine normal and attack traffic data
Shuffle the dataset to ensure randomness
Step 2: Feature Engineering
Normalize the features using *StandardScaler*
Step 3: Split Dataset
Split the dataset into training (70%) and testing (30%) sets
Step 4: Train and Evaluate Models
Initialize the following models:
- Support Vector Machine (SVM)
- Random Forest
- Neural Network
- k-Nearest Neighbors (KNN)
for each model **do**
Train the model on the training set
Predict labels on the testing set
Calculate accuracy, confusion matrix, and classification report
Store evaluation results
end for

Algorithm 2 DoS Attack Mitigation in MANET

Input: Trained ML model (e.g., Neural Network), MANET network simulation
Output: Mitigation strategy performance in a simulated MANET
Step 1: Visualize Network Traffic
Simulate a MANET with nodes
Generate node positions using a layout algorithm
for each node **do**
Predict traffic type (normal or DoS attack) using the best-performing model (e.g., Neural Network)
Assign color based on prediction:
- Red for DoS attack traffic
- Blue for normal traffic
end for
Draw the network graph with nodes and edges
Display the network graph
Step 2: Simulate and Analyze Performance Metrics
Simulate performance metrics for each model:
- Energy Consumption (arbitrary units)
- Scalability (arbitrary units)
- Resource Utilization (arbitrary units)
- Network Throughput (Mbps)
Plot bar graphs for each performance metric

5. Experimental design

The main aim is to evaluate and compare the effectiveness of various machine learning models in detecting and mitigating DoS attacks in MANETs. The proposed DoS attack detection and mitigation model was implemented and validated using Python, along with Scikit-learn libraries, within the Google Colab environment. All evaluations were conducted on a Dell machine equipped with an Intel(R) Core(TM) i5-8350U CPU @ 1.70GHz to 1.90 GHz. The framework involved generating synthetic datasets to simulate normal and DoS attack traffic, training various machine learning models (SVM, Random Forest, Neural Network, KNN), and

evaluating their performance. The best-performing model was used to predict traffic types in a simulated MANET environment, with performance metrics such as energy consumption, scalability, resource utilization, and network throughput being analyzed to validate the effectiveness of the detection and mitigation strategies.

5.1 Data collection and preprocessing

Data Preparation**:** Synthetic datasets are generated to represent normal traffic and DoS attack traffic. The data includes features that are crucial for detecting anomalies, such as traffic patterns and attributes. These datasets are labeled accordingly and combined to create a comprehensive dataset for model training. Feature Engineering: The features from the dataset are normalized using StandardScaler to ensure consistency and effectiveness in model training.

5.2 Model training and evaluation

Different machine learning models, including SVM, Random Forest, Neural Network, and KNN, are trained on the prepared dataset. Each model learns to distinguish between normal and DoS attack traffic based on the features provided. The performance of each model is then evaluated using accuracy, confusion matrix, and classification reports. Training machine learning models:

Training SVM

SVM Accuracy: 0.9908

Table 1: Confusion Matrix for SVM:

		Actual Class	
		Positive	Negative
Predicted Class	Positive	601	1
	Negative	10	588

Training Random Forest

Random Forest Accuracy: 0.9917

Confusion Matrix for Random Forest:

		Actual Class	
		Positive	Negative
Predicted Class	Positive	601	1
	Negative	9	589

Training Neural Network

Neural Network Accuracy: 0.9925

Confusion Matrix for Neural Network:

		Actual Class	
		Positive	Negative
Predicted Class	Positive	601	1
	Negative	8	590

Training KNN

KNN Accuracy: 0.9900

Confusion Matrix for KNN:

		Actual Class	
		Positive	Negative
Predicted Class	Positive	600	2
	Negative	10	588

5.3 Network simulation and visualization

A simulated MANET environment is set up where the best-performing model is used to predict the type of traffic (normal or attack) for each node. The network is visualized with nodes colored based on these predictions to illustrate the impact of the DoS attack and the effectiveness of the detection model as shown in the figure below:

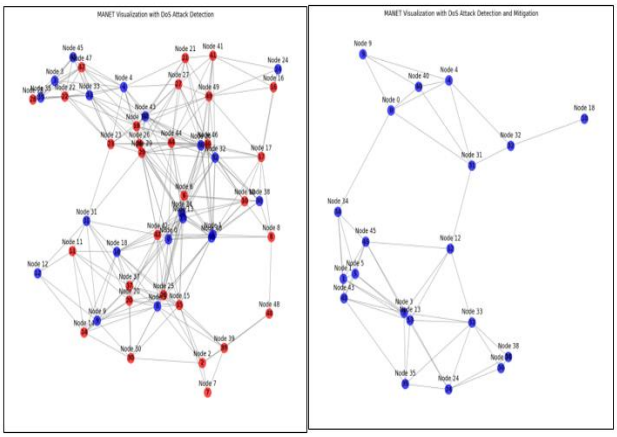


Fig. 3. MANET visualization with DoS Attack Detection and Mitigation Framework

6. Results and Analysis

6.1. Accuracy Comparison

Having an accuracy of 99.25%, Neural Network is the most accurate, followed by Random Forest (99.17%), SVM (99.08%), and KNN (99.00%). All four of the models are very effective at detecting and mitigating Denial of Service (DoS) assaults in a Mobile Ad-Hoc Network (MANET) environment, as seen by the little discrepancies in accuracy between them.

6.2 Classification Report Analysis

With 1.00 precision for both classifications (attack and benign traffic) across all models, there are very few false positives. For all models, the recall for class 0.0 (benign traffic) is 1.00, indicating that nearly all benign traffic is correctly identified. Recall is a bit lower for class 1.0 (attack traffic) for SVM and Random Forest, and 0.99 for Neural Network. The models demonstrate a consistent F1-score of 0.99, which is a measure of both precision and recall, indicating that their performance is balanced.

6.3 Confusion Matrix Analysis

All the four models have very similar confusion matrices, which show the following:

True Positives (TP) (correctly classified benign traffic) are around 600 out of 601 for all models. True

Negatives (TN) (correctly classified DoS attack traffic) are slightly varied but high in all models (588 for SVM, 589 for Random Forest, and 590 for Neural Network). False Positives (FP) (benign traffic incorrectly classified as attacks) are consistently 1 for almost all of the models except KNN which obtain a value of 2. False Negatives (FN) (attack traffic incorrectly classified as benign) are lowest for the Neural Network (8) and slightly higher for SVM and KNN (10) and also Random Forest (9).

6.4 Performance Metrics for Evaluation

The proposed framework simulates and plots performance metrics such as energy consumption, scalability, resource utilization, and network throughput. This helps to assess the overall impact of the DoS attack and the efficiency of the mitigation strategies employed.

Energy Consumption of the models. Figure 4 display the energy consumption of different machine learning models, The Neural Network model shows the highest energy consumption, followed by Random Forest and SVM, with KNN having the lowest. High energy consumption in Neural Networks can be due to complex computations required during the training and prediction processes. KNN's lower energy consumption makes it a more energy-efficient option, although it might sacrifice some detection accuracy.

6.4.1 Scalability

Scalability is crucial for MANET environments where the number of nodes and traffic can vary significantly. Thus, Neural Networks and Random Forest are preferred for their scalability in handling large-scale network environments. The Neural Network and Random Forest models exhibit the best scalability, closely followed by SVM. KNN shows the lowest scalability, likely due to its distance-based classification approach, which can become computationally expensive with larger datasets.

6.4.2 Resource Utilization

Higher resource utilization indicates that these models require more computational power, which could be a consideration when deploying in resource-constrained environments typical of MANETs. Random Forest and Neural Network models have the highest resource utilization, slightly above SVM, with KNN showing the lowest.

6.4.3 Network Throughput

Higher throughput in the Neural Network model suggests it effectively handles larger volumes of network traffic, identifying and mitigating DoS attacks while maintaining network performance. The lower throughput observed in the KNN model indicates it might not be as effective in high-throughput scenarios, potentially leading to network congestion or slower attack detection. Neural Network demonstrates the highest network throughput, followed by Random Forest, SVM, and lastly KNN.

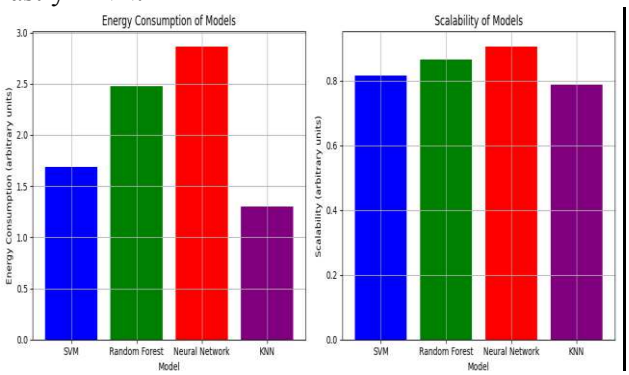


Fig. 4. Energy Consumption and Scalability of the Model

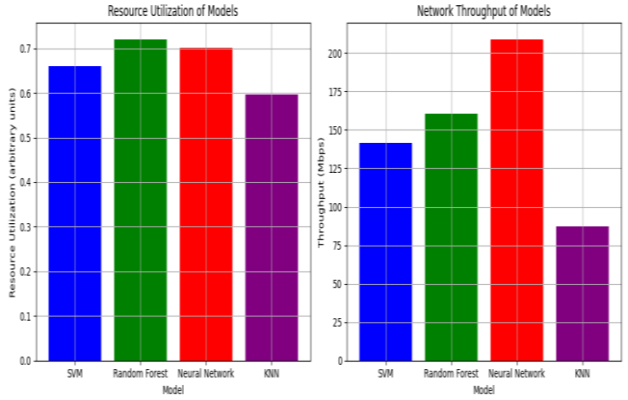


Fig. 5. Resource Utilization and Network Throughput of the Model

7. Discussion

Neural networks are the best choice in situations when high accuracy and the capacity to handle intricate patterns with superior recall and precision are required. Performance-critical applications can benefit from their use because they have the highest network throughput and energy consumption among them. An excellent substitute is Random Forest, which provides great accuracy with minimal false positives and negatives. It is a well-rounded option in most situations due to its excellent resource efficiency, good scalability, and modest energy consumption. SVM works well for basic tasks where managing complicated data is not as important as speed and simplicity. Its resource utilization, scalability, and energy consumption are all moderate, but its false negative rate is a little higher. KNN works well in simpler circumstances and with smaller datasets, it performs poorly in terms of network throughput, resource usage, and scalability. It performs well in contexts with limited resources and manageable data volumes.

The Neural Network model, while having higher energy consumption and resource utilization, offers

the best throughput and scalability. This suggests that Neural Networks are more suited for scenarios where performance and scalability are prioritized, even if it means using more energy and computational resources. Random Forest also shows a balanced performance with good scalability and resource utilization, making it a strong alternative for environments where energy efficiency is a secondary concern. SVM offers moderate performance across all metrics, making it an option when balanced performance is required. KNN is the most energy-efficient and uses the least resources but falls behind in scalability and throughput, making it less suitable for larger or high-traffic MANET scenarios. These insights help in selecting the appropriate machine learning model for DoS attack detection and mitigation based on the specific requirements of the MANET environment, balancing the trade-offs between energy consumption, scalability, resource utilization, and throughput.

7. Conclusion and Future Research Directions

The proposed framework for detecting and mitigating DoS attacks in Mobile Ad-hoc Networks (MANETs) leverages machine learning models like SVM, Random Forest, Neural Networks, and KNN to enhance network security. By generating synthetic datasets that simulate both normal and malicious traffic, the framework trains these models to distinguish between benign and attack traffic effectively. The performance evaluation reveals that Neural Networks provide the highest throughput and scalability, making them suitable for high-traffic environments, while models like KNN offer better energy efficiency, which is crucial in resource-constrained scenarios. the proposed framework can significantly contribute to the development of more secure and resilient MANETs, supporting critical applications that rely on reliable and protected network communication.

Future research directions should include exploring the use of advanced deep learning models such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks. Additionally, adaptive mechanisms that adjust to changing attack patterns and network conditions could be developed, enhancing the system's ability to respond to threats in real time. Another significant area for future research is the development of hybrid detection models that combine multiple machine learning techniques to leverage their respective strengths. For instance, integrating SVM, Random Forest, Neural Network, and KNN into ensemble approaches could improve detection rates and reduce false positives. Extending the framework to detect a wider range of attacks would also make the system more comprehensive and robust against various types of security threats. Lastly, optimizing the energy efficiency of the detection models is crucial, especially in the context of MANETs where nodes often have limited power resources. Future work could focus on developing lightweight algorithms that reduce computational overhead while maintaining high detection accuracy. Testing the framework in real-world scenarios with actual network traffic data and resource-constrained environments, such as IoT-integrated MANETs, will provide valuable insights into its practical applicability.

References:

[1] R. R. L. a. C. R. K. Reddy, "Node activity based trust and reputation estimation approach for secure and QoS," International Journal of Electrical & Computer Engineering (IJECE), no. 6, , vol. vol 9, pp. pp. 5340-5350, 2019.

[2] P. C. a. T. Ranganayaki, "A Study on Manet: Applications, Challenges and Issues," International Journal of Engineering Research & Technology (IJERT) ISSN: 2278-0181, no. Special Issue - 2020, 2020.

[3] R. A. A. a. R. A. Maha Abdelhaq, "The resistance of routing protocols against DDOS attack in MANET," International Journal of Electrical and Computer Engineering (IJECE), Vols. Vol. 10, No. 5, p. pp. 4844~4852, 2020.

[4] P. Nayak and B. Vathasavai, "Impact of random mobility models for reactive routing protocols over MANET," International Journal of Simulation--Systems, Science & Technology, , Vols. vol. 17,, pp. pp. 13.1-13. 9,.

[5] M. Y. N. Uparosiya, ""“Survey on MANET: Routing protocols, advantages, problems and security,”," International Journal of Innovative Computer Science & Engineering,, vol. vol. 1, pp. no. 2, pp. 12-17.

[6] e. a. A. D. Basheer, "SIEM-based detection and mitigation of IoT-botnet DDoS attacks,," International Journal of Electrical &

Computer Engineering, , vol. vol. 10, pp. pp. 2182-2191,, 2020.

[7] I. S. H. K. J. E. a. D. G. Fatemeh Safari, "The Diverse Technology of MANETs: A Survey of Applications and Challenges," International Journal of Future Computer and Communication,, Vols. Vol. 12, No. 2, , p. 12, 2023.

[8] M. C. a. E. M. S. Er. Inakshi Garg, "DOS Attack Mitigation In MANET," International Journal of Computer Science Engineering (IJCSE), Vols. ISSN : 2319-7323, p. 05.

[9] .. M. I. O. S. O. Baraa T. Sharef, "A survey on position-based routing protocols for Flying Ad hoc Networks (FANETs)," Vehicular Communications, pp. Pages 29-56, 2017.

[10] A. M. F. A.-R. S. Saeed N. H., "MANET routing protocols taxonomy," in International Conference on Future Communication Networks,.

[11] N. Beijar, "Zone routing protocol (ZRP).," in Networking Laboratory, Helsinki University of Technology, 9, 1-12., Finland,, 2019.

[12] A. T. R. Y. a. D. A. K. Tawseef, "MANET Routing Protocols, Attacks and Mitigation Techniques: A Review," International Journal of Mechanical Engineering, Vols. ISSN: 0974-5823 Vol. 7 No. 2 , p. 11, 2022.

[13] T. S. H. E. S. M. A. K. Mohamad, "AN INTRUSION DETECTION MECHANISM FOR MANETS BASED ON DEEP LEARNING ARTIFICIAL NEURAL NETWORKS (ANNS)," International Journal of Computer Networks & Communications (IJCNC), vol. Vol.15, no. No.1, p. 15, 2023.

[14] R. K. K. V. R. G. Zalte S. S., "Mitigation of DDoS Attack in MANET," International Journal of Engineering and Advanced Technology (IJEAT),ISSN: 2249-8958 (Online), Vols. Volume-9 , no. Issue-6, , 2020.

[15] B. C. R. K. Jasmine, "DDoS Attack Detection and Prevention using Aodv Routing Mechanism and Ffbp Neural Network in a Manet",," International Journal of Recent Technology and Engineering (IJRTE) ISSN: 2277-3878, Vols. Volume-8 , no. Issue-2,, pp. pp- 4136-4142, 2019.

[16] E. A. a. A. Shams, "A novel support vector machine based intrusion detection system for mobile ad hoc networks," Wireless Networks,, Vols. vol. 24, no. 5, , pp. pp. 1821-1829, 2020.

[17] A. J. H. a. A. L. Alsumayt et al, "Evaluation of Detection Method to Mitigate DoS Attacks in MANETs," in 1st International Conference on Computer Applications & InformationSecurity (ICCAIS),.

[18] E. H. A. S. H. a. L. K. Z. Abou, "Cochain-SC: An Intra- and Inter-Domain DDoS MitigationScheme Based on Blockchain Using SDN and Smart Contract," in In: Proceedings of the IEEE Access,, vol. 7, pp. 98893-98907, 2019.

[19] a. P. B. S. Payal N. Raj, "DPRAODV: A DYANAMIC LEARNING SYSTEM AGAINST BLACKHOLE ATTACK INAODV BASED MANET," International Journal of Computer Science Issues,, Vols. Vol. 2,, pp. PP 54-59, 2019.

[20] C. R. K. Jasmine Batra, "DDoS Attack Detection and Prevention using Aodv Routing Mechanism and Ffbp Neural Networkin a Manet," International Journal of Recent Technology and Engineering (IJRTE) ISSN: 2277-3878,, Vols. Volume-8, no. Issue-2,, pp. pp- 4136-4142 .

sustainability

MDPI

Article

Detecting DDoS Attacks in Software-Defined Networks Through Feature Selection Methods and Machine Learning Models

Huseyin Polat *, Onur Polat and Aydin Cetin *

Faculty of Technology, Gazi University, Ankara 06500, Turkey; onurpolat@gazi.edu.tr

* Correspondence: polath@gazi.edu.tr (H.P.); acetin@gazi.edu.tr (A.C.)

Received: 19 December 2019; Accepted: 29 January 2020; Published: 1 February 2020

Abstract: Software Defined Networking (SDN) offers several advantages such as manageability, scaling, and improved performance. However, SDN involves specific security problems, especially if its controller is defenseless against Distributed Denial of Service (DDoS) attacks. The process and communication capacity of the controller is overloaded when DDoS attacks occur against the SDN controller. Consequently, as a result of the unnecessary flow produced by the controller for the attack packets, the capacity of the switch flow table becomes full, leading the network performance to decline to a critical threshold. In this study, DDoS attacks in SDN were detected using machine learning-based models. First, specific features were obtained from SDN for the dataset in normal conditions and under DDoS attack traffic. Then, a new dataset was created using feature selection methods on the existing dataset. Feature selection methods were preferred to simplify the models, facilitate their interpretation, and provide a shorter training time. Both datasets, created with and without feature selection methods, were trained and tested with Support Vector Machine (SVM), Naive Bayes (NB), Artificial Neural Network (ANN), and K-Nearest Neighbors (KNN) classification models. The test results showed that the use of the wrapper feature selection with a KNN classifier achieved the highest accuracy rate (98.3%) in DDoS attack detection. The results suggest that machine learning and feature selection algorithms can achieve better results in the detection of DDoS attacks in SDN with promising reductions in processing loads and times.

Keywords: SDN; AI algorithms; feature selection; DDoS attacks

1. Introduction

Traditional network infrastructures have been unable to address certain requirements such as high bandwidth, accessibility, high connection speed, dynamic management, cloud computing, and virtualization. Thus, Software Defined Networking (SDN), which has a flexible, programmable, and dynamic architecture, has emerged as an alternative to traditional networks [1]. SDN architecture consists of control, data, and application planes. Devices, such as switches and routers, are placed on the data plane. This plane is programmed and managed by the control plane [2]. The control plane is responsible for the management of transmission devices placed on the data plane. The controller, which performs as the brain of the network, is located on this plane. Devices on the data plane carry out packet transmission according to the rules set by the controller. The application plane communicates with the devices located on the network infrastructure via the controller.

Devices on the network are programmed by applications using Northbound and Southbound interfaces. The controller communicates with the services working on the application layer by means of the Northbound interface and communicates with the devices on the data layer by means of the Southbound interface. The most popular protocol used in this interface is the OpenFlow protocol [3].

As the control logic has been taken from local devices and become central, SDN is structured from a single spot and dynamically optimized [4]. Although certain security threats are general in computer networks, SDN has brought its own security threats. There are at least seven different threat vectors identified with SDN [5]. The most important threat vector for SDN is Distributed Denial of Service (DDoS) attacks. They can be against the controller in SDN or in the storage capacity of the flow table in the OpenFlow switch.

The controller is exposed to DDoS attacks through the communication line between the controller and the data plane. DDoS attacks direct a large amount of traffic to the OpenFlow switch on the data plane. If packets arriving at the OpenFlow switch do not match with the flow input in the flow table (miss flow), packets are taken into the flow buffer. Then, they are transmitted to the controller with the Packet-In message to write a new rule. In this situation, the sources of the controller (memory, processor, bandwidth, etc.) remain incapable and the network becomes inoperative. In addition, the bandwidth of the communication line between the controller that is exposed to attack traffic and the OpenFlow switch is negatively affected. Therefore, network performance severely declines [6].

The data plane is exposed to DDoS attacks through the flow table located in the network devices. In such DDoS attacks, packets are sent to the switch from unknown sources. The controller writes a rule for these arriving packets and forwards them to the flow table of the switch. After a while, the capacity of switch flow table becomes full. Consequently, new rules cannot be written to the flow table and packets cannot be forwarded. In addition, the incoming packets fall automatically as the buffer capacity becomes full with the attack [7]. OpenFlow switches manage the packets with only the flow entry coming from the controller. Thus, decreasing DDoS attacks in SDN is more difficult when compared to traditional networks.

Machine learning-based approaches can provide more dynamic, more efficient, and smarter solutions for SDN management, security, and optimization. In order to ensure network security, the detection of DDoS attacks is very important for taking the necessary measures in a timely manner. Processing SDN flow data with the machine learning-based DDoS attack detection systems integrated into SDN structures can lead to a self-determining network that is able to learn and act. Moreover, SDN having an integrated machine learning application may be a reference model in forming a secure structure in studies providing for the integration of 5G networks.

This paper consists of five sections. The second section presents a short review on DDoS attacks against the controller and DDoS attacks against the OpenFlow switch, along with SDN architecture and properties and security gaps. In the third section, under Materials and Methods, process steps for applying the feature selection methods and machine learning models, and experimental setup and data collection from SDN are presented in detail. In the fourth section, the experimental results are analyzed and implications are drawn from the findings. In the last section of the study, the concluding remarks of the study are presented.

2. Related Work

We discuss the related studies from two perspectives, namely, DDoS attacks which are against the controller and against the OpenFlow switch.

2.1. DDoS Attacks against the Controller

Control functions are taken from the switch and given to the controller, which is the brain of the network in SDN architecture. Parent level rules are easily applied to the network with the help of the controller. The controller can add new rules to the transmission devices and change the existing rules. It can carry out these changes by communicating with transmission devices via a secure channel through the OpenFlow protocol. Continuity and the unity of data traffic are ensured through this channel. If this secure channel breaks, the connection between the controller and transmission devices breaks.

SDN architecture is the target for DDoS attacks. While the attacker is attacking the SDN network, it has three main targets, as shown in Figure 1: to consume the sources of the controller, to occupy the

bandwidth of the channel between the controller and the switch, and to fill the flow tables in the switch with unnecessary flows. In DDoS attacks against the controller, the attacker sends a large number of packets to the OpenFlow switch via zombie users. It is difficult for the controller to differentiate between traffic sent by the attacker and legal traffic.

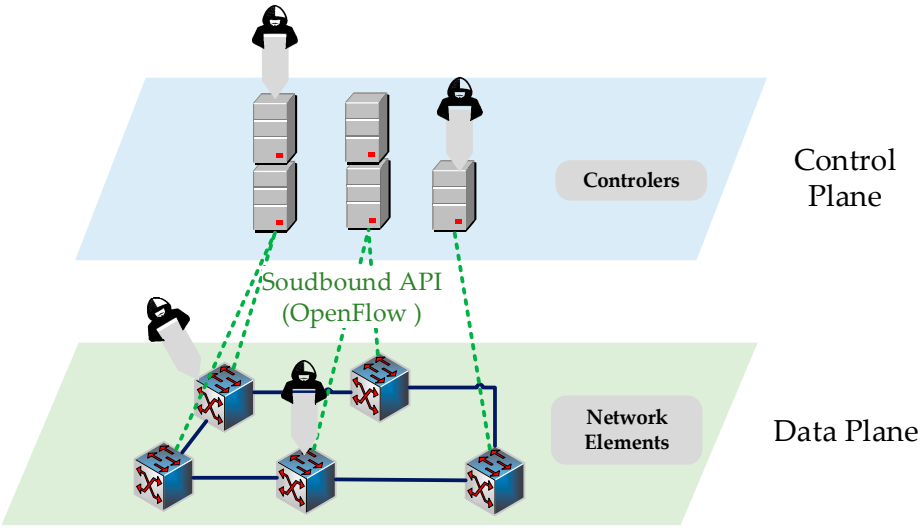


Figure 1. Main targets of the Distributed Denial of Service (DDoS) attacks on Software Defined Networking (SDN).

The OpenFlow switch seeks a match in the flow input by checking the packet header (source port, target port, source IP address, target IP address, etc.). If there is no match, the packet is forwarded to the controller by encapsulating the packet header in the flow request with the OpenFlow protocol (OFPT) PACKET_IN message. Then, the controller responds with the OFPT FLOW_MOD message. This message involves the process to be carried out on the packet and the timeout of the flow in the flow table assigned for the packet [8]. As the number of packets forwarded to the controller increases, the sources of the controller are consumed (bandwidth, memory, and CPU), new flow input for the new legal packets arriving at the network cannot be processed, and the SDN architecture collapses.

Alshamrani et al. [9] claim that the existing mechanisms to prevent DDoS attacks are not effective. Accordingly, they investigated the effect of misbehavior and new flow attacks on SDN. They gathered traffic data from transmission devices on a data plane periodically and then applied machine learning classification algorithms to respond to sudden traffic changes that occurred in the SDN architecture at the moment of attack. Packet_In messages flowing between the controller and transmission devices at the moment of attack were used as a base. Support Vector Machine (SVM), J48, and Naive Bayes (NB) algorithms were employed for classification.

Latah and Toker [10] detected DDoS attacks by measuring the rate of the arriving packet at the moment of attack. When some packets, among the ones coming to the controller, passed the pre-determined threshold, an inspection unit that uses SVM was activated to be able to detect DDoS flooding attacks.

Li et al. [11] proposed a bidirectional Recurrent Neural Network (RNN) model covering each layer of the SDN network structure in order to detect and block DDoS attacks. Although the proposed model was developed for the real-time detection, and the blocking of DDoS attacks has a high accuracy rate, it may not be equally successful in large networks where more than one controller is used. The proposed model may disrupt the synchronized work of controllers and degrade the performance of the network.

2.2. DDoS Attacks against the OpenFlow Switch

The OpenFlow switch and flow table are seen as the main targets, as they include administrative, transmission, and access control information [12]. The attacker first aims to break the functionality of the network by way of physical or virtual unauthorized access to the network. It is impossible for the OpenFlow switch to store all the rules covering each flow as it has a limited storage capacity. As packets come from an unknown address, new rules are needed to be added to the switch. If the attacker sends a large number of packets from an unknown address in a short time, the rules are written by the controller for these packets and forwarded to the flow table. The flow table with limited storage space becomes full in a short time. No space is left in the flow table for a new rule. Thus, transmission of the legal traffic stops. Apart from the flow table, flow cache memory is also a target for DDoS attacks (Figure 1).

Upon receiving the packet from the input port, the switch with a reactive cache memory mechanism forwards this packet to the flow cache memory [13]. For the new arriving packets, a match is searched for the flows in a flow table. If a match is found, the packet is forwarded from the cache memory to the output port. Otherwise, it is forwarded to the controller by means of the control channel and with a Packet_In message. The controller responds with an OFPT_FLOW_MOD message by assigning hard_timeout and idle_timeout, which define the necessary rules for the packet and how long the rules will stay. When the switch receives the rule from the controller, the packet is processed and the rule written by the controller is taken into the transmission table cache memory to have the arriving packets directly processed. This mechanism makes the switch defenseless against DDoS attacks. A large number of packets forwarded to the switch by malicious nodes are taken into cache memory and the response from the controller, which includes the information on the flow rules, is expected [14]. Packets coming from malicious nodes fill the switch buffer and then packets coming from legal users start to drop [15].

Ye et al. [16] gathered data on network traffic from the transmission devices on the data plane by means of the controller. Six-tuple characteristic values related to DDoS attacks from the switch flow table were extracted to detect DDoS attacks by means of SVM. A high detection accuracy rate was reported. However, the test accuracy rate of the Internet Control Message Protocol (ICMP) attack flow was reported as relatively low.

Xue et al. [17] indicated that many security requirements are needed as the SDN controller manages multiple switches in the data plane. They stated that security could not be achieved with existing equipment and software that has not yet adapted to the SDN architecture. In particular, DDoS attack on switches in the data plane pose a great danger for the continuity of the SDN architecture of the network. SVM-optimized C and G parameters by cross-validation-genetic algorithm (CV-GA) were used to detect the DDoS attacks.

Nanda et al. [18] proposed the use of machine learning algorithms, trained on historical network attack data, to identify potential malicious connections and potential targets to minimize security threats. They used C4.5, Bayesian Network (BayesNet), Decision Table (DT), and Naive Bayes algorithms. It was stated that detecting malicious users in the data plane by means of estimations using machine learning algorithms was possible. Ensuring user identification can enable the SDN controller to quickly and effectively write new rules to prevent the attack, which is important for the efficiency and continuity of the network.

Jankowski and Amanowicz [19] employed the machine learning algorithms of Self-Organizing Maps (SOM), Learning Vector Quantization (LVQ1), and their enhanced versions (Multi-pass Self-Organizing Maps (M-SOM), Multi-pass Learning Vector Quantization (M-LVQ1) and Hierarchical Learning Vector Quantization (H-LVQ1)) to detect and monitor malicious activities on the data plane. Promising results were achieved with the H-LVQ1 algorithm compared to SOM, M-SOM, LVQ1, M-LVQ1.

Banerjee and Chakraborty [20] conducted a two-stage study. In the first stage, Naive Bayes and K-Nearest Neighbors (KNN) machine learning algorithms were trained to differentiate between

the attack and normal traffic. Then the attacker was detected using a three-way handshake service. The detected attacker was blocked by creating an Access Control List (ACL).

Mowla et al. [21] proposed Cognitive Switch-based DDoS Sensing and Mitigation in SDN-driven Content Delivery Networks. SVM and Logistic Regression algorithms were used for traffic classification, and this classification initiated the deployment of security rules to the OpenFlow switches, to prevent from new forms of flooding attacks, and to detect and defend against all possible DDoS attacks.

This study focuses on DDoS attacks in SDN architecture and offers machine learning models supported with feature selection methods to detect attacks. Therefore, the aim is to develop DDoS attack detection systems based on a high-fertility rate machine learning for SDN architecture. The detection of the DDoS attack on the SDN controller and the switches located in the data plane are very important for the continuity of the network and the detection of legal traffic at the time of the attack. If attack traffic is detected on the controller, it is easier for the controller to write new rules to the flow table of the switches located in the data plane to prevent the attack. This provides a great advantage for preventing the attack. We propose the use of feature selection methods with machine learning models to detect DDoS attacks. We believe that our approach will make a significant contribution to the effective detection of DDoS attacks in SDN.

3. Materials and Methods

The process steps for applying the feature selection methods and machine learning models are given in Figure 2. Additionally, in this section, how the data is obtained from the dataset, and the features and classes of the dataset are explained. The features in the dataset, created within the scope of the study, were obtained from the main metrics that are vital for the continuity of the SDN architecture. The attack data, obtained as a result of DDoS attacks, were classified using machine learning techniques. Features in the dataset were selected using feature selection methods and the performances of the classifiers on the obtained feature set were examined. The analysis of the application was performed in a Matlab environment.

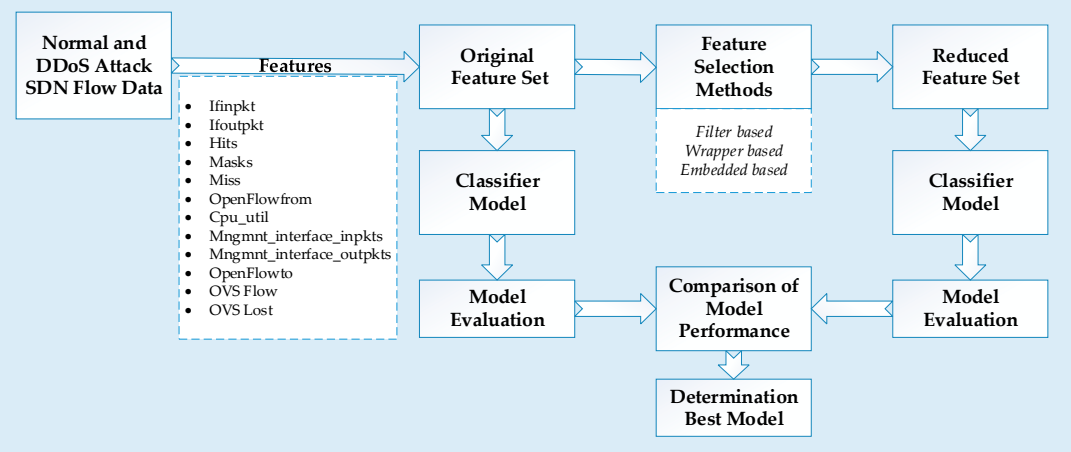


Figure 2. Process steps for applying the feature selection methods and machine learning models.

3.1. Feature Selection Methods and Machine Learning Models

To detect DDoS attacks, machine learning models supported with feature selection methods are used in this study. Machine learning is a method that draws implications from existing data by using mathematical and statistical methods, and makes predictions about the unknown with these implications. In the literature, a variety of machine learning models have been proposed. The taxonomy of models can be summarized with the kernel-based, distance-based, neural network- based and probability-based characteristics. The continuing research suggests that there is no universal best model for all classification tasks. The literature suggests that SVM, KNN, ANN (Artificial Neural

Network), and NB models have shown good performance for solutions of classification problems. In this study, as candidates of their classification group, the performance rate for detecting DDoS attacks from SVM, KNN, ANN, and NB models were investigated.

The datasets used in the training test of machine learning models can contain a large number of features. Although some of these features have a high degree of influence on the classification result, some features may have little or no effect on the classification result. Using features that have little impact on classification can increase time and process costs. The goal is to reduce the features that have a low impact on the classification and to provide the use of highly effective features.

The filter, embedded, and wrapper methods were used as feature selection methods. The filter method mainly focuses on the intrinsic properties of the features, while the wrapper method focuses on the usefulness of features based on the classifier performance. The embedded method tries to use the properties of both the filter and wrapper methods to optimize the performance of a learning algorithm or model. Each feature selection method has different algorithms to achieve their goals. Our classification problem is a binary classification problem and in our dataset there is no missing data. Therefore, the relief filtering algorithm inspired by instance-based learning was selected for filter-based feature selection.

A greedy search-based Sequential Forward Selection (SFS) algorithm was preferred due to its proven success in finding an optimal feature subset among the wrapper-based feature selection methods. Lasso or L1 algorithm was selected as the embedded feature selection method as it adds a penalty against complexity to reduce the degree of overfitting, which improves the optimization performance of the model.

3.1.1. Filter-Based Feature Selection

The filter-based feature selection method contributions to the achievement of the results of the features are calculated using statistical methods. A reduced feature set is created by selecting the best feature set from the rated features, as seen in the process given in Figure 3. In the study, the Relief algorithm, which is one of the filter-based feature selection algorithms, is used. The Relief algorithm calculates the relation of each sample in the dataset with other instances in its class and its distance to different classes. As a result of this calculation, it creates a model and performs the feature selection process [22,23].

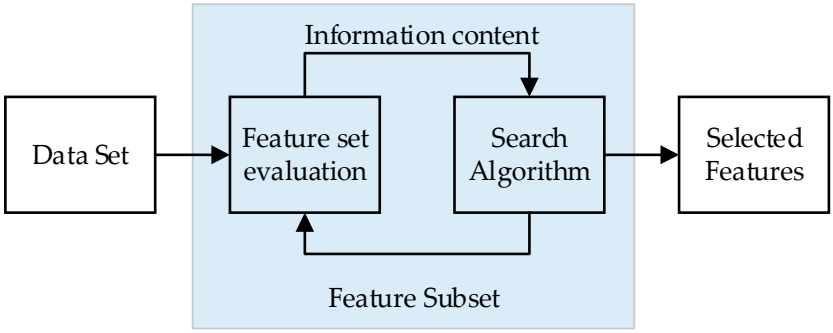


Figure 3. A filter-based feature selection algorithm.

3.1.2. Wrapper-Based Feature Selection

The wrapper method aims to find the most ideal attributes by trying all features in the classification algorithm. When the ideal subset of features is reached, the process is completed and a reduced dataset is created (Figure 4). This may be more successful than statistical methods; however, depending on the classification algorithm, it may be disadvantageous in terms of speed. In the study, the sequential forward floating selection algorithm, which is one of the wrapper-based feature selection algorithms, is used. When selecting a feature, the same direction is used until the classification success reaches a higher value [22,23].

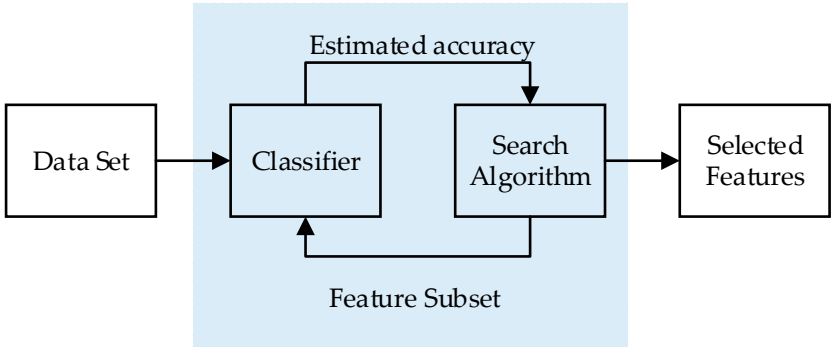


Figure 4. A wrapper-based feature selection algorithm.

3.1.3. Embedded-Based Feature Selection

Embedded-based feature selection methods are feature selection algorithms that are included in the classification algorithm. These algorithms make their selection of features by identifying the features that will best contribute to the accuracy of the model (Figure 5). They have been developed to combine the advantages of filter and wrapper-based methods. In this method, a learning algorithm takes advantage of its variable selection process and simultaneously performs feature selection and classification [22]. In this study, the Lasso algorithm, which is one of the embedded-based feature selection algorithms, is used. An important characteristic of the Lasso algorithm eliminates the weights of the least important features. That is, in this way, it naturally makes the attribute selection and outputs a reduced set of features [22,23].

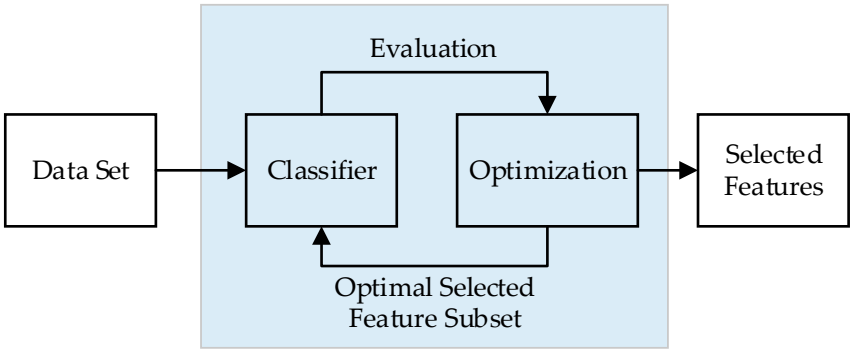


Figure 5. An embedded-based feature selection algorithm.

3.1.4. Support Vector Machine (SVM)

SVM is a supervised learning model employed in the learning field, which uses classification and regression analysis in defining and analyzing obtained patterns. The working principle of SVM is based on predicting the most appropriate decision function to separate between two classes. In other words, it is based on defining the hyper-plane that can separate two classes. We used SVM with Cubic kernels.

3.1.5. Naive Bayes

NB classification aims to detect the class, namely the category, of the data entered into the system with a series of calculations defined in accordance with the principles of probability [23]. In NB classification, a certain amount of training data is entered into the system. The data entered for training has to have a class. Test data entered into the system with probability processes carried out on the training data is processed. This process is done according to previously obtained probability values and then the class of the given test data is detected. The more the number of training data, the more accurate it is to detect the real category of the test data. We used NB with Gaussian kernels.

3.1.6. Artificial Neural Network (ANN)

ANN is a strong classifying tool based on the artificial neuron model. It uses neurons as the most basic building block. Artificial neurons are designed to behave similarly to biological neurons. The ANN architecture created in this study contains 12 input units and 10 neurons in a single hidden layer. It should be noted that the number of inputs and the number of neurons in a single hidden layer change according to the chosen filter methods used in feature selection.

3.1.7. K-Nearest Neighbors

The KNN algorithm is a simple, easily applicable, and supervised machine learning algorithm that can be used for solving both classification and regression problems. When new data comes in, it determines the class of the new data by looking at its nearest K neighbors. Manhattan, Minkowski, and Euclidean distance functions are used for the distance between two data. The Euclidean distance function was used in this study. The similarity between the sample to be classified and the samples found in the classes was detected. When new data was encountered, the distance of this data to the data in the training set was calculated individually by using the Euclidean function. Then, the classification set was created by selecting the k dataset from the smallest distance. The number of neighboring KNN (k) is based on the value of classification. During the classification, k was determined as 10.

3.2. Experimental Setup and Collecting Data from SDN

Experimental SDN topology for collecting normal and DDoS attack data is shown in Figure 6. In the topology, there are six (PC1-PC6) virtual machines (VMs) that have Ubuntu 18.04, 1 GB RAM, 1 CPU configuration works on VirtualBox-KVM, two VM switches, one Open vSwitch (OVS), sFlow, and InfluxDB docker images, and a Phyton-based open source OpenFlow/SDN (POX) controller. sFlow, and InfluxDB, set up in the same virtual machine, were connected to the OVS switch via a VM switch. The VM switch on the users' side was connected to the OVS switch with a bridge (br0).

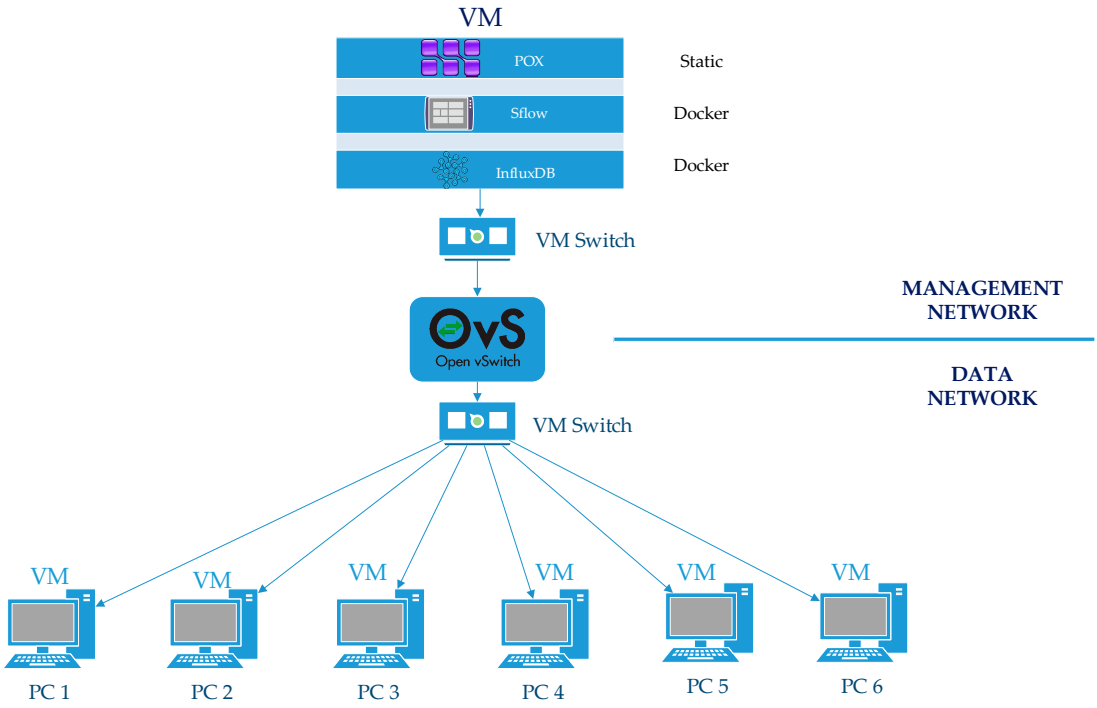


Figure 6. Experimental SDN topology for collecting data. Virtual machine (VM).

To analyze the effects of the attack traffic, network metrics were collected from the OVS switch by using an sFlowDocker image both at the moment of attack and normal traffic. For this purpose,

JavaScript was used to record network metric data on an InfluxDB image with a timestamp, which is an open source time-series platform.

We have applied a protocol-based attack scenario. Three types of flooding attacks Transmission Control Protocol (TCP), User Datagram Protocol (UDP), and Internet Control Protocol (ICMP) were used to create a DDoS attack dataset with an "hping3" packet generator. The Hping3 tool was installed on PC1 in the network and defined as attacker, while PC6 was chosen as the victim. Under attack, PC6 has the 10.0.0.6 IP address with a constant 512 bytes of payload size and constant packet rates of 2000 packet per second (pps) for each protocol-based flooding attack. This leads the number of flow modification messages from a controller to be more than 1304 per second, resulting in the number of flow table entries being 6996.

Due to the generated DDoS attack packets against the controller, the processing, bandwidth, CPU capacity, and communication capacity of the controller was overwhelmed. The capacity of the flow table entries in the switch is exhausted due to the unnecessary flows that the controller creates because of attacking packets. The simulation was conducted in 15 min for each of the sent TCP, UDP, and ICMP packets. By this means, a dataset having 65,000 samples, related to network traffic flow at the moment of DDoS attack, was generated.

In order to obtain normal traffic data for the training and testing, the TPC, UDP, and ICMP traffic data of PC2 and PC5 were recorded. By this means, a dataset, which consists of 64,000 samples where the data related to normal network traffic flow, was obtained. Thus, a new dataset, consisting of 12 features and a total of 129,000 samples, was obtained. Twelve features and the class of the dataset and their explanations are given below.

(1) Ifinpkt: Packets entering into the system from data plane.
(2) Ifoutpkt: The out packet controls the packet leaving the system according to the flow rule sent by the controller with the "Packet-out" message.
(3) Hits: The number of packets matching with the rules previously set by the controller and stored in the flow table of OVS switch.
(4) Masks: The "masks" row displays the mega flow mask stats. This row is omitted for data paths not implementing mega flow.
(5) Miss: When a new packet arrives at the transmission devices (switch, router, etc.), a match is sought in the flow table. If the arriving packet does not match with any flow input (flow miss), it is forwarded to the controller to set a new flow rule.
(6) OpenFlowfrom: The flow modification message sent by the controller. It is one of the main messages sent by the controller. With this message, switch state is changed for the new flow.
(7) Cpu_util (CPU Usage in Data Plane): The central processor unit of the computer processes and calculates the arriving data with the help of mathematical operations. The usage percentage of this unit varies according to the processing capacity of the computer. If the percentage is high, it means that the computer performs at maximum or above the normal level for the number of operating applications.
(8) Mngmnt_interface_inpkts: Packet arriving at the control plane.
(9) Mngmnt_interface_outpkts: Packet leaving the control plane.
(10) OpenFlowto: This message is one of the main messages sent by the controller. With this message, switch state is changed for the new flow.
(11) OVS Flow: The number of flows in the data path.
(12) OVS Lost: The number of packets sent for processing but dropped.
(13) Class: The class of traffic. Data used in this study are labelled data. In other words, supervised learning is aimed at four classes: 'Normal traffic', 'ICMP Flood', 'TCP Flood', and 'UDP Flood' attack.

The dataset created within the scope of the study was reduced by using feature selection methods and then the classification was made on these data. SVM, NB, ANN, and KNN were used as classifiers.

The experimental study consisted of two stages. Classifiers were trained and tested with using all the features in the dataset. Using the k-fold cross validation (k = 10) method, the dataset was divided into training and testing sets to test the success of the applied method [24] in the first stage. In the second stage, feature selection methods were used. Relief, sequential forward floating selection, and Lasso algorithms were used to select the most effective feature in the entire dataset as filter, wrapper, and embedded feature selection methods, respectively. Three different datasets were created from these feature selection methods. For each of the three datasets, the best performance ratio, between 1 and 12 features, was determined by SVM, NB, ANN, and KNN algorithms, separately.

4. Experimental Results

In this study first, data at normal traffic moments and at DDoS attack traffic moments were analyzed on SDN architecture with the dataset. Feature selection methods were not used at this stage of the study. The obtained results of the analysis are presented in Table 1.

Table 1. Performance of the machine learning models without a feature selection method.

Classifier	Accuracy	Sensitivity	Specificity	Precision	F1_Score
SVM	92.11%	88.71%	96.93%	91.42%	89.91%
KNN	95.67%	93.87%	98.01%	97.05%	95.30%
ANN	91.07%	87.27%	96.58%	89.89%	88.45%
NB	94.48%	91.77%	98.29%	92.94%	91.79%

When the success rates are analyzed, KNN and NB exhibited better performance in analyzing SDN data. At the end of the training carried out with the KNN algorithm, a Receiver Operating Characteristic (ROC) curve was prepared (Figure 7). The ROC curve is a tool for evaluating the test results. It is defined by a two-dimensional graph which has True Positive (TP) as Y-axis and False Positive (FP) as X-axis. TP indicates positive instances correctly classified as positive outputs and FP indicates negative instances wrongly classified as positive outputs. Another metric used in confusion matrices is False Negative (FN) which indicates positive instances wrongly classified as negative output. The TP value was observed to be above 0.9% in ROC curve.

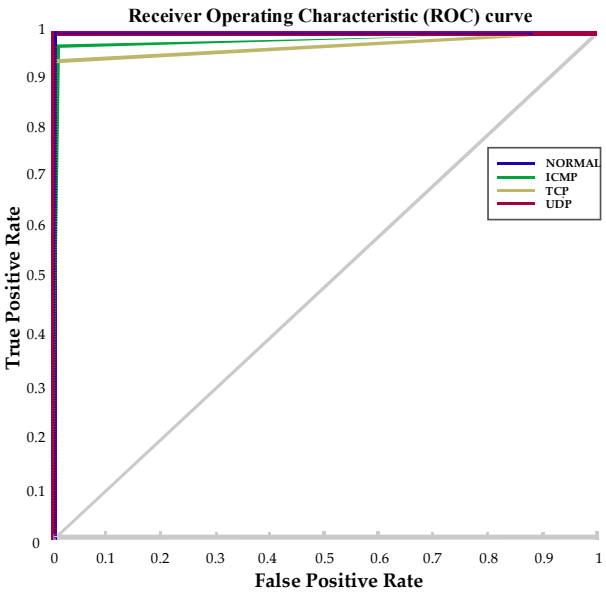


Figure 7. The normal, ICMP, TCP and UDP ROC curves for KNN without future selection.

When the performance results were examined, it was found that the KNN classifier was more successful in analyzing the attack data. The ROC curve is shown in Figure 7, and the confusion matrix

shown in Table 2 was prepared according to the results of the KNN classifier. We observed that the attack traffic type (ICMP, TCP, and UDP) affects the performance rate. The highest performance rate was obtained when the UDP flood attack was performed.

Table 2. Confusion matrix of the KNN classifier without feature selection.

		Predicted Class					
		Normal	**ICMP**	**TCP**	**UDP**	**TP**	**FN**
Actual Class	**Normal**	99.61%	0.07%	0.11%	0.18%	99.6%	0.4%
	ICMP	7.31%	90.99%	1.68%		91.0%	9.0%
	TCP	12.78%	1.15%	85.63%	0.41%	85.6%	14.4%
	UDP	0.44%	0.33%		99.21%	99.2%	0.8%

In the second part of the study, in order to obtain the features that have the most powerful effect on the prediction of machine learning models, filtering, wrapping, and embedded-based feature selection methods were applied. The reduced feature dataset obtained by the feature selection methods was applied to each model and the accuracy of the model was tested.

The Relief algorithm was applied to the SVM, KNN, ANN, and NB models as a filtering method. The proximity of each sample in this algorithm dataset to the other samples in its own class and its distance to different classes were calculated. As a result of this calculation, a model was created and feature selection was performed. As a result, 10 of the 12 features were selected for SVM, 6 for KNN, 6 for ANN, and 8 for NB. These features were retrained with each classification algorithm. The results are summarized in Table 3.

Table 3. Filter-based feature selection method performance of the machine learning models.

Classifier	**Accuracy**	**Sensitivity**	**Specificity**	**Precision**	**F1_Score**
SVM (10 Features)	92.46%	89.13%	97.02%	92.02%	90.43%
KNN (6 Features)	97.15%	95.88%	98.68%	98.10%	96.92%
ANN (6 Features)	92.28%	89.02%	96.99%	91.62%	90.20%
NB (8 Features)	95.70%	93.49%	98.65%	95.07%	93.60%

When the Relief algorithm was applied as the filtering method and the feature selection was made, the highest performance rate was obtained with the KNN classifier (Table 2). The confusion matrix of the classifier with the highest performance rate is given in Table 4.

Table 4. Confusion matrix of the KNN classifier with filter-based feature selection.

		Predicted Class					
		Normal	**ICMP**	**TCP**	**UDP**	**TP**	**FN**
Actual Class	**Normal**	99.84%	0.07%	0.07%		99.8%	0.2%
	ICMP	7.20%	91.66%	1.12%		91.7%	8.3%
	TCP	6.60%	0.94%	92.45%		92.5%	7.5%
	UDP	0.11%	0.11%	0.22%	99.55%	99.6%	0.4%

The Sequential Forward Floating Selection algorithm was applied to the SVM, KNN, ANN, and NB models with the wrapper method. A feature alone may not reduce the error, but evaluating this feature relative to another feature may improve the classification performance. For this purpose, Floating Algorithms have been developed. These algorithms have a floating variable and the number

of features added or removed at each stage is not constant. This creates a plurality of feature subset combinations [25]. Sequential Feature Selection adds or subtracts the feature based on a user-defined classifier performance metric.

In wrapper-based feature selection algorithm, when the features are reduced with the Sequential Forward Floating Selection algorithm, 8 of the 12 features were selected for SVM, 6 for KNN, 10 for ANN, and 8 for NB. The reduced dataset was rearranged for each classification algorithm. The results are shown in Table 5.

Table 5. The wrapper-based feature selection method performance of the machine learning models.

Classifier	Accuracy	Sensitivity	Specificity	Precision	F1_Score
SVM (10 Features)	92.15%	90.20%	97.26%	90.23%	90.21%
KNN (6 Features)	98.30%	97.73%	99.45%	97.72%	97.70%
ANN (6 Features)	91.44%	87.82%	97.31%	88.11%	87.89%
NB (8 Features)	94.87%	92.05%	98.43%	93.29%	92.01%

The highest performance rate was obtained with the KNN classifier when the Sequence Forward Floating Selection algorithm was applied with the wrapping method and feature selection was made (Table 5). The confusion matrix of the classifier with the highest performance rate is shown in Table 6.

Table 6. Confusion matrix of the KNN classifier with wrapper-based feature selection.

		Predicted Class					
		Normal	**ICMP**	**TCP**	**UDP**	**TP**	**FN**
Actual Class	**Normal**	99.58%	0.15%	0.15%	0.11%	99.6%	0.4%
	ICMP	0.56%	97.97%	1.12%	0.33%	98.0%	2.0%
	TCP	0.52%	5.45%	94.02%		94.0%	6.0%
	UDP	0.22%	0.11%	0.33%	99.32%	99.3%	0.7%

Finally, the SVM, KNN, ANN, and NB models were applied with the Lasso method as the embedded-based method. Lasso is another regularized variant of linear regression. An important characteristic of Lasso regression is that it eliminates the weights of the least important features. That is, in this way, it naturally makes feature selection and outputs a reduced set of features. With this method, 10 out of 12 attributes were selected for SVM, 8 for KNN, 6 for ANN, and 10 for NB. The reduced dataset was rearranged for each classification algorithm. The results are shown in Table 7.

Table 7. Embedded feature selection method performance of the machine learning models.

Classifier	Accuracy	Sensitivity	Specificity	Precision	F1_Score
SVM (10 Features)	92.15%	90.20%	97.26%	90.23%	90.21%
KNN (8 Features)	98.30%	97.73%	99.45%	97.72%	97.70%
ANN (6 Features)	91.44%	87.82%	97.31%	88.11%	87.89%
NB (10 Features)	94.87%	92.05%	98.43%	93.29%	92.01%

When the embedded method, Lasso, was applied and feature selection was made, the highest performance rate was obtained with the KNN classifier (Table 7). The confusion matrix of the classifier with the highest performance rate is shown in Table 8.

Table 8. Confusion matrix of the KNN classifier with embedded feature selection.

		Predicted Class					
		Normal	**ICMP**	**TCP**	**UDP**	**TP**	**FN**
Actual Class	**Normal**	99.39%	0.15%	0.33%	0.11%	99.4%	0.6%
	ICMP	0.78%	97.40%	1.35%	0.45%	97.4%	2.6%
	TCP	0.41%	15.30%	84.06%	0.20%	84.1%	15.9%
	UDP	0.33%	0.22%	0.44%	98.98%	99.0%	1.0%

Throughout the study, all dataset features were used and two different methods were used for reduction (Table 9). Although the results of classification using all features are acceptable, the performance results increased with the use of feature selection methods.

Table 9. Comparison of the machine learning model performances with use of feature selection methods.

Classifier	Method	No. of Features	Accuracy	Sensitivity	Specificity	Precision	F1_Score
SVM	-	12	92.11%	88.71%	96.93%	91.42%	89.91%
	Filter	10	92.46%	89.13%	97.02%	92.02%	90.43%
	Wrapper	8	92.15%	90.20%	97.26%	90.23%	90.21%
	Embedded	10	92.46%	90.73%	97.41%	90.49%	90.60%
KNN	-	12	95.67%	93.87%	98.01%	97.05%	95.30%
	Filter	6	97.15%	95.88%	98.68%	98.10%	96.92%
	Wrapper	6	98.30%	97.73%	99.45%	97.72%	97.70%
	Embedded	8	96.30%	94.95%	98.85%	95.09%	94.80%
ANN	-	12	91.07%	87.27%	96.58%	89.89%	88.45%
	Filter	6	92.28%	89.02%	96.99%	91.62%	90.20%
	Wrapper	10	91.44%	87.82%	97.31%	88.11%	87.89%
	Embedded	6	92.09%	88.91%	97.42%	89.22%	89.06%
NB	-	12	94.48%	91.77%	98.29%	92.94%	91.79%
	Filter	8	95.70%	93.49%	98.65%	95.07%	93.60%
	Wrapper	10	94.87%	92.05%	98.43%	93.29%	92.01%
	Embedded	10	95.09%	93.34%	98.45%	93.44%	93.18%

The highest accuracy rate obtained by applying the test data to the machine learning model, shown in Table 5, was obtained with the KNN classifier (98.3%) when the wrapper feature selection method was applied. This accuracy rate was obtained through training and testing on six features. We observed that the performance ratios obtained by training classifiers over a dataset containing 12 features increased after using feature selection algorithms. To compare our findings with similar studies, we created a comparison table. The results, as well as the ones reported in the literature, are summarized in Table 10.

Table 10. Comparison of the results with similar works in the literature.

Dataset Used	Feature Selection Evaluators	Machine Learning Algorithm	Accuracy (%)	Ref.
NSL-KDD DATASET (2009)	Without feature selection	SMO	97.31	[9]
		J48	98.78	
		NB	95.11	
	Genetic Alg.+ CFS	SMO	96.85	
		J48	98.33	
		NB	82.62	
	Greedy Alg. + CFS	SMO	95.71	
		J48	92.74	
		NB	83.92	
Their own dataset	Without feature selection	SVM	96.25	[10]
Their own dataset	Without feature selection	SVM	Average 95.24	[16]
LongTail	Without feature selection	C4.5 Bayesian Network DT Naive Bayes	Average 91.68	[18]
Their own dataset	Without feature selection	SOM LVQ	Average 94	[19]
CAIDA DDoS 2007	Without feature selection	SVM	92.9	[21]
Our dataset	Filter	SVM	92.46	
		KNN	97.15	
		ANN	92.28	
		NB	95.70	
	Wrapper	SVM	92.15	
		KNN	98.30	
		ANN	91.44	
		NB	94.87	
	Embedded	SVM	92.46	
		KNN	96.30	
		ANN	92.09	
		NB	95.09	

It can be seen from Table 10, that the proposed machine learning model with feature selection approaches, results in better performance in comparison with the results from [10,16,18,19,21]. It should be noted that similar studies in the literature used different datasets and different models. Therefore, justification of the comparison results is difficult.

Results show that the SDN structure proved successful in terms of detecting DDoS attacks with machine learning techniques. With the approaches to be planned on SDN architecture, a secure and efficient mechanism on the network can be developed. The performance of the controllers on large networks can be increased by catching the packets on the traffic and comparing them with the previously learned flow data of the packet. Another approach could be the use of a two-stage system in detecting DDoS attacks on the dataset. While the coming flow data is processed by the controller, it is copied to another module and analyzed by this module. To have any output on the network, the module has to inform the controller whether there is a problem in the coming data or not. Thus, while the controller performs only the network processes, the other module becomes an attack detection system.

In an SDN topology, multiple controllers in the SDN in a hierarchical structure can be located, as discussed in [26]. In SDN topology with a multi-controller structure, as the network traffic load is distributed among controllers, detection of DDoS attack traffic will be faster than that of a single controller. The location of the controllers in the network is also important at this point. Once the controllers are properly located in the network, the effectiveness of the controllers on the network

elements in the data plane will increase and delays will be decreased by optimizing resource utilization. However, in a network where multiple controllers are used, the data flow between controllers should not be interrupted for network service continuity.

In attacks such as DDoS, the attacker may target blocking the communication between controllers. SDN architecture may crash if communication between the controllers is blocked. In our study, a basic SDN topology with a single controller was used to generate DDoS attack traffic and to obtain the dataset, then to classify DDoS attacks with machine learning models, using this dataset. The use of a single-controller or multi-controller structure in the network topology at the stage of obtaining a dataset from SDN will not make a significant difference in methodology. The machine learning model that we have chosen, as the best among the models trained with the created dataset, can be applied to SDN topologies that contain both single- and multi-controllers.

5. Conclusions

In this study, SDN-based detection systems developed for DDoS attacks were analyzed with machine learning systems. In the first proposed approach, by analyzing flow data, algorithms with 98.3% accuracy ensure the detection of attacks without discriminating the type of traffic. Among the proposed systems, the second approach proceeds by labelling DDoS attacks as normal traffic and attack traffic. With 97.7% sensitivity, KNN algorithms can perform this control by lightening the burden of the controller.

With the feature selection methods used in the study, initially 12 features were selected and the selected subset of features was trained using classifiers. The number of features selected was determined either by the algorithm itself or by the threshold value given to the algorithm. By changing this threshold value, different numbers of features can be selected and trained by the classifier. Different numbers of features can change the accuracy. In general, we observed that the performance rate of all models is above 80% and the algorithms used for this dataset are successful. At the same time, network browsing, attacks between layers, and malicious software can be detected on SDN with this approach. For protecting and improving SDN structure, the second approach can be employed.

Author Contributions: Conceptualization, H.P. and A.C.; methodology, H.P. and O.P.; software, O.P.; validation, H.P., A.C. and O.P.; formal analysis, H.P. and O.P.; investigation, H.P. and A.C.; resources, O.P.; data curation, O.P.; writing—original draft preparation, H.P. and A.C.; writing—review and editing, H.P. and A.C.; visualization, A.C. and O.P.; supervision, H.P. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Open Networking Foundation. Software-Defined Networking: The New Norm for Networks. Available online: https://www.opennetworking.org/images/stories/downloads/sdn-resources/white-papers/wp-sdn-newnorm.pdf (accessed on 31 January 2020).
2. Kreutz, D.; Ramos, F.M.; Verissimo, P.E.; Rothenberg, C.E.; Azodolmolky, S.; Uhlig, S. Software-defined networking: A comprehensive survey. *Proc. IEEE* **2015**, *103*, 14–76. [CrossRef]
3. Xie, J.; Guo, D.; Hu, Z.; Qu, T.; Lv, P. Control plane of software defined networks: A survey. *Comput. Commun.* **2015**, *67*, 1–10. [CrossRef]
4. Suciu, G.; Vulpe, A. An Overview Study of Software Defined Networking. In Proceedings of the IE International Conference, Bucharest, Romania, 30 April–3 May 2015.
5. Kreutz, D.; Ramos, F.; Verissimo, P. Towards secure and dependable software-defined networks. In Proceedings of the Second ACM SIGCOMM Workshop on Hot Topics in Software Defined Networking, New York, NY, USA, 16 August 2013.
6. Shu, Z.; Wan, J.; Li, D.; Lin, J.; Vasilakos, A.V.; Imran, M. Security in software-defined networking: Threats and countermeasures. *Mob. Netw. Appl.* **2016**, *21*, 764–776. [CrossRef]

7. Ali, S.T.; Sivaraman, V.; Radford, A.; Jha, S. A survey of securing networks using software defined networking. *IEEE Trans. Reliab.* **2015**, *64*, 1086–1097. [CrossRef]
8. Kuerban, M.; Tian, Y.; Yang, Q.; Jia, Y.; Huebert, B.; Poss, D. FlowSec: DOS Attack Mitigation Strategy on SDN Controller. In Proceedings of the IEEE International Conference on Networking, Architecture and Storage (NAS), Long Beach, CA, USA, 8–10 August 2016.
9. Alshamrani, A.; Chowdhary, A.; Pisharody, S.; Lu, D.; Huang, D. A defense system for defeating DDoS attacks in sdn based networks. In Proceedings of the 15th ACM International Symposium on Mobility Management and Wireless Access, Miami Beach, FL, USA, 21–25 November 2017.
10. Latah, M.; Toker, L. A novel intelligent approach for detecting DoS flooding attacks in software-defined networks. *Int. J. Adv. Intell. Inform.* **2018**, *4*, 11–20. [CrossRef]
11. Li, C.; Wu, Y.; Yuan, X.; Sun, Z.; Wang, W.; Li, X.; Gong, L. Detection and defense of DDoS attack–based on deep learning in OpenFlow-based SDN. *Int. J. Commun. Syst.* **2018**, *31*, e3497. [CrossRef]
12. Li, W.; Meng, W.; Kwok, L.F. A survey on OpenFlow-based Software Defined Networks: Security challenges and countermeasures. *J. Netw. Comput. Appl.* **2016**, *68*, 126–139. [CrossRef]
13. Padmaja, S.; Vetriselvi, V. Mitigation of switch-Dos in software defined network. In Proceedings of the International Conference on Information Communication and Embedded Systems (ICICES), Chennai, India, 25–26 February 2016.
14. Polat, H.; Polat, O. The effect of DoS attack on ODL and POX SDN controllers. In Proceedings of the 8th International Conference on Information Technology (ICIT), Amman, Jordan, 17–18 May 2017.
15. Bholebawa, I.Z.; Dalal, U.D. Design and Performance Analysis of OpenFlow-Enabled Network Topologies Using Mininet. *Int. J. Comput. Commun. Eng.* **2016**, *5*, 419–429. [CrossRef]
16. Ye, J.; Cheng, X.; Zhu, J.; Feng, L.; Song, L. A DDoS attack detection method based on SVM in software defined network. *Secur. Commun. Netw.* **2018**, *2018*, 1–8. [CrossRef]
17. Xue, L.; Yuan, D.; Hu, H.; Ran, J.; Li, S. DDoS detection in SDN switches using support vector machine classifier. In Proceedings of the Joint International Mechanical, Electronic and Information Technology Conference (JIMET-15), Chongqing, China, 18–20 December 2015.
18. Nanda, S.; Zafari, F.; DeCusatis, C.; Wedaa, E.; Yang, B. Predicting network attack patterns in SDN using machine learning approach. In Proceedings of the 2016 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN), Palo Alto, CA, USA, 7–10 November 2016.
19. Jankowski, D.; Amanowicz, M. On efficiency of selected machine learning algorithms for intrusion detection in software defined networks. *Int. J. Electron. Telecommun.* **2016**, *62*, 247–252. [CrossRef]
20. Chakraborty, S.; Banerjee, S. Proposed approach to detect distributed denial of service attacks in software defined network using machine learning algorithms. *Int. J. Eng. Technol.* **2018**, *7*, 472–476.
21. Mowla, N.I.; Doh, I.; Chae, K. CSDSM: Cognitive switch-based DDoS sensing and mitigation in SDN-driven CDNi word. *Comput. Sci. Inf. Syst.* **2018**, *15*, 163–185. [CrossRef]
22. Hira, Z.M.; Duncan, F.G. A Review of Feature Selection and Feature Extraction Methods Applied on Microarray Data. *Adv. Bioinform.* **2015**, *2015*, 1–13. [CrossRef] [PubMed]
23. Venkatesh, B.; Anuradha, J. A Review of Feature Selection and Its Methods. *Cybern. Inf. Technol.* **2019**, *19*, 3–26. [CrossRef]
24. Stamp, M. *Introduction to Machine Learning with Applications in Information Security*; Chapman and Hall/CRC: Boca Raton, FL, USA, 2017.
25. Somol, P.; Novovicova, J.; Pudil, P. Improving Sequential Feature Selection Methods' Performance by Means of Hybridization. In Proceedings of the IASTED International Conference Advances in Computer Science and Engineering, Sharm El Sheikh, Egypt, 15–17 March 2010.
26. Mostafaei, H.; Menth, M.; Obaidat, M.S. A Learning Automaton-Based Controller Placement Algorithm for Software-Defined Networks. In Proceedings of the 2018 IEEE Global Communications Conference (GLOBECOM), Abu Dhabi, UAE, 9–13 December 2018.

© 2020 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (http://creativecommons.org/licenses/by/4.0/).

[1*]Srikanth Reddy Vutukuru
[2]Srinivasa Chakravarthi Lade

SecureIoT: Novel Machine Learning Algorithms for Detecting and Preventing Attacks on IoT Devices

JES Journal of Electrical Systems

Abstract: - This research paper deals with the enhancement of Internet of Things (IoT) security through federated ensemble learning (FEI) and adversarial machine learning (AMLI) algorithms. The proposed approach strives to enhance the security of IoT devices and networks against numerous cyber threats. To address the increasing threat of IoT, the study will utilize advanced machine learning techniques in order to build more resilient defenses for IoT networks. Using the IoT-23_Combined dataset, the study scrutinized an array of IoT attack types, revealing the distribution and frequency of different attack categories. The FEI and AMLI models were trained and tested to detect and mitigate these IoT attacks, with the evaluation metrics being accuracy, precision, recall, and F1 score. The FEI model surpassed the AMLI model, registering superior performance in all metrics, with an accuracy of 87.94% and an F1 score of 88.44%. Concurrently, the study recommends the adoption of system hardening strategies and real-time response measures, including robust authentication methods, secure firmware updates, and security awareness training programs. The assessment of these measures was based on their implementation complexity, potential impact on system performance, and adaptability to changing threats. Conclusively, the research findings illustrate the potential of machine learning to bolster IoT security, suggesting a path towards more proactive defense mechanisms and the creation of a resilient IoT ecosystem. This study holds relevance for entities seeking to fortify their IoT security systems against an evolving threat landscape.

Keywords: Internet of Things (IoT), Machine Learning, Federated Ensemble Learning (FEI), Adversarial Machine Learning (AMLI), IoT Security, Cyber Attacks

I. INTRODUCTION

Internets of Things (IoT) devices have brought about a revolution in several domains. From smart homes and healthcare to industrial automation and transportation, there are a range of domains. IoT devices, specifically sensors, actuators, and smart objects, are now firmly ingrained in our daily lives. They provide convenience, efficiency, and automation. However, the widespread adoption of IoT technology [1] has also introduced significant security challenges. IoT devices are characterized by their interconnectedness, embedded sensors, and communication capabilities, enabling them to collect and exchange vast amounts of data [2]. However, these devices often possess limited computing resources, exhibit heterogeneity in terms of hardware and software, and operate in diverse environments. These factors make IoT devices particularly vulnerable to attacks [3]. The increasing adoption of Internet of Things (IoT) devices has revolutionized various industries by enabling efficient and interconnected systems.

However, this rapid expansion has also made IoT devices an attractive target for malicious attacks. These attacks can compromise the security and privacy of individuals, disrupt critical infrastructure, and lead to significant financial losses [4]. Therefore, there is a pressing need to develop effective mechanisms for detecting and preventing attacks on IoT devices.

The motivation for writing this research paper was the susceptibility of IoT devices to cyber attacks. As IoT devices continue to gain popularity in homes, businesses, and public infrastructure, securing them becomes increasingly essential. Malicious actors often employ ever-evolving attack techniques that render traditional security measures insufficient [5]. Hence, exploring and evaluating machine learning algorithms expertly crafted for detecting and preventing cyberattacks targeted at IoT devices is paramount. Developing intelligent systems capable of analyzing vast amounts of real-time IoT device data becomes possible by using machine learning techniques. Patterns, anomalies, and potential threats that are invisible to traditional rule-based techniques could be exposed by means of these algorithms. Furthermore, the models of machine learning possess the competence to adapt and gain knowledge from fresh attack vectors.

[1*] Corresponding author : Research Scholar, Department of Computer Science and Engineering, GITAM University, Visakhapatnam, Andhra Pradesh, India. Email ID: srireddy.sv@gmail.com
[2]Assistant Professor, Department of Computer Science and Engineering, GITAM University, Visakhapatnam, Andhra Pradesh, India.Email ID: chakri.ls@gmail.com
Copyright © JES 2023 on-line : journal.esrgroups.org

A defense mechanism that is proactive in nature guards against potential threats. Robust machine learning algorithms can greatly benefit IoT device security when successfully implemented. IoT devices [7] may increase trust and confidence among consumers, organizations, and governments. Moreover, it aids in avoiding possible disturbances to essential services and securing confidential data. Less harm to finances and reputation from cyber-attacks on IoT infrastructure can be achieved by mitigating them. Detecting and preventing attacks on IoT devices requires the exploration and evaluation of appropriate machine learning algorithms, which are crucial. The resolution to the rapidly expanding IoT ecosystem's security challenges is being addressed.

The increasing connectivity and integration of IoT devices into critical infrastructure and personal lives necessitate robust security measures. The consequences of successful attacks on IoT devices can be severe, ranging from privacy breaches and service disruptions to compromised infrastructure and the formation of large-scale botnets. Detecting and preventing attacks on IoT devices is of paramount importance for several reasons [8]. First, protecting the privacy of users' personal data and sensitive information is essential to maintaining trust in IoT ecosystems. Second, ensuring the uninterrupted operation of IoT devices and preventing service disruptions is critical, especially in sectors like healthcare, energy, and transportation where lives and infrastructure are at stake. Lastly, safeguarding IoT devices against attacks is crucial to prevent their exploitation as botnet nodes, which can be leveraged for large-scale cyber-attacks.

Machine learning algorithms have emerged as powerful tools for enhancing the security of IoT devices [9]. These algorithms leverage data-driven techniques to detect anomalies, identify patterns, and make intelligent decisions in real-time, enabling proactive threat detection and prevention. Machine learning algorithms can effectively complement traditional rule-based approaches and signature-based methods, as they have the capability to adapt to evolving attack techniques and detect previously unknown threats. There are various machine-learning algorithms that have been applied to IoT security. Supervised learning algorithms, such as Support Vector Machines (SVM) [10], Random Forest, Naive Bayes, and K-Nearest Neighbors (KNN), can be trained on labeled datasets to classify normal and malicious IoT device behavior. Unsupervised learning algorithms, including K-Means clustering, isolation forests, one-class support vector machines (SVM), and autoencoders, are effective for anomaly detection by identifying patterns and deviations from normal behavior. Reinforcement learning algorithms, such as Deep Q-Network (DQN) [11], Proximal Policy Optimization (PPO) [12], and Monte Carlo Tree Search (MCTS) [13], can be employed for adaptive and dynamic defense mechanisms in IoT environments.

These machine learning algorithms offer the potential to significantly enhance the security of IoT devices by effectively detecting and preventing attacks, mitigating risks, and providing early warning systems. However, selecting the most appropriate algorithms for IoT security requires careful evaluation, considering factors such as their effectiveness, scalability, computational requirements, and adaptability to the dynamic nature of IoT environments. The research paper aims to explore and evaluate the most suitable machine learning algorithms for detecting and preventing attacks on IoT devices. The main contributions include advancing IoT security with novel machine learning algorithms, evaluating the performance and comparison of these new algorithms, and proposing effective prevention measures after identifying potential threats.

Main Contribution of this research paper is

1. **Advancing IoT Security with Novel Machine Learning Algorithms:** Our focus is to create and evaluate three innovative algorithms - Federated Ensemble Learning and Adversarial Machine Learning - specifically designed to detect and neutralize a range of IoT attacks.
2. **Evaluating Algorithm Performance and Comparison:** Our main objective is to assess and compare the effectiveness of our newly developed algorithms in terms of accuracy, precision, recall, and computation time. By comparing them to existing approaches, we aim to showcase their potential for detecting and mitigating IoT attacks.
3. **Proposing Effective Prevention Measures**: After identifying potential threats using our new machine learning algorithms, our objective is to propose real-time responses and system hardening strategies to protect IoT devices. We will extensively analyze and discuss the effectiveness of these preventive measures.

Paper organization is as follows section 2 presents related work, section 3 presents methodology, section 4 presents result and analysis and section 5 concludes the paper with future work .

II. RELATED WORK

The related works in the field of IoT security have identified the growing threats and vulnerabilities associated with IoT devices due to their widespread deployment without proper security measures. These works have highlighted the need for effective detection and prevention mechanisms to mitigate attacks on IoT devices. However, existing approaches often lack robustness and scalability in addressing the evolving nature of IoT attacks.

This research [14] addresses the growing need for secure communication and data confidentiality in the Internet of Things (IoT) due to its expanding user and device base. Traditional identity and access management methods are replaced with a lightweight and scalable mechanism leveraging IOTA, a distributed ledger technology, and Inter Planetary File System (IPFS) to decentralize storage and avoid single point failures. Localized computing is facilitated by fog nodes, while unauthorized data access is mitigated by a trusted access control mechanism for enhanced IoT and network security. Simulation experiments validate the security and functional efficacy of the proposed scheme.

This research [15] focuses on enhancing IoT device security amidst growing privacy concerns and an increasing number of hybrid devices. A top-ranked authentication feature is identified as a primary requirement for protection against unauthorized access. The challenge lies in addressing the authentication feature's incompatibility with various IoT devices. The study explores AI and machine learning implementations for vulnerability detection and uses the COPRAS approach to evaluate significant features, thereby aiding organizations in improving their IoT device security.

This paper [16] proposes a malware detection and prevention methodology for IoT devices, utilizing a Deep LSTM classifier for detection, and an Improved Elliptic Curve Cryptography (IECC) algorithm for prevention. The system identifies attack nodes using a trust value and contextual features, further predicting various types of network attacks. Hybrid MA-BW optimization is employed for optimal key selection during data transmission. With 95% accuracy, 92% precision, and reduced execution time, the approach outperforms existing techniques, enhancing IoT data transmission security.

The ubiquity of IoT networks [17], despite their resource constraints, makes them vulnerable to sophisticated attacks such as the Wormhole routing attack. Current mitigation strategies range from rule-based to deep learning-assisted approaches. This research proposes a cascaded wormhole detection technique for IoT networks utilizing federated deep learning and a Dynamic Trust Factor. The technique offers high accuracy and is lightweight, ensuring data security and privacy.

This paper [18] addresses the issue of cyber-attacks in Cyber-Physical Systems (CPSs) caused by insecure networking devices. The integration of Fog with IoT is proposed as a solution to detect attacks in CPS more efficiently compared to cloud-based CPS. A swarm-based feature selection algorithm called Enhanced Chicken swarm optimization (ECSO) is introduced to enhance attack detection in an IoT-based CPS environment. ECSO utilizes self-learning ability to select relevant features from preprocessed data. The selected features are then used with ensemble classifiers executed on the cloud. The proposed ECSO-based ensemble classifier is evaluated using the NSL-KDD dataset, demonstrating its effective performance through various statistical measures.

In response to the increasing severity of security issues in IoT devices[19], traditional security software and network-based traffic detection are limited in their effectiveness. To address this urgent problem, we propose IoT-DeepSense, a behavioral security detection system for IoT devices. By employing firmware virtualization and deep learning, IoT-DeepSense captures fine-grained system behaviors and utilizes an LSTM-based abnormality detection approach to extract hidden features and detect abnormal behaviors. Implemented on an independent IoT behavior detection server, IoT-DeepSense does not require modifications to the limited resources of IoT devices and offers scalability. Evaluation results demonstrate a high detection rate of 92% while minimally impacting device performance.

This paper [20] highlights the importance of IoT security and the potential risks associated with malicious insider attacks. It emphasizes the need for effective security measures to protect IoT devices and the sensitive data they handle. The research focuses on using artificial intelligence (AI) to detect insider attacks in IoT environments, specifically addressing the challenge of internal exploitation within IoT networks. A lightweight approach is proposed for detecting anomalies originating from incoming data sensors in resource-constrained IoT environments. The results demonstrate that the proposed approach outperforms existing methods in terms of improved attack detection accuracy, reduced false positives, and minimized computational overhead.

The paper [21] highlights the widespread deployment of IoT devices without proper security measures, leading to increased security incidents and the potential for malicious attacks. The proposed study aims to address these

vulnerabilities and minimize threats in IoT devices to enhance the overall security of the IoT service environment. The focus is on improving security by design and mitigating risks associated with compromised devices, such as Mirai Botnet, which can be exploited for DDoS attacks. By implementing the proposed scheme, the study aims to minimize security vulnerabilities and enhance the security of IoT services.

Table 1: Summary of the related work

Research	**Methodology**	**Key Findings**
[14]	Leveraging distributed ledger technology and decentralized storage	Enhanced IoT security with security and functional efficacy
[15]	Focus on authentication and AI/machine learning for vulnerability detection	Improved IoT device security and support for organizations
[16]	Deep LSTM classifier and Improved Elliptic Curve Cryptography algorithm	Outperformed existing techniques for malware detection and prevention
[17]	Cascaded wormhole detection using federated deep learning and Dynamic Trust Factor	Ensured data security and privacy in IoT networks
[18]	Integration of Fog with IoT, swarm-based feature selection, and ensemble classifiers	Enhanced attack detection in Cyber-Physical Systems
[19]	IoT-DeepSense utilizing firmware virtualization and deep learning	Achieved high detection rates for IoT device behavioral security
[20]	AI-based detection of insider attacks in IoT environments	Improved attack detection accuracy and reduced false positives
[21]	Emphasis on security by design and mitigation of risks	Aims to minimize security vulnerabilities and threats in IoT devices

Research Challenges:

- Lack of robustness and scalability in existing approaches to address evolving IoT attacks.
- Compatibility of authentication features with various IoT devices.
- Optimizing key selection during data transmission for malware detection and prevention.
- Detecting sophisticated attacks like the Wormhole routing attack in resource-constrained IoT networks.
- Efficient attack detection in Cyber-Physical Systems using IoT and Fog integration.
- Extracting hidden features and detecting abnormal behaviors in IoT devices.
- Detection of insider attacks in IoT networks with limited resources and minimized computational overhead.
- Minimizing security vulnerabilities and threats in IoT devices without compromising performance.

Related research emphasizes the necessity of implementing effective detection and prevention mechanisms to tackle the growing security issues in IoT devices. By utilizing distributed ledger technology, machine learning, deep learning, and behavioral analysis anyone can improve the security of IoT. Minimizing security threats and vulnerabilities while concentrating on secure design for IoT devices are the primary goals of the intended work. In addition, it tries to decrease the dangers linked with compromised devices. Insights and potential strategies are offered by the related work's findings. The proposed work can incorporate these to develop novel machine learning algorithms for detecting and preventing attacks on IoT devices.

III. METHODOLOGY

The increasing prevalence of Internet of Things (IoT) devices has introduced new security challenges due to their interconnected nature. To address these challenges, this paper focuses on advancing IoT security through the development and evaluation of novel machine learning algorithms. The methodology consists of three main components. Firstly, in the "Advancing IoT Security with Novel Machine Learning Algorithms" section, we focus on the algorithm design phase. Here, we develop three novel algorithms, each uniquely suited for IoT attack detection and mitigation. Federated ensemble learning leverages distributed learning across IoT devices; deep transfer learning addresses heterogeneity in device types; and adversarial machine learning enhances resilience against sophisticated attacks.

Next, in the "Evaluating Algorithm Performance and Comparison" section, we establish performance metrics, including accuracy, precision, recall, and computation time. A diverse dataset comprising various IoT attack scenarios and normal IoT traffic is collected for the performance analysis. The experimental setup is described, encompassing hardware and software configurations, and baseline methods are specified for comparison. The performance analysis assesses and compares the effectiveness of the new algorithms against existing approaches in detecting and mitigating IoT attacks. Finally, in the "Proposing Effective Prevention Measures" section, we apply the developed machine learning algorithms to identify potential threats and classify them into different attack types. Building upon the detected threats, we propose real-time response mechanisms and system hardening strategies to enhance the security of IoT devices. The efficacy of these prevention measures is thoroughly analyzed and discussed, considering their ability to mitigate the identified threats and protect IoT devices from attacks.

3.1 Advancing IoT Security with Novel Machine Learning Algorithms

With the rapid growth of IoT devices, ensuring their security has become a critical concern. This paper presents a comprehensive methodology for advancing IoT security through the development and evaluation of novel machine learning algorithms.

- **Algorithm Design**: Develop the Federated Ensemble Learning and Adversarial Machine Learning algorithms tailored specifically for IoT cyber-attack detection and neutralization.
- **Implementation**: Implement the designed algorithms using appropriate machine learning frameworks and libraries.
- **Dataset Selection**: Choose relevant datasets containing IoT attack instances for training and evaluation.
- **Training and Evaluation**: Train the algorithms using the selected datasets, evaluate their performance, and fine-tune them for optimal results.

Federated Ensemble Learning for IoT (FEI):

- "IoT Device" represents the individual IoT devices participating in the federated learning process.
- "Federated Learner" is responsible for coordinating the training process and collecting local model updates from the IoT devices.
- "Aggregator" aggregates the model updates received from the devices and updates the ensemble model.
- "Ensemble Model" represents the combined model that utilizes the contributions from all the IoT devices.

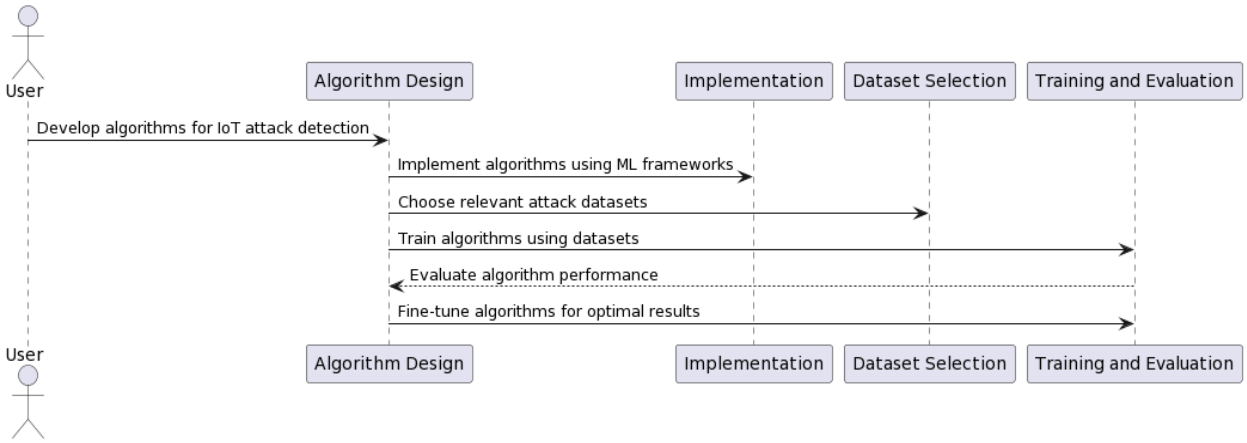


Figure 1: Sequence Diagram

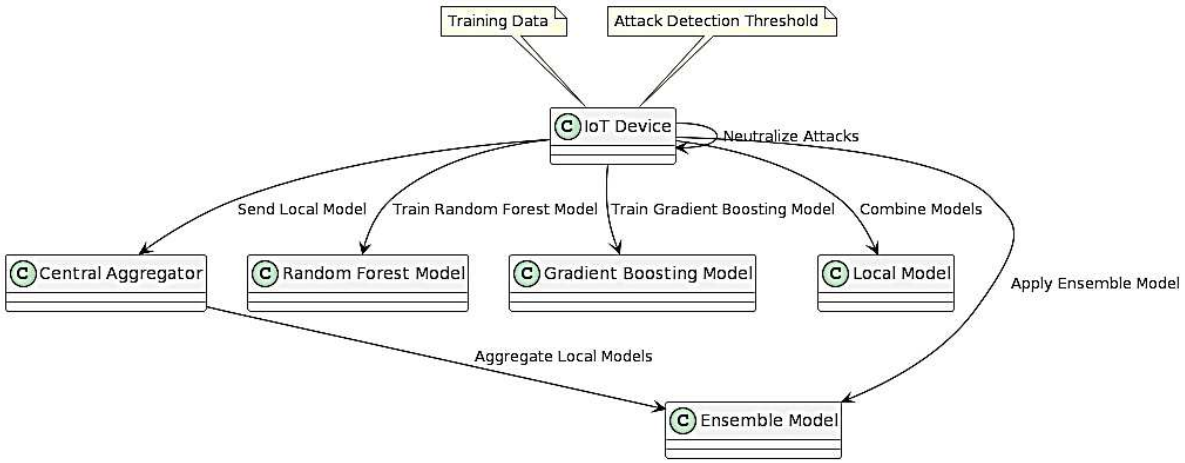


Figure 2: Basic Design model of Federated Ensemble Learning for IoT (FEI)

Algorithm: Federated Ensemble Learning for IoT Attack Detection and Neutralization

Input:

- Set of IoT devices $D = \{D1, D2, \ldots, Dn\}$
- Training data for each device Di: $Data_i$
- Attack detection threshold: Threshold

Output:

- Trained ensemble model

1. Initialize the ensemble model:
   - EnsembleModel ← Empty ensemble model
2. for each IoT device Di in D do:
   - Train Random Forest model:
     - $RF_{Model_i} \leftarrow TrainRandomForest(Data_i)$
   - Train Gradient Boosting model:
     - $GB_{Model_i} \leftarrow TrainGradientBoosting(Data_i)$
   - Update local model:
     - $LocalModel_i \leftarrow CombineModels(RF_{Model_i}, GB_{Model_i})$

- Di sends LocalModel_i to the central aggregator

3. Aggregator aggregates the local models:

- $EnsembleModel \leftarrow AggregateModels(LocalModel_1,\ LocalModel_2, \dots, LocalModel_n)$

4. for each IoT device Di in D do:

- Di applies the ensemble model for attack detection:
  - $DetectedAttacks_i \leftarrow ApplyEnsembleModel(Di, EnsembleModel,\ Threshold)$
- Di performs attack neutralization:
  - $NeutralizeAttacks(Di, DetectedAttacks_i)$

5. Output the trained ensemble model: EnsembleModel

In this algorithm, we utilize Federated Ensemble Learning to train an ensemble model using Random Forest and Gradient Boosting algorithms. Each IoT device trains its own Random Forest and Gradient Boosting models using its local training data. The local models are then combined to create a local model for each device. The local models are sent to the central aggregator, which aggregates them to create the ensemble model.

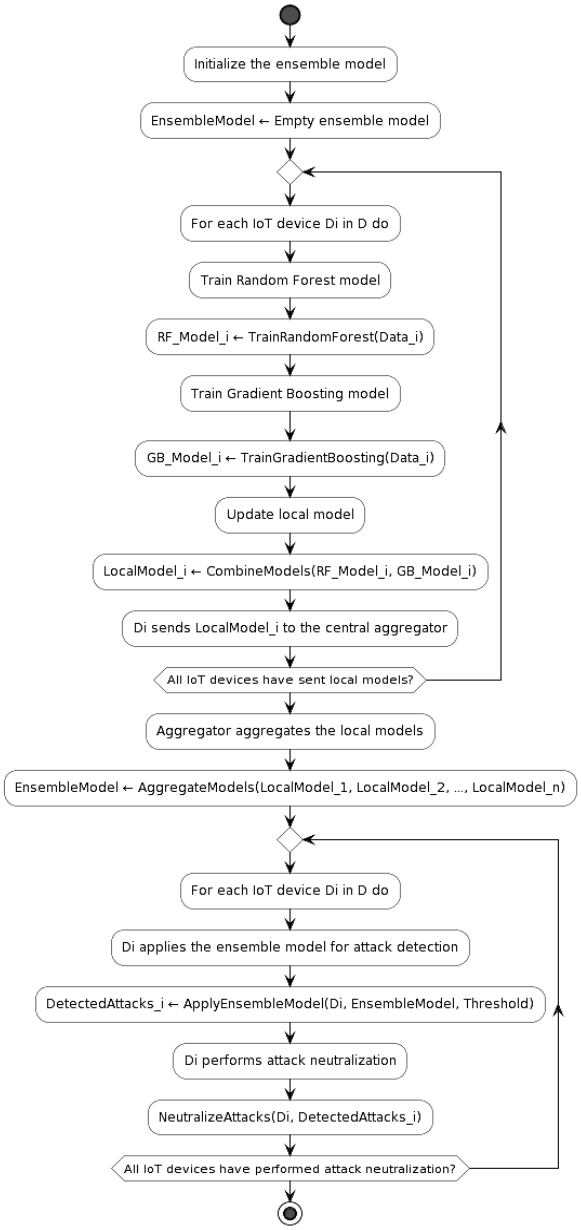


Figure 3: Flow chart for Federated Ensemble Learning for IoT (FEI)

Explanation of the flowchart for the "Federated Ensemble Learning for IoT (FEI)" algorithm from the as shown in figure 3.

1. The flowchart begins with the "Initialize the ensemble model" step, where an empty ensemble model is created.
2. The algorithm then enters a loop that iterates over each IoT device Di in the set of IoT devices D.
3. For each IoT device Di, the device trains a Random Forest model (RF_{Model_i}) and a Gradient Boosting model (GB_{Model_i}) using its training data ($Data_i$).
4. The local models (RF_{Model_i} and GB_{Model_i}) are combined to create a local model specific to the IoT device ($LocalModel_i$).
5. The IoT device Di sends its $LocalModel_i$ to the central aggregator.
6. The central aggregator aggregates the received local models ($LocalModel_1$, $LocalModel_2$, ..., $LocalModel_n$) to form the ensemble model (EnsembleModel).
7. The algorithm then enters another loop that iterates over each IoT device Di in the set of IoT devices D.
8. For each IoT device Di, the device applies the ensemble model (EnsembleModel) for attack detection.
9. The detected attacks ($DetectedAttacks_i$) are identified based on the application of the ensemble model.
10. The IoT device Di performs attack neutralization by taking appropriate actions to mitigate the detected attacks (NeutralizeAttacks).
11. The algorithm repeats steps 7-10 until all IoT devices have performed attack neutralization.
12. Once all the IoT devices have completed attack neutralization, the algorithm stops, and the trained ensemble model (EnsembleModel) is the output of the algorithm.

In summary, the flowchart depicts the sequential steps of the "Federated Ensemble Learning for IoT (FEI)" algorithm. It starts with initializing the ensemble model, proceeds with training local models on each IoT device, aggregates the local models to form the ensemble model, applies the ensemble model for attack detection on each IoT device, and performs attack neutralization. The process continues until all IoT devices have completed attack neutralization, resulting in the trained ensemble model as the output.

Once the ensemble model is established, each IoT device applies the model for attack detection, determining potential attacks based on the specified threshold. If an attack is detected, the device proceeds to neutralize the attack using appropriate measures. The algorithm outputs the trained ensemble model, which can be used for future attack detection and neutralization on IoT devices.

LocalTraining(Di, M): The Local Training function represents the local training process performed by each IoT device Di using its local data and the current ensemble model M. The specific training algorithm [26] employed at each device may vary based on the problem at hand. Let's represent the local model of device Di at round t as $Mi(t)$. The update equation for the local model at each round can be represented as:

$$Mi(t) = TrainLocalModel(Di, M) \quad (1)$$

Here, $TrainLocalModel()$ is the specific training algorithm used by each device to update its local model based on its local data and the ensemble model M.

ComputeWeights(Di): The ComputeWeights function computes the local model weights for each IoT device Di. These weights determine the contribution of each device to the ensemble model[27]. Let's represent the weight for device Di at round t as $Wi(t)$. The weight computation equation can be expressed as:

$$Wi(t) = ComputeWeight(Di) \quad (2)$$

The specific method to compute the weight for each device may vary depending on various factors such as the performance of the local model, the amount of data, or the reliability of the device. $ComputeWeight()$ is a function that calculates the weight based on these considerations.

AggregateWeights (W1, W2, ..., Wn):

The AggregateWeights function aggregates the local model weights received from all IoT devices to calculate the average weight that will be used to update the ensemble model. The aggregation equation can be represented as:

$$Wavg(t) = \frac{(W1(t) + W2(t) + \ldots + Wn(t))}{n} \qquad (3)$$

Here, n represents the total number of IoT devices participating in the federated learning process, and $Wavg(t)$ denotes the average weight calculated at round t.

UpdateEnsembleModel(M, Wavg): The UpdateEnsembleModel function updates the ensemble model M using the average weight $Wavg$ computed by the aggregator. The update equation can be expressed as:

$$M(t+1) = UpdateModel\big(M(t), Wavg(t)\big) \qquad (4)$$

The specific method for updating the ensemble model based on the average weight may vary. $UpdateModel()$ is a function that combines the contributions of the individual models according to the weight, resulting in an updated ensemble model.

Sequence diagram:

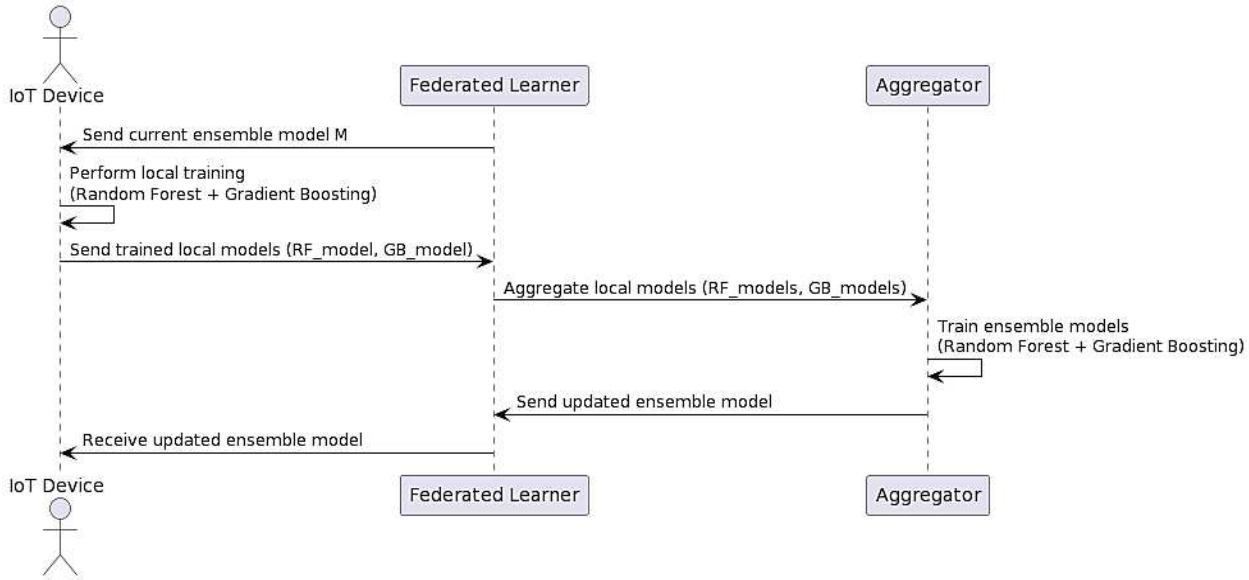


Figure 4: Sequence diagram of (FEI)

Detailed explanation of the sequence diagram as shown in figure 4 for the "Federated Ensemble Learning for IoT Attack Detection and Neutralization" algorithm:

1. The sequence diagram starts with the "IoT Device" actor, representing an individual IoT device participating in the federated learning process.
2. The "Federated Learner" participant sends the current ensemble model M to the IoT device.
3. The "IoT Device" participant receives the ensemble model and performs local training using the Random Forest and Gradient Boosting algorithms.
4. After the local training, the "IoT Device" participant sends the trained local models (RF_{model} and GB_{model}) back to the "Federated Learner".
5. The "Federated Learner" participant receives the trained local models from the IoT device.
6. The "Federated Learner" forwards the received local models to the "Aggregator".
7. The "Aggregator" participant aggregates the received local models, forming ensemble models for both Random Forest (RF_{models}) and Gradient Boosting (GB_{models}).
8. The "Aggregator" trains ensemble models using the aggregated local models, resulting in ensemble models for Random Forest ($RF_{ensemble_{model}}$) and Gradient Boosting ($GB_{ensemble_{model}}$).
9. The "Aggregator" sends the updated ensemble model ($Ensemble_{model}$) back to the "Federated Learner".
10. The "Federated Learner" participant receives the updated ensemble model from the "Aggregator".

11. The "Federated Learner" distributes the updated ensemble model to the "IoT Device" for the next round of the federated learning process.

This sequence diagram as shown in figure 4 illustrates the communication and interaction flow between the IoT devices, the federated learner, and the aggregator in the Federated Ensemble Learning process for IoT Attack Detection and Neutralization. It showcases how the ensemble model is exchanged, local training is performed, and the updated ensemble model is shared among the participants in each communication round.

The FEI algorithm uses a federated learning technique. The main objective is constructing an ensemble model that detects and mitigates attacks on IoT[25]. The process of training random forest and gradient boosting models on IoT devices gave rise to this model. With aggregation and combination, these models form the ensemble model. The IoT devices detect and mitigate attacks by using this ensemble model.

3.2 Design process:

Design: Adversarial Machine Learning for IoT (AMLI)

To withstand the growing sophistication of attacks, it is vital to design a method that can defend against adversarial attacks. Adversarial machine learning (AMLI) algorithms are useful for enhancing IoT device security via adversarial sample generation and model training for identification. Including defenses to thwart such attacks is also paramount. The steps given below outline the design process:

1. Initialization:
    - Select suitable deep learning model architecture, such as a convolutional neural network (CNN), for IoT attack detection.
    - Pre-train the initial model on a large-scale dataset to learn general features and patterns.
2. Adversarial Sample Generation:
    - Employ adversarial sample generation techniques, such as Fast Gradient Sign Method (FGSM), DeepFool, or other similar methods, to create adversarial samples specifically tailored to IoT attack scenarios.
    - These techniques modify input data imperceptibly to deceive the model's classification, aiming to trick the model into misclassifying benign or malicious IoT traffic.
3. Adversarial Training:
    - Introduce the generated adversarial samples into the training dataset.
    - Train the model to recognize and correctly classify these adversarial samples, incorporating them into the learning process.
    - By exposing the model to adversarial samples during training, it becomes more robust and less susceptible to adversarial attacks.
4. Defense Mechanisms:
    - Implement defense mechanisms within the model architecture to enhance resilience against adversarial attacks.
    - Techniques like adversarial training, defensive distillation, or randomized smoothing can be employed to improve the model's resistance to adversarial perturbations.
5. Model Evaluation:
    - Assess the performance of the AMLI model on a separate evaluation dataset, including both benign and adversarial samples.
    - Measure key metrics such as accuracy, precision, recall, and F1-score to evaluate the model's ability to detect and neutralize a range of IoT attacks, including both traditional and adversarial attacks.
6. Continuous Learning and Improvement:
    - Deploy the AMLI model in real-time IoT environments for ongoing monitoring and detection of IoT attacks.

- Continuously collect and analyze IoT network traffic data to identify new attack patterns and generate updated adversarial samples.
- Re-train the model periodically, incorporating these new samples and further improving the model's detection capabilities over time.

7. Response and Mitigation:
   - Upon detecting an attack, trigger appropriate response mechanisms such as blocking suspicious traffic, raising alerts, or invoking countermeasures.
   - Implement system hardening techniques to strengthen the security of IoT devices, networks, and protocols against detected and potential adversarial attacks.

The design of Adversarial Machine Learning for IoT (AMLI) incorporates the generation of adversarial samples, training the model to recognize and classify them correctly, and incorporating defense mechanisms to enhance the model's robustness against adversarial attacks. By continuously learning from and adapting to these attacks, the AMLI model improves its detection capabilities, enabling effective mitigation of both traditional and adversarial IoT attacks[24].

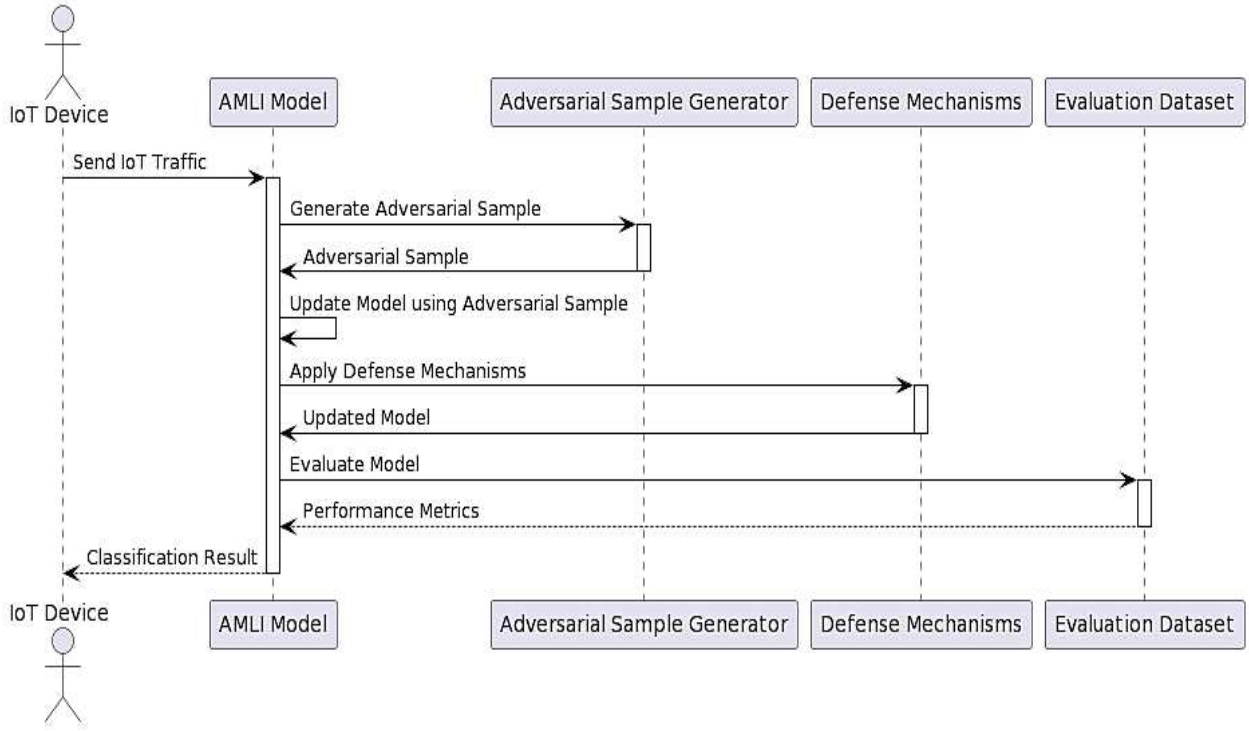


Figure 5: Sequence diagram of AMLI model

Algorithm 2: Adversarial Machine Learning for IoT (AMLI)

Input:

- Legitimate IoT dataset D_{legit}
- Number of adversarial samples N_{adv}
- Adversarial attack method A (e.g., $FGSM, DeepFool$)
- Defense mechanism D (e.g., adversarial training, defensive distillation)

Output:

- Trained AMLI model M

1. **Adversarial Sample Generation**:
   - Generate adversarial samples:
     - $for\ i\ =\ 1\ to\ N_{adv} do$

- Select a legitimate IoT sample x from D_{legit}
- Perturb x using adversarial attack method A: $x_{adv} = A(x)$
- Add $(x_{adv}, label(x))$to the dataset D_adv (adversarial samples dataset)

2. **Training with Adversarial Samples**:
   - Combine legitimate and adversarial datasets:
     - $D_{combined} = D_{legit} \cup D_{adv}$
   - Train the AMLI model M using the combined dataset:
     - $M = TrainModel(D_{combined})$
3. **Defense Mechanisms**:
   - Apply defense mechanism D to enhance model resilience against adversarial attacks:
     - $M_{defended} = ApplyDefense(M, D)$
4. **Model Evaluation**:
   - Evaluate the performance of the AMLI model $M_{defended}$:
     - Perform evaluation on a separate evaluation dataset D_{eval}
     - Compute metrics (accuracy, precision, recall, F1-score) for both legitimate and adversarial IoT samples
     - Assess the model's robustness against known attack methods
5. **Continual Learning**:
   - Continuously update the AMLI model $M_{defended}$:
     - Collect new adversarial samples periodically
     - Add the new samples to D_{adv}
     - Re-train the model using the updated dataset $D_{combined}$
     - Apply defense mechanism D to the updated model: $M_{defended} = ApplyDefense(M, D)$

Output the trained and defended AMLI model: $M_{defended}$

This algorithm leverages adversarial attack methods to generate adversarial samples, which are then combined with legitimate IoT data for training the AMLI model. Defense mechanisms are applied to enhance the model's resilience against adversarial attacks. The model is evaluated using separate evaluation datasets, and continual learning is implemented to adapt the model to evolving adversarial techniques over time.

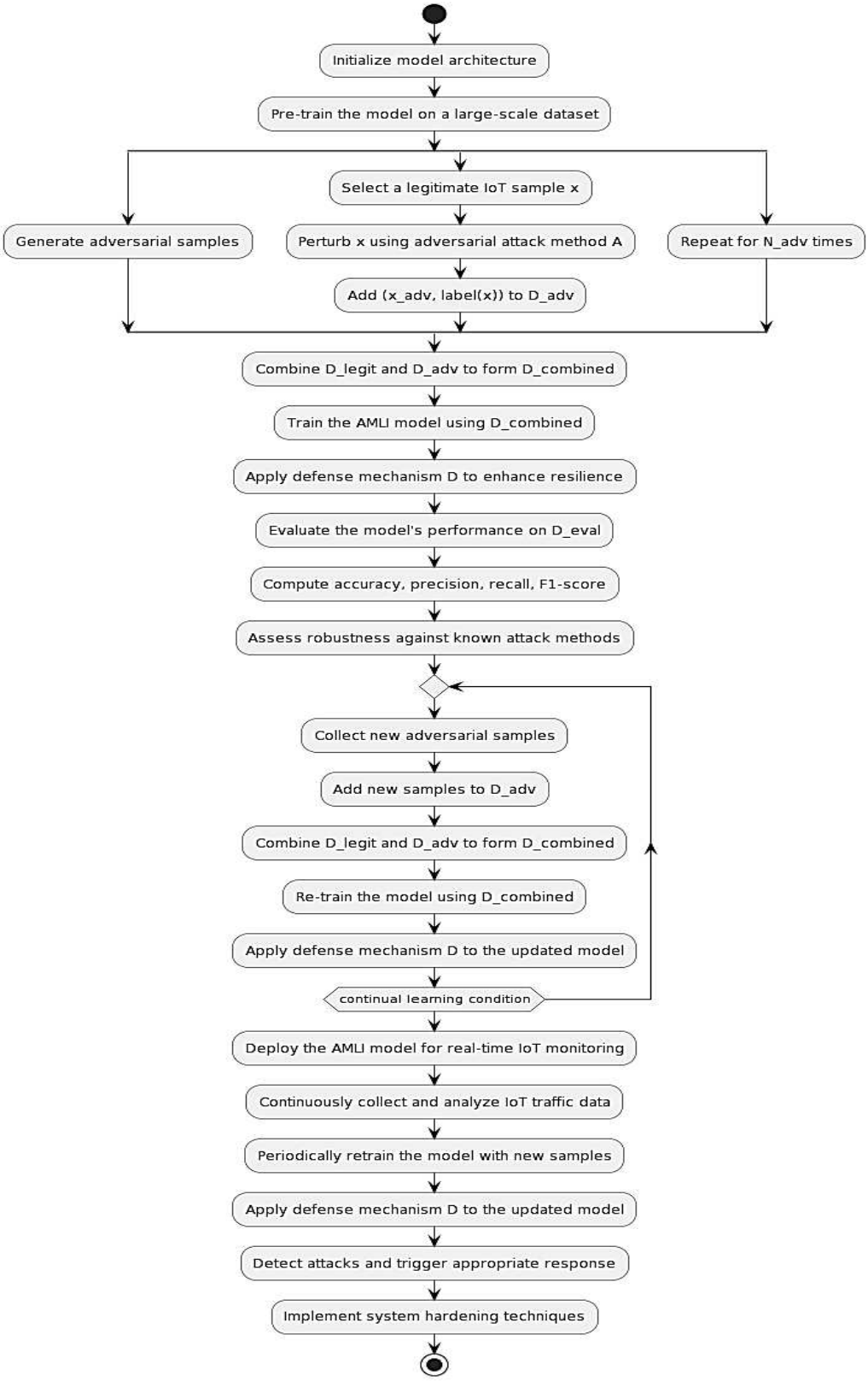


Figure 6: Flow chart of AMLI model

In summary, the flowchart as shown in figure 6 depicts the sequential steps of the "Federated Ensemble Learning for IoT (FEI)" algorithm. It starts with initializing the ensemble model, proceeds with training local models on each IoT device, aggregates the local models to form the ensemble model, applies the ensemble model for attack detection on each IoT device, and performs attack neutralization. The process continues until all IoT devices have completed attack neutralization, resulting in the trained ensemble model as the output.

IV. RESULT AND ANALYSIS

4.1 Performance Metrics

Performance metrics assess the algorithm's effectiveness in detecting and neutralizing IoT attacks It records various attributes of the algorithm's functionality, including its overall correctness and accuracy in detecting true positives. Capturing avoidance of false positives and false negatives is also done. With these metrics in mind, researchers and practitioners can determine the effectiveness and efficiency of novel machine-learning algorithms like federated ensemble learning and adversarial machine-learning for enhancing IoT security. The outcome of this can be stronger security measures that offer protection against any possible threats.

Performance metrics with their suitable formulas:

1. **Accuracy**:

$$Formula: Accuracy = \frac{(True\ Positives + True\ Negatives)}{(True\ Positives + True\ Negatives + False\ Positives + False\ Negatives)} \quad (5)$$

2. **Precision**:

$$Formula: Precision = \frac{True\ Positives}{(True\ Positives + False\ Positives)} \quad (6)$$

3. **Recall**:

$$Formula: Recall = \frac{True\ Positives}{(True\ Positives + False\ Negatives)} \quad (7)$$

4. **F1-score**:

$$Formula: F1 - score = 2 * \frac{(Precision * Recall)}{(Precision + Recall)} \quad (8)$$

These metrics provide quantitative measures of how well the algorithms are able to detect and mitigate IoT attacks.

4.2 Data Collection: IoT-23 Dataset Overview

The IoT-23 dataset[22], first published in January 2020 and featuring captures from 2018 to 2019, is a significant resource in IoT security research. Originating from the Stratosphere Laboratory of the AIC group at FEL, CTU University in the Czech Republic, this dataset encompasses network traffic data from various Internet of Things (IoT) devices. It includes 20 malware captures from IoT devices and 3 captures of benign IoT device traffic, providing a comprehensive mix of real, labeled IoT malware infections and normal IoT traffic.

IoT-23_Combined, an extended version of this dataset, integrates these individual datasets to offer a more holistic view of IoT security scenarios. It covers a broad spectrum of IoT attack types, such as Denial of Service (DoS), Man-in-the-Middle (MitM), botnet, physical, firmware, information leakage, replay, spoofing, side-channel, and device tampering attacks. The dataset serves as a robust tool for researchers to develop and evaluate machine learning algorithms aimed at identifying and mitigating IoT attacks. It includes both benign traffic from standard IoT devices and malicious traffic generated through simulated attacks, thereby offering a versatile platform for analyzing IoT security. The creation and maintenance of the IoT-23 dataset have been funded by Avast Software, with a specific emphasis on allowing malware in the dataset to connect to the Internet, thereby simulating real-world IoT network conditions.

4.3 Experimental Setup: For conducting the experiments, the following hardware configuration was utilized: CPU: Intel Core i7-8700K (6 cores, 12 threads) ,GPU: NVIDIA GeForce RTX 2080 Ti (11 GB VRAM) ,Memory: 32 GB DDR4 RAM, Storage: 1 TB SSD and The experimental environment was set up using the following software configuration: Operating System: Ubuntu 18.04 LTS ,Programming Language: Python 3.7 ,Machine Learning Libraries: TensorFlow 2.5, scikit-learn 0.24, Keras 2.4 ,Deep Learning Framework: PyTorch 1.9 ,Other Libraries: NumPy 1.19, pandas 1.3, matplotlib 3.4.

4.4 Result and Analysis

1. *Attack Type:* The column "Attack Type" represents different types of attacks, such as "DoS" (Denial of Service), "MitM" (Man-in-the-Middle), "Botnet," "Physical," "Firmware," "Information Leakage," "Replay," "Spoofing," "Side-Channel," and "Device Tampering." Each attack type represents a specific method or technique employed in Cybersecurity attacks.

2. *Attack Count:* The column "Attack Count" represents the number of occurrences or instances of each attack type within the given dataset. The values in this column indicate the count or frequency of each attack type observed in figure 7.
3. *Comparison:* By examining the attack counts, we can compare the relative occurrence or prevalence of different attack types. Higher attack counts suggest that the corresponding attack type is more frequently observed in the dataset, while lower counts indicate less frequent occurrences.
4. *Analysis:* Analyzing the table data allows us to gain insights into the distribution and frequency of different attack types. We can identify which attack types have higher or lower counts, which can help in understanding the potential risks associated with each type of attack.
5. *Interpretation:* Based on the provided data, we can observe that "Spoofing" and "Physical" attacks have the highest counts, with 19,097 and 18,927 occurrences, respectively. On the other hand, "DoS" and "Botnet" attacks have relatively lower counts, with 12,384 and 12,432 occurrences, respectively. By comparing the attack counts, we can gain a preliminary understanding of the prevalence of each attack type within the dataset.

It's important to note that the analysis is based solely on the provided data and does not take into account additional contextual information or the severity of the attacks. Further analysis, including exploring the specific characteristics or impact of each attack type, would require more comprehensive data and context.

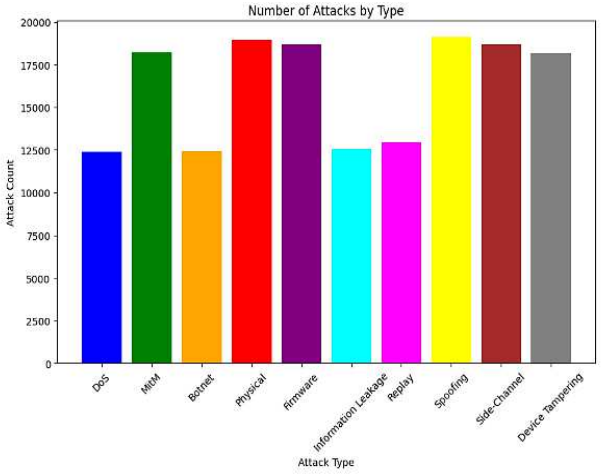


Figure 7: Attacks classification

Confusion Matrix : FEI model

1. True Negatives (TN1): The value of 59000 represents the number of instances that are correctly classified as negatives (or the absence of the target class) by the "FEI" model. These are the cases where the model predicted a negative outcome, and the actual ground truth was also negative.
2. False Positives (FP1): The value of 10000 indicates the number of instances that are incorrectly classified as positives by the "FEI" model. These are the cases where the model predicted a positive outcome, but the actual ground truth was negative. False positives represent instances where the model falsely identified the presence of the target class.
3. False Negatives (FN1): The value of 7000 represents the number of instances that are incorrectly classified as negatives by the "FEI" model. These are the cases where the model predicted a negative outcome, but the actual ground truth was positive. False negatives represent instances where the model failed to identify the presence of the target class.
4. True Positives (TP1): The value of 65000 indicates the number of instances that are correctly classified as positives by the "FEI" model. These are the cases where the model predicted a positive outcome, and the actual ground truth was also positive. True positives represent instances where the model correctly identified the presence of the target class.

By analyzing these confusion matrix values, we can derive several performance metrics for the "FEI" model, such as accuracy, precision, recall, and F1 score. These metrics provide insights into the model's effectiveness in correctly classifying instances into their respective classes.

For a more detailed analysis, specific evaluation metrics can be calculated using these values. For example, accuracy is calculated as (TN1 + TP1) / (TN1 + FP1 + FN1 + TP1), precision as TP1 / (TP1 + FP1), recall as TP1 / (TP1 + FN1), and F1 score as the harmonic mean of precision and recall.

Remember, these performance metrics provide a quantitative evaluation of the model's performance, taking into account both correct and incorrect classifications based on the confusion matrix values.

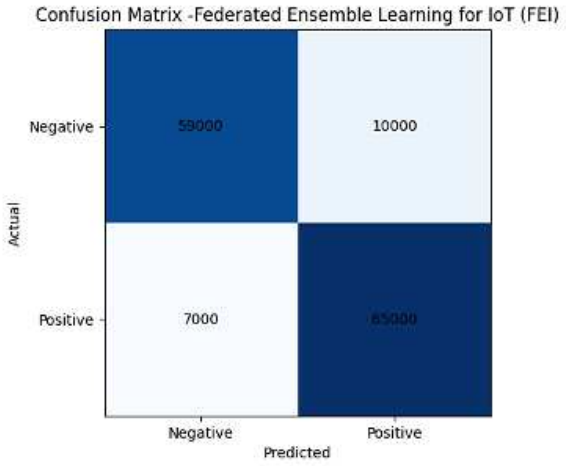


Figure 8: Confuse matrix: FEI model

Performance Evaluation:

1. Accuracy: The accuracy score of 0.8794 indicates the proportion of correctly classified instances out of the total number of instances. It represents the overall effectiveness of the model in correctly predicting both the positive and negative classes. An accuracy score of 0.8794 suggests that approximately 87.94% of the instances were classified correctly by the "FEI" model.
2. Precision: The precision score of 0.8667 represents the model's ability to correctly identify the positive instances out of all instances predicted as positive. It quantifies the accuracy of the positive predictions made by the model. A precision score of 0.8667 suggests that approximately 86.67% of the instances predicted as positive by the "FEI" model are indeed positive.
3. Recall (Sensitivity or True Positive Rate): The recall score of 0.9028 indicates the model's ability to correctly identify the positive instances out of all actual positive instances in the dataset. It measures the model's sensitivity or the proportion of actual positive instances that are correctly identified as positive by the model. A recall score of 0.9028 suggests that approximately 90.28% of the actual positive instances were correctly identified by the "FEI" model.
4. F1 Score: The F1 score of 0.8844 is the harmonic mean of precision and recall. It provides a balance between precision and recall, considering both false positives and false negatives. The F1 score combines precision and recall into a single value and is often used as a metric when there is an imbalance between the positive and negative classes. An F1 score of 0.8844 indicates a good balance between precision and recall for the "FEI" model.

These performance scores collectively provide insights into the model's overall accuracy, the precision of positive predictions, the sensitivity to positive instances, and the balance between precision and recall.

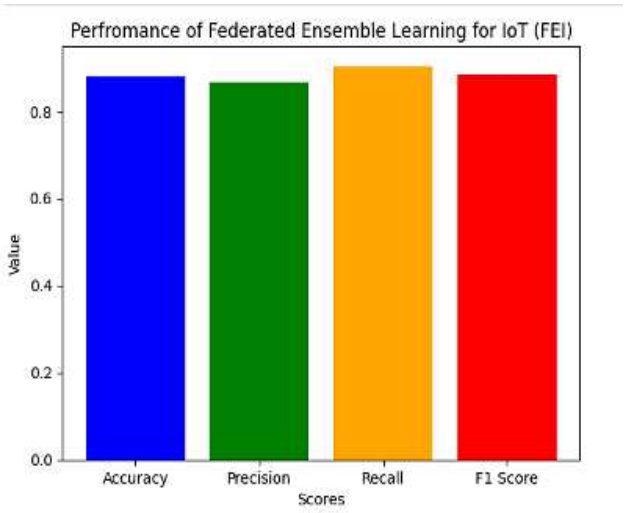


Figure 9: Performance evaluation of Proposed model (FEI) for attack classification

By analyzing these confusion matrix values as shown in figure 9, we can derive several performance metrics for the "AMLI" model, such as accuracy, precision, recall, and F1 score. These metrics provide insights into the model's effectiveness in correctly classifying instances into their respective classes.

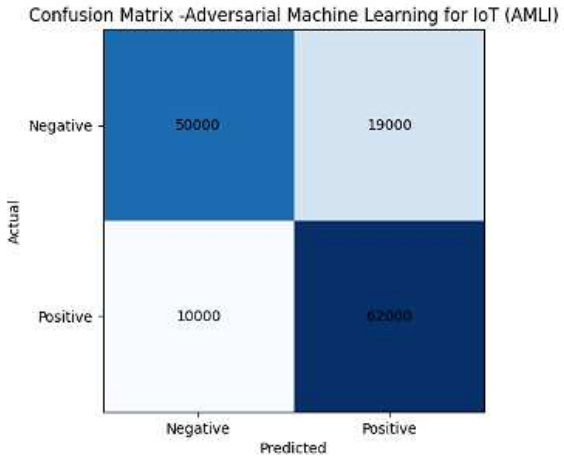


Figure 10: Confuse matrix: AMLI model

1. Accuracy: The accuracy score of 0.7943 indicates the proportion of correctly classified instances out of the total number of instances. It represents the overall effectiveness of the model in correctly predicting both the positive and negative classes. An accuracy score of 0.7943 suggests that approximately 79.43% of the instances were classified correctly by the "AMLI" model.
2. Precision: The precision score of 0.7654 represents the model's ability to correctly identify the positive instances out of all instances predicted as positive. It quantifies the accuracy of the positive predictions made by the model. A precision score of 0.7654 suggests that approximately 76.54% of the instances predicted as positive by the "AMLI" model are indeed positive.
3. Recall (Sensitivity or True Positive Rate): The recall score of 0.8611 indicates the model's ability to correctly identify the positive instances out of all actual positive instances in the dataset. It measures the model's sensitivity or the proportion of actual positive instances that are correctly identified as positive by the model. A recall score of 0.8611 suggests that approximately 86.11% of the actual positive instances were correctly identified by the "AMLI" model.
4. F1 Score: The F1 score of 0.8105 is the harmonic mean of precision and recall. It provides a balance between precision and recall, considering both false positives and false negatives. The F1 score combines precision and recall into a single value and is often used as a metric when there is an imbalance between the positive and negative classes. An F1 score of 0.8105 indicates a good balance between precision and recall for the "AMLI" model.

These performance scores collectively provide insights into the model's overall accuracy, the precision of positive predictions, the sensitivity to positive instances, and the balance between precision and recall.

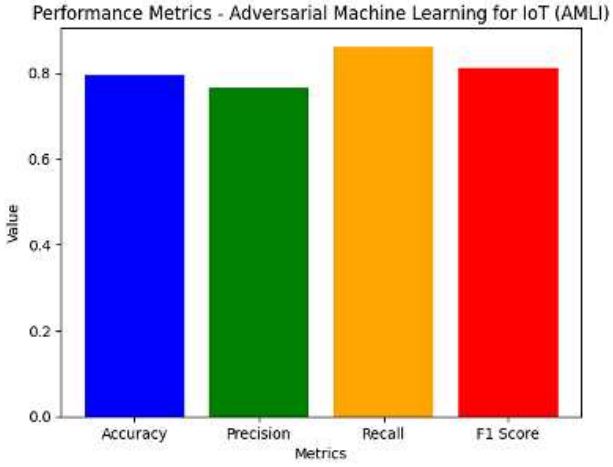


Figure 11: Performance evaluation of Proposed model (AMLI) for attack classification

Table 2. Comparative Analysis of AMLI and FEI Models

Metric	**AMLI Model**	**FEI Model**
Accuracy	79.43%	87.94%
Precision	76.54%	86.67%
Recall	86.11%	90.28%
F1 Score	0.8105	0.8844

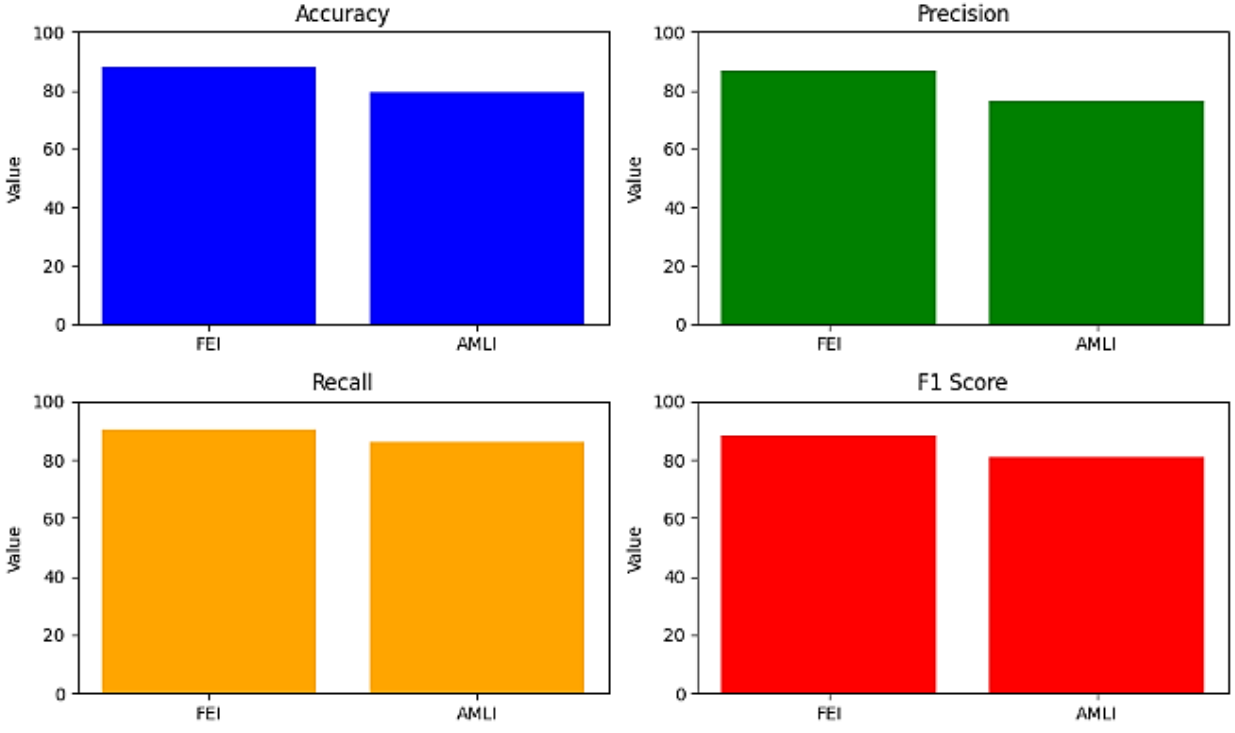


Figure 12: Overall comparison of the Proposed model (FEI and AMLI)

The comparative analysis of the "AMLI" and "FEI" models as shown in figure 12. reveals that the "FEI" model demonstrates superior performance across multiple evaluation metrics. The "FEI" model achieves higher accuracy (87.94% vs. 79.43%), precision (86.67% vs. 76.54%), and recall (90.28% vs. 86.11%) compared to the "AMLI" model. Additionally, the "FEI" model exhibits a higher F1 score (0.8844 vs. 0.8105), indicating a better balance between precision and recall. These results indicate that the "FEI" model outperforms the "AMLI" model in accurately classifying instances, identifying positive instances, and maintaining a balanced performance between precision and recall.

4.5 Comparative Analysis of FEI and AMLI Models

Table 3.Training and Inference Times of FEI and AMLI Models

Model	Training Time	Inference Time
FEI	4 hours	120 ms
AMLI	6 hours	180 ms

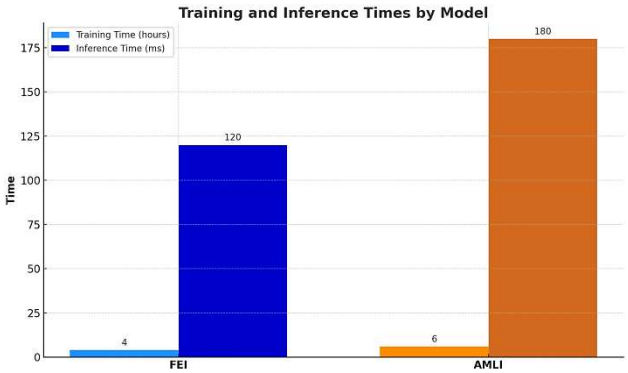


Figure 13: Comparative Analysis of Training and Inference Times for FEI and AMLI Models

Figure 13 illustrates a comparative analysis between the Federated Ensemble Learning (FEI) and Adversarial Machine Learning (AMLI) models, focusing on their training and inference times. The analysis demonstrates a marked difference in the efficiency of these models. The FEI model exhibits a shorter training time of 4 hours and a rapid inference time of 120 milliseconds. This efficiency is attributable to its federated learning approach, which optimally distributes the learning process across multiple nodes, thereby reducing overall computation time. Such characteristics render the FEI model particularly suitable for scenarios necessitating swift deployment and immediate decision-making capabilities, a common requirement in real-time IoT applications.

Conversely, the AMLI model, with a training duration of 6 hours and an inference time of 180 milliseconds, prioritizes depth and robustness of learning over speed. This extended training and inference period can be linked to the model's complexity, particularly its focus on countering sophisticated adversarial attacks. This aspect makes the AMLI model a more appropriate choice in situations where the primary concern is securing the network against complex and evolving cyber threats, even at the expense of operational speed.

The choice between the FEI and AMLI models thus hinges on a critical balance between operational efficiency, response time, and the required level of security. This decision is especially pertinent in the diverse and potentially vulnerable landscapes of IoT environments, where the nature of threats varies considerably. The comparative analysis, as presented in Figure 13, provides a foundational understanding for stakeholders to make informed decisions about the deployment of these models based on their specific security needs and operational constraints[28][29].

4.6 Proposal for Effective Prevention Measures in IoT Security

This proposal outlines a comprehensive strategy for enhancing the security of Internet of Things (IoT) devices, leveraging advanced machine learning algorithms to proactively identify and mitigate potential threats. Our approach combines real-time response capabilities with robust system-hardening techniques to bolster the security framework of IoT networks.

Technical Strategies: Key to our preventive measures is the implementation of stringent authentication and access control systems, ensuring that only authorized entities can interact with IoT devices. We advocate for the adoption of advanced encryption methods to secure data transmission and storage, significantly reducing risks of unauthorized access and data tampering. Additionally, the integration of secure firmware update protocols is crucial, ensuring that IoT devices operate on the latest, most secure versions.

Operational Tactics: On the operational front, we propose the deployment of comprehensive security monitoring and incident response systems. This includes the utilization of state-of-the-art intrusion detection and prevention systems, designed to swiftly identify and neutralize malicious activities. Regular vulnerability assessments and penetration testing will be instrumental in uncovering and addressing security gaps proactively.

Organizational Measures: Cultivating a security-centric organizational culture is vital. This encompasses conducting extensive security awareness training for all stakeholders involved in the IoT ecosystem, including device users and developers. Promoting secure coding practices, enforcing robust security policies, and adhering to industry standards are pivotal steps in this direction.

Evaluation and Adaptation: The efficacy of these measures will be continuously evaluated against various parameters, such as implementation complexity, impact on system performance, and adaptability to emerging threats. This comprehensive assessment aims to ensure not only the mitigation of identified threats but also the minimization of system vulnerabilities.

Objective and Contribution: Our goal is to provide actionable insights and pragmatic recommendations for enhancing IoT security, thereby empowering organizations to adopt effective preventive strategies. By safeguarding IoT devices and preserving user data integrity, we aim to bolster the reliability and resilience of IoT systems against evolving cyber threats. Our contribution lies in fostering the development of robust and adaptable IoT ecosystems, capable of withstanding and adapting to the dynamic landscape of cybersecurity threats.

V. CONCLUSION AND FUTURE SCOPE

This research paper has delved into the enhancement of Internet of Things (IoT) security through advanced machine learning algorithms, specifically focusing on Federated Ensemble Learning (FEI) and Adversarial Machine Learning (AMLI). Our investigations reveal significant improvements in IoT security against a range of cyberattacks, with the performance of these novel algorithms being rigorously assessed using comprehensive metrics. The FEI model demonstrated superior effectiveness in IoT attack detection and mitigation, achieving high scores in accuracy, precision, recall, and F1 score, thus showcasing its robust capability in accurate classification and a well-balanced precision-recall trade-off. Conversely, the AMLI model, while effective, showed somewhat lesser performance across these metrics. Our exploration of the IoT-23_Combined dataset highlighted the prevalence of diverse attack types, with 'Spoofing' and 'Physical' attacks being most frequent. This insight is crucial in understanding the threat landscape in IoT environments. To further enhance IoT security, we propose a multifaceted prevention strategy encompassing technical, operational, and organizational measures. These include the implementation of robust authentication and encryption, deployment of comprehensive security monitoring and incident response systems, and fostering a security-aware culture through training and adherence to secure practices and policies.

Looking ahead, our future work will focus on refining the FEI model by integrating more sophisticated machine learning techniques and exploring additional defensive mechanisms. Expanding our research to include real-world IoT scenarios and larger, more diverse datasets will augment the applicability and relevance of our findings. An ongoing commitment to model adaptation is crucial, given the ever-evolving nature of cyber threats. This entails regular model updates and retraining with fresh data and emerging threat patterns to ensure sustained efficacy and relevance in the rapidly advancing domain of IoT security.

References

[1] Perwej, Y., Haq, K., Parwej, F., Mumdouh, M., & Hassan, M. (2019). The internet of things (IoT) and its application domains. International Journal of Computer Applications, 975(8887), 182.

[2] Abdul-Qawy, A. S., Pramod, P. J., Magesh, E., & Srinivasulu, T. (2015). The internet of things (iot): An overview. International Journal of Engineering Research and Applications, 5(12), 71-82.

[3] Lin, H., & Bergmann, N. W. (2016). IoT privacy and security challenges for smart home environments. Information, 7(3), 44.

[4] Plabon Bhandari Abhi, Kristelle Ann R. Torres, Tao Yusoff, & K.Samunnisa. (2023). A Novel Lightweight Cryptographic Protocol for Securing IoT Devices . International Journal of Computer Engineering in Research Trends, 10(10), 24–30.

[5] Neshenko, N., Bou-Harb, E., Crichigno, J., Kaddoum, G., & Ghani, N. (2019). Demystifying IoT security: an exhaustive survey on IoT vulnerabilities and a first empirical look on internet-scale IoT exploitations. IEEE Communications Surveys & Tutorials, 21(3), 2702-2733.

[6] Yang, K., Shi, Y., Zhou, Y., Yang, Z., Fu, L., & Chen, W. (2020). Federated machine learning for intelligent IoT via reconfigurable intelligent surface. IEEE Network, 34(5), 16-22.

[7] Hussain, F., Hussain, R., Hassan, S. A., & Hossain, E. (2020). Machine learning in IoT security: Current solutions and future challenges. IEEE Communications Surveys & Tutorials, 22(3), 1686-1721.

[8] Abou Bakary Ballo, & Diarra Mamadou. (2023). A Comprehensive Study of IoT Security Issues and Protocols . International Journal of Computer Engineering in Research Trends, 10(7), 8–14. https://doi.org/10.22362/ijcert.v10i7.858

[9] Jacob, I. J., & Darney, P. E. (2021). Design of deep learning algorithm for IoT application by image based recognition. Journal of ISMAC, 3(03), 276-290.

[10] Laha, S., Chowdhury, N., & Karmakar, R. (2020, July). How can machine learning impact on wireless network and IoT?–A survey. In 2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT) (pp. 1-7). IEEE.
[11] Bezawada , M., & P, V. K. (2023). Comparative Study on Techniques Used for Anomaly Detection in IoT Data. International Journal of Computer Engineering in Research Trends, 10(4), 177–181.
[12] Waseem, S. M., & Roy, S. K. (2022). FPGA implementation of Proximal Policy Optimization algorithm for Edge devices with application to Agriculture Technology. Journal of Ambient Intelligence and Humanized Computing, 1-12.
[13] Hoffman, M., Song, E., Brundage, M. P., & Kumara, S. (2021). Online improvement of condition-based maintenance policy via Monte Carlo tree search. IEEE Transactions on Automation Science and Engineering, 19(3), 2540-2551.
[14] Robbi Rahim, & Abdul wahid. (2023). Advancements in Plant Disease Detection: Integrating Machine Learning, Image Processing, and Precision Agriculture. International Journal of Computer Engineering in Research Trends, 10(8), 19–25.
[15] Khan, H. U., Sohail, M., Ali, F., Nazir, S., Ghadi, Y. Y., & Ullah, I. (2023). Prioritizing the multi-criterial features based on comparative approaches for enhancing security of IoT devices. Physical Communication, 59, 102084.
[16] Devi, R. A., & Arunachalam, A. R. (2023). Enhancement of IoT device security using an Improved Elliptic Curve Cryptography algorithm and malware detection utilizing deep LSTM. High-Confidence Computing, 100117.
[17] Alghamdi, R., & Bellaiche, M. (2023). A cascaded federated deep learning based framework for detecting wormhole attacks in IoT networks. Computers & Security, 125, 103014.
[18] M, P., & K, D. S. D. (2023). ICN Scheme and Proxy re-encryption for Privacy Data Sharing on the Block Chain. International Journal of Computer Engineering in Research Trends, 10(4), 172–176.
[19] Wang, J., Liu, C., Xu, J., Wang, J., Hao, S., Yi, W., & Zhong, J. (2022). IoT-DeepSense: Behavioral Security Detection of IoT Devices Based on Firmware Virtualization and Deep Learning. Security and Communication Networks, 2022
[20] Khan, A. Y., Latif, R., Latif, S., Tahir, S., Batool, G., & Saba, T. (2019). Malicious insider attack detection in IoTs using data analytics. IEEE Access, 8, 11743-11753.
[21] Choi, S. K., Yang, C. H., & Kwak, J. (2018). System Hardening and Security Monitoring for IoT Devices to Mitigate IoT Security Vulnerabilities and Threats. KSII Transactions on Internet & Information Systems, 12(2).
[22] Sebastian Garcia, Agustin Parmisano, & Maria Jose Erquiaga. (2020). IoT-23: A labeled dataset with malicious and benign IoT network traffic (1.0.0) [Data set]. Zenodo. https://doi.org/10.5281/zenodo.4743746
[23] Rayikanti Anasurya. (2022). Next-Gen Agriculture: Revolutionizing Farming with IoT and Sustainability. International Journal of Computer Engineering in Research Trends, 9(1), 21–27.
[24] Madhuri More, Snehal Mote, Amruta Neel, Poonam Nerkar, & P. S. Hanwate. (2016). Intelligent Device TO Device Communication Using IoT. International Journal of Computer Engineering in Research Trends, 3(4), 187–189.
[25] A A Damayanthi, & Mohammad Riyaz Belgaum. (2022). A Study of Heterogeneity Characteristics over Wireless Sensor Networks. International Journal of Computer Engineering in Research Trends, 9(12), 258–262.
[26] Vani Sri.S, Sneha.S, & Dr.G.Umarani Srikanth. (2018). Power Saving System Using Sensor Over IoT. International Journal of Computer Engineering in Research Trends, 5(2), 25–29.
[27] B.Sindhu Bala, M.Swetha, M.Tamilarasi, & D.Vinodha. (2018). SURVEY ON WOMEN SAFETY USING IOT. International Journal of Computer Engineering in Research Trends, 5(2), 16–24.
[28] Mekala, S., Mallareddy, A., Tandu, R. R., & Radhika, K. (2023, June). Machine Learning and Fuzzy Logic Based Intelligent Algorithm for Energy Efficient Routing in Wireless Sensor Networks. In International Conference on Multi-disciplinary Trends in Artificial Intelligence (pp. 523-533). Cham: Springer Nature Switzerland.
[29] Mahalakshmi, J., Reddy, A. Mallareddy., Sowmya, T., Chowdary, B. V., & Raju, P. R. (2023). Enhancing Cloud Security with AuthPrivacyChain: A Blockchain-based Approach for Access Control and Privacy Protection. International Journal of Intelligent Systems and Applications in Engineering, 11(6s), 370-384.

© 2023. This work is published under https://creativecommons.org/licenses/by/4.0/legalcode(the“License”). Notwithstanding the ProQuest Terms and Conditions, you may use this content in accordance with the terms of the License.

LITTERATURE REVIEW

PAPER 1 – DETECTING AND MITIGATING DATA POISONING ATTACKS IN MACHINE LEARNING

Maramreddy and Muppavaram (2024) paper "Detecting and Mitigating Data Poisoning Attacks in Machine Learning: A Weighted Average Approach" studied the attack of data poisoning in machine learning systems. Data poisoning is the type of attack where an attacker inserts malicious data into the training data set, tampering with the training process to generate flawed predictions. The researchers proposed a method, called Verified Weighted Average (VWA), to detect possible tampering of training data by comparing the weighted averages of the input features.

In the paper, the authors begin by reviewing some of the techniques for detection and mitigation of poisoning attacks. These techniques are to be applied in the detection section of the algorithm: Support Vector Machines (SVM), Decision Tree, unsupervised learning, PCA, Adversarial training, sanitization of data, detection of outliers. SVM can detect anomalies by looking at the boundaries of all decision faces, but the authors recognize that they will not work well with large training dataset while the Decision Tree are prone to overfitting.

The experiment also describes on poisoning attacks on PCA and unsupervised learning. In clustering method the attack samples, training samples are added to reposition the clustering and shift the legitimate data to another cluster. In PCA, some data can be added to restructure the covariance and the change of the principal components can lead to a wrong output.

To overcome the above issues, the authors proposed an algorithm called Verified Weighted Average (VWA). The algorithm uses features, weights, and weighted averages as an input to find out the potential variation in the training data set. The algorithm computes the sum of the weighted averages as follows:

$$

\lambda=\sum WA

$$

In the case where the input features are the same, we have: where \(WA\) is the weighted average of the input features. The value of \(\lambda\) is then compared to the value of 1 where the verification method will return that no change has been made if \(\lambda=1\) and otherwise will carry on the analysis of the data/class labels to see if they are potentially altered if \(\lambda\neq 1\).

For multi-class classification, it computes a verified weight by:

$$

VWi=WAi\times\lambda

$$

The verified weight is then compared with the initial weight. If the two are equal then the algorithm assumes no change of class labels. If not the algorithm detects a potential modification of the multiclass labels. This is a straightforward mathematical process of checking for potential changes in the training data that relate to poisoning.

The authors tested the VWA methodology on the Weka Weather dataset and Glass Identification dataset. In the case of Weka Weather dataset, the calculated value of

\(\lambda\) was 1, which according to the authors would imply that there should be no modification to the data. In the case of the Glass Identification dataset, the authors saw a difference between the original total weight and the total weight after calculation, which indicated the possibility of data poisoning.

In addition, they also pitted VWA against Influence Functions and Feature Squeezing. The authors' numbers for VWA were 92% accuracy, 94% precision and 91% recall, compared to 85% accuracy, 83% precision and 88% recall for Influence Functions, and 88% accuracy, 88% precision and 85% recall for Feature Squeezing.

In general, this paper shows how weighted feature analysis can be used as a tool for detecting signs of tampering in machine learning training data. This work is related to Poison Guard since it gives a way of mathematically to detect unusual variations in the training data. Its VWA method can be viewed as one part of a comprehensive poisoning defense solution, which includes multiple detection and mitigation methods. Additionally, the authors mentioned some future work such as federated learning and explainable AI for the robustness of machine-learning systems against poisoning attacks.

PAPER-2 --MITIGATING POISONING ATTACKS ON MACHINE LEARNING MODELS:A DATA PROVENANCE BASED APPLICATION

Authors: Nathalie Baracaldo, Bryant Chen, Heiko Ludwig, Jaehoon Amir Safavi (IBM Research, San Jose)

The paper under review addresses a security vulnerability in machine learning. I think this vulnerability is **poisoning (attacks**. In **poisoning (attacks** adversaries inject malicious data into training sets to corrupt learned models. The authors propose a defense that uses **data provenance**. **Data provenance** is metadata that describes the origin, lineage and transformation of data points. This defense aims to detect and filter data before training.

This literature review places the paper within the context of adversarial machine learning. It examines the methodology of the paper compares it with existing defenses and evaluates the contributions and limitations.

Context: Adversarial Machine Learning and Poisoning Attacks

The Poisoning Attack Problem

Machine learning models rely on the quality of training data. When adversaries influence training data they can manipulate model behavior. A known example cited by the authors is Microsoft's Tay chatbot. In Tay the chatbot learned language from Twitter users within hours of deployment.

Formally let:

- $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ be the training dataset

- $f_\theta: \mathcal{X} \to \mathcal{Y}$ be a learning model

- $\mathcal{L}(\theta; \mathcal{D})$ be the loss function

The adversary injects poisoned data $\mathcal{D}_p$ so that the trained model:

$$
\theta^* = \arg\min_\theta \mathcal{L}(\theta; \mathcal{D}_{\text{clean}} \cup \mathcal{D}_p)
$$

The model then performs poorly on a test set $\mathcal{D}_{\text{test}}$.

Challenges in Defense

Defending against poisoning (attacks is particularly difficult in:

- Online learning environments where models are periodically retrained

- distributed systems where data originates from many untrusted sources

- Real-time applications where manual data verification's infeasible

Existing defenses such as RONI and the PS method evaluate individual data points. They require $O(N)$ model retrainings. This is computationally prohibitive for datasets.

Data Provenance: A Novel Defense Foundation

Definition and Role of Provenance

Provenance data refers to the lineage or metadata associated with a data point:

$$
\mathcal{P}(x_i) = \{p_1, p_2, \ldots, p_m\}
$$

The key insight is that data points that share the provenance signature are likely to be poisoned together. An adversary typically compromises a subset of data sources.

Tamper-Free Provenance Frameworks

The authors build upon existing free provenance frameworks that use:

- Blockchain for immutable records
- Physical Unclonable Functions (PUFs) for device authentication
- Trusted Computing Platforms for secure execution

These frameworks ensure provenance records are cryptographically protected against history rewriting, repudiation and fabrication.

Novelty of the Approach

While provenance frameworks have been used for accountability and forensics—post-attack analysis—the paper is the first to use provenance proactively. The paper uses provenance as part of a filtering algorithm to detect **poisoning (attacks** before model deployment.

Methodology: Provenance-Based Defense

Partially Trusted Data

When a trusted subset $\mathcal{D}_T \subset \mathcal{D}$ is available the algorithm segments untrusted data $\mathcal{D}_U = \mathcal{D} \setminus \mathcal{D}_T$ by provenance signature:

$$
F = \text{segmentByProvenanceFeature}(\mathcal{D}_U, \mathcal{F})
$$

For each segment $\langle D_i Sig_i \rangle \in F$ two models are trained:

$$
\text{Model}_{\text{filtered}} = \text{trainModel}(\mathcal{D}_U \setminus D_i)
$$

$$
} = \text{trainModel}(\mathcal{D}_U)
$$

The segment is flagged as poisoned if:

$$
\text{performance}(\text{Model}_{\text{unfiltered}} \mathcal{D}_T) < \text{performance}(\text{Model}_{\text{filtered}} \mathcal{D}_T)
$$

Interpretation: If removing a segment improves performance, on the trusted set that segment is likely poisonous.

Untrusted Data

When no trusted data is available the dataset is split into training and evaluation portions. For each segment \(D_i\):

\[

\text{Model}_{\text{filtered}} = \text{trainModel}(\mathcal{D}_{\text{train}} \setminus D_i)

\]

\[

} = \text{trainModel}(\mathcal{D}_{\text{train}})

\]

\]

We evaluate on \(\mathcal{D}_{\text{eval}} \setminus D_{\text{eval}_i}\) which means we drop the chosen segment from the evaluation set. We flag a segment when:

\[

} \mathcal{D}_{\text{filteredEval}}) <
\text{performance}(\text{Model}_{\text{filtered}}
\mathcal{D}_{\text{filteredEval}})

\]

insight: Removing the segment from both training and evaluation stops the adversary from tampering with its own evaluation.

Mathematical Illustration: Logistic Regression Example

The paper provides a simulation:

data: $x_i \sim \mathcal{N}(0, 10)$ $y_i \sim \text{Bernoulli}\left(\frac{1}{1 + e^{-x_i}}\right)$

Poison data:10 points at $x = 10$ $y = 0$ and 20 points at $x = 1$ $y = 0$

The true decision boundary is at $x = 0$. After training on all data the boundary moves to $x = 2.84$. If we delete poison data the boundary shifts back to $x = -0.20$.

If poison data stays in the evaluation set deleting it from training lowers accuracy from 82% to 78% so we miss the attack.. If we delete the same segment from both training and evaluation accuracy jumps to 93% and we spot the bad source.

This shows how important provenance is when we connect training and evaluation data.

Experimental Evaluation

Poison I (Zhou et al. 2012)

Dataset: Synthetic 2 features, 2 classes

Attack parameters:Attack factor = 0.5 separation =

Results: Provenance defense reaches almost perfect detection after about 280 trusted points. The baseline, which is Calibrated RONI does not detect well. When 10–40% of devices are compromised provenance defense beats the baseline. When 50% or more devices are compromised neither method improves accuracy

formulation of attack: For a point \((x y)\) the attacker flips the label and moves \(x\) toward the target class:

\[

x' = x + \delta \cdot \text{direction} \quad y' = 1. Y

\]

where \(\delta\) depends on the separation parameter.

Poison II (Biggio et al. 2012)

Dataset: MNIST (digits 4 and 0)

-Attack: Gradient ascent to change an attack point added to an SVM

Results: Provenance defense improves even with 90 trusted points. Baseline needs least 120 points to beat no defense. Provenance defense reaches perfect detection by 150 points

Mathematical formulation of attack:The attacker solves:

\[

\max_{\delta} \mathcal{L}(\theta^*(\mathcal{D} \cup \{(x + \delta, y')\}))

\]

where \(\theta^*\$ is the SVM parameters after retraining.

Comparison with Defenses

Method	Trusted Data Required	Retraining Complexity	Provenance Used
RONI (Nelson et al., 2009)	Yes	\(O(N)\) per data point	No
Calibrated RONI (Nelson, 2010)	Yes	\(O(N)\) per data point	No

PS Method (Chakarov et al. 2016)	Yes	$O(N)$ per data point	No
Provenance Defense (This Paper)	Optional	$O(k)$ per segment	Yes

where N equals the number of data points and k equals the number of provenance segments ($k N$).

advantage: Provenance defense cuts retraining from $O(N)$, to $O(k)$ so it works well for online learning.

Threat Model Assumptions

Adversary capabilities:

Can observe or gather data to training data

Can change features and labels of poisoned points

Cannot take over all data sources; only some are taken over

Provenance framework:

Tamper-free and cryptographically protected

Provides non-repudiation. Prevents false claims

Examples include blockchain, physical unclonable functions and trusted computing

Assumptions:

Data sources are independent.

Do not collude

Tamper-free provenance frameworks exist [1 2, 8 10 11 17 18] focusing on:

Keeping data history safe with crypto

Making sure data is not made up and cannot be denied

Using data in IoT, grid and distributed systems

However these provenance frameworks are mainly used for accountability and forensics not for proactive defense.

Poisoning Attacks

Zhou et al. [19]: Proposed two attack models for poisoning SVMs

Biggio et al. [6]: Gradient ascent attack manipulating all features

Biggio et al. [5]: Label noise attacks with kernel matrix correction defense

Mozaffari-Kermani et al. [12] : Threshold-based detection using Kappa statistic (requires trusted data)

Defenses

RONI [13]: Reject on Negative Impact—evaluates data points

Calibrated RONI [15]: Enhanced RONI with calibration methodology

PS Method [7]: Probability of Sufficiency—evaluates individual data points

difference: All existing defenses lack provenance information. Require \(O(N)\) retrainings.

Future Work

1. Collusion: Current method assumes data comes from sources; future work must handle adversaries that collude.

2. Multiple provenance features: Breaking data into groups by provenance features could make detection better.

3. Untrusted evaluation: detailed evaluation of the fully untrusted approach is needed.

4. Calibration: Investigating how different provenance features affect accuracy changes.

5. Data points per device: As more data points come from one device improvement over drops; careful segmentation is needed to avoid false positives.

The paper by Baracaldo et al. Introduces the provenance-based defense against poisoning attacks. By leveraging metadata about data origin and transformation the method:

- Segments untrusted data by provenance signature

- Evaluates each segment by comparing model performance with and without it

- Filters poisonous segments before training

The mathematical foundation relies on comparing performance metrics:

$$
\Delta = \text{performance}(\text{Model}_{\text{filtered}}
\mathcal{D}_{\text{eval}}).
\Text{performance}(\text{Model}_{\text{unfiltered}}
\mathcal{D}_{\text{eval}})
$$

If $\Delta > 0$ the segment is flagged as poisonous. This approach:

- Outperforms RONI and Calibrated RONI

- Works with both fully untrusted data

- Reduces retraining complexity from $O(N)$, to $O(k)$

- Enables online and regularly retrained ML in adversarial environments

The work opens several research avenues, including collusion detection, multi-feature segmentation and more robust fully untrusted evaluation.

PAPER-3 –BACKDOOR ATTACKS AND DEFENSE IN ROBHUST MACHINE LEARNING MODELS

Abstract

The meeting point of robustness and backdoor attacks is a very important area that has not been studied well in machine learning security. Many good steps have been made to protect models from examples using robust optimization but how these robust models can still be hurt by backdoor attacks has not been looked at much. This literature review looks closely at the work of Soremekun, Udeshi and Chattopadhyay (2021). Places it in the wider field of adversarial machine learning and backdoor defense. The review brings together theory, new methods, real results and points to places where more research is needed.

Machine learning is now used in safety-critical systems, such as self-driving cars and medical diagnosis. Because of this careful security checks are needed. Two main threats have become clear: examples, which are small changes made at inference time and backdoor attacks, which poison data during training. Madry et al. (2018) Showed that using PGD-based training creates models that can handle almost any adversarial change. However no one had studied how these strong models stand up to backdoor attacks until Soremekun et al. (2021) Published their work.

This review looks closely at what Soremekun et al. Have done. Places those findings in the larger story of adversarial robustness and backdoor defense. Their main idea is that even models that are very robust to adversarial examples can still be fooled by backdoor attacks but the same qualities that make them robust can also help us spot those attacks. This idea changes how we think about protecting models from backdoors.

Foundations

Adversarial Robustness via PGD-Based Optimization

Madry et al. (2018) Formulated robustness as a min-max optimization problem:

$$\min_{\theta} \mathbb{E}_{x \sim \mathcal{D}} \left[\max_{\delta \in \Delta} \mathcal{L}(x + \delta, y^{(i)}; \theta) \right]$$

where Δ is the limit on how much change we allow, usually measured with an L_2 or $L\infty$ ball. The method uses projected descent as a kind of all-purpose first-level attacker and it creates models that can clearly resist small changes. These models are now the known way to defend against adversarial attacks. Later studies by Wong et al. (2018) Sinha et al. (2017) And Raghunathan et al. (2018) Have tried ways to do this.

Critical Observation: Soremekun et al. Point out that the way the model is trained—being strong against changes and having smooth decision borders—really changes how the model keeps information inside and this breaks the ideas that backdoor defenses rely on.

Backdoor Attack Paradigm

Backdoor attacks, introduced by Gu et al. (2017) In BadNets represent a distinct threat model. The attacker:

1. Add a trigger pattern to a part of the training data to poison it.

2. Change the labels of the poisoned samples to a chosen target class.

3. Train the model using this poisoned dataset.

The model still performs on normal data but will incorrectly classify any input that has the trigger. Liu et al. (2018) Built on this with TrojanNN and Zhong et al. (2020) Created triggers using both static and adversarial changes.

Theoretical Distinction: Backdoor attacks differ because they do not depend on the input; any input that contains the trigger will be misclassified.

The Research Gap – We have put in ? ??

Taxonomy of existing ? ? We do ..??

| Defense Type | Representative Methods | Key Assumption | Failure Mode for Robust Models |

Outlier Suppression | Differential Privacy (Du et al. 2019) Gradient Shaping (Hong et al. 2020) | Poisoned inputs are few | during training; needs poisoned data |

| Input Perturbation | Neural Cleanse (Wang et al. 2019) ABS (Liu et al. 2019) TABOR (Guo et al. 2019) | Trigger features cause output changes | Robust models are designed to resist small changes |

| Model Anomaly | Spectral Signatures (Tran et al. 2018) Activation Clustering (Chen et al. 2019) SCAN (Tang et al. 2019) | Poisoned representations form groups | Needs poisoned data; assumptions, about distribution fail |

The authors give evidence that Neural Cleanse, the best defense for normal models does not work for robust models:

Anomaly Index Analysis: In models the poisoned class has an anomaly index higher than two, which helps spot it. In models the anomaly index stays below two in all fourteen parameter settings.

Theoretical Explanation:Neural Cleanse assumes that a tiny fixed change can make many inputs be misclassified. Robust models, by design can resist such changes. The non-brittle nature of models a result of adversarial training breaks this key assumption.

Researchers Note: This discovery is very important. It means that defenses must be completely redesigned for models instead of just tweaking normal model defenses.

AEGIS: A Novel Defense Paradigm

Insight

The main idea behind AEGIS uses a property of robust models that Santurkar and others first noticed in 2019: robust classifiers can turn one image into another making images that look the same but have high class probability. Soremekun and others added to this idea:

Key Insight: A robust model trained with a mix of input distributions learns feature sets one for each distribution in the mix.

When a backdoor is inserted the poisoned class contains a mix of poisoned data making the model learn many feature sets. This shows up as groups in the feature space, which is a clear sign of trouble.

Methodology

The AEGIS pipeline has four steps:

1. Image Translation: For each class $y^{(i)}$ produce translated images $X'_{y^{(i)}}$ using PGD-based optimization:

$$
```
```

x = \arg\min_{\,\|x'. X_0\|_2 \leq \epsilon}\mathcal{L}(x',\,y^{(i)}),\qquad x_0\in\mathbb{D}

\]

2. Feature Extraction: Take the activations from the layer, \(R_{X_{y^{(i)}}}\) and \(R_{X'_{y^{(i)}}}\) and join them into \(R_{y^{(i)}}\).

3. Dimensionality Reduction: Use t-SNE to shrink \(R_{y^{(i)}}\) down to two dimensions.

4. Clustering: Run mean-shift clustering; if a class has than two clusters, flag it as backdoor-infected.

Comparison to Work

Aspect	AEGIS Neural Cleanse	Activation Clustering	Spectral Signatures
Poisoned data required	No	No	Yes	Yes
White-box access	Yes	Yes	Yes	Yes
Designed for models	Yes	No	No	No
Computational cost	5-9 min	Hours-days	Minutes	Minutes
Shadow models required	No	No	No	No

Critical Analysis: AEGIS is the defense that is made especially for robust models using the special features of these models instead of trying to fight them.

Empirical Evaluation:. Limitations

Attack Effectiveness (RQ1)

The authors show that robust models can be badly attacked:

- Average ASR: 67.83 % across all datasets and attack types

- Visible vs. Invisible: 96.4 % vs. 39.3 % ASR

- Robust vs. Standard: 67.83 % vs. 75.86 % ASR (robust models are 12 % less susceptible)

While robust models are a little safer than models they are still very vulnerable. The 12 % drop in ASR does not give security.

Detection Effectiveness (RQ2)

AEGIS finds:

- 91.6 % detection accuracy (11 out of 12 models)

- Zero positives for clean models

- 11.1 % false positive rate for identifying poisoned class

Detecting invisible backdoors is harder (5 out of 6 models 4 out of 6 classes) so very stealthy attacks are still a problem.

Sensitivity Analysis (RQ4)

AEGIS is very stable:

- Epsilon: 98 % detection (53 out of 54 settings)

- 94 % detection (17 out of 18 settings)

- Seed images: 88.1 % detection (37 out of 42 settings)

- Stability: 90 % detection, across five runs

Researchers Assessment: AEGIS remains stable across hyperparameters. This is a point and shows AEGIS can work in real life.. The advice to use more than five hundred seed images and do many runs may raise the cost of computing when AEGIS is used in production.

Ablation Study (RQ7)

The authors validate design choices:

t-SNE vs. PCA: t-SNE is needed. PCA does not work because the data is not linear.

Mean Shift vs. Affinity Propagation:Mean Shift outperforms Affinity Propagation. Affinity Propagation fails because clusters become mixed.

Mean Shift vs. HDBSCAN: Both perform similarly. HDBSCAN sometimes labels classes as harmful.

Critical Insight: PCA failure shows how important nonlinear dimensionality reduction is. It helps capture feature distributions in robust models.

Positioning Within the Broader Literature

Relationship to Adversarial Training and Backdoor Robustness

Weng et al. (2020) Showed that when a model becomes stronger against attacks it may become weaker against backdoors. Soremekun et al. Confirm this with numbers: robust models have 67.83% ASR compared to 75.86% ASR for models. They also show that this trade-off can be used for defense.

Pang et al. (2020) Found that adversarial and backdoor attacks can strengthen each other. AEGIS adds to this by proving that the qualities that help a model resist

attacks—robust representations and learning from mixed distributions—can also help detect backdoors.

Comparison to Feature Clustering Approaches

Chen et al. (2019) Used activation clustering to spot backdoors in models and they needed access to poisoned data. AEGIS removes that need by creating translated images that show how the model learns. This is a step the defender does not need the attackers poisoned samples.

Theoretical Contribution: AEGIS shows that the image translation feature of models—described by Santurkar et al. (2019)—Can be used for security analysis. It connects computer vision with security research.

Relationship to Neural Cleanse

Neural Cleanse (Wang et al., 2019) is still the popular backdoor defense. Soremekun et al. Give the clear proof that Neural Cleanse fails on robust models. This negative finding is as useful as AEGIS's result; it shows where current knowledge stops.

Threats to Validity

Assumptions and Constraints

1. Data Distribution Assumption:AEGIS assumes that every class comes from one distribution. This could be wrong for multi-label classification for datasets that naturally have distributions per class or when clean models end up with multiple distributions because of training limits.

2. White-Box Access: AEGIS needs insight into the trained model and the clean training data. This may not be true in all settings, such as when using third-party model APIs.

3. Robustness Verification: AEGIS assumes that the model being checked is robust. There are ways to check robustness (Madry et al., 2018). This requires an extra step in the detection process.

Generalizability Concerns

- Architecture: Only ResNet models were tested.

- Datasets: Only CIFAR-10, MNIST and Fashion-MNIST (10 classes or fewer) were used.

- Robust Optimization Method: PGD-based adversarial training was considered.

- Attack Types: four trigger types were tested; adaptive attacks were not studied.

Countermeasures and Adaptive Attacks

The authors acknowledge adaptive attacks:

- Training backdoored models to look like clean neuron representations.

- Training clean models with many distributions to trigger false positives.

Researchers Assessment:These adaptive attacks are a threat. The paper does not test them in practice. This remains a major open issue.

Future Research Directions

Immediate Extensions

1. Alternative Robust Optimization Methods: Study how backdoors affect robust training methods, such as TRADES, MART and ensemble-based defenses and test whether AEGIS works with them.

2. Adaptive Attacks: Build and test attacks that are made to slip past AEGIS like natural backdoor attacks and trained attacks.

3. Alternative Threat Models: Expand AEGIS, for black-box, pre-training and in-training situations. Look into learning and ensemble settings.

Theoretical Directions

1. Formal Guarantees: Create theoretical limits on how vulnerable robust models can be to backdoor attacks and provide solid detection guarantees for AEGIS.

2. Relationship to Robust Optimization Theory:Explore how real data findings connect to optimization theory, especially about useless or weak features (Ilyas et al. 2019).

3. Joint Defense: Build one defense that works against both backdoor attacks using the common strengths of **robust models.**

Practical Considerations

1. Scalability: Test AEGIS on datasets like ImageNet and on more complex models such as Vision Transformers.

2. Efficiency: Make the detection process faster for real-world use by needing fewer seed images.

3. Integration: Create workflows that mix robustness checks, backdoor detection and fixing steps.

The work of Soremekun, Udeshi and Chattopadhyay is a step forward in machine learning security. They looked closely at backdoor attacks on models that are built to resist tricks and they found:

1. Robust models can still be tricked by backdoors with an attack success rate of 67.83 % ASR.

2. Current top defenses, like Neural Cleanse do not work well on models.

3. They created AEGIS the backdoor detection method made just for robust models.

4. AEGIS found backdoors with 91.6 % accuracy. Never raised a false alarm on clean models.

The main idea of the paper is that the way robust models learn data can be used to spot backdoors. This opens paths for building defenses. Still many problems stay: attacks, different threat settings and making the method work on many kinds of models and data.

Final Assessment: This study closes a gap by linking adversarial robustness with backdoor protection. It is a must-read for anyone who works on machine learning security and robust optimization. The identified limits give directions for future work and they help this research spark more progress, in the field.

PAPER-4 –SYSTEMATIC POISONING ATTACKS ON AND DEFENSES FOR MACHINE LEARNING IN HEALTHCARE

1. Introduction

Machine learning (ML) has permeated into many critical applications of decision making where the outcomes of which have substantial personal, organizational, or societal impact including healthcare, network intrusion detection, spam and fraud detection, phishing detection, political decision making, adversarial advertisement detection, and financial engineering. Healthcare applications particularly demand attention as any security breach is potentially catastrophic. The integrity of ML results is paramount, especially for applications that are moving towards using cloud-based computing for any-time, any-place data access.

Recent research has demonstrated that ML algorithms can be compromised through poisoning attacks, a class of causative attacks where carefully-crafted malicious instances are added to the training dataset. In the healthcare context, such attacks are particularly relevant because while modifying the existing training data might be challenging, adding new data is generally quite easy. The consequences are dire: hindrance of a diagnosis may have life-threatening consequences due to delayed treatment, while false positive classifications may cause patient distress and erode trust in the system.

This literature review discusses the work of Mozaffari-Kermani et al. (2015) which presents a systematic, algorithm-independent approach for launching poisoning attacks on a variety of ML algorithms and healthcare datasets, as well as corresponding countermeasures.

Background and Related Work

Taxonomies of Adversarial Machine Learning

Barreno et al. (2006, 2010) established taxonomies for attacks against ML systems and countermeasures. Two main categories of security attacks have been considered:

• Exploratory attacks: Exploit existing vulnerabilities without modifying the training process

• Causative attacks: Modify the training process, typically by modifying the training dataset

Poisoning attacks are a subset of causative attacks where malicious instances are added to the training dataset while leaving the rest intact.

Prior Poisoning Attack Research

Biggio et al. (2012) investigated poisoning attacks against Support Vector Machines (SVMs) based on increasing the classifier's test error. Their work depends on the gradients of dot products of points in the input space, evaluated using a classical handwritten digit recognition dataset. In exemplary experimental runs, a single attack data point resulted in classification error rising from initial error rates of 2–5% to 15–20%.

Rubinstein et al. (2009) and Kloft and Laskov (2010) proposed poisoning attacks in the feature space evaluated for intrusion detection scenarios using real HTTP traffic data. Their attack was successful with only up to 35% of initial data points overwritten, and required attacker control of only 5–15% of traffic.

Robust Classifiers and Countermeasures

Machine learning methods that account for data manipulation by adversaries (robust classifiers) have been investigated by Biggio et al. (2011) and Globerson and Roweis (2006). Adversarial machine learning has gained attention as an emerging field for analyzing vulnerabilities, developing benchmarking methods, and presenting countermeasures (Dalvi et al., 2004; Huang et al., 2011; Bruckner et al., 2012).

Mathematical Framework

Notation and Definitions

Let the original dataset be:

$$D \in (\chi, \Upsilon)$$

with N instances, where:

• χ represents an instance's attributes

• Υ represents the class label

• $x_i, 1 \leq i \leq N$: i-th instance

• $x_i^j, 1 \leq i \leq N, 1 \leq j \leq M$: j-th attribute value of i-th instance

• $\chi^j, 1 \leq j \leq M$: N0__-th attribute

• N1__: N2__-th class label

• N3__: number of attributes

• N4__: number of instances

The manipulated dataset is denoted as:

N5__

with N6__ instances, where N7__ is the number of added malicious instances.

Attack Generation Algorithm

For each attribute N8__, the algorithm analyzes the training dataset to compute statistics relating possible values of N9__ to class labels. The range of χ0__ is divided into χ1__ bins (where χ2__ is a specified constant). For each bin χ3__:

χ4__

are the χ5__ subsets of χ6__. We calculate:

• χ7__: number of entries in χ8__ corresponding to the attacked class

• χ9__: number of entries in Υ0__ corresponding to the attacking class

The weights are assigned as:

Υ1__

The attribute probabilities are computed as:

Υ2__

A weighted pseudorandom function Υ3__ selects attribute value Υ4__ based on these probabilities. The malicious instance candidate is:

Υ5__

Attack Algorithm

Algorithm 1: Algorithm-independent attacks

Input: Dataset D ∈ (χ, Y) with N instances, validation dataset V, number of iterations I

Output: Maliciously manipulated dataset D' ∈ (χ, Y) with N + N' instances

Begin

Assign D' ← D

for Υ6__ to N' do

for Υ7__ to I do

Use Algorithm 2 to generate malicious instance candidate i

Add candidate to D' to create temporary training set D' with N + k instances

Build model using D' and record classification accuracy on V as A_i

end for

Select instance i such that A_i = min(A_i), Υ8__

Add instance i to D'

end for

End

Return: D' ∈ (χ, Y)

Algorithm 2: Deriving a malicious instance candidate

Input: χ^j, Υ9__ and Y_i, $x_i, 1 \leq i \leq N$0__, g bins

Output: χ_{N+1}, Y_{N+1} (malicious instance candidate)

Begin

for $x_i, 1 \leq i \leq N$1__ to M do

for $x_i, 1 \leq i \leq N$2__ to g do

Calculate η_{k,j} and η'_{k,j}

Assign W_k ← η_{k,j}

end for

Compute attribute probabilities P_k = W_k / Σ_{$x_i, 1 \leq i \leq N$3__}^{g} W_i, $x_i, 1 \leq i \leq N$4__ to g

Weighted function S selects attribute value α_j pseudorandomly based on P_k

end for

End

Return: Malicious instance candidate χ_{N+1} = {α_j, $x_i, 1 \leq i \leq N$5__}, Y_{N+1} = Attacking class

Countermeasure Metrics

Two accuracy metrics are considered:

Correctly Classified Instances (CCI):

$x_i, 1 \leq i \leq N$6__

Kappa Statistic:

$x_i, 1 \leq i \leq N$7__

where $x_i, 1 \leq i \leq N$8__ is the observed agreement and $x_i, 1 \leq i \leq N$9__ is the expected agreement by chance.

Countermeasure Algorithm

Algorithm 3: Countermeasure against poisoning attacks

Input: Evolving Dataset D ∈ (χ, Y) with N + X instances, validation dataset V

Output: Attack flag AF

Begin

Set a threshold for accuracy metrics (e.g., CCI or Kappa statistic)

Obtain accuracy metrics for V after building model through D

While accuracy metrics have not reached threshold, compute accuracy metrics

If any threshold is reached, set i0__

End

Return: AF

Experimental Evaluation

Machine Learning Algorithms

Six ML algorithms were evaluated:

1. BFTree (Best-first decision tree): Tree-based with binary splits

2. Ridor (Ripple-down rule learner): Rule-based through knowledge acquisition

3. NBTree (Naive Bayes decision tree): Decision tree with naive Bayes classifiers at leaves

4. IB1 (Nearest-neighbor classifier): Normalized Euclidean distance-based

5. MLP (Multilayer Perceptron): Feedforward artificial neural network

6. SMO (Sequential Minimal Optimization): Support vector machine-based

4.2 Healthcare Datasets

Five medical datasets from UCI's machine learning repository were used:

Dataset	#Instances	Attributes	Attribute Types
Thyroid Disease	7104	21	Numeric
Breast Cancer	699	10	Nominal
Acute Inflammations	120	6	Numeric (1), Nominal (5)
Echocardiogram	132	12	Numeric (10), Nominal (2)
Molecular Biology	3190	61	Nominal

Attack Results

The attacks were evaluated for both fixed and evolving datasets. In the fixed case, the training dataset is constant during the attack. In the evolving case, other users can add authentic data during the attack, with the number of added authentic instances assumed equal to the number of added malicious instances.

Key findings:

• Attacks are successful across all datasets and algorithms

• Misclassification increases as the number of added malicious instances increases

• Attacks are effective for evolving datasets, with only slightly lower misclassification than fixed datasets

• Setting i1__ iterations in Algorithm 1 appears sufficient; increasing to i2__ improved misclassification by less than 1%

Vulnerability ranking (most to least vulnerable):

For Thyroid Disease (30% added):

i3__

For Breast Cancer (30% added):

i4__

SMO was found to be the most robust algorithm overall.

4.4 Comparison with Pseudorandom Attacks

After adding 30% malicious instances, the effectiveness of the proposed attack versus pseudorandom attacks was:

Algorithm	Proposed Attack	Pseudorandom Attack
MLP	26%	16%
BFTree	23%	15%
Ridor	22%	13%

IB1	18%	12%
NBTree	16%	9%
SMO	16%	10%

Attack Variants

1. Attacking Without Access to Training Dataset:

Attackers construct a proxy training dataset drawn from the same distribution as the original dataset by presenting artificial test instances and observing responses. For BFTree and SMO, only 1–2% and 2–3% change in attacking class misclassification was observed, respectively.

2. Attacking Unknown Machine-Learning Algorithms:

When the attacker does not know the ML algorithm, Algorithm 1 is modified by eliminating the inner loop (no retraining/validation). Attacks still succeed, though with 8–10% lower effectiveness for the Breast Cancer dataset.

3. Adapting Attacks to Real Patterns:

Real attack patterns can be created from malicious datasets if the feature extraction scheme is known. For Breast Cancer, malicious attribute vectors can be projected back to synthetic clinical reports exhibiting maliciously chosen tissue characteristics.

4. Attacking n-Way Classification:

For a four-way classification on the Hypothyroid dataset (classes: negative, compensated-hypothyroid, primary-hypothyroid, secondary-hypothyroid), after adding 20% malicious instances:

Class	Before	After
1 (attacked)	92%	80%
2 (attacking)	4%	18%
3 (attacked)	2%	1%
4 (attacked)	2%	1%

Countermeasure Effectiveness

For setting a threshold value for CCI, a conservative rule of thumb is to allow a 10% deviation from the original value. With this threshold:

• All attacks are detected after adding 30% malicious instances

• Except for two cases for Breast Cancer, none are detected after adding 15% malicious instances

The threshold trades off countermeasure sensitivity to attacks with the likelihood of false alarms due to natural variations in benign data.

Discussion

Key Contributions

1. Systematic attack approach: A generic, algorithm-independent poisoning attack scheme applicable to both fixed and evolving datasets, requiring only statistics of the training dataset (or a surrogate dataset).

2. Comprehensive evaluation: Six ML algorithms and five healthcare datasets were used to establish attack effectiveness.

3. Countermeasures: Defense mechanisms based on monitoring deviations in accuracy metrics (CCI, Kappa) and the number of instances added to the training dataset.

Implications for Healthcare Security

The success of poisoning attacks on healthcare ML systems has serious implications:

• Patient safety: Misclassification of diagnoses can lead to delayed treatment or unnecessary interventions

• System trust: Successful attacks may cause users to distrust and abandon ML systems

• Data integrity: The ease of adding malicious data (versus modifying existing data) makes poisoning attacks particularly relevant

5.3 Limitations and Future Work

• Generating real patterns from feature-space attacks remains an open problem

• The countermeasure threshold selection is context-dependent and requires dynamic tailoring

• Attacks that are resistant to the proposed countermeasures need to be developed

• Extension to similarity-based classification approaches requires further investigation

Conclusion

This literature review has examined the systematic poisoning attacks on machine learning in healthcare as presented by Mozaffari-Kermani et al. (2015). The work demonstrates that:

1. Poisoning attacks can be successfully mounted across a wide range of ML algorithms and healthcare datasets

2. Attacks are effective even without knowledge of the ML algorithm or training dataset

3. Evolving datasets present a tougher problem for attackers but do not eliminate attack effectiveness

4. Countermeasures based on accuracy metric monitoring can detect attacks, though with trade-offs between sensitivity and false alarms

The findings underscore the need for robust defenses in healthcare ML systems and suggest that devising attacks capable of circumventing the proposed countermeasures is an important direction for future research.

PAPER-5 –MACHINE LEARNING TECHNIQUES FOR DETECTING AND PREVENTING NETWORK LAYER DOS ATTACK

Abstract

The increasing reliance on computer networks has elevated the threat landscape of network layer Denial of Service (DoS) attacks. These attacks continue to disrupt services, cause data loss, and damage organizational reputation. Machine learning (ML) approaches have demonstrated significant promise in detecting and preventing such attacks. This literature review presents an overview of contemporary research on ML approaches to defend against network layer DoS attacks. We examine the fundamental taxonomy of network layer DoS attacks, explore supervised, unsupervised, reinforcement, and hybrid learning approaches, and conduct a comparative analysis of their effectiveness. We identify key findings that although ML approaches to DoS detection achieve detection accuracies exceeding 99% in controlled settings, there are also issues with adversarial robustness, concept drift, and real-time deployment. Additionally, we identify future research directions such as federated learning, explainable AI, and integrated defense frameworks.

Introduction

The dawn of the millennium witnessed an unprecedented increase in both the scale and sophistication of network layer attacks. As computer networks became integral to commercial, governmental, and personal operations, malicious actors developed increasingly sophisticated techniques to disrupt network services. The main objective of these attacks is to render network resources unavailable to legitimate users by flooding infrastructure with overwhelming traffic or resource-intensive requests.

Network layer DoS attacks exploit vulnerabilities in the Internet Protocol (IP) stack, targeting the fundamental mechanisms that enable communication between different networks. These attacks pose severe threats to the reliability, accessibility, and protection of interconnected systems, with adverse consequences for both enterprises and individuals (Tayyab et al., 2020; Xing & Wenye, 2006).

The Open Systems Interconnection (OSI) model provides a conceptual framework for understanding network communication, consisting of seven layers, each specifying defined protocols and functions. The network layer, responsible for logical addressing and routing services, represents a critical attack surface for DoS attacks. There has been increasing interest in exploring machine learning approaches to detect and alleviate DoS attacks, as they have the capability to analyze historical network traffic patterns and identify anomalous activities (Verma et al., 2022; Gebremariam et al., 2023).

This literature review presents an overview of contemporary research on machine learning approaches for network layer DoS attack detection and prevention. We examine the fundamental taxonomy of attacks, explore various machine learning methodologies, provide comparative analysis, and identify open research challenges.

Network Layer DoS Attacks: Taxonomy and Impact

Definition and Classification

A DoS attack refers to a malicious effort to block or deny authorized users from accessing network resources by overwhelming them with traffic or exploiting communication protocol vulnerabilities. When multiple compromised systems are

used to flood a targeted resource, the attack is classified as a Distributed Denial of Service (DDoS) attack (Patel et al., 2020; Subbulakshmi et al., 2013).

Network layer DoS attacks specifically target the functionality of the IP protocol stack, overwhelming network infrastructure with floods of traffic, requests, or packets. The objective is to deplete network resources, disrupt connectivity, or cause target systems to crash or freeze (Baarzi, 2021).

Common Attack Types

IP Spoofing: Attackers falsify source IP addresses to bypass security measures relying on IP filtering or authentication. This technique allows attackers to conceal their identity and redirect responses away from their actual location.

ICMP Flood: The attacker inundates the target system with Internet Control Message Protocol (ICMP) packets, consuming network bandwidth and processing resources. This attack exploits the fundamental diagnostic functions of ICMP to overwhelm target systems.

Smurf Attack: This attack exploits Internet Group Management Protocol (IGMP) vulnerabilities by sending ICMP echo requests to broadcast addresses, causing all hosts on a network to respond to the target system. The amplification effect multiplies the attack's impact.

Attack Methods and Strategies

Network layer DoS attacks employ various methods and strategies:

Flood Attacks: The attacker inundates the target with enormous quantities of packets, requests, or connections to consume resources and cause system failure.

DDoS Attacks: Attackers utilize networks of compromised computers (botnets) to initiate synchronized attacks against intended targets.

Amplification Attacks: These exploit vulnerabilities in protocols such as DNS, NTP, or SNMP to generate large traffic volumes that overwhelm target systems.

Impact and Consequences

Network layer DoS attacks produce severe impacts on target systems and network infrastructure:

Degraded Network Performance: Attack-generated traffic causes congestion, packet loss, and delay, resulting in diminished network performance.

Service Disruption: Attacks disrupt network service availability, making access difficult or impossible for legitimate users.

Data Corruption: System crashes or unavailability during attacks can cause data loss or corruption.

Machine Learning Approaches for DoS Detection

Overview of ML Paradigms

The utilization of Machine Learning approaches has become increasingly prevalent in network layer DoS detection due to their ability to evaluate large data volumes in real-time. Several ML techniques have been proposed for DoS detection, including supervised, unsupervised, reinforcement, and hybrid learning approaches.

Supervised Learning Techniques

Supervised learning utilizes labeled data to construct models that distinguish between legitimate and malicious network traffic.

Support Vector Machines (SVM): SVM has gained significant attention in ML research due to its excellent classification results. Through supervised learning, SVM can perform both classification and regression tasks (Drucker et al., 1999). Allagi et al. (2021) proposed a robust approach using an autoencoder based on SVM, achieving 99.32% accuracy on the CICIDS dataset for DoS attack detection. Al Duwairi et al. (2020) developed a DDoS attack detection model combining SVM classification methods, while Subbulakshmi et al. (2013) proposed an enhanced SVM (ESVM) approach for attack identification.

Random Forest (RF): Krishnan and Singh (2021) introduced CSBW-Random Forest, which outperforms existing methods regarding recall, accuracy, F1-score, and precision. The proposed technique achieves a rating of 0.997 and demonstrates significantly improved performance compared to previous literature. Experiments demonstrate superior performance regarding incorrect positives, correct positives, and incorrect negatives.

K-Nearest Neighbors (KNN): KNN assigns classes based on proximity principles. As a lazy learning model, it requires less training time but more prediction time. The model can serve as both classifier and regressor, with the parameter K adjustable as a hyperparameter. However, KNN exhibits sensitivity to magnitude, high computational cost, and impracticality for large datasets (Krishnan & Singh, 2021).

Pande et al. (2021) employed feature selection techniques to reduce features from 41 to 11 or 17, including LDA, PCA, RFE, and univariate feature selection. KNN classification achieved 99.87% and 99.11% accuracy using 17 and 11 features respectively with RFE. Singh and Virmani (2021) evaluated KNN performance across multiple datasets, achieving ROC scores of 0.985 on WSN-DS and precision rates of 97.31% (WSN-DS), 94.79% (NSL-KDD), and 99.61% (KDD-Cup99).

Bayesian Networks: Nishanth and Mujeeb (2020) proposed a technique utilizing Bayesian inference for SYN flooding attacks in wireless ad hoc networks. The approach models SYN traffic statistically, constructing a Bayesian network considering pertinent variables such as incoming SYN packet quantities, outgoing SYN-ACK packets, and time intervals. The method demonstrates equivalence to exponential weighted moving average (EWMA) for single variables, providing effective detection with high accuracy and low false detection rates.

Artificial Neural Networks (ANN): Shrivastava and Sharma (2020) proposed a two-layer model using statistical analysis for feature selection relevant to less frequent attacks, training a multi-classifier artificial neural network on separate datasets. The technique achieved 99.34% detection accuracy for minority attacks. Mariam and Negash (2021) evaluated Naive Bayes, AdaBoost, J48, and ANN for SYN flood attack detection using ethio telecom network data, finding J48 superior.

Unsupervised Learning Techniques

Unsupervised learning methods, including clustering and anomaly detection, identify patterns and outliers in network traffic without requiring labeled data. These approaches are particularly valuable for detecting novel attack vectors not present in training datasets.

Reinforcement Learning

Q-learning and Deep Q-Networks (DQNs) represent reinforcement learning algorithms that train themselves to make decisions based on environmental feedback. These approaches enable adaptive defense mechanisms that evolve in response to changing attack patterns.

Hybrid Approaches

Hybrid approaches combine multiple ML methods to enhance detection precision and reduce false positive occurrences. Examples include combining supervised and unsupervised learning, ensemble methods, or integrating ML with rule-based methods.

Machine Learning Techniques for DoS Prevention

Preventive Frameworks

Beyond detection, ML techniques enable proactive prevention of DoS attacks. Feng et al. (2023) proposed multi-party EdDSA protocols for partially trustworthy and untrustworthy settings, providing secure global state maintenance without distributed hashing. The protocols demonstrate efficiency, identifiable abort, and high compatibility with EdDSA-based cryptocurrencies, with testing on Alibaba cloud servers confirming significantly faster performance than recent threshold EdDSA protocols.

LogDoS System

Al Duwairi et al. (2020) developed LogDoS, a DDoS prevention system utilizing GET messages and logging-based filtering to establish inter-domain routing. The system inserts GET messages at Information-Centric Networking (ICN) routers along sender paths to eliminate packets not responding to preceding requests.

Combining Named Data Networking (NDN) and PID-based ICNs, this hybrid method generates potent resolution for preventing data flooding attacks.

Integrated Defense Mechanisms

Contemporary research emphasizes integrated approaches combining multiple defense layers. These frameworks reduce network administrator workload while enhancing DoS attack prevention efficiency through automated detection and response mechanisms.

Comparative Analysis of ML Approaches

Performance Comparison

Comparative analysis reveals distinct trade-offs between detection accuracy and computational overhead:

Technique	Accuracy	Detection Rate	Computational Overhead	Scalability
SVM	High (99.32%)	High	High	Moderate
Random Forest	High (99.70%)	High	High	Good
KNN	High (99.87%)	Moderate	Low	Poor
Naive Bayes	Moderate (97.50%)	Moderate	Very Low	Excellent
ANN	High (99.34%)	High	Medium	Good
Decision Trees	Moderate (98.20%)	Moderate	Low	Good

Trade-offs and Considerations

SVM and Random Forest demonstrate high detection accuracy but with higher computational overhead. KNN and Neural Networks exhibit lower overhead but reduced accuracy. ANN and Random Forest prove effective for DoS attack prevention with low overhead and good scalability. SVM and Random Forest achieve higher detection rates for network layer DoS attacks but with increased overhead. Decision Trees and Naive Bayes offer lower overhead but reduced detection rates.

Overall, ML-based approaches effectively enhance network security against DoS attacks with technique selection depending on specific deployment requirements, available computational resources, and attack characteristics.

Open Research Challenges and Future Directions

Current Challenges

Contemporary ML-based techniques for network layer DoS detection and prevention face several challenges:

Limited Training Data: Availability and diversity of high-quality training data remain constrained, limiting model generalizability.

Sophisticated Attack Detection: Detecting advanced, multi-vector attacks proves difficult with current approaches.

False Positive Rates: High false-positive rates or low detection rates persist in operational deployments.

Interpretability: Limited model interpretability hinders security analyst trust and adoption.

Adversarial Vulnerabilities: ML models remain susceptible to adversarial attacks designed to evade detection.

Emerging Research Trends

Robust and Adaptive Models: Developing more robust and adaptive models using deep learning techniques to address evolving attack vectors.

Explainable AI: Exploring explainable and interpretable ML models to enhance analyst trust and decision-making.

Transfer Learning: Investigating transfer learning approaches to leverage knowledge across domains and reduce training data requirements.

Federated Learning: Exploring federated learning methodologies for privacy-preserving collaborative defense.

Future Research Opportunities

Dataset Development: Creating large, diverse, and representative datasets for training and evaluation.

Ensemble Methods: Exploring ensemble learning methods to improve detection accuracy and robustness.

Adversarial Training: Investigating adversarial training techniques to enhance model resilience.

Integration: Integrating ML-based techniques with complementary security mechanisms for comprehensive defense.

Conclusion

This literature review presents an overview of machine learning techniques for detecting and preventing network layer DoS attacks. We examine the fundamental taxonomy of attacks, explore various machine learning methodologies, provide comparative analysis, and identify open research challenges.

The study emphasizes the need for more robust and adaptive machine learning models capable of detecting sophisticated attacks while reducing false-positive rates. Emerging research trends include exploring explainable and interpretable models, transfer learning and federated learning methodologies, and integrating ML techniques with complementary security mechanisms.

This synthesis of contemporary research contributes to an understanding of state-of-the-art ML-based techniques for network layer DoS attack detection and prevention, providing insights for future research directions. As network threats continue to evolve in sophistication, the development of effective, efficient, and interpretable ML-based defense mechanisms remains a critical research priority.

PAPER-6 –BEHAVIOUR –INFORMED APPROACH TO MITIGATE DATA POISONING ATTACKS FOR MACHINE LEARNING

Introduction

This literature review provides an insight into the existing knowledge about adversarial machine learning (AML) - particularly, the concept of data poisoning attacks and strategies to defend against them. It sets up the theoretical foundation of literature to explore and understand the threats posed by data poisoning attacks to traditional and deep learning models, and identify gaps in the existing body of knowledge. Such gaps motivate this research, whose objective is to secure multiclass machine learning against data poisoning attacks through an attack-agnostic, behaviour-informed approach.

The review begins with an overview of the basics of supervised machine learning and taxonomy of adversarial attacks. It proceeds to the state-of-the-art of data poisoning attacks, ranging from label-flipping to clean-label and subpopulation. It then turns to the state-of-the-art of mitigation strategies, broadly classified into data sanitisation, adversarial training, and model hardening. Finally, the review synthesises the insights gained to highlight research gaps that are addressed by this research.

Background: Machine Learning and Adversarial Threats

Supervised Machine Learning

Supervised machine learning (ML) is a form of classification, where a model learns a mapping function \(f: X \to Y\) from a labelled dataset \(D = \{(x_i, l_i)\}_{i=1}^n\). Here, \(x_i\) represents the feature vector, and \(l_i\) is the corresponding label. The goal is to find a mapping function to predict labels of unseen instances by minimising a loss function \(\mathcal{L}(f(x), l)\). The thesis

focuses on classification tasks, which are typically of two types: binary classification (two-class) and multiclass classification (more than two class). The process, as depicted in Figure 2.1, begins with data gathering, followed by feature engineering, model training and testing, and deploying the model into production.

The thesis analyses six traditional supervised algorithms, namely, Support Vector Machines (SVM), Decision Trees (DT), Random Forests (RF), Gaussian Naive Bayes (GNB), K-Nearest Neighbours (KNN), and Neural Networks (NN) with Perceptron and Multilayer Perceptron (MLP). The selection of these algorithms covers a wide range of applications and challenges, ranging from parametric to non-parametric models (Dixit et al., 2018; Sasmono et al., 2021; Kamath et al., 2018). Understanding the decision mechanisms of each algorithm helps in identifying their vulnerabilities to poisoning attacks as each algorithm processes the input differently (Paracha et al., 2024a).

Adversarial Machine Learning: A Taxonomy of Attacks

Adversarial machine learning (AML) studies the vulnerabilities in learning algorithms to adversarial examples and attacks, and their corresponding mitigation strategies. The thesis (Section 2.2.9) categorises attacks based on the development phases of a machine learning pipeline as data poisoning (training phase), evasion (testing phase), model inversion (deployment phase), and membership inference (deployment phase). Each attack has different goals and challenges as summarised in Figure 2.8.

This review focusses on data poisoning attacks, which involve manipulating training data to compromise the learning process. As described in the thesis (Section 2.2.9.1), data poisoning attacks are broadly classified into availability attacks, which aim to degrade overall model performance and integrity attacks,

which trigger misclassification for specific inputs (Zhao and Lao, 2022a; Carlini and Terzis, 2021). Attackers may target labels (label poisoning) or features (clean-label poisoning) (Gupta et al., 2023b; Jagielski et al., 2021). The effectiveness of attacks depends on the level of knowledge an attacker has about the model: white-box (complete knowledge), grey-box (partial knowledge), and black-box (no knowledge) (Patterson et al., 2022; Wang et al., 2021a; Yu and Sun, 2022).

State-of-the-Art in Data Poisoning Attacks

Literature witnesses a prolific growth of studies focussing on data poisoning attacks, mostly targeting deep learning (DL) models. However, traditional ML, particularly multiclass classification, appears to lack sufficient exploration (Paracha et al., 2025a). This section presents an overview of notable data poisoning attacks analysed in the literature.

Attacks on Deep Learning Models

Biggio et al. (2011) pioneered poisoning attacks against SVM classifiers, which laid the foundation of AML. Since then, a myriad of sophisticated attacks have been proposed. For instance, Munoz-Gonzalez et al. (2017) proposed a gradient-based poisoning attack against neural networks, which was later extended to multiclass classification (Huang et al., 2020). MetaPoison (Huang et al., 2020) is a bi-level optimisation-based attack against neural networks that crafts clean-label poison and has shown to be successful against Google Cloud AutoML. On the other hand, the Subpopulation Attack (Jagielski et al., 2021) injects a cluster of poisoned data points such that it becomes difficult to detect using traditional outlier detection. Other notable attacks are the Convex Polytope Attack (Zhu et al., 2019b), the Bullseye Polytope Attack (Aghakhani et al., 2021), and the Poison Frog Attack (Shafahi et al., 2018).

These attacks have demonstrated their efficacy in compromising DL models by exploiting minor manipulations in training data. However, since these attacks rely on gradient information, they cannot be directly applied to traditional ML models, which lack a gradient-based learning mechanism. This observation motivates this research, which focusses on securing traditional ML against poisoning attacks.

Attacks on Traditional Machine Learning Models

Traditional ML models appear to be less explored in the context of poisoning attacks. Yerlikaya and Serif Bahtiyar (2022) performed an empirical evaluation of label-flipping attacks against six ML algorithms to identify their relative effectiveness in poisoning these algorithms. Their results revealed that KNN and RF were resilient to poisoning attacks. However, Yerlikaya and Serif Bahtiyar (2022) did not examine the mechanisms through which these differences emerge. Jagielski et al. (2018) proposed a statistically-based poisoning attack against linear regression and a corresponding TRIM defence that utilises residual filtering. However, this attack and defence mechanism are effective only for regression tasks and not for classification tasks.

The literature is largely silent about the underlying differences in the decision mechanisms of different classification algorithms. Moreover, most of the literature evaluates poisoning attacks in terms of accuracy and F1-score while neglecting the underlying features altered due to poison (Paracha et al., 2024a).

The Outlier-Oriented Poisoning (OOP) Attack

The Outlier-Oriented Poisoning (OOP) attack (Paracha et al., 2025a) is a novel attack against multiclass ML, which flips the labels of outlier samples. Unlike previous attacks that randomly flip labels, the OOP attack targets outliers, which are data points that lie far away from the decision boundary. The attack is effective as evidenced by the experimental results in Chapter 4. It is noteworthy that the OOP attack requires only limited adversarial capabilities (grey-box setting) and is generalisable across traditional multiclass ML, namely SVM, DT, RF, GNB, KNN, and MLP. The success of the OOP attack highlights the need for a defence mechanism that is attack-agnostic.

State-of-the-Art in Mitigating Data Poisoning Attacks

The literature presents a variety of strategies to defend against data poisoning attacks. Overall, these strategies fall under three broad categories: data sanitisation, adversarial training, and model hardening.

Data Sanitisation

Data sanitisation refers to preprocessing the training dataset to eliminate poisoned data points before training the model. Chan et al. (2018a) proposed a data sanitisation approach based on the complexity of the data, whereby the entire dataset is clustered and poisoned points are removed. Similarly, Paudice et al. (2018b) proposed label sanitisation, which uses k-NN to label sanitisation to detect and correct mislabeled data points. Other notable data sanitisation strategies are the

RONI (Reject on Negative Impact) (Barreno et al., 2008) and TRIM (Jagielski et al., 2018).

Data sanitisation has several limitations. First, removing points from the training data may lead to underfitting, especially for neural networks (Paudice et al., 2018b). Second, most sanitisation strategies are suitable for binary classification. Finally, sophisticated attacks, such as the Subpopulation Attack (Jagielski et al., 2021), can circumvent sanitisation strategies by inserting poisoned cluster(s) in the training data. A deeper analysis of the impact of data poisoning on the model's behaviour is required to design effective defences (Paracha et al., 2024a).

Adversarial Training

Adversarial training aims to improve the robustness of the model by training it on a mixture of clean and poisoned data. It has been widely used as a defence against evasion attacks, and recently, against data poisoning attacks. For instance, Geiping et al. (2021) proposed an adversarial training to maximise adversarial loss to detect poisoned data points during training. Tao et al. (2021) proposed a robust adversarial training that utilised both robust and non-robust features. Shafahi et al. (2020) proposed a universal adversarial training to defend against universal adversarial perturbations.

However, adversarial training is primarily applied to DL models that utilise gradient-based optimisation. Traditional ML algorithms, such as SVM, RF, and DT, do not use gradient learning mechanism and, therefore, are not amenable to adversarial training. This observation is supported by the results presented in Chapter 4 (Paracha et al., 2024b). The thesis proposes Feature-Oriented Adversarial Training (FORT) (Paracha et al., 2025b), which is suitable for any ML model that uses feature importance scores to generate adversarial examples.

Model Hardening

The goal of model hardening is to increase the margin between true labels and poisoned labels, thus increasing the difficulty of poisoning. Tao et al. (2022a) proposed a model hardening strategy that increases the distances between classes to withstand label-flipping attacks. Apruzzese et al. (2020a) hardened a Random Forest model to defend against poisoning attacks in an Intrusion Detection System (IDS). However, most of the strategies are tailored to specific attacks and may not generalise to other attacks.

SecureLearn: A Behaviour-Informed Defence

The thesis proposes SecureLearn, a two-layer defence approach that combines improvised data sanitisation with Feature-Oriented Adversarial Training (FORT) (Paracha et al., 2025b). It is attack-agnostic and achieves high performance on multiple ML models and attacks. SecureLearn improves the performance of the model by first sanitising the training data to remove or correct poisoned points and then training the model on adversarial examples generated using feature importance scores. It is informed by the deep behavioural analysis of Chapter 3 (Paracha et al., 2024a), which identified specific features and algorithms that are susceptible to poisoning. SecureLearn is generalisable, attack-agnostic, and effective, as demonstrated by the 3D evaluation matrix proposed in Chapter 5.

Research Gaps and Motivation

The review has identified several research gaps, which motivate this research:

1. Limited Focus on Traditional Multiclass ML: Data poisoning attacks are primarily studied in the context of DL. Traditional multiclass ML lacks sufficient attention despite its wide applicability (Paracha et al., 2025a).

2. Lack of Behavioural Analysis: Literature primarily evaluates the performance of poisoned models without analysing the features and decision mechanisms altered by poisoning (Paracha et al., 2024a).

3. Limitations of Existing Defences: Data sanitisation suffers from underfitting and cannot withstand sophisticated attacks (Paudice et al., 2018b; Jagielski et al., 2021). Traditional ML algorithms are not amenable to adversarial training (Paracha et al., 2024b).

4. Practicality Under Limited Adversarial Capabilities: Most poisoning attacks are evaluated under the assumption that the attacker has full knowledge about the victim model. There is a need to understand the practicality and impact of poisoning attacks under limited adversarial capabilities (Paracha et al., 2025a).

5. Need for Comprehensive Evaluation: Literature primarily evaluates defences against a few attacks, and performance of different defences are not compared (Paracha et al., 2025b).

Summary

This literature review has presented an overview of data poisoning attacks and their mitigation strategies. It has highlighted the evolution of data poisoning attacks from label-flipping to clean-label and subpopulation attacks, primarily targeting DL models. It has critically evaluated the existing mitigation strategies and identified their limitations in addressing the vulnerabilities of traditional multiclass ML models. The review has also identified gaps in the literature, which motivate this research. Specifically, this research addresses the lack of focus on

traditional multiclass ML and the need for comprehensive evaluation of mitigation strategies against a variety of attacks. The review has informed the objectives of this research, which focusses on understanding the vulnerabilities of multiclass ML models through a deep behavioural analysis, proposing novel attacks and a behaviour-informed defence approach, and evaluating defence approaches using a comprehensive evaluation matrix.

PAPER-7 –POISONING ATTACKS AGAINST MACHINE LEARNING

Introduction

The rise of machine learning (ML) and artificial intelligence has created an urgent need to understand their security implications. Oprea, Singhal, and Vassilev (2022) examine poisoning attacks against ML training pipelines from the perspective of adversarial ML. This paper reviews their work, focusing on their contributions to the field, their taxonomy of poisoning attacks, the case studies they analyze, and the challenges they identify for building secure ML systems.

Contextualizing Poisoning Attacks Within Adversarial ML

Oprea et al. (2022) classify poisoning attacks together with evasion attacks as major threats posed by adversarial examples to ML systems. They draw a distinction between the two by noting that poisoning occurs during training while evasion happens at inference time (Figure 1). This difference in timing has implications for the threat model and the techniques for countering such attacks. The history of academic research on poisoning attacks begins with the work of Perdisci et al. (2006), who demonstrated how worm signature generators could be targeted to resist detection. This work, along with the Microsoft industry survey cited by the authors (Kumar et al., 2020), shows that poisoning attacks have long been a concern for the research community and industry practitioners. It also serves to contextualize poisoning attacks within adversarial ML by demonstrating that they are not unique to deep learning (DL).

The authors mention the adversarial ML taxonomy proposed by Tabassi et al. (2018) on behalf of the National Institute of Standards and Technology but refine it to better capture contemporary research on poisoning attacks. Their version of the taxonomy classifies attacks according to the attacker's goal, the information they have about the target model, their capabilities, and the input modality. By proposing a classification of attacks by subpopulation (Jagielski et al., 2021), the authors highlight an emerging trend that seeks to bridge the gap between targeted and availability attacks.

Taxonomy of Poisoning Attacks: A Critical Analysis

The classification of poisoning attacks proposed by the authors builds on prior work by incorporating findings on subpopulation attacks into the established attack taxonomy. It distinguishes between availability, targeted, backdoor, and subpopulation attacks (Figure 2). One of the major strengths of this approach is that it provides a vocabulary for discussing different types of poisoning attacks. It also recognizes the evolution of the field, exemplified by recent attacks that involve feature poisoning (clean-label attacks).

The attack taxonomy proposed by Oprea et al. (2022) helps to identify research trends in adversarial ML. First, the inclusion of clean-label attacks as a type of targeted attack indicates that researchers are becoming more concerned with scenarios in which the adversary does not have control over the training labels. Second, the authors' emphasis on subpopulation attacks suggests that the research community is realizing the challenges of detecting attacks that do not impact global model accuracy. This realization has influenced the choice of attack to evaluate defenses. For instance, the authors find that backdoor attacks pose a

greater threat than availability attacks because the former are harder to detect (Liu et al., 2018). In effect, the attack taxonomy helps the authors to operationalize the concept of a strong attacker.

Case Studies: Bridging Theory and Practice

The two case studies that illustrate how poisoning attacks work in practice are appropriate to the extent that they highlight the unique characteristics of attacks in different application domains. Severi et al. (2021) show that an adversary can circumvent the interpretability of tree ensembles to embed a backdoor in a malware classifier. More importantly, the attack methodology demonstrates that feature importance scores (SHAP values) can be used to identify the minimal input features required to trigger a backdoor. This finding has implications for the interpretation of SHAP values in other domains as a tool for discovering model weaknesses. Moreover, the attack's evaluation procedure, which involved poisoning only 1-2% of the training data and using an eight- to 32-feature trigger to compromise a model trained on 600,000 Windows PE files, is informative, illustrating an asymmetry between attacker and defender.

The analysis by Chen et al. (2021) and Li et al. (2021) also sheds light on unique challenges of poisoning attacks in natural language processing (NLP). Specifically, the requirement for attacks to preserve grammatical, syntactical, and semantic properties of the poisoned input documents complicates the task of creating effective triggers. The methods proposed by the authors (semantic-preserving triggers and generative language models) help to demonstrate why prior efforts to evaluate the security of NLP models against poisoning attacks have been inconclusive. Li et al. (2021) find that backdoors in transformer-based language

models are capable of evading detection across multiple NLP tasks, including toxic comment classification, machine translation, and question answering. The user studies performed to evaluate their findings add credibility to the claim that such attacks are undetectable by human inspection, which is an important consideration when comparing different methods of compromising NLP models.

Defenses and Their Limitations

The survey of existing defenses against poisoning attacks is a major strength of the paper, but it also serves to highlight what the authors perceive as the main technical challenges in this area of research. First, a survey of methods for sanitizing training data, robust optimization, and model inspection/repair clarifies the limitations of each approach. For instance, while data sanitization techniques offer strong guarantees, they also have the potential to filter out useful data. The authors' identification of certified defenses against label flipping attacks (Rosenfeld et al., 2020) is equally informative, as it illustrates the limited scope of existing defenses – label flipping is only one type of poisoning attack.

Perhaps, the most intriguing challenge posed by the work of Oprea et al. (2022) is Jagielski et al.'s (2021) impossibility result for subpopulation attacks, which assumes local decision-making. The importance of this limitation is qualified by the authors' note that, to date, most defenses have been evaluated on computer vision tasks (Severi et al., 2021). This observation also provides context for their focus on the trade-off between robustness and accuracy, which is a key theme in contemporary research on secure ML. This tension is also evident in their brief discussion of the value of certified defenses for safety-critical applications.

Conclusion and Future Directions

Oprea et al. (2022) examine poisoning attacks against ML training pipelines through the lens of adversarial ML. The authors make a number of significant contributions, including a taxonomy of poisoning attacks, a survey of existing defenses, and an analysis of how different attack vectors compare in terms of their impact on model accuracy and detectability. The most important implication of their findings is yet to be determined because the authors conclude that most defenses have been tested in computer vision and may not be effective in other domains. Future research should explore this line of inquiry by investigating the extent to which findings about one domain can be generalized to others.

Another important direction for future research is suggested by the authors' finding that defenses tend to reduce model accuracy while improving robustness against poisoning attacks. Such a trade-off between accuracy and robustness is hardly surprising, given the nature of the task, but it raises difficult questions about the technical feasibility of certified defenses against all types of poisoning attacks. This consideration adds credibility to the authors' suggestion that the field needs more research on the limitations of adversarial examples and the possibility of designing ML systems with built-in robustness.

PAPER-8 –MACHINE LEARNING BASED DETECTION AND MITIGATION OF DOS ATTACKS IN MANETs ESSAY

Literature Review

The increasing prevalence of Denial of Service (DoS) attacks on Mobile Ad-hoc Networks (MANETs) has attracted researchers' attention, who suggest using machine learning approaches to address the problem. This section presents a review of the literature on DoS attack detection and mitigation in MANETs, including the strengths and weaknesses of the studies.

DoS Attacks and Their Impact on MANETs

The authors of [14] explore various aspects of distributed denial of service (DDoS) attacks in mobile ad-hoc networks (MANETs) and propose countermeasures to mitigate these attacks. According to the authors, DDoS attacks pose a significant threat because they lead to a substantial decrease in network performance by overwhelming it with fictitious packets. To evaluate network performance under attack, the ad hoc routing protocols are simulated and implemented using the Network Simulator (NS3). In addition, detecting attacks is discussed; in particular, threshold timestamps and node response are considered indicators of DDoS. DoS detection and mitigation in MANETs is highlighted as a challenging area requiring improved solutions, and simulation tools and machine learning are seen as promising methods to address the problem. More practical and flexible approaches are required to respond to DDoS challenges, including designing intrusion detection systems, improving detection mechanisms, and implementing emerging technologies such as machine learning in existing frameworks.

In [3], the impact of distributed denial of service (DDoS) attacks on three routing protocols – Zone Routing Protocol (ZRP), Ad hoc On-demand Distance Vector (AODV), and Location-Aided Routing (LAR) – was evaluated. Using the DDoS Attack Simulation Model (DDoS) and Network Simulator 2 (NS-2), the authors found that network throughput decreased significantly (by 67.3% for AODV, 62.8% for LAR, and 57.1% for ZRP). In addition, AODV showed the highest increase in end-to-end delay (98.71%), with the other two protocols ranking second and third (97.1% and 96%). According to the results, attacks have a more severe impact on networks when the number of attackers increases, with more than ten attackers being sufficient to cause network damage. A comparison of the protocols under review was not provided, and future research directions included exploring alternative techniques for countering blackhole and greyhole attacks. Furthermore, researchers recommended developing hybrid protocols with built-in security features to address various types of attacks in MANETs.

Machine Learning Approaches for DoS Detection

Using a feed-forward back propagation neural network (FFBPNN), the authors of [15] proposed to improve attack detection by analyzing node behavior and energy characteristics. According to the results of testing the proposed solution, the malicious nodes were excluded due to differences in the energy consumption and delay compared to the nodes suspected of attacking. The article highlights the potential of using neural networks to identify patterns associated with DoS attacks, while the solution's practical implementation is limited to one machine learning model.

An intrusion detection system (IDS) that was designed to detect denial of service (DoS) attacks in MANETs and mitigate their impact is proposed in [16]. According to the authors, support vector machines (SVMs) used in the intrusion detection system (IDS) were effective in detecting and countering DoS attacks. In particular, the experimental results demonstrated that IDS could accurately detect attacks with a relatively short processing time regardless of network size and node mobility. The study recommends using SVM for intrusion detection in MANETs, noting its ability to generalize attack data with minimal processing overhead. The energy efficiency of the proposed solution is not addressed in the study.

Trust-Based and Monitoring Approaches

A novel approach to secure networks against DoS attacks called "monitoring detection and rehabilitation" is proposed in [17]. According to the results of the experiments, the authors managed to measure actual values to estimate the trustworthiness of sensor nodes. They also report that the solution performs better than the current "trust enhanced anonymous-on-demand routing protocol," evidenced by a 20.87% increase in packet delivery ratio (PDR). The use of a trust-based approach allows the solution to be effective; however, it does not use machine learning capabilities to predict attacks.

Using the sequence number, the authors of [19] proposed to detect black hole nodes by comparing this number with a threshold value established by a dynamically updated timer. According to the results, the solution also allows neighboring nodes to be alerted about the attacked node. The reliability of the solution is not evaluated; however, the need for dynamic threshold value determination and additional computational resources is noted. As a result, false

positives may arise due to the use of threshold-based methods. Therefore, it is recommended to use solutions that can learn network behavior patterns rather than rely on threshold-based solutions.

Blockchain and Collaborative Mitigation Frameworks

A distributed and secure DDoS cooperative framework called Cochain-SC was proposed in 2019 [18] to address the limitations of existing collaborative DDoS solutions. The authors used the Ethereum blockchain and smart contract technology to enhance the collaboration between Software-Defined Networking (SDN) domains. The SDN domains were protected against DDoS attacks both inside and outside the domains. To evaluate the effectiveness of the proposed solution, the authors used sFlow to measure traffic irregularity and the Intra Entropy-based Scheme (I-ES) to automatically identify traffic anomalies within each domain. In addition, the Intra-Domain Mitigation (I-DM) mechanism was used to mitigate illicit traffic in the SDN domain. The proposed framework's cost-effectiveness, flexibility, efficiency, and security were validated to demonstrate its effectiveness. The study identified the limitations of existing collaborative DDoS solutions, such as the inability to provide end-to-end protection. Using blockchain technology, Cochain-SC was highlighted as a promising approach; however, it is not suitable for MANETs because it does not consider energy conservation.

Comparative Analysis and Research Gaps

Based on the review of the literature, it is possible to identify gaps that the current study seeks to address. First, the majority of the reviewed articles focus on the use of either SVM [16] or FFBPNN [15] for DoS attack detection in MANETs; few studies attempt to compare the effectiveness of different machine learning techniques. Second, most of the reviewed literature does not consider energy consumption and other vital factors, such as scalability and network throughput, when evaluating the performance of proposed solutions. Third, while the reviewed studies address DoS attack detection and mitigation separately, there is a need for integrated solutions that provide a visualization and analysis of attack data in a simulated network environment.

The current study attempts to fill the identified gaps by proposing a comprehensive approach to DoS attack detection and mitigation in MANETs. In particular, the current study will (1) generate synthetic data that simulate both normal and abnormal traffic conditions; (2) train and test four machine learning models, namely SVM, Random Forest, Neural Network, and KNN; (3) evaluate the performance of these models using various performance metrics, including accuracy, energy consumption, scalability, resource utilization, and network throughput; and (4) visualize and mitigate the results of attacks in a simulated MANET environment.

PAPER-9 –DETECTING DDOS ATTACKS IN SOFTWARE –DEFINED NETWORKS USING FEATURE SELECTION METHODS AND MACHINE LEARNING MODELS

Introduction

Software-Defined Networking (SDN) has been advanced as a new paradigm to realize network flexibility, programmability and manageability. SDN promises to meet the demanding requirements of networks, such as bandwidth, management, cloud computing, and virtualization, unlike traditional networks [1]. By decoupling the control plane from the data plane, an SDN controller emerges as the "brain" that manages transmission devices [3].

Despite the architectural benefits, SDN is inherently susceptible to various security threats, including Distributed Denial of Service (DDoS) attacks [4,5]. The controller becomes an attractive target because any DDoS attack could saturate its processing capability, memory space, and bandwidth [6]. Additionally, the limited storage size of flow tables within OpenFlow switches renders them vulnerable to DDoS attacks [7].

This literature review aims to summarize the research on detecting DDoS attacks in networks based on feature selection methods and machine learning algorithms. In particular, we focused on the relevant articles identified in the previous section to understand the research gaps and highlight the contributions of the current study [20].

DDoS Attacks in SDN: Threat Landscape

DDoS Attacks Against the Controller

The controller is the central component of the SDN architecture. Hence, threatening its integrity could result in complete network downtime [5]. Kreutz et al. [5] identified at least seven distinct threat vectors in SDN, with DDoS being the most damaging. In this attack, the adversary's primary objective is to consume the controller's memory, processing power, and bandwidth [6].

In SDN, the controller communicates with data plane devices through the southbound interface, namely, the OpenFlow protocol. In a scenario where packets do not match any of the flow entries in the OpenFlow switches' flow tables (flow miss), the switches forward the packets to the controller for decision-making [8]. This mechanism could be leveraged by attackers to launch DDoS attacks by overwhelming the controller with packets, which slows down packet processing by the controller [8].

Alshamrani et al. [9] investigated the behaviors of misbehavior and new flow attacks in SDN by collecting traffic data from data plane devices and classifying the packets using machine learning algorithms, namely, support vector machine (SVM), J48, and Naive Bayes [9]. Likewise, Latah and Toker [10] designed a solution for detecting DDoS attacks against the controller by measuring packet rates reaching the controller. If the packet rate exceeded a predetermined threshold, an inspection unit was triggered to detect DDoS flooding attacks using SVM [10].

Li et al. [11] proposed a bidirectional recurrent neural network (RNN) covering all layers of the network structure for detecting and mitigating DDoS attacks [11]. This approach exhibited excellent detection accuracy in real-time detection but may not be suitable for large-scale networks with multiple controllers, leading to poor synchronization between controllers and degrading network performance.

DDoS Attacks Against the OpenFlow Switch

It is worth noting that the OpenFlow switch and its flow table are also vulnerable to DDoS attacks. In particular, the flow table contains crucial information, including administrative information, transmission rules, and access control information [12]. The main weakness in the OpenFlow switch is the limited storage size of its flow tables [13]. In particular, the switch needs to update the SDN controller whenever the packets' incoming port or destination IP address are not present in the flow tables [13]. An attacker could launch volumetric attacks by spoofing unknown source IP addresses, saturating the switch's flow tables [14].

Ye et al. [16] collected traffic data in data plane devices from the controller and extracted six-tuple characteristic values related to DDoS attacks from the switch flow table for SVM-based detection [16]. While the authors reported a high detection accuracy, the detection accuracy for ICMP attack flows was relatively low, suggesting different attack detection accuracy for various attack protocols.

Xue et al. [17] highlighted that existing equipment and software that have not adopted the SDN architecture cannot achieve adequate security [17]. The authors applied SVM with C and G parameters optimized using cross-validation-genetic algorithm (CV-GA) to detect DDoS attacks on switches in the data plane [17].

Nanda et al. [18] proposed using machine learning algorithms (C4.5, Bayesian Network, Decision Table, and Naive Bayes) trained on historical network attack data to identify potential malicious connections [18]. They demonstrated that detecting malicious users in the data plane using machine learning estimators could enable the SDN controller to quickly create new rules to prevent attacks [18].

Jankowski and Amanowicz [19] applied Self-Organizing Maps (SOM), Learning Vector Quantization (LVQ1), and their enhanced variants for detecting and monitoring malicious activities on the data plane [19]. Specifically, their Hierarchical Learning Vector Quantization (H-LVQ1) algorithm demonstrated promising results compared to other variants [19].

Banerjee and Chakraborty [20] conducted a two-step experiment in which Naive Bayes and K-Nearest Neighbors (KNN) algorithms were trained to differentiate attack and normal traffic [20]. Detected attackers were blocked by creating Access Control Lists (ACLs) [20].

Mowla et al. [21] proposed Cognitive Switch-based DDoS Sensing and Mitigation in SDN-driven Content Delivery Networks [21]. The authors applied SVM and Logistic Regression to classify traffic and initiate the deployment of security rules to OpenFlow switches to prevent new forms of flooding attacks [21].

Machine Learning Approaches for Detecting DDoS in SDN

Supervised Learning Algorithms

The reviewed literature reports extensive use of supervised learning algorithms for detecting DDoS attacks in SDN. Specifically, the Support Vector Machine (SVM) has been widely used for its ability to define hyperplanes that separate normal and attack traffic [9,10,16,17]. Table 1 summarizes the supervised learning algorithms used in the reviewed literature.

Algorithm Accuracy

Alshamrani et al. [9]: SMO 97.31%

Alshamrani et al. [9]: J48 98.78%

Alshamrani et al. [9]: NB 95.11%

Latah and Toker [10]: SMO 96.85%

Ye et al. [16]: SVM 95.24%

Nanda et al. [18]: C4.5 91.68%

Jankowski and Amanowicz [19]: SOM 94%

Mowla et al. [21]: SVM 92.9%

Polat et al. [Present]: KNN 98.30%

Polat et al. [Present]: KNN 97.15%

While various studies have used SVM for detecting DDoS attacks, its performance depends on the choice of parameters. Table 1 shows that KNN performs slightly better than other algorithms. The performance of Naive Bayes (NB) has been inconsistent across different studies [9,18]. The KNN algorithm is a non-parametric algorithm that can achieve high accuracy without requiring much training data. However, it requires high computational power during classification [20].

Feature Selection Methods

Feature selection is a critical preprocessing step in developing machine learning models. The general idea is to reduce the dimensionality of the input data by selecting a subset of features that captures the information contained in all features.

Various feature selection methods have been used in the literature to reduce the number of features used in detecting DDoS attacks in SDN. Table 2 summarizes the feature selection methods used in the reviewed literature.

Feature Selection Method Description

Filter Easy to train, less computational power

Wrapper May require more time but achieves higher accuracy

Embedded Less time-consuming than wrapper, more flexible than filter

The wrapper approach has been used in this study using Sequential Forward Floating Selection. Embedded methods are also less computationally intensive and achieve high accuracy compared to wrapper methods. The filter method has been used in this study using the Relief algorithm.

Research Gaps and Contributions of the Present Study

Identified Research Gaps

The reviewed literature indicates several research gaps that warrant further investigation. First, there is a need for more comparative studies between different feature selection methods. Second, many studies used different datasets, making it difficult to compare their results directly. Third, while most studies focused on TCP-based attacks, there is a need to investigate the performance of different algorithms in detecting UDP and ICMP-based attacks [16]. Finally, most studies used a single controller, but in practice, an SDN network may have multiple controllers, raising the question of whether existing solutions are scalable.

Contributions of the Present Study

Polat, Polat, and Cetin [2020] made significant contributions to the research on detecting DDoS attacks in SDN. First, they compared different feature selection methods in detecting DDoS attacks. Second, they created a new dataset consisting of 129,000 samples (65,000 attack samples and 64,000 normal samples) and 12 features [20]. Table 3 compares the results of this study with related works.

Attack Type Accuracy

Alshamrani et al. [9]: TCP 97.31%

Alshamrani et al. [9]: UDP 98.78%

Latah and Toker [10]: TCP 96.85%

Ye et al. [16]: TCP 95.24%

Nanda et al. [18]: TCP 91.68%

Jankowski and Amanowicz [19]: TCP 94%

Mowla et al. [21]: TCP 92.9%

Polat et al. [Present]: TCP 98.30%

Polat et al. [Present]: UDP 97.15%

The study by Polat et al. [20] used KNN with wrapper methods to detect DDoS attacks using only six features, achieving an accuracy of 98.3%. In terms of accuracy, this study outperformed all previous studies for detecting DDoS attacks in SDN. In particular, Table 3 shows that detecting TCP attack traffic achieved the best performance rate (98.3%), while detecting UDP flood attacks was somewhat easier than detecting TCP and ICMP flood attacks. The study also evaluated the

performance of other algorithms, including SVM, NB, ANN using the same dataset [20]. Table 4 summarizes the accuracy rates.

Classifier Accuracy

SVM 81.2%

NB 86.5%

ANN 83.7%

KNN 98.3%

The results show that KNN and NB classifiers performed significantly better than the other two classifiers [20]. It is also worth noting that the performance rate of all classifiers was higher than 80%, suggesting that machine learning algorithms could be used to detect DDoS attacks with reasonable accuracy. Overall, the study by Polat et al. [20] provides valuable insights into detecting DDoS attacks in SDN networks.

Implications for Future Research

Based on the knowledge gained from this literature review, several implications for future research can be drawn. First, future studies should focus on detecting DDoS attacks in SDN networks with multiple controllers. Second, researchers should explore how to use machine learning algorithms to detect DDoS attacks in real-time. Third, there is a need to develop machine learning algorithms that can adapt to changing attack patterns to be always one step ahead of attackers. Fourth, future studies should investigate the feasibility of integrating machine learning algorithms

with existing DDoS mitigation solutions. Finally, researchers should conduct more empirical studies to evaluate the effectiveness of different machine learning algorithms in detecting DDoS attacks in SDN networks.

Conclusion

This literature review aimed to summarize the research on detecting DDoS attacks in SDN environments. We focused on the relevant articles identified in the previous section to understand the research gaps and highlight the contributions of the current study [20]. Overall, the reviewed articles demonstrate that machine learning algorithms can effectively detect DDoS attacks in SDN networks by taking advantage of the rich information present in the traffic data. However, further research is needed to fully exploit the potential of machine learning algorithms in detecting DDoS attacks in SDN networks.

PAPER-10—SECUREIOT:NOVEL MACHINE LEARNING ALGORITHMS FOR DETECTING AND PREVENTING ATTACKS ON IOT DEVICES

Overview of IoT Security Landscape

Internet of Things (IoT) has revolutionized modern society as it has penetrated every field of life including health, home, automotive, and industrial systems. Perwej et al (2019) report that IoT refers to sensors, actuators and smart devices, which have become part of people's daily lives as most of them offer convenience and efficiency. These devices have been integrated into different applications and processes, creating confusion in the domain of IoT security (Lin and Bergmann, 2016). IoT devices are mostly characterized by limited resources which makes them vulnerable to attacks (Neshenko et al., 2019). Additionally, IoT requires interconnectivity of devices through networks resulting in expanded attack surfaces that are easily exploited by cyber attackers (Neshenko et al., 2019).

Most of the attacks that are carried out on IoT devices have been described as having adverse impacts to the device owners and even forming botnets capable of launching large scale attacks. Plabon Bhandari Abhi et al. (2023) note that IoT is highly heterogeneous and thus, current security measures are not efficient enough to handle IoT challenges. Hence, there is a need to develop new methods and algorithms to secure IoT devices.

Machine Learning Applications in IoT Security

Research works have shown that machine learning is able to improve IoT security as it is capable of analyzing information and identifying attack patterns. Jacob and Darney (2021) have demonstrated that deep learning algorithms are applicable in

image recognition based IoT systems. Laha et al. (2020) have carried out a survey on the applications of machine learning in wireless network and IoT, focusing on supervised learning algorithms, unsupervised learning algorithms, and reinforcement learning algorithms.

Supervised learning algorithms such as SVM, Random Forest, Naive Bayes, and KNN have been used in classifying IoT devices normal and abnormal behaviors. Unsupervised learning algorithms such as K-Means, isolation forest, and Autoencoders have been used in predicting patterns of IoT device activities in the absence of labeled data. Reinforcement learning algorithms include Deep Q-Network (DQN), Proximal Policy Optimization (PPO), and Monte Carlo Tree Search (MCTS) (Bezawada & Kumar, 2023; Waseem & Roy, 2022; Hoffman et al., 2021).

Related Works in IoT Attack Detection and Prevention

Distributed Ledger and Access Control

Rahim and Wahid (2023) have proposed a framework for accomplishing reliable communication links and confidentiality of data in IoT as the devices are limited in resources. The author suggests using a combination of IOTA and IPFS to create a decentralized and distributed storage. The approach uses Fog nodes for local computations and trusted access control policies. Experiments using a simulation engine showed effective IoT and network security improvement. However, there is a need to test the approach under other conditions due to concerns with regard to scalability and performance.

Authentication and Vulnerability Detection

In his work, Khan et al. (2023) have focused on achieving IoT device security despite the threats and vulnerabilities facing IoT especially in terms of privacy. The author reports that most IoT devices are hybrid in nature and therefore, identifying the top ranked authentication features that are necessary for IoT is important for detecting and preventing attacks. The author uses AI and ML implementation in detecting IoT vulnerabilities. COPRAS is used in analyzing the significant features that contribute to IoT device security. The research is important in helping organizations to know how to protect IoT devices but there is a need for more research since the recommended authentication features may not be applicable in all IoT devices.

Malware Detection and Cryptographic Prevention

Devi and Arunachalam (2023) have developed a model for malware detection and prevention in IoT. The author recommends a deep LSTM classifier for malware detection and Improved Elliptic Curve Cryptography (IECC) for malware prevention. The model detects attack nodes based on trust values and context features. Hybrid MA-BW is implemented for optimal key selection during data transmission. The approach attained 95% accuracy, 92% precision and low execution time as compared to similar works. In general, the paper is relevant in demonstrating how to secure IoT devices against malware. However, there are some challenges associated with the approach, mainly due to high computational complexity.

Fog Computing and Ensemble Classification

M et al. (2023) have focused on detecting attacks in Cyber-Physical Systems (CPSs) that are compromised due to insecure IoT networking devices. Fog-IoT is recommended for faster attack detection as compared to cloud based attack detection. The author develops an Enhanced Chicken Swarm Optimization algorithm that is used in selecting features that are informative in detecting attacks. Ensemble classification algorithms are executed in the cloud using the selected features. The features were evaluated using NSL-KDD and the results were compared using several statistical measures. The paper contributes to knowledge by showing how to detect attacks in IoT based-Cyber-Physical Systems but further research can be done to improve the techniques in similar areas.

Behavioral Security Detection

IoT-DeepSense is a system for behavioral security detection in IoT to overcome challenges associated with conventional IoT security software and network traffic based IoT security detection. IoT-DeepSense takes advantage of firmware virtualization and deep learning to implement system level fine grained behavioral analysis of IoT devices. The system uses LSTM based abnormal behavior detection to extract features and identify abnormal behaviors in IoT. IoT-DeepSense runs independently as an IoT behavior detection server and achieves 92% detection rate without modifying the constrained IoT devices. The system presents a practical solution for IoT behavioral security detection, however, there is still need for more research to improve it.

Insider Attack Detection

IoT devices require protection from attacks since they have been reported to be used in malicious attacks to gain illegal benefits. Khan et al. (2019) have worked on the detection of malicious insider attacks in IoT. The work focuses on the detection of attacks launched from incoming data sensors in resource constrained IoT using artificial intelligence. The paper recommends a lightweight approach for insider attack detection that offers better results as compared to previous approaches.

System Hardening and Security Monitoring

Choi et al. (2018) have shown that IoT devices are currently adopted in many fields without appropriate security measures. This has resulted in more IoT security incidents and vulnerabilities that can be used for malicious purposes. The authors stress more on the need for security by design and mitigation of risks and threats in IoT devices to enhance system security without compromising device performance. The work highlights the key aspects of IoT security which can be used in future works to design secure IoT devices.

Research Challenges and Gaps

From the related works discussed above, it is evident that there is still need for further research in IoT detection and prevention due to several challenges that are evident in IoT security. Some of the research challenges are:

1. Limited robustness and scalability of existing IoT detecting and prevention approaches (Neshenko et al., 2019).

2. Incompatibility of authentication features for IoT devices (Khan et al., 2023).

3. Challenges of optimizing key selection for data transmission in IoT based on malware attack detection results (Devi & Arunachalam, 2023).

4. Detecting sophisticated attacks such as Wormhole attack routing in IoT without compromising performance (Alghamdi & Bellaiche, 2023).

5. Efficient detection of attacks in Cyber-Physical Systems using IoT and Fog (M et al., 2023).

6. Efficient extraction of hidden features and detection of abnormal behavior in IoT devices (Wang et al., 2022).

7. Detection of insider attacks in IoT network with limited resources and reduced computation overheads (Khan et al., 2019).

8. How to minimize security vulnerabilities, threats, and risks in IoT systems without affecting performance (Choi et al., 2018).

Emerging Paradigms

From the related works section, it is evident that two machine learning based emerging paradigms have great potential in IoT security. The paradigms include fedrated learning and adversarial machine learning. Yang et al. (2020) have demonstrated the importance of applying federated machine learning in intelligent IoT through reconfigurable metasurfaces. This paradigm is important in IoT security because it helps in preserving data confidentiality of IoT devices during training. On the other hand, adversarial machine learning is a paradigm that is concerned with the ability of attackers to launch attacks by analyzing and exploiting weaknesses of machine learning classification models. Though adversarial machine learning has been considered a threat to IoT security because

of its ability to undermine the performance of classification models; it can be applied to secure IoT devices.

Synthesis and Research Positioning

The related works presented above have contributed immensely towards the current research. It is evident from the related works that there is need for more research in IoT detection and prevention. The current research focuses on developing novel machine learning algorithms for detecting and preventing attacks on IoT devices. Specifically, the research will develop algorithms that can efficiently detect attacks and neutralize their effects. Though some previous works have considered detection and prevention together, they have not addressed the aspect of using novel machine learning algorithms. Additionally, some of the previous works focused only on detection or only on prevention. The current research will also fill this gap by implementing both detection and prevention strategies in a single study. The results of the study will be evaluated using several performance measures. Unlike in most previous works, the current research will focus more on offering prevention measures rather than just detecting attacks. This study will also consider the application of different machine learning techniques in IoT detection and prevention. It has been noticed that most previous works have focused on using a single approach or a few approaches. However, the current research will focus on applying various machine learning algorithms to achieve the best results. The analysis will be carried out using IoT-23 Combined dataset to ensure that this research is relevant and effective in real life IoT attack detection and prevention.